**CS 6200 — Georgia Tech OMSCS**

# Graduate Introduction to Operating Systems

## 講義ノート

kinginu

Fall 2026

# 目次


**1 オペレーティングシステム入門 1**
1.1 オペレーティングシステムとは何か . . . 1
1.2 抽象化，機構，方針 . . . 2
1.3 設計原則 . . . 3
1.4 ユーザ/カーネル保護境界 . . . 3
1.5 システムコール . . . 4
1.6 オペレーティングシステムのサービス . . . 5
1.7 オペレーティングシステムの構成 . . . 6
1.8 Linux と Mac OS X . . . 7

**2 プロセスとプロセス管理 10**
2.1 プログラムとプロセス . . . 10
2.2 プロセスのアドレス空間 . . . 10
2.3 実行状態とプロセス制御ブロック . . . 12
2.4 コンテキストスイッチ . . . 12
2.5 プロセスのライフサイクル . . . 14
2.6 プロセスの生成 . . . 14
2.7 CPU スケジューラ . . . 16
2.8 プロセス間通信 . . . 17

**3 スレッドと並行性 20**
3.1 プロセスとスレッド . . . 20
3.2 スレッドの有用性 . . . 21
3.3 スレッドの基本機構 . . . 22
3.4 相互排除 . . . 23
3.5 条件変数 . . . 24
3.6 リーダ・ライタ問題 . . . 25
3.7 よくある誤りとスプリアスウェイクアップ . . . 27
3.8 デッドロック . . . 28
3.9 カーネルレベルスレッドとユーザレベルスレッド . . . 29
3.10 マルチスレッドのパターン . . . 31

**4 スレッドのケーススタディ: PThreads 34**
4.1 POSIX スレッド . . . 34
4.2 スレッドの生成 . . . 34
4.3 デタッチスレッド . . . 36
4.4 Pthreads プログラムのコンパイル . . . 37
4.5 スレッド生成の例 . . . 37
4.6 ミューテックス . . . 39
4.7 条件変数 . . . 41
4.8 生産者・消費者の例 . . . 41

**5 スレッド設計上の考慮事項 45**
5.1 スレッドのデータ構造 . . . 45
5.2 SunOS 5.0 のスレッド . . . 46
5.3 ライブラリとカーネルの相互作用 . . . 48
5.4 実装上の問題 . . . 49
5.5 割り込みとシグナル . . . 51
5.6 割り込みとシグナルの処理 . . . 51
5.7 割り込みとシグナルの無効化 . . . 53
5.8 スレッドとしての割り込み . . . 54
5.9 スレッドとシグナル処理 . . . 55
5.10 Linux におけるタスク . . . 56

**6 スレッド性能上の考慮事項 58**
6.1 スレッドモデルの比較 . . . 58
6.2 スレッドの有用性 . . . 59
6.3 性能指標 . . . 59
6.4 マルチプロセスサーバとマルチスレッドサーバ . . . 60

6.5 イベント駆動モデル . . . 61
6.6 ブロッキング操作とヘルパ . . . 63
6.7 Flash ウェブサーバ . . . 64
6.8 Apache ウェブサーバ . . . 66
6.9 実験の方法論 . . . 66
6.10 実験結果 . . . 67
6.11 実験の設計と実施 . . . 68

**7 スケジューリング 70**
7.1 CPU スケジューリングの概観 . . . 70
7.2 実行完了型スケジューリング . . . 71
7.3 プリエンプティブスケジューリング . . . 71
7.4 優先度逆転 . . . 73
7.5 ラウンドロビンスケジューリングとタイムシェアリング . . . 74
7.6 タイムスライスの長さの選択 . . . 75
7.7 多段フィードバックキュー . . . 77
7.8 Linux の $O(1)$ スケジューラ . . . 78
7.9 Linux の Completely Fair Scheduler . . . 79
7.10 マルチプロセッサ上のスケジューリング . . . 80
7.11 ハイパースレッディング . . . 81
7.12 ハイパースレッディング対応プラットフォームのスケジューリング . . . 82
7.13 ハードウェア性能カウンタ . . . 83
7.14 スケジューリング指標としての命令当たりサイクル数 . . . 84

**8 メモリ管理 85**
8.1 メモリ管理の目標 . . . 85
8.2 ハードウェアによる支援 . . . 86
8.3 ページテーブル . . . 87
8.4 ページテーブルエントリ . . . 88
8.5 ページテーブルの大きさ . . . 89
8.6 多段ページテーブル . . . 89
8.7 変換の高速化：TLB . . . 90
8.8 逆ページテーブル . . . 91
8.9 セグメンテーション . . . 91
8.10 ページの大きさ . . . 92
8.11 メモリの割り当て . . . 93
8.12 Linux カーネルのアロケータ . . . 94
8.13 デマンドページング . . . 95
8.14 ページ置換 . . . 96
8.15 コピーオンライト . . . 97
8.16 チェックポインティング . . . 97

**9 プロセス間通信 100**
9.1 プロセス間通信 . . . 100
9.2 メッセージベースの IPC . . . 100
9.3 メッセージパッシングの形態 . . . 101
9.4 共有メモリ IPC . . . 102
9.5 コピーとマップ . . . 103
9.6 System V 共有メモリ . . . 103
9.7 POSIX 共有メモリ . . . 105
9.8 共有メモリと同期 . . . 105
9.9 IPC のコマンドラインツール . . . 107
9.10 共有メモリ上の通信の設計 . . . 107

**10 同期機構 110**
10.1 ミューテックスと条件変数の先へ . . . 110
10.2 スピンロック . . . 110
10.3 セマフォ . . . 111
10.4 リーダ・ライタロック . . . 112
10.5 モニタ . . . 113
10.6 その他の同期機構 . . . 114
10.7 同期のためのハードウェア支援 . . . 116

10.8 共有メモリ型マルチプロセッサとキャッシュ . . . 117
10.9 スピンロックの性能 . . . 119

**11 I/O 管理 123**
11.1 I/O デバイス . . . 123
11.2 CPU とデバイスのインターコネクト . . . 123
11.3 デバイスドライバ . . . 124
11.4 デバイスの種類 . . . 124
11.5 CPU とデバイスの相互作用 . . . 125
11.6 デバイスアクセス：プログラム制御 I/O と DMA . . . 126
11.7 典型的なデバイスアクセス . . . 127
11.8 同期アクセスと非同期アクセス . . . 128
11.9 ブロックデバイススタック . . . 129
11.10 仮想ファイルシステム . . . 130
11.11 VFS の抽象化 . . . 130
11.12 第 2 拡張ファイルシステム . . . 131
11.13 i ノード . . . 132
11.14 ディスクアクセスの最適化 . . . 134

**12 仮想化 136**
12.1 仮想化とは何か . . . 136
12.2 仮想化の利点 . . . 137
12.3 仮想化のモデル . . . 138
12.4 ハードウェアの保護レベル . . . 139
12.5 プロセッサの仮想化 . . . 140
12.6 ハードウェア支援以前の x86 の仮想化 . . . 140
12.7 バイナリ変換 . . . 141
12.8 準仮想化 . . . 142
12.9 メモリの仮想化 . . . 142
12.10 デバイスの仮想化 . . . 144
12.11 仮想化のためのハードウェア支援 . . . 145

**13 遠隔手続き呼び出し 148**
13.1 なぜ遠隔手続き呼び出しか . . . 148
13.2 RPC への要件 . . . 149
13.3 RPC システムの構造 . . . 149
13.4 インタフェース定義言語 . . . 150
13.5 マーシャリングとアンマーシャリング . . . 151
13.6 バインディングとレジストリ . . . 151
13.7 引数としてのポインタ . . . 152
13.8 部分障害 . . . 152
13.9 Sun RPC . . . 153
13.10 rpcgen によるインタフェースのコンパイル . . . 154
13.11 Sun RPC のレジストリとバインディング . . . 155
13.12 XDR のデータ型とルーチン . . . 156
13.13 XDR の符号化 . . . 157
13.14 Java RMI . . . 157

**14 分散ファイルシステム 160**
14.1 分散ファイルシステム . . . 160
14.2 遠隔ファイルサービス . . . 160
14.3 ステートレスなファイルサーバとステートフルなファイルサーバ . . . 162
14.4 DFS におけるキャッシュ状態 . . . 163
14.5 ファイル共有セマンティクス . . . 164
14.6 ファイルとディレクトリ . . . 166
14.7 レプリケーションとパーティショニング . . . 166
14.8 Network File System . . . 167
14.9 NFS のバージョン . . . 168
14.10 Sprite 分散ファイルシステム . . . 169
14.11 Sprite におけるファイルアクセス操作 . . . 170

**15 分散共有メモリ** **173**
15.1 ピア分散アプリケーション . . . 173
15.2 分散共有メモリ . . . 173
15.3 ハードウェア DSM とソフトウェア DSM . . . 174
15.4 共有の粒度 . . . 175
15.5 アクセスアルゴリズム . . . 175
15.6 マイグレーションとレプリケーション . . . 176
15.7 一貫性管理 . . . 177
15.8 DSM のアーキテクチャ . . . 178
15.9 DSM の実装 . . . 179
15.10 一貫性モデル . . . 181

**16 データセンタ技術** **184**
16.1 インターネットサービス . . . 184
16.2 スケールアウト構成 . . . 184
16.3 クラウドコンピューティングの動機 . . . 187
16.4 クラウドコンピューティング . . . 188
16.5 デプロイメントモデルとサービスモデル . . . 189
16.6 クラウドへの要件 . . . 190
16.7 クラウドを可能にする技術 . . . 191
16.8 ビッグデータエンジンとしてのクラウド . . . 192

**参考文献** **194**

**索引** **197**

**English index** **202**

第 1 章

# オペレーティングシステム入門

| | |
|---|---|
| **講義動画** | P1L2 |
| **教科書** | OSTEP [1] ch. 2，6；Silberschatz ら [2] ch. 1–2 |
| **キーワード** | オペレーティングシステム，抽象化，調停，機構，方針，カーネルモード，ユーザモード，トラップ，システムコール，シグナル，モノリシック OS，モジュール型 OS，マイクロカーネル |

あらゆるアプリケーションは，実行されるマシンの CPU，メモリ，デバイスを使うためにオペレーティングシステムに依存している．この章では，オペレーティングシステムとは何か，どのような役割を果たすかを定義し，それを構成する要素–抽象化，機構，方針–と，その設計を導く原則を導入する．次に，アプリケーションとオペレーティングシステムの間の保護境界と，アプリケーションがシステムコールによってそれを越える仕組みを調べ，オペレーティングシステムの構成の仕方を Linux と Mac OS X を例に比較する．ここで導入するプロセス，スレッド，メモリページ，ファイルといった抽象化は，以降の章の主題である．

## 1.1 オペレーティングシステムとは何か

計算機システムは，ハードウェア–1 つ以上の CPU（それぞれ複数のコアを持ちうる），主記憶，ハードディスクや SSD，USB フラッシュドライブなどの記憶装置，Ethernet や WiFi のためのネットワークインタフェース，グラフィックスプロセッサ（GPU）–と，それを使うアプリケーションからなる．通常は複数のアプリケーションが同時に実行され，そのすべてがこれらのハードウェア構成要素を必要とする．オペレーティングシステムは，両者の間に位置するソフトウェアの層である（図 1.1）．

**出典** P1L2，オペレーティングシステムとは何か，オペレーティングシステムの定義；OSTEP ch. 2；Silberschatz ら ch. 1．

▷ operating system

**定義 1.1** (オペレーティングシステム). **オペレーティングシステム**（OS）とは，次の性質を持つシステムソフトウェアの層である．

- 下位のハードウェアに直接かつ特権的にアクセスできる．
- ハードウェアの複雑さをアプリケーションから隠す．
- あらかじめ定められた方針に従って，1 つ以上のアプリケーションに代わってハードウェアを管理する．
- アプリケーションが互いに隔離され保護されることを保証する．

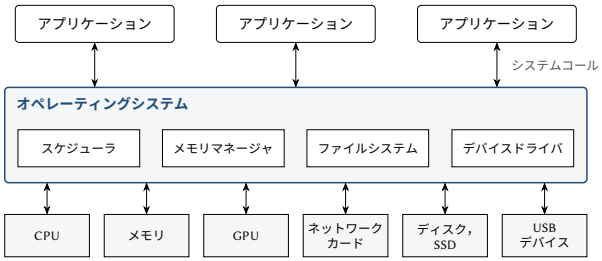



**図 1.1:** アプリケーションとハードウェアの間のオペレーティングシステム．アプリケーションはシステムコールを通じてサービスを要求し，スケジューラ，メモリマネージャ，ファイルシステム，デバイスドライバなどの構成要素がそれをハードウェア上で実行する．

要するに，オペレーティングシステムはハードウェアを**抽象化**してアプリケーションから見た姿を単純にし，その利用を**調停**して，アプリケーションがハードウェアをどのように共有するかを監督・制御する．この定義は 3 つの役割を挙げている．

1 つのファイルという抽象化が，桁違いに性能の異なるデバイスを覆う．フラッシュのページ読み出しは 25–75 µs（OSTEP ch. 44），ディスクのシークは 4–9 ms（OSTEP ch. 37）である．

▷ abstraction

**ハードウェアの複雑さの隠蔽** オペレーティングシステムは，**抽象化**，すなわち背後にあるデバイスの詳細を隠す，より単純で高水準なインタフェースを通じてハードウェアをアプリケーションに提示する．データを保存するアプリケーションは，特定のディスクのセクタやブロックを指定しない．**ファイル**に書き込み，オペレーティングシステムがファイル操作を，実際に備わっている記憶装置に対する操作へ変換す

る．同じように**ソケット**は，ネットワークインタフェースによらずネットワーク通信を表す．デバイスが変わっても抽象化は変わらないので，アプリケーションは異なるハードウェア上でも変更なしに動作する．

▷ arbitration

**資源管理** オペレーティングシステムは**調停**によってハードウェア資源を管理する．すなわち，どのアプリケーションがどの資源をどれだけ使うかを決め，アプリケーションにメモリを割り当て，それらを CPU 上にスケジュールし，デバイスへのアクセスを制御する．その判断は方針に従う．例えば，資源へのアクセスが公平であること，あるいは 1 つのアプリケーションが使う資源に上限を設けることである．

▷ isolation

▷ protection

**隔離と保護** 同時に実行されるアプリケーションは互いに干渉してはならない．オペレーティングシステムは，各アプリケーションに資源の取り分–例えば物理メモリの自分の領域–を与えることで**隔離**を提供し，その実行が他のアプリケーションに影響されないようにする．また，割り当てられていない資源，例えば他のアプリケーションやオペレーティングシステム自身のメモリへのアクセスを禁じることで**保護**を提供する．つまり隔離は，あるアプリケーションが他のアプリケーションに及ぼす影響を制限し，保護は，アプリケーションが触れてよいものを制限する．

したがってオペレーティングシステムは，CPU スケジューラ，メモリマネージャ，デバイスドライバ，ファイルシステムといった構成要素からなる．表計算ソフトやメディアプレーヤのように，これらのサービスを利用するだけのアプリケーションプログラムは，たとえ一緒に配布されていても，その一部ではない．

**注意**「オペレーティングシステム」という語は緩く使われる．厳密にはカーネル，すなわちカーネルモードで実行される部分である（1.4 節）．Ubuntu のようなディストリビューションは，シェル，ライブラリ，ユーティリティも同梱している（Silberschatz ら ch. 1）．

**例 1.2.** オペレーティングシステムのある機能は，デバイスへのインタフェースを単純にするときには抽象化であり，資源をアプリケーション間で分けるときには調停である．プログラムが，プリンタの機種の命令言語を知らずに文書をプリンタへ送れるようにすることは抽象化であり，同時に印刷する 3 つのプログラムの文書がどの順でプリンタに届くかを決めることは調停である．同様に，Ethernet でも WiFi でも同じソケットインタフェースを提供することは抽象化であり，1 つのアプリケーションが消費してよいネットワーク帯域を制限することは調停である．

### 1.1.1 オペレーティングシステムの例

**出典** P1L2，オペレーティングシステムの例．

オペレーティングシステムは，対象とする環境によって異なる．デスクトップやサーバのマシンでは，主な例は Microsoft Windows と UNIX 系のシステムであり，後者には Mac OS X（BSD UNIX に由来する）や Linux が含まれる．スマートフォンなどの組込みデバイスでは，Linux に基づく Android，iOS，Symbian などがある．いずれもその環境に適した設計上の選択を行っている．デスクトップシステムは，1 人の利用者の対話的なアプリケーションの応答性を重視する．組込みシステムは，小さなメモリと限られたバッテリの中で，しばしば実時間の期限のもとで動作しなければならない．サーバシステムは，多数の利用者と要求を高いスループットで処理し，長期間動作し続けなければならない．以降の章では主に Linux を例として用いる．

Android は Linux カーネルの上で，GNU のツールではなく独自のユーザ空間–Bionic C ライブラリと ART ランタイム–を動かす（source.android.com）．

## 1.2 抽象化，機構，方針

**出典** P1L2，OS の構成要素，OS の構成要素の例；Silberschatz ら ch. 2．

オペレーティングシステムは 3 種類の要素から構築される．その抽象化は，実行中のアプリケーション–プロセスとスレッド–と，ハードウェア資源–ファイル，ソケット，メモリページ–を表す．オペレーティングシステムはこれらの抽象化に対する操作を提供し，その操作をどのように適用するかを決める．

▷ mechanism

▷ policy

**定義 1.3** (機構と方針). **機構**とは，プロセスの生成やスケジューリング，ファイルのオープンや書き込み，メモリの割り当てなど，オペレーティングシステムがその抽象化に対して提供する操作である．**方針**とは，メモリに対する LRU（least recently used）やスケジューリングに対する EDF（earliest deadline first）のように，ハードウェアを管理するために機構をどう使うかを定める規則である．

x86-64 ではページは既定で 4 KiB であり，2 MiB と 1 GiB のヒュージページもある．第 8 章．

**ヒント** 方針と機構を見分けるには，実際の処理を行うコードを変えずに置き換えられるかを問えばよい．LRU は LFU に置き換えられるが，ページをディスクへ移すことはそのコード自体である．

メモリ管理は，3 つの要素がどのように組み合わさるかを示している（表 1.1）．抽象化はメモリページ，すなわち固定長のメモリブロックである．機構はページを割り当て，それをプロセスのアドレス空間に対応付けて，プロセスがそれを使えるようにする．物理メモリ（DRAM）は通常すべてのプロセスの要求の合計より小さいので，オペレーティングシステムは一部のページをディスクへ移し，再びアクセスされたときに戻す．どのページを移すかを決めるのが方針である．LRU では，最も長くアクセス
されていないページを，近いうちに必要になる可能性が最も低いという仮定のもとで移す．

表 1.1: オペレーティングシステムの要素．一般の場合とメモリ管理の場合．

| 要素 | 例 | メモリ管理 |
|---|---|---|
| 抽象化 | プロセス，スレッド，ファイル，ソケット，メモリページ | メモリページ |
| 機構 | 生成，スケジュール，オープン，書き込み，割り当て | ページを割り当て，プロセスに対応付ける |
| 方針 | LRU，EDF | LRU: 最も長くアクセスされていないページをディスクへ移す |

## 1.3 設計原則

出典 P1L2，OS の設計原則；Silberschatz ら ch. 2．
▷ separation of mechanism and policy

これらの要素の設計を導く原則が 2 つある．第一は**機構と方針の分離**である．オペレーティングシステムは多くの方針を支えられる柔軟な機構を実装し，方針は機構を変更することなく選択したり置き換えたりできる．ディスクへ移すページの選択では，ページアクセスの記録，ページのディスクへの移動，その対応付けの更新という同じ機構が，LRU，LFU（least frequently used），さらにはランダム選択も支える．どれが最も適しているかは，システムが使われる状況に依存する．

**例 1.4.** 物理メモリが 4 ページを保持しており，5 つ目のページが必要になったので，4 つのうち 1 つをディスクへ移さなければならない．時刻 160 ms における記録されたアクセスは次のとおりである．

| ページ | A | B | C | D |
|---|---|---|---|---|
| 最終アクセス（ms） | 120 | 40 | 95 | 150 |
| アクセス回数 | 9 | 25 | 3 | 12 |

LRU は最終アクセスが最も過去にあるページ B を，LFU はアクセス回数が最も少ないページ C を移し，ランダム選択は各ページを確率 1/4 で移す．いずれの場合も実行される機構は同じであり，選ばれたページは変更されていればディスクへ書き出され，その対応付けが無効化される．

▷ optimize for the common case

第二の原則は**頻繁に起こる場合の最適化**である．設計者は，オペレーティングシステムがどこで使われるか，利用者がそのマシンでどのアプリケーションを実行するか，そのワークロードの要件が何かを理解しなければならない．その上で，この頻繁に起こる場合に適し，かつ下位の抽象化と機構で支えられる方針を選ぶ．例えば主にデータベースを実行するサーバは多数のランダムな読み書きを行うのに対し，メディアサーバは大きなファイルを逐次に読む．一方のワークロードに適した方針は，他方にはうまく合わない．

> **要点**
> 抽象化は複雑さを単純にし，機構は抽象化に対して操作を行い，方針は機構をどう使うかを決める．機構を一般的に保つことで，同じオペレーティングシステムが，その頻繁に起こる場合に合った方針を採用できる．

## 1.4 ユーザ/カーネル保護境界

出典 P1L2，ユーザ/カーネル保護境界；OSTEP ch. 6；Silberschatz ら ch. 1．

オペレーティングシステムはハードウェアに直接アクセスする必要があるので，アプリケーションが持たない特権をもって実行されなければならない．そこで計算機プラットフォームは，少なくとも 2 つの実行モードを区別する．

**定義 1.5 (カーネルモードとユーザモード).** 特権モードである**カーネルモード**では，ソフトウェアはプロセッサのすべての命令を実行でき，ハードウェアに直接アクセスできる．非特権モードである**ユーザモード**では，ハードウェアに直接アクセスする命令は禁じられる．

▷ kernel mode
▷ user mode

カーネルモードで実行されるオペレーティングシステムの部分が，その**カーネル**である．アプリケーションはユーザモードで実行される．2 つのモードの区別はハードウェアによって支えられる．CPU は現在のモードを記録する**モードビット**を持ち，ハードウェアを直接操作する命令，すなわち**特権命令**は，モードビットがカーネルモードを示す場合にのみ実行される．ユーザモードで特権命令を実行しようとしても効果はなく，トラップが発生する．

▷ kernel
▷ mode bit
▷ privileged instruction

▷ trap

**定義 1.6** (トラップ). **トラップ**とは，実行中のアプリケーションからオペレーティングシステムへの制御の移行であり，ハードウェアによって行われる．アプリケーションは中断され，CPU はカーネルモードに切り替わり，オペレーティングシステムがあらかじめ登録しておいたカーネル内の位置から実行が続く．

トラップは同期的であり，実行中の命令によって引き起こされる．これに対して割り込みは，実行中の命令とは無関係にデバイスが発生させる．どちらも，オペレーティングシステムがあらかじめ登録しておいたハンドラのアドレスの表を通じて制御を移す．

x86 プロセッサは 4 つの特権レベル（第 12 章の保護リング）を備え，OS はうち 2 つ以上を使う．ARM はそのモードを例外レベル EL0–EL3 と呼び，RISC-V はユーザモード，スーパバイザモード，マシンモードを持つ（Arm Architecture Reference Manual；RISC-V Privileged Specification）．

特権命令や，特権を必要とするメモリアクセスによるトラップの後，オペレーティングシステムはトラップの原因が何か，そのアクセスを許すべきかを判定する．アプリケーションが不正なことを試みた場合には，それを終了させることもある．アプリケーションとオペレーティングシステムは 3 つの方法でやり取りする（図 1.2）．

**注意** トラップはエラーではない．すべてのシステムコールは意図的なトラップであり，終了につながるのは不正な命令やメモリアクセスだけである．

**トラップ** アプリケーションが特権を必要とする操作を行ったときに，制御をオペレーティングシステムへ移す．

▷ system call

**システムコール** 明示的に要求される．**システムコール**とは，アプリケーションが自分に代わってサービスや特権的なアクセスを行うようオペレーティングシステムに依頼できるように，オペレーティングシステムが公開している操作である．例えばファイルに対する open，ソケットに対する send，メモリに対する mmap がある．これらの操作の集合が**システムコールインタフェース**である．

▷ signal

**シグナル** 逆向きに伝わる．**シグナル**とは，オペレーティングシステムがアプリケーションに渡す通知であり，例えばアプリケーションが設定したタイマが満了したことを伝える．

カーネルはユーザモードへ戻るたびに保留中のシグナルを調べる．割り込みはそのような復帰を任意の時点で引き起こしうるので，ハンドラは任意の 2 命令の間で実行されうる．シグナル：第 5 章．

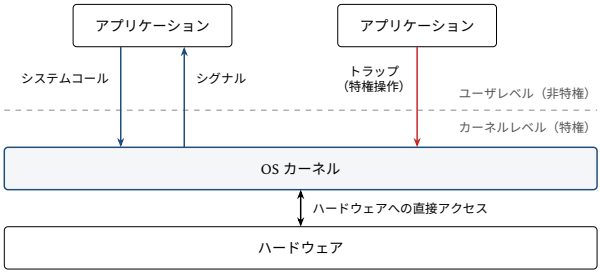



**図 1.2:** ユーザ/カーネル保護境界．ハードウェアにアクセスするのはカーネルだけであり，アプリケーションはシステムコールまたはトラップによってカーネルに入り，カーネルはシグナルでアプリケーションに通知する．

## 1.5 システムコール

**出典** P1L2，システムコールのフローチャート；OSTEP ch. 6；Silberschatz ら ch. 1–2．

図 1.3 は，ユーザプロセスがシステムコールを実行するときの一連の出来事を示している．プロセスは，オペレーティングシステムのサービスが必要になってシステムコールを呼び出すまでユーザモードで実行される．この呼び出しはカーネルへトラップする．モードビットがカーネルモードに切り替わり，実行コンテキストがユーザプロセスのものからカーネルのものへ変わり，引数が渡され，そのコールを実装するカーネルのコードへ実行が移る．カーネルはカーネルモードでコールを実行する．完了すると，モードビットがユーザモードに戻され，プロセスの実行コンテキストが復元され，結果が返され，プロセスはコールの次の命令から実行を続ける．

システムコールを行うには，アプリケーションは引数とその他の必要なデータを定められた場所に書き，**システムコール番号**によってコールを呼び出す．カーネルはこの番号から，どの操作が要求されているか，引数がいくつあるか，どこにあるかを判断する．引数は，値として直接渡されるか，それらを保持するメモリ領域のアドレスとして間接的に渡される．そのようなアドレスはどこを指していてもおかしくないので，カーネルは使う前に，それが呼び出し元プロセスのメモリ内にあることを確かめる．隔離はこの検査に依存している．コールの結果，またはエラーコードはレジスタで返される．ここで述べる同期的な形式では，プロセスはシステムコールが完了するまで待つ．非同期のシステムコールは後の章で扱う．

すべての呼び出しが境界を越えるわけではない．Linux は vDSO を各プロセスにマップするので，`clock_gettime` はユーザモードで実行される．`vdso(7)`．

**例 1.7.** 次の C プログラムのどちらの呼び出しも，標準出力へ 6 バイトを書くよう Linux に依頼する．

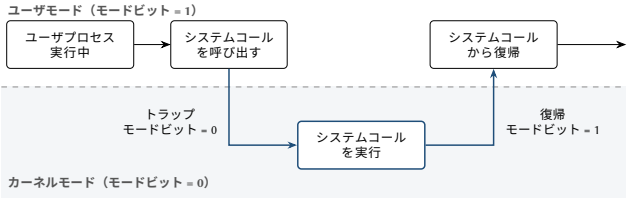



**図 1.3:** システムコールの実行．トラップによってカーネルモードに切り替わり，カーネルがコールを実行し，復帰によってユーザモードに戻る．モードビットは Silberschatz ら [2] の慣習に従い，カーネルモードで 0，ユーザモードで 1 である．

```c
#define _GNU_SOURCE
#include <unistd.h>
#include <sys/syscall.h>

int main(void) {
  const char msg[] = "hello\n";
  /* システムコールの C ライブラリラッパ */
  write(1, msg, sizeof msg - 1);
  /* 同じシステムコールを番号で呼び出す */
  syscall(SYS_write, 1, msg, sizeof msg - 1);
  return 0;
}
```

ライブラリ関数 `write` は，`write` システムコールの番号とその 3 つの引数をカーネルが期待する場所に置き，トラップを実行する．ファイルディスクリプタ 1（標準出力）と長さは直接渡され，文字列はそのアドレス `msg` を通じて間接的に渡される．

x86-64 Linux では，番号は `rax`，引数は `rdi`，`rsi`，`rdx` に置き，`syscall` 命令がトラップする．

### 1.5.1 ユーザ/カーネル境界の横断

ユーザモードとカーネルモードの間の移行は，アプリケーションの実行に欠かせない部分である．ファイルを読んだり，ネットワークでデータを送ったり，メモリを割り当てたりするアプリケーションは，境界を越えなければならない．この移行には 3 つの性質がある．

- 不正な命令や特別な特権を必要とするメモリアクセスに対するトラップを通じて，ハードウェアに支えられている．
- 状態を保存・切り替えるために相当数の命令を実行する必要がある．Linux が動作する 2 GHz のマシンでは，1 回の移行におよそ 50–100 ns かかる．[1]
- 局所性が切り替わる．カーネルはアプリケーションとは異なるコードを実行し異なるデータを使うので，その命令がプロセッサキャッシュ内のアプリケーションのデータを追い出す．アプリケーションが再開すると，データの一部を再び主記憶から取ってこなければならず，その分だけ遅くなる．キャッシュは境界について何も知らない．モード切り替えそのものは何もフラッシュせず，アプリケーションがデータを失うのは，カーネルのアクセスがそれを追い出すからにすぎない．モード切り替えでは同じプロセスとアドレス空間が保たれる．一方，CPU 上で実行されるプロセスを替えるコンテキストスイッチ（第 2 章）は，アドレス空間も入れ替え，キャッシュされた状態を通常より多く失う．

したがってユーザ/カーネル間の移行は安価ではない．

**注意** システムコールは C の手続き呼び出しのように見えるが，コストははるかに高い．

**出典** P1L2，OS 境界の横断．

[1] Core i7 では，カーネルへの出入りはそれぞれ 79 サイクルと 71 サイクルであるが，`pwrite` は 32 KiB の L1 データキャッシュの半分を追い出し，アプリケーションが元の命令実行率に戻るまでに最大 14 000 サイクルを要する [3]．

## 1.6 オペレーティングシステムのサービス

オペレーティングシステムは，サービスを公開することでアプリケーションにハードウェアへのアクセスを与える．最も基本的なレベルでは，サービスはハードウェア構成要素に対応する．スケジューラは CPU へのアクセスを制御し，メモリマネージャは同時に実行されるアプリケーションに物理メモリを割り当て，それらが互いのメモリを上書きしないようにし，ブロックデバイスドライバはディスクなどのブロックデバイスへのアクセスを与える．その他のサービスは，1 つのハードウェア構成要素に直接対応しない，より高水準の抽象化を提供する．ファイルシステムのファイルやディレクトリがその例である．目的ごとにまとめると，オペレーティングシステムはプロセス管理，ファイル管理，デバイス管理，メモリ管理，ストレージ管理，セキュリティを提供する．

**出典** P1L2，OS のサービス；Silberschatz ら ch. 2.

プロセス管理：第 2 章；スレッド：第 3 章；スケジューリング：第 7 章；メモリ：第 8 章；I/O とファイルシステム：第 11 章；分散ファイルシステム：第 14 章．

アプリケーションはこれらすべてのサービスにシステムコールを通じて到達する．スケジューラには，プロセスを生成する fork や CPU を譲る sched_yield などのコールで，メモリマネージャには mmap と brk で，ファイルシステムには open と close で到達し，ブロックデバイスドライバには，ファイルに対する read と write を通じて間接的にのみ到達する．コールの名前や構成はオペレーティングシステムによって異なるが，表 1.2 の Windows と Unix のコールが示すように，提供される機能は同等である．

x86-64 の Linux 7.0 は 386 個のシステムコールに番号を与えている（unistd_64.h）．C ライブラリはそれらをはるかに大きな API に包む．

**表 1.2:** Windows と Unix のシステムコールの例（Silberschatz ら [2] による）．

| 分類 | Windows | Unix |
|---|---|---|
| プロセス制御 | CreateProcess(), ExitProcess(), WaitForSingleObject() | fork(), exit(), wait() |
| ファイル操作 | CreateFile(), ReadFile(), WriteFile(), CloseHandle() | open(), read(), write(), close() |
| デバイス操作 | SetConsoleMode(), ReadConsole(), WriteConsole() | ioctl(), read(), write() |
| 情報管理 | GetCurrentProcessId(), SetTimer(), Sleep() | getpid(), alarm(), sleep() |
| 通信 | CreatePipe(), CreateFileMapping(), MapViewOfFile() | pipe(), shm_open(), mmap() |
| 保護 | SetFileSecurity(), InitializeSecurityDescriptor(), SetSecurityDescriptorGroup() | chmod(), umask(), chown() |

## 1.7 オペレーティングシステムの構成

**出典** P1L2，モノリシック OS，モジュール型 OS，マイクロカーネル；Silberschatz ら ch. 2.

オペレーティングシステムは，そのサービスをどう構成するかによって異なる．図 1.4 は 3 つの基本的な構成を示し，表 1.3 はそれらの利点と欠点をまとめたものである．

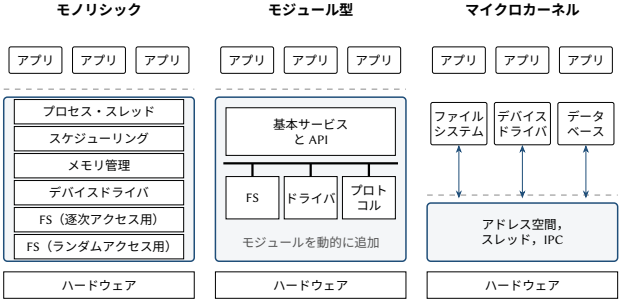



**図 1.4:** オペレーティングシステムの 3 つの構成．破線はユーザ/カーネル境界である．モノリシックカーネルはすべてのサービスを含み，モジュール型カーネルは共通のインタフェースを通じてサービスをモジュールとして追加し，マイクロカーネルはアドレス空間，スレッド，IPC だけを保持して他のすべてのサービスをユーザレベルで実行する．

### 1.7.1 モノリシック OS

▷ monolithic operating system

歴史的には，オペレーティングシステムはモノリシックであった．**モノリシック OS** では，どのアプリケーションが必要としうるサービスも，どの種類のハードウェアが要求しうるサービスも，すべてオペレーティングシステムの一部である．メモリ管理，デバイスドライバ，ファイル管理，プロセスとスレッド，スケジューリングがそうである．このようなシステムは，逐次アクセスに特化したものと，データベースが使うようなランダムアクセスに最適化したもののように，複数のファイルシステムを含むことさえある．これらのコードはすべて一緒にコンパイルされるので，コンパイラは構成要素の境界を越えて，例えばインライン化によって最適化できる．その代償は，カスタマイズ，移植，保守，デバッグ，更新が難しい大量のコードと状態，そしてアプリケーションに残されるメモリを減らし，その性能を低下させうる大きなメモリフットプリントである．

Silberschatz らは Linux を，ロード可能なモジュールを持つモノリシックカーネルとして説明している．

### 1.7.2 モジュール型 OS

Linux を標準的な例とする**モジュール型 OS** は，基本的なサービスと API の集合を備え，それ以外のものは**カーネルモジュール**として追加できる．オペレーティングシステムは，モジュールがその一部となるために実装しなければならないインタフェースを，例えばファイルシステム用のインタフェースとして定める．モジュールはワークロードに応じて動的にインストールできる．データベースを実行するマシンは，ランダムアクセスに最適化されたファイルシステムをロードできる．必要なモジュールだけがロードされるので，システムはより小さく，保守と更新が容易で，アプリケーションにより多くの資源を残す．モジュールインタフェースを介する間接参照は多少の性能低下を招きうるが，通常はわずかである．一方，モジュールはまったく異なるコードベースに由来しうるので，バグの発生源となる．

▷ modular operating system

▷ kernel module

**ヒント** Linux では，`lsmod` はロードされているモジュールを列挙し，`modprobe` はモジュールをロードし，`uname -r` は実行中のカーネルのバージョンを表示する．

Linux のソースツリーは，コメントと空行を含めて 2025 年 1 月に 4000 万行を超えた（Phoronix）．実行中のカーネルがロードするのはその一部にすぎない．

### 1.7.3 マイクロカーネル

**マイクロカーネル**は，オペレーティングシステムのレベルで最も基本的なプリミティブ，すなわち実行中のアプリケーションのメモリを表すアドレス空間と，その実行コンテキストを表すスレッドだけを提供する．それ以外のすべて–ファイルシステム，デバイスドライバ，データベース–は，カーネルの外，ユーザレベルで別々のプロセスとして実行される．[2] したがって 1 つの要求を処理するにはプロセス間の多数のやり取りが必要であり，プロセス間通信（IPC）は，アドレス空間とスレッドと並ぶマイクロカーネルの中核的な抽象化かつ機構である．マイクロカーネルは小さいので，カーネル自体のメモリフットプリントは小さく，アプリケーションによってはオーバーヘッドが低く性能もよい．そのコードは検証もしやすい．そのため，オペレーティングシステムの正しい振る舞いが決定的に重要な組込みデバイスや制御システムでは，マイクロカーネルが魅力的である．他方でマイクロカーネルは下位のハードウェアに特化していることが多く，移植性が制限される．また共通のオペレーティングシステム構成要素を見つけにくいのでソフトウェア開発はより複雑になり，ユーザ/カーネル境界を頻繁に越えることはコストが高い．[3]

▷ microkernel

[2] この考えは Brinch Hansen の RC 4000 のニュークリアス（1969 年）にさかのぼり，Mach（Carnegie Mellon，1985 年から）によって広く知られるようになった．QNX，MINIX 3，seL4 は現役のマイクロカーネルであり，seL4 は機能的正しさについて機械検査された証明を持つ [4]．

[3] どれだけ高いかは設計の問題である．Liedtke の L3 [5] は 50 MHz の i486 上で，アドレス空間をまたぐ 8 バイトのメッセージを 5.2 µs で転送した．同じプロセッサ上の Mach は 115 µs を要した．後継の L4 がこれに続いた．今日の seL4 は，Arm，x86-64，RISC-V のコアで IPC コール 1 回当たり 350–800 サイクルと報告している（`se14.systems/performance.html`）．

**表 1.3:** 3 つの構成の利点と欠点．

| 構成 | 利点 | 欠点 |
|---|---|---|
| モノリシック | すべてを含む．構成要素を越えた最適化ができる | カスタマイズ，移植，管理が難しい．メモリフットプリントが大きい |
| モジュール型 | 保守しやすい．フットプリントが小さい．必要な資源が少ない | インタフェースを介する間接参照．別々に開発されたモジュールに起因するバグ |
| マイクロカーネル | カーネルのフットプリントが小さい．検証しやすい | 移植性が限られる．ソフトウェア開発が複雑．ユーザ/カーネル境界を頻繁に越える |

**例 1.8.** ディスク上のファイルからの読み出しを考える．モノリシックカーネルやモジュール型カーネルではファイルシステムとディスクドライバがカーネル内で実行されるので，アプリケーションは 1 回のシステムコールを行い，要求は入るときと戻るときの 2 回，ユーザ/カーネル境界を越える．マイクロカーネルでは，アプリケーションがファイルシステムプロセスへ IPC メッセージを送り，そのプロセスがドライバプロセスへメッセージを送る．ドライバはファイルシステムへ返答し，ファイルシステムはアプリケーションへ返答する．4 つのメッセージはそれぞれカーネルに入り，また出ていくので，合計 8 回の横断となる．1 回の横断を 80 ns，読み出しを毎秒 50 000 回とすると，モノリシックな設計は 1 秒当たり $50\,000 \times 2 \times 80\,\text{ns} = 8\,\text{ms}$ を，マイクロカーネルは $50\,000 \times 8 \times 80\,\text{ns} = 32\,\text{ms}$ を横断に費やす．これには，関係するプロセスのアドレス空間の切り替えは含まれていない．

> **要点**
> モノリシックカーネルは完全で全体として最適化できるが，大きく管理が難しい．モジュール型カーネルは必要なサービスだけを追加するが，間接参照の代償を払う．マイクロカーネルは小さく検証しやすいが，頻繁なユーザ/ カーネル境界の横断と IPC の代償を払う．

## 1.8 Linux と Mac OS X

Linux システムは階層状に構成される（図 1.5 左）．最下層はハードウェアである．Linux カーネルはその上で直接カーネルモードで実行される．その上位のユーザモードでは，標準ライブラリが `open`，`close`，`read`，`write`，`fork` などの関数としてシステムコール

**出典** P1L2，Linux と Mac OS のアーキテクチャ；Tanenbaum と Bos [6] ch. 10；Silberschatz ら ch. 2．

**注意** printf や malloc はライブラリ関数であってシステムコールではない．必要なときにだけ write や brk/mmap を呼ぶ．

**ヒント** strace ./prog はプログラムが実際に行うシステムコールを列挙する．ライブラリ呼び出しには ltrace を使う．

インタフェースを提供し，シェル，エディタ，コンパイラなどの標準ユーティリティプログラムがそのライブラリを使い，利用者はユーティリティプログラムや自分のアプリケーションとやり取りする．これらの層の間の境界が，システムコールインタフェース，ライブラリインタフェース，ユーザインタフェースである．

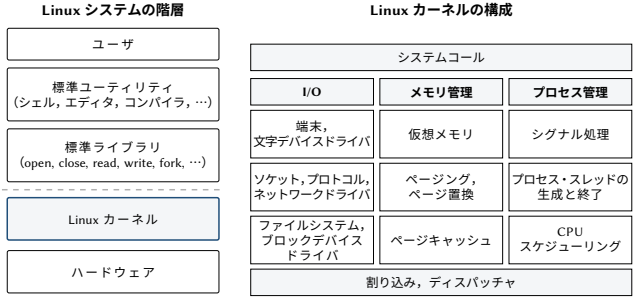



**図 1.5:** 左: Linux システムの階層．破線はユーザ/カーネル境界である．右: Linux カーネルの構成要素（Tanenbaum と Bos [6] による）．

Linux 6.6（2023 年）以降，スケジューラは EEVDF であり，第 7 章の CFS を置き換えた（カーネルドキュメント，sched-eevdf）．

カーネル自体（図 1.5 右）へは上部のシステムコールから入る．下部では割り込みを処理し，CPU をプロセス間で切り替えるディスパッチャ（第 7 章）を備える．その間に 3 つの構成要素がある．I/O の構成要素は，端末と文字デバイスのドライバ，ソケット，ネットワークプロトコルとネットワークデバイスのドライバ，ファイルシステムとブロックデバイスのドライバを含む．メモリ管理の構成要素は，仮想メモリ（第 8 章），ページングとページ置換，ページキャッシュ（第 11 章）を実装する．プロセス管理の構成要素は，シグナル（第 5 章）を扱い，プロセスとスレッドを生成・終了し，CPU をスケジュールする．各構成要素は他から独立に変更・置換でき，これがモジュール型の方式を可能にしている．

Mach，BSD，I/O Kit，カーネル拡張は 1 つのカーネルアドレス空間で実行されるので，純粋なマイクロカーネルではなくハイブリッドな設計である（Silberschatz ら ch. 2）．

Mac OS X（現在の macOS）は Mach マイクロカーネルを中心に構築されている（図 1.6）．Mach は，メモリ管理，スレッドスケジューリング，そして遠隔手続き呼び出し（第 13 章）を含む IPC という主要なプリミティブを実装する．BSD の構成要素は，BSD のコマンドラインインタフェース，POSIX API のサポート，ネットワーク I/O によって Unix との相互運用性を提供する．すべてのアプリケーション環境はこれら 2 つの上に層をなす．その下では，I/O Kit がデバイスドライバを開発するための環境であり，カーネル拡張は動的にカーネルへロードされる構成要素である．

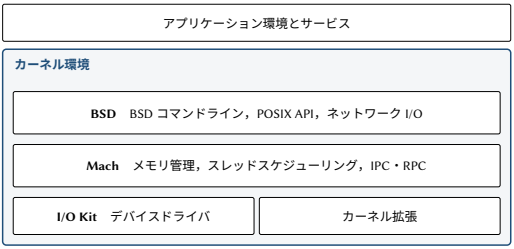



**図 1.6:** Mac OS X の構造．BSD 層，I/O Kit，カーネル拡張を伴う Mach マイクロカーネルであり，これらはすべてカーネルモードで実行される．

**要点 — 章のまとめ**
オペレーティングシステムは，アプリケーションとハードウェアの間にある特権的なソフトウェアの層である．ハードウェアの複雑さを抽象化の背後に隠し，ハードウェア資源の利用を調停し，アプリケーションを互いに隔離し保護する．それは抽象化，機構，方針から構築され，機構と方針の分離と，頻繁に起こる場合の最適化がその設計を導く．ハードウェアに支えられたユーザモードとカーネルモードがオペレーティングシステムを保護し，アプリケーションは安価ではないシステムコールとトラップによって境界を越え，オペレーティングシステムはシグナルでアプリケーションに通知する．モノリシック，モジュール型，マイクロカーネルの構成は，完全性，柔軟性，規模，性能を互いにトレードオフしており，モジュール型の Linux カーネルと Mach に基づく Mac OS X がそれを例示している．

第 2 章

# プロセスとプロセス管理

| | |
|---|---|
| **講義動画** | P2L1 |
| **教科書** | OSTEP [1] ch. 4–7，13；Silberschatz ら [2] ch. 3 |
| **キーワード** | プロセス，アドレス空間，仮想アドレス，ページテーブル，プロセス制御ブロック，コンテキストスイッチ，ホットキャッシュ，プロセス状態，fork，exec，CPU スケジューラ，タイムスライス，プロセス間通信，共有メモリ |

第 1 章では，オペレーティングシステムを，アプリケーションに代わってハードウェアを抽象化し調停する層として述べた．その最も基本的な抽象化がプロセス，すなわち実行中のアプリケーションの表現である．この章では，プロセスとは何か，どのような状態からなるか，オペレーティングシステムがその状態をどのように表現するか，そして多数のプロセスを同時にどのように管理するか–プロセスの間で CPU を切り替え，プロセスを生成し，スケジュールし，互いに通信させる–を述べる．

## 2.1 プログラムとプロセス

ディスクやフラッシュメモリに保存されたアプリケーションは**プログラム**である．すなわち，命令と初期データからなるファイルであり，それ自体は何もしない静的な実体である．プログラムが起動されると，オペレーティングシステムはそれをメモリへロードして実行を開始し，プログラムは能動的な実体となる．

**出典** P2L1，プロセスとは何か；OSTEP [1] ch. 4；Silberschatz ら [2] ch. 3.

▷ process

**定義 2.1** (プロセス). **プロセス**とは，実行中のプログラムのインスタンスである．すなわち，メモリにロードされた，実行中のプログラムの状態である．プロセスは**タスク**あるいは**ジョブ**とも呼ばれる．

同じプログラムを 2 回起動すれば 2 つのプロセスが生じる．両者は同じコードを実行するが，それぞれが自分の状態–例えば異なる入力データや，プログラム内の異なる位置–を持つ．プロセスは**能動的な**アプリケーションの実行状態を表すが，それはプロセスが常に CPU を使っていることを意味しない．ユーザの入力を待っているかもしれないし，自分の順番を待つ間に別のプロセスが CPU 上で実行されているかもしれない．プロセスは，実行状態とデータのほかに，読み書きの対象となる I/O デバイスなどのハードウェアも必要としうる．

## 2.2 プロセスのアドレス空間

プロセスは，実行中のアプリケーションのすべての状態–そのコード，データ，および実行中に生成するすべての変数–をカプセル化する．この状態の各要素は，そのアドレスによって一意に識別できなければならない．

**出典** P2L1，プロセスはどのように見えるか，プロセスのアドレス空間；OSTEP ch. 4，13.

▷ address space

**定義 2.2** (アドレス空間). プロセスの**アドレス空間**とは，オペレーティングシステムがプロセスの状態をカプセル化するために用いる抽象化であり，$V_0$ から $V_{\max}$ までのアドレスの範囲と，それらのアドレスに格納された状態からなる．これはプロセスのメモリ上の表現である．

アドレス空間内の状態にはいくつかの種類があり，図 2.1 のように配置される．

▷ text segment

**テキストとデータ** **テキストセグメント**はプログラムのコード，すなわちコンパイラが生成した命令を保持する．プログラムのロード時に初期化される大域変数などの静的データを保持するのが**データセグメント**である．どちらも静的な状態であり，プロセスが最初にロードされた時点から存在する．

▷ data segment

▷ heap

**ヒープ** **ヒープ**は，プロセスが実行中に動的に生成する状態，例えば `malloc` で確保したメモリを保持する．上位アドレスに向かって伸びる．

▷ stack

**スタック** **スタック**はアドレス空間の動的な部分であり，実行中に後入れ先出しの順序で伸縮する．プロセスが手続きを呼び出すと，呼び出し元を再開するために必要な状態，すなわち戻り番地と退避されたレジスタの値がスタックに積まれ，呼び出された手続きの局所変数がそこに確保される．手続きから戻ると，それらはすべて取り除かれる．スタックはアドレス空間の上端から始まり，下位アドレスに向かって伸びる．

`cat /proc/self/maps` は実際のアドレス空間を表示する．1 行が 1 つの領域であり，その開始アドレスと終了アドレス，アクセス権，背後のファイルを示す（proc(5)）．Linux のメインスレッドのスタック上限の既定値は 8 MiB である（`ulimit -s`）．それより深い再帰はセグメンテーション違反で終わる（getrlimit(2)）．

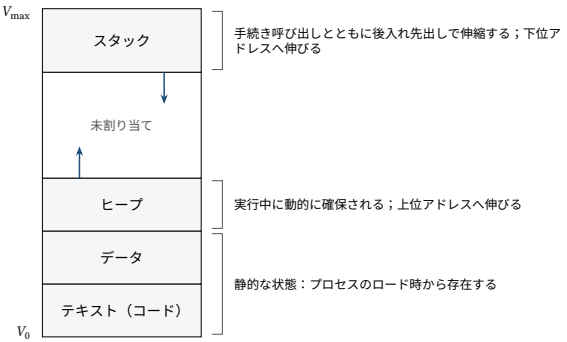



**図 2.1:** プロセスのアドレス空間．ヒープとスタックは互いに向かって伸びる．その間のアドレスは割り当てられていない．

### 2.2.1 仮想アドレスと物理アドレス

プロセスが用いる $V_0$ から $V_{max}$ までのアドレスは，マシンの物理メモリ上の実際の位置に対応している必要はない．

**出典** P2L1，プロセスのアドレス空間；OSTEP ch. 13.

**定義 2.3** (仮想アドレスと物理アドレス). プロセスのアドレス空間内のアドレスであり，プロセス自身が用いるものを**仮想アドレス**と呼ぶ．これに対して，マシンの物理メモリ（DRAM）上の位置のアドレスが**物理アドレス**である．

▷ virtual address
▷ physical address

メモリ管理ハードウェアとオペレーティングシステムは，プロセスが用いるすべての仮想アドレスを物理アドレスへ変換する．両者の対応付けは**ページテーブル**に記録され，オペレーティングシステムがこれをプロセスごとに保持する．2 種類のアドレスを分離することで，アドレス空間の配置が物理メモリの配置から解放される．

▷ page table

**例 2.4.** あるプロセスが変数 x を仮想アドレス 0x00402A10 に格納しているとする．そのページテーブルはこのアドレスを物理アドレス 0x1F3A0A10 へ対応付けている．プロセスが x を読み書きするときに使うのは 0x00402A10 であり，0x1F3A0A10 への変換はプロセスの関与なしに行われる．

**ヒント** 変換はページ単位—x86 では 4 KiB—で行われるので，アドレスの下位 12 ビットは変換を素通りする．ここでの 2 つのアドレスがともに A10 で終わるのはそのためである．

### 2.2.2 アドレス空間とメモリ管理

この対応付けは，物理メモリの使い方についてオペレーティングシステムに自由を与える．それが必要な理由は 2 つある．第一に，プロセスがアドレス空間の全体を使うことはまれであり，ヒープとスタックの間の領域のように，大きな部分はある時点では割り当てられていない．第二に，割り当てられたすべての状態を収めるだけの物理メモリがないかもしれない．32 ビットの仮想アドレスでは 1 つのプロセスが $2^{32}$ バイト，すなわち 4 GiB をアドレス指定でき，複数のプロセスを合わせれば，必要なメモリはマシンが備える量を容易に超える．

**出典** P2L1，アドレス空間とメモリ管理．

したがってオペレーティングシステムは，どのアドレス空間のどの部分を物理メモリのどこに置くかを動的に決める．一部は一時的にディスクへ移され，プロセスが必要としたときにメモリへ戻される（スワッピング，第 8 章）．オペレーティングシステムは，各プロセスについて，アドレス空間の各部分が現在どこに—メモリ上かディスク上か—あるかを知っていなければならない．これがページテーブルに保持される情報である．同じ情報によって，メモリアクセスの検証，すなわちプロセスが使おうとしているアドレスへのアクセスが実際に許されているかの確認も行える．

x86-64 ではハードウェアが 48 ビットのアドレスを変換し，Linux はその範囲の下半分，128 TiB を各プロセスに与える．近年のプロセッサの 5 段ページングはアドレスを 57 ビットへ拡張する（Linux の *5-level paging* 文書）．

プロセスはそれぞれ自分のアドレス空間を持ち，異なるプロセスのアドレス空間は同じ仮想アドレスの範囲にわたる．したがって同時に実行される 2 つのプロセスがまったく同じ仮想アドレスを使うことがあり，それぞれのページテーブルがそれを異なる物理的な位置へ対応付ける（図 2.2）．仮想アドレスは，それを使うプロセスと合わせて初めて位置を特定する．

**ヒント** 4 KiB のページを用いる 32 ビットのアドレスは，20 ビットのページ番号と 12 ビットのオフセットに分かれる．変換されるのはページ番号だけである（第 8 章）．

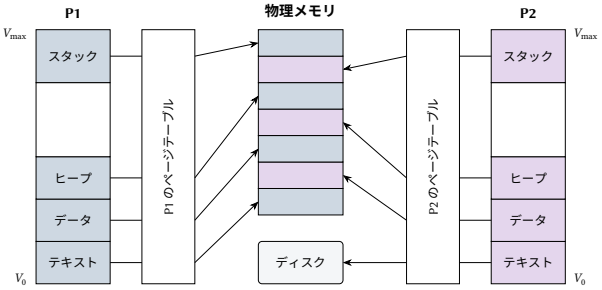



**図 2.2:** 同じ仮想アドレスの範囲を持つ 2 つのプロセス．それぞれのページテーブルは，割り当てられた領域を異なる物理フレームへ対応付けている．P2 のテキストは現在ディスク上にあり，割り当てられていない領域はどこにも対応付けられていない．

## 2.3 実行状態とプロセス制御ブロック

出典 P2L1，プロセスの実行状態，プロセス制御ブロックとは何か；OSTEP ch. 4.

オペレーティングシステムがプロセスを止めるときには，後でまったく同じ地点からそのプロセスを再開できるように，プロセスが何をしていたかを記録しなければならない．プログラムはバイナリ，すなわち命令の列にコンパイルされるが，その命令が厳密に順番どおりに実行されるとは限らない．ジャンプ，ループ，割り込みが順序を変える．CPU は，この列のどこにプロセスがいるかを常に知っていなければならない．

▷ program counter

▷ stack pointer

**プログラムカウンタ**（PC）は，次に実行する命令のアドレスを保持する．プロセスの実行中は，CPU のレジスタに保持される．CPU のその他のレジスタは，データの値，アドレス，そして実行の順序に影響する状態情報を保持する．**スタックポインタ**（SP）はスタックの先頭のアドレスを保持する．これらの値が合わさってプロセスの実行状態をなす．

▷ process control block

**定義 2.5** (プロセス制御ブロック). **プロセス制御ブロック**（PCB）とは，オペレーティングシステムがプロセスごとに保持するデータ構造である．プロセス状態，プロセス番号，プログラムカウンタとその他の CPU レジスタ，メモリの範囲と仮想から物理への対応付け，オープンしたファイルの一覧，優先度，シグナルマスク，CPU スケジューリング情報を含む．

**注意** Linux の 1 つの `task_struct` は 1 つのスレッド（第 3 章）を表す．プロセスはスレッドグループ，すなわち 1 つのスレッドグループ ID を共有するタスクの集まりであり，プロセスごとの状態はそれらが共有する構造体 `mm_struct`，`files_struct`，`signal_struct` に置かれる．

Linux では PCB は `struct task_struct` である．そのフィールドには，プロセス識別子（`pid`），状態，メモリの対応付けを記述する `mm`，オープンしたファイルの表である `files` などがある．

PCB の各フィールド（図 2.3）は，異なる時点で維持される．

- PCB はプロセスの生成時に作られ，初期化される．例えばプログラムカウンタはプログラムの最初の命令に設定される．
- 一部のフィールドは，対応する状態が変化したときに更新される．プロセスがより多くのメモリを要求すると，オペレーティングシステムはそれを割り当て，メモリの範囲と対応付けを更新する．
- その他のフィールドは，PCB 内で最新に保つにはあまりに頻繁に変化する．プログラムカウンタは命令ごとに変化するので，CPU が自分のレジスタに保持する．オペレーティングシステムは，CPU がプロセスのために保持している値を集め，そのプロセスが CPU 上での実行を止めるたびに PCB へ退避する．

`/proc/PID/status` はこれらのフィールドの多くを表示する．x86-64 では各タスクが 16 KiB のカーネルスタックも持つ（Linux の *Kernel Stacks* 文書）．

## 2.4 コンテキストスイッチ

出典 P2L1，PCB はどのように使われるか，コンテキストスイッチ；OSTEP ch. 6.

P1 と P2 を 2 つのプロセスとし，その PCB がオペレーティングシステムのメモリに格納されているとする．P1 の実行中，CPU のレジスタは P1 の実行状態を保持している．オペレーティングシステムが P1 に割り込むと，レジスタの内容を P1 の PCB へ退避し，P2 の状態を P2 の PCB からレジスタへ復元する．そして P2 が実行され，P1 は休止する．後にオペレーティングシステムが P2 に割り込むと，P2 の状態をその PCB へ退避し，P1 の状態を復元する．P1 は，止められたまさにその地点から再開する（図 2.4）．このスイッチはカーネルモードで実行され，アクティブなアドレス空間も切り替える．オペレーティングシステムは，CPU のページテーブルベースレジスタ（第 8 章）に，これから実行するプロセスのページテーブルを設定する．

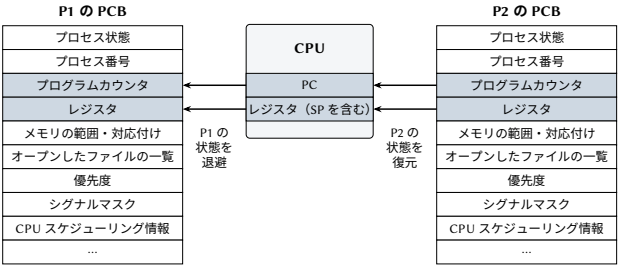



**図 2.3:** P1 と P2 のプロセス制御ブロック．網掛けのフィールドは，プロセスの実行中に CPU がレジスタに保持する値であり，プロセスが実行を止めるときに PCB へ退避され，再び実行されるときに PCB から復元される．

**定義 2.6 (コンテキストスイッチ).** **コンテキストスイッチ**とは，オペレーティングシステムが CPU をあるプロセスのコンテキストから別のプロセスのコンテキストへ切り替えるために用いる機構である．

▷ context switch

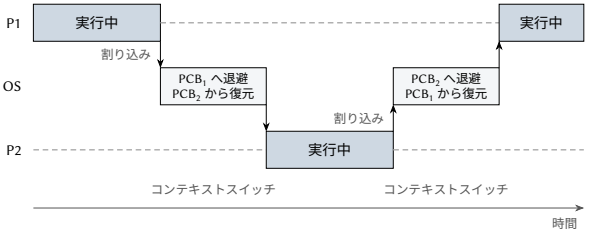



**図 2.4:** P1 と P2 の間の 2 回のコンテキストスイッチ．各プロセスは他方が実行される間は休止（破線）しており，その状態は PCB の中で待っている．

コンテキストスイッチが高価である理由は 2 つある．

**直接のコスト** 一方のプロセスの状態を退避し他方の状態を復元するロード命令とストア命令の実行，およびページテーブルベースレジスタの再設定に費やされる CPU サイクル．

**間接のコスト** プロセッサキャッシュへの影響．キャッシュアクセスは数サイクル程度で済むのに対し，主記憶へのアクセスは数百サイクル程度を要する．

Intel Skylake では L1 ヒットが 4–5 サイクル，L2 が約 12 サイクル，L3 が約 42 サイクル，DRAM はさらに 50 ns 程度（7-cpu.com）．

プロセスの実行中は，そのプロセスが使うデータの多くがプロセッサキャッシュに存在する．そのようなキャッシュは**ホットキャッシュ**であり，ほとんどのアクセスがキャッシュから供給され，プロセスは最良の性能を発揮する．コンテキストスイッチの後，次のプロセスがこのデータの多くを自分のデータで置き換える．最初のプロセスが再開したときに見出すのは**コールドキャッシュ**である．自分のデータはもはやキャッシュになく，キャッシュが再び満たされるまで，そのアクセスは主記憶から供給されるほかないキャッシュミスとなる．したがってオペレーティングシステムは，コンテキストを切り替える頻度を抑えるべきである．

▷ hot cache

▷ cold cache

**例 2.7.** レジスタの退避と復元に 1 000 サイクルかかり，アクセスのコストはデータがキャッシュにあるとき 4 サイクル，主記憶から読まなければならないとき 200 サイクルであるとする．P1 が再開した後，P2 の実行中にキャッシュから追い出された 5 000 個のデータ項目にアクセスするなら，それぞれへの最初のアクセスは $200 - 4 = 196$ サイクルを余分に要し，合計で $5\,000 \times 196 = 980\,000$ サイクルとなる．間接のコストはスイッチの直接のコストのほぼ 1000 倍である．

Li ら [7] の測定では，データにアクセスしないプロセスの間のスイッチは 1 回 3.8 µs であり，ワーキングセットが 512 KiB の L2 に収まらず大きなストライドでアクセスされると，1 回あたり最大で約 1.5 ms になる（2.0 GHz の Xeon，2007 年）．

**注 2.8.** オペレーティングシステムは，キャッシュの状態にかかわらずコンテキストを切り替える．例えば，より優先度の高いプロセスが実行可能になったときや，実行中のプロセスが割り当てられた時間を使い切ったとき（2.7 節）である．スイッチが残すコールドキャッシュは，CPU を共有するために払う代償の一部である．

**要点**
PCB は，オペレーティングシステムがプロセスを止めて後で再開するために必要なすべてを保持する．コンテキストスイッチは，あるプロセスの CPU 状態をその PCB へ退避し，別のプロセスの状態をその PCB から復元する．その隠れたコスト，すなわち残されるコールドキャッシュは，退避と復元のコストをはるかに上回りうる．

## 2.5 プロセスのライフサイクル

**出典** P2L1，プロセスのライフサイクル，プロセスのライフサイクル: 状態；OSTEP ch. 4.

CPU から見ると，プロセスは実行中と非実行中を交互に繰り返す．割り込まれて切り替えられた実行中のプロセスは休止する．すなわち，もはや CPU 上にない．それは依然として実行の準備ができており，スケジューラが CPU へディスパッチすれば再び実行される．プロセスが休止する理由は他にもあり，オペレーティングシステムはこれらの状況を**プロセス状態**によって区別し，PCB に記録する．プロセスは次のいずれかの状態にある（図 2.5）．

**注意** ここでの休止は CPU 上にないという意味であり，実行可能なプロセスがないときに動くカーネルのアイドルタスクのことではない．

▷ process state
▷ new state

**新規** 生成されつつあるプロセスは**新規状態**にある．オペレーティングシステムは**受け入れ制御**を行い，そのプロセスを受け入れられるかを判断する．受け入れる場合は PCB といくらかの初期資源を割り当てて初期化し，プロセスは**受け入れ**られる．

▷ ready state

**実行可能** **実行可能状態**のプロセスは，CPU 以外に実行に必要なものをすべて備えている．スケジューラが選びさえすれば実行できるのであり，それまでは順番を待つ．

▷ running state

**実行** **実行状態**のプロセスは CPU 上で実行されている．**スケジューラによるディスパッチ**の後，PCB に格納された地点から実行を続ける．実行中のプロセスがこの状態を離れるのは次の 3 通りである．割り込まれて–例えば CPU 上の時間を使い切って–コンテキストスイッチの後に実行可能状態へ戻る．ディスクからの読み出しやキーボード・タイマのイベント待ちのような長い操作を始めて待ち状態に入る．あるいは終了する．

▷ waiting state

**待ち** **待ち状態**（**ブロック状態**とも呼ぶ）のプロセスは，I/O 操作の完了などの何らかの事象が起こるまで先へ進めない．事象が起これば，プロセスは再び実行可能になる．

**注意** 事象の完了はプロセスを実行可能にするのであって，実行中にはしない．CPU は，直ちに行われることもあるディスパッチによって与えられる．

**終了** すべての操作を終えたプロセス，あるいは誤りを検出したプロセスは *exit* を呼び，成功か誤りかを示す終了コードを返す．不正なメモリアクセスなどの障害を引き起こしたプロセスは，オペレーティングシステムによって終了させられる．いずれの場合もプロセスは**終了状態**に入る．

▷ terminated state

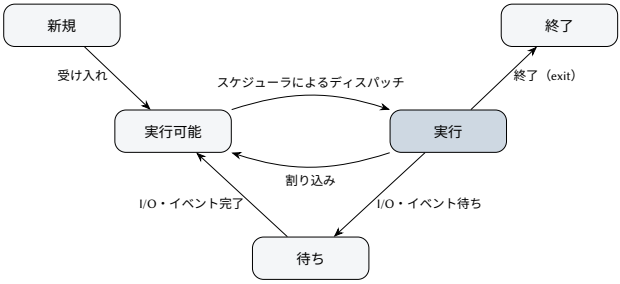



**図 2.5:** プロセスの状態と，その間の遷移．

**注意** Linux は実行可能と実行を区別しない．どちらも TASK_RUNNING であり，ps はどちらも R と表示する．S と D は割り込み可能・不可能の 2 種類の待ちであり，T は停止したプロセスである．

表 2.1 は，1 つのプロセスをそのライフサイクルに沿ってたどったものである．プロセスに CPU を与える遷移はディスパッチだけであり，他のすべての遷移は CPU を取り上げるか，CPU を持たないままにする．

## 2.6 プロセスの生成

**出典** P2L1，プロセスのライフサイクル: 生成；OSTEP ch. 5；Silberschatz ら ch. 3.

プロセスは他のプロセスによって生成される．システムが起動してオペレーティングシステムがロードされると，いくつかの初期プロセスが生成され，その一部は特権的なアクセスを持つ．ユーザがログインすると，そのユーザのためにシェルプロセスが生成され，シェルから起動されたプログラム，例えば ls や emacs は，シェルが生成した新しいプロセスとして実行される．別のプロセスを生成したプロセスは，そのプロセスの**親プロセス**であり，生成された側のプロセスは，生成した側の**子プロセス**である．最初のプロセスを除くすべてのプロセスが親を持つので，システムのプロセスは木をなす（図 2.6 (a)）．Unix

▷ parent process
▷ child process

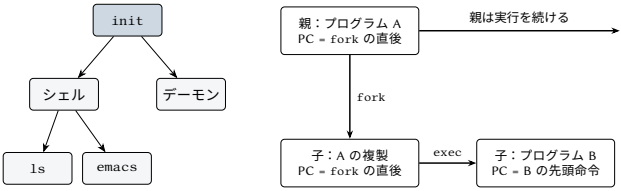
**表 2.1:** 実行され，一度割り込まれ，ファイルを読み，終了するプロセスの状態の推移．待つことになるキューと，レディキューから取り出すディスパッチについては 2.7 節 で述べる．

| 事象 | 遷移 | 遷移後の状態 | 所在 |
|---|---|---|---|
| プロセスが生成される | — | 新規 | — |
| PCB を初期化し受け入れ | 受け入れ | 実行可能 | レディキュー |
| スケジューラが選ぶ | ディスパッチ | 実行 | CPU |
| CPU 上の時間が満了 | 割り込み | 実行可能 | レディキュー |
| スケジューラが選ぶ | ディスパッチ | 実行 | CPU |
| ディスク読み出しを発行 | I/O・イベント待ち | 待ち | ディスク I/O キュー |
| ディスク読み出しが完了 | I/O・イベント完了 | 実行可能 | レディキュー |
| スケジューラが選ぶ | ディスパッチ | 実行 | CPU |
| `exit` を呼ぶ | 終了（exit） | 終了 | — |

系システムではその根は `init`，すなわちブート後にカーネルが最初に開始するユーザプロセスであり，他のすべてのユーザプロセスはそこから派生する．（現在の多くの Linux ディストリビューションでは，この最初のプロセスとして動くプログラムは `systemd` である．）

▷ init

`pstree -p` は，1 を含むすべての PID を付けてプロセスの木を表示する（pstree(1)）．




*(a)* プロセスの木　　*(b)* 新しいプログラムの起動

**図 2.6:** (a) プロセスは初期プロセスを根とする木をなす．(b) `fork` で生成された子は親の複製であり，`exec` がそのプログラムを新しいものに置き換える．

ほとんどのオペレーティングシステムは，プロセスを生成する 2 つの機構を提供する．Unix ではそれらを `fork` と `exec` と呼ぶ．

`fork` オペレーティングシステムは，`fork` システムコールによって子のために新しい PCB を作り，親の PCB をそこへコピーし，親のアドレス空間の複製を子に与える．子は親の複製であり，同じプログラムと同じプログラムカウンタを持つので，呼び出しの後，親も子も `fork` の次の命令から実行を続ける．

▷ fork

`exec` プロセスは，`exec` システムコールによって自分のイメージを新しいプログラムで置き換える．オペレーティングシステムはプロセスのアドレス空間を新しいプログラムで置き換え，それを記述するように PCB を更新する．プログラムカウンタは新しいプログラムの最初の命令を指すようになり，プロセス識別子とオープンしたファイルは保たれる．ただし close-on-exec の印が付いたもの（FD_CLOEXEC, O_CLOEXEC）は除く．

▷ exec

異なるプログラムを実行するプロセスを生成するには，親が `fork` を呼び，続いて子が `exec` を呼ぶ（図 2.6 (b)）．親と子は同じ `fork` から同じ場所へ戻るので，その戻り値によって互いを区別する．`fork` は，子では 0 を，親では子のプロセス識別子を返し，子を生成できなかった場合は親で −1 を返す．

**注意** `fork` がコピーするのはメモリではなくページテーブルである．ページが複製されるのは，どちらかが書き込むときだけである（コピーオンライト，第 8 章）．

**ヒント** `fork` は 2 回，それぞれのプロセスで 1 回ずつ戻る．0 は「自分は子である」を意味する．

Android は，アプリケーションを素早く起動するために `fork` を用いる．そのアプリケーションプロセスは，ブート時に開始され，アプリケーションの起動だけを目的とするデーモンである **Zygote** から派生する．Zygote はランタイムとよく使われるライブラリを一度だけロードしておく．[1] アプリケーションを起動するときには Zygote が fork し，Zygote がロードしたものをすでにすべて含む子が，そのアプリケーションのプロセスとなる．

▷ Zygote

[1] `fork` は親のページをコピーオンライトで共有するので，どのアプリケーションプロセスも，ランタイムとあらかじめロードされたクラスがすでに対応付けられた状態で始まり，誰も書き込まない限り物理ページは 1 組のままである．Zygote はこうして起動時間とメモリを同時に節約する．

**例 2.9.** fork で子プロセスを生成し，その中で exec により ls -l を実行し，子が終わるまで待つプログラムを示す．[2]

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {                 /* 子 */
        execlp("ls", "ls", "-l", (char *)NULL);
        _exit(127);                 /* exec 失敗時のみ */
    } else if (pid > 0) {           /* 親 */
        waitpid(pid, NULL, 0);      /* 子の終了を待つ */
    }
    return 0;
}
```

execlp が成功すれば，子は ls のコードを実行する．この呼び出しが戻ることはない．

[2] exit したプロセスは，親が終了コードを読み取る—ここでは waitpid で—まで PCB を保持し続ける．それまではゾンビ（状態 Z）である．親が先に終了した子は，最も近いサブリーパーの祖先（PR_SET_CHILD_SUBREAPER，例えば systemd のユーザマネージャ）があればそれに，なければ init に引き取られ回収される．

## 2.7 CPU スケジューラ

出典 P2L1，CPU スケジューラの役割；OSTEP ch. 6–7．

CPU がプロセスの実行を開始するには，そのプロセスが実行可能でなければならず，実行可能なプロセスはいつでも複数ありうる．オペレーティングシステムは，そのうちのどれを次に実行するか，そしてどれだけの間実行させるかを決めなければならない．

▷ CPU scheduler

**定義 2.10** (CPU スケジューラ)．**CPU スケジューラ**とは，プロセスが CPU をどのように使うかを管理するオペレーティングシステムの構成要素である．実行可能なプロセスのどれを CPU へディスパッチするか，そしてどれだけの間実行させてよいかを決める．

▷ ready queue

実行可能なプロセスは**レディキュー**（Linux ではランキュー）で CPU を待つ．そのうちの 1 つに CPU を与えるために，オペレーティングシステムは 3 つの操作を行う．

▷ preemption

**プリエンプト** 実行中のプロセスに割り込み，そのコンテキストを PCB へ退避する．これを**プリエンプション**と呼ぶ．

**スケジュール** スケジューラがアルゴリズムを実行し，レディキューから次のプロセスを選ぶ．

▷ dispatch

**ディスパッチ** オペレーティングシステムは選ばれたプロセスのコンテキストへ切り替え，CPU 上で実行させる．これがプロセスの**ディスパッチ**である．

CPU はプロセスが必要とする資源であるから，その時間はできるだけ，オペレーティングシステムの実行ではなくプロセスの実行に費やされるべきである．そのためスケジューラには，効率的なアルゴリズムと，その効率的な実装，そして待機中のプロセスとそのスケジューリング情報を表す効率的なデータ構造—とりわけレディキュー—が必要である．

Linux 2.6.23 は O(1) スケジューラを CFS で置き換えた．CFS のランキューは赤黒木であり，次のタスクの選択は O(1)，挿入は O(log n) である（カーネル文書 sched-design-CFS）．第 7 章．

### 2.7.1 スケジューラを動かす頻度

出典 P2L1，プロセスの長さ；OSTEP ch. 7．

スケジューラを動かす回数が多いほど，プロセスから奪う CPU 時間は増える．各プロセスが時間 $T_p$ だけ実行された後，$t_{\text{sched}}$ を要するスケジューラが次のプロセスを選ぶとする（図 2.7）．時間 $t_{\text{sched}}$ は，2 つのタイムスライスの間にオペレーティングシステムが行うことのすべて，すなわちスケジューリングアルゴリズムの実行，キューのデータ構造の更新，そして 2.4 節 のコンテキストスイッチからなる．これを $n$ 回繰り返すと，有用な処理に費やされる CPU 時間の割合は次のようになる．

$$\text{有用な CPU 処理} = \frac{\text{総処理時間}}{\text{総時間}} = \frac{nT_p}{nT_p + nt_{\text{sched}}} = \frac{T_p}{T_p + t_{\text{sched}}}. \tag{2.1}$$

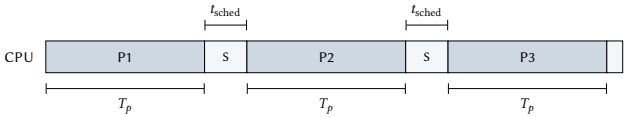



**図 2.7:** 各プロセスは $T_p$ ずつ実行され，その間にスケジューラ（S）が $t_{\text{sched}}$ だけ実行される．

▷ timeslice

プロセスが CPU 上で割り当てられる時間 $T_p$ が，そのプロセスの**タイムスライス**であ

る．$t_{\text{sched}}$ に対してタイムスライスが長いほど，有用な割合は大きくなる．タイムスライスがスケジューラ 1 回分の長さしかなければ CPU 時間の半分が失われ，その 10 倍の長さのタイムスライスなら有用な割合は $10/11 \approx 91\,\%$ となる．

**例 2.11.** $t_{\text{sched}} = 0.5\,\text{ms}$ とする．タイムスライスが 2 ms なら，式 (2.1) より有用な処理は $2/2.5 = 80\,\%$ であり，タイムスライスが 20 ms なら $20/20.5 \approx 97.6\,\%$ となる．

実際の値もこの程度である．Linux の CFS は 6 ms の周期を実行可能なタスクで分け合わせ，1 タスクあたり 0.75 ms を下限とした．Linux 6.6 の EEVDF は 0.75 ms の基本スライスを用いる（いずれも基本値であり，既定では CPU 数に応じて大きくされる）．

しかし，長いタイムスライスにも代償がある．1 つのプロセスが CPU を握っている間，他の実行可能なプロセスは順番を待つ．したがってスケジューラを選ぶには設計上の判断が要る．どのようなタイムスライスの値が適切か，そしてどの指標によって次に実行するプロセスを決めるかである．第 7 章ではこれらの問いを扱う．

### 2.7.2 プロセスと I/O

実行中のプロセスは，例えばディスクから読み出すために I/O 要求を発行することがある．オペレーティングシステムはその要求をデバイスへ届け，プロセスをそのデバイスに対応する *I/O* **キュー**に置く．プロセスは，デバイスが操作を完了して要求に応答するまでそこで待つ．その後プロセスは再び実行可能になる．システムの負荷に応じてレディキューに置かれるか，他に待っているプロセスがなければ直ちに CPU へディスパッチされることもある．

**出典** P2L1，I/O はどうなるか；Silberschatz ら ch. 3.

図 2.8 は，プロセスがレディキューに入る，あるいは再び入る経路を示している．

- I/O 要求がデバイスによって処理された．
- タイムスライスが満了した．
- `fork` によって新しいプロセスが生成された．子がレディキューに置かれる．
- 割り込みを待っていて，その割り込みが発生した．

この待ちは，この章の他のすべてと比べて長い．4 KiB の読み出しには，NVMe SSD で数十マイクロ秒，回転ディスクで 10 ms 程度を要する．

割り込みの処理自体はコンテキストスイッチではない．ハンドラは，割り込まれたプロセスのコンテキストのまま，カーネル内で実行される（第 5 章）．

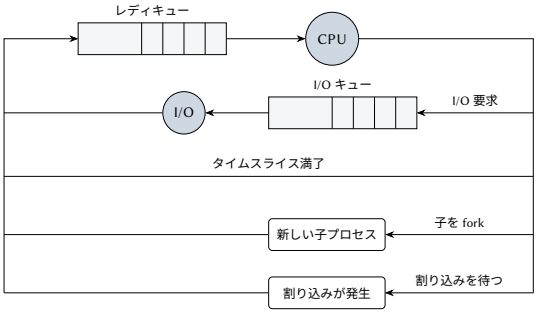



**図 2.8:** プロセスがレディキューに入る経路．どの経路もそこで終わる．`fork` の経路でキューに加わるのは，呼び出しを発行したプロセスではなく新しい子である．

I/O キューにあるプロセスは，デバイスが要求を完了するまでスケジューラの制御の外にある．スケジューラが選ぶのはレディキューにあるプロセスだけであり，いつ，どのプロセスへコンテキストを切り替えるかを決める．

> **要点**
> スケジューラは実行中のプロセスをプリエンプトし，レディキューから次のプロセスを選んでディスパッチする．スケジューラは同じ CPU 上で実行されるので，それ自身の時間はオーバーヘッドであり（式 (2.1)），$t_{\text{sched}}$ に対してタイムスライスが長くなるほど小さくなる．

## 2.8 プロセス間通信

多くのアプリケーションは，互いにやり取りしなければならない複数のプロセスとして構成される．例えばウェブアプリケーションは，クライアントの要求を処理するウェブサーバプロセスと，データを保存するデータベースプロセスからなりうる．しかしオペレーティングシステムはプロセスを互いに隔離する（第 1 章）．各プロセスは自分のアドレス空間を持ち，どのプロセスも他のプロセスのメモリにアクセスできない．

**出典** P2L1，プロセスは相互作用できるか；Silberschatz ら ch. 3.

▷ inter-process communication

**定義 2.12 (プロセス間通信).** プロセスが相互作用するためにオペレーティングシステムが提供する機構を**プロセス間通信**（IPC）と呼ぶ．IPC の機構は，アドレス空間の間でデータや情報を転送し，プロセスの保護と隔離を保ち，アプリケーションに柔軟性と良好な性能を与える．

**注意** IPC はプロセスのためのものである．1 つのプロセスのスレッド（第 3 章）は，そのいずれも使わずにアドレス空間を共有する．

IPC には基本となる 2 つの種類がある（図 2.9）．

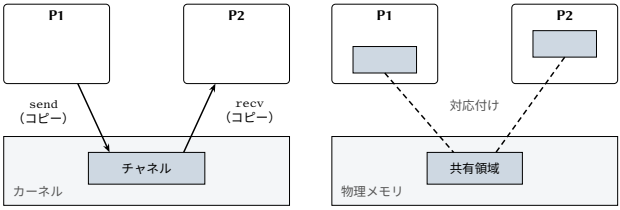
▷ message passing

**メッセージパッシング　メッセージパッシング**では，オペレーティングシステムが通信チャネル，例えばカーネルメモリ内の共有バッファを確立する．プロセスはシステムコールを用いて，チャネルに書き込むことでメッセージを送り，チャネルから読むことでメッセージを受け取る．オペレーティングシステムがチャネルを管理し，send や recv のような，明確に定義された同一のインタフェースをすべてのアプリケーションに提供する．その代償はオーバーヘッドである．あらゆる情報が，送信側のアドレス空間からカーネル内のチャネルへ，さらにそこから受信側のアドレス空間へとコピーされ，その両側でカーネル境界の横断が生じる．

▷ shared memory

**共有メモリ　共有メモリ**では，オペレーティングシステムが物理メモリの領域を確保し，それを両方のプロセスのアドレス空間へ対応付ける．プロセスはこのメモリを，自分のメモリと同じように直接読み書きし，オペレーティングシステムは介在しない．その代償は，オペレーティングシステムがそのメモリの使い方を定めなくなることである．明確に定義された固定のインタフェースはなく，プロセスはデータの形式について合意し，アクセスを自分たちで調整しなければならない（同期：第 10 章）．開発者はアプリケーションごとにこのコードを書き直すことになりかねず，誤りも生じやすい．

POSIX での名前は pipe，mq_open，mmap と組み合わせる shm_open である（pipe(7)，mq_overview(7)，shm_overview(7)）．第 9 章．




*(a)* メッセージパッシング　　*(b)* 共有メモリ

**図 2.9:** (a) メッセージパッシングは，すべてのメッセージをカーネル内のチャネルへコピーし，そこから再びコピーする．(b) 共有メモリでは，同じ物理ページが両方のアドレス空間へ対応付けられる．

共有メモリは，通信のコストを個々のやり取りから設定時へ移す．領域が一度対応付けられれば，1 回のやり取りにコピーもカーネル境界の横断も伴わないが，対応付けの作成自体が高価なので，2 つの機構のどちらが安上がりかは，プロセスが何回やり取りするかで決まる．

**ヒント** 損益分岐点は，設定のコストを 1 回のやり取りあたりの節約で割った値である．ここでは $300/(6-1) = 60$ 個のメッセージとなる．

**例 2.13.** カーネルのチャネルを通じてメッセージを 1 つ送るコストが 6 µs（2 回のシステムコールと 2 回のコピー）であり，同じデータを共有メモリでやり取りするコストが，一度限りの 300 µs で対応付けを確立した後は 1 µs であるとする．$n$ 個のメッセージに対し，共有メモリのコストは $300 + n$ マイクロ秒，メッセージパッシングのコストは $6n$ マイクロ秒である．共有メモリが安上がりになるのは $300 + n < 6n$ のとき，すなわちメッセージが 60 個を超えるときだけである．

現在の x86 Linux では，パイプを通る往復は数マイクロ秒から 10 マイクロ秒程度，Unix ドメインソケットではそれよりやや大きく，最小のシステムコールは 100 ナノ秒程度である（lmbench 流の測定：lat_pipe，lat_unix，lat_syscall；マシンに依存する）．

Unix 系システムの IPC 機構とその利用は第 9 章の主題である．

**要点 — 章のまとめ**
プロセスとは実行中のプログラムであり，そのアドレス空間–テキスト，データ，ヒープ，スタックからなり，ページテーブルによって仮想アドレスから物理アドレスへ変換される–と，オペレーティングシステムが PCB に保持する実行状態とによって表される．コンテキストスイッチは，あるプロセスの状態をその PCB へ退避し，別のプロセスの状態を復元するもので，直接のコストと，コールドキャッシュという間接のコストを伴う．プロセスは新規，実行可能，実行，待ち，終了の状態の間を移り，`fork` と `exec` によって生成される．CPU スケジューラは次に実行する実行可能なプロセスを選び，そのオーバーヘッドはタイムスライスに依存する．そして IPC は，メッセージパッシングまたは共有メモリによって，隔離されたプロセスの間の通信を可能にする．

第 3 章

# スレッドと並行性

| | |
|---|---|
| **講義動画** | P2L2 |
| **教科書** | OSTEP [1] ch. 26–32；Silberschatz ら [2] ch. 4–8 |
| **キーワード** | スレッド，マルチスレッド，並行性，競合状態，ミューテックス，条件変数，リーダ・ライタ問題，スプリアスウェイクアップ，デッドロック，カーネルレベルスレッド，ユーザレベルスレッド，マルチスレッドモデル，ボス・ワーカーパターン，パイプラインパターン |

第 2 章で述べたプロセスは実行コンテキストを 1 つしか持たないので，一度に 1 つの命令列しかたどれない．この章では，1 つのアドレス空間を共有する複数の実行コンテキストをプロセスに与えるスレッドと，そうしたスレッドが共有データへのアクセスを協調させるための機構–ミューテックスと条件変数–を導入する．説明は Birrell によるスレッドプログラミングのチュートリアル [8] に沿って進め，その後デッドロック，ユーザレベルスレッドとカーネルレベルスレッドの関係，マルチスレッドアプリケーションを構成するための一般的なパターンを扱う．

## 3.1 プロセスとスレッド

プロセスは，アドレス空間（ページテーブルに保持される仮想アドレスと物理アドレスの対応付けを通じて到達するコード，データ，ヒープ），開いたファイルなどオペレーティングシステムがそのプロセスのために保持する資源，そして実行コンテキスト，すなわちプログラムカウンタとスタックポインタを含む CPU レジスタの値とスタックによって表される．オペレーティングシステムはこれらすべてをプロセス制御ブロックに記録する．このようなプロセスはシングルスレッドである．

**出典** P2L2，プロセスとスレッドの違い；Birrell [8]；OSTEP ch. 26.

▷ thread

**定義 3.1** (スレッド). **スレッド**とは，プロセス内の 1 つの実行コンテキストである．各スレッドは独自のプログラムカウンタ，スタックポインタ，レジスタ値，スタックを持つ．プロセスのすべてのスレッドは，そのプロセスのアドレス空間（コード，データ，ヒープ）と，開いたファイルなどのその他の資源を共有する．

▷ multithreading

1 つのプロセスの中で複数のスレッドを実行することを**マルチスレッド**と呼び，そのようなプロセスを**マルチスレッドプロセス**と呼ぶ．スレッドは仮想アドレスと物理アドレスの対応付けを共有するので，すべてのスレッドが同じコードとデータにアクセスできる．しかし任意の時点で，各スレッドが実行している命令，処理している入力の部分，進行中の手続き呼び出しの連鎖は，それぞれ異なりうる．図 3.1 のスレッドごとの状態は，まさにこれらの違いを捉えている．プログラムカウンタとレジスタはスレッドがどこにいて何を計算しているかを記録し，専用のスタックは実行中の呼び出しの局所変数と戻りアドレスを保持する．

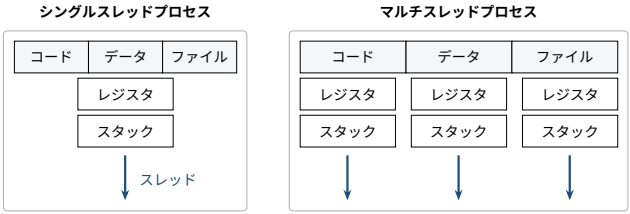



**図 3.1:** シングルスレッドプロセスとマルチスレッドプロセス．網掛けの状態（コード，データ，開いたファイル）はすべてのスレッドが共有する．各スレッドは PC と SP を含む独自のレジスタと，独自のスタックを持つ．

Linux ではスレッドは `clone(2)` に CLONE_VM を指定して生成されるタスクであり，カーネルは別個のスレッドオブジェクトを持たない（第 5 章）．

マルチスレッドプロセスに対するオペレーティングシステムの表現は，それに応じてシングルスレッドプロセスの PCB より多くの情報を含む．すべてのスレッドが共有する情報は 1 つだけ保持し，各スレッドについてはその実行コンテキストを記録する別個のデータ構造を保持する．

## 3.2 スレッドの有用性

複数の CPU を持つマシンでは，スレッドは 2 通りの方法でアプリケーションを高速化できる．

**同じコード，異なるデータ** すべてのスレッドが同じコードを実行し，それぞれが入力の異なる部分を処理する．例えば行列積では，各スレッドが結果の行の異なる部分集合を計算でき，スレッドは異なる CPU 上で並列に実行される．1 つのタスクの作業をこのように分割することを**並列化**と呼ぶ．

**異なるコード** スレッドごとにプログラムの異なる部分を実行する．あるスレッドが入力を処理し別のスレッドが画面を描画することもあれば，種類の異なる要求にそれぞれ専用のスレッドを割り当てることもある．スレッドを特定のタスクに専念させることを**特化**と呼ぶ．

出典 P2L2，スレッドはなぜ有用か，単一 CPU におけるマルチスレッドの利点，アプリケーションと OS コードにとっての利点；OSTEP ch. 26．

▷ parallelization

▷ specialization

特化にはさらに 2 つの利点がある．第一に，スレッドを異なる方法で管理できる．例えば，より重要なタスクや顧客を扱うスレッドに高い優先度を与えられる．第二に，性能はスレッドの状態のうちどれだけがプロセッサキャッシュにあるかに依存するので，コードの小さな部分を繰り返し実行するスレッドは，すべてを実行するスレッドよりもホットキャッシュ（第 2 章）で動作することが多い．

### 3.2.1 スレッドとプロセスの比較

同じ並列性は，シングルスレッドのプロセスを複数実行することでも得られる．それでもマルチスレッドによる実装は，3 つの点でより効率的である．

- **メモリ**: スレッドは 1 つのアドレス空間を共有するが，プロセスはそれぞれ独自のアドレス空間と独自の状態のコピーを必要とする．マルチスレッドアプリケーションは必要なメモリが少なく，したがってスワッピングを必要とする可能性も低い．
- **コンテキストスイッチ**: 同じプロセスのスレッド間の切り替えではアドレス空間を切り替えない．仮想アドレスと物理アドレスの対応付けはそのまま残るので，プロセスのコンテキストスイッチのうちコストの高い部分を避けられる．
- **通信**: スレッドは共有変数を通じてデータを交換する．プロセスにはプロセス間通信（IPC）が必要である．メッセージパッシングはやり取りのたびにカーネルに入り，共有メモリはまずオペレーティングシステムによって設定されなければならないので，プロセス間の通信はよりコストが高い．

**例 3.2.** あるアプリケーションが，プロセスごとに別々に保持しなければならない 400 MB のデータ（オペレーティングシステムがプロセス間で共有できるコードページはここでは無視する）を持ち，スタックその他の実行コンテキストごとの状態に約 1 MB を必要とするとする．それぞれがデータのコピーを持つ 4 つのプロセスとして実行すると約 $4\times401 = 1604$ MB を占め，1 つのプロセスの 4 つのスレッドとして実行すると約 $400 + 4\times1 = 404$ MB を占める．

Linux ではスレッドのスタックは既定でアドレス空間 8 MiB（RLIMIT_STACK）であり，実際に触れたページだけがメモリを占める．10 000 個のスレッドは約 78 GiB のアドレス空間を予約するが，実際に確保するのはそれぞれ数ページにすぎず，スレッドごとの 16 KiB のカーネルスタックを含めても合計で数百メガバイトである．

### 3.2.2 単一 CPU 上のスレッド

スレッドは，CPU よりスレッドの方が多い場合でも有用である．スレッドがディスク読み出しのようなブロックする要求を発行すると，要求が完了するまでそのスレッドにはすることがなく，その間の時間 $t_{\text{idle}}$ だけ CPU はアイドルになってしまう．代わりに CPU は別のスレッドに切り替え，要求が完了したら切り替えて戻ることができる（図 3.2）．このとき別のスレッドは $t_{\text{idle}} - t_{\text{ctx}}$ の CPU 時間を得る一方，ブロックしたスレッドは本来より $t_{\text{ctx}}$ だけ遅れて再開する．2 回の切り替えのコストは合わせて $2t_{\text{ctx}}$ である．切り替えが有利になるのは，得られる時間が遅れを上回るとき，すなわち

$$t_{\text{idle}} \;>\; 2\,t_{\text{ctx}} \tag{3.1}$$

が成り立つときである．スレッド間のコンテキストスイッチはアドレス空間の切り替えを避けるので，その $t_{\text{ctx}}$ はプロセス間の切り替えより小さく，式 (3.1) はより短いアイドル期間でも成り立つ．したがってマルチスレッドにより，1 つの CPU でもプログラムはアイドル時間を他のスレッドの実行に活用できる．

ディスクのシーク（10 ms 程度）や SSD の読み出し（100 µs 程度）は 2 回の切り替えを十分に償うが，メモリアクセス程度の待ち（100 ns 未満）は決して償わない．短いロック待ちでは，ブロックするよりスピンする方が有利である（第 10 章）．

この丸めた値は計算を簡単にするためのものであり，実測値はマイクロ秒の単位である．lmbench（McVoy と Staelin，USENIX 1996）は 1996 年の 200-MHz P6 で 1 回のコンテキストスイッチに 6 µs を報告しており，2–3 GHz のコアでは 1 マイクロ秒を下回る（OSTEP ch. 6）．

**例 3.3.** スレッドのコンテキストスイッチに 0.05 ms，プロセスのコンテキストスイッチに 0.5 ms かかるとする．スレッドなら 0.1 ms より長いブロッキング待ちはすべて別のスレッドが利用できるが，プロセスでは 1 ms より長い待ちに限られる．4 ms の待ちは，別のスレッドに $4 - 0.05 = 3.95$ ms の CPU 時間を与え，ブロックしたスレッドを 0.05 ms 遅らせる．別のプロセスの場合，これらの値は 3.5 ms と 0.5 ms である．

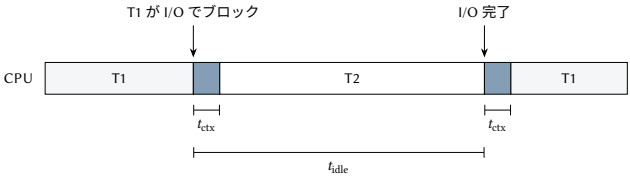



**図 3.2:** T1 は I/O でブロックする．CPU はアイドルになる代わりに T2 に切り替え，I/O の完了を契機に切り替えて戻る．切り替えのコストがアイドル時間より小さければ，切り替えは有利になる（式 (3.1)）．

### 3.2.3 マルチスレッドのアプリケーションとカーネル

アプリケーションもオペレーティングシステムも恩恵を受ける．マルチスレッドアプリケーションは，複数の CPU では並列化による高速化を達成し，1 つの CPU ではアイドル時間を有効に活用する．マルチスレッドのオペレーティングシステムカーネルは，一部のスレッドをアプリケーションのために実行し，他のスレッドをデーモンやデバイスドライバなどのオペレーティングシステムサービスを提供するために実行する．マルチプロセッサでは，これらのカーネルスレッドは異なる CPU 上で並行に実行される．

> **要点**
> スレッドはプロセスのアドレス空間を共有するので，メモリ，コンテキストスイッチ時間，通信の点でプロセスより安価である．同じ共有のためにスレッドは互いに干渉しうるので，協調のための明示的な機構が必要となる．

## 3.3 スレッドの基本機構

スレッドを実現するには 3 つのものが必要である．すなわち，スレッドを識別しその資源使用を追跡できるようにするための，スレッドを表すデータ構造，スレッドを生成・管理する機構，そして同じアドレス空間で同時に実行されるスレッドが安全に協調するための機構である．

**出典** P2L2，スレッドの基本機構，スレッドの生成；Birrell [8]；OSTEP ch. 26–27．

2 つのスレッドの実行が時間的に重なるとき，それらは並行に実行されるという．1 つの CPU 上で交互に実行される場合も，異なる CPU 上で同時に実行される場合も同様である．こうして生じる**並行性**は，プロセスにはない問題を生む．オペレーティングシステムは異なるプロセスに異なる対応付けを与えるので，あるプロセスが別のプロセスのメモリに触れることはできない．スレッドは対応付けを共有するので，どのスレッドもプロセスのどのデータでも正当に読み書きできる．2 つのスレッドが同じデータを同時に更新したり，一方が読んでいる間に他方が変更したりすると，データは不整合な状態になりうる．

▷ concurrency

**定義 3.4 (競合状態).** **競合状態**とは，計算結果が，共有データにアクセスするスレッド（少なくとも 1 つはそれを変更する）の相対的なタイミングに依存する状況である．同じデータに対する，同期されていない 2 つの並行アクセスで，少なくとも一方が書き込みであるものを**データ競合**と呼ぶ．データ競合は競合状態のよくある原因である．

▷ race condition

**注意** データ競合は C11 と C++11 では未定義動作である（C11 §5.1.2.4）．コンパイラはアクセスの順序を入れ替えたりキャッシュしたりしてよいので，言語のレベルでは「無害な」競合は存在しない．

スレッドが共有データへのアクセスと互いの進行を協調させるために用いる機構を**同期**機構と呼ぶ．この章では，どちらも Birrell が述べている次の 2 つの基本機構を全体を通じて用いる．

- **相互排除**: 一度に 1 つのスレッドだけがある操作を実行できる．これは**ミューテックス**が提供する（3.4 節）．
- **待機**: あるスレッドが，別のスレッドが特定の条件を成立させるまで待ち，そのスレッドによって起こされる．これは**条件変数**が提供する（3.5 節）．

▷ synchronization

### 3.3.1 スレッドの生成

スレッドは，その識別子，プログラムカウンタ，スタックポインタその他のレジスタ値，スタック，そしてスレッドライブラリがスレッドを管理するために使う属性を含むスレッドデータ構造で表される．Birrell の記法では，**スレッド生成**は呼び出し `t1 = Fork(proc, args)` で行う．この呼び出しは新しいスレッドを生成し，そのデータ構造を `t1` に格納し，新しいスレッドに `proc(args)` の実行を開始させる．生成したスレッドは `Fork` の次の文から実行を続けるので，この時点から両方のスレッドが並行に実行される．名前

▷ thread creation

に反して Fork は同じプロセス内にスレッドを生成するものであり，新しいプロセスを生成する第 2 章の Unix の fork とは無関係である．

子スレッドが終了したら，その結果を親に届けなければならない．子が共有アドレス空間内の決められた位置に結果を格納するか，親が**ジョイン**操作で結果を得る．Join(t1) は，スレッド t1 が完了するまで呼び出し元のスレッドをブロックし，その結果を返す．この時点で子に割り当てられていた資源は解放される．これを除けば親と子は対等である．どちらもプロセスのすべての資源にアクセスでき，異なるのは実行コンテキストだけである．

▷ join

**例 3.5.** 親スレッドと子スレッドがどちらも共有リストに挿入する．

```
Thread t1;
Shared_list list;

t1 = Fork(safe_insert, 7);   // 子が 7 を挿入
safe_insert(9);              // 親が 9 を挿入
Join(t1);                    // ここでは省略可
```

どちらの挿入が先に起こるかはスレッドのスケジューリングに依存するので，先頭に挿入する場合，リストは最終的に 7 → 9 または 9 → 7 のどちらかになる．親は子の結果を使わないので，Join は正しさのためには必要ない．

Drepper と Molnar [9] によれば，Linux でかつて 15 分かかった 100 000 個のスレッドの起動と停止は，NPTL と作り直されたカーネルのスレッド終了処理によって 2 秒になった．

挿入はアトミックではない．リストの先頭に要素 $e$ を挿入するには，$e$ を生成する，先頭ポインタを読んで $e$.next に格納する，$e$ のアドレスを先頭ポインタに書き込む，という 3 つのステップが必要である．2 つのスレッドがこれらのステップを交互に実行すると，表 3.1 のトレースが示すように挿入の 1 つが失われうる．

**表 3.1:** 初めに要素 3 を 1 つだけ含むリストへの，同期されていない 2 つの挿入．どちらのスレッドも，いずれかが先頭を書き込む前に古い先頭を読むので，ステップ 6 の書き込みがステップ 5 の書き込みを上書きし，要素 7 が失われる．

| ステップ | スレッド T1（7 を挿入） | スレッド T2（9 を挿入） | リスト |
|---|---|---|---|
| 1 | e7 を生成 | | 3 |
| 2 | e7.next = head (→ 3) | | 3 |
| 3 | | e9 を生成 | 3 |
| 4 | | e9.next = head (→ 3) | 3 |
| 5 | head = e7 | | 7 → 3 |
| 6 | | head = e9 | 9 → 3 |

## 3.4 相互排除

表 3.1 の競合は，挿入の 3 つのステップを一度に 1 つのスレッドしか実行できなければ消える．このとき 3 つのステップは，他のスレッドから見て 1 つの分割不可能な操作のように振る舞わなければならない．すなわち，ステップの間に他のスレッドがリストを観測したり変更したりすることはできない．この要件を**相互排除**と呼び，それを強制する基本機構がミューテックスである．

**出典** P2L2，ミューテックス；Birrell [8]；OSTEP ch. 28；Silberschatz ら ch. 6．

▷ mutual exclusion

**定義 3.6 (ミューテックス).** **ミューテックス**（相互排除ロック）は，ロックとアンロックの 2 つの操作を持つ同期プリミティブである．空いているミューテックスをロックしたスレッドはそれを獲得し，その所有者となる．別のスレッドが所有するミューテックスをロックしようとしたスレッドは，ミューテックスが解放されるまでブロックする．

▷ mutex

ミューテックスは，少なくともロックされているかどうか，どのスレッドが所有しているか，そのミューテックスでブロックしているスレッドのリストを記録するデータ構造で表される．ミューテックスを保持している間に実行されるコードが**クリティカルセクション**である．これは，一度に 1 つのスレッドしか実行してはならない操作，典型的には共有状態の更新を行うプログラムの部分である．

▷ critical section

Birrell の記法では，クリティカルセクションをブロックとして書く．

**ヒント** C++ の std::lock_guard はスコープの終わりでミューテックスを解放し，次に示す Birrell のブロック記法を再現する．すべての経路でアンロックされる．

```
Lock(m) {
  // クリティカルセクション
} // unlock
```

ミューテックスはブロックの終わりで暗黙に解放される．第 4 章の POSIX スレッドを含

む多くのスレッドインタフェースでは，代わりにクリティカルセクションの前後で別々の呼び出し lock(m) と unlock(m) を行い，クリティカルセクションを通るすべての経路がアンロックで終わるようにすることはプログラマの責任となる．

**注意** ミューテックスは再帰的ではない．すでに所有しているミューテックスをロックしたスレッドは，自分自身とデッドロックする．これは POSIX が PTHREAD_MUTEX_NORMAL に定める挙動であり，Linux の既定の型の挙動でもある．再帰的ミューテックスは明示的に要求しなければならない．

**例 3.7.** リストを保護するミューテックスを用いると，3.3.1 節の挿入は安全になる．

```
list<int> my_list;
Mutex m;

void safe_insert(int i) {
  Lock(m) {
    my_list.insert(i);
  } // unlock
}
```

親が先に m をロックすると，子は親の挿入が完了するまで Lock(m) でブロックし，その後自分の挿入を行う．表 3.1 の状況では，リストは最終的に 7 → 9 → 3 または 9 → 7 → 3 となり，要素が失われることはない．

複数のスレッドがブロックしているミューテックスを所有者が解放すると，そのうち 1 つだけがミューテックスを獲得し，残りはブロックしたままである．ミューテックスの仕様は，どれが獲得するかを定めていない．スレッドが要求した順にミューテックスを獲得できる保証はなく，ミューテックスが解放された瞬間に Lock を呼んだスレッドが，しばらく待っていたスレッドより先に獲得することもある．したがって，プログラムの正しさは，待っているスレッドがミューテックスを獲得する順序に依存してはならない．

競合のない Linux のロックはユーザ空間の 1 つのアトミック命令であり，カーネルに入るのは競合したときだけである（futex(2)；Drepper，*Futexes Are Tricky*）．第 10 章．

## 3.5 条件変数

**出典** P2L2，生産者/消費者の例，条件変数，条件変数の API；Birrell [8]；OSTEP ch. 30；Silberschatz ら ch. 6．

▷ producer-consumer problem

他のスレッドが成立させる条件をスレッドが待たなければならない場合，相互排除だけでは不十分である．ミューテックスを保持しているスレッドは，単に停止して条件を待つことはできない．ミューテックスを保持している間は，他のスレッドが共有状態を変更できないからである．標準的な例は**生産者・消費者問題**である．複数の生産者スレッドが共有リストに項目を挿入し，1 つ以上の消費者スレッドが，リストが満杯になったらすべての項目を表示して取り除かなければならない．

ミューテックスだけを使う場合，消費者はミューテックスをロックし，リストが満杯かどうかを調べ，満杯でなければミューテックスを解放して後で再試行しなければならない．ほとんどの検査ではリストはまだ満杯でないので，この繰り返しの検査は CPU 時間を浪費する．消費者は眠って待ち，リストを満杯にする挿入を行った生産者によって起こされる方がよい．

▷ condition variable

**定義 3.8** (条件変数). **条件変数**は，ミューテックスと組み合わせて用いる同期プリミティブである．スレッドは共有状態に関するある条件が成り立つまで条件変数上で待ち，他のスレッドは条件が成り立った可能性があるときに条件変数を通じて待っているスレッドに通知する．

条件変数 list_full を用いると，両者は次のようになる．

```
// 消費者: print_and_clear
Lock(m) {
  while (my_list.not_full())
    Wait(m, list_full);
  my_list.print_and_remove_all();
} // unlock
```

```
// 生産者: safe_insert
Lock(m) {
  my_list.insert(my_thread_id);
  if (my_list.full())
    Signal(list_full);
} // unlock
```

### 3.5.1 条件変数のインタフェース

▷ wait operation

Birrell のインタフェースには，条件変数 cond に対する 3 つの操作がある．
Wait(mutex, cond) **wait 操作**は mutex を保持した状態で呼び出す．この操作はミュー

テックスを解放し，スレッドを cond の待ちキューで停止させる．スレッドが起こされると，操作は戻る前にミューテックスを再獲得する．

`Signal(cond)` **signal 操作**は，cond で待っているスレッドがあれば，少なくとも 1 つ（通常は 1 つ）を起こす．

`Broadcast(cond)` **broadcast 操作**は，cond で待っているすべてのスレッドを起こす．

▹ signal operation

▹ broadcast operation

条件変数は，そのミューテックスへの参照と待っているスレッドのリストを保持するデータ構造で表される．wait 操作は次のように進む．

```
Wait(mutex, cond) {
  // アトミックに mutex を解放し，
  //   スレッドを cond の待ちキューに入れる
  // ... 待つ ...
  // スレッドを待ちキューから取り除く
  // mutex を再獲得する
  // 戻る
}
```

**注意** POSIX の `pthread_cond_wait` は (cond, mutex) を引数にとり，Birrell の `Wait(mutex, cond)` とは順序が逆である．第 4 章．

待っている間にミューテックスを解放することは不可欠である．さもなければ，他のどのスレッドもクリティカルセクションに入って状態を変更し，signal することができない．戻る前にミューテックスを再獲得することで，待っていたスレッドが共有状態を再検査するときに再びミューテックスを保持していることが保証される．その結果，起こされたスレッドはミューテックスを再び保持するまで先に進めない．ミューテックスがまだ保持されている場合，例えば signal したスレッドが保持している場合には，起こされたスレッドは条件変数の待ちキューからミューテックスの待ちキューへ移る（図 3.3）．broadcast の後は起こされたすべてのスレッドがミューテックスを奪い合うので，それらは 1 つずつ wait 操作から戻る．

条件変数の signal 操作は，第 1 章の Unix シグナルとは無関係である．

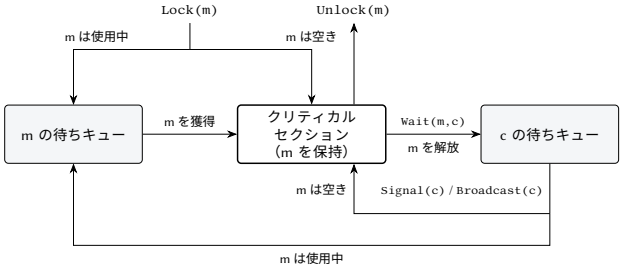



**図 3.3:** 条件 c で待つスレッドはミューテックス m を解放する．signal されると，スレッドはまず m を再獲得しなければならない．m が保持されていれば，ミューテックスの待ちキューに移る．

消費者が `if` ではなく `while` ループでリストを検査するのはこのためである．`Signal` から消費者がミューテックスを再獲得するまでの間に，他のスレッドが先にミューテックスを獲得してリストを変更するかもしれない—例えば，リストを空にする 2 番目の消費者である．条件変数への signal は，条件が成り立っているかもしれないことを伝えるだけである．起こされたスレッドがすぐにミューテックスを獲得できる保証はなく，獲得したら条件を再検査しなければならない．[1]

[1] ループで再検査するのは Mesa セマンティクス（Lampson と Redell [10]）を反映している．signal したスレッドがロックを保持し続ける．Hoare のモニタは起こされたスレッドにロックを引き渡すので，そこでは `if` で十分である．モニタは第 10 章で扱う．

> **要点**
> `Wait` は必ずミューテックスを保持した状態で，条件を再検査する `while` ループの中で呼び出す．signal または broadcast は，条件が依存する状態を変更した後に行う．

## 3.6 リーダ・ライタ問題

**リーダ・ライタ問題**では，一部のスレッド（リーダ）はファイルなどの共有資源を読むだけで，他のスレッド（ライタ）はそれを変更する．任意の数のリーダが同時に資源にアクセスしてよいが，ライタは排他的なアクセスを必要とする．すなわち，ライタは高々 1 つで，書き込み中はリーダもいてはならない．

▹ readers-writers problem

**出典** P2L2，リーダ/ライタ問題とその例，（代理変数を用いた）クリティカルセクションの構造；OSTEP ch. 31；Silberschatz ら ch. 7.

資源を 1 つのミューテックスで保護するのは正しいが，制限が強すぎる．リーダ同士も互いに排除してしまうからである．アクセス規則は，活動中のリーダとライタの数に

依存する．
- 活動中のリーダもライタもいない: リーダもライタも入ってよい．
- 活動中のリーダが 1 つ以上いる: 別のリーダは入ってよいが，ライタは入れない．
- 活動中のライタがいる: リーダもライタも入れない．

これらの規則は 1 つのカウンタで表現できる．資源は，resource_counter = 0 のとき**空き**，resource_counter = $n > 0$ のとき $n$ 個のリーダによって**読み出し中**，resource_counter = $-1$ のとき**書き込み中**である（図 3.4）．

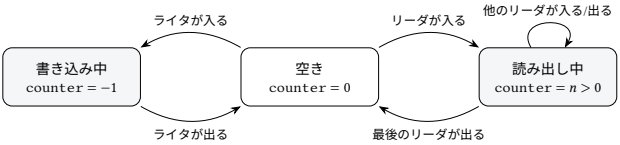



**図 3.4:** 共有資源の状態と，それに対応する代理変数の値．counter は resource_counter の略である．

▷ proxy variable

このカウンタは**代理変数**の一例である．代理変数とは，ミューテックスで保護され，資源の状態を表す共有変数である．スレッドがミューテックスを保持するのは代理変数を検査・更新する間だけであり，実際の読み書きは，代理変数が強制する方針のもとでミューテックスの外で行われる．解は 1 つのミューテックスと 2 つの条件変数を用いる．

```
Mutex counter_mutex;
Condition read_phase, write_phase;
int resource_counter = 0;

// リーダ
Lock(counter_mutex) {
  while (resource_counter == -1)
    Wait(counter_mutex, read_phase);
  resource_counter++;
} // unlock
// ... データを読む ...
Lock(counter_mutex) {
  resource_counter--;
  if (resource_counter == 0)
    Signal(write_phase);
} // unlock
```

```
// ライタ
Lock(counter_mutex) {
  while (resource_counter != 0)
    Wait(counter_mutex, write_phase);
  resource_counter = -1;
} // unlock
// ... データを書く ...
Lock(counter_mutex) {
  resource_counter = 0;
  Broadcast(read_phase);
  Signal(write_phase);
} // unlock
```

**ヒント** 先に進めるスレッドが高々 1 つで，どれでもよいときは signal する．変化によって複数のスレッドが入れるとき，あるいは同じ条件変数で異なる述語を待つスレッドがいるときは broadcast する．

退出処理でどの通知を用いるかは，誰が待っている可能性があるかから決まる．
- 他のリーダが残っている間に出ていく**リーダ**は，待っているスレッドに関係する状態を何も変えない．最後のリーダが出ていくとき，待っている可能性があるのはライタだけであり（リーダが待つのはライタが活動中のときだけである），入れるのはそのうち 1 つだけなので，write_phase への Signal で十分である．
- 出ていく**ライタ**は，どちらの種類のスレッドに対しても資源を空ける．待っているリーダはすべて一緒に入ってよいので Broadcast(read_phase) を，ライタは 1 つしか入れないので Signal(write_phase) を行う．実際にどのスレッドが先に入るかは，どれが先に counter_mutex を獲得するかに依存する．リーダが先に入れば，起こされた他のリーダも入り，ライタは while ループでカウンタ $\neq 0$ を見て再び待つ．ライタが先に入れば，リーダはカウンタ $= -1$ を見て再び待つ．

**注意** この方針はリーダを優先する．新しいリーダが到着し続ける限りカウンタは 0 に戻らず，待っているライタは決して入れない．glibc の pthread_rwlock も既定はリーダ優先であり，ライタ優先の種類も提供している（第 10 章）．

**例 3.9.** 表 3.2 は，2 つのリーダ，それらの読み出し中に到着するライタ，ライタの書き込

み中に到着する 3 番目のリーダをトレースしたものである．

**表 3.2:** リーダ・ライタ問題の解のトレース．各ステップの後の代理変数の値を示す．

| ステップ | イベント | カウンタ |
|---|---|---|
| 1 | R1 が入る | 1 |
| 2 | R2 が入る | 2 |
| 3 | W1 はカウンタ ≠ 0 を見て write_phase で待つ | 2 |
| 4 | R1 が出る．カウンタ ≠ 0 なので通知しない | 1 |
| 5 | R2 が出る．カウンタ = 0 なので write_phase に signal | 0 |
| 6 | W1 がミューテックスを再獲得し，再検査して入る | −1 |
| 7 | R3 はカウンタ = −1 を見て read_phase で待つ | −1 |
| 8 | W1 が出る．read_phase に broadcast，write_phase に signal | 0 |
| 9 | R3 がミューテックスを再獲得し，再検査して入る | 1 |

### 3.6.1 クリティカルセクションの構造

リーダ・ライタ問題の解は，ミューテックスと条件変数を組み合わせて使うコードの一般的な形を示している．待つ必要がありうるクリティカルセクションは次のように書く．

```
Lock(mutex) {
  while (!predicate_indicating_access_ok)
    Wait(mutex, cond_var);
  update state => update predicate;
  Signal and/or Broadcast(
      cond_var_with_correct_waiting_threads);
} // unlock
```

代理変数を用いる場合は，資源に対する操作をミューテックスの外に出し，全体を 3 つの部分に分ける．

```
// クリティカルセクションに入る
Lock(mutex) {
  while (!predicate_for_access)
    Wait(mutex, cond_var);
  update predicate;
} // unlock

perform critical operation  // 例：共有ファイルの読み書き

// クリティカルセクションから出る
Lock(mutex) {
  update predicate;
  Signal/Broadcast(cond_var);
} // unlock
```

**要点**
ミューテックスだけで強制できる方針は「一度に 1 つのスレッド」の 1 つしかない．短いクリティカルセクションで更新される代理変数を用いると，「任意の数のリーダ，または 1 つのライタ」のようなより複雑なアクセス方針を，同じミューテックスと条件変数のプリミティブから構築できる．

## 3.7 よくある誤りとスプリアスウェイクアップ

ミューテックスと条件変数を使うプログラムでは，いくつかの誤りが繰り返し現れる．

- どのミューテックスと条件変数がどの資源に属するかを見失う．変数を宣言する箇所に，例えば `Mutex m1; // file1 を保護` のように対応関係を記録すれば，これを避けられる．
- ロックやアンロックを忘れる，あるいは誤ったミューテックスをロック・アンロックする．コンパイラや解析ツールの中には，このような場合に警告を出せるものがある．

**出典** P2L2，よくある誤り，スプリアスウェイクアップ；Birrell [8]．

Clang の `-Wthread-safety` はコンパイル時にロックの注釈を検査し，ThreadSanitizer（`-fsanitize=thread`）は実行時に競合を見つける．典型的な代価は実行時間 5–15 倍，メモリ 5–10 倍である．

- 同じ資源に複数のミューテックスを使う．ファイルのリーダが m1 を，ライタが m2 をロックすると，読み出しと書き込みが並行に実行されてしまう．
- 誤った条件変数に signal する．
- Broadcast が必要なところで Signal を使う．起こされるのは 1 つのスレッドだけで，他のスレッドはいつまでも待ち続けるかもしれない．
- 通知によって実行順序が決まると仮定する．signal を発行した順序は，起こされたスレッドが実行される順序を決めない．優先度を守らなければならない場合は，別の機構が必要である．

さらに 2 つの問題を詳しく見る．性能を損なうスプリアスウェイクアップと，プログラムを停止させるデッドロック（3.8 節）である．

### 3.7.1 スプリアスウェイクアップ

通知の中には，それを利用できないスレッドを起こしてしまうものがある．

▷ spurious wake-up

**定義 3.10** (スプリアスウェイクアップ). **スプリアスウェイクアップ**とは，実行されても先に進めないスレッドを起こすことであり，それによって生じるコンテキストスイッチは無駄になる．

**注意** 講義はこれをスプリアスウェイクアップと呼ぶ．Birrell はこれを *spurious lock conflict*（見かけのロック競合）と呼び，*spurious wake-up* を述語が偽であるスレッドを起こすことに用いる．

POSIX では，signal がまったくないのに wait から戻ることもスプリアスウェイクアップと呼ぶ．while ループはどちらにも対処する．

よくある原因は，ミューテックスを保持したまま signal することである．3.6 節のライタの退出処理では，broadcast はクリティカルセクションの中で発行される．起こされたリーダは read_phase の待ちキューから出るが，ライタがまだ保持している counter_mutex を再獲得するまで Wait から戻れない．それらは単にミューテックスの待ちキューに移されるだけである（図 3.3）．

通知が共有状態に依存しない場合，対策は通知をアンロックの後に移すことである．

```
// ライタの退出処理（通知はミューテックスの外）
Lock(counter_mutex) {
  resource_counter = 0;
} // unlock
Broadcast(read_phase);
Signal(write_phase);
```

これは常に可能なわけではない．リーダの退出処理では，signal するかどうかの判断が resource_counter に依存し，この変数はミューテックスを保持している間しか読めないので，Signal を単純にアンロックの後に移すことはできない．移せるのは，クリティカルセクションの中で判断を局所変数に記録し，その後で signal する場合だけであり，これは局所変数 doSignal を用いる Birrell 自身の対策である．スプリアスウェイクアップが正しさに影響することはなく（先に進めないスレッドは while ループによって待ちに戻される），影響するのは性能だけである．

Linux には同じ目的の FUTEX_CMP_REQUEUE がある．条件変数で待つスレッドを起こす代わりにミューテックスの待ちキューへ移すもので，futex(2) はこれを「thundering herd」の回避と説明している．

## 3.8 デッドロック

一部のミューテックスを保持したまま別のミューテックスを要求するスレッドは，互いを永久にブロックしうる．

**出典** P2L2，デッドロック；OSTEP ch. 32；Silberschatz ら ch. 8.

▷ deadlock

**定義 3.11** (デッドロック). **デッドロック**とは，競合する 2 つ以上のスレッドがそれぞれ他のスレッドの完了を待っているが，どれも決して完了しない状況である．[2]

[2] デッドロックには 4 つの条件が同時に必要である（Coffman ら，1971 年）．相互排除，保持と待機，横取り不可，循環待ちである．ロック順序は最後の条件を取り除く（OSTEP ch. 32；Silberschatz ら ch. 8）．

最も単純な場合は，2 つのスレッドと 2 つのミューテックスが関わるものである．どちらのスレッドも foo(A, B) を実行するために資源 A と B を必要とするが，それらを保護するミューテックスを逆の順序でロックする．

```
// スレッド T1                  // スレッド T2
Lock(m_A) {                    Lock(m_B) {
  Lock(m_B) {                    Lock(m_A) {
    foo(A, B);                     foo(A, B);
  }                              }
}                              }
```

T1 が m_A を，T2 が m_B を，どちらも 2 番目の Lock に到達する前に獲得すると，T1 は m_B を待ってブロックし，T2 は m_A を待ってブロックする．どちらも先に進めず，どちらも相手が必要とするミューテックスを決して解放しない．

▷ wait graph

この状況は**待ちグラフ**で記述される．待ちグラフは，資源を待っているスレッドからその資源を所有するスレッドへの辺を持つ．図 3.5 は，ミューテックスを明示的に描いて

この例を示したものである．デッドロックは待ちグラフの閉路に対応する．所有者が 1 つだけであるロックについては，閉路はデッドロックの必要十分条件である．

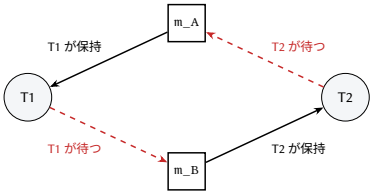



**図 3.5:** このデッドロックの待ちグラフ．T1 から辺をたどると，m_B，T2，m_A を経て T1 に戻る．

### 3.8.1 例のデッドロックの回避

例からデッドロックを取り除く方法は 3 つある．

1. **B をロックする前に A をアンロックする．**一度に 1 つのミューテックスだけを保持すれば（細粒度のロック）閉路は生じえないが，ここでは使えない．スレッドは両方の資源を同時に必要とするからである．
2. **すべてのロックを最初に獲得する．**開始前に必要なすべてのロックを一度に（アトミックに，すなわちすべてを獲得するか 1 つも獲得しないかのどちらかで，1 つでも得られなければ保持しているものを解放して）獲得し，最後にすべて解放するスレッドはデッドロックしえない．その極端な場合が，すべての資源に対する 1 つの「メガロック」である．並列性を制限するので，多くのアプリケーションにとってこれは制限が強すぎる．
3. **ロックの順序を守る．**すべてのスレッドがミューテックスを同じ順序，すなわち先に m_A，次に m_B の順で獲得すれば，デッドロックは起こりえない．

3 番目の方法である**ロック順序**は，一般に通用する．ミューテックスに番号を付け，すべてのスレッドが番号の昇順で獲得すれば，待ちグラフの閉路を一周する間ずっと番号が増加しなければならず，これは不可能である．難しさは実践面にある．多数のミューテックスを持つ大規模なプログラムでは，すべてのコード経路が順序を守ることを保証するのは難しく，ロックの獲得順序を検査するツールが役立つ．

▷ lock ordering

Linux の *lockdep*（カーネル文書 locking/lockdep-design）はロッククラスの獲得順序を記録し，デッドロックが起きる前に順序の逆転を警告する．ThreadSanitizer もロック順序の逆転を報告する．

### 3.8.2 デッドロックへの一般的な対処

オペレーティングシステムとアプリケーションがデッドロックに対処する方法は 3 つある．

**防止　デッドロック防止**では，閉路の形成を決して許さない．コードを変更して閉路を不可能にする（スレッドが別のロックを要求する前に保持しているロックを解放する，あるいはすべてのスレッドがロック順序に従う）か，すべてのロック要求を実行時に検査し，それを許可すると後に閉路につながりうる場合には要求を遅らせる．実行時の検査はコストが高く，各スレッドがどのロックを要求しうるかを事前に知る必要がある．教科書ではこの変種を**デッドロック回避**と呼び，**防止**という語を構造的な規則に限って用いる．

▷ deadlock prevention

**検出と回復　デッドロックの検出と回復**では，システムが定期的に待ちグラフを解析して閉路を探し，見つかった場合には関係するスレッドの一部の実行をロールバックする．ロールバックには以前の時点を復元するのに十分な保存状態が必要であり，スレッドがすでに外部からの入力を消費したり外部への出力を生成したりしている場合には不可能である．

▷ deadlock detection and recovery

**何もしない　ダチョウアルゴリズム**は，デッドロックは稀であり他の方法はコストが高いという理由で，デッドロックを完全に無視する．実際にデッドロックが起きたら，システムを再起動する．

▷ ostrich algorithm

> **要点**
> デッドロックは待ちグラフの閉路である．すべてのミューテックスを 1 つの大域的な順序で獲得すれば，実行時コストなしにそのような閉路を防止できる．すべてのロック要求の実行時検査やロールバックによる回復は，汎用的だがコストが高い．

## 3.9 カーネルレベルスレッドとユーザレベルスレッド

**出典** P2L2，カーネルレベルスレッドとユーザレベルスレッド，マルチスレッドモデル，マルチスレッドのスコープ；Silberschatz ら ch. 4–5．

スレッドはシステムの 2 つのレベルに存在する．オペレーティングシステムのカーネル

自体がマルチスレッドでありうるし，アプリケーションもプロセス内にスレッドを生成できる．問題は，この 2 種類のスレッドが互いにどう関係するかである．

▷ kernel-level thread

▷ user-level thread

**定義 3.12** (カーネルレベルスレッドとユーザレベルスレッド). **カーネルレベルスレッド**とは，カーネルから見え，CPU スケジューラなどのカーネルの構成要素によって管理されるスレッドである．**ユーザレベルスレッド**とは，ユーザレベルのスレッドライブラリを通じて生成される，アプリケーションから見たスレッドである．マルチスレッドモデルに応じて，ライブラリはそれを自ら管理するか，操作をカーネルに渡す．

**ヒント** `ps -eLf` はすべてのプロセスのカーネルレベルスレッドを一覧表示し（LWP 列），`top -H` はそれらを個別に表示する．

カーネルレベルスレッドの中には，アプリケーションのユーザレベルスレッドを支えるために存在するものもあれば，デーモンなどのオペレーティングシステムサービスを実行するものもある．CPU スケジューラは，どのカーネルレベルスレッドをどの CPU で実行するかを決める．一方ユーザレベルスレッドは，カーネルレベルスレッドに対応付けられ，スケジューラがそのカーネルレベルスレッドを CPU 上で実行するときにのみ実行できる．

### 3.9.1 マルチスレッドモデル

ユーザレベルスレッドをカーネルレベルスレッドにどう対応付けるかは，3 つのマルチスレッドモデルで記述される（図 3.6）．[3]

[3] 多対多モデルは野心的なモデルであったが，ほぼ放棄された．Solaris 2 はこれを実装したが，2 つのレベルの間の協調が，それによって得られる利益より高くつくことが分かり，Sun は Solaris 9（2002 年）で一対一のライブラリを既定とした．Linux も多対多の NGPT ではなく一対一の NPTL に落ち着いた（第 5 章）．このモデルは，スケジューラがランタイムの一部である言語処理系に生き残っている．Go のプログラムは最小 2 KiB のスタックで始まるゴルーチンを，少数のカーネルレベルスレッド上に多重化する．

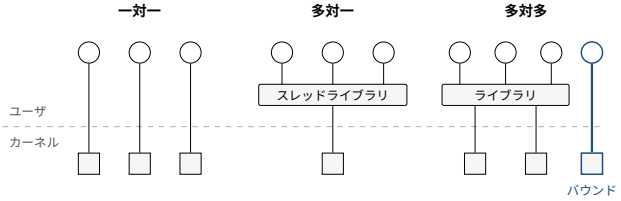



**図 3.6:** マルチスレッドモデル．円はユーザレベルスレッド，四角はカーネルレベルスレッドである．多対多モデルでは，バウンドスレッドは専用のカーネルレベルスレッドを持ち続ける．

▷ one-to-one model

**一対一** **一対一モデル**では，各ユーザレベルスレッドが専用のカーネルレベルスレッドを持つ．カーネルはスレッド，その同期，ブロッキングを把握・理解し，それらを複数の CPU にスケジュールできる．一方で，スレッドの生成と，スレッドをブロックさせたり起こしたりするすべての操作にはシステムコールが必要であり，コストが高い（Linux では，競合のないロック操作は futex によってユーザ空間にとどまる）．アプリケーションはカーネルがサポートする方針とスレッド数の上限に制約され，このサポートを提供するカーネルにしか移植できない．

▷ many-to-one model

**多対一** **多対一モデル**では，プロセスのすべてのユーザレベルスレッドが 1 つのカーネルレベルスレッドに対応付けられ，ユーザレベルのスレッドライブラリが，その上でどのユーザレベルスレッドを実行するかを随時決める．スレッド管理にシステムコールは不要で，カーネルの制限や方針にも依存しないので，完全に移植可能である．しかしカーネルはアプリケーションが何を必要としているかを知る手段を持たず，1 つのユーザレベルスレッドが I/O でブロックするとカーネルレベルスレッドがブロックし，それとともにプロセス全体がブロックする．

▷ many-to-many model

▷ bound thread

**多対多** **多対多モデル**では，一部のユーザレベルスレッドはカーネルレベルスレッドの集合上に多重化され，他のユーザレベルスレッドはカーネルレベルスレッドを専有する．カーネルレベルスレッドに恒久的に対応付けられたユーザレベルスレッドを**バウンドスレッド**と呼ぶ．バウンドスレッドには，例えば高い優先度やよりよい応答性を与えられる．プロセスは複数のカーネルレベルスレッドを持つので，I/O でブロックしたユーザレベルスレッドは自分のカーネルレベルスレッドだけをブロックし，他のユーザレベルスレッドは残りのカーネルレベルスレッド上で実行を続ける．このモデルは他の 2 つの利点を兼ね備えるが，その代償としてユーザレベルとカーネルレベルのスレッド管理機構の間の協調が必要になる．

### 3.9.2 マルチスレッドのスコープ

スレッド管理は 2 つのレベルで行われ，そのレベルによって資源を共有する単位が決まる．**プロセススコープ**では，ユーザレベルのスレッドライブラリが 1 つのプロセス内のスレッドを管理し，スレッドはそのプロセスの他のスレッドとだけ資源を奪い合う．カー

▷ process scope

ネルはスレッドを個別には見ない．**システムスコープ**では，スレッドはオペレーティングシステムのスレッド管理機構，特に CPU スケジューラによってシステム全体で管理され，システム内のすべてのスレッドと資源を奪い合う．

▷ system scope

Linux はシステムスコープしかサポートしない．PTHREAD_SCOPE_PROCESS を指定した pthread_attr_setscope は ENOTSUP で失敗する（pthread_attr_setscope(3)）．第 4 章．

**例 3.13.** プロセス P は 6 つのユーザレベルスレッドを，プロセス Q は 2 つを実行する．プロセススコープではカーネルには 2 つのプロセスしか見えず，それぞれに CPU を均等に割り当てると，P の各スレッドは CPU の $\frac{1}{2} \cdot \frac{1}{6} = \frac{1}{12}$ を，Q の各スレッドは $\frac{1}{4}$ を受け取る．システムスコープではカーネルに 8 つのスレッドすべてが見え，CPU をそれらの間で均等に分けると各スレッドは $\frac{1}{8}$ を受け取り，P 全体では $\frac{3}{4}$ を受け取る．

## 3.10 マルチスレッドのパターン

マルチスレッドアプリケーションは，一般にボス・ワーカー，パイプライン，階層化の 3 つのパターンのいずれかに従って構成される．

**出典** P2L2，マルチスレッドのパターン；スレッドプールについては Silberschatz ら ch. 4．

### 3.10.1 ボス・ワーカー

**ボス・ワーカーパターン**では，1 つのボススレッドが要求を受け付けてワーカースレッドに割り当て，各ワーカーはタスク全体を実行する．すべての要求がボスを通るので，システムのスループットはボスによって制限される．ボスが要求ごとに時間 $t_{\text{boss}}$ を費やすなら，ワーカーがいくつあってもシステムは単位時間当たり高々 $1/t_{\text{boss}}$ 個の要求しか受け付けられない．したがって，ボスが要求ごとに行う仕事はできるだけ少なくしなければならない．

▷ boss-workers pattern

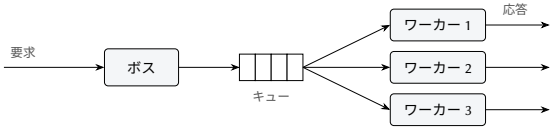



**図 3.7:** 作業キューを用いるボス・ワーカー．ボスは要求をキューに入れるだけであり，各ワーカーはキューから要求を取り出してタスク全体を実行する．

ボスがワーカーに仕事を渡す方法は 2 つある（図 3.7 は 2 番目を示す）．

- **特定のワーカーに直接通知する．**ワーカー間の同期は不要だが，ボスは各ワーカーが何をしているか，どれが空いているかを把握しなければならない．これは $t_{\text{boss}}$ を増やし，スループットを下げる．
- **仕事をキューに置く．**ワーカーはそこから仕事を取り出す．これは生産者・消費者の構成である．ボスはワーカーについて何も知る必要がないが，ワーカー（とボス）は共有キューへのアクセスを同期しなければならない．ボスの仕事を少なく保てるので，通常はこちらの方がスループットがよい．

ワーカーは要求が到着したときに必要に応じて生成することもできるが，要求ごとにスレッドを生成するとレイテンシが増える．より一般的には，アプリケーションは事前に生成したワーカーの集合である**スレッドプール**を用いる．プールは静的でも動的でもよい．動的なプールは，キューが長くなると（典型的には 1 つずつではなく一度に複数のスレッドずつ）拡大し，ワーカーがアイドルになると縮小する．

▷ thread pool

**ヒント** 時間の割合 $b$ をブロックして過ごすワーカーで $c$ 個の CPU を使い切るには，およそ $c/(1-b)$ 個のワーカーが必要である．それ以上増やしても何も得られない．

このパターンは単純だが，スレッドプールの管理が必要であり，局所性を無視する．ボスは，どのワーカーが似たタスクを直前に実行し，次のタスクをホットキャッシュで実行できるかを知らない．そこで変種として，すべてのワーカーを同等にする代わりに，特定の種類の要求にワーカーを特化させるものがある．特化したワーカーは局所性がよく，アプリケーションはより重要な種類の要求により多くのスレッドを割り当てることで異なるサービス品質を提供できる．代償は負荷分散であり，各種類にいくつのスレッドを割り当てるべきかの判断が難しくなる．

### 3.10.2 パイプライン

**パイプラインパターン**では，タスクを一連のサブタスクに分割し，各サブタスクをそれ専用のスレッド（またはスレッドのグループ）が実行する．ステージは共有バッファを通じて通信するので，複数の要求がそれぞれ異なるステージで並行に処理される（図 3.8）．スループットは最も弱い環，すなわち最も遅いステージで決まる．対策は，各ステージをスレッドプールにし，遅いステージにより多くのスレッドを与えることである．ステー

▷ pipeline pattern

シェルのパイプライン a | b | c は，このパターンをプロセスで実現したものである．ステージは，Linux では既定で 64 KiB のカーネルバッファでつながれる（pipe(7)）．

ジ $i$ が要求当たり時間 $t_i$ を要し，$p_i$ 個のスレッドを持つとき，スループットは

$$X = \min_i \frac{p_i}{t_i} \tag{3.2}$$

である．

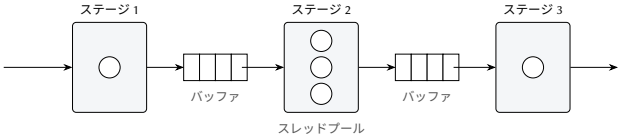



**図 3.8:** 3 ステージのパイプライン．ステージはバッファを通じて要求を受け渡す．遅い第 2 ステージは 3 スレッドのプールである．

パイプラインでは各スレッドが同じサブタスクを繰り返し実行するので，特化と局所性が得られる．コストは，ステージのバランスを保つ手間とバッファの同期オーバーヘッドである．

**例 3.14.** 3 ステージのパイプラインのステージ時間が 10 ms，40 ms，20 ms であるとする．各ステージ 1 スレッドの場合，式 (3.2) から $\min(100, 25, 50) = 25$ 要求/秒となり，ステージ 2 が制約となる．ステージ 2 を 4 スレッドにするとスループットは $\min(100, 100, 50) = 50$ 要求/秒になり，今度はステージ 3 がボトルネックとなる．

### 3.10.3 階層化

▷ layered pattern

3 層のウェブアプリケーション（プレゼンテーション，ロジック，データ）が典型例であり，各層が独自のスレッドプールを持つ．

**階層化パターン**では，関連するサブタスクを層にまとめて各層を専用のスレッドが担当し，すべての要求がすべての層を下って再び上っていく（図 3.9）．パイプラインと同様に特化をもたらすが，粒度はより粗い．すべてのアプリケーションに適しているわけではなく，各層とその上下の層との間の同期が必要である．




**図 3.9:** 階層化パターン．各層は関連するサブタスクをまとめ，専用のスレッドを持つ．要求は層を下って再び上っていく．

### 3.10.4 ボス・ワーカーとパイプラインの比較

第 6 章はマルチプロセス，マルチスレッド，イベント駆動のウェブサーバを比較する．Apache の worker MPM はプロセスごとにプールを持つボス・ワーカーであり（`ThreadsPerChild` は既定で 25），Flash（AMPED）は 1 つのイベントループとディスク I/O のためのヘルパで動作する．

一群の要求をどちらのパターンが速く処理し終えるかは，要求の数に依存する．それぞれが同じ総処理時間を要する $n$ 個の要求を考える．$w$ 個のワーカーを持ちボスの時間 $t_{\text{boss}}$ が無視できるボス・ワーカー設計は，要求を $w$ 個ずつ完了し，各グループにタスク全体の時間 $t$ がかかる．時間 $s$ の等しい $k$ 個のステージからなるパイプラインは，$k$ ステージ時間の後に最初の要求を完了し，その後は 1 ステージ時間ごとに 1 要求を完了する．したがって

$$T_{\text{bw}}(n) = \left\lceil \frac{n}{w} \right\rceil t, \qquad T_{\text{pl}}(n) = (k + n - 1)\, s \tag{3.3}$$

である．

**注意** これらの式が比べているのは一群の要求全体の完了時間である．パイプラインは 1 つの要求を速くはしない．要求は依然としてすべての $k$ 個のステージを通り，次のステージが塞がっていればバッファで待つ．

**例 3.15.** どちらの設計も 4 つのスレッドを使い，1 要求に 100 ms の作業が必要であるとする．ボス・ワーカー設計は 1 つのボスと $w = 3$ 個のワーカーを持ち $t = 100$ ms，パイプラインは $s = 25$ ms のステージを $k = 4$ 個持つ．100 ms 当たりに完了する要求はボス・ワーカーが 3，定常状態のパイプラインが 4 なので，ボス・ワーカーが先行できるのは，パイプラインの立ち上がり時間 $k\,s$ がまだ効いている少数の要求においてだけである．そこでも結果は $n$ に依存する．式 (3.3) の $T_{\text{bw}}$ は階段状に増えるからである．$n = 2$，3，6 ではボス・ワーカーが速く，$n = 4$，7，8 ではパイプラインが速く，$n = 1$，5，9 では両者が等しく，$n \geq 10$ では常にパイプラインが勝つ．

| $n$ | ボス・ワーカー（ms） | パイプライン（ms） |
|---|---|---|
| 3 | 100 | 150 |
| 4 | 200 | 175 |
| 6 | 200 | 225 |
| 12 | 400 | 375 |

**要点 — 章のまとめ**
スレッドはプロセスのアドレス空間を共有する実行コンテキストであり，並列性，特化，アイドル時間の利用（式 (3.1)）をプロセスより安価に提供する．共有データはミューテックスで保護しなければならず，スレッドは条件変数で条件を待ち，その条件は常に `while` ループで再検査する．代理変数を用いるとリーダ・ライタのようなより複雑な方針を構築でき，ミューテックスの外で signal するとスプリアスウェイクアップを避けられ，一貫したロック順序はデッドロックを防ぐ．ユーザレベルスレッドは一対一，多対一，多対多のいずれかのモデルに従ってカーネルレベルスレッド上で実行され，アプリケーションはボス・ワーカー，パイプライン，階層として構成される．

第 4 章

# スレッドのケーススタディ: PThreads

| | |
|---|---|
| **講義動画** | P2L3 |
| **教科書** | OSTEP [1] ch. 27，30；Silberschatz ら [2] ch. 4，7；Kerrisk [11] ch. 29–30 |
| **キーワード** | POSIX，POSIX スレッド，スレッド生成，スレッド属性，ジョイン可能スレッド，デタッチスレッド，スレッドへの引数，ミューテックス，trylock，条件変数，安全な使い方，生産者・消費者問題，有界バッファ |

第 3 章では，スレッドと，スレッドが協調するための機構を Birrell [8] の抽象的な記法–`Fork` と `Join`，ミューテックス，条件変数–で述べた．この章では，これらの機構を提供し，ほとんどの Unix 系オペレーティングシステムがサポートしている具体的なインタフェースである POSIX スレッド，すなわち Pthreads を扱う．機構ごとに対応するデータ型と呼び出しを示し，実際のインタフェースが Birrell のモデルに付け加える細部–スレッド属性，デタッチスレッド，同期構成要素の初期化–と，それらを安全に使うための規則を述べる．最後に生産者・消費者プログラムの全体を示す．

## 4.1 POSIX スレッド

▷ Portable Operating System Interface

**出典** P2L3，レッスンの概要；POSIX.1 [12]；OSTEP ch. 27．

**POSIX** は，オペレーティングシステムがアプリケーションに提供するインタフェースを規定する IEEE 標準の一群である．システムコールとライブラリ関数の名前，引数，意味を定めるが，オペレーティングシステムがそれらをどう実装するかは定めない．その目的は移植性である．POSIX に従って書かれたプログラムは，最初に書かれたオペレーティングシステムだけでなく，この標準に準拠するすべてのオペレーティングシステムでコンパイルして実行できる．

POSIX.1 は IEEE と The Open Group が共同で保守しており，Single UNIX Specification の中核をなす．本章は 2017 年版 [12] を引用する．

**定義 4.1** (POSIX スレッド). POSIX のうち，スレッドを生成・管理するためのインタフェースと，スレッドが協調するための同期構成要素–ミューテックスと条件変数–を規定する部分を **POSIX スレッド**，または *Pthreads* と呼ぶ．

▷ POSIX threads

Pthreads は POSIX.1 基本標準の一部である．POSIX の他の部分と同様に，Pthreads も実装は規定しない．例えば Pthreads ライブラリは，第 3 章のマルチスレッドモデルのいずれに従ってスレッドをカーネルレベルスレッドに対応付けてもよい．その関数と型の名前は `pthread_` で，定数の名前は `PTHREAD_` で始まる．データ型は `_t` で終わり，不透明であるため，プログラムはインタフェースの関数を通じてのみそれらを操作する．

Linux の Pthreads ライブラリは NPTL であり，各スレッドに専用のカーネルタスク（`clone(2)`）を与える．一対一モデルである（第 5 章）．

このインタフェースは第 3 章の機構に忠実に対応している．表 4.1 は Birrell の各操作に対応する Pthreads の要素を挙げたものである．以下の節ではそれらを順に扱う．

**表 4.1:** Birrell の機構（第 3 章）と対応する Pthreads の要素．`t`，`m`，`c` はそれぞれスレッド，ミューテックス，条件変数を表す．

| 操作 | Birrell | Pthreads |
|---|---|---|
| スレッド型 | `Thread` | `pthread_t` |
| 生成 | `Fork(proc, args)` | `pthread_create(&t, &attr, proc, arg)` |
| ジョイン | `Join(t)` | `pthread_join(t, &status)` |
| ミューテックス型 | `Mutex` | `pthread_mutex_t` |
| ロック | `Lock(m) {` | `pthread_mutex_lock(&m)` |
| アンロック | `}` | `pthread_mutex_unlock(&m)` |
| 条件変数型 | `Condition` | `pthread_cond_t` |
| wait | `Wait(m, c)` | `pthread_cond_wait(&c, &m)` |
| signal | `Signal(c)` | `pthread_cond_signal(&c)` |
| broadcast | `Broadcast(c)` | `pthread_cond_broadcast(&c)` |

## 4.2 スレッドの生成

**出典** P2L3，PThread の生成；OSTEP ch. 27；Kerrisk ch. 29．

Pthreads では，スレッドは Birrell の `Thread` に対応する `pthread_t` 型の変数で表される．この変数はスレッドを識別し，ライブラリはこれを通じて，識別子，実行状態，その

スレッドを管理するために必要なその他の情報を持つスレッドのデータ構造に到達する．スレッドの生成とジョインには 2 つの関数を用いる．

```
#include <pthread.h>

int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
int pthread_join(pthread_t thread, void **status);
```

pthread_create は Birrell の Fork(proc, args) に対応する．生成された新しいスレッドは start_routine(arg) の実行から始まり，そのスレッドの識別子は *thread に格納される．スタートルーチンは void * を受け取って void * を返すので，任意の型のデータへのポインタをスレッドに渡せる．複数の引数は構造体にまとめ，その構造体へのポインタとして渡す．第 2 引数の attr に対応するものは Birrell の Fork にはなく，これについては後述する．この呼び出しは，スレッドが生成されれば 0 を，そうでなければ–例えばシステムにもう 1 つスレッドを作るだけの資源がない場合には–エラー番号を返す．

pthread_join は Join(t) に対応し，thread が終了するまで呼び出し元のスレッドをブロックする．スレッドが終了するのは，そのスタートルーチンが return したとき，あるいは pthread_exit(status) を呼んだときである．スタートルーチンが返した値，あるいは pthread_exit に渡した値は，*status に格納される．結果を必要としない呼び出し元はここに NULL を渡す．pthread_create と同様に，pthread_join も成功したときは 0 を，失敗したときはエラー番号を返す．

生成とジョインの往復は，現在の x86-64 Linux では 10 µs 程度である．スレッドプール（第 3 章）はこれをワーカーごとに一度だけ払う．

**注意** 不透明なので，2 つの識別子は == ではなく pthread_equal(t1, t2) で比較する．POSIX は pthread_t が構造体であることを許している．

**注意** 同じスレッドを 2 回ジョインすること，およびデタッチされたスレッドをジョインすることは，POSIX では単なるエラーの戻り値ではなく未定義動作である（POSIX.1）．

### 4.2.1 スレッド属性

**定義 4.2 (スレッド属性).** スタックサイズ，ジョインできるかどうか，スケジューリング方針と優先度，スコープなど，スレッドの生成時に設定されるスレッドの性質を**スレッド属性**と呼ぶ．新しいスレッドの属性は，pthread_attr_t 型の属性オブジェクトとして pthread_create に渡す．

▷ thread attribute

属性オブジェクトの代わりに NULL を渡すと，既定の属性を持つスレッドが生成される．属性オブジェクト自体も不透明であり，関数を通じてのみ操作する．

```
int pthread_attr_init(pthread_attr_t *attr);
int pthread_attr_destroy(pthread_attr_t *attr);
/* 属性 xxx ごとに: */
int pthread_attr_setxxx(pthread_attr_t *attr, ...);
int pthread_attr_getxxx(const pthread_attr_t *attr, ...);
```

pthread_attr_init はオブジェクトを既定値で埋める．この関数は，オブジェクトに他の関数を適用する前に呼ばなければならず，pthread_attr_destroy がそれを解放する．オブジェクトが参照されるのは pthread_create の実行中だけなので，スレッドが生成された後はすぐに破棄してよく，別のスレッドのために再利用してもよい．表 4.2 は，この講義で重要になる属性を挙げたものである．スコープ属性は，第 3 章のシステムスコープとプロセススコープのどちらかを選ぶ．PTHREAD_SCOPE_SYSTEM ではスレッドはシステム内のすべてのスレッドと CPU を奪い合い，PTHREAD_SCOPE_PROCESS では自分のプロセスのスレッドとだけ奪い合う．

既定のスタックは RLIMIT_STACK のソフトリミットで，通常は 8 MiB のアドレス空間である（第 3 章）．最小値は PTHREAD_STACK_MIN で，x86-64 では 16 KiB である（pthread_attr_setstacksize(3)）．

SCHED_OTHER は第 7 章の公平スケジューラである（CFS，Linux 6.6 で EEVDF に置き換えられた）．SCHED_FIFO と SCHED_RR はリアルタイムであり，優先度は 1–99 である．クォンタムを持つのは SCHED_RR だけで，既定値は 100 ms である（sched_rr_get_interval(2)）．

Linux はシステムスコープしかサポートしておらず，pthread_attr_setscope は PTHREAD_SCOPE_PROCESS を ENOTSUP で拒否する．したがって属性を設定する関数の戻り値もすべて検査しなければならない．

**表 4.2:** スレッド属性とそれを設定する関数．いずれにも対応する get 関数がある．

| 属性 | 設定する関数 | 意味 |
|---|---|---|
| スタックサイズ | pthread_attr_setstacksize | スレッドのスタックの大きさ |
| デタッチ状態 | pthread_attr_setdetachstate | ジョイン可能（既定）またはデタッチ（4.3 節） |
| スコープ | pthread_attr_setscope | システムスコープまたはプロセススコープ |
| 継承 | pthread_attr_setinheritsched | 方針と優先度を生成元から継承するか，オブジェクトから取るか |
| スケジューリング方針 | pthread_attr_setschedpolicy | 例: SCHED_OTHER，SCHED_FIFO，SCHED_RR |
| 優先度 | pthread_attr_setschedparam | 方針の中での優先度 |

## 4.3 デタッチスレッド

▷ joinable thread

**定義 4.3** (ジョイン可能スレッド). **ジョイン可能スレッド**とは，他のスレッドがジョインできるスレッドである．終了すると，その識別子と終了ステータスは pthread_join が回収するまで保持される．Pthreads のスレッドは，デタッチされた状態で生成されるか後からデタッチされない限り，ジョイン可能である．

**出典** P2L3，PThread のデタッチ；Kerrisk ch. 29.

プロセスのスレッドは対等である．ジョイン可能なスレッドをジョインできるのは，それを生成したスレッドだけではなく，どのスレッドでもよい．それでも通常の構成は，親スレッドが子を生成して後でジョインするというものであり，ジョインは結果を回収する以上の働きをする．終了したがジョインされていないジョイン可能なスレッドは**ゾンビ**スレッドとして残る．その識別子と終了ステータスは，いずれかのスレッドがジョインするまで保持され，誰もジョインしなければプロセスが生きている間ずっと資源を占有する．さらにメインスレッドは，子が終わる前に main から return してはならない．return するとプロセス全体が終了し，実行中のすべてのスレッドもそれとともに終了するからである．

第 2 章のゾンビプロセスと同じ勘定である．ワーカーを決してジョインしないサーバは，やがて新しいスレッドを生成できなくなる．

▷ detached thread

**定義 4.4** (デタッチスレッド). **デタッチスレッド**とは，ジョインできないスレッドである．終了すると，その資源は直ちに解放される．

**ヒント** 誰もジョインしないスレッドは，最初の文で pthread_detach(pthread_self()) を呼んで自分をデタッチする．

スレッドは生成後に，自分自身または他のスレッドによってデタッチできる．また属性オブジェクトでデタッチ状態を設定すれば，デタッチされた状態で生成できる．

```
int pthread_detach(pthread_t thread);

pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
```

デタッチされたスレッドを再びジョイン可能にすることはできない．デタッチすればジョインは不要になる．誰もそのスレッドのステータスを回収する必要がなく，資源は終了と同時に回収される．一方でスレッド間の関係は変わらず，どちらの場合もスレッドは対等である．スレッドがプロセスを終了させずに自分だけ終了するには pthread_exit(status) を呼ぶ．この呼び出しは呼び出し元のスレッドだけを終了させ，プロセスは最後のスレッドが終了するまで続く．残りのスレッドがジョイン可能かデタッチされているかは関係ない（図 4.1）．

**注意** main からの return や exit の呼び出しは，デタッチスレッドも含めてプロセス全体を終了させる．

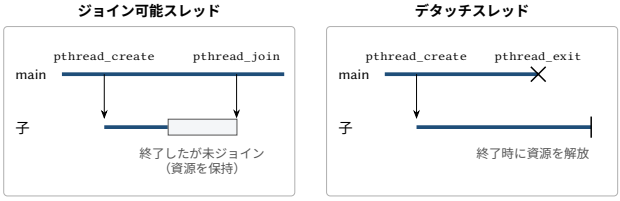



**図 4.1:** 左: ジョインされる前に終了したジョイン可能な子は，pthread_join がステータスを回収するまで資源を保持する．右: デタッチされた子は自分で資源を解放する．メインスレッドは pthread_exit で終了し，残りのスレッドがジョイン可能かデタッチされているかによらずプロセスは動き続ける．

**例 4.5.** メインスレッドは，システムスコープを持つデタッチされたログ出力スレッドを起動して終了し，ログ出力スレッドに作業を終えさせる．ログ出力スレッドは普通のスタートルーチンである．

```
void *logger(void *arg) {          /* スレッドのメイン */
  printf("logger: flushing records\n");
  pthread_exit(NULL);
}
```

メインスレッドは属性オブジェクトを用意し，ログ出力スレッドを生成して終了する．

```
int main(void) {
  pthread_t tid;
  pthread_attr_t attr;

  pthread_attr_init(&attr);        /* 必須 */
  pthread_attr_setdetachstate(&attr,
                              PTHREAD_CREATE_DETACHED);
  pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
  pthread_create(&tid, &attr, logger, NULL);
  pthread_attr_destroy(&attr);
  pthread_exit(NULL);              /* return 0 は不可 */
}
```

`main` が `return 0` で終わっていたら，ログ出力スレッドが何かを表示する前にプロセスが終了しうる．

> **要点**
> Pthreads のスレッドは，デタッチされない限りジョイン可能である．ジョイン可能なスレッドはすべてジョインするか，結果が誰にも必要ないスレッドはデタッチする．プロセスを終了させてはならないメインスレッドは，return する代わりに `pthread_exit` を呼ぶ．

## 4.4 Pthreads プログラムのコンパイル

Pthreads を使うプログラムには 3 つのものが必要である．

1. **ヘッダ．** Pthreads を使うソースファイルはすべて`<pthread.h>` をインクルードする．このヘッダが型，定数，関数を宣言する．
2. **コンパイラのフラグ．** プログラムは `-pthread` または`-lpthread` を付けてコンパイル・リンクする．

   ```
   gcc -o main main.c -pthread
   gcc -o main main.c -lpthread
   ```

   `-lpthread` は Pthreads ライブラリをリンクするだけだが，`-pthread` はマルチスレッドのコードのためにプラットフォームが必要とするプリプロセッサ定義も設定する．より移植性が高いのは後者である．
3. **戻り値の検査．** Pthreads の関数は，`errno` ではなく戻り値で失敗を報告する．成功なら 0，そうでなければエラー番号である．処理を続けられない呼び出し元は，例えば `strerror` でその番号を報告して終了する．`pthread_create` の結果を無視するプログラムは，後になって，生成されなかったスレッドを，設定されなかった識別子を使ってジョインしてしまうことがある．

**出典** P2L3，PThreads プログラムのコンパイル；OSTEP ch. 27．

GNU/Linux では `-pthread` は `_REENTRANT` も定義する．glibc 2.34 以降，Pthreads の関数は `libc` の一部である．

後で用いる競合検出器 `-fsanitize=thread` はリンク時にも指定しなければならず，さもなければリンクが未定義の`__tsan` シンボルで失敗する．`-g` を付けると報告にソースの行が加わる．

**注意** Pthreads は他の `posix_*` 関数と同様，システムコールの慣習（−1 を返し `errno` を設定）に従わない．`errno` は当てにならず，`perror` は無関係な値を表示しうる．

## 4.5 スレッド生成の例

3 つの小さなプログラムで，スレッドを生成してジョインする方法と，新しいスレッドにデータを届ける方法を示す．

**出典** P2L3，PThread の生成例 1–3；OSTEP ch. 27．

### 4.5.1 複数のスレッドの生成とジョイン

最初のプログラムは，既定の属性を持つスレッドを決まった数だけ生成し，そのすべてを待つ．

```
#include <stdio.h>
#include <pthread.h>
#define NUM_THREADS 3

void *greet(void *arg) {        /* スレッドのメイン */
  printf("Hello from a worker\n");
  return NULL;
}

int main(void) {
  pthread_t tid[NUM_THREADS];
  int i;

  for (i = 0; i < NUM_THREADS; i++)  /* 生成     */
    pthread_create(&tid[i], NULL, greet, NULL);
  for (i = 0; i < NUM_THREADS; i++)  /* ジョイン */
    pthread_join(tid[i], NULL);
  return 0;
}
```

このプログラムは 2 つの段階からなる．最初のループがスレッドを生成し，2 番目のループがそれらをジョインする．各ワーカーは 1 行を表示する．ワーカーは何のデータも共有しないので同期は不要であり，すべてが同じ文字列を表示するので，出力はスレッドがスケジュールされる順序に依存しない．ジョインは，すべてのワーカーが自分の行を表示する前に `main` が return して–それによってプロセスを終了させて–しまうのを防ぐ．

行が混ざることはない．POSIX は stdio の各呼び出しにストリームをロックさせる（`flockfile(3)`）ので，`printf` にミューテックスは要らない．

### 4.5.2 引数を渡す

各ワーカーは自分の番号を表示するものとする．スタートルーチンは引数を通じて番号を読む．

```
void *show_index(void *arg) {   /* スレッドのメイン */
  int *p = (int *)arg;
  int my_num = *p;
  printf("Thread %d\n", my_num);
  return NULL;
}
```

番号を値として `(void *)(intptr_t)i` の形で渡し，キャストで戻す書き方もある．共有変数はなくなるが，この変換は処理系定義である．

最初の試みでは，ループ変数のアドレスを渡す．

```
for (i = 0; i < NUM_THREADS; i++)
  pthread_create(&tid[i], NULL, show_index, &i);
```

**ヒント** 競合検出器を使えばこれを確実に検出できる．`-fsanitize=thread` を付けてビルドするか，`valgrind --tool=helgrind` の下で実行する．

このプログラムが 0，1，2 を表示する保証はない．どのスレッドも同じアドレス，すなわち `main` のスタックフレームにある唯一の変数 `i` のアドレスを受け取り（図 4.2 左），スレッドが値を読むのは `*p` を実行した時点である．その時点で `main` はすでに `i` を増やしているかもしれない．`i` はそのアドレスを持つすべてのスレッドから見えるので，あるスレッドが加えた変更はすべてのスレッドに見え，`main` が `i` を変更している間にそれを読むスレッドはデータ競合（第 3 章）にある．したがってワーカーは，別のワーカーに属する番号を表示することもあれば，生成ループの後でしか `i` が取らない値–ループ終了時の 3 や，`i` を再利用するジョインループが設定した値–を表示することもある．表 4.3 は起こりうる実行の 1 つを示す．

### 4.5.3 専用のデータを渡す

各スレッドが，自分だけに属し他のどのスレッドも変更しないデータのアドレスを受け取るようにすれば，競合はなくなる．

```
int t_num[NUM_THREADS];

for (i = 0; i < NUM_THREADS; i++) {
  t_num[i] = i;
  pthread_create(&tid[i], NULL, show_index, &t_num[i]);
}
```

生成元が待てない場合の通常の構成は，スレッドごとに 1 つの引数構造体を `malloc` で確保し，スレッド自身が解放するというものである．

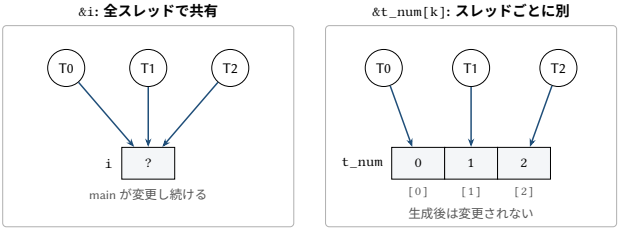



**図 4.2:** 左: すべてのスレッドがループ変数 `i` のアドレスを受け取り，その値はスレッドの起動中に変化する．右: 各スレッドは自分専用の配列要素のアドレスを受け取り，その値はスレッドの生成後に変化しない．

**表 4.3:** `&i` を渡すプログラムを 3 スレッドで実行した例．T0 は `main` が `i` を増やした後に読み，T2 はジョインループが `i` を 0 に戻した後に読む．自分の番号を表示するのは T1 だけであり，プログラムは 1，1，0 を表示する．

| ステップ | メインスレッド | i | ワーカー |
|---|---|---|---|
| 1 | `&i` で T0 を生成 | 0 | |
| 2 | `i++` | 1 | |
| 3 | `&i` で T1 を生成 | 1 | |
| 4 | | 1 | T0 が `*p` を読む: 1 を表示 |
| 5 | | 1 | T1 が `*p` を読む: 1 を表示 |
| 6 | `i++`，`&i` で T2 を生成 | 2 | |
| 7 | `i++`，生成ループ終了 | 3 | |
| 8 | ジョインループ: `i = 0`，T0 をジョイン | 0 | |
| 9 | | 0 | T2 が `*p` を読む: 0 を表示 |

各ワーカーは今度は `t_num` の自分の要素を読む．その値はスレッドの生成前に設定され，その後は決して変化しない（図 4.2 右）．すべての番号がちょうど一度ずつ表示される．ただし行の**順序**は依然として決まらない．それはスレッドがどうスケジュールされるかに依存し，3 行のどの並びも起こりうる．配列は `main` のスタックフレームにあり，`main` はすべてのスレッドをジョインするまで return しないので，スレッドがそれを使っている間このメモリは有効なままである．

3 行の並び方は $3! = 6$ 通りあり，ある実行で順に表示されても残りの 5 通りについては何も言えない．

**注意** スレッドがデータを読む前に return しうる関数の局所変数のアドレスを，スレッドに渡してはならない．

> **要点**
> アドレスで渡した引数は共有されるのであって，コピーされるのではない．各スレッドには，そのスレッドが使い終わるまで生き続け，他のどのスレッドも変更しない専用のデータ–配列の要素や，そのスレッドのために確保した構造体–を与える．

## 4.6 ミューテックス

Pthreads のミューテックスは，第 3 章のミューテックスと同様に，並行するスレッドの間の相互排除の問題を解決する．ミューテックスの型は `pthread_mutex_t` であり，基本的な操作は 2 つである．

**出典** P2L3，PThread のミューテックス；OSTEP ch. 27–28；Silberschatz ら ch. 7．

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

意味は Birrell のミューテックスと同じである．空いているミューテックスをロックしたスレッドがその所有者となり，他のスレッドが所有するミューテックスをロックしようとしたスレッドは，それが解放されるまでブロックする．[1] 違いは構文上のものだが，重要である．Birrell の `Lock(m) {…}` はブロックの終わりで暗黙にミューテックスを解放する．Pthreads ではクリティカルセクションは 2 つの呼び出しの間のコードであり，プログラムはそこから出るすべての経路–早期の return やエラー処理の経路も含めて–で明示的に `pthread_mutex_unlock` を呼ばなければならない．

[1] 空いているミューテックスのロックは，ユーザ空間のアトミックなリードモディファイライト命令 1 つで済む．glibc が `futex(2)` を通じてカーネルに入るのは，スレッドがブロックしなければならないときと，他のスレッドを起こさなければならないときだけである．したがって競合のないロックは数十ナノ秒で済み，競合するロックはシステムコールとコンテキストスイッチの分だけかかる．Drepper，「Futexes Are Tricky」[13]．

**例 4.6.** 第 3 章の安全なリスト挿入を C と Pthreads で書くと次のようになる．

```
struct node { int value; struct node *next; };

struct node *head = NULL;  /* 共有リスト */
pthread_mutex_t list_m = PTHREAD_MUTEX_INITIALIZER;

void safe_insert(int value) {
  struct node *e = malloc(sizeof *e);
  e->value = value;

  pthread_mutex_lock(&list_m);
  e->next = head;          /* クリティカルセクション */
  head = e;
  pthread_mutex_unlock(&list_m);
}
```

共有の先頭ポインタを読み書きする 2 つの文だけがクリティカルセクションにある．新しい要素の確保と値の設定は共有データに関わらないので，ロックを取る前に行う．こうしてクリティカルセクションは短く保たれる．

### 4.6.1 その他のミューテックス操作

型は小さい．x86-64 の glibc では pthread_mutex_t が 40 B，pthread_cond_t が 48 B なので，保護する構造体ごとにロックを埋め込める．

基本的なミューテックスのインタフェースは，さらに 3 つの操作で完結する．

```
int pthread_mutex_init(pthread_mutex_t *mutex,
                       const pthread_mutexattr_t *attr);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

型も属性である．既定の型のミューテックスを再びロックすることは POSIX では未定義である（glibc はこれを NORMAL に対応付けるのでデッドロックする．第 3 章）．PTHREAD_MUTEX_ERRORCHECK は EDEADLK を返し，PTHREAD_MUTEX_RECURSIVE は回数を数える．

pthread_mutex_timedlock（POSIX.1）は絶対時刻で与えた期限まで待つだけで，その後は ETIMEDOUT を返す．

pthread_mutex_init は，最初に使う前にミューテックスを初期化する．その属性オブジェクトは pthread_mutexattr_t 型であり，例えばプロセス間で共有してよいかどうか（第 9 章）といったミューテックスの振る舞いを指定する．NULL は既定値を選ぶ．既定の属性を持つミューテックスは，例 4.6 のように，マクロ PTHREAD_MUTEX_INITIALIZER を使って静的に初期化することもできる．

pthread_mutex_trylock は，ミューテックスが空いていればそれをロックして 0 を返す．ミューテックスがロックされている場合はブロックせず，直ちにエラー EBUSY を返す．呼び出したスレッドは別のことをして，後で再試行できる．

pthread_mutex_destroy は，不要になったミューテックスのデータ構造を解放する．ミューテックスはロックされていてはならない．

**例 4.7.** あるスレッドが，主たる仕事を遅らせることなく，ミューテックスがたまたま空いているときだけ補助的な統計情報を更新する．

```
if (pthread_mutex_trylock(&stats_m) == 0) {
  stats.requests += local_count;
  pthread_mutex_unlock(&stats_m);
  local_count = 0;
}  /* 空きがなければ局所的に数え，後で再試行 */
```

trylock をループの中で呼ぶとスピンロックになり，他のスレッドがミューテックスを保持している間 CPU を浪費する．第 10 章がスピンとブロックを比較する．

### 4.6.2 ミューテックスの安全な使い方

ミューテックスに関する最もよくある誤りは，いくつかの規則で避けられる．

- **共有データ 1 つにつきミューテックス 1 つ．**共有変数へのすべてのアクセスは，同じミューテックスを通らなければならない．異なるコード経路が同じデータを異なるミューテックスで保護しても，互いに排除し合うことはない．
- **ミューテックスは，それを使うすべてのスレッドから見えなければならない．**したがってミューテックスは大域スコープで，あるいはこれらのスレッドすべてが到達できる構造体の中で宣言する．スレッド関数の局所変数として宣言したミューテックスは，スレッドごとに別のミューテックスになる．
- **ミューテックスは大域的な順序でロックする．**すべてのスレッドが同じ順序でミューテックスを獲得すれば，第 3 章のデッドロックは起こりえない．
- **必ずアンロックし，正しいミューテックスをアンロックする．**アンロックを忘れると，そのミューテックスを必要とする他のすべてのスレッドがブロックする．誤ったミューテックスをアンロックすると，一方のクリティカルセクションが保護されないまま，もう一方がロックされたままになる．

**ヒント** ミューテックスに番号を付け，常に番号の小さい方から順にロックする．より小さい番号のものが必要になったスレッドは，保持しているものを解放してやり直す．

## 4.7 条件変数

pthread_cond_t 型の Pthreads の条件変数は，Birrell の 3 つの操作を提供する．

出典 P2L3，PThread の条件変数；OSTEP ch. 27，30．

```
int pthread_cond_wait(pthread_cond_t *cond,
                      pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

意味は第 3 章のとおりである．pthread_cond_wait は mutex を保持した状態で呼び出す．この関数はミューテックスをアトミックに解放して呼び出したスレッドを cond の待ちキューに入れ，スレッドが起こされると，戻る前にミューテックスを再獲得する．pthread_cond_signal は待っているスレッドの少なくとも 1 つを，pthread_cond_broadcast はそのすべてを起こす．どちらも，待っているスレッドがなければ何も起こらない．

pthread_cond_timedwait は相対的なタイムアウトではなく絶対時刻の期限を取る．時計は既定で CLOCK_REALTIME である（pthread_condattr_setclock）（POSIX.1）．

注意 引数の順序は Birrell の Wait(m, c) と逆である．pthread_cond_wait は条件変数を先に取る．

POSIX は，どのスレッドも signal していないのに pthread_cond_wait が戻ることを許しており，これもスプリアスウェイクアップと呼ぶ．これは，利用できない通知によってスレッドが起こされる第 3 章の場合とは別の事象である．どちらも起こされたスレッドが先に進めない点は同じであり，どちらも同じ規則で処理される．すなわち，起こされたスレッドは述語を再検査する．したがって第 3 章の while ループは，単に望ましい習慣であるだけでなく必須である．[2]

[2] POSIX.1 [12] の根拠はこの許容を次のように説明している．wait から無駄に戻ることが決してないことを要求すると，マルチプロセッサでは条件変数が遅くなり，実装が pthread_cond_signal を broadcast として実現することも許されなくなる．どのみち述語は検査しなければならないので，ループに費用はなく，余分な通知に対してプログラムが強くなる．

```
pthread_mutex_lock(&m);
while (!predicate)
  pthread_cond_wait(&c, &m);
/* 述語が成り立ち m を保持している */
update_shared_state();
pthread_mutex_unlock(&m);
```

ミューテックスと同様に，条件変数も明示的に初期化・破棄する．

```
int pthread_cond_init(pthread_cond_t *cond,
                      const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

pthread_condattr_t 型の属性オブジェクトは，例えばプロセス間で共有されるかどうかといった条件変数の振る舞いを指定する．NULL は既定値を選び，静的初期化子 PTHREAD_COND_INITIALIZER も同じ既定値を与える．

プロセス間で共有するには，条件変数とそのミューテックスの両方に PTHREAD_PROCESS_SHARED を設定し，どちらも共有メモリに置かなければならない（第 9 章）．

### 4.7.1 条件変数の安全な使い方

- **待っているスレッドへの通知を忘れない．**述語が依存する状態を変更したスレッドは必ず，その述語を待つスレッドがブロックしている条件変数に signal または broadcast しなければならない．
- **迷ったら broadcast する．**signal で足りるところで broadcast しても正しさは保たれる．起こされたスレッドはそれぞれ述語を再検査し，先に進めなければ再び待つからである．代償は性能であり，これらのスレッドは無駄に起こされてスケジュールされる．
- **signal や broadcast にミューテックスは必要ない．**通知するかどうかの判断が，ミューテックスを保持している間しか読めない共有状態に依存しない場合，通知はミューテックスを解放した後に発行できる．そうすれば，起こされたスレッドは，signal したスレッドがまだ保持しているミューテックスを待たされずに済む．この待ちが，第 3 章で調べたスプリアスウェイクアップの原因である．通知がまったくないのに pthread_cond_wait が戻ることは依然としてありうるので，これによって while ループが不要になるわけではない．

待っているスレッドが $n$ 個あると，broadcast は $n$ 回の起床と $n$ 回のミューテックス獲得を要し，そのうち $n-1$ 回は無駄である．第 3 章の「thundering herd」である．

> **要点**
> Pthreads のミューテックスと条件変数は Birrell と同じ意味を持つ．それに加わるのは，明示的なアンロック，明示的な初期化と破棄，そして属性オブジェクトである．wait は while ループの中で行い，述語が変化した可能性があるときは必ず通知し，通知するかどうかの判断が許すならアンロックの後に通知する．

## 4.8 生産者・消費者の例

出典 P2L3，PThreads を用いた生産者と消費者の例；OSTEP ch. 30．
▷ bounded buffer

第 3 章の生産者・消費者問題は，ここまでの内容によって Pthreads の完全なプログラムとして解ける．生産者スレッドが共有の**有界バッファ**—循環キューとして使う BUF_SIZE 個

のスロットの配列—に値を挿入し，消費者スレッドが挿入された順にそれらを取り出す．バッファは 4 つの変数で記述される．配列 buffer，次に埋めるスロットの添字 add，次に空けるスロットの添字 rem，そしてバッファ内の値の個数 num である．どちらの添字も BUF_SIZE を法として進むので，キューは配列の末尾で先頭に戻る（図 4.3）．

1 つのミューテックス m がこれらの変数を保護する．生産者はバッファが満杯の間 c_prod で待ち，消費者は空の間 c_cons で待つ．共有状態はすべて，ミューテックスの安全な使い方が要求するとおり大域変数であり，静的に初期化される．

1 つのスロットを空けておけば num は不要になる．そのときバッファが空なのは add == rem のとき，満杯なのは (add + 1) % BUF_SIZE == rem のときである．

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define BUF_SIZE  4     /* 共有バッファのスロット数  */
#define NUM_ITEMS 10    /* 生産する値の個数          */

int buffer[BUF_SIZE];   /* 共有循環バッファ          */
int add = 0;            /* 次に埋めるスロット        */
int rem = 0;            /* 次に空けるスロット        */
int num = 0;            /* バッファ内の値の個数      */

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c_cons = PTHREAD_COND_INITIALIZER;
pthread_cond_t c_prod = PTHREAD_COND_INITIALIZER;

void *producer(void *param);
void *consumer(void *param);
```

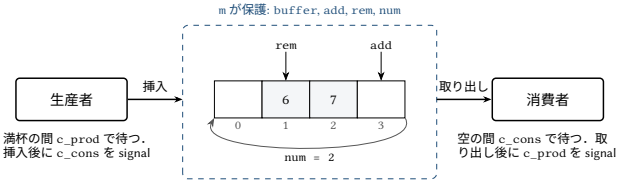



**図 4.3:** BUF_SIZE = 4 の生産者・消費者プログラム．1 つのミューテックスがバッファの状態をすべて保護し，各スレッドは自分専用の条件変数で待つ．

これは OSTEP ch. 30 の有界バッファである．OSTEP ch. 31 は同じプログラムをセマフォで構成する．

メインスレッドは 2 つのスレッドを生成してジョインする．

```
int main(void) {
  pthread_t tid1, tid2;

  if (pthread_create(&tid1, NULL, producer, NULL) != 0 ||
      pthread_create(&tid2, NULL, consumer, NULL) != 0) {
    fprintf(stderr, "cannot create threads\n");
    exit(1);
  }
  pthread_join(tid1, NULL);   /* 生産者を待つ */
  pthread_join(tid2, NULL);   /* 消費者を待つ */
  printf("parent done\n");
  return 0;
}
```

**ヒント** BUF_SIZE が定数の 2 のべきなら，コンパイラはすでに除算を避ける．add と rem を unsigned で宣言するか，& (BUF_SIZE - 1) と書けば，折り返しは AND 命令 1 つになる．

生産者は値 1,…,NUM_ITEMS を挿入する．

```
void *producer(void *param) {
  int i;
  for (i = 1; i <= NUM_ITEMS; i++) {
    pthread_mutex_lock(&m);
    while (num == BUF_SIZE)          /* 満杯: 待つ */
      pthread_cond_wait(&c_prod, &m);
    buffer[add] = i;                 /* 満杯でない */
    add = (add + 1) % BUF_SIZE;
    num++;
    pthread_mutex_unlock(&m);
    pthread_cond_signal(&c_cons);    /* 空でない   */
```

```
    printf("produced %d\n", i);
  }
  return NULL;
}
```

消費者は対称である．

各スレッドは自分のスタックに自分の i と n を持つ（第 3 章）．共有されているのは buffer，add，rem，num だけである．

```
void *consumer(void *param) {
  int i, n;
  for (n = 0; n < NUM_ITEMS; n++) {
    pthread_mutex_lock(&m);
    while (num == 0)                /* 空: 待つ   */
      pthread_cond_wait(&c_cons, &m);
    i = buffer[rem];                /* 空でない   */
    rem = (rem + 1) % BUF_SIZE;
    num--;
    pthread_mutex_unlock(&m);
    pthread_cond_signal(&c_prod);   /* 満杯でない */
    printf("consumed %d\n", i);
  }
  return NULL;
}
```

このプログラムは，この章と第 3 章の規則を適用している．

- 各スレッドは m を保持したまま while ループで述語を検査する．wait から戻ったときスレッドは再び m を保持しており，バッファに触れる前に述語を再検査する．
- 各スレッドはアンロックの後に signal するので，起こされたスレッドは m を待たずに済む．誰も待っていないときの signal は何もしないが，これは無害である．スレッドは待つ前に述語を検査するので，すでに起きた変化を待つことは決してないからである．
- 生産者は c_prod だけで，消費者は c_cons だけで待つので，通知は必ず，それを利用できる種類のスレッドに届く．
- 消費者は NUM_ITEMS 個の値を取り出したら停止する．無限にループする消費者は決して終了せず，main の 2 番目の pthread_join は決して戻らない．

printf がクリティカルセクションの外にあるのも同じ理由である．端末への書き込みはブロックしうるし，m を保持したままブロックすればもう一方が止まる．

**注意** 複数の生産者または消費者が 1 つの条件変数を共有すると，消費者しか進めないときに signal が生産者を起こしうる．そのような設計には broadcast が必要である．

**例 4.8.** 表 4.4 は，消費者が動き出す前に生産者が先行してバッファを満たす場合の実行を 1 つトレースしたものである．取り除かれた値は- で示す．

**表 4.4:** BUF_SIZE = 4 における生産者（P）と消費者（C）のトレース．buffer 列はスロット 0–3 を示し，add，rem，num の値は各ステップの後のものである．

| ステップ | イベント | buffer | add | rem | num |
|---|---|---|---|---|---|
| 1 | P が 1 を挿入 | 1 - - - | 1 | 0 | 1 |
| 2 | P が 2，3，4 を挿入 | 1 2 3 4 | 0 | 0 | 4 |
| 3 | P は満杯を見て c_prod で待つ | 1 2 3 4 | 0 | 0 | 4 |
| 4 | C が 1 を取り出し c_prod に signal | - 2 3 4 | 0 | 1 | 3 |
| 5 | P は m を再獲得，再検査して 5 を挿入 | 5 2 3 4 | 1 | 1 | 4 |
| 6 | C が 2，3，4 を取り出す | 5 - - - | 1 | 0 | 1 |
| 7 | C が 5 を取り出す | - - - - | 1 | 1 | 0 |
| 8 | C は空を見て c_cons で待つ | - - - - | 1 | 1 | 0 |
| 9 | P が 6 を挿入し c_cons に signal | - 6 - - | 2 | 1 | 1 |
| 10 | C は m を再獲得，再検査して 6 を取り出す | - - - - | 2 | 2 | 0 |

**要点 — 章のまとめ**

POSIX スレッドは，Birrell の機構を具体的な型と呼び出しで実現する（表 4.1）．`pthread_create` は，1 つの `void *` 引数と，例えばスタックサイズ，スコープ，デタッチ状態を設定する属性オブジェクトとともにスレッドを起動する．ジョイン可能なスレッドはジョインしなければならず，デタッチされたスレッドは自分で資源を解放し，`pthread_exit` はプロセスを終了させずにスレッドを終了させる．プログラムは `<pthread.h>` をインクルードし，`-pthread` でビルドし，戻り値を検査し，各スレッドに専用の引数データを与える．共有資源ごとに大域から見える 1 つのミューテックスを置き，大域的な順序でロックして必ずアンロックする．条件変数は `while` ループの中で wait し，述語が変化した可能性があるときは必ず通知し，通知するかどうかの判断が許すならアンロックの後に通知する．生産者・消費者プログラムがその例である．

第 5 章

# スレッド設計上の考慮事項

| | |
|---|---|
| **講義動画** | P2L4 |
| **教科書** | OSTEP [1] ch. 6，26–28；Silberschatz ら [2] ch. 4；Kerrisk [11] ch. 20–22，28 |
| **キーワード** | ハードプロセス状態，ライトプロセス状態，軽量プロセス，レッドゾーン，適応型ミューテックス，リーパスレッド，割り込みハンドラ，シグナルハンドラ，シグナルマスク，トップハーフ，ボトムハーフ，リアルタイムシグナル，タスク，NPTL |

第 3 章ではカーネルレベルスレッドとユーザレベルスレッド，および一方を他方に対応付けるモデルを導入し，第 4 章ではアプリケーションがそれらを利用するための POSIX スレッドのインタフェースを扱った．この章では実装の内側を見る．すなわち，カーネルとユーザレベルのスレッドライブラリがどのようなデータ構造を保持するか，2 つのレベルがどのように判断を協調させるか，そして割り込みとシグナルがマルチスレッドとどう関わるかである．説明は SunOS 5.0 のマルチスレッドカーネル [14] とそのユーザレベルスレッドライブラリ [15] に関する 2 つの論文に沿って進め，最後に Linux がスレッドを表現する方法で締めくくる．

## 5.1 スレッドのデータ構造

スレッドを支えるには，2 つのレベルでデータ構造が必要である．カーネルレベルスレッドのためのカーネル内のものと，ユーザレベルスレッドのためのユーザレベルライブラリ内のものである．すべてのユーザレベルスレッドを 1 つのカーネルレベルスレッド上で実行するプロセス（第 3 章の多対一モデル）が最も単純な場合である．カーネルはこのプロセスを PCB で記述し，PCB は仮想アドレスの対応付けと，カーネルレベルスレッドの実行コンテキスト，すなわちそのレジスタ，スタックポインタ，スタックを保持する．ユーザレベルライブラリは，ユーザレベルスレッドごとにデータ構造を保持し，そこにスレッドの識別子，保存されたレジスタ，スタックを収める．これらのコンテキストの 1 つをカーネルレベルスレッドに読み込むことで，ライブラリはどのユーザレベルスレッドを実行するかを決める．

**出典** P2L4，スレッドのデータ構造：単一 CPU の場合と大規模な場合，ハードプロセス状態とライトプロセス状態，データ構造を分ける根拠；Eykholt ら [14]；OSTEP ch. 26．

プロセスが複数のカーネルレベルスレッドを持つ場合，1 つの PCB ではすべてを保持できない．カーネルレベルスレッドはそれぞれ独自の実行コンテキストを必要とするからである．各スレッドについて PCB 全体を複製すると，プロセスのすべてのスレッドで同一である仮想アドレスの対応付けまで複製することになる．そこでカーネルは，カーネルレベルスレッドごとにそのレジスタ，スタックポインタ，スタックを収める別個のカーネルレベルスレッドデータ構造を保持し，PCB にはプロセスに共通の情報を残す．

PCB そのものは第 2 章で扱った．ここではその内容を分割し，1 つのプロセスのスレッドがアドレス空間の部分を共有できるようにする．

多数のプロセス，多数のスレッド，複数の CPU がある場合には，これらの構造の間の関係も記録しなければならない（図 5.1）．

- ユーザレベルライブラリは，自プロセスのすべてのユーザレベルスレッドと，それらを実行できるカーネルレベルスレッドを把握する．
- PCB は，そのプロセスのために実行されるカーネルレベルスレッドを把握する．
- 各カーネルレベルスレッドは自プロセスの PCB を指し，カーネルはそのスレッドを実行するときにアドレス空間を見つけられる．
- CPU ごとのデータ構造が，とりわけその CPU で現在実行中のスレッドを記録し，実行中のカーネルレベルスレッドは自分の CPU を指す．

x86-64 の Linux では，タスクはそれぞれ 16 KiB（4 ページ）のカーネルスタックを持つが，アドレス空間は共有される．カーネル文書 `x86/kernel-stacks`．

### 5.1.1 ハードプロセス状態とライトプロセス状態

カーネルが同じプロセスの 2 つのカーネルレベルスレッドを切り替えるとき，仮想アドレスの対応付けは有効なままであり，保存・復元しなければならないのは個々のスレッドに属する状態だけである．その状態の一部はユーザレベルスレッドに関わる．どのシグナルがマスクされているか，および実行中のシステムコールの引数は，プロセス全体ではなく，そのカーネルレベルスレッド上で現在実行しているユーザレベルスレッドに属する．こうしてプロセスの状態は 2 つの部分に分かれる．

**ヒント** プロセスのすべてのスレッドについて有効ならハード状態，現在実行中のスレッドについてだけ有効ならライト状態である．

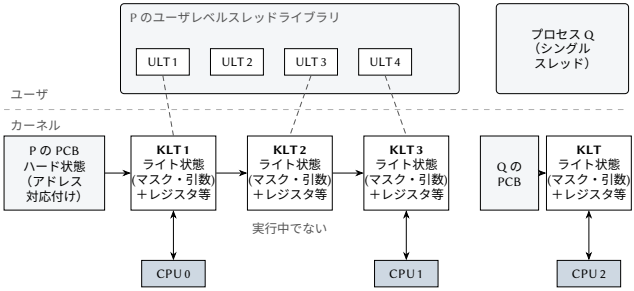



**図 5.1:** 大規模な場合のスレッドのデータ構造．プロセスの PCB は，そのすべてのスレッドが共有する状態と，カーネルレベルスレッド（KLT）のリストを保持する．各 KLT は独自の状態を保持し，実行中は CPU に結び付けられる．破線は，ライブラリが現在どのユーザレベルスレッド（ULT）をどの KLT に配置しているかを示す．KLT から PCB への逆向きのポインタは省いてある．

▷ hard process state

▷ light process state

**定義 5.1** (ハードプロセス状態とライトプロセス状態). **ハードプロセス状態**とは，プロセスの状態のうち，その中で実行されるすべてのユーザレベルスレッドに関係する部分であり，仮想アドレスの対応付けなどがこれにあたる．**ライトプロセス状態**とは，特定のカーネルレベルスレッドに現在結び付けられているユーザレベルスレッドの部分集合にのみ関係する部分であり，シグナルマスクやシステムコールの引数などがこれにあたる．

**注意** この 2 つの用語は講義のものである．教科書ではプロセスごとの状態，スレッドごとの状態と呼ぶ．

### 5.1.2 データ構造を複数に分ける理由

1 つの PCB と，相互に参照する小さな構造の集合とでは，コストが異なる（表 5.1）．1 つの PCB は大きな連続した構造であり，その内容が共有されている場合でも実行コンテキストごとに専有される．コンテキストスイッチのたびに全体が保存・復元され，状態が変化すればそのたびに更新される．ポインタで互いを参照する小さな構造であれば，共有される状態は 1 つだけ保持すればよく，コンテキストスイッチで保存・復元するのは変化する部分だけで済み，ユーザレベルライブラリは自ら管理する部分をカーネルを介さずに更新できる．この分割はスケーラビリティ，性能，柔軟性を改善し，オーバーヘッドを減らす．

**表 5.1:** 1 つの PCB と複数の小さなデータ構造の比較．

| | 1 つの PCB | 複数のデータ構造 |
|---|---|---|
| 構造 | 大きな連続した 1 つの構造 | ポインタで連結された小さな構造 |
| 共有 | 実行コンテキストごとに専有 | 共有される状態は 1 つだけ保持 |
| コンテキストスイッチ | 全体を保存・復元 | 変化する部分だけ |
| 更新 | 変更のたびに PCB を書き換え | ライブラリは自分の部分だけを更新 |
| 帰結 | スケーラビリティが低く，オーバーヘッドが大きく，性能が低く，柔軟性に乏しい | スケーラビリティが高く，オーバーヘッドが小さく，性能が高く，柔軟性に富む |

## 5.2 SunOS 5.0 のスレッド

**出典** P2L4，SunOS 5.0 のスレッドモデル，ユーザレベルスレッドのデータ構造，カーネルレベルのデータ構造；Eykholt ら [14]；Stein と Shah [15]．

SunOS 5.0，すなわち Solaris 2 のカーネルは共有メモリ型マルチプロセッサ向けに設計されており，カーネル自体がマルチスレッドである．そのスレッドモデル（図 5.2）は次の要素からなる．

- プロセスはシングルスレッドでもマルチスレッドでもよい．ユーザレベルライブラリは，バウンドスレッドを含む多対多モデルと一対一モデルの両方に対応し，プロセスはすべてのユーザレベルスレッドを 1 つの軽量プロセス上で実行することもできる．

- ユーザレベルスレッドを実行するカーネルレベルスレッドは，すべて軽量プロセスと結び付けられる．
- それ以外のカーネルレベルスレッドは軽量プロセスを持たず，カーネル自身の仕事を行う．
- カーネルレベルスケジューラが，カーネルレベルスレッドを CPU にスケジュールする．

**定義 5.2 (軽量プロセス).** SunOS 5.0 における**軽量プロセス**（LWP）とは，ユーザレベルスレッドを実行するカーネルレベルスレッドに結び付けられたカーネルのデータ構造であり，その実行コンテキストのライトプロセス状態を保持する．ユーザレベルライブラリから見ると，LWP はユーザレベルスレッドをスケジュールする先の仮想 CPU を表す．

▹ lightweight process

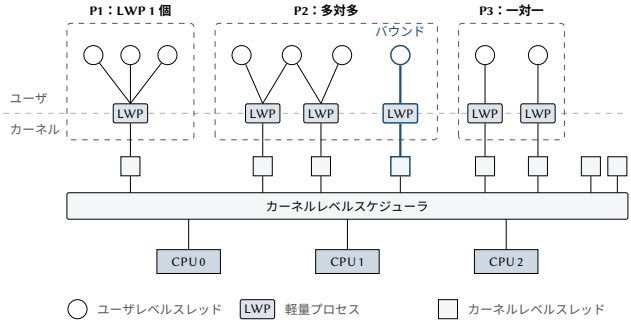



**図 5.2:** SunOS 5.0 のスレッドモデル．ユーザレベルスレッドを実行するカーネルレベルスレッドはそれぞれ LWP を持つ．右側の 2 つのカーネルレベルスレッドはカーネルの仕事を行い，LWP を持たない．P1 はすべてのユーザレベルスレッドを 1 つの LWP 上で実行している．カーネルレベルスケジューラがカーネルレベルスレッドを CPU に配置する．

### 5.2.1 ユーザレベルスレッドのデータ構造

Stein と Shah のライブラリは，スレッドを生成するとスレッド識別子を返す．この識別子はスレッドデータ構造へのポインタではなく，構造へのポインタの表への添字である．この間接参照は古くなった識別子から保護する．スレッドがすでに終了してその構造のメモリが再利用されている場合，直接のポインタは無関係なデータを黙って指してしまうが，表のエントリにはスレッドの状態を記録できるので，ライブラリは意味のあるエラーを返せる．

スレッドデータ構造は，実行コンテキスト（スタックポインタを含むレジスタ），シグナルマスク，優先度，**スレッドローカルストレージ**，スタックを含む．スレッドローカルストレージは，プログラムがスレッドごとに固有と宣言した変数を保持する．その合計サイズはプログラムの開始時に実行時リンカが計算し，ライブラリは新しいスレッドのスレッドローカルストレージをそれぞれのスタックの基部に割り当てる．スレッド構造そのものもスレッドローカルストレージに置かれる．スタックのサイズは，ライブラリの既定値によって，あるいはアプリケーションによって定まる．すべての部分のサイズがスレッドの生成時に分かっているので，ライブラリは連続したメモリ上に次々とスレッドの構造を隣り合わせに割り当てることができ，局所性がよくなる．

連続割り当てには危険がある．ライブラリはスタックがどこまで伸びるかを制御できず，オペレーティングシステムはそもそもユーザレベルスレッドを知らない．したがって，スタックがあふれたスレッドは隣のスレッドのデータ構造に書き込んでしまい，その被害は，その別のスレッドが実行されて異常を起こすときに—原因から遠く離れたところで—初めて現れる．

**注意** 講義と論文の違い：講義は，スレッドローカルストレージはコンパイル時に分かっているので，すべてのスレッドに場所を確保できると述べる．Stein と Shah では，そのサイズはプログラムの開始時に初めて，実行時リンカによって定まる．

**定義 5.3 (レッドゾーン).** **レッドゾーン**とは，あるスレッドのスタックと隣のスレッドのデータ構造の間にある，メモリに対応付けられていない仮想アドレス空間の領域であり，そこへのアクセスはすべてフォールトを起こす．

▹ red zone

**注意** 同じ語の別の意味：x86-64 の ABI におけるレッドゾーンは，スタックポインタの下の 128 B であり，リーフ関数がスタックポインタを動かさずに使ってよい領域を指す．

スタックがレッドゾーンに伸びると直ちに，あふれさせた当のスレッドでフォールトが起こるので，誤りの診断が容易になる（図 5.3）．

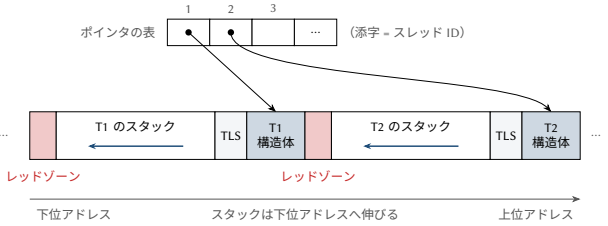



**図 5.3:** 連続して割り当てられたユーザレベルスレッドのデータ構造．スレッド識別子は，構造へのポインタの表への添字である．T2 のスタックがあふれても，T1 の構造に達する前にレッドゾーンに入る．

**例 5.4.** あるライブラリが各スレッドに 64 KiB のブロックを与える．下端に 4 KiB のレッドゾーン，56 KiB のスタック，上端にスレッド構造とそのスレッドローカルストレージのための 4 KiB である．領域の先頭からの KiB 単位で測ると，ブロック $k$ は $[64k, 64k + 64)$ を占める．したがってスレッド 3 はレッドゾーンを [192, 196) に，スタックを [196, 252) に，構造を [252, 256) に持ち，スレッド 2 の構造は [188, 192) にある．スレッド 3 が 56 KiB を超えるスタックを必要とすると，次のプッシュは [192, 196) に入ってフォールトする．レッドゾーンがなければ，スレッド 2 の構造を上書きしてしまう．1024 スレッドでは領域は 64 MiB の仮想アドレス空間に及ぶが，そのうち 4 MiB のレッドゾーンは物理メモリを使わない．

x86 の NPTL も同じ配置を保つ．スレッド記述子はスタックブロックの上端にあり，glibc はスタックの下に既定で 1 ページ（4 KiB）のガードページを置く．`pthread_attr_setguardsize` で変更できる．

### 5.2.2 カーネルレベルのデータ構造

SunOS 5.0 のカーネルは，プロセスとスレッドについて 4 種類の構造を保持する（表 5.2）．LWP はユーザレベルスレッドのデータ構造に似ているが，カーネルから見える．その内容はプロセスが実行中の間だけ必要なので，LWP はプロセスとともにスワップアウトできる．カーネルレベルスレッド構造は，スケジューリング情報のように，プロセスが実行中でないときにもカーネルが必要とする情報を保持し，決してスワップアウトされない．CPU 構造は，各 CPU で現在実行中のスレッドを示す．

予備のレジスタを持つ SPARC プロセッサでは，SunOS は現在のスレッドへのポインタを専用のレジスタに保持する．Linux は CPU ごとに 1 つのランキュー `struct rq` と，CPU ごとの現在のタスクへのポインタを保持する（`kernel/sched`）．SunOS の CPU 構造も同じ役割を果たす．

**表 5.2:** SunOS 5.0 におけるプロセスとスレッドのためのカーネルデータ構造．

| 構造 | 内容 |
|---|---|
| プロセス | カーネルレベルスレッドのリスト，仮想アドレス空間，ユーザの資格情報，シグナルハンドラ |
| LWP | ユーザレベルのレジスタ，システムコールの引数，資源使用情報，シグナルマスク．スワップ可能 |
| カーネルレベルスレッド | カーネルレベルのレジスタ，スタックポインタ，スケジューリング情報，結び付けられた LWP・プロセス・CPU へのポインタ．スワップ不可 |
| CPU | 現在のスレッド，アイドルスレッド，ディスパッチと割り込み処理の情報 |

## 5.3 ライブラリとカーネルの相互作用

**出典** P2L4，スレッド管理の基本的な相互作用，スレッド管理の可視性と設計；Stein と Shah [15]；Silberschatz ら ch. 4．

ユーザレベルライブラリはカーネルの中で何が起きているかを知らず，カーネルはライブラリの中で何が起きているかを知らない．システムコールと特別なシグナルが，2 つのレベルの協調を可能にする．アプリケーションは，自分のスレッドのうち何個が同時に実行されるべきか—`pthread_setconcurrency` で設定する**並行度**—をライブラリに伝え，ライブラリはシステムコールでカーネルにカーネルレベルスレッドを要求し，足りなくなればさらに要求する．カーネルは，カーネルレベルスレッドがブロックしたことなど，ライブラリのスレッドに影響する事象をライブラリに通知できる．（`pthread_setconcurrency` は Linux では何の効果も持たない．一対一のライブラリでは選ぶ余地がなく，POSIX も実装がこのヒントを無視することを許している．）

**例 5.5.** あるプロセスが 6 つのユーザレベルスレッドを持ち，そのうち同時に実行される必要があるのは高々 3 つであるとする．表 5.3 は，ライブラリとカーネルがカーネルレベルスレッドの数をどう調整するかをたどったものである．

**表 5.3:** 6 つのユーザレベルスレッド（U1–U6）を持つプロセスにおける，ユーザレベルライブラリとカーネルの協調．

| ステップ | 事象 | KLT |
|---|---|---|
| 1 | ライブラリがカーネルに 3 つのカーネルレベルスレッドを要求し，カーネルが K1–K3 を生成する． | K1–K3 |
| 2 | U1–U3 が K1–K3 上で実行され，U4–U6 はユーザレベルの条件で待つ． | 3 実行中 |
| 3 | U1 と U2 が端末またはソケットに対するブロッキング読み出し（無期限の待ち）を発行し，K1 と K2 がカーネル内でブロックする．その間に U4 が実行可能になる． | 2 ブロック，1 実行中 |
| 4 | U3 も同様の読み出しを発行するので，U4 を実行できるカーネルレベルスレッドがなくなり，プロセス全体が停止してしまう．そこでカーネルは，K3 がブロックするときにライブラリに通知する—SunOS 5.0 ではシグナル SIGWAITING を用いる．ライブラリはもう 1 つカーネルレベルスレッドを要求し，カーネルが K4 を生成して，その上で U4 が実行される． | 3 ブロック，1 実行中 |
| 5 | I/O が完了し，U1–U3 が実行を続ける．その後 K4 は長い間アイドルのままとなり，ライブラリはそれを破棄する． | K1–K3 |

### 5.3.1 スレッド管理の可視性

どちらのレベルも全体像の一部しか見ていない．
**カーネル** カーネルレベルスレッド，CPU，カーネルレベルスケジューラの判断が見える．
**ユーザレベルライブラリ** ユーザレベルスレッドと，自プロセスに割り当てられたカーネルレベルスレッドが見える．
ライブラリは，第 3 章のバウンドスレッドのように，ユーザレベルスレッドをカーネルレベルスレッドに結び付けることができ，カーネルはカーネルレベルスレッドを特定の CPU に固定することができる．どちらの判断も，もう一方のレベルからは見えない．

可視性の欠如は問題を引き起こす．あるユーザレベルスレッドがライブラリの実装するミューテックスを獲得し，その後カーネルが，例えばタイムスライスが尽きたために，そのスレッドが動いているカーネルレベルスレッドをプリエンプトしたとする．そのミューテックスを必要とする他のユーザレベルスレッドは，たとえカーネルが，それらがスケジュールされているカーネルレベルスレッドを実行していても，先に進めない．プリエンプトされたカーネルレベルスレッドをカーネルが再び実行するまで何も変わらないが，ミューテックスが見えていないカーネルには，それをすぐに実行する理由がないからである．多対多モデルでは，ユーザレベルのスケジューリングの判断や，ユーザレベルスレッドのカーネルレベルスレッドへの対応付けの変更は，ライブラリにしか見えない．一対一モデルはこれらの問題の多くを避ける．カーネルがすべてのスレッドを見るからである．[1]

ライブラリのスケジューラはプロセスのアドレス空間にある通常のコードであるから，実行がライブラリに入ったときにしか動かない．

- ユーザレベルスレッドが明示的に実行権を譲る．
- ライブラリが設定したタイマが満了する．
- ユーザレベルスレッドが，ミューテックスのロックやアンロックなど，どのスレッドが実行可能かを変えうるライブラリ関数を呼ぶ．
- ブロックしていたユーザレベルスレッドが実行可能になる．
- 例えばタイマのために，カーネルからシグナルが届く．

## 5.4 実装上の問題

マルチプロセッサはライブラリにさらなる問題を突きつけ，スレッド生成のコストはスレッドの再利用を促す．

### 5.4.1 複数 CPU 上のユーザレベルスレッド

マルチプロセッサでは，ライブラリは複数のカーネルレベルスレッド上で同時に実行され，ある CPU で下した判断が別の CPU での行動を必要とすることがある．T1，T2，T3

**注意** 講義と論文の違い：講義はブロッキング I/O 一般について述べる．SunOS が SIGWAITING を送るのは，プロセスのすべての LWP が端末やソケットからの読み出しのような無期限の待ちにあるときだけであり，ディスクの読み出しは含まれない．またライブラリがこれを捕捉するのは，LWP より多くのスレッドを持つ間だけである．

POSIX ではバウンドスレッドは PTHREAD_SCOPE_SYSTEM であり，NPTL はこのスコープにしか対応しない．カーネルレベルスレッドを CPU に固定するのは第 7 章のキャッシュアフィニティである．

[1] 多対多モデルに対する一般的な対策は，実行中のスレッド数を変える事象をカーネルがすべてライブラリに知らせることである．Anderson ら [16] はこのアップコールをスケジューラアクティベーションと呼んだ．スレッドがブロックし，ブロックを解かれ，あるいはプリエンプトされるたびに，カーネルは新しいアクティベーションをライブラリに渡し，ライブラリは直ちに再スケジュールできる．SIGWAITING は同じ考え方の限定された形である．NetBSD と FreeBSD はアクティベーションを実装したが，のちに削除した．多対多モデル自体と同様に，得られるものより高くついたのである．

**出典** P2L4，複数 CPU における問題，同期に関する問題，スレッドの破棄；Eykholt ら [14]；OSTEP ch. 28.

を，優先度が T3 > T2 > T1 であり，ライブラリが 2 つの CPU 上の 2 つのカーネルレベルスレッド K1 と K2 で実行するユーザレベルスレッドとする（図 5.4）．T2 は CPU 1 でミューテックス m を保持している．T3 は CPU 2 で実行され，m をロックしようとしてブロックし，ライブラリは K2 に T1 をスケジュールする．T2 が m をアンロックすると，CPU 1 で実行されているライブラリは，T3 が再び実行可能になり，実行中の T1 より優先度が高いことを知る．T1 をプリエンプトしなければならないが，CPU 1 上のライブラリのコードは CPU 2 のレジスタを変更できない．そこでライブラリは K2 にシグナルを送る．このシグナルによって K2 は CPU 2 でライブラリを実行し，ライブラリが T1 をプリエンプトして T3 をスケジュールする．

シグナルは割り込みとして CPU 2 に届く．K2 がそこで実行中であれば，カーネルはその CPU にプロセッサ間割り込みを送り，その CPU はカーネルに入ってシグナルを配送する（Linux では `kick_process`）．

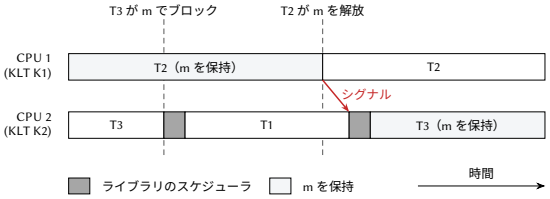



**図 5.4:** 2 つの CPU 上の，優先度が T3 > T2 > T1 であるユーザレベルスレッド．T2 が m をアンロックすると，CPU 1 のライブラリは K2 にシグナルを送り，CPU 2 のライブラリが T1 をプリエンプトして T3 を実行する．

### 5.4.2 適応型ミューテックス

マルチプロセッサは，ミューテックスを待つ最良の方法も変える．CPU 1 であるスレッドがミューテックスを保持しているときに，CPU 2 で実行されている別のスレッドがそれをロックしようとしたとする．通常，2 番目のスレッドはミューテックスの待ちキューに置かれてブロックし，そのために今 1 回，起こされるときにもう 1 回のコンテキストスイッチが必要になる．クリティカルセクションが短く，その所有者が別の CPU で実行中であれば，これらのコンテキストスイッチが完了するより前に所有者がミューテックスを解放する可能性が高く，2 番目のスレッドは**スピン**する方が–自分の CPU にとどまり，ミューテックスが空いたかどうかを繰り返し調べる方が–安上がりである．所有者が実行中でなければ，所有者が再びスケジュールされるまでミューテックスは解放されえないので，スピンは無意味である．

**ヒント** 待ち時間の見込みがコンテキストスイッチ 2 回分より短いときだけスピンし，そうでなければブロックせよ．

▷ adaptive mutex

**定義 5.6** (適応型ミューテックス). **適応型ミューテックス**とは，そのロック操作が，ミューテックスの所有者が別の CPU で実行中の間はスピンし，そうでなければ呼び出したスレッドをブロックするミューテックスである．

適応型ミューテックスが意味を持つのはマルチプロセッサに限られ–単一の CPU では，別のスレッドがスピンしている間に所有者が実行中であることはありえない–かつ短いクリティカルセクションに限られる．長いクリティカルセクションにはブロックの方が適する．スピンとブロックのどちらを選ぶかを決めるには，ロック操作が所有者の実行中かどうかを知らなければならないので，ミューテックスのデータ構造はその所有者を記録する．スピンロックの基礎としてのスピンは第 10 章で改めて扱う．

OSTEP では，しばらくスピンしてから眠るロックを 2 相ロックと呼んでいる．Linux カーネルのミューテックスも，ここと同じく所有者が実行中の間だけスピンする．

### 5.4.3 スレッドの破棄

スレッドの生成は，そのデータ構造とスタックを割り当てて初期化することであり，破棄はそれらを解放することである．アプリケーションはスレッドを頻繁に生成・破棄するので，これらの構造を再利用することには利点がある．デタッチされたスレッドが終了しても，その構造はすぐには解放されず，スレッドは**デスロー**（death row）と呼ばれるリストに置かれる．（デタッチされていないスレッドは，別のスレッドがそれをジョインするまでゾンビのリストで待つ．）**リーパスレッド**は，デスロー上のスレッドを破棄してそのデータ構造を解放する．リーパスレッドが実行されるのは，ライブラリにアイドルな LWP があるとき，またはデスローが上限に達したときである．解放されたスタックはキャッシュに入り，ライブラリは新しいスレッドのスタックをそこから取り出すので，それを割り当てて初期化するコストが節約される．

glibc も NPTL のスレッドに対して同じことを行う．解放されたスタックは最大 40 MiB のキャッシュ（チューナブル `glibc.pthread.stack_cache_size`）に残り，次の `pthread_create` に渡される．

▷ reaper thread

**注意** 講義と論文の違い：講義では新しいスレッドが，リーパの実行前にまだデスロー上にあるスレッドの構造を再利用する．Stein と Shah では，再利用はリーパが満たすスタックキャッシュを通じて行われる．

**要点**
カーネルとユーザレベルライブラリは，それぞれ自分のスレッドしか見ない．システムコールとシグナルによって両者は協調できるが，ロックの所有者をプリエンプトする，あるいは別の CPU 上のスレッドをプリエンプトするといった判断には，レベルをまたぐ明示的な通知が必要である．複数の CPU はスレッドの待ち方も変え–適応型ミューテックスは所有者が実行中の間だけスピンする–終了したスレッドは直ちに解放されるのではなく再利用のために保持される．

## 5.5 割り込みとシグナル

出典 P2L4，割り込みとシグナルの導入；OSTEP ch. 6；Kerrisk [11] ch. 20.

スレッドを通常の実行から逸らす事象には 2 種類ある．割り込みとシグナルである．シグナルは第 1 章で，オペレーティングシステムがアプリケーションに渡す通知として導入した．割り込みは，ハードウェアが CPU に届ける，これに対応する通知である．

▷ interrupt

**定義 5.7** (割り込み). **割り込み**とは，I/O デバイス，タイマ，他の CPU といった構成要素によって CPU の外部で生成され，何らかの条件が対応を要することを CPU に通知する事象である．

どの割り込みが起こりうるかは物理的なプラットフォームで決まり，割り込みは CPU が何を実行しているかとは無関係に非同期に現れる．一方シグナルは，CPU とその上で動くソフトウェアが引き起こす事象である．どのシグナルが存在するかはオペレーティングシステムが決め，シグナルは同期的にも非同期的にも現れうる．とはいえ，2 つの機構には共通点が多い（表 5.4）．どちらも，ハードウェアまたはオペレーティングシステムが定めた一意な識別子を持ち，どちらもマスクできる，すなわち無効化・抑止できる．そして有効であれば，どちらもハンドラを起動する．割り込みハンドラはオペレーティングシステムがシステム全体に対して設定し，シグナルハンドラは各プロセスが自分のために設定する．

**表 5.4:** 割り込みとシグナル．

| | 割り込み | シグナル |
|---|---|---|
| 生成元 | デバイス，タイマ，他の CPU | CPU とその上で動くソフトウェア |
| 定める主体 | ハードウェアプラットフォーム | オペレーティングシステム |
| 発生 | 非同期 | 同期または非同期 |
| マスク | CPU ごと | 実行コンテキストごと |
| ハンドラの設定 | オペレーティングシステムがシステム全体に対して | 各プロセスが自分のために |

## 5.6 割り込みとシグナルの処理

出典 P2L4，割り込みの処理，シグナルの処理；OSTEP ch. 6，36；Kerrisk ch. 20–21.

デバイスは対応を必要とするとき，CPU との間をつなぐ PCI Express などのインターコネクトを通じて CPU に割り込みを送る．初期のシステムは割り込みごとに専用の物理線を用いていたが，現代のデバイスはインターコネクト自体を通じてメッセージとして割り込みを送り，メッセージの内容が割り込みを，したがってデバイスを識別する．専用の線を駆動する代わりにこのようなメッセージを送って発生させる割り込みを，**メッセージシグナル割り込み**（MSI）と呼ぶ．

▷ message signaled interrupt

**定義 5.8** (割り込みハンドラ). **割り込みハンドラ**とは，特定の割り込みに応じてオペレーティングシステムが実行するルーチンである．**割り込みハンドラテーブル**は，各割り込み番号をそのハンドラの先頭アドレスに対応付ける．

▷ interrupt handler
▷ interrupt handler table

CPU は割り込み番号で割り込みハンドラテーブルを引き，そこで見つけた先頭アドレスをプログラムカウンタに設定してハンドラを実行する（図 5.5a）．どの割り込みが起こるかはハードウェアに固有であり，それらをどう処理するかはオペレーティングシステムに固有である．

この表は割り込みベクタテーブルとも呼ばれ，x86 では割り込みディスクリプタテーブルである．

シグナルはオペレーティングシステムが生成する．例えばスレッドが自分のプロセスに割り当てられていないメモリにアクセスすると，オペレーティングシステムは SIGSEGV（Linux ではシグナル 11）を発生させる．オペレーティングシステムはプロセスごとに，各シグナル番号を取るべき動作に対応付ける**シグナルハンドラテーブル**を保持する（図 5.5b）．シグナルの集合はオペレーティングシステムに固有であり，POSIX は標準的な集合を定

Linux は 31 個の標準シグナルと，ひとまとまりのリアルタイムシグナルを定める．glibc は後者の最初の 2 つを NPTL のために予約するので，SIGRTMIN は 34，SIGRTMAX は 64 である（signal(7)）．

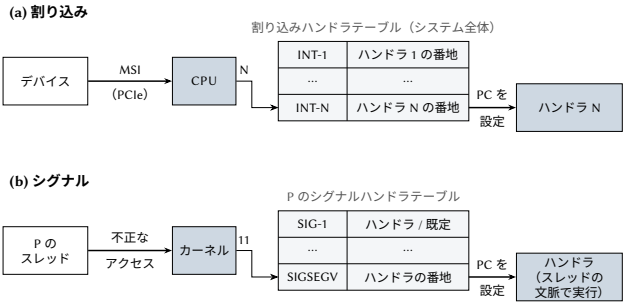



**図 5.5:** (a) デバイスが発生させた割り込みは，システム全体の割り込みハンドラテーブルからハンドラを選ぶ．(b) 不正なメモリアクセスによりカーネルが SIGSEGV を発生させ，これがプロセスのシグナルハンドラテーブルからハンドラを選ぶ．

めている．すべてのシグナルには既定の動作があり，それはプロセスの終了，シグナルの無視，プロセスの終了とコアダンプ，プロセスの停止，停止したプロセスの再開のいずれかである．

▷ signal handler

**定義 5.9** (シグナルハンドラ)．**シグナルハンドラ**とは，特定のシグナルが配送されたときに，それを受け取るスレッドの文脈で実行されるようプロセスが設定する関数である．

プロセスはハンドラを signal で，あるいはより移植性の高い sigaction で設定する．ほとんどのシグナルはこの方法で捕捉できるが，いくつかはできない．特に SIGKILL と SIGSTOP には，常に既定の動作が適用される．

**例 5.10.** signal.h と unistd.h を用いて，タイマが満了したときに作業ループを止めるプログラム．

**注意** ハンドラが代入してよいのは volatile sig_atomic_t のオブジェクトだけであり，呼んでよいのは signal-safety(7) に列挙された関数だけである．printf や malloc はそこに含まれない．

```
static volatile sig_atomic_t expired = 0;

static void on_alarm(int signo) {
    (void)signo;
    expired = 1;            /* ハンドラは最小限に保つ */
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = on_alarm;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGALRM, &sa, NULL);
    alarm(2);               /* 2 秒後に SIGALRM */
    while (!expired)
        do_work();
    return 0;
}
```

シグナルは，それを受け取るスレッドとの関係によって分類される．

▷ synchronous signal

▷ asynchronous signal

**定義 5.11** (同期シグナルと非同期シグナル)．**同期シグナル**とは，特定のスレッドの動作に応じて発生し，そのスレッドに配送されるシグナルである．保護されたメモリへのアクセスに対する SIGSEGV や，ゼロ除算に対する SIGFPE がその例である．**非同期シグナル**とは，受け取るスレッドが何を実行しているかとは無関係に発生するシグナルである．他のプロセスが kill で送る SIGKILL や，タイマの満了による SIGALRM がその例である．

シグナルがユーザ空間に届くのは，受け取るスレッドがカーネルから戻るときだけである（第 1 章）．カーネルはそのような復帰のたびに，ブロックされていない保留シグナルを調べる．

シグナルは，例えば pthread_kill で特定のスレッドに明示的に送ることもできる．その場合，同期シグナルと同様に，そのスレッドにのみ配送される．

## 5.7 割り込みとシグナルの無効化

割り込みハンドラとシグナルハンドラは，割り込まれたスレッドの文脈で実行される．そのスレッドのプログラムカウンタがハンドラに設定され，ハンドラはスケジューラを介さずにそのスレッドの代わりに実行される．このときシグナルハンドラはスレッド自身のスタックを，割り込みハンドラはカーネルスタックを用いる．このことは，ハンドラが共有状態にアクセスするときに問題を生む．割り込まれたスレッドがミューテックス m を保持していて，ハンドラが m をロックしようとすると，ハンドラはブロックする．m の所有者は割り込まれたスレッド自身であり，そのスレッドはハンドラが戻るまで先に進めない．こうしてスレッドは自分自身のハンドラとデッドロックする．

出典 P2L4，なぜ割り込みやシグナルを無効化するのか，シグナルマスクの詳細，マルチコアシステムにおける割り込み，シグナルの種類；Kerrisk ch. 20–22.

これを避ける方法は 2 つある．1 つは，ハンドラのコードを単純に保ち，決してミューテックスをロックしないようにすることであるが，これはしばしば制限が強すぎる．もう 1 つは，ハンドラが必要とするミューテックスを保持している間は該当する割り込みやシグナルを無効化することで，ハンドラがいつ実行されうるかをコードの側で制御することである．

**定義 5.12** (割り込みマスクとシグナルマスク). **割り込みマスク**および**シグナルマスク**とは，割り込みまたはシグナル 1 つにつき 1 ビットを持つビット列であり，そのビットの値が，その割り込みまたはシグナルが有効（1）か無効（0）かを示す．

▷ interrupt mask
▷ signal mask

クリティカルセクションの前後でシグナルを無効化するスレッドは，次の形をとる．

```
clear_field_in_mask(mask);  // 無効化
lock(mutex) {
  ...                       // ここではハンドラは動かない
} // unlock
reset_field_in_mask(mask);  // 有効化
```

無効化されている間に発生した割り込みやシグナルは失われない．それは**保留**のまま残り，マスクが再びそれを有効にすると直ちにそのハンドラが実行される．無効化されている間に同じシグナルが複数回発生しても，通常ハンドラは 1 回しか実行されない．シグナルを複数回発生させても，ハンドラが複数回実行される保証はない（5.7.3 節）．

**注意** POSIX のシグナル集合はブロックされるシグナルを列挙する．集合の要素であるシグナルは無効であり，上のビットの規約とは逆である．

### 5.7.1 マスクの設定

割り込みマスクは CPU ごとに保持される．ある CPU のマスクが割り込みを無効にしていれば，ハードウェアの割り込み配送機構はその割り込みをその CPU に届けない．カーネルはハードウェアに書き込むことでマスクを設定する．シグナルマスクは実行コンテキストごとに保持される．カーネルは，マスクがそのシグナルを無効にしているカーネルレベルスレッドには割り込まない．そして，ユーザレベルスレッドがカーネルレベルスレッドの上で実行される場合，カーネルにはカーネルレベルのマスクしか見えない．これについては 5.9 節で改めて扱う．シングルスレッドのプロセスは `sigprocmask` で，マルチスレッドプロセスのスレッドは `pthread_sigmask` でシグナルマスクを変更する．どちらも `SIG_BLOCK`，`SIG_UNBLOCK`，`SIG_SETMASK` のいずれかの操作を取る．

**例 5.13.** 上の形を POSIX の呼び出しで書くと，同じミューテックスをロックする `SIGALRM` のハンドラに対して次のようになる．

```
sigset_t set, old;
sigemptyset(&set);
sigaddset(&set, SIGALRM);
pthread_sigmask(SIG_BLOCK, &set, &old);     // 無効化
pthread_mutex_lock(&m);
/* ... クリティカルセクション ... */
pthread_mutex_unlock(&m);
pthread_sigmask(SIG_SETMASK, &old, NULL);  // 復元
```

**注意** `pthread_mutex_lock` は非同期シグナル安全ではない．ハンドラはフラグを立てるにとどめ，それを実行しうるすべてのスレッドがそのシグナルをブロックしなければならない．

### 5.7.2 マルチコアシステムにおける割り込み

マルチコアシステムでは，割り込みはそれを有効にしているどの CPU にも届けられうる．ある割り込みを 1 つのコアだけで有効にし，そのコアだけが処理するようにすれば，他のすべてのコアでその処理が引き起こすオーバーヘッドと乱れを避けられる．[2]

[2] Linux では `/proc/interrupts` が割り込み源ごとの回数を CPU ごとに 1 列ずつ示し，`/proc/irq/N/smp_affinity` は割り込み $N$ を配送してよい CPU のマスクを保持する．`irqbalance` デーモンが動いていれば，それがこれらのマスクを調整する．

**要点**
ハンドラは割り込まれたスレッドの文脈で実行される．ハンドラが必要とするミューテックスを保持するコードは，それを保持している間，対応する割り込みまたはシグナルを無効化しなければならない．その間に発生した事象は，再び有効になるまで保留のまま残る．

### 5.7.3 シグナルの種類

シグナルは，ハンドラが実行される前に同じシグナルが複数回発生したときに何が起こるかで異なる．

▷ one-shot signal

**ワンショットシグナル** **ワンショットシグナル**では，保留中の $n$ 回分の発生は 1 回分と等価である．ハンドラは少なくとも 1 回は実行されることが保証されるが，$n$ 回実行されるとは限らない．システムによっては，ハンドラは実行後に既定の動作に戻されるので，プロセスは再びそのシグナルを捕捉するために明示的に設定し直さなければならない．

▷ real-time signal

**リアルタイムシグナル** **リアルタイムシグナル**では，個々の発生がキューに入れられる．シグナルが $n$ 回発生すれば，ハンドラは $n$ 回呼ばれる．POSIX はこれらのために SIGRTMIN から SIGRTMAX までの範囲を予約している．

**注意** キューに入るかどうかは呼び出しではなくシグナル番号で決まる．リアルタイムシグナルは `kill` で送ってもキューに入り，標準シグナルの保留は決して 1 個を超えない．SA_SIGINFO を伴う sigqueue はデータ値を付け加えるだけである（signal(7)）．

**例 5.14.** あるスレッドが，標準のシグナルである SIGUSR1 と，リアルタイムシグナルである SIGRTMIN の両方をブロックする．ブロックされている間に各シグナルが 3 回発生し，その後スレッドはそれらのブロックを解除する．SIGUSR1 のハンドラは，3 回分の発生が 1 個の保留シグナルにまとまるので 1 回実行され，SIGRTMIN のハンドラは 3 回実行される．

## 5.8 スレッドとしての割り込み

**出典** P2L4，スレッドとしての割り込み，割り込みをスレッドとする方式の性能；Eykholt ら [14]．

SunOS 5.0 は，スレッドとそのハンドラの間のデッドロックを別の方法で避けている．割り込みをスレッドとして処理するのである．割り込みハンドラが独自のスレッドとして実行されるなら，それは他のスレッドと同様にミューテックスでブロックできる．所有者—それは割り込まれたスレッドかもしれない—がミューテックスを解放するまで待ち，その後スケジューラがそれを実行する．割り込みごとに新しいスレッドを生成するのはコストが高すぎるので，割り込みスレッドはあらかじめ生成・初期化しておき，そのうちの 1 つが完全なスレッドになるかどうかは実行時に決める．割り込みが起こると，そのようにあらかじめ割り当てられたスレッドのスタックに切り替わり，割り込まれたスレッドはその下にピン留めされたままとなって実行できない．ブロックしないハンドラはそこで終わり，ピン留めされたスレッドが再開する．ハンドラがミューテックスでブロックした場合に限り，割り込みスレッドは完全なスレッドとなり，スケジューラはそれを他のスレッドと同様に扱って，ピン留めされていたスレッドは直ちに実行を続ける．

Linux の遅延実行機構のうち，ブロックしてよいのはワークキューとスレッド化ハンドラだけである．

これに関連する設計として，割り込み処理の作業を 2 つの部分に分ける方法があるが，SunOS 5.0 はこれを採用しなかった（図 5.6）．Eykholt らがこれを退けたのは，デバイスドライバの再構成を必要とし，常に完全なコンテキストスイッチのコストがかかるからである．これに対して割り込みスレッドは，そのハンドラがブロックしたときにだけ完全なスレッドになる．

▷ top half

▷ bottom half

**定義 5.15** (トップハーフとボトムハーフ). 割り込み処理の**トップハーフ**は，割り込みが発生すると直ちに実行される部分であり，最小限の処理だけを行い，高速でブロックしないものでなければならない．**ボトムハーフ**は遅延される部分であり，任意の複雑さを持ちうる．スレッドがそれを実行する場合，それは他のスレッドと同様にスケジュールされ，ブロックしてもよい．

割り込みをスレッドとして処理することには代償がある．SunOS 5.0 では，割り込みごとにおよそ 40 個の SPARC 命令が加わる．一方で他のところでは作業を節約する．ミューテックスを必要とするハンドラがブロックできるようになったので，ミューテックスの操作は，ミューテックスを保持している間ハンドラが実行されないようにするために割り込みマスクを変更する必要がなくなり，ミューテックス操作 1 回当たりおよそ 12 命令が節約される．割り込み 1 回当たり $c$ 命令が加わり，ミューテックス操作 1 回当たり $s$ 命令が節約されるとき，この方式が引き合うのは

$$c N_{\text{int}} < s N_{\text{mutex}} \iff \frac{N_{\text{int}}}{N_{\text{mutex}}} < \frac{s}{c}, \tag{5.1}$$

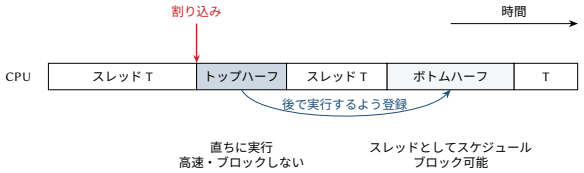



**図 5.6:** トップハーフとボトムハーフに分けた割り込み処理．トップハーフはスレッド T に割り込み，ボトムハーフはスケジューラが後で実行する．

のときである．ここで $N_{\text{int}}$ と $N_{\text{mutex}}$ は，割り込みとミューテックス操作の回数である．このモデルは固定的なコストだけを数え，ハンドラがめったにブロックしないことを仮定している．割り込みスレッドを完全なスレッドに変換するにははるかに大きなコスト，すなわち SPARC のレジスタウィンドウのフラッシュがかかり，Eykholt らも，割り込みがあまり頻繁にブロックしない限りにおいてのみ正味の利得を主張している．ミューテックスのロックとアンロックは割り込みの発生よりはるかに頻繁なので，収支は正になる．

Linux も同じ取引をしている．`threadirqs` を指定して起動するとハンドラはスレッドで実行され，Linux 6.12 で本流に入った PREEMPT_RT では，IRQF_NO_THREAD の付かない，CPU ごとでないハンドラはすべてこれが既定になる．

**例 5.16.** あるシステムが毎秒 2 000 回の割り込みを処理し，150 000 回のミューテックス操作を行うとする．$c = 40$，$s = 12$ とすると，割り込みスレッドは毎秒 $40 \times 2\,000 = 80\,000$ 命令を費やし，$12 \times 150\,000 = 1\,800\,000$ 命令を節約するので，差し引き毎秒 1 720 000 命令の節約になる．式 (5.1) により，この方式が損になるのは，ミューテックス操作当たりの割り込みが $12/40 = 0.3$ 回を超えるとき，すなわちこのミューテックス頻度では毎秒 45 000 回を超える割り込みがあるときだけである．

> **要点**
> 頻出ケースを最適化せよ．スレッドとしての割り込みは，稀な事象である割り込みをわずかに高価にする代わりに，頻繁な事象であるミューテックス操作を安価にする．

## 5.9 スレッドとシグナル処理

**出典** P2L4，スレッドとシグナル処理，ケース 1–4；Stein と Shah [15]．

多対多の実装では，シグナルマスクは 2 つのレベルに存在する．ユーザレベルライブラリはユーザレベルスレッドごとのマスクをそのスレッドデータ構造に保持し，カーネルはカーネルレベルスレッドごとのマスクをその LWP に保持する．ユーザレベルスレッドは自分のマスクを，システムコールなしにライブラリを通じて変更する．カーネルにはカーネルレベルスレッドのマスクしか見えない．シグナルが到着すると，カーネルは，その上で現在実行中のユーザレベルスレッドのマスクがどうであれ，マスクがそのシグナルを有効にしているカーネルレベルスレッドに割り込む．

NPTL では 2 つのレベルが一致するので，この食い違いは生じない．`pthread_sigmask` はシステムコール（`rt_sigprocmask`）であり，そのスレッドに対するカーネルのマスクを直接設定する．

SunOS のライブラリは，プロセスのシグナルハンドラテーブルに，それらのシグナルのハンドラとして自分のルーチンを設定することで，この食い違いを解消する．このルーチンが最初に実行され，ユーザレベルのマスクを調べて，適切なユーザレベルスレッドでアプリケーションのハンドラを呼び出す．生じるケースは 4 つある（図 5.7）．

**ケース 1** シグナルが，カーネルレベルスレッドでも，その上で実行中のユーザレベルスレッドでも有効である．カーネルはカーネルレベルスレッドに割り込み，ライブラリのルーチンは現在のユーザレベルスレッドでシグナルが有効であることを見つけて，そこでハンドラを実行する．

**ケース 2** 実行中のユーザレベルスレッドではシグナルが無効だが，実行可能で実行中ではない別のユーザレベルスレッドでは有効である．カーネルは，マスクがシグナルを有効にしているカーネルレベルスレッドに割り込む．ライブラリのルーチンは現在のスレッドでシグナルがマスクされていることを知り，実行可能なそのスレッドをこのカーネルレベルスレッドにスケジュールする．ハンドラはその中で実行される．

**ケース 3** シグナルが有効なユーザレベルスレッドが，別のカーネルレベルスレッド上で実行されている．割り込まれたカーネルレベルスレッド上のライブラリのルーチンは，そのスレッドがどこで実行されているかを知っているので，もう一方のカーネルレベルスレッドを宛先に**指定した**シグナルを送るシステムコールを行う．カーネルはそのカーネルレベルスレッドに割り込み，そこでライブラリのルーチンはシグナルが有効であることを見つけてハンドラを実行する．

**ケース 4** シグナルを有効にしているユーザレベルスレッドは 1 つもないが，カーネルレベルスレッドのマスクはまだそれを有効にしている．割り込まれたカーネルレベルスレッド上のライブラリのルーチンは，そのカーネルレベルスレッドのマスクでシ

グナルを無効にするシステムコールを行い，プロセスに対してシグナルを再送出する．カーネルはそれを，まだ有効にしている別のカーネルレベルスレッドに配送し，そこでも同じ手順が続く．これは，すべてのカーネルレベルスレッドでシグナルが無効になり，シグナルが保留のまま残るまで繰り返される．後にあるユーザレベルスレッドがそのシグナルを有効にすると，ライブラリはシステムコールを行ってカーネルレベルスレッドでも再び有効にし，保留されていたシグナルが配送される．

**ヒント** 4 つのケースは 1 つの規則から生じる．ユーザレベルのマスクがシグナルを有効にしているところでハンドラを実行し，どのスレッドも有効にしていなければ，カーネルレベルのマスクを落として再送出する．

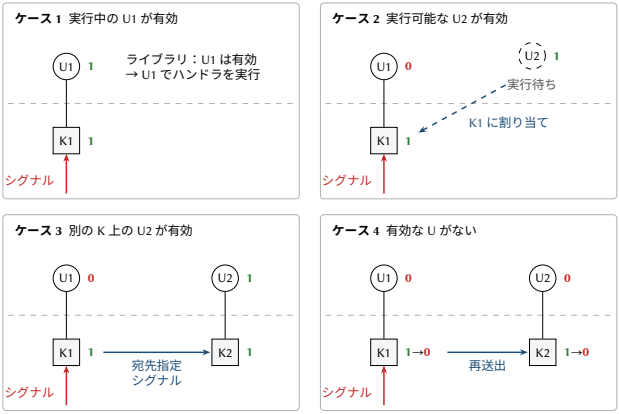



**図 5.7:** ユーザレベルスレッド（U）とカーネルレベルスレッド（K）を用いたシグナル処理の 4 つのケース．各スレッドの脇の数字は，そのシグナルに対するマスクのビットであり，1 が有効，0 が無効を表す．

この設計は，スレッドとしての割り込みと同じ原理に従う．シグナルの発生は，スレッドがシグナルマスクを変更する頻度–典型的にはクリティカルセクションごとである–よりはるかに低い．ユーザレベルのマスクの変更は安価なライブラリ操作であるのに対し，カーネルレベルのマスクをそれに合わせて保つには，変更のたびにシステムコールが必要になる．その代わりにシグナルの配送は高価になる–ライブラリのルーチンが最初に実行され，シグナルを転送したりシステムコールを行ったりしなければならないことがある–が，これは稀にしか起こらない．

## 5.10 Linux におけるタスク

**出典** P2L4，Linux におけるタスク；Silberschatz ら ch. 4；Kerrisk ch. 28．

Linux カーネルはプロセスとスレッドを区別しない．どちらも 1 種類の実行コンテキストから作られ，マルチスレッドプロセスは資源を共有するそうしたコンテキストの集まりである．

▷ task

**定義 5.17** (タスク). Linux カーネルにおける**タスク**とは，基本的な実行コンテキストであり，`struct task_struct` で表されるカーネルレベルスレッドである．シングルスレッドプロセスは 1 つのタスクからなり，マルチスレッドプロセスは複数のタスクからなる．

この構造体の主なフィールドは次のとおりである．

```
struct task_struct {        // 抜粋，Linux 3.x
    // ...
    pid_t pid;              // タスクの識別子
    pid_t tgid;             // スレッドグループの識別子
    int prio;
    volatile long state;
    struct mm_struct *mm;          // アドレス空間
    struct files_struct *files;    // 開いたファイル
    struct list_head tasks;
    int on_cpu;
    cpumask_t cpus_allowed;
    // ...
};
```

すべてのタスクは独自の識別子 pid を持つ．1 つのプロセスのタスクはスレッドグループを形成し，その識別子 tgid はグループの先頭タスクの pid である．これが getpid の返すプロセス識別子である．タスク構造体はプロセスの資源そのものを含まない．mm と files は，アドレス空間と開いたファイルのために別に割り当てられた構造体を指す．5.1 節と同様に，状態は複数のタスクが共有できる構造に分けられている．

**注意** pid という名前のフィールドはスレッドの識別子である．getpid は tgid を返し，gettid が pid を返す．ps -L はこの 2 つを PID と LWP として表示する．

▷ clone

タスクは **clone** システムコールで生成される．

```
clone(function, stack_ptr, sharing_flags, args);
```

新しいタスクはスタック stack_ptr 上で function(args) を実行する．フラグ sharing_flags の各ビットは，状態の一部を生成元のタスクと共有するか複製するかを選ぶ（表 5.5）．これらのフラグをすべて立て，さらに新しいタスクを生成元のスレッドグループに入れる CLONE_THREAD を立てると，新しいタスクは生成したプロセスのスレッドになる．すべてを落とすと新しいプロセスになり，fork はこのように実装されている．

**表 5.5:** clone の共有フラグ．

| フラグ | 立てた場合 | 落とした場合 |
|---|---|---|
| CLONE_FILES | ファイルディスクリプタ表を共有 | ファイルディスクリプタ表を複製 |
| CLONE_FS | ファイルシステム情報を共有 | ファイルシステム情報を複製 |
| CLONE_SIGHAND | シグナルハンドラテーブルを共有 | シグナルハンドラテーブルを複製 |
| CLONE_VM | アドレス空間を共有 | 新しいアドレス空間（複製） |

現在の Linux システムにおける POSIX スレッドは，**NPTL**（Native POSIX Thread Library）によって実装されている．その設計 [9] はいくつかの点に基づく．

▷ Native POSIX Thread Library

- 一対一モデル．各ユーザレベルスレッドが 1 つのタスクであるので，カーネルはすべてのスレッドを，そのシグナルマスクなどの情報とともに見ることができ，5.3 節と 5.9 節の協調の問題は生じない．
- futex（第 4 章）の上に築かれた同期．カーネルに入るのは待つときと起こすときだけである．
- カーネル内に実装された POSIX のシグナルの意味論．プロセスに送られたシグナルは，それをブロックしていないスレッドの 1 つに届き，致命的なシグナルはプロセス全体を終了させ，停止と再開のシグナルはそのすべてのスレッドに作用する．
- マネージャスレッドの廃止．clone が拡張され，スレッドは他の助けなしに開始・終了できる．スレッドの終了時にカーネルはユーザメモリ上のスレッド識別子を消去し，それに対する futex を起こす．pthread_join はこれを待つ．
- スケーラビリティ．当時のカーネルスケジューラはスレッドの数によらず一定時間で動作し，スレッドローカルストレージに対する新しいカーネルの支援によって，LinuxThreads が IA-32 で抱えていたプロセス当たり 8192 スレッドという上限がなくなった．

かつて一対一モデルに反対する論拠となっていた欠点は薄れた．カーネルへのトラップははるかに安価になり，メモリとスレッド識別子の範囲は多数のカーネルレベルスレッドに十分なほど大きい．NPTL は LinuxThreads を置き換えた．LinuxThreads も 1 スレッドにつき 1 タスクを生成していたが，シグナル処理などの点で POSIX に準拠していなかった．同じ時期に開発された多対多のライブラリである Next Generation POSIX Threads（NGPT）は 5.3 節の協調の問題を抱えており，NPTL のために放棄された．

Drepper と Molnar のベンチマークでは，NPTL は LinuxThreads や NGPT より数倍速くスレッドを生成・破棄する．第 3 章の 15 分対 2 秒という数字は，作り直されたカーネルの終了処理によるものである．

LinuxThreads は多対多と説明されることがあるが，当時の多対多ライブラリは NGPT である．

### 要点 — 章のまとめ

スレッドの状態は，すべてのスレッドが共有するハードプロセス状態，カーネルレベルスレッドごとのライトプロセス状態，そしてレッドゾーンを伴うユーザレベルの構造に分けられる．SunOS 5.0 では LWP が 2 つのレベルをつなぎ，両者はシステムコールとシグナルで協調する．ハンドラは割り込まれたスレッドの文脈で実行されるので，マスクがデッドロックを防ぐ．スレッドとしての割り込み，トップハーフとボトムハーフ，ユーザレベルのシグナルマスクは，いずれも頻出ケースを最適化する．Linux はすべてのスレッドを clone で生成されるタスクとし，NPTL がそれを一対一に対応付ける．

第 6 章

# スレッド性能上の考慮事項

| | |
|---|---|
| 講義動画 | P2L5 |
| 教科書 | OSTEP [1] ch. 26，33；Silberschatz ら [2] ch. 4–5；Kerrisk [11] ch. 49–50，63；Pai ら [17] |
| キーワード | 性能指標，実行時間，スループット，応答時間，マルチプロセスモデル，マルチスレッドモデル，イベント駆動モデル，イベントディスパッチャ，非同期 I/O，ヘルパ，AMPED，Flash，ワークロード，ベースライン |

第 3 章から第 5 章ではスレッド，そのプログラミングインタフェース，およびその実装を導入し，マルチスレッド化がアプリケーションをより速く，より効率的にすると論じた．この章では，そのような主張をどのように評価するのかを問う．どの設計が優れているかは性能指標とワークロードに依存することを示し，ウェブサーバのマルチプロセス，マルチスレッド，イベント駆動の各アーキテクチャを比較し，実験の設計の事例研究として Pai ら [17] の Flash ウェブサーバの評価をたどる．

## 6.1 スレッドモデルの比較

**出典** P2L5，どのスレッドモデルが優れているか．

第 3 章では，ボス・ワーカーパターンとパイプラインパターンを，一群の要求を処理し終えるのに要する総時間で比較した．これは妥当な基準のうちの 1 つにすぎず，別の基準を取れば判定は逆転しうる．5 つのスレッドを持つサーバが $n$ 個の要求の一群を受け取り，各要求の処理に 125 ms を要するとする．

- ボス・ワーカー構成では，1 つのスレッドがボスであり，その要求当たりの時間は無視でき，$w = 4$ 個のスレッドがワーカーで，そのそれぞれが $t = 125\,\text{ms}$ で 1 つの要求全体を処理する．
- パイプライン構成では，$k = 5$ 個のスレッドのそれぞれがステージであり，どのステージも $s = 25\,\text{ms}$ を要する．

ボス・ワーカーでは要求は $w$ 個ずつ完了するので，要求 $i$ は時刻 $\lceil i/w \rceil t$ に完了する．パイプラインでは要求 $i$ は $(k + i - 1)s$ に完了する．これらの完了時刻から 2 つの指標が導かれる．一群全体を処理し終える時間は，最後の要求の完了時刻である．1 つの要求の**平均完了時間**は

**注意** $n$ 個の要求はすべて時刻 0 に存在しており，完了はそこから数えている．到着が時間的に散らばる場合に意味を持つ指標は応答時間（6.3 節）である．

$$\overline{T}_{\text{bw}}(n) = \frac{t}{n} \sum_{i=1}^{n} \left\lceil \frac{i}{w} \right\rceil, \qquad \overline{T}_{\text{pl}}(n) = k\,s + \frac{(n-1)\,s}{2} \tag{6.1}$$

である．

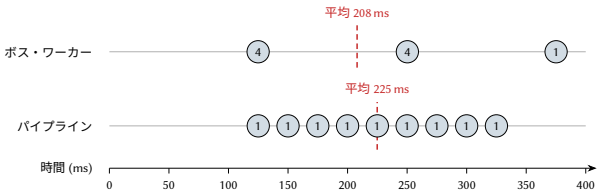



**図 6.1:** 9 個の要求の完了時刻．各円はその時刻に完了する要求の数を示す．4 つのワーカーを持つボス・ワーカーは 125 ms ごとに 4 個の要求を完了し，5 段のパイプラインは最初の 125 ms の後は 25 ms ごとに 1 個の要求を完了する．破線は平均完了時間を示す．

**例 6.1.** $n = 9$ 個の要求の一群（図 6.1）では，ボス・ワーカーは 125 ms に 4 個，250 ms に 4 個，最後の 1 個を 375 ms に完了する．パイプラインは最初の要求を 125 ms に完了し，その後は 25 ms ごとに 1 個ずつ，最後の 1 個を 325 ms に完了する．一群を処理し終えるのはパイプラインの方が早い．しかし 式 (6.1) によれば，平均完了時間はボス・ワーカーが $(4 \cdot 125 + 4 \cdot 250 + 375)/9 \approx 208.3\,\text{ms}$，パイプラインが $125 + 8 \cdot 25/2 = 225\,\text{ms}$ であり，平均的な要求に応じるのはボス・ワーカーの方が早い．$n = 40$ では両方の指標でパイプラインが勝つ．一群を処理し終えるのが 1250 ms ではなく 1100 ms，平均が 687.5 ms ではなく 612.5 ms である．

ボス・ワーカーは $t$ の後に $w$ 個の要求をまとめて完了するので，多くの要求が早く終わる．一方パイプラインは 1 ステージ時間当たり 1 個の要求しか送り出さない．しかし定常状態では，パイプラインが毎秒 $1/s = 40$ 個の要求を完了するのに対しボス・ワーカーは $w/t = 32$ 個であり，大きな一群ではこの高い完了率が支配的になる．したがってどちらのモデルが優れているかは，指標にも要求の数にも依存する．

## 6.2 スレッドの有用性

出典 P2L5，スレッドは有用か；OSTEP ch. 26.

これまでの章では，スレッドを使う理由がいくつか挙げられた．

- **並列化**：複数の CPU 上では，同じタスクの一部分ずつを担当するスレッドがそのタスクを高速化する．
- **特化**：特定のタスクに専念するスレッドはホットキャッシュで動作し，異なる優先度を与えることもできる．
- **効率**：スレッドはアドレス空間を共有するので，プロセスより必要なメモリが少なく，通信と同期も安価である．
- **アイドル時間の隠蔽**：単一の CPU 上でも，あるスレッドが I/O を待つ間に別のスレッドが実行できる．

これらの利点がスレッドに基づく設計を有用にするかどうかは，何を測るかに依存し，立場が異なれば測るものも異なる．行列を乗算するアプリケーションにとって意味のある尺度は，その実行時間である．ウェブサービスにとっては，単位時間当たりに処理するクライアントの要求数と，クライアントが応答を待つ時間であり，後者は平均，最大，あるいは 95 パーセンタイルのようなパーセンタイルとして報告されうる．ハードウェアの所有者にとっては，CPU などの資源の使用率である．したがって，要求に応じるあらゆるシステムについて，3 つの尺度が繰り返し現れる．単位時間当たりに完了する要求の数，1 つの要求に応答するまでの時間，そしてシステムの能力のうちどれだけが使われているか，である．

ヒント スループットと平均応答時間の積はシステム内にある要求の数であり，6.9 節のリトルの法則である．測定した組を手早く確かめられる．

**例 6.2.** あるサーバが 200 個の要求を処理し，そのうち 180 個は 20 ms 後に，20 個は 400 ms 後に応答を受け取るとする．平均応答時間は $(180 \cdot 20 + 20 \cdot 400)/200 = 58$ ms である．95 パーセンタイルとは，少なくとも 95 % の要求がそれを超えない最小の値であり，昇順に並べたときの 190 番目の応答時間は 400 ms なので，95 パーセンタイルは 400 ms である．平均は，10 個に 1 個の要求が大多数の 20 倍待つという事実を覆い隠す．

注意 パーセンタイルは最も遅い要求の平均ではない．95 パーセンタイルは最悪の 5 % をまったく考慮しない．

## 6.3 性能指標

出典 P2L5，性能指標，スレッドは本当に有用か；Silberschatz ら ch. 5.

ある設計が別の設計より良い性能を示すという主張は，指標を定めて初めて意味を持つ．

**定義 6.3 (性能指標).** システムの測定基準，すなわち対象とするシステムの，測定可能で定量化できる性質であって，その振る舞いを評価する–例えば別の実装と比較する–ために使えるものを**性能指標**という．

▷ performance metric

指標は理想的には，実際のマシンに配備された実際のソフトウェアを実際のワークロードの下で実験して得られる．これはしばしば実行不可能である．マシンが利用できないかもしれず，ソフトウェアが完成していないかもしれず，実際の利用者に実験のために待つよう求めることもできない．そこで測定は代わりに，現実的な状況を代表するように設計された小規模な実験や，シミュレーションから得られる．このように，実験を行って指標を得るために用意された環境を**テストベッド**という．

▷ testbed

最もよく使われる指標は次のものである．

**実行時間** タスクの**実行時間** とは，その開始から完了までの時間である．行列乗算のようなバッチ計算にとって自然な指標である．

▷ execution time

**スループット** システムの**スループット**とは，単位時間当たりに完了するタスク，例えば要求の数である．これは**要求レート**，すなわち単位時間当たりに到着する要求の数とは異なる．要求レートに追いつけないシステムでも，要求を完了する速さはそのスループットにとどまり，未処理の要求は溜まっていく．

▷ throughput

**応答時間** 要求の**応答時間**とは，その発行からクライアントが応答を受け取るまでの時間であり，処理される前に要求が待つ時間も含む．多数の要求については，平均，最大，あるいはパーセンタイルで要約される．

▷ response time

**待ち時間** タスクの**待ち時間**とは，その投入から実行が始まるまでの時間である．利用者が，ジョブがいつ終わるかよりいつ始まるかを気にする場合に重要になる．

▷ wait time

**CPU 使用率** **CPU 使用率**とは，CPU が動いている時間の割合である．他の資源の使用率も同じように定義される．

▷ CPU utilization

▷ service level agreement

さらに多くの指標が，性能をその費用や特定の立場の関心に結び付ける（表 6.1）．サービスの提供者はしばしば，**SLA**（サービスレベル合意）で性能目標を約束する．例えば，要求の 99 % が 200 ms 以内に応答を受け取る，といったものである．そのとき，合意に違反した要求の割合がそれ自体 1 つの指標となる．

**表 6.1:** 性能指標と，典型的にそれに関心を持つ立場．

| 指標 | 測るもの | 関心を持つ立場 |
|---|---|---|
| 実行時間 | タスクを完了するまでの時間 | バッチジョブの利用者 |
| スループット，要求レート | 単位時間当たりに完了するタスク，到着する要求 | 運用者 |
| 応答時間 | 要求に応答するまでの時間 | クライアント |
| 待ち時間 | タスクが開始するまでの時間 | 利用者 |
| CPU 使用率 | CPU が動いている時間の割合 | 運用者 |
| プラットフォーム効率 | 使用した資源に対するスループット | データセンタの運用者 |
| ドル当たり・ワット当たり性能 | 費用や電力に対する性能 | 運用者，購入者 |
| SLA 違反 | 合意した目標を外れた要求の割合 | 提供者，クライアント |
| クライアント体感性能 | クライアントから見た品質．例えば動画のフレームレート | クライアント |
| 総合性能 | 多数のタスクやクライアントにわたって平均した指標 | 運用者 |
| 平均資源使用量 | 例えば時間にわたって使用されたメモリ | 運用者 |

### 6.3.1 指標とワークロードへの依存

したがって，スレッドが有用かという問いに単一の答えはない．答えは指標に依存する．6.1 節が平均完了時間と総完了時間について示したとおりである．そして答えはワークロード（6.9 節）にも依存する．2 つのスレッドモデルのどちらが優れているかは，要求の数とともに変わった．同じことが，オペレーティングシステムのほとんどの設計上の判断について成り立つ．グラフの種類が違えば有利な最短経路アルゴリズムも違い，ファイルアクセスの型が違えば有利なファイルシステムの設計も違い，要求の負荷が違えば有利なサーバのアーキテクチャも違う．

> **要点**
> 一般に最良の設計というものはない．比較が意味を持つのは指標とワークロードを明示した場合に限られ，2 つの設計のどちらが優れているかは，たいていその両方に依存する．

## 6.4 マルチプロセスサーバとマルチスレッドサーバ

**出典** P2L5，マルチプロセス対マルチスレッド；Silberschatz ら ch. 4．

この章の残りでは，並行サーバを構成する方法を，静的なファイルを提供するウェブサーバを例に比較する．クライアントが要求を送り，サーバは図 6.2 の各ステップでそれを処理し，ファイルを送って応答する．要求の解析や応答ヘッダの計算のように，CPU 時間だけを必要とするステップもある．他のステップはブロックしうる．接続の受け付けはクライアントを待ち，要求の読み込みとデータの送信はネットワークを待ち，ファイルの検索と読み込みはディスクを待つ場合がある．

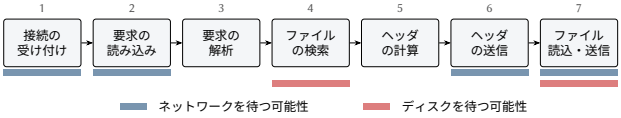



**図 6.2:** 静的な内容を提供する単純なウェブサーバの処理ステップ．ステップの下の帯は，そのステップがどの資源を待つ可能性があるかを示す．

これらのステップを一度に 1 つの要求について実行するサーバは，1 つのクライアントにしか応じられず，自らがブロックしている間でさえ他のすべてのクライアントを待たせる．最も単純な対策は，そのコピーを複数実行することである．

**定義 6.4** (マルチプロセスモデル). **マルチプロセスモデル**（MP）では，サーバは複数のプロセスからなり，各プロセスは，一度に 1 つの要求についてすべてのステップを実行する単一スレッドのサーバを動かす（図 6.3a）．

▷ multi-process model

マルチプロセスモデルは，各プロセスが普通の逐次プログラムであるため，プログラムしやすい．その代償はアドレス空間が分かれていることから生じる．多数のプロセスは多くのメモリを必要とし，その分ファイルをキャッシュするメモリが減る．プロセス間の切り替えは高価である．ファイルデータや応答ヘッダのキャッシュのように，プロセスが共有すべき状態は，プロセス間通信か明示的な共有メモリを必要とするため，維持するのが難しく高価である．最後に，すべてのプロセスが同じポートで接続を受け付けなければならず，その準備にはいくらかの注意を要する．典型的には，親プロセスが待ち受けソケットを作ってからサーバプロセスを fork し，各サーバプロセスはそのソケットを継承して，いずれもそれに対して accept を呼ぶ．

**定義 6.5** (マルチスレッドモデル). **マルチスレッドモデル**（MT）では，サーバは 1 つのプロセスであり，1 つのアドレス空間を共有するそのスレッドが，それぞれ並行に要求を処理する（図 6.3b）．

▷ multithreaded model

スレッドの構成方法はいくつかある．すべてのスレッドが自分で接続を受け付け，すべてのステップを実行してもよい．あるいは第 3 章のボス・ワーカーパターンのように，ボススレッドが接続を受け付けてワーカーに渡してもよい．あるいは各ステップをパイプラインのステージとしてもよい．どの構成でもスレッドはアドレス空間を共有するので，キャッシュのような共有状態に直接アクセスでき，メモリ使用量は小さく，スレッド間のコンテキストスイッチは安価である．その代わり実装は単純ではない．共有状態は同期を必要とし，それに伴うオーバーヘッドと誤りの危険を伴う．このモデルはまた，オペレーティングシステムがカーネルレベルスレッドに対応していることに依存する．これらのアーキテクチャが最初に比較された当時は普遍的ではなかったが，今日では問題にならない．

**注意** ここでいう「マルチスレッド」はこの章の（Pai らの）サーバモデルであり，第 3 章の一対一モデルや多対一モデルといった対応付けのモデルではない．

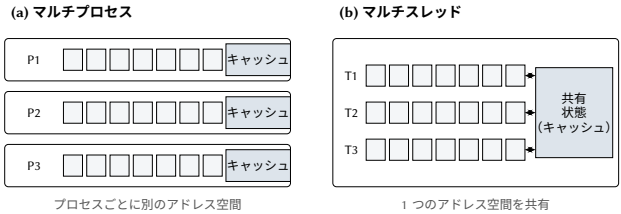



**図 6.3:** (a) マルチプロセスモデル．各プロセスが 1 つの要求のすべての処理ステップ（マス）を実行し，キャッシュのような状態を自分の複製として保持する．(b) マルチスレッドモデル．1 つのアドレス空間の中のスレッドが各ステップを実行し，状態を共有する．

## 6.5 イベント駆動モデル

第 3 のアーキテクチャは，複数の実行コンテキストを用いずに並行性を実現する．

**出典** P2L5，イベント駆動モデル；OSTEP ch. 33；Kerrisk [11] ch. 63.

**定義 6.6** (イベント駆動モデル). 1 つのアドレス空間の中で単一の制御の流れを持つ単一のプロセスとして動作し，イベントに応じて仕事を行うアプリケーションは，**イベント駆動モデル**に従う．

▷ event-driven model

このようなアプリケーションの中心にあるのが**イベントディスパッチャ**である．これは，イベントを繰り返し待ち，イベントに応じて，それに登録されたハンドラを 1 つ以上呼び出すループである（図 6.4）．ウェブサーバの場合，イベントには接続要求の到着，接続上での要求データの到着，送信の完了（その接続がさらにデータを受け取れることを意味する），そしてディスク読み込みの完了が含まれる．Pai らは，このように構築されたサーバを**単一プロセスイベント駆動モデル**（SPED）の一例と呼ぶ．

▷ event dispatcher

▷ single-process event-driven model

**イベントハンドラ**とは，1 つのイベントに応じて実行されるコードであり，その呼び出しは単にそのコードを呼ぶことにすぎない．ハンドラは最後まで実行される．ブロックすることになる場合，ハンドラは待たない．ブロッキング操作を開始し，要求がどこまで進んだかを記録して，制御をディスパッチャに返す．操作の完了は後に別のイベントとして届く．ディスパッチャとハンドラは合わせて状態機械を形成する．スレッドであ

▷ event handler

**注意** 最後まで実行するとは，ブロックする呼び出しを一切含まないということである．`getaddrinfo` による名前解決のようにライブラリ関数に隠れたものも，1 つではなく全部の接続を止める．

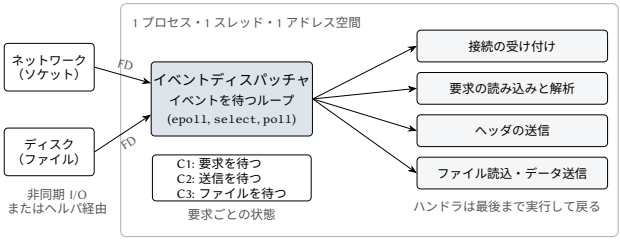



**図 6.4:** イベント駆動のウェブサーバ．ディスパッチャはファイルディスクリプタ（FD）上のイベント—準備の整ったソケット，および非同期 I/O やヘルパ（6.6 節）が完了を通知するディスク読み込み—を待ち，イベントごとにそのハンドラを呼び出す．処理中の各要求の状態は，明示的なレコードに保持される．

ればプログラムカウンタとスタックに暗黙のうちに保持する情報が，ここでは要求ごとのレコードに明示的に保持される．

OSTEP はこの明示的な管理を手動スタック管理と呼んでいる．

### 6.5.1 並行実行

単一のスレッドしか持たないにもかかわらず，イベント駆動のサーバは多数の要求を並行に処理する．処理中の各要求は，その処理のいずれかのステップにある．ディスパッチャは 1 つの要求の 1 ステップを実行し，それから次のイベントに移る．そのイベントは別の要求のものかもしれず，こうして要求の処理は互いに織り込まれる．表 6.2 は 3 つの接続をたどったものである．例えばステップ 6 の後には，1 つのスレッドが 3 つの要求を処理中で，それぞれ異なるステップにある．この追跡では，ファイル読み込みの完了がイベントとして届くことを仮定しており，そのためには非同期 I/O か 6.6 節のヘルパが必要である．ファイルの検索もディスクを待つ場合がある．

**表 6.2:** 3 つの接続を持つイベント駆動サーバの追跡．状態は，R が要求待ち，F がファイルデータ待ち，S が送信可能になるのを待つ状態，— が未接続を表す．

| ステップ | イベント | ハンドラの動作 | C1 | C2 | C3 |
|---|---|---|---|---|---|
| 1 | 接続要求 | C1 を受け付ける | R | — | — |
| 2 | 接続要求 | C2 を受け付ける | R | R | — |
| 3 | C1 上の要求 | 解析，ファイル検索，読み込み開始 | F | R | — |
| 4 | 接続要求 | C3 を受け付ける | F | R | R |
| 5 | C2 上の要求 | 解析，ファイル検索，読み込み開始 | F | F | R |
| 6 | C1 のファイルデータ | ヘッダとデータを送信 | S | F | R |
| 7 | C3 上の要求 | 解析，ファイル検索，読み込み開始 | S | F | F |
| 8 | C1 がデータを受け取れる | 残りを送信して閉じる | 完了 | F | F |
| 9 | C2 のファイルデータ | ヘッダとデータを送信 | 完了 | S | F |

### 6.5.2 このモデルが機能する理由

第 3 章では，単一の CPU 上で，あるスレッドが I/O を待つ間に別のスレッドへ切り替えることが，アイドル時間が 2 回のコンテキストスイッチのコストを上回るとき，すなわち $t_{\text{idle}} > 2t_{\text{ctx}}$ のときに引き合うことを示した．アイドル時間がなければ，コンテキストスイッチは純粋なオーバーヘッドである．要求を処理できたはずの CPU サイクルを消費するからである．イベント駆動モデルは，このコストなしに利点を得る．要求が待たなければならなくなるまでその要求を処理し，それから別の要求を処理するのであって，コンテキストスイッチは介在しない．複数の CPU を持つマシンでは，このモデルを CPU ごとに 1 つ適用する．すなわち，それぞれが独自のディスパッチャを持つ複数のイベント駆動プロセスが，到着する接続を分担する．

この章がコンテキストスイッチのオーバーヘッドとして数える時分割を行う CPU スケジューラは，第 7 章で改めて扱う．その費用は第 2 章にある．1 回の切り替えの直接の仕事はおよそ 3.8 µs，キャッシュを詰め直す分まで含めると 1000 µs を超える [7]．

### 6.5.3 実装

ウェブサーバのイベントの発生源には，ファイルディスクリプタを通じてアクセスする．ネットワークにはソケット，ディスクにはファイル，そして以下で導入するヘルパにはパイプである．ソケットやパイプの場合，イベントとはそのディスクリプタが入力または出力の準備を整えたことである．通常のファイルの場合，準備が整っているということは，読み込みがディスクを待たなければならないかどうかについて何も語らない．この制限には 6.6 節で立ち返る．どのディスクリプタの準備が整っているかを知るために，ディスパッチャは 3 つのシステムコールのいずれかを使う．`select` と `poll` は，呼び出しのたびに関心のあるディスクリプタの集合全体を受け取る．カーネルはそのすべてを調べ，アプリケーションは結果を走査するが，準備が整っているのはたいていごく少数である．したがってそのコストは，開いている接続の数とともに増える．Linux で利用できる `epoll` は，このオーバーヘッドの大半を取り除く．ディスクリプタは一度登録すればよく，各呼び出しは準備の整ったものだけを返す．[1]

**例 6.7.** `epoll` に基づくディスパッチャの骨組み．各要求の状態は `struct conn` に保持する．これはディスクリプタ，バッファ，ファイル，到達したオフセットを保持し，`handler` に次のステップの関数へのポインタを保持する．

```
int ep = epoll_create1(0);   /* 登録は epoll_ctl */
for (;;) {                           /* ディスパッチャ */
    struct epoll_event ev[64];
    int n = epoll_wait(ep, ev, 64, -1);   /* 待つ */
    for (int i = 0; i < n; i++) {
        struct conn *c = ev[i].data.ptr;
        c->handler(c);           /* 最後まで実行される */
    }
}
```

待ち受けソケットのハンドラは接続を受け付け，要求を読むハンドラを持つ `struct conn` を確保し，新しいディスクリプタを登録する．どのハンドラも 1 つのステップを実行し，`handler` を次のステップに設定する．

このモデルの利点は，単一のアドレス空間と単一の制御の流れから生じる．処理中の 1 つの要求が要するのはその状態レコードだけであり，スタックを伴うスレッドではないので，必要なメモリは小さい．実行コンテキスト間のコンテキストスイッチは存在しない．そしてハンドラが並行に実行されることはないので，同期は不要である．

> **要点**
> イベント駆動のサーバは，多数の要求の処理を 1 つのスレッドの中で互いに織り込む．ディスパッチャがファイルディスクリプタ上のイベントを待ち，それぞれについてハンドラを実行する．どのハンドラもブロックしない限り，メモリ，コンテキストスイッチ，同期を節約できる．

## 6.6 ブロッキング操作とヘルパ

イベント駆動モデルには 1 つ重大な問題がある．ブロックするハンドラがプロセス全体をブロックしてしまうことである．単一のスレッドがディスクを待つ間，ディスパッチャは実行されず，他のどの要求も前に進まない．データを送る準備が整っている要求さえも進まない．準備完了の通知はディスク読み込みの役に立たない．`select` と `poll` は，通常のファイルを読むのにディスクを待たなければならない場合でも常に準備が整っていると報告し，`epoll` は通常のファイルをそもそも受け付けない．

**定義 6.8 (非同期 I/O).** **非同期 I/O** では，I/O 操作を開始するシステムコールがその操作の完了前に戻り，呼び出したスレッドは実行を続ける．カーネルはその呼び出しから，操作を遂行するのに必要な情報をすべて得て，結果をどこへ届ければよいかを知るか，あるいは後で結果をどこから取り出せばよいかを呼び出し側に伝える．

非同期 I/O はカーネルからの支援を必要とする．例えば，操作のブロックする部分を実行するカーネルレベルスレッドである．あるいはデバイスからの支援を必要とする．例えば，DMA（第 11 章）でデータを転送し，完了を割り込みで知らせるデバイスである．非同期 I/O はイベント駆動モデルによく適合する．非同期操作の完了は，ディスパッチャにとってもう 1 つのイベントにすぎないからである．

しかし非同期の呼び出しは常に利用できるわけではなく，存在する場合でもあらゆる

**注意** `select` には上限もある．Linux では `FD_SETSIZE` = 1024 未満のディスクリプタしか扱えない．`poll` にこの制限はなく，`epoll` の上限は `/proc/sys/fs/epoll/max_user_watches` である（select(2)，epoll(7)）．

[1] 1 台のマシンで 1 万の接続を扱うことは，Kegel が 1999 年にその手法をまとめたことにちなんで *C10K* 問題と呼ばれた．答えがこれらのスケーラブルなインタフェースであり，FreeBSD 4.1 の `kqueue`（2000 年）と Linux 2.5.44 の `epoll`（2002 年，一般に使えるようになったのは Linux 2.6）である．

`EPOLLET` を使うと `epoll` は状態が変わるたびに 1 度だけディスクリプタを報告するので，ハンドラは `EAGAIN` が返るまで読み書きしなければならない．

**出典** P2L5，非同期 I/O，ヘルパ；OSTEP ch. 33；Pai ら．

▹ asynchronous I/O

POSIX は `aio_read` と `aio_write` を定めており，glibc はこれらをスレッドで実装している．Linux でバッファ付きのファイル I/O まで扱えるのは `io_uring`（2019 年）である．

種類のデバイスについて対応しているとは限らない．そのときイベント駆動のサーバは，ブロッキング操作を委譲する．

▷ helper

**定義 6.9 (ヘルパ).** **ヘルパ**とは，イベント駆動のサーバが，自らのディスパッチャに代わってブロッキング操作を実行させるためだけに用いるスレッドまたはプロセスである．

ディスパッチャは操作をパイプでヘルパに渡し，ヘルパは完了を第 2 のパイプで報告する．パイプはデータを一方向にしか運ばないからである．ソケットの対なら両方向を一度に担える．パイプもソケットもファイルディスクリプタなので，ディスパッチャはこれらの報告を，他のイベントと同じように `select` や `poll` で待つ．ブロックするのはヘルパであって，ディスパッチャはブロックせず，したがって他のすべての要求も止まらない．イベント駆動のプロセスとヘルパプロセスを組み合わせたものが，**非対称マルチプロセスイベント駆動モデル**（AMPED）である．このモデルが非対称であるのは，MP モデルや MT モデルの同等なプロセスやスレッドとは違い，ヘルパが特定の操作だけを行うからである．Pai らがヘルパにプロセスを用いたのは，当時はすべてのオペレーティングシステムがカーネルレベルスレッドに対応していたわけではなかったからである．プロセスの代わりにヘルパスレッドを用いれば，サーバは**非対称マルチスレッドイベント駆動モデル**（AMTED）に従う．

▷ asymmetric multi-process event-driven model

▷ asymmetric multithreaded event-driven model

ヘルパは，基本のイベント駆動モデルの移植性の問題を解消する．サーバがカーネルの非同期 I/O 支援に依存しなくなるからである．またヘルパは，要求ごとにワーカーを置く場合よりもメモリ使用量を小さく保つ．ヘルパが必要なのは同時にブロックしている操作の分だけであり，接続ごとではないからである．他方でこの手法は，ある種のアプリケーションにしか適用できず，マルチプロセッサ上ではイベントの振り分けを複雑にする．ヘルパが報告する完了も含め，各イベントは対応する要求を扱っているディスパッチャに届かなければならない．

**例 6.10.** あるサーバが 2 000 個の接続を開いており，そのうち任意の時点でディスクを待っているのは高々 25 個であるとする．マルチスレッドモデルでは接続ごとにスレッドが必要であり，スタックとデータ構造にスレッド当たり 64 kB を要するとすれば，スレッドは 128 MB を使う．AMPED のサーバは，接続ごとに例えば 2 kB の状態レコード（4 MB）と，プロセス固有のページ，ページテーブル，カーネル構造に例えば 1 個当たり 1 MB を要する 25 個のヘルパプロセス（25 MB）を保持し，合計でおよそ 29 MB となる．その差をファイルデータのキャッシュに使えば，ディスクを待たなければならない要求の数が減る．

これは実際に使う分である．Pthreads のスタックはアドレス空間を 8 MiB 予約するが，ページを使うのは触れた分だけである（第 4 章）．

表 6.3 は 4 つのアーキテクチャをまとめたものである．

**表 6.3:** 並行ウェブサーバのアーキテクチャ．

| | MP | MT | SPED | AMPED |
|---|---|---|---|---|
| 実行コンテキスト | 並行する要求ごとに 1 プロセス | 並行する要求ごとに 1 スレッド | 1 つ | 1 つとヘルパ |
| 要求当たりのメモリ | アドレス空間 | スレッドのスタックと構造 | 状態レコード | 状態レコード |
| コンテキストスイッチ | プロセス間 | スレッド間 | なし | ブロッキング操作のときだけ |
| 同期 | プロセス間通信による | 共有状態のロック | なし | なし |
| ブロックするディスク I/O | 1 プロセスがブロック | 1 スレッドがブロック | サーバがブロック | 1 つのヘルパがブロック |

## 6.7 Flash ウェブサーバ

**出典** P2L5，Flash；Pai ら；Kerrisk ch. 49–50．

▷ Flash

Pai らは，AMPED モデルに基づくイベント駆動のウェブサーバを構築した．彼らのサーバ **Flash** では，ブロックしうるディスク操作をヘルパが行う．すなわち，ディレクトリの読み込みを伴う場合のある，要求のパス名からファイルへの対応付けと，ファイルデータの読み込みである．ヘルパはディスパッチャとパイプで通信する．Flash はファイルデータにメモリマップドファイル（第 9 章）を通じてアクセスするので，ファイルのページは最初にアクセスされたときにディスクから読まれる．

**ヒント** mincore はページ当たり 1 バイトで答える．x86-64 の 4 KiB ページなら，4 MiB の対応付けに必要なベクタは 1024 バイトである．

ハンドラがファイルデータを送る前に，ディスパッチャはシステムコール `mincore` を使って，そのデータのページがメモリ上にあるかどうかを調べる（図 6.5）．メモリ上にあれば，ハンドラは直ちにデータを送り，これは通常ブロックしない（判定と送信の間

にページが追い出される可能性は残る）．メモリ上になければ，ディスパッチャはその要求をパイプでヘルパに渡す．ヘルパはそれらのページにアクセスし，それによってページがメモリに読み込まれるが，その間ブロックしうる．そしてヘルパは完了をパイプでディスパッチャに報告する．ディスパッチャはその報告をイベントとして受け取り，ハンドラを呼び出す．ハンドラは今度はデータをメモリ上に見つける．ウェブサーバへの要求の大半は，データがすでにメモリ上にある人気のファイルに対するものであり，そうした要求では Flash はヘルパを使わずに済み，ヘルパとの通信のコストも避けられる．これは大きな節約になりうる．

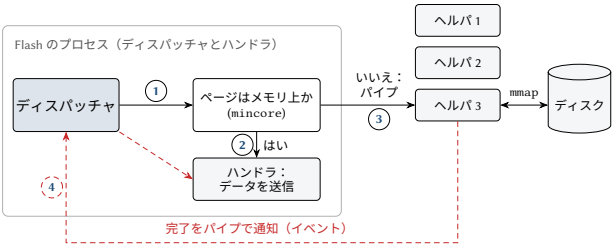



**図 6.5:** Flash におけるファイルデータの提供．(1) ディスパッチャが `mincore` でデータがメモリ上にあるかを調べる．(2) あれば，ハンドラが直接それを送る．(3) なければ，ヘルパにパイプでページへのアクセスを依頼し，ヘルパがディスクから読み込む．(4) その完了報告がイベントとなり，その後ハンドラがデータを送る．

**例 6.11.** `sys/mman.h`，`unistd.h`，`stdbool.h` を用いた C での判定．ファイルデータはページ境界に整列したアドレスに対応付けられているものとする．

```
static bool in_memory(void *addr, size_t len)
{
    size_t pg = (size_t)sysconf(_SC_PAGESIZE);
    size_t n = (len + pg - 1) / pg;
    unsigned char vec[n];           /* ページに1バイト */
    if (mincore(addr, len, vec) != 0)
        return false;
    for (size_t i = 0; i < n; i++)
        if (!(vec[i] & 1))          /* ページが非常駐 */
            return false;
    return true;
}

/* ディスパッチャ，ファイルデータを送る前 */
if (in_memory(c->map, c->len))
    send_data(c);                   /* ブロックしない */
else
    ask_helper(c);                  /* パイプ経由 */
```

### 6.7.1 追加の最適化

Flash はこのアーキテクチャに，要求当たりの作業量を減らす最適化を組み合わせている．

**アプリケーションレベルのキャッシュ** Flash はデータと計算結果の両方をキャッシュする．要求のパス名からファイルへの対応付け，ファイルだけで決まる応答ヘッダ，そして対応付けられたファイル自体である．パス名のキャッシュにヒットすれば，ヘルパへの往復の依頼も省ける．

**ギャザ I/O** 応答ヘッダとファイルデータは，1 回のベクタ I/O 操作 `writev` で送られる．これは複数のメモリ領域を一度に渡すのでシステムコールを 1 回節約する．それでもカーネルは各領域をユーザ空間から自身のソケットバッファへコピーする．スキャッタギャザ DMA を備えたネットワークインタフェースは，これらのカーネルバッファを 1 つにまとめることなく集めて転送できるが，そのコピーを避けるわけではない．

**整列** `writev` に渡された領域のカーネルへのコピーは，領域が整列していないと遅くなる．そこで Flash は，すべての応答ヘッダを 32 バイト境界に整列させ，可変長のヘッダフィールドを伸ばしてその長さを 32 バイトの倍数に揃える．これは対象と

Linux はさらに進んでいる．`sendfile`（2.2，1999 年）はファイルデータをユーザ空間を経由せずカーネル内でソケットへ複写する．

したマシンの 32 バイトのキャッシュラインに合わせたものであり，ヘッダに続くファイルデータも整列した位置でコピーされる．

**注意** この最適化は通常 *DMA* のための整列と呼ばれるが，カーネルバッファへのコピーを避けるのではなく，そのコピーを安価にするものである．

これらの最適化は，今日のウェブサーバでは一般的である．これらはサーバのアーキテクチャとは独立であり，この事実は 6.9 節の評価において重要になる．

## 6.8 Apache ウェブサーバ

▷ Apache

**出典** P2L5，Apache ウェブサーバ．

オープンソースの **Apache** HTTP サーバは，実運用のサーバがこれらのモデルをどう組み合わせるかを示している．そのコアがサーバの基本的な骨組みを提供し，接続を受け付けて実行コンテキストを管理する–Apache 2 では，この節の最後の注で述べるように，専用のモジュールを通じてそれを行う．要求を処理する機能はモジュールとして実装され，各モジュールは要求処理の段階であるフックにハンドラを登録する（図 6.6a）．要求が進むにつれて，コアは各段階に登録されたハンドラを呼び出す．この制御の流れはイベント駆動モデルに似ている．しかしイベントハンドラとは違い，これらのハンドラは要求に割り当てられたスレッド上で実行され，ブロックしうる．

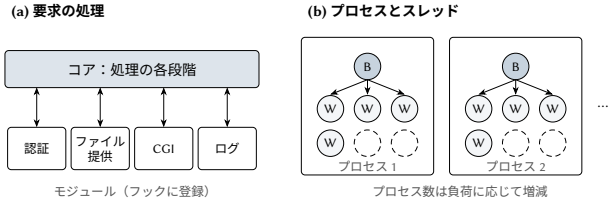



**図 6.6:** Apache ウェブサーバ．(a) モジュールは，コアが提供する要求処理の段階にハンドラを登録する．(b) 複数のプロセスがあり，各プロセスはボス（B）とワーカー（W）のプールからなる．破線の円は，講義のモデルで負荷が増えたときに追加されるワーカーである．一方 `worker` モジュールはプロセスごとに一定の数を保つ．

実行コンテキストは，マルチプロセスモデルとマルチスレッドモデルを組み合わせている（図 6.6b）．講義のモデルでは，Apache は複数のプロセスを実行し，各プロセスは動的なスレッドプールを持つボス・ワーカーのサーバである．各プールのスレッド数もプロセス数も，例えばアイドルなワーカーの数や未処理の接続の数によって，負荷に合わせて調整される．以下で述べる実際の `worker` モジュールは別の方法で規模を変える．各子プロセスは一定数のワーカースレッド（`ThreadsPerChild`）と 1 つのリスナースレッドを生成し，負荷には子プロセスを生成または終了させることで追従して，アイドルなスレッドの数を `MinSpareThreads` と `MaxSpareThreads` の間に保つ．

**注 6.12.** Apache 2 では，骨組み自体が 1 つのモジュールである．接続を受け付け，プロセスとスレッドを管理するのはマルチプロセッシングモジュールの仕事であり，これが実行コンテキストの構造を選ぶ．`prefork` はプロセスだけを使い，`worker` はプロセスと，その各々の中の一定数のスレッドを組み合わせ，`event` はアイドルな接続を監視するイベントループを加える．

Apache httpd 2.4 は Linux では `event` MPM を既定とし，`prefork` と `worker` も選択できる（Apache の文書「Multi-Processing Modules」）．

## 6.9 実験の方法論

**出典** P2L5，実験の方法論；Pai ら．

実験による比較は 3 つの問いに答えなければならない．どのシステムを比較するか，どの入力を用いるか，どの指標で測るかである．Flash の評価はそのそれぞれを例示している．

### 6.9.1 比較の対象

Flash は，この章で論じたすべてのアーキテクチャのサーバと比較された（表 6.4）．MP，MT，SPED の各サーバは Flash のコードから作られており，アーキテクチャだけが Flash と異なり，最適化は共通である．したがってそれらの間の差は，アーキテクチャに帰することができる．2 つの独立したサーバは，当時使われていたサーバの中で Flash がどの位置にあるかを示す．

### 6.9.2 ワークロード

▷ workload

**定義 6.13** (ワークロード)**.** 実験の**ワークロード**とは，試験対象のシステムに与える入力の

表 6.4: Flash の評価で比較されたサーバ．

| サーバ | アーキテクチャ |
|---|---|
| MP | マルチプロセス．各プロセスは単一スレッドで，一度に 1 つの要求を扱う |
| MT | マルチスレッド，ボス・ワーカー |
| SPED | 単一プロセスイベント駆動，ヘルパなし |
| Zeus | バージョン 1.30，商用の SPED サーバ．トレースに基づく実験では 2 つのプロセスで実行 |
| Apache | バージョン 1.3.1，マルチプロセス |
| Flash | AMPED |

集合である．ウェブサーバであれば，到着時刻と要求されるファイルを伴う要求の列である．

ワークロードは現実的であるべきで，例えば実際のサーバが受けるウェブページ要求の時間分布を再現すべきである．また制御可能かつ再現可能であって，どのサーバも同じ入力で測定されるようにすべきである．**トレース**，すなわち実際のシステムで観測された要求の記録を実験で再生するものは，この両方の要件を満たす．**合成ワークロード**は，選んだパラメータからプログラムが生成するものであり，現実性は劣るが，各パラメータを系統的に変化させられる．Flash は，ライス大学のウェブサーバで収集された 2 つのトレースで評価された．大きなファイル集合を持つ計算機科学科の *CS* **トレース**と，学生の個人ページを提供するサーバの，より小さな集合を持つ *Owlnet* **トレース**である．合成ワークロードは，大きさを変えながら 1 つのファイルを繰り返し要求するものであった．

▷ trace
▷ synthetic workload

サーバマシンは 333 MHz の Pentium II で，メモリ 128 MB，100 Mbit/s のインタフェースを複数持ち，Solaris 2.6 と FreeBSD 2.2.6 で動作した [17]．

### 6.9.3 指標

ウェブサーバは，届けるデータと応じるクライアントによって，2 つの指標で測られる．

$$\begin{aligned} \text{帯域幅} &= \frac{\text{転送した総バイト数}}{\text{総時間}}, \\ \text{接続レート} &= \frac{\text{クライアント接続の総数}}{\text{総時間}} \end{aligned} \tag{6.2}$$

どちらもファイルサイズの関数として評価された．ファイルが大きいほど，サーバは接続の固定コストをより多くのバイトに分散でき，帯域幅は上がる．しかし同時に接続当たりの作業も増えるので，接続レートは下がる．これは簡単なモデルで定量的に表せる．大きさ $S$ のファイルを提供するのに，接続当たり固定の時間 $c$ とバイト当たり $b$ がかかるとすれば，一度に 1 つの接続を扱うサーバは

$$R(S) = \frac{1}{c + bS}, \qquad B(S) = S \cdot R(S) = \frac{S}{c + bS} < \frac{1}{b} \tag{6.3}$$

を達成する．ここで $R$ は接続レート，$B$ は帯域幅である．

ヒント リトルの法則が両者を結ぶ．レート $R$，平均応答時間 $T$ なら同時に開いている接続は $RT$ 個であり，毎秒 1000 接続で 100 ms なら 100 個である．

**例 6.14.** $c = 1$ ms，キロバイト当たり $b = 0.02$ ms とすると，10 kB のファイルには 1.2 ms かかり，$R \approx 833$ 接続毎秒，$B \approx 8.3$ MB/s となる．100 kB のファイルには 3 ms かかり，$R \approx 333$ 接続毎秒だが，$B \approx 33.3$ MB/s となる．ファイルが大きくなるにつれて $B$ は $1/b = 50$ MB/s に近づき，一方 $R$ は下がり続ける．

## 6.10 実験結果

評価の結果は何よりも，要求されたデータがメモリ上にあるかどうかに依存する．表 6.5 は以下で述べる知見をまとめたものである．

出典 P2L5，実験結果，性能結果のまとめ；Pai ら．

ヒント 表の各行は「ワークロード：勝者」と読み，そのワークロードを制約する資源は何かを問うとよい．キャッシュされたデータなら CPU，大きなデータ集合ならディスクである．

**単一ファイル（最良の場合）** 合成ワークロードでは，すべての要求が同じファイルに対するもので，そのファイルは常にキャッシュされている．帯域幅，すなわち $n$ 個の要求とファイルサイズの積を時間で割った値が，200 kB までのファイルサイズについて測定された．どのサーバも似た傾向を示す．最も性能が高いのは SPED である．Flash はそれに迫るが，ここでは決して必要にならない `mincore` の判定を要求ごとに行う分を支払う．FreeBSD 上では Zeus は 10–100 kB のファイルで異常を示し，Pai らはこれが上で述べた整列の問題に起因すると推測している．MT と MP は同期とコンテキストスイッチのために遅く，最適化を欠く Apache が最も遅い．

表 6.5: Flash の評価における性能結果のまとめ．

| ワークロード | 順位 | 理由 |
|---|---|---|
| データがキャッシュ上 | SPED > Flash | Flash はメモリ上にあるかの判定を不要に行う |
| | SPED，Flash > MT，MP | 同期とコンテキストスイッチのオーバーヘッド |
| ディスクを多用 | Flash > SPED<br>Flash > MT，MP | SPED は非同期 I/O を欠きブロックする<br>Flash はメモリ効率が高く，コンテキストスイッチも少ない |

**Owlnet トレース** データ集合は小さく，その大半がキャッシュに収まるので，傾向は最良の場合に似る．しかし時折，ディスクを必要とする要求がある．そのとき SPED は完全にブロックしてしまうが，Flash はヘルパのおかげで他の要求に応じ続けられる．

**CS トレース** データ集合はより大きく，多くの要求がディスク I/O を必要とする．Flash のコードから作られたサーバの中では，非同期 I/O を欠く SPED が最も性能が低い．MT は MP より良い．メモリ使用量が小さい分だけファイルのキャッシュに使えるメモリが増えてディスク I/O が減り，スレッド間で状態を共有する方が，プロセスに必要なプロセス間通信より安価だからである．最も性能が高いのは Flash である．メモリ使用量が小さいのでキャッシュに使えるメモリが増え，ブロッキング I/O を必要とする要求が減り，しかも同期を必要としない．Apache はここでも，測定されたすべてのサーバの中で最も遅い．

Pai らによれば，Flash は「幅広い実際のワークロードにわたって，既存のウェブサーバの性能に並ぶか，最大で 50 % 上回りうる」[17]．

**最適化** パス名，応答ヘッダ，対応付けたファイルの各キャッシュをさまざまに組み合わせて Flash を測定すると，どの最適化も相当に寄与していることが分かる．Apache もこれらから恩恵を受けたはずである．

> **要点**
> データがメモリ上にあるときはイベント駆動のサーバが勝ち，Flash の余分な判定がわずかな代償となる．ワークロードがディスクを必要とするときは，Flash のヘルパがブロックを避け，メモリ使用量が小さいのでキャッシュに使えるメモリが増え，ディスク I/O が減る．Flash は個々のワークロードで最良というより，ワークロード全体にわたって良い性能を示す．

## 6.11 実験の設計と実施

実験を行う価値があるのは，それが関連性を持つ場合に限られる．すなわち，他者が信じ，関心を持つ解決策についての主張を支えるものでなければならない．

出典 P2L5，実験の設計に関する助言，実験の実施に関する助言．

### 6.11.1 関連性と指標

関連性は関係者に依存する．ウェブサーバの場合，クライアントは応答時間を気にし，サーバの運用者はスループットと費用を気にする．[2] 応答時間とスループットの両方を改善する新しい設計は，明らかに価値がある．一方を改善し他方を悪化させない設計も，やはり改善である．他方を犠牲にして一方を改善する設計も，悪化した指標が許容範囲に留まるならば，一部の関係者には有用でありうる．要求レートが増えても応答時間を変えないことも，目標になりうる．このように目標が，どの指標を測るか，実験をどう構成するかを決める．

[2] クライアントに平均ではなくパーセンタイルを示すのは，最も遅い応答が全体を決めるからである．1 つの利用者要求が 100 台のサーバの応答を待ち，100 回に 1 回が 1 秒を超えるなら，その要求が遅くなる確率は $1 - 0.99^{100} \approx 63\,\%$ である．Dean と Barroso はこれを *tail at scale* と呼ぶ [18]．

指標はそれに応じて選ぶべきである．標準的な指標はより広い読者に届く．読者がその解釈と比較の仕方を知っているからである．同じ理由で，実験ではしばしば**ベンチマーク**，すなわちコミュニティが合意した，実施方法と結果の報告の規則を伴う標準的なワークロードが使われる．それに加えて，指標は，誰が結果に関心を持つのか，何に関心を持つのか，なぜかという問いに答えるものであるべきである．クライアントの性能であれば応答時間やタイムアウトした要求の数を，運用者の費用であればスループットと費用を求めることになる．

▷ benchmark

### 6.11.2 構成空間

システムの振る舞いは多くの要因に依存し，それらが合わさってその構成空間をなす．

- **システム資源**：CPU やメモリのようなハードウェアと，スレッド数やキューの大きさのようなソフトウェア．
- **ワークロード**：ウェブサーバであれば，要求レート，並行する要求の数，ファイルサイズ，アクセスパターン．

空間全体を探索することはできないので，実験では構成パラメータの部分集合を選び，変化させる各パラメータについて値の範囲を決め，関連性のあるワークロードを用いる．最良の場合と最悪の場合のシナリオは含めるべきである．要因の組み合わせは有用なものであるべきで，多くの組み合わせは同じ点を繰り返すだけである．

比較を律する規則が 2 つある．第 1 に，**同じ条件のもの同士**を比較することである．結果を比較する 2 つの構成は，調べている要因においてのみ異なっていなければならない．あるサーバを大きなマシン上の軽いワークロードで測り，別のサーバを小さなマシン上の重いワークロードで測ったのでは，サーバについて何も結論できない．差がワークロードから来るのか，資源から来るのか，サーバから来るのか分からないからである．第 2 に，**ベースライン** と比較することである．これは性能が理解されている参照システムであり，最先端のもの，最も一般的な手法，あるいは理想的な最良・最悪の場合のシナリオなどである．ある設計が優れているという主張は，それがベースラインより優れているという主張である．

▹ baseline

**注意** ベースラインは新しい設計と同じだけ丁寧に調整しなければならない．既定の設定のまま比較することが，改善を過大に見せる最もありがちなやり方である．

### 6.11.3 実験の実施

実験を設計したら，各試験事例を何度も実行し，実行にわたって指標を，例えば平均とばらつきとして計算し，結果を，主張が明確に伝わる形式–典型的にはグラフ–で表す．最後に，結論を導かなければならない．実験の目標に照らして解釈されない測定値は，どんな主張も支えない．

**例 6.15.** あるチームが，新しい AMPED のサーバが写真共有サイトの応答時間の 95 パーセンタイルを，スループットを下げずに短縮することを示したいとする．ハードウェアは 1 台のサーバマシンに固定し，ワークロードはそのサイトのトレースを，毎秒 1 000，2 000，4 000，8 000，16 000 要求と要求レートを上げながら再生したものとする．すべてのファイルがメモリ上にある最良の場合と，データ集合がメモリの 4 倍ある最悪の場合とが範囲を挟む．ベースラインは現在使われているマルチスレッドのサーバである．2 つのサーバ，5 つのレート，2 つの場合，それぞれ 5 回の繰り返しで，この研究は $2 \cdot 5 \cdot 2 \cdot 5 = 100$ 回の実行からなり，各構成についてパーセンタイルとスループットの平均を，そのばらつきとともに報告する．

**要点 — 章のまとめ**

どの設計が優れているかは，指標とワークロードに依存する．並行サーバはプロセス（MP），スレッド（MT），あるいは 1 つのイベント駆動のスレッド（SPED）を使える．非同期 I/O が利用できないとき，ヘルパはブロッキング操作をイベントに変える（AMPED，AMTED）．AMPED のサーバである Flash は，データがキャッシュされているときは SPED に迫り，ワークロードがディスクを必要とするときは他の選択肢に並ぶかそれを上回る．Apache はプロセスとスレッドのプールを組み合わせている．健全な実験は，関連性のある指標とワークロードを選び，調べている要因において異なる構成同士を比較し，ベースラインと比較し，実行を繰り返し，結論を導く．

第 7 章

# スケジューリング

| | |
|---|---|
| **講義動画** | P3L1 |
| **教科書** | OSTEP [1] ch. 7–10；Silberschatz ら [2] ch. 5；Fedorova ら [19] |
| **キーワード** | ランキュー，FCFS，SJF，優先度逆転，ラウンドロビン，タイムスライス，MLFQ，O(1) スケジューラ，CFS，キャッシュアフィニティ，NUMA，同時マルチスレッディング，ハードウェア性能カウンタ |

第 2 章では，レディキューから次に実行するプロセスを選ぶ CPU スケジューラを導入し，タイムスライスの長さがスケジューリングのオーバーヘッドと，実行可能なプロセスが待つ時間とを釣り合わせることを示した．この章では，その選択がどのように行われるかを調べる．単純な実行完了型の方針から，プリエンプティブなもの，優先度に基づくもの，タイムシェアリングを行うものに至るスケジューリングアルゴリズム，それらを実装するランキューのデータ構造，そして Linux カーネルの 2 つのスケジューラを扱う．続いて，マルチプロセッサとハイパースレッディング対応のプロセッサが提起する問題を，Fedorova ら [19] の論文に沿って論じる．アルゴリズムの比較には第 6 章の性能指標を用いる．

## 7.1 CPU スケジューリングの概観

**出典** P3L1，スケジューリングの概観と CPU スケジューラ；OSTEP ch. 7；Silberschatz ら ch. 5．

CPU スケジューラは，プロセスと，その中のスレッドが，共有された CPU にいつどのようにアクセスするかを決定する．ユーザレベルのプロセスやスレッドを実行するタスクも，カーネル自身のカーネルレベルスレッドも，ともにスケジュールする．第 5 章の Linux の用語にならい，この章ではそのようなスケジュール可能な実体をすべて**タスク**と呼ぶ．

最も単純なスケジューラでさえ 1 つの目標を体現している．3 つの初歩的な戦略が，ありうる目標の幅を示す．

- タスクを到着した順に直ちにディスパッチする．スケジューリングの判断は自明であり，そのオーバーヘッドは小さい．
- 単純で短いタスクを先にディスパッチする．これは早い時期に最も多くのタスクを完了させ，したがってワークロード全体より短いどの区間についても，単位時間当たりに完了するタスク数であるスループットを最大化する．
- 複雑で長いタスクを先にディスパッチする．これは資源を占有し続け，CPU，デバイス，メモリの使用率を最大化する．

スケジューラは，CPU を別のタスクに与えなければならない可能性があるときに必ず動作する．

- CPU がアイドルになったとき．例えば実行中のタスクが I/O を待ってブロックした場合である．
- 新たなタスクが実行可能になったとき．例えば新しく生成されたタスクや，I/O 操作が完了したタスクである．スケジューラは，その新しいタスクが実行中のタスクより重要であって，それに割り込むべきかどうかを判断しなければならない．
- 実行中のタスクのタイムスライスが満了したとき．

満了に気付くのはタイマのティックである．Linux カーネルは CONFIG_HZ を 100，250，300，1000 のいずれかで構築され，ティックは 10，4，3.3，1 ms ごとに入る．

そしてスケジューラは実行可能なタスクの 1 つを選んでディスパッチする．選ばれたタスクへのコンテキストスイッチを行い，ユーザモードに入り，プログラムカウンタをそのタスクの次の命令に設定する．タスクはそこから実行を続ける．

どのタスクが選ばれるかはスケジューリング方針，すなわちアルゴリズムが決める．選択がどのように実行されるかは，実行可能なタスクを保持するデータ構造に依存する．

**定義 7.1** (ランキュー)**.** スケジューラが実行可能なタスクを保持しておくデータ構造を，その**ランキュー**という．

Linux は主な方針を SCHED_OTHER（公平スケジューラ），SCHED_FIFO，SCHED_RR，SCHED_DEADLINE と名付けている（sched(7)）．

▷ runqueue

第 2 章のレディキューはそのようなランキューの 1 つである．名前に反して，ランキューは FIFO キューである必要はない．多くのスケジューラは木や複数のキューを用いる．ランキューとスケジューリングアルゴリズムは密接に結び付いている．アルゴリズムを効率的に実装できるのは，その選択を安価にするランキューがある場合に限られ，ランキューの側も，それが支えるアルゴリズムに合わせて設計される．

## 7.2 実行完了型スケジューリング

最初に考えるスケジューラは，タスクから CPU を決して取り上げない．これらを単純化した仮定の下で解析する．一群のタスクをスケジュールするものとし，各タスクの実行時間はあらかじめ分かっており，プリエンプションはなく，CPU は 1 つであるとする．アルゴリズムは，スループット，平均完了時間，平均待ち時間，CPU 使用率（第 6 章）によって比較する．タスクの完了時間はその到着から完了までで測り，待ち時間はその到着から実行が始まるまでで測る．

出典 P3L1，実行完了型スケジューリング，SJF の性能；OSTEP ch. 7；Silberschatz ら ch. 5．

**定義 7.2** (実行完了型スケジューリング). **実行完了型スケジューリング**では，いったんディスパッチされたタスクは完了するまで CPU 上で実行され，プリエンプトされることはない．

▷ run-to-completion scheduling

**定義 7.3** (FCFS). **先着順スケジューリング**（FCFS）に従うスケジューラは，タスクを到着した順に実行する．

▷ first-come first-served scheduling

FCFS のランキューは FIFO キューである．到着したタスクは末尾に追加され，スケジューラは先頭のタスクを取り出す．どちらの操作も定数時間で済む．

**定義 7.4** (SJF). **最短ジョブ優先**（SJF）という方針に従うスケジューラは，タスクを実行時間の短い順に実行する．

▷ shortest job first

SJF のランキューは実行時間で順序付けられていなければならない．順序付きキューを使えばスケジューラは先頭のタスクを定数時間で取り出せるが，タスクの挿入にはその位置の探索が必要で，キューの長さに比例する時間がかかる．実行時間で順序付けた平衡木ならタスクの挿入は対数時間で済み，スケジューラは最左の節点を取り出す．

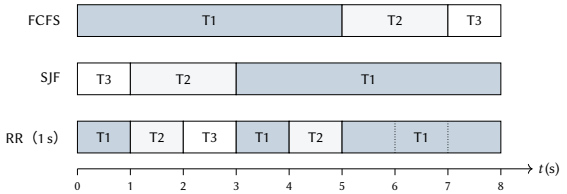



**図 7.1:** 時刻 0 に T1，T2，T3 の順で到着し，実行時間がそれぞれ 5 s，2 s，1 s である 3 つのタスクを，FCFS，SJF，およびタイムスライス 1 s のラウンドロビン（7.5 節）でスケジュールした場合．点線は同じタスクの連続するタイムスライスの区切りを示す．

**例 7.5.** 3 つのタスクが時刻 0 に T1，T2，T3 の順で到着し，実行時間はそれぞれ 5 s，2 s，1 s であるとする（図 7.1）．FCFS ではそれらは 5，7，8 s に完了する．スループットは毎秒 $3/8 = 0.375$ タスク，平均完了時間は $(5+7+8)/3 \approx 6.67$ s，平均待ち時間は $(0+5+7)/3 = 4$ s である．SJF は T3，T2，T1 の順に実行し，それらは 1，3，8 s に完了する．スループットは毎秒 0.375 タスクのままだが，平均完了時間は $(1+3+8)/3 = 4$ s に，平均待ち時間は $(0+1+3)/3 \approx 1.33$ s に下がる．

総仕事量はどちらの順序でも同じであるから，一群全体にわたって測ったスループットは変わらない．しかし短いタスクを先に実行することで，SJF は一群が終わる前のどの時点でもより多くのタスクを完了させており，2 つの平均をともに下げる．実際，同時に到着し，実行完了型でスケジュールされるタスクについては，これより平均待ち時間の小さい順序は存在しない．

**注意** 短いタスクが到着し続けると，SJF は長いタスクをスタベーションに陥らせうる．スタベーションには 7.3 節で立ち返る．

## 7.3 プリエンプティブスケジューリング

実際にはタスクは異なる時刻に到着し，到着したタスクの方が実行中のタスクより CPU を得るにふさわしいことがある．より短いかもしれず，より重要かもしれない．実行完了型のスケジューラは，実行中のタスクが完了するまで，それがどれほど長くかかろうとも待たせる．

出典 P3L1，プリエンプティブスケジューリング：SJF とプリエンプション，優先度；OSTEP ch. 7–8；Silberschatz ら ch. 5．

**定義 7.6** (プリエンプティブスケジューリング). **プリエンプティブスケジューリング**では，スケジューラは実行中のタスクを完了前に中断し，CPU を別のタスクに与えることができる．

▷ preemptive scheduling

この方針は最短残余時間優先とも呼ばれる．OSTEP はこれを最短完了時間優先（STCF）と呼ぶ．

Silberschatz らは指数平均 $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$ を用いる．保持すべきは $\tau_n$ だけである．$\alpha = 1/2$ とすれば，古い実行区間ほど重みが半分ずつになる．

SJF は次のようにしてプリエンプティブになる．タスクが到着するたびに，スケジューラはその実行時間と，実行中のタスクの残りの実行時間とを比較する．新しいタスクの方が短ければ，実行中のタスクはプリエンプトされてランキューに戻され，新しいタスクが実行される．

タスクの実行時間があらかじめ分かっていることはまれである．そこでスケジューラは，そのタスクの履歴から，最後に CPU 上にあったときの実行時間や，直近 $n$ 回の実行時間の窓平均といった経験則を用いて，それを推定する．

### 7.3.1 優先度スケジューリング

タスクは重要さの点でも異なる．オペレーティングシステムにとって不可欠な仕事を行うカーネルレベルスレッドは，ユーザプロセスのスレッドより重要でありうるし，ユーザプロセスの中にも他より緊急なものがある．

▷ priority
▷ priority scheduling

**定義 7.7 (優先度).** タスクの**優先度**とは，スケジューラにとってのそのタスクの重要さを表す値である．**優先度スケジューリング**を用いるスケジューラは，実行可能なタスクのうち優先度が最も高いものを次に実行し，より高い優先度のタスクが実行可能になったときには実行中のタスクをプリエンプトする．

優先度スケジューラのランキューは，優先度の水準ごとに 1 つのキューを持ち，空でない最上位の水準の先頭のタスクを選ぶ形にできる．あるいは優先度で順序付けた木にすることもできる．

**例 7.8.** 3 つのタスクが次のように到着するとする（図 7.2）．

| タスク | 到着（s） | 実行時間（s） | 優先度 |
|---|---|---|---|
| T1 | 0 | 7 | 中 |
| T2 | 1 | 2 | 低 |
| T3 | 2 | 3 | 高 |

プリエンプティブ SJF では，T1 は 1 s に T2 が到着するまで実行される．T2 は T1 に残る 6 s より短いのでこれをプリエンプトする．2 s に T3 が到着したとき，T2 の残りは 1 s しかないので T2 が実行を続け，3 s に完了する．T3 は T1 の残りより短いので次に実行され，6 s に完了する．T1 は 12 s に完了する．到着から測ると各タスクは 12，2，4 s を要し，平均は 6 s である．プリエンプティブな優先度スケジューリングでは，T2 は T1 をプリエンプトしないが T3 はプリエンプトする．T3 が 2 から 5 s まで実行され，次に T1 が 10 s に完了し，T2 は 12 s になってようやく完了する．

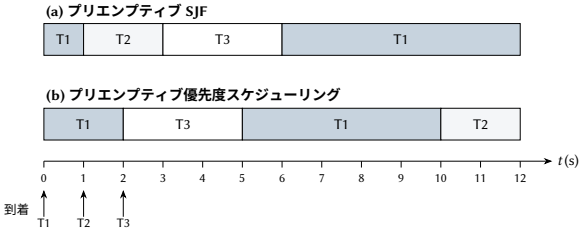



**図 7.2:** 例 7.8 のタスクを (a) プリエンプティブ SJF と (b) プリエンプティブな優先度スケジューリングで実行した場合．

### 7.3.2 スタベーションとエージング

優先度スケジューリングには危険がある．より高い優先度のタスクが到着し続ける限り，低い優先度のタスクはランキューに留まり続ける．

▷ starvation

**定義 7.9 (スタベーション).** スケジューラが常に他のタスクを優先するために，実行可能でありながら CPU を無期限に与えられないタスクは，**スタベーション**に陥る．

▷ priority aging

その対策が**エージング**である．スケジューラが用いる優先度は，タスク本来の優先度だけではなく，本来の優先度と，そのタスクがランキューで過ごした時間との関数である．

タスクが待つ時間が長いほどその実効的な優先度は高くなり，やがて到着するタスクの優先度を上回ってそのタスクが実行される．こうしてエージングはスタベーションを防ぐ．

## 7.4 優先度逆転

優先度は同期と相互に作用する．優先度が T1 > T2 > T3 である 3 つのタスクを取り上げる．このうち T1 と T3 は同じミューテックス m をロックする（図 7.3a）．どのタスクも 3 s の CPU 時間を要する．T3 は実行の最初の 2 s の間 m を保持し，T1 は 1 s 実行した後 1 s の間それを保持する．T3 は時刻 0 に到着し，直ちに m をロックする．時刻 1 に T2 が到着して T3 をプリエンプトし，時刻 2 に T1 が到着して T2 をプリエンプトする．時刻 3 に T1 は m をロックしようとするが，これは T3 が保持しているのでブロックする．残る 2 つのタスクのうち優先度が高いのは T2 であるから，T3 が実行を再開できるようになる前に T2 が最後まで実行される．その後ようやく T3 はクリティカルセクションを終えて m をアンロックし，T1 が先へ進める．T2 は T1 より早く時刻 5 に完了し，T1 は時刻 8 に完了する．T3 は最後に時刻 9 に完了する．

出典 P3L1，優先度逆転；Silberschatz ら ch. 5–6．

**定義 7.10 (優先度逆転).** **優先度逆転**とは，高い優先度のタスクが，より低い優先度のタスクが保持する資源を待っている一方で，その資源を必要としない中間の優先度のタスクがその間に実行されることである．高い優先度のタスクは，あたかも優先度が逆転しているかのように遅れる．

▷ priority inversion

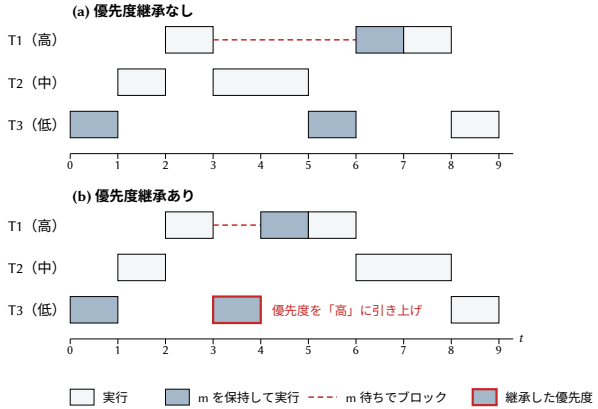



**図 7.3:** T1 と T3 はミューテックス m を共有する．(a) T1 が m でブロックしている間に，中優先度の T2 が T1 に先んじて完了まで実行される．(b) 優先度継承があると，T3 は m をアンロックするまで T1 の優先度で実行され，T1 は T2 より先に完了する．

解決策は，ミューテックスの所有者の優先度を一時的に引き上げることである（図 7.3b）．T1 が m でブロックすると，T3 に T1 の優先度が与えられ，T2 ではなく T3 が実行されてクリティカルセクションを終える．T3 が m をアンロックすると，その優先度は本来のものに戻され，T1 がミューテックスを獲得して実行され，T1 は T2 に先んじて時刻 6 に完了する．T2 は時刻 8 に完了する．

**定義 7.11 (優先度継承).** **優先度継承**では，ロックの所有者は，そのロックでブロックしているタスクのうち最も優先度の高いものの優先度で一時的に実行され，ロックを解放すると自分本来の優先度に戻る．

▷ priority inheritance

所有者の優先度を引き上げるには，ロック操作が誰が所有者であるかを知らなければならない．これも，第 5 章の適応型ミューテックスに加えて，ミューテックスのデータ構造に所有者を記録しておく（第 3 章）理由の 1 つである．[1]POSIX スレッド（第 4 章）は，優先度継承をミューテックスのプロトコル属性として提供する．

[1] 1997 年 7 月，火星に着陸した Mars Pathfinder は再起動を繰り返した．優先度の低い気象観測タスクが共有情報バスのミューテックスを保持し，優先度の高いバスのタスクがそれでブロックし，代わりに中優先度の通信処理が実行され，最後にウォッチドッグタイマが探査機をリセットしたのである．JPL は地上で優先度逆転を再現し，そのミューテックスで優先度継承を有効にする修正を送信した．

```
pthread_mutexattr_t attr;
pthread_mutex_t m;

pthread_mutexattr_init(&attr);
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT);
pthread_mutex_init(&m, &attr);
```

**要点**
プリエンプションによって，スケジューラは短いタスクや重要なタスクを先に処理できる．優先度は 2 つの危険をもたらす．1 つはスタベーションであり，エージングによって回避される．もう 1 つは優先度逆転であり，ロックの所有者にそのロックでブロックしているタスクの優先度を継承させることで回避される．

**注意** POSIX は優先度上限プロトコル PTHREAD_PRIO_PROTECT も定義している．所有者は，ブロックしているタスクの有無によらず，ミューテックスに宣言された上限の優先度で実行される．講義は継承のみを扱う．

## 7.5 ラウンドロビンスケジューリングとタイムシェアリング

実行時間が分からないタスクのためのスケジューラは，単純にタスクを順番に実行させればよい．このようなスケジューラには，実行時間の推定も，基本的な形では優先度も必要ない．

**出典** P3L1，ラウンドロビンスケジューリング，タイムシェアリングとタイムスライス；OSTEP ch. 7.

▷ round robin scheduling

**定義 7.12** (ラウンドロビン). **ラウンドロビンスケジューリング**（RR）を用いるスケジューラは，FCFS と同様にランキューの先頭のタスクを取り出す．FCFS と異なり，タスクは完了まで実行される必要はない．例えば I/O を待つために CPU を手放すことができ，再び実行可能になったときにはランキューの末尾に置かれる．

ラウンドロビンは優先度と組み合わせられる．同じ優先度のタスクが順番に実行され，より高い優先度のタスクが実行可能になれば，実行中のタスクをプリエンプトする．最も一般的な形では，ラウンドロビンは各タスクが実行してよい時間を制限することで，タスクが自ら CPU を手放さない場合にもタスクを交互に実行する．

第 2 章のタイムスライスは，タスクに与えられる，中断されない時間の最大値であり，タイムクォンタムとも呼ばれる．タスクはタイムスライスより短い時間しか実行しないこともある．I/O や同期機構を待ってブロックし，対応するキューに置かれるかもしれないし，より高い優先度のタスクが実行可能になってプリエンプトするかもしれない．タイムスライスが満了すると，実行中のタスクはプリエンプトされてランキューの末尾に戻る．

Silberschatz らの ch. 5 は典型的なタイムクォンタムを 10–100 ms としており，これは数マイクロ秒のコンテキストスイッチ（第 2 章）よりはるかに長い．Linux は今もリアルタイムのラウンドロビンクラスには固定のタイムクォンタムを用いる．既定は 100 ms であり，sysctl の sched_rr_timeslice_ms で設定する．

▷ timesharing

**定義 7.13** (タイムシェアリング). タイムスライスによってタスクを交互に実行することを，CPU の**タイムシェアリング**という．どのタスクも一度に高々 1 タイムスライスしか実行されないので，CPU 集約的なタスクもタイムスライスが満了すればプリエンプトされ，完了するまで CPU を占有し続けることはできない．

**例 7.14.** 例 7.5 のタスクを，コンテキストスイッチの費用を無視して，タイムスライス 1 s のラウンドロビンで実行する（図 7.1）．順序は T1，T2，T3，T1，T2，T1 であり，T3 は 3 s，T2 は 5 s，T1 は最後の 3 タイムスライスを単独で実行した後 8 s に完了する．指標は 表 7.1 で比較する．実行時間を一切知らなくても，ラウンドロビンは平均完了時間を 6.67 s から 5.33 s へ，FCFS から SJF の 4.00 s までのちょうど中間まで下げる．また，各タスクが最初に実行されるまでに待つのは，自分より前にあるタスクそれぞれについて高々 1 タイムスライス分であるから，その平均待ち時間は SJF より短くさえある．

**表 7.1:** 図 7.1 の 3 つのスケジュールの指標．

| | FCFS | SJF | RR（1 s） |
|---|---|---|---|
| スループット（タスク/秒） | 0.375 | 0.375 | 0.375 |
| 平均完了時間（s） | 6.67 | 4.00 | 5.33 |
| 平均待ち時間（s） | 4.00 | 1.33 | 1.00 |

タイムシェアリングにはいくつかの利点がある．短いタスクがより早く完了する．実行可能なタスクはどれも限られた時間内に実行されるので，システムの応答性が高まる．

そして，長い I/O 操作を開始するタスクがそれに早く到達するので，他のタスクが計算している間にその操作を進められる．一方で，先の例が無視した費用もある．タイムスライスの終わりごとにタイマが実行中のタスクに割り込み，スケジューラが動作し，コンテキストスイッチが行われる．このオーバーヘッドを小さく保つには，スケジューラ単独の時間について 式 (2.1) が示したように，タイムスライスはスケジューラとコンテキストスイッチに要する時間よりはるかに長くなければならない．

## 7.6 タイムスライスの長さの選択

タイムスライスがどれだけ長くあるべきかは，タスクがそれをどう使うかに依存する．タイムスライスは，タイムシェアリングの利点とそのオーバーヘッドとを釣り合わせなければならず，その釣り合いは，主に計算するタスクと，主に I/O を待つタスクとで異なる．

出典 P3L1，タイムスライスはどれだけ長くあるべきか，CPU バウンドのタイムスライス長，I/O バウンドのタイムスライス長，タイムスライス長のまとめ；OSTEP ch. 7．

### 7.6.1 CPU バウンドのタスク

ほとんどの時間を計算に費やし，めったにブロックしないタスクを **CPU バウンドタスク** という．

▷ CPU-bound task

**例 7.15.** 2 つの CPU バウンドのタスク T1 と T2 が，それぞれ 6 s の CPU 時間を必要とし，時刻 0 に到着するとする．コンテキストスイッチには 0.2 s を要する．タイムスライスが 1 s のとき，2 つのタスクは 12 回交互に実行され，タイムスライスの間に 11 回のコンテキストスイッチが入る．T2 は 1.2 s に開始する．T1 の最後のタイムスライスは，10 回のタイムスライスと 10 回のコンテキストスイッチの後，12 s に開始するので，T1 は 13 s に完了し，T2 はもう 1 回のコンテキストスイッチの後 14.2 s に完了する．タイムスライスが 3 s のときは 4 回交互に実行され，コンテキストスイッチは 3 回である．T2 は 3.2 s に開始し，T1 は 9.4 s に，T2 は 12.6 s に完了する．タイムスライスが無限のときは，T1 が 6 s に完了するまで実行され，T2 は 6.2 から 12.2 s まで実行される．得られる指標を表 7.2 に挙げる．

注意 0.2 s は説明のための値であって測定値ではない．第 2 章によればコンテキストスイッチの直接の費用は数マイクロ秒であり，実際の系がタイムスライスごとに失うものはこの例よりはるかに小さい．

**表 7.2:** コンテキストスイッチが 0.2 s のときの，6 s の CPU バウンドのタスク 2 つ（例 7.15）．

| タイムスライス | スループット（タスク/秒） | 平均待ち時間（s） | 平均完了時間（s） |
|---|---|---|---|
| 1 s | $2/14.2 \approx 0.141$ | $(0 + 1.2)/2 = 0.6$ | $(13 + 14.2)/2 = 13.6$ |
| 3 s | $2/12.6 \approx 0.159$ | $(0 + 3.2)/2 = 1.6$ | $(9.4 + 12.6)/2 = 11.0$ |
| ∞ | $2/12.2 \approx 0.164$ | $(0 + 6.2)/2 = 3.1$ | $(6 + 12.2)/2 = 9.1$ |

CPU バウンドのタスクにとっては，タイムスライスが長いほどコンテキストスイッチに費やされる時間が減るので，スループットと平均完了時間が改善する．伸びるのは待ち時間だけであり，バッチ計算のような CPU バウンドのタスクにとって，それはほとんど問題にならない．その利用者が見るのは計算が完了する時刻であって，開始する時刻ではないからである．極限としての無限のタイムスライス，すなわち実行完了型が，それらにとっては最良である．

### 7.6.2 I/O バウンドのタスク

頻繁に I/O でブロックし，I/O 操作の間にわずかしか計算しないタスクを **I/O バウンドタスク** という．すべてのタスクが I/O バウンドであれば，タイムスライスの長さは，それらの短い計算の区間を超えてさえいれば，ほとんど問題にならない．どのタスクもタイムスライスが満了する前に I/O でブロックし，自ら CPU を離れるからである．長さが問題になるのは，I/O バウンドのタスクが CPU バウンドのタスクと CPU を共有するときである．

▷ I/O-bound task

**例 7.16.** CPU バウンドのタスク T1 が，5 ms 計算してから 10 ms を要する I/O 操作を発行する I/O バウンドのタスク T2 と CPU を共有し，コンテキストスイッチには 2 ms を要するとする（図 7.4）．タイムスライスが 5 ms のとき，T2 の I/O 操作は T1 の 2 回目のタイムスライスの途中で完了し，T2 は再び実行されるまで 4 ms 待つ．このパターンは 19 ms ごとに繰り返される．T2 は 19 ms ごとに I/O 操作を発行し，デバイスは時間の $10/19 \approx 53\,\%$ の間動いている．タイムスライスが 30 ms のときは，T2 は各 I/O 操作の後に 24 ms 待ち，パターンは 39 ms ごとに繰り返され，デバイスが動いているのは時間の $10/39 \approx 26\,\%$ にすぎない．一方 T1 は，短いタイムスライスでは 19 ms ごとに 10 ms（$\approx 53\,\%$）を得るが，長いタイムスライスでは 39 ms ごとに 30 ms（$\approx 77\,\%$）を得る．

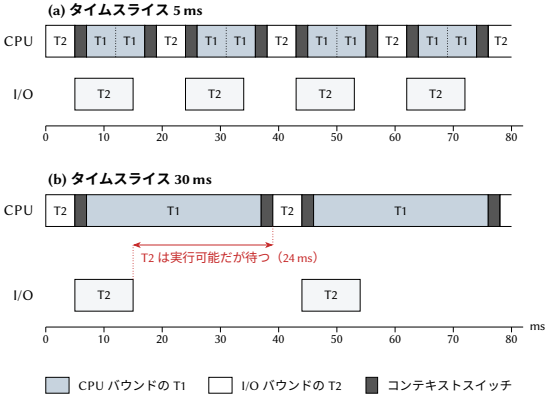



**図 7.4:** ラウンドロビンの下での CPU バウンドのタスク T1 と I/O バウンドのタスク T2（例 7.16）．T2 がまだ I/O を待っている間に T1 のタイムスライスが満了した場合，T1 はそのまま実行を続ける．

### 7.6.3 まとめ

2 種類のタスクは正反対のタイムスライス長を好む．

**CPU バウンドのタスクは長いタイムスライスを好む．** コンテキストスイッチが少ないほどオーバーヘッドが抑えられ，CPU 使用率とスループットが高く保たれる．

**I/O バウンドのタスクは短いタイムスライスを好む．** I/O バウンドのタスクは I/O の完了後すぐに実行され，次の I/O 操作をより早く発行する．これによりデバイスが動き続け，対話的なタスクは典型的に I/O バウンドであるから，利用者が体感する性能も良くなる．

例の 10 ms はディスクの値である．7200 rpm のドライブは平均しておよそ 4 ms の回転待ちと同程度のシークを要するが，NVMe の SSD は読み出しに数十マイクロ秒しか要しない．

### 7.6.4 CPU 使用率

CPU 時間のうち有用な仕事に費やされる割合（式 (2.1)）は，振る舞いの異なるタスクが混在する場合にも拡張できる．スケジュールの中で，どのタスクも実行の機会を得ている繰り返しの区間を 1 つ選ぶ．その区間の中で，CPU がタスクの実行に費やす総時間を $t_{\text{run}}$，コンテキストスイッチに費やす総時間を $t_{\text{ctx}}$ とすると，

$$\text{CPU 使用率} = \frac{t_{\text{run}}}{t_{\text{run}} + t_{\text{ctx}}} \tag{7.1}$$

である．

**ヒント** タイムスライス $q$ ごとに費用 $c$ のスイッチが 1 回入るなら，この割合は $q/(q+c)$ である．損失を 1 % 未満に抑えるには，タイムスライスをコンテキストスイッチの費用の 100 倍以上にすればよい．

**例 7.17.** 3 つの I/O バウンドのタスクがそれぞれ 2 ms 計算してから 5 ms の I/O 操作を発行し，2 つの CPU バウンドのタスクは決してブロックせず，コンテキストスイッチには 0.5 ms を要し，どのタスクも長時間実行されるとする．タイムスライスが 2 ms のときは，CPU バウンドかどうかによらずどのタスクも 2 ms で CPU を離れ，どの実行の後にもコンテキストスイッチが続くので，使用率は $2/(2+0.5) = 80\,\%$ である．タイムスライスが 20 ms のときは，1 巡は $3 \cdot 2 + 2 \cdot 20 = 46$ ms の計算と 5 回のコンテキストスイッチからなり，使用率は $46/(46+2.5) \approx 94.8\,\%$ である．12.5 と 48.5 ms というどちらの 1 巡も I/O 操作より長いので，I/O バウンドのタスクは自分の順番が来たときには常に実行可能である．

**注意** 式 (7.1) はアイドル時間を無視している．どのタスクも I/O を待っている区間は $t_{\text{run}}$ にも $t_{\text{ctx}}$ にも寄与しないので，CPU に実行するものがない時間が長くても，この比は 100 % になりうる．

> **要点**
> タイムシェアリングは，実行時間の知識なしにシステムを応答性の高いものにするが，その代償としてタイムスライスごとに割り込み，スケジューラの実行，コンテキストスイッチが必要になる．CPU バウンドのタスクは長いタイムスライスを，I/O バウンドのタスクは短いタイムスライスを好む．

## 7.7 多段フィードバックキュー

I/O バウンドのタスクと CPU バウンドのタスクに異なるタイムスライスを与えるには，スケジューラはすべてのタスクを 1 つのランキューに保持して選択時に各タスクの種類を調べてもよいし，2 種類のタスクを別々の構造に保持してもよい．複数のキューからなるランキューは後者の方法を取る（図 7.5）．

- 1 番目のキューは最も I/O 集約的なタスクを保持し，最も高い優先度を持ち，そのタスクに 8 ms のタイムスライスを与える．
- 2 番目のキューは I/O と計算が混在するタスクを保持し，タイムスライスは 16 ms である．
- 最後のキューは CPU 集約的なタスクを保持し，最も低い優先度を持ち，無限のタイムスライス，すなわち FCFS でそれらをスケジュールする．

I/O バウンドのタスクはタイムシェアリングの利点を受け，CPU バウンドのタスクはそのオーバーヘッドを避けられる．

出典 P3L1，ランキューのデータ構造；OSTEP ch. 8；Corbató ら [20]．

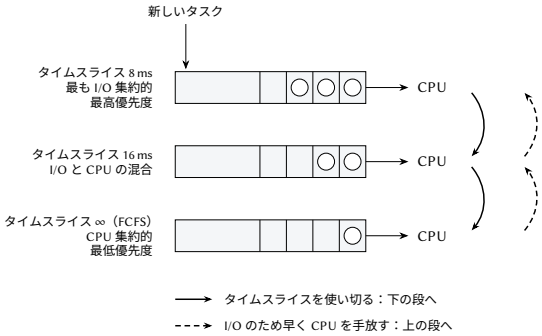



**図 7.5:** 3 段の多段フィードバックキュー．各段はそれぞれのタイムスライスでタスクを処理し，タスクはタイムスライスの使い方に応じて段の間を移動する．

この設計は，あるタスクが I/O 集約的か CPU 集約的かをスケジューラがどう知るのか，という問題を提起する．タスクの履歴に基づく経験則は，新しいタスクにも，振る舞いを変えるタスクにも役立たない．その解決策は，タスクがタイムスライスをどう使うかから，スケジューラにタスクの種類を学習させることである．

1. 新しいタスクは最上段のキューに入る．
2. タスクがタイムスライスの満了前に CPU を手放したなら，段の選択は適切であったのであり，そのタスクはこの段に留まる．
3. タスクがタイムスライス全体を使い切ったなら，それはその段が想定するより CPU 集約的であり，1 段下へ移される．
4. 下の段にあるタスクが I/O を待つために早く CPU を解放したなら，優先度の引き上げを受けて上へ移される．

注意 OSTEP の MLFQ は 2 点で異なる．どの段もタスクをラウンドロビンで実行し，最下段も例外ではない．また周期 $S$ ごとにすべてのタスクが最上段へ戻され，どのタスクもスタベーションに陥らない（OSTEP ch. 8）．

**定義 7.18** (多段フィードバックキュー). **多段フィードバックキュー**（MLFQ）とは，複数の段からなり，各段が独自のスケジューリング方針を持ち，観測された CPU の使い方に応じてタスクが段の間を移動するランキューである．

▷ multi-level feedback queue

MLFQ は単なる優先度キューの集まりではない．その各段は，長さの異なるタイムスライスと最下段の FCFS によってタスクを異なる仕方で扱い，フィードバック機構が時間とともに各タスクの段を調節する．この設計は Fernando Corbató のタイムシェアリングシステム CTSS [20] にさかのぼる．Corbató は 1990 年にチューリング賞を受賞した．

**例 7.19.** 新しいタスクが 8 ms の段に入り，ブロックせずに 30 ms 計算するとする．このタスクは最初のタイムスライス全体を使い切って 16 ms の段へ移り，そのタイムスライスも使い切って最下段へ移り，そこで残りの 6 ms を完了する．I/O 操作の間に 3 ms 計算するタスクは，8 ms のタイムスライスを使い切ることが決してなく，最上段に留まる．

## 7.8 Linux の $O(1)$ スケジューラ

▷ O(1) scheduler

**出典** P3L1，Linux の O(1) スケジューラ；Bovet と Cesati [21] ch. 7.

Linux の $O(1)$ **スケジューラ**は，システム内のタスク数によらず，実行するタスクの選択とランキューへのタスクの追加を定数時間で行う．これはプリエンプティブで優先度に基づくスケジューラであり，140 段階の優先度を持ち，番号が小さいほど優先度が高いように番号付けられている．0 から 99 までの水準はリアルタイムタスク用，100 から 139 までの水準はタイムシェアリングのタスク用である．

▷ nice value

ユーザプロセスの既定の優先度は 120 である．これはプロセスの **nice 値**によって調整でき，nice 値は −20 から 19 の範囲を取り，既定値に加算されるので，タイムシェアリングのタスクの優先度は 100 から 139 の範囲となる．プロセスは，例えば `nice` システムコールによって自分の優先度を下げられるが，上げるには権限が必要である．

**ヒント** `nice -n 5 cmd` はコマンドを nice 値 5 で起動する．`renice` は後からその値を変更し，`chrt` はリアルタイムの方針を選ぶ．

タスクのタイムスライスはその優先度に依存する．優先度の低いタスクには最も短いタイムスライスが，優先度の高いタスクには最も長いタイムスライスが与えられる．スケジューラはさらに，タスクの**スリープ時間**，すなわちタスクが待機またはアイドルで過ごす時間に基づくフィードバックも用いる．

- 長い時間スリープするタスクは対話的であり，その優先度は最大 5 水準引き上げられる（その番号は最大 5 減る）．
- ほとんどスリープしないタスクは計算集約的であり，その優先度は最大 5 水準引き下げられる（その番号は最大 5 増える）．

**注意** 2 つの番号付けは向きが逆である．優先度の番号が大きいほど，すなわち nice 値が大きいほど優先度は低いが，`SCHED_FIFO` と `SCHED_RR` が用いる `sched_priority`（1–99）は大きいほど優先度が高い．

**例 7.20.** Bovet と Cesati が記述している Linux 2.6 カーネルでは，nice 値が与える優先度である静的優先度が $p$ であるタイムシェアリングのタスクには，$p < 120$ なら $(140 - p) \times 20$ ms の，そうでなければ $(140 - p) \times 5$ ms のタイムスライスが与えられる．nice 値 5 のプロセスは優先度 125 を持ち，タイムスライスは $15 \times 5 = 75$ ms である．nice 値 0 のプロセスは 100 ms，nice 値 −10 のプロセスは $30 \times 20 = 600$ ms である．nice 値 5 のプロセスが対話的であれば，フィードバックによって優先度は 120 まで高められ，スリープせずに計算するのであれば 130 まで下げられる．

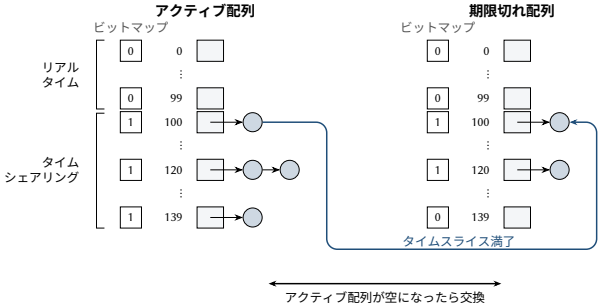



**図 7.6:** $O(1)$ スケジューラのランキュー．各配列は優先度の水準ごとに 1 本のリストと，空でないリストを示すビットマップを持つ．タイムスライスが満了したタスクは期限切れ配列へ移り，アクティブ配列が空になると 2 つの配列が交換される．

▷ active array

ランキューは，優先度の水準ごとに 1 本のリストを持つ，タスクのリストの配列 2 つからなる（図 7.6）．スケジューラが次のタスクを取り出す方の配列を**アクティブ配列**という．ビットマップがその 140 本のリストのうちどれが空でないかを記録しており，スケジューラは最初に立っているビットを見つけてそのリストの先頭のタスクを取り出す．タスクの追加はリストの末尾への追加とビットの設定である．水準の数が固定であるから，どちらの操作も定数時間で済む．タスクは，そのタイムスライスが満了するまでアクティブ配列に留まる．それより前にブロックしたタスクは，再び実行可能になったときに，残りのタイムスライスとともにアクティブ配列へ戻る．

▷ expired array

**期限切れ配列**は，タイムスライスが満了して非アクティブとなったタスクを保持する．タスクのタイムスライスが満了すると，スケジューラはその優先度とタイムスライスを再計算し，タスクを期限切れ配列へ移す．アクティブ配列にタスクが残らなくなると，スケジューラは 2 つの配列を交換する．これにはポインタを 2 つ入れ替えるだけでよい．この仕組みは，優先度の低いタスクにも実行の機会を与える．優先度の高いタスクは長いタイムスライスを受け取るものの，それを使い切れば期限切れ配列へ移り，アクティブ配列のすべてのタスクが，優先度の低いものも含めて，自分のタイムスライスを使い

切るまでそこで待つからである．

*O*(1) スケジューラは Ingo Molnar によって Linux 2.5 の開発系列で導入され，Linux 2.6 のスケジューラとなった．ストリーミングメディアのようなアプリケーションによってワークロードがより対話的になるにつれ，そのジッタが問題となり，Linux 2.6.23 では，同じく Molnar が書いた Completely Fair Scheduler に置き換えられた．

Linux 2.6 カーネルでは，対話的と判定されたタスクは，期限切れ配列のタスクがすでに待ちすぎている場合を除き，通常はアクティブ配列へ戻された．

## 7.9 Linux の Completely Fair Scheduler

*O*(1) スケジューラには 2 つの問題がある．第 1 に，対話的なタスクの性能が悪い．いったんタスクが期限切れ配列に入ると，アクティブ配列のすべてのタスクがタイムスライスを使い切るまでスケジュールされないので，対話的なタスクが実行されるまでに予測できない時間待つことがある．第 2 に，公平性の保証がない．ここで公平性とは，与えられた時間区間の中で，どのタスクもその優先度に比例した時間だけ実行されるべきだということである．

**出典** P3L1，Linux の CFS スケジューラ；OSTEP ch. 9.

**CFS**（完全公平スケジューラ）は，その両方の問題に対処する．これはバージョン 2.6.23 から 6.5 まで，リアルタイムでないタスクのための Linux の既定のスケジューラであった．そのランキューは**赤黒木**である．これは自己平衡二分探索木であり，根から葉へのどの経路も他のどの経路の 2 倍より長くならないので，すべての経路の長さがほぼ等しくなる．木はタスクの仮想実行時間で順序付けられている（図 7.7）．

▷ Completely Fair Scheduler

▷ red-black tree

**定義 7.21** (仮想実行時間)**.** タスクの**仮想実行時間**（`vruntime`）とは，そのタスクが CPU 上で実行に費やした時間であり，ナノ秒の粒度で測られ，タスクの優先度に応じて重み付けされる．

▷ virtual runtime

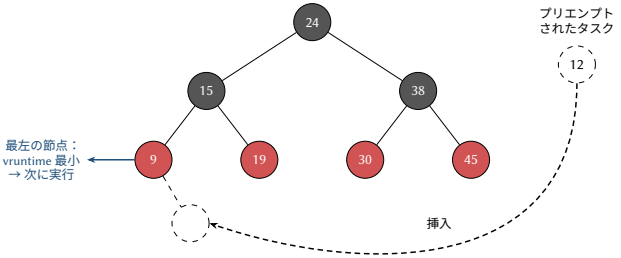



**図 7.7:** CFS のランキュー．実行可能なタスクを仮想実行時間で順序付けた赤黒木である（灰色の節点が黒，赤い節点が赤）．最左のタスクが次に実行され，プリエンプトされたタスクはその仮想実行時間が定める位置に挿入される．

CFS のアルゴリズムは単純である．

- 常に最左の節点のタスク，すなわち仮想実行時間が最小のタスク，言い換えればこれまでに受け取った CPU 時間が最も少ないタスクをスケジュールする．
- タスクが実行されている間，CFS は定期的にその仮想実行時間を増やし，木の中の最左のタスクの仮想実行時間と比較する．
- 実行中のタスクの仮想実行時間の方がまだ小さければそのタスクは実行を続け，大きければそのタスクはプリエンプトされて，その仮想実行時間が定める位置に木へ挿入され，最左のタスクが実行される．

Linux 6.6 以降，公平スケジューリングクラスは EEVDF アルゴリズムでタスクを選ぶが，CFS の仮想実行時間の計上はそのまま残っている．

優先度は，仮想実行時間が進む速さによって表される．優先度の低いタスクでは仮想実行時間が実時間より速く進むので，それらはすぐに CPU を失う．優先度の高いタスクではより遅く進むので，それらはより長く実行される．こうして 1 本の木がすべての優先度のタスクを保持する．スケジューラは最左の節点へのポインタを保持しているのでタスクの選択は定数時間で済み，タスクの追加は $N$ 個のタスクに対して $O(\log N)$ 時間で済む．言い換えれば，CFS は実行可能なすべてのタスクが共有するスケジューリング周期を，タスクの重みに比例するタイムスライスに分割する（OSTEP ch. 9）．

**例 7.22.** CFS はこの速さを nice 値ごとの重みから導く．nice 値 0 の重みは 1024 であり，nice 値の 1 段階ごとに重みは約 1.25 倍変化する．nice 値 5 の重みは 335 である．重み $w$ のタスクが時間 $\Delta t$ だけ実行されると，その仮想実行時間は $\Delta t \cdot 1024/w$ だけ進む．タスク A（nice 値 0）とタスク B（nice 値 5）がそれぞれ 6 ms 実行されると，A の仮想実行時間は

**ヒント** この係数は，nice 値 1 段が CPU のおよそ 10 % に相当するように選ばれている．1 段差なら 1.25/2.25 ≈ 55 : 45，5 段差ならここでの 75:25 である．

6 ms 進むが，B のそれは 6 · 1024/335 ≈ 18.3 ms 進む．CFS は各タスクの仮想実行時間を近い値に保つので，B が実行される時間は A の約 3 分の 1 となり，2 つのタスクは CPU の 1024/1359 ≈ 75 % と 25 % をそれぞれ受け取る．

表 7.3: Linux の 2 つのスケジューラ．

| | *O*(1) スケジューラ | CFS |
|---|---|---|
| Linux のバージョン | 2.6.0–2.6.22 | 2.6.23–6.5 |
| ランキュー | 140 本のリストからなるアクティブ配列と期限切れ配列 | 仮想実行時間で順序付けた赤黒木 |
| タスクの選択 | *O*(1) | *O*(1) |
| タスクの追加 | *O*(1) | *O*(log *N*) |
| 優先度 | 選択の順序，すなわち空でない最上位のリストと，タイムスライスの長さ．スリープ時間によるフィードバックが優先度を調整する | 仮想実行時間が進む速さ |
| 対話的なタスク | アクティブ配列全体を待つことがある | 仮想実行時間が最小になり次第実行される |

## 7.10 マルチプロセッサ上のスケジューリング

**出典** P3L1，マルチプロセッサ上のスケジューリング；OSTEP ch. 10；Silberschatz ら ch. 5．

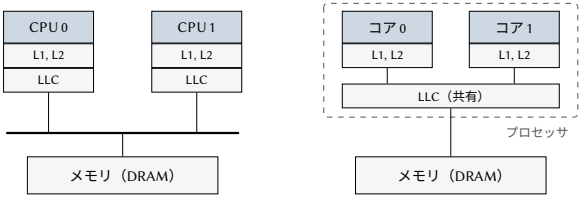
CPU が複数あると，スケジューリングはより複雑になる．2 つのアーキテクチャを区別しなければならない（図 7.8）．

▷ shared memory multiprocessor

**定義 7.23** (共有メモリ型マルチプロセッサ)．**共有メモリ型マルチプロセッサ**は複数の CPU を持ち，各 CPU は自分専用のキャッシュを持ち，それらすべてが主記憶を共有する．

▷ multicore processor

**定義 7.24** (マルチコアプロセッサ)．**マルチコアプロセッサ**は複数のコアを含み，各コアは自分専用のキャッシュを持つ CPU である．コアはプロセッサのラストレベルキャッシュ（LLC）と主記憶を共有する．




(a) 共有メモリ型マルチプロセッサ　　(b) マルチコアプロセッサ

図 7.8: (a) 共有メモリ型マルチプロセッサ．どの CPU も自分専用のキャッシュを持つ．(b) マルチコアプロセッサ．コアは専用のキャッシュを持ち，ラストレベルキャッシュを共有する．

オペレーティングシステムから見れば，マルチコアプロセッサの各コアは 1 つの CPU であり，システムは複数のマルチコアプロセッサを持つことで両方のアーキテクチャを組み合わせることもある．

### 7.10.1 キャッシュアフィニティと負荷分散

直近 sched_migration_cost_ns（既定は 500 µs）以内に実行されたタスクはキャッシュがホットとみなされ，負荷分散はそれを動かさない．Linux 5.13 以降，この調整値は debugfs の /sys/kernel/debug/sched/migration_cost_ns にある．

タスクの性能はキャッシュの状態に依存する．タスクがある CPU 上で実行される間，その CPU のキャッシュはそのタスクのデータで満たされる．次回も同じ CPU 上で実行されればホットキャッシュを見出すが（第 2 章），別の CPU 上で実行されればコールドキャッシュから始まり，そのキャッシュが満たされるまで実行は遅くなる．

▷ cache affinity

**定義 7.25** (キャッシュアフィニティ)．タスクが，自分のデータを保持するキャッシュを持つ CPU 上で実行されることから得る利益を，その**キャッシュアフィニティ**という．スケジューラは，各タスクをできる限り同じ CPU 上に保つことでこれを維持する．

そこでマルチプロセッサのためのスケジューラは階層的なアーキテクチャを取る．各 CPU が自分のランキューを持ち，その CPU のためのスケジューラはそこからのみタスクを選ぶので，タスクは同じ CPU 上で実行され続ける．CPU ごとのスケジューラの上には**負荷分散**の構成要素があり，ある CPU が過負荷である一方で別の CPU の仕事が少なすぎる，ということがないようにランキューの間でタスクを移動させる．負荷分散はランキューの長さに基づいて動作するほか，ある CPU がアイドルになったときに他の CPU のランキューからタスクを取ってくることもある．

▷ load balancing

Linux では，アプリケーションは `sched_setaffinity` でスレッドを CPU の集合に限定できる．

## 7.10.2 NUMA 対応スケジューリング

マルチプロセッサは複数のメモリノードを持つこともある（図 7.9）．

**定義 7.26** (NUMA). **NUMA**（不均一メモリアクセス）を持つシステムでは，主記憶は複数のメモリノードからなり，各ノードは 1 つ以上のプロセッサのソケットの近くに置かれる．CPU から自分のローカルなメモリノードへのアクセスは，ソケット間のインターコネクト，例えば Intel の UPI（旧 QPI）や AMD の Infinity Fabric を経由するリモートのノードへのアクセスより速い．

▷ non-uniform memory access

`numactl --hardware` はノードとその間の距離を表示し，`numactl --cpunodebind=0 --membind=0 cmd` はプログラムを 1 つのノードに固定する（numactl(8)）．

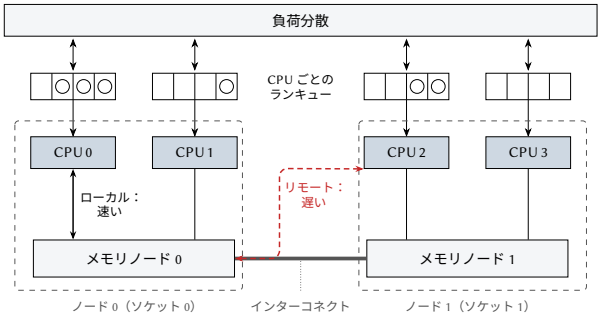



**図 7.9:** NUMA システム上の階層的なスケジューリング．どの CPU も自分のランキューからタスクをスケジュールし，負荷分散がランキューの間でタスクを移動させる．CPU は自分のノードのメモリには直接，もう一方のノードのメモリにはインターコネクトを経由して到達する．

**NUMA 対応スケジューリング**では，スケジューラはタスクを，その状態を保持するメモリノードに近い CPU 上に保ち，メモリアクセスがローカルなままであるようにする．負荷分散はこれを考慮しなければならない．タスクを別のノードの CPU へ移すとランキューの均衡は取れるかもしれないが，そのタスクの主記憶へのアクセスはすべてインターコネクトを越えることになり，遅くなる．

▷ NUMA-aware scheduling

**例 7.27.** あるタスクが実行時間の 40 % を主記憶へのアクセスに費やし，リモートのアクセスはローカルのアクセスの 1.5 倍の時間を要するとする．負荷分散器がこのタスクを，そのメモリはそのままにして，もう一方のノードの CPU へ移すと，そのタスクの実行時間は $0.6 + 0.4 \times 1.5 = 1.2$ 倍，すなわち 20 % 増える．

# 7.11 ハイパースレッディング

CPU を複数のタスクで多重化しなければならないのは，タスクの実行コンテキスト，すなわちプログラムカウンタやスタックポインタといったそのレジスタの値が，タスクの実行中は CPU のレジスタに保持されるからである．別のタスクに切り替えるには，そのコンテキストを退避し，別のコンテキストを読み込まなければならない．その代わりに，ハードウェアが 1 つの CPU 上に複数のレジスタ集合，したがって複数のハードウェアで支援された実行コンテキストを提供することもできる．その CPU は，コンテキストに共有される 1 組の実行資源を持つ単一の CPU のままであるが，あるコンテキストから別のコンテキストへ移るのに状態の退避と復元を必要としない．

**出典** P3L1，ハイパースレッディング；Silberschatz ら ch. 5；Fedorova ら [19]．

**定義 7.28** (同時マルチスレッディング). **同時マルチスレッディング**（SMT）では，1 つの CPU が複数のハードウェア実行コンテキスト，すなわちハードウェアスレッドを提供

▷ simultaneous multithreading

する．それぞれが自分のレジスタ状態を持ち，CPU のパイプラインとキャッシュを共有する．

▷chip multithreading

この技法は，ハードウェアマルチスレッディング，Intel が用いる名称であるハイパースレッディング，あるいはチップマルチスレッディングとも呼ばれる．より正確には，**チップマルチスレッディング**（CMT）は複数のコアとコア当たり複数のハードウェアスレッドを組み合わせたプロセッサを指し，これは Fedorova らが研究したプラットフォームである．実際にはこれらの名称はしばしば同じ意味で使われる．コア当たり 8 つまでのハードウェアスレッドを持つプロセッサも作られてきたが，最も一般的な構成は 2 つであり，SMT は起動時に有効にも無効にもできる．

Intel の最初の実装は，ダイ面積の 5% 未満の増加で，一般的なサーバアプリケーションのベンチマークにおいて最大 30% 性能を高めた（Marr ら，*Intel Technology Journal*，2002）．

設計によって，コンテキストがパイプラインをどれだけ密に共有するかは異なる．最も単純なものはコンテキストを交互に実行し，短ければ 1 サイクルごとにパイプラインを別のコンテキストへ渡す．狭義の同時マルチスレッディング，例えば Intel の Hyper-Threading では，複数のハードウェアスレッドの命令が同じサイクルにパイプラインの各段を占める．

各ハードウェアスレッドは，オペレーティングシステムには個別の仮想 CPU として見え，スケジューラはその上にタスクを配置できる．したがってスケジューラは，同じ CPU のハードウェアスレッド上でどのタスクを一緒にスケジュールするかを決めなければならない．ハードウェアスレッド間のコンテキストスイッチは数サイクル程度であるのに対し，キャッシュミスの後にメモリからデータを読み込むには 100 サイクル程度を要する．第 3 章の議論によれば，切り替えが有利になるのはアイドル時間が切り替えの費用の 2 倍を超えるときであり（式 (3.1)），ここではそれが大きな余裕をもって成り立つ．したがって SMT により，1 つの CPU 上のマルチスレッドが I/O のレイテンシを隠すのとちょうど同じように，CPU はメモリアクセスのレイテンシを隠せる．

## 7.12 ハイパースレッディング対応プラットフォームのスケジューリング

問題は，どのタスクが CPU を共有すべきかである．Fedorova らが研究したプラットフォームに倣ったマルチスレッド CPU の単純なモデルが，これに答える．このモデルは 4 つの仮定に立つ．

**出典** P3L1，ハイパースレッディング対応プラットフォームのスケジューリング；Fedorova ら [19]．

1. スレッドは CPU の毎サイクルに 1 つの命令を発行し，パイプラインは 1 サイクル当たり高々 1 つの命令を実行する．
2. メモリバウンドのスレッドのメモリアクセスは 4 サイクルを要し，その間そのスレッドは命令を発行しない．この小さな値はタイミング図を短く保つためのものであり，メモリまで到達するアクセスの 100 サイクル程度ではなく，L1 キャッシュで処理されるロードの遅延に当たる．
3. ハードウェアスレッド間の切り替えは瞬時に行われる．
4. CPU は 2 つのハードウェアスレッドを持つ．

Fedorova らがシミュレートしたコアは，4 つのハードウェアスレッドを 1 サイクルごとに交互に実行する．

このモデルでは，CPU バウンドのタスクは毎サイクルにパイプラインを必要とする．**メモリバウンドタスク** とは，その進行がメモリアクセスによって制限されるタスクであり，ほとんどのサイクルをメモリからのデータの到着を待って費やす．図 7.10 は，ありうる 3 通りの組み合わせを示している．

▷memory-bound task

**CPU バウンドのスレッド 2 つ．** どちらのスレッドも毎サイクル実行可能だが，パイプラインは 1 サイクル当たり 1 命令しか実行しない．2 つのスレッドは干渉し合ってパイプラインを奪い合う．それぞれは単独のときの半分の速度でしか進まず，2 倍の性能低下となる．その間，メモリの側はアイドルである．

**メモリバウンドのスレッド 2 つ．** どちらのスレッドもほとんどのサイクルをメモリ待ちで過ごし，図の 10 サイクルのうち 6 サイクルではどちらも実行すべき命令を持たない．これらの CPU サイクルは無駄になる．

**CPU バウンドとメモリバウンドのスレッド．** CPU バウンドのスレッドは，メモリバウンドのスレッドが待っているサイクルを使う．すなわち，10 サイクルのうち 8 サイクルで実行し，その間メモリバウンドのスレッドは単独のときと同じ速さで進む．アイドルのサイクルはなく，CPU もメモリもよく活用される．CPU バウンドのスレッドは 5 サイクルに 1 つを失うので干渉は残るが，それは小さい．

> **要点**
> SMT の CPU では，CPU バウンドのタスクとメモリバウンドのタスクを一緒にスケジュールすることで，パイプラインの奪い合いとアイドルサイクルの両方を避けられる．したがってスケジューラは，各 CPU のハードウェアスレッド上で 2 種類のタスクを混ぜるべきである．

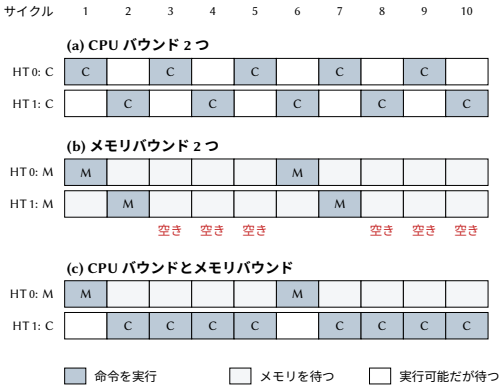



**図 7.10:** 1 つの SMT CPU のハードウェアスレッド（HT）上に 2 つのスレッドを一緒にスケジュールした場合．C は CPU バウンドの，M はメモリバウンドのスレッドを表す．各列は共有パイプラインの 1 サイクルである．

## 7.13 ハードウェア性能カウンタ

CPU バウンドのタスクとメモリバウンドのタスクを混ぜるには，スケジューラがどちらがどちらであるかを知る必要がある．I/O バウンドのタスクに用いた履歴に基づく経験則は通用しない．$O(1)$ スケジューラが用いるスリープ時間は，メモリバウンドのタスクを明らかにしない．メモリを待つスレッドはスリープしているのではなく，オペレーティングシステムからは見えないプロセッサの内部で待っているからである．個々のメモリアクセスをソフトウェアで追跡することもできない．それには時間がかかりすぎる．情報はハードウェアから得なければならない．

**出典** P3L1，CPU バウンドかメモリバウンドか，ハードウェアカウンタを用いたスケジューリング；Fedorova ら．

**定義 7.29 (ハードウェア性能カウンタ).** **ハードウェア性能カウンタ**とは，プロセッサの実行に伴って更新され，性能に関係する事象を数えるプロセッサのレジスタである．

▷ hardware performance counter

プロセッサは通常いくつかのカウンタを備えており，利用できるカウンタはアーキテクチャによって異なる．例えば次のようなものを数える．

- L1，L2 から LLC に至るキャッシュ階層の各水準でのキャッシュミス．
- 命令数とサイクル数．ここからサイクル当たり命令数（IPC）や，その逆数である命令当たりサイクル数が導かれる．
- 一部のプロセッサでは電力とエネルギーの消費．

ソフトウェアのインタフェースやツールがこれらのカウンタを利用可能にする．例えば，アーキテクチャごとに利用できるカウンタを文書に列挙している `oprofile` や，Linux の `perf` ツールがある．

カウンタの値から，スケジューラはタスクがどのような資源を必要とするかを推定できる．LLC ミスの少ないタスクはデータをキャッシュ内に見出しており CPU バウンドである可能性が高く，LLC ミスの多いタスクはメモリを待ち続けておりメモリバウンドである可能性が高い．単一のカウンタで決着が付くことはまれである．そこでスケジューラは複数のカウンタを組み合わせ，よく理解されたワークロードの特性に基づいてアーキテクチャごとに閾値を調整したモデルを用いる．

**ヒント** コマンド `perf stat -e cycles,instructions,LLC-load-misses` *cmd* は，プログラム *cmd* についてこれらの計数を報告する．

**定義 7.30 (命令当たりサイクル数).** タスクの **CPI**（命令当たりサイクル数）とは，実行した命令 1 つ当たりに要する CPU サイクル数，すなわちサイクル数の計数と命令数の計数の比である．

▷ cycles per instruction

**例 7.31.** 100 ms の区間の間に，カウンタが，タスク A は $4.6 \times 10^8$ サイクルで $4.2 \times 10^8$ 命令を実行し，1000 命令当たりの LLC ミスは 0.2 であったと報告し，タスク B は同じサイクル数で $0.9 \times 10^8$ 命令を実行し，1000 命令当たりの LLC ミスは 25 であったと報告したとする．A の CPI は約 1.1，B の CPI は約 5.1 である．どちらの指標も A を CPU バウンド，B をメモリバウンドと分類しており，7.12 節に従うスケジューラなら，これらを同じ CPU の 2 つのハードウェアスレッド上に配置するであろう．

## 7.14 スケジューリング指標としての命令当たりサイクル数

**出典** P3L1，CPI の実験，CPI の実験結果；Fedorova ら．

Fedorova らは，カウンタから導かれる単一の指標である CPI によってタスクを区別することを提案した．この着想は魅力的である．メモリバウンドのタスクは多くのサイクルをメモリ待ちに費やすので CPI が高く，CPU バウンドのタスクは CPI が低い．1 サイクルに 1 命令を実行するモデルでは 1 であり，スーパースカラのプロセッサでは 1 を下回る．そうであれば，スケジューラは各コアのハードウェアスレッド上に CPI の異なるタスクを混ぜればよいことになる．

**注意** CPI が高いのは分岐予測ミスやパイプラインの奪い合いのためでもありうるので，CPI が高いことだけではスレッドがメモリバウンドであることの証明にはならない．

この着想を検証するために，彼らは 4 つのコアがそれぞれ 4 つのハードウェアスレッドを持ち，合計 16 のハードウェアスレッドを持つチップマルチスレッディングのプロセッサをシミュレートした．ワークロードは合成されたもので，CPI が大きく異なるスレッドからなっていた．スレッドは，どのコアもさまざまな CPI のスレッドを混ぜて実行する割り当てから，どのコアも同じ CPI のスレッドを実行する割り当てまで，いくつかの方法でコアへ割り当てられた．講義は割り当てを合計の IPC（単一発行のコア 4 つでは最大 4）で比較するが，論文は合計のスループットを毎秒のトランザクション数で報告している．

CPI を混ぜた割り当ての性能が最も高く，各コア上のスレッドの CPI が似ている割り当ての性能が最も低かった．これらのワークロードでは CPI によるスケジューリングが明らかに有益であり，この実験はプロセッサのパイプラインの奪い合いが重要であることを裏付けている．しかし現実的なワークロードは異なる振る舞いを示す．標準的なベンチマークスイートのような実際のアプリケーションについて測定された CPI は，それほど明確には分かれない．それらは狭い範囲に集中する．[2] したがって CPI では，現実のワークロードのタスクを，スケジューラを導くのに十分なほど区別することはできない．合成されたワークロードでうまくいくことが，実際にはうまくいかないのである．

[2] ワークショップ論文自体は CPI について楽観的であった．測定された CPI はアプリケーション間で 1 桁の幅で変動し，スレッドの近い将来の CPI は予測可能であると報告している．そこで挙げられている限界は別のものである．CPI はキャッシュの奪い合いも浮動小数点ユニットの奪い合いも明らかにせず，忙しいシステム上で測ると歪む．

**注 7.32.** それでもこの研究は 3 つの教訓を残している．SMT のプロセッサ上のタスクはプロセッサのパイプラインを奪い合うこと，ハードウェアカウンタはワークロードを特徴付けられること，そしてスケジューラは単に負荷を均衡させるだけでなく，資源の奪い合いを意識しなければならないことである．同じグループによる後の研究 [22] は，奪い合いの指標としては CPI より，ラストレベルキャッシュの時間的な使用状況の方が優れていることを見出した．

> **要点 — 章のまとめ**
> スケジューラは自分のランキューから次のタスクを選び，ランキューの構造はアルゴリズムとともに設計される．FCFS と SJF はタスクを完了まで実行する．プリエンプションと優先度は緊急なタスクに応えるが，エージングで回避されるスタベーションと，優先度継承で回避される優先度逆転をもたらす．タイムスライスを用いるラウンドロビンはシステムを応答性の高いものにする．CPU バウンドのタスクは長いタイムスライスを，I/O バウンドのタスクは短いタイムスライスを好み，多段フィードバックキューは各タスクの種類を学習することでそれを実現する．Linux は，アクティブ配列と期限切れ配列を持つ $O(1)$ スケジューラを，赤黒木から仮想実行時間が最小のタスクを実行する CFS に置き換えた．マルチプロセッサでは，CPU ごとのランキューがキャッシュアフィニティを保ち，負荷分散がタスクを分散させ，NUMA 対応スケジューリングがタスクをそのメモリの近くに保つ．SMT のプロセッサでは，CPU バウンドのタスクとメモリバウンドのタスクが CPU を共有すべきであり，ハードウェア性能カウンタがそれらを区別する助けとなるが，CPI だけでは十分な指標ではない．

第 8 章

# メモリ管理

| | |
|---|---|
| 講義動画 | P3L2 |
| 教科書 | OSTEP [1] ch. 13，15–23；Silberschatz ら [2] ch. 9–10；Bovet と Cesati [21] ch. 2，8，17 |
| キーワード | 仮想メモリ，ページング，セグメンテーション，ページテーブルエントリ，ページフォールト，多段ページテーブル，TLB，ラージページ，バディアロケータ，スラブアロケータ，デマンドページング，ページ置換，コピーオンライト，チェックポインティング |

第 7 章では，オペレーティングシステムが CPU を複数のプロセスでどのように分け合うかを述べた．この章では，あらゆるプロセスが必要とする第 2 の資源，すなわち物理メモリを扱う．オペレーティングシステムは，各プロセスに専用の仮想アドレス空間（第 2 章）を与え，使用中の部分だけを物理メモリに保持し，他の部分は必要になったときに二次記憶から持ち込む．この対応付けを構成する 2 つの方式であるページとセグメント，アドレス変換を高速かつコンパクトにするハードウェアとデータ構造，物理メモリを管理するアロケータ，どのページがメモリを去るかを決める方針，そしてメモリ管理のハードウェアによって可能になる 2 つのサービス，すなわちコピーオンライトとチェックポインティングを順に調べる．

## 8.1 メモリ管理の目標

**出典** P3L2，レッスンの概観，メモリ管理の目標；OSTEP [1] ch. 13，15；Silberschatz ら [2] ch. 9．

オペレーティングシステムは，マシンの物理メモリ（DRAM）を，その上で動作するプロセスに代わって管理する．プロセスは物理アドレスを使わない．各プロセスは仮想アドレス空間を持ち，その範囲は物理メモリよりはるかに大きくなりうる．32 ビットの仮想アドレスでは，どのプロセスも 4 GiB をアドレス指定でき，すべてのプロセスのアドレス空間を合わせれば，搭載されているメモリを容易に超える．

**定義 8.1** (仮想メモリ)．オペレーティングシステムが，プロセスの仮想アドレス空間を物理メモリから切り離すために用いる抽象化を**仮想メモリ** と呼ぶ．オペレーティングシステムは，各アドレス空間のうち使用中の部分を物理メモリに対応付け，他の部分を二次記憶に置き，プロセスが使うすべてのアドレスを変換し，検証する．

▷ virtual memory

プロセスは自分の状態のすべてを同時に必要とするわけではない．どの時点でもアドレス空間の一部しか使わない．したがってオペレーティングシステムは，プロセスが次に必要とする状態を速やかに利用可能にできる限り，その部分だけを物理メモリに保持すればよい．ここから 2 つの仕事が生じる．

Linux の x86-64 では，ユーザ空間は 48 ビット空間の下半分 128 TiB を得て，カーネルは上半分を使う（Documentation/arch/x86/x86_64/mm.rst）．

**割り当て** オペレーティングシステムは，どのアドレス空間のどの部分を物理メモリのどこに置くかを決め，仮想アドレスから物理アドレスへの対応付けを確立する．物理メモリはプロセスが使うメモリより小さいことがあるので，一部の状態はディスクへ移し，必要になったときに戻さなければならない．したがって割り当てには**置換**，すなわち場所を空けるためにメモリを去る状態の選択が含まれる．

**調停** すべてのメモリアクセスは，仮想アドレスから物理アドレスへ変換され，かつ検証されなければならない．そのアドレスはアドレス空間の割り当て済みの領域に属していなければならず，そのアクセスは許可されていなければならない．仮想アドレスを，それが対応付けられた物理アドレスへ変換することを**アドレス変換** と呼ぶ．これはメモリアクセスのたびに起こるので，高速でなければならない．

▷ address translation

オペレーティングシステムがアドレス空間を単位に分割する方法は 2 つある（図 8.1）．

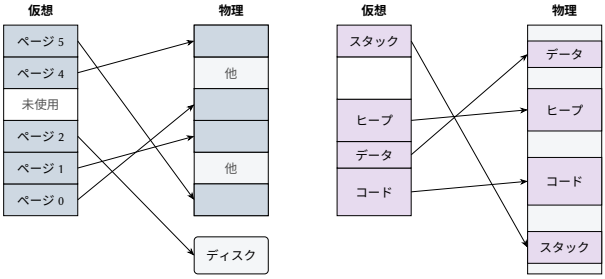
**ページに基づくメモリ管理 ページング**では，仮想アドレス空間は固定長のブロックに分割され，その各々を**ページ**と呼び，物理メモリも同じ大きさのブロックに分割され，その各々を**ページフレーム**と呼ぶ．割り当てはページをページフレームに対応付け，調停はページテーブルを用いる．

▷ paging
▷ page
▷ page frame

**セグメントに基づくメモリ管理 セグメンテーション**では，アドレス空間は可変長のブロックに分割され，その各々を**セグメント**と呼ぶ．セグメントは，コード，ヒープ，スタックのような，プロセスの論理的な構成要素に対応する．割り当てはセグメントを同じ大きさの物理メモリの領域に対応付け，調停はセグメントレジスタを用いる．

▷ segmentation
▷ segment

現在のオペレーティングシステムではページングが主流である．セグメンテーションは一部のアーキテクチャ，とりわけ x86 でサポートされており，そこではページングと組み合わされる（8.9 節）．




(a) ページング　　(b) セグメンテーション

**図 8.1:** (a) ページングは固定長のページをページフレームに対応付ける．ページ 2 はディスク上にあり，未使用のページには対応付けがない．(b) セグメンテーションは大きさの異なるセグメントを，同じ大きさの物理メモリの領域に対応付ける．

## 8.2 ハードウェアによる支援

**出典** P3L2，メモリ管理：ハードウェアによる支援；OSTEP ch. 15，18–19.

すべてのメモリアクセスをソフトウェアで変換し検証するのは，あまりにも遅い．そこでメモリ管理は CPU パッケージ内のハードウェアに依存する．ハードウェアがすべてのアクセスを処理し，オペレーティングシステムはデータ構造を用意して例外を処理する（図 8.2）．

▷ memory management unit

**定義 8.2 (メモリ管理ユニット).** **MMU**（メモリ管理ユニット）とは，CPU が発行する仮想アドレスを物理アドレスへ変換し，変換が不可能なときにフォールトを起こす，CPU パッケージ内のハードウェア構成要素である．

MMU がオペレーティングシステムにフォールトを報告するのは 3 つの場合である．割り当てられていない仮想アドレスへのアクセス（不正アクセス），読み出し専用メモリへの書き込みのように許可に違反するアクセス，そして物理メモリに存在しておらず先にディスクから持ち込まなければならないページへのアクセスである．MMU は，実行中のプロセスのページテーブルを指すレジスタを用いる．セグメンテーションの場合には，セグメントのベースアドレス，リミット，その他の情報を保持するレジスタを用いる．メモリ上のページテーブルを辿るのは遅いので，MMU は有効な変換のキャッシュである TLB も備える（8.7 節）．

**注意** ハードウェアだけで変換するアーキテクチャばかりではない．MIPS や SPARC では，TLB ミスが例外を起こし，カーネルのハンドラがソフトウェアで TLB を埋める．OSTEP ch. 19.

変換そのものはすべてハードウェアで行われる．オペレーティングシステムは，ハードウェアが使うページテーブルなどのデータ構造を作り，保守するが，その形式はハードウェアが定め，したがってどのようなメモリ管理の機構が可能かもハードウェアが定める．例えば，ページの大きさやページテーブルエントリのレイアウトである．

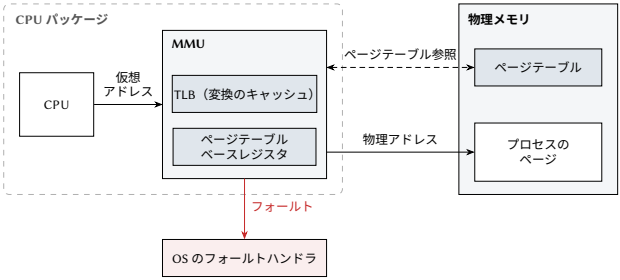



**図 8.2:** メモリ管理のためのハードウェアによる支援．MMU は TLB の助けを借りて仮想アドレスを変換し，変換がキャッシュされていないときは物理メモリ上のページテーブルを辿り，アクセスが不可能なときはオペレーティングシステムへフォールトを起こす．

## 8.3 ページテーブル

出典 P3L2，ページテーブル；OSTEP ch. 18；Silberschatz ら ch. 9.

ページテーブル（第 2 章）は，仮想アドレス空間のすべてのページを，それを保持するページフレームに対応付ける．ページとページフレームは同じ大きさなので，ページテーブルはアドレスを 1 つずつ対応付ける必要はない．ページ内でのアドレスの位置は仮想メモリでも物理メモリでも同じであり，変換が必要なのはページだけだからである．

$2^d$ バイトのページでは，仮想アドレスは 2 つの部分に分けられる．

**仮想ページ番号** 上位のビットが**仮想ページ番号**（VPN）を成し，これがページテーブルの索引となる．

▹ virtual page number

**オフセット** 下位 $d$ ビットが**オフセット**を成す．これはページ内でのアドレスの位置であり，変換されない．

▹ offset

ページテーブルの各エントリを**ページテーブルエントリ**（PTE）と呼ぶ．PTE は，他の情報（8.4 節）とともに，そのページを含むフレームの**物理フレーム番号**（PFN）を保持する．物理アドレスは，PFN と変換されないオフセットを組み合わせて得られる（図 8.3）．

▹ page table entry

▹ physical frame number

注意 PFN の幅は VPN の幅と同じとは限らない．物理アドレス拡張（PAE）を用いる 32 ビットの x86 は，20 ビットの VPN を 24 ビットの PFN へ対応付け，64 GiB の物理メモリを扱う．

$$\mathrm{VPN} = \left\lfloor \frac{VA}{2^d} \right\rfloor, \qquad \mathrm{offset} = VA \bmod 2^d, \qquad PA = \mathrm{PFN} \cdot 2^d + \mathrm{offset}. \tag{8.1}$$

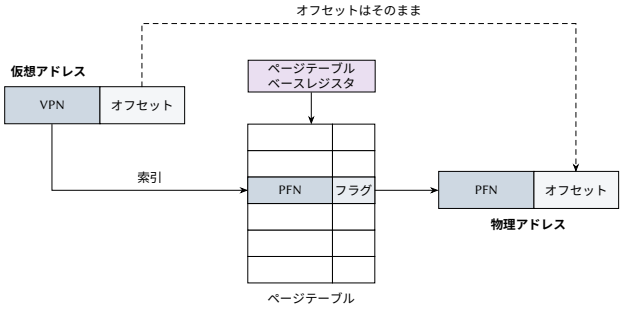



**図 8.3:** ページテーブルによるアドレス変換．VPN が，実行中のプロセスのページテーブルのエントリを選ぶ．そのエントリの PFN と，変換されないオフセットとを合わせたものが物理アドレスになる．

**例 8.3.** 32 ビットの仮想アドレスと 4 KiB のページ（$d = 12$）では，VPN は 20 ビット，オフセットは 12 ビット，すなわち 16 進で 3 桁である．仮想アドレス 0x00035A7C の VPN は 0x00035，オフセットは 0xA7C である．ページテーブルのエントリ 0x35 が PFN 0x001C2 を保持していれば，物理アドレスは 0x001C2A7C となる．0x00035000 から 0x00035FFF までのすべてのアドレスが同じエントリで変換される．

### 8.3.1 初回アクセス時の割り当てと回収

プロセスが，例えば大きな配列のために仮想アドレスの範囲を予約しても，その時点では物理メモリは割り当てられない．割り当てられるのは，プロセスがそのメモリにアクセスしたときである．この方針を**初回アクセス時の割り当て**と呼ぶ．ページへの最初のアクセスはフォールトを起こし，そこでオペレーティングシステムがそのページにページフレームを割り当て，ページテーブルに対応付けを作る．

▹ allocation on first touch

**例 8.4.** 次のコードは 256 MiB を予約するが，ページフレームを受け取るのは，ループが $2^{28}/2^{12} = 65\,536$ 個のページに 1 つずつアクセスしていくのに応じてである．

```
size_t n = 256UL << 20;          /* 256 MiB */
char *buf = malloc(n);           /* まだフレームなし */
for (size_t i = 0; i < n; i += 4096)
    buf[i] = 1;                  /* 初回参照でフォールト */
```

逆に，あるページが長く使われていないとき，オペレーティングシステムはそのフレームを回収してよい．ページの内容をディスクへ移し，対応付けを無効と記す（8.4 節）．プロセスがそのページに再びアクセスすると，MMU はフォールトを起こしてオペレーティングシステムへトラップする．オペレーティングシステムは，そのアクセスが許可され

ていることを確かめ，内容を，場合によっては別のページフレームへ，メモリに戻し，対応付けを作り直す．プロセスは，遅延のほかには何も気づかないまま実行を続ける．

オペレーティングシステムはプロセスごとに 1 つのページテーブルを保持する．MMU は，実行中のプロセスのページテーブルを**ページテーブルベースレジスタ**を通じて見つける．これはそのページテーブルのアドレスを保持する CPU のレジスタであり，x86 ではレジスタ CR3 である．コンテキストスイッチの際，オペレーティングシステムは次のプロセスのページテーブルのアドレスをこのレジスタに読み込む．

▷ page table base register

CR3 への書き込みは，プロセスコンテキスト識別子（PCID）を使っていない限り，グローバルでない TLB エントリをすべてフラッシュする（Intel SDM vol. 3A，§4.10）．

## 8.4 ページテーブルエントリ

PFN のほかに，どの PTE もフラグを含む．MMU はアクセスのたびにこれを検査し，オペレーティングシステムはこれを使ってメモリを管理する．最も重要なフラグは次のものである．

**有効ビット 有効ビット**は，**存在ビット**とも呼ばれ，対応付けが有効かどうか，すなわちエントリが指すフレームにそのページが物理メモリ上に存在するかどうかを示す．

**ダーティビット** ハードウェアは，ページが書き込まれるたびに**ダーティビット**を立てる．例えばファイルの内容をキャッシュするページは，ダーティビットが立っているときにだけファイルへ書き戻せばよい．

**アクセスビット** ハードウェアは，ページが読み出しでも書き込みでもアクセスされるたびに**アクセスビット**（参照ビット）を立てる．

**保護ビット 保護ビット**は，そのページに対して許される操作–読み出し，書き込み，実行–を指定する．

オペレーティングシステムはダーティビットとアクセスビットを読み，それらに基づいて処置を行った後にクリアする．

▷ valid bit

▷ dirty bit

▷ access bit

▷ protection bits

PAE を用いない 32 ビットの x86 では，PTE は 32 ビットである（図 8.4）．上位 20 ビットが PFN を保持し，下位のビットには存在ビット P，ダーティビット D，アクセスビット A が含まれる．R/W は保護ビットであり，0 ならページは読み出し専用，1 なら書き込みもできる．U/S は特権レベルを制限し，0 ならページはスーパバイザ（カーネル）モードでのみ，1 ならユーザモードでもアクセスできる．この形式には実行の許可がない．実行禁止ビットは，PAE と x86-64 の 64 ビットのエントリにだけ存在する．PWT と PCD はキャッシュに関する情報（ライトスルー，キャッシュ禁止）を伝え，ビット 9–11 はハードウェアが使わず，オペレーティングシステムが自由に使える．

**注意** 講義は x86 と同じく，2 つの概念を 1 つのビットにまとめている．OSTEP（ch. 18，21）はこれらを区別し，有効ビットは変換が割り当てられているかどうかを，存在ビットはページが物理メモリ上にあるかどうかを示す．

x86-64 では PTE は 8 バイトであり，ビット 63 が NX（実行禁止），ビット 12–51 が PFN を保持する（Intel SDM vol. 3A，§4.5）．




**図 8.4:** 32 ビット x86 のページテーブルエントリ．グローバルビット G とページ属性ビット PAT はここでは扱わない．

### 8.4.1 ページフォールト

MMU は PTE を，アドレスの変換にもアクセスの検査にも使う．

**定義 8.5** (ページフォールト). **ページフォールト**とは，現在のページテーブルエントリではアクセスを実行できないときに MMU が起こす例外である．エントリが有効でないか，アクセスがその保護ビットに違反する場合である．

▷ page fault

ページフォールトが起こると，CPU はエラーコードをカーネルスタックに置いてカーネルへトラップし，カーネルはページフォールトハンドラを実行する．x86 ではエラーコードが，ページが存在しなかったのかアクセスが保護ビットに違反したのか，アクセスが読み出しだったのか書き込みだったのか，ユーザモードとカーネルモードのどちらから来たのかを伝え，フォールトした仮想アドレスはレジスタ CR2 に保存される．ハンドラは，この情報と，自分が保持するアドレス空間の記録とから，取るべき動作を決める（表 8.1）．

I/O を必要としないフォールトをマイナーフォールト，ディスクから読み込むものをメジャーフォールトと呼ぶ．Linux は両方をプロセスごとに数える（`/proc/<pid>/stat` の `minflt`，`majflt`）．

不正なアクセスを行ったプロセスはシグナル SIGSEGV（第 5 章）を受け取り，その既定の動作はプロセスを終了させる．他のフォールトはすべてオペレーティングシステムが解決し，その後フォールトした命令が再び実行される．

**表 8.1:** ページテーブルエントリとオペレーティングシステム自身の記録に応じた，アクセスの扱い．

| PTE | 状況 | 動作 |
|---|---|---|
| 有効 | アクセスは許可 | なし：MMU が変換する |
| 無効 | 割り当て済み，未アクセス | フレームを割り当てる（初回アクセス） |
| 無効 | ページはディスク上 | 持ち込む（8.13 節） |
| 無効 | アドレスは未割り当て | 不正アクセス：SIGSEGV |
| 有効 | アクセスが保護に違反 | SIGSEGV，またはコピー（8.15 節） |

## 8.5 ページテーブルの大きさ

出典 P3L2，ページテーブルの大きさ；OSTEP ch. 20．

平坦なページテーブルは，ページが使われているかどうかにかかわらず，すべての仮想ページ番号に対して 1 つのエントリを持つ．$n$ ビットの仮想アドレス，$2^d$ バイトのページ，$e$ バイトのページテーブルエントリでは，エントリ数とテーブルの大きさは

$$N_{\text{PTE}} = \frac{2^n}{2^d} = 2^{n-d}, \qquad S_{\text{PT}} = 2^{n-d} \cdot e \tag{8.2}$$

である．4 KiB のページと 4 バイトのエントリを持つ 32 ビットアーキテクチャでは，ページテーブルは $2^{20}$ 個のエントリを持ち 4 MiB を占める．64 ビットのアドレスと 8 バイトのエントリでは，1 つのプロセスにつき $2^{52}$ 個のエントリ，$2^{55}$ バイト，すなわち 32 PiB を占めることになる．どのプロセスも自分のページテーブルを持つので，この代価にはさらにプロセスの数が掛かる．しかもプロセスはアドレス空間のごく一部しか使わないので，これらのエントリのほとんどは，決して使われないページを記述することになる．

ヒント ページが大きいほどエントリは少ない．ページの大きさを 2 倍にすれば $N_{\text{PTE}}$ は半分になる（8.10 節）．

**例 8.6.** x86-64 アーキテクチャは 48 ビットの仮想アドレスを使う．4 KiB のページと 8 バイトのエントリでは，平坦なページテーブルはプロセスごとに $2^{36}$ 個のエントリ，$2^{39}$ バイト，すなわち 512 GiB を必要とする．

## 8.6 多段ページテーブル

出典 P3L2，多段ページテーブル；OSTEP ch. 20；Silberschatz ら ch. 9．

したがって，大きなアドレス空間に対して平坦なページテーブルは実用的でない．アドレス空間の大部分は使われていないので，ページテーブルは代わりに階層的に構成され，大きな未使用領域のページにはページテーブルエントリが不要になる．

**定義 8.7 (多段ページテーブル).** **多段ページテーブル**とは，テーブルの木である．外側のテーブルである**ページディレクトリ**は内側のページテーブルへのポインタを保持し，内側のページテーブルが PFN を保持する．内側のページテーブルは，有効なページを含むアドレス空間の領域に対してだけ割り当てられる．

▷ multi-level page table
▷ page directory

2 段のテーブルでは，仮想アドレスは 3 つの部分に分けられる（図 8.5）．$p_1$ がページディレクトリの索引，$p_2$ がディレクトリのエントリが指す内側のページテーブルの索引となり，オフセット $d$ がフレーム内のアドレスを選ぶ．$p_2$ が $k$ ビットなら，内側のページテーブル 1 つが仮想アドレス空間の $2^{k+d}$ バイトを覆う．この大きさのアドレス空間の隙間には内側のページテーブルはまったく不要であり，そのディレクトリエントリを無効と記すだけでよい．プロセスが後にその領域にメモリを割り当てたら，オペレーティングシステムはそのための内側のページテーブルを割り当て，ディレクトリエントリを埋める．

ヒント 8 バイトのエントリ $2^9$ 個のテーブルはちょうど 4 KiB である．1 段当たり 9 ビットの索引にすると，どの段のテーブルも 1 つのページフレームに収まる．

**例 8.8.** 20 ビットの仮想アドレスと 1 KiB のページ（$d = 10$）を考える．平坦なページテーブルは $2^{10} = 1024$ 個のエントリを持つ．$p_1$ が 4 ビット，$p_2$ が 6 ビットの 2 段のテーブルでは，ページディレクトリは 16 個のエントリを持ち，64 個のエントリからなる内側のテーブルはそれぞれ 64 KiB を覆う．あるプロセスが最下位の 100 KiB（テキスト，データ，ヒープ，ページ 0–99）と最上位の 20 KiB（スタック，ページ 1004–1023）を使うとする．このプロセスには，ページ 0–63，64–127，960–1023 のための内側のテーブルが必要で，合計 $16 + 3 \cdot 64 = 208$ 個のエントリ，平坦なテーブルの約 5 分の 1 である．

階層はさらに段を加えて拡張できる．3 段目はページディレクトリへのポインタのテーブルを，4 段目はそのテーブルへのポインタのテーブルを加える．段を増やすことが最も効くのは 64 ビットアーキテクチャであり，そのアドレス空間はより大きく，かつより疎である．木の上位にある 1 つのエントリが覆う領域が大きいほど，その下にテーブルを必要としない隙間も大きくなる．

x86-64 は 12 ビットのオフセットの上に 9 ビットずつ 4 段を置く．$4 \cdot 9 + 12 = 48$ ビットのアドレスである．

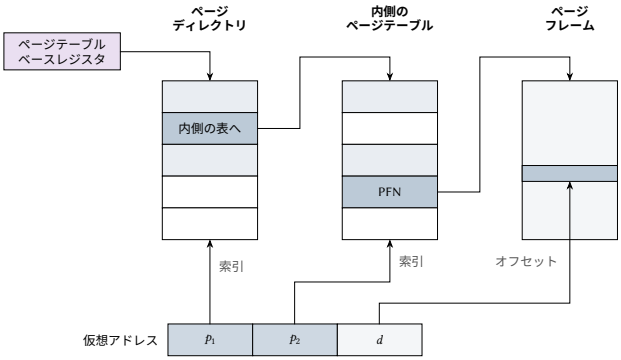



**図 8.5:** 2 段のページテーブル．白いディレクトリエントリには内側のページテーブルがない．変換には，ページディレクトリへの 1 回と内側のページテーブルへの 1 回のアクセスを要する．

代価は変換のレイテンシである．どの段のエントリも順にメモリから読まれるので，1 段ごとにアドレスの変換に 1 回のメモリアクセスが加わる．段が多いほどページテーブルのためのメモリは節約できるが，1 回の変換は遅くなる．

x86-64 が実装するのは 64 ビットのうち 48 ビットだけである．Intel の 5 段ページングは 5 段目を加えてこれを 57 ビット（128 PiB）まで広げる．Linux は 4.14 以降これを支援する．

## 8.7 変換の高速化：TLB

1 段のページテーブルでは，どのメモリ参照もページテーブルエントリへの 1 回とデータへの 1 回の計 2 回のメモリアクセスを要し，4 段では 5 回を要する．MMU はこれらのアクセスの大半をキャッシュで避ける．

**出典** P3L2，変換の速度：TLB；OSTEP ch. 19；Silberschatz ら ch. 9．

▷ translation lookaside buffer

**定義 8.9 (変換索引バッファ).** **TLB**（変換索引バッファ）とは，MMU に組み込まれた，最近使われたアドレス変換のキャッシュである．TLB ヒットの際はページテーブルにアクセスせずにアドレスが変換され，TLB ミスの際はページテーブルが辿られ，その変換が TLB に格納される．

MMU は上位の段のエントリもキャッシュするので（paging-structure caches），ミスの代価が $L$ 回のアクセスに達することはまれである．Intel SDM vol. 3A，4.10.3．

TLB のエントリは，その変換の保護ビットと有効ビットも保持するので，ページテーブルなしにアクセスを検査できる．メモリアクセスには**局所性**があるので，少数のエントリで高いヒット率が得られる．今アクセスされたページは間もなく再びアクセスされる可能性が高く（時間的局所性），それに近いアドレス—たいてい同じページの中にある—が次にアクセスされる可能性が高い（空間的局所性）．例えば Sandy Bridge 世代の Intel Core i7 プロセッサは，コアごとに 64 エントリのデータ TLB，128 エントリの命令 TLB，および両方の変換が共有する 512 エントリの第 2 段 TLB を持つ（いずれも 4 KiB のページに対して．Intel 64 and IA-32 Architectures Optimization Reference Manual）．

コンテキストスイッチは TLB のエントリを古くする．アドレス空間でタグ付けされていない限り，エントリはフラッシュされる（第 12 章）．

TLB の参照時間を $t_{\text{TLB}}$，メモリアクセス時間を $t_{\text{mem}}$，ヒット率を $h$，ページテーブルを $L$ 段とすると，メモリアクセスの実効時間は

$$t_{\text{eff}} = h\,(t_{\text{TLB}} + t_{\text{mem}}) + (1-h)\,\big(t_{\text{TLB}} + (L+1)\,t_{\text{mem}}\big) \tag{8.3}$$

である．

**ヒント** 式 (8.3) は $t_{\text{eff}} = t_{\text{TLB}} + \big(1 + L(1-h)\big)t_{\text{mem}}$ と書き直せる．参照の代価は常に払い，テーブルを辿る代価はミスのときだけ払う．

**例 8.10.** $t_{\text{mem}} = 80\,\text{ns}$，$t_{\text{TLB}} = 1\,\text{ns}$，$L = 4$ とする．TLB がなければ，どのアクセスも $5 \cdot 80 = 400\,\text{ns}$ かかる．ヒット率が 99 % なら，式 (8.3) より $0.99 \cdot 81 + 0.01 \cdot 401 \approx 84.2\,\text{ns}$ となり，変換のないアクセスより 5 % 増えるだけである．ヒット率が 90 % なら $0.9 \cdot 81 + 0.1 \cdot 401 = 113\,\text{ns}$ となる．

> **要点**
> 平坦なページテーブルは仮想アドレス空間に比例し，大きすぎる．多段ページテーブルは使用中の領域に対してのみテーブルを割り当て，その代価は 1 段当たり 1 回のメモリアクセスである．局所性を利用する TLB が，ほとんどすべての変換でそのアクセスを省く．

## 8.8 逆ページテーブル

ページテーブルを小さく保つもう 1 つの方法は，仮想メモリではなく物理メモリに従ってテーブルを構成することである．物理メモリは，すべてのプロセスの仮想アドレス空間を合わせたものよりはるかに小さく，どのページフレームも同時にはたかだか 1 つのページしか保持しない．

出典 P3L2，逆ページテーブル；OSTEP ch. 20；Silberschatz ら ch. 9．

**定義 8.11** (逆ページテーブル). **逆ページテーブル**は，物理メモリのページフレームごとに 1 つのエントリを持つ．フレーム $i$ のエントリは，フレーム $i$ が保持するページのプロセス識別子と仮想ページ番号を記録する．

▷ inverted page table

システム全体で逆ページテーブルは 1 つで済み，その大きさは物理メモリに比例し，プロセスの数にもそのアドレス空間の大きさにも依存しない．仮想アドレスは，プロセス識別子（pid），ページ番号 $p$，オフセット $d$ からなる．これを変換するには，(pid, $p$) に一致するエントリをテーブルから探す．そのエントリの索引 $i$ がフレーム番号であり，物理アドレスは $i$ と $d$ から作られる（図 8.6 (a)）．

**注意** フレームごとに 1 つのエントリということは，フレームごとに (pid, $p$) の組が 1 つということである．複数のプロセスが共有するフレームはこの構造に収まらず，共有メモリの扱いが難しくなる．

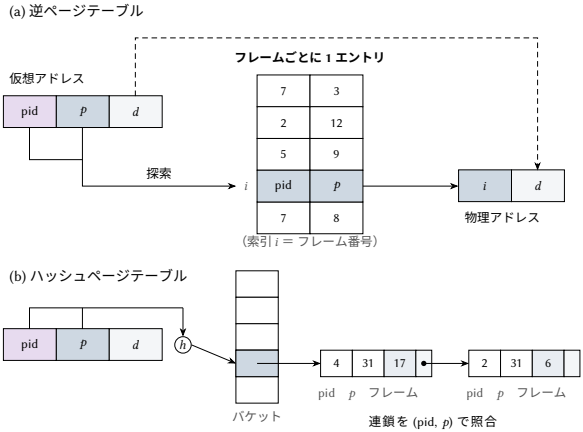



**図 8.6:** (a) 逆ページテーブルを (pid, $p$) で探索する．一致したエントリの索引がフレーム番号である．(b) ハッシュページテーブルは，ハッシュ関数 $h$ が選んだバケットの連鎖だけを探索する．

テーブルの線形探索は遅い．変換の大半は TLB が処理するが，ミスしたときの探索もやはり速くなければならず，これはハッシュ法で加速される．**ハッシュページテーブル**では，ページ番号にプロセス識別子を合わせてハッシュ関数を適用し，バケットを選ぶ．バケットは，キーがそのバケットにハッシュされるエントリの連結リストの先頭である（図 8.6 (b)）．各要素はプロセス識別子，ページ番号，フレーム番号，および次の要素へのポインタを保持するので，1 回の変換は 1 本の短いリストだけを探索する．

▷ hashed page table

ハッシュ化した逆ページテーブル：64 ビットの UltraSPARC と PowerPC（Silberschatz ら ch. 9），Itanium のハッシュ化 VHPT．x86 は一貫して多段ページテーブルである．

**例 8.12.** 16 GiB の物理メモリと 4 KiB のフレームを持つマシンは $2^{34}/2^{12} = 2^{22}$ 個のフレームを持つ．16 バイトのエントリなら，その逆ページテーブルは $2^{26}$ バイト，すなわち 64 MiB を占める．マシンが 10 個のプロセスを動かしていようと 1000 個を動かしていようと同じである．

## 8.9 セグメンテーション

セグメンテーションでは，アドレス空間は任意の大きさのセグメントに分割され，各セグメントは，コード，データ，ヒープ，スタックのような，プロセスの論理的に意味のある構成要素に対応する．仮想アドレスは，x86 では**論理アドレス**と呼ばれ，**セグメントセレクタ**とセグメント内のオフセットからなる．最も単純な形では，セグメントは物理メモリの連続した領域であり，それが始まるアドレスである**ベース**と，その大きさである**リミット**によって記述される．物理アドレスはベースにオフセットを加えたものであり，オフセットがリミット未満のときにだけ有効である．そうでなければアクセスはフォールトする．

出典 P3L2，セグメンテーション；OSTEP ch. 16；Silberschatz ら ch. 9；Bovet と Cesati ch. 2．

**例 8.13.** あるプロセスが，ベース 0x4000，リミット 0x1800 のコードセグメントと，ベース 0x9000，リミット 0x0800 のスタックセグメントを持つとする．論理アドレス（コード，0x0A20）は 0x0A20 < 0x1800 なので 0x4A20 に変換される．論理アドレス（スタック，0x0900）は，そのオフセットがスタックセグメントのリミットを超えるのでフォールトする．

記述子は 8 バイトである．セレクタは 13 ビットの索引—ゆえに 8192 エントリ—と，GDT か LDT かを選ぶビット，2 ビットの特権レベルを持つ．

x86 では，セレクタはセグメントレジスタに保持される．コード用の CS，データ用の DS，スタック用の SS などである．セレクタは**セグメント記述子表**—グローバル記述子表（GDT）またはローカル記述子表（LDT）—の索引であり，そのエントリであるセグメント記述子が，セグメントのベース，リミット，および許可を保持する．ベースとオフセットの和が**リニアアドレス**である．セグメンテーションはページングと組み合わされる．リニアアドレスはページテーブルによって物理アドレスへ変換される（図 8.7）．

- 32 ビットの x86（IA-32）では両方の機構が有効である．記述子表は最大 8192 個のセグメント記述子を保持する．Linux は，多くのオペレーティングシステムと同様に，アドレス 0 から始まりアドレス空間全体にわたるセグメントを定義し，変換と保護はページングに任せる．
- x86-64 では，セグメンテーションは後方互換のためにのみ残されている．64 ビットモードでは，プロセッサはセグメントのベースを 0 として扱い（FS と GS を除く），リミットも検査しないので，アドレスの変換はページングだけが行う．[1]

[1] 2 つの例外は実際に使われている．Linux は，実行中のスレッドのスレッドローカル記憶を FS のベースに，カーネルの CPU ごとのデータを GS のベースに置く．システムコールでカーネルに入るとき，命令 swapgs が後者のユーザ側とカーネル側の値を入れ替える．

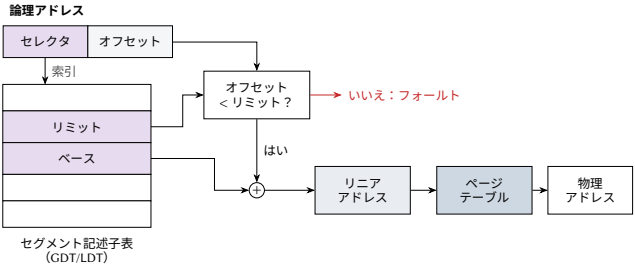



**図 8.7:** x86 における，ページングと組み合わされたセグメンテーション．セレクタがセグメント記述子表の索引となる．リミット内のオフセットがベースに加えられ，得られたリニアアドレスがページテーブルによって変換される．

## 8.10 ページの大きさ

**出典** P3L2，ページの大きさ；OSTEP ch. 20，23；Silberschatz ら ch. 9．

ページの大きさは，そのオフセットのビット数で決まる．10 ビットのオフセットは 1 KiB のページを，12 ビットのオフセットは 4 KiB のページを与え，後者が x86 での既定である．多くのアーキテクチャは，複数のページサイズを同時にサポートする．

▷ large page

▷ huge page

x86-64 では，既定のほかに 2 つの大きなページサイズが使える．**ラージページ**は 2 MiB の大きさを持ち，そのオフセットは 21 ビットである．オフセットが 30 ビットの 1 GiB のページは**ヒュージページ**と呼ばれる．表 8.2 は 3 つの大きさを比較したものである．

**表 8.2:** x86 のページサイズと，1 GiB のメモリを対応付けるのに必要なページテーブルエントリ数．

| ページの大きさ | オフセット幅 | 1 GiB のエントリ数 | 削減率 |
|---|---|---|---|
| 4 KiB（既定） | 12 | 262 144 | 1 |
| 2 MiB（ラージ） | 21 | 512 | $2^9 = 512$ |
| 1 GiB（ヒュージ） | 30 | 1 | $2^{18} = 262\,144$ |

**注意** 用語：講義のラージ（2 MiB）とヒュージ（1 GiB）のページを，Linux はどちらもヒュージページと呼ぶ（Transparent Huge Pages，`/sys/kernel/mm/transparent_hugepage/`）．

▷ internal fragmentation

大きなページには 2 つの利点がある．必要なページテーブルエントリが少ないのでページテーブルが小さくなり，TLB のエントリ 1 つがより多くのメモリを覆うので TLB ヒットとなるアクセスが増える．欠点はメモリの無駄である．

**定義 8.14** (内部断片化)**.** 割り当てが単位全体—ここではページ全体—に切り上げられるために，割り当てられていながら使われないメモリは，**内部断片化**によって失われる．

大きなページは，プロセスがその一部しか使わなくても全体が対応付けられるので，ページが大きいほど内部断片化が無駄にしうるメモリも大きくなる．

**例 8.15.** あるプロセスが 1 GiB のメモリを対応付ける．4 KiB のページでは 262 144 個のエントリを要し，64 エントリの TLB はそのうち 256 KiB にしか届かない．2 MiB のページでは 512 個のエントリで済み，同じ TLB が 128 MiB に届く．しかし 5222 KiB の別の領域は，4 KiB のページなら 1306 ページを占めて 2 KiB を無駄にするが，2 MiB のページなら 3 ページを占めて 6144 − 5222 = 922 KiB を無駄にする．

## 8.11 メモリの割り当て

アドレス変換は，仮想アドレスから物理アドレスへの対応付けを適用する．ページテーブルがその対応付けを記録し，MMU がすべてのアクセスをそれに照らして検査するが，対応付けそのもの–どのプロセスのどのページをどの物理メモリが裏づけ，どのメモリがまだ空いているか–を決めるのは別の構成要素である．

**出典** P3L2，メモリの割り当て，メモリ割り当ての課題；OSTEP ch. 17；Silberschatz ら ch. 9–10．

**定義 8.16** (メモリアロケータ). **メモリアロケータ**は，仮想メモリのある領域をどの物理メモリが裏づけるか，したがって仮想アドレスから物理アドレスへの対応付けを決定する．ページテーブルと MMU は，この対応付けを記録し，アクセスの正当性とその許可を検査するだけである．

▷ memory allocator

アロケータは 2 つの水準に存在する．

**カーネルレベルのアロケータ** プロセス制御ブロックのようなカーネルの状態のために，またプロセスの静的な状態–そのコード，スタック，初期化済みデータ–のために，メモリ領域すなわちページを割り当てる．システム内の空きメモリの管理も行う．

**ユーザレベルのアロケータ** `malloc` や `free` のような呼び出しを通じて，プロセスの動的な状態であるヒープを管理する．カーネルがプロセスにメモリの領域を与え，ユーザレベルのアロケータは，再びカーネルを介することなくその領域を個々の割り当てに分ける．dlmalloc，jemalloc，Hoard，tcmalloc などがその例である．

アロケータの主要な課題は，空きメモリの断片化である．

glibc の `malloc` は `brk` でヒープを伸ばすが，mmap しきい値を超える要求には専用の `mmap` を用いる．しきい値は初め 128 KiB であり，`M_MMAP_THRESHOLD` が設定されていなければ，glibc はこれを動的に引き上げる（64 ビットでは最大 32 MiB）．

**定義 8.17** (外部断片化). 空きメモリが連続しない穴に分かれ，総量としては十分なメモリが空いているにもかかわらず連続した領域の要求を満たせなくなるとき，空きメモリは**外部断片化**に悩まされる．

▷ external fragmentation

連続した物理メモリは，例えばラージページのために，デバイスが直接アクセスするバッファのために，そしてカーネルのデータ構造のために必要である．アロケータは外部断片化に 2 つの方法で対抗する．空きメモリが連続を保つように割り当てを配置すること，そしてメモリが解放されたときに，解放された領域を隣接する空き領域と併合してより大きな領域にすることである．

**例 8.18.** 12 個のページフレームからなるプールが，A（3 フレーム），B（2），C（3），D（2）の要求を受ける．その後 A と C が解放され，E の要求が連続した 4 フレームを必要とする（図 8.8）．A と C が離して配置されていれば，8 フレームが空いているものの，それは 3，3，2 フレームの穴に分かれており，E は失敗する．A と C が隣り合って配置されていれば，それらを解放すると 6 フレームの連続した穴が残り，E は成功する．




**図 8.8:** 12 フレームのプールにおける外部断片化．同じ要求と解放の並びが，アロケータが A と C をどこに配置したかに応じて，(a) 3 つの小さな穴を残すか，(b) 1 つの大きな穴を残す．

## 8.12 Linux カーネルのアロケータ

**出典** P3L2，Linux カーネルのアロケータ；OSTEP ch. 17；Silberschatz ら ch. 10；Bovet と Cesati ch. 8；Bonwick [23]．

Linux カーネルは 2 つのアロケータを組み合わせる．ページフレームのためのバディアロケータと，その上に載るカーネルオブジェクトのためのスラブアロケータである．

### 8.12.1 バディアロケータ

▷ buddy allocator

**定義 8.19** (バディアロケータ). **バディアロケータ**は，空きメモリを 2 の冪の大きさのブロックで管理する．要求は 2 の冪に切り上げられ，それを満たす最小の空きブロックから割り当てられる．要求された大きさのブロックができるまで，より大きなブロックは半分ずつに分割され，その一対をバディと呼ぶ．ブロックが解放され，そのバディも空いていれば，両者は併合され，併合は木を上へ向かって続く．

それでも断片化は起こりうるが，併合はよく働き，しかも速い．フレーム $a$ から始まる $2^k$ フレームのブロックのバディは，フレーム $a \oplus 2^k$ から始まる．すなわち 2 つのアドレスは 1 ビットだけ異なるので，アロケータは探索せずにバディを見つけられる．ただし 2 の冪への切り上げは内部断片化を生む．

Linux は 0–10 の次数，すなわち 1 個から 1024 個のフレーム（4 KiB から 4 MiB）ごとに空きリストを持つ．`/proc/buddyinfo` が次数ごとの空きブロック数を示す．

**例 8.20.** あるバディアロケータが 16 個のページフレームを管理する．表 8.3 は 5 つの操作をたどり，図 8.9 は 3 番目の後のブロックを示す．3 フレームの要求は 4 に，7 フレームの要求は 8 に切り上げられ，それぞれ 1 フレームを無駄にする．B が解放されると，そのバディ 6–7 は空いているので，両者は 4–7 に併合される．4–7 のバディは $0 = 4 \oplus 4$ よりブロック 0–3 であり，これも空いているので併合は 0–7 まで続く．そのバディ 8–15 は割り当て済みなので，そこで止まる．

**表 8.3:** 16 個のページフレームを持つバディアロケータのトレース．

| ステップ | 操作 | 分割・併合 | 空きブロック |
|---|---|---|---|
| 0 | — | — | 0–15 |
| 1 | A を割り当て（3 → 4） | 0–15，0–7 を分割；A = 0–3 | 4–7，8–15 |
| 2 | B を割り当て（2） | 4–7 を分割；B = 4–5 | 6–7，8–15 |
| 3 | C を割り当て（7 → 8） | C = 8–15 | 6–7 |
| 4 | A を解放 | 4–7 は分割済み，併合なし | 0–3，6–7 |
| 5 | B を解放 | 4–5 を 6–7 と，次に 0–3 と併合 | 0–7 |

### 8.12.2 スラブアロケータ

[2] Bonwick が最初のスラブアロケータを書いたのは Solaris 2.4 のためである．Linux にはこの考えの実装が 3 つあった—SLAB，SLUB，小規模システム向けの SLOB．2.6.23 以降の既定である SLUB は，SLOB が Linux 6.4 で，SLAB が 6.8 で削除された現在，唯一の実装である．

▷ slab allocator

カーネルのオブジェクトの大きさは 2 の冪ではないので，バディアロケータはそれらに与えるメモリの多くを無駄にしてしまう．そこで Linux カーネルは，自分のオブジェクトをスラブアロケータ [23] で割り当てる．[2]

**定義 8.21** (スラブアロケータ). **スラブアロケータ**は，よく使われるオブジェクトの型または大きさごとにキャッシュを持つ．各キャッシュは**スラブ**，すなわち連続して割り当てられた物理メモリの領域からなり，スラブはオブジェクトの大きさちょうどのスロット

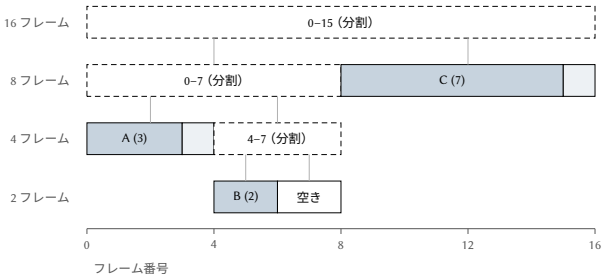



**図 8.9:** 例 8.20 のバディアロケータの，ステップ 3 の後のブロック．括弧内は要求されたフレーム数である．破線のブロックは分割されており，薄く塗ったフレームは内部断片化で失われている．

に分割される．割り当ては，そのオブジェクトの型のキャッシュから空きスロットを取り，解放はそのスロットをキャッシュへ返す．

スロットはオブジェクトの大きさちょうどなので，スラブアロケータはバディアロケータの切り上げによる無駄を避けられる—使われないのはスラブ末尾の余りだけである—し，外部断片化も問題にならない．解放されたスロットは，同じ型の次のオブジェクトにちょうど合うからである（図 8.10）．その下にあるスラブはバディアロケータから得る．

各キャッシュは CPU ごとの空きスロットのリストも持つ．割り当ての大半は共有された状態に触れず，ロックを必要としない（第 10 章）．

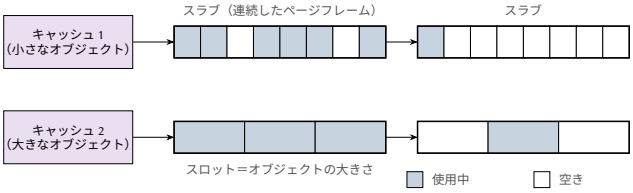



**図 8.10:** 小さなオブジェクト用と大きなオブジェクト用のキャッシュを持つスラブアロケータ．どのスラブもオブジェクトの大きさのスロットに分割される．

**例 8.22.** あるカーネルオブジェクトが 1200 バイトを占めるとする．バディアロケータから個別に割り当てると，各オブジェクトは少なくとも 4 KiB の 1 ページを受け取り，2896 バイトを無駄にする．1 ページのスラブはこのようなオブジェクトを $\lfloor 4096/1200 \rfloor = 3$ 個保持し，$4096 - 3600 = 496$ バイトを未使用のまま残す．オブジェクト当たり約 165 バイトである．

カーネルの構造体は `kmem_cache_create` で自分のキャッシュを得る．`task_struct`（第 2 章の PCB）も専用のキャッシュを持つ．`/proc/slabinfo` が各キャッシュとそのオブジェクトの大きさを列挙する．

> **要点**
> Linux はページフレームをバディアロケータで割り当てる．その 2 の冪のブロックは探索なしに見つけて併合できるが，要求は切り上げられる．カーネルオブジェクトはスラブアロケータで割り当てる．そのスロットはオブジェクトの大きさを持ち，同じ型の次のオブジェクトに再利用される．

## 8.13 デマンドページング

仮想メモリは物理メモリよりはるかに大きいので，仮想ページが常にページフレームに存在しているとは限らない．ページフレームの内容は二次記憶へ退避され，そこから復元される．

**出典** P3L2，デマンドページング；OSTEP ch. 21；Silberschatz ら ch. 10．

**定義 8.23 (スワッピングとデマンドページング).** ページの内容を物理メモリと二次記憶との間で移動することを**スワッピング**と呼ぶ．ページは**スワップ領域**に格納される．これはディスク，フラッシュメモリ，あるいは遠隔マシンのメモリの一部である．**デマンドページング**では，ページがメモリへスワップインされるのは，プロセスがそれにアクセスしたときだけである．

▷ swapping
▷ swap partition
▷ demand paging

プロセスがメモリにないページにアクセスすると，次の手順が踏まれる（図 8.11）．

1. プロセスがメモリアドレスを参照し，MMU はその PTE の有効ビットがクリアされていることを見つける．
2. MMU がページフォールトを起こし，オペレーティングシステムへトラップする．
3. オペレーティングシステムは，そのアクセスが正当であること，およびページがスワップ領域のどこに格納されているかを判断する．
4. I/O 操作を用いて，スワップ領域からページを空いているページフレームへ読み込む．
5. 新しい PFN で PTE を更新し，有効ビットを立てる．
6. プログラムカウンタをフォールトした命令に戻してプロセスを続行させる．参照は再び実行され，今度は成功する．

スワップインの後にページが占めるページフレームは，一般に以前占めていたものとは異なる．空いているページフレームがなければ，オペレーティングシステムはまず 1 つを解放しなければならない（8.14 節）．

**注意** Silberschatz ら（§9.5）は，プロセス全体についてはスワッピング，個々のページについてはページングという語を用いる．講義と Linux はページについてスワップという．

後の例の 8 ms は回転ディスクの値である．NVMe SSD は数十マイクロ秒（典型的なベンダ仕様値）でページを読むので，許容できるフォールト率は 2 桁高くなる．

スワップしてはならないページもある．**ピン留めページ** とは，スワッピングが禁止され，物理メモリに留まり続けるページである．例えば，ダイレクトメモリアクセス（第 11 章）の際にデバイスが物理アドレスで直接アクセスするページはピン留めされる．

▷ pinned page

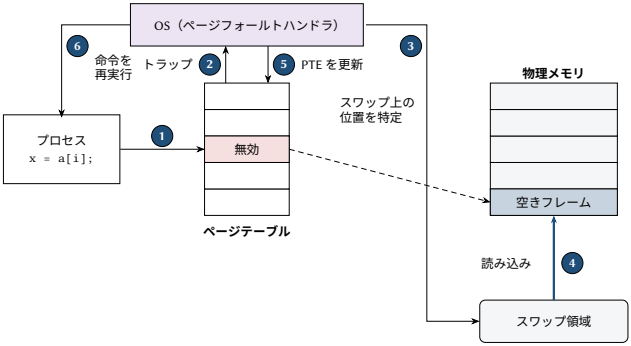



**図 8.11:** デマンドページング．プロセスがスワップ領域にあるページを参照したときに踏まれる手順．

**例 8.24.** メモリアクセスに 100 ns，ディスクからページを読むページフォールトの処理に 8 ms，すなわち $8 \cdot 10^6$ ナノ秒，つまり 80 000 倍の時間がかかるとする．アクセスのうち割合 $p$ がフォールトするなら，平均のアクセスは $(1-p) \cdot 100 + p \cdot 8 \cdot 10^6$ ナノ秒かかる．$p = 10^{-4}$ ではこれは約 900 ns であり，9 倍遅い．100 ns の 10 % 以内に収めるには，$p$ は $1.25 \cdot 10^{-6}$，すなわち 800 000 回のアクセスに 1 回のフォールトを超えてはならない．

## 8.14 ページ置換

出典 P3L2，物理メモリの解放；OSTEP ch. 22；Silberschatz ら ch. 10；Bovet と Cesati ch. 17.

物理メモリが不足すると，オペレーティングシステムはページをスワップアウトしてページフレームを解放しなければならない．スワップアウトされたページがすぐにまた必要になると，ページフォールトと I/O 操作の代価がかかる．8.13 節の例が示したように，これは高くつく．したがってオペレーティングシステムは，スワップアウトするページを注意深く選ばなければならない．この選択を**ページ置換**と呼び，そこには 2 つの問いがある．いつページをスワップアウトすべきか，そしてどのページをスワップアウトすべきかである．

▷ page replacement

▷ page-out daemon

注意 用語が異なる．講義の高水位はメモリ使用量，低水位は CPU 使用率のしきい値である．Linux の min/low/high の水位はすべて空きメモリのしきい値である（low を下回ると回収が始まり，high を上回ると止まる）．

ページをスワップアウトするのは**ページアウトデーモン**である．これは，メモリの使用量がしきい値である**高水位**を超えたときに走るカーネルスレッドである．アプリケーションをできるだけ妨げないよう，CPU の使用率がしきい値である**低水位**を下回っている間に走ることが望ましい．

理想的な犠牲ページは，将来使われないページである．将来は分からないので，オペレーティングシステムはアクセスの履歴からそれを予測する．

- **LRU**（最長時間未使用）方針は，最後のアクセスが最も過去にあるページをスワップアウトする．最近使われていないページは近いうちにも使われないだろう，という仮定に基づく．PTE のアクセスビットが，ビットが最後にクリアされてからそのページが参照されたかどうかを教える．
- 書き出す必要のないページは置換の代価が小さい．ダーティビットは，ページが変更されたかどうかを教える．すでにディスクに格納されているクリーンなページの内容は，単に捨ててよい．
- ピン留めページやカーネルの特定のページのように，メモリに留まらなければならないページは決して選ばれない．

▷ least recently used

Linux のページアウトデーモンはカーネルスレッド kswapd である．水位は /proc/zoneinfo に，スワップデバイスは /proc/swaps に示される．

Linux は，回収が始まり，また止まるタイミングを調整するパラメータを備え，ページを回収可能なページやスワップ可能なページといった種類に分類し，ページに二度目の機会を与える LRU の変種を用いる．

**定義 8.25** (セカンドチャンスアルゴリズム). **セカンドチャンスアルゴリズム**は，アクセスビットを用いて LRU を近似する．ページフレームを巡回順に走査する．アクセスビットが立っているページには二度目の機会が与えられる．そのビットはクリアされ，走査は次へ進む．アクセスビットがクリアされているページ，すなわち前回の走査がそのビットをクリアしてから参照されていないページが，スワップアウトされる．

▷ second chance algorithm

**例 8.26.** 5 個のページフレームがページ A–E を保持し，そのアクセスビットは 1，0，1，1，0 で，走査は A から始まる．表 8.4 は 2 回の置換をたどる．2 回目の走査は F を受け取ったフレームの次から再開するので，C，D，E だけを調べる．C と D には二度目の機

会が与えられ，ビットがクリアされている E がスワップアウトされる．A はまったく調べられない．最初の走査がそのビットをクリアした後にプロセスが再び A を参照したので，次に走査が A に達したとき，A には二度目の機会が与えられる．そして，ステップ 3 でビットがクリアされ，その後参照されていない C が犠牲となる．

**表 8.4:** セカンドチャンスアルゴリズムのトレース．括弧内は読み込まれるページである．アクセスビットは各ステップの後の値を走査順に並べてある．新たに読み込まれたページはビットが立っている．

| ステップ | 事象 | 調べたページ | アクセスビット |
|---|---|---|---|
| 0 | — | — | A1 B0 C1 D1 E0 |
| 1 | フォールト（F） | A クリア，B スワップアウト | A0 F1 C1 D1 E0 |
| 2 | A と D を参照 | — | A1 F1 C1 D1 E0 |
| 3 | フォールト（G） | C，D クリア，E スワップアウト | A1 F1 C0 D0 G1 |

**要点**
デマンドページングは，最初にそのページを必要とするページフォールトの際にページを持ち込み，ページアウトデーモンはメモリが尽きる前にフレームを解放する．どのページが去るかは，アクセスビットを読んでクリアするセカンドチャンスアルゴリズムのような LRU の近似で決められる．ダーティビットは，犠牲となるページをそもそも書き出す必要があるかどうかを教える．

## 8.15 コピーオンライト

MMU はアドレスを変換し，アクセスを追跡し，保護を強制する．これらの能力は，他のサービスや最適化を作るのにも役立つ．その 1 つは，プロセスが生成されるときの不要なコピーを避ける．

出典 P3L2，コピーオンライト；OSTEP ch. 23；Silberschatz ら ch. 10．

fork で生成されたプロセスは，親のアドレス空間全体のコピーを受け取る（第 2 章）．そのページの多くは，コードのように静的で決して変わらず，しかも子はしばしば生成の直後に exec を呼ぶので，コピーの大半は無駄になってしまう．

**定義 8.27 (コピーオンライト).** **コピーオンライト**（COW）では，オペレーティングシステムはプロセスを生成するときにアドレス空間をコピーしない．新しいプロセスの仮想ページを元のプロセスのページフレームに対応付け，両方のプロセスのページテーブルでそれらのページを読み出し専用にする．そのようなページへの書き込みはページフォールトを起こし，そのときになって初めてオペレーティングシステムはそのページをコピーする．

▷ copy-on-write

両方のプロセスが共有ページを読むだけである限り，メモリも，コピーの時間も費やされない．どちらかがページに書き込むと，保護フォールトによってオペレーティングシステムはそれが COW ページであることを認識する．そしてこの 1 ページだけを新しいページフレームへコピーし，書き込んだプロセスのページを書き込み許可付きでそのコピーに対応付け，書き込みをやり直させる（図 8.12）．コピーの代価は，実際に変更されたページに対してだけ支払われる．

書き込まれる私的な領域のページテーブルは，それでもコピーされる．4 KiB のページで 1 GiB なら 8 バイトのエントリ $2^{18}$ 個，2 MiB である．

**例 8.28.** 4 KiB のページ 25 600 個（100 MiB）のアドレス空間を持つプロセスが fork し，子は exec を呼ぶ前にスタックとデータの 40 ページに書き込むとする．コピーオンライトは 25 600 ページではなく 40 ページ，すなわちアドレス空間の 0.16 % をコピーする．

**注意** 節約は無条件ではない．最初の書き込みはいずれもカーネルへのトラップを伴い，その代価は現在のハードウェアでマイクロ秒の程度である（典型的な値）．ほとんどすべてのページに書き込む子では，先にアドレス空間をコピーしてしまう方が速い．

## 8.16 チェックポインティング

同じハードウェアによる支援は，障害管理にも役立つ．

出典 P3L2，障害管理：チェックポインティング；OSTEP では扱われていない；ライブマイグレーションは Clark ら [24]．

**定義 8.29 (チェックポインティング).** プロセスの状態を定期的に保存する障害・回復管理の技法を**チェックポインティング**と呼ぶ．障害の後，プロセスは最初からではなく，最新のチェックポイントから再開される．

▷ checkpointing

障害を排除することはできないが，チェックポイントは失われる作業の量を限定し，それによって回復を短くする．最も単純な方法は，プロセスを停止して状態全体をコピーすることである．大きなアドレス空間では停止が長くなり，繰り返すなら代価も高い．MMU はより良い方法を可能にする．

同じ書き込み保護とフォールトの手法は，分散共有メモリにおいて書き込みを検出するのにも使われる（第 15 章）．

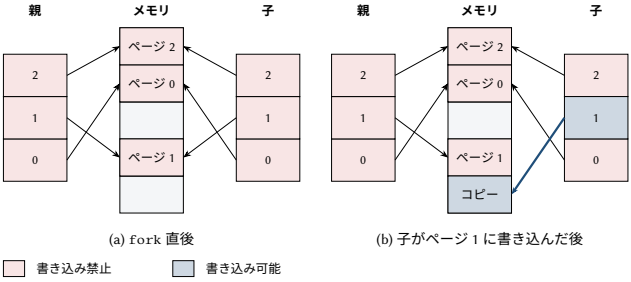



**図 8.12:** コピーオンライト．(a) fork の後，親と子は同じ書き込み禁止のページフレームを対応付ける．(b) 子のページ 1 への書き込みがフォールトし，オペレーティングシステムはそのページだけをコピーする．

- オペレーティングシステムはプロセスのすべてのページを書き込み禁止にし，プロセスを走らせたまま状態全体を一度コピーする．
- それ以降は，書き込みフォールトまたはダーティビットによって，プロセスがどのページを変更したかを記録し，後のチェックポイントはこれらの汚れたページだけをコピーする．このような**増分**チェックポイントは，前のチェックポイントとの差分を格納する．
- プロセスを復元するには，完全なコピーとそれに続く差分から状態を再構成する．差分はバックグラウンドで併合しておくこともできる．

チェックポインティングは，さらなるサービスの基礎となる．

**デバッグ　巻き戻し再実行**では，プログラムをチェックポイントから再開（巻き戻し）して実行し直す．誤りが再現しなければ，見つかるまで順により古いチェックポイントを使う．

▷ process migration

**マイグレーション　プロセスマイグレーション**では，プロセスのチェックポイントを別のマシンへ転送し，そこでプロセスを続行する．マイグレーションは，災害復旧や，プロセスをより少ないマシンへ集約するのに役立つ．

マイグレーションの停止時間を短く保つため，チェックポイントは速い繰り返しで行われる．オペレーティングシステムは，プロセスを走らせたまま全ページをコピーし，次にそのコピーの間に汚れたページを，次に 2 回目のコピーの間に汚れたページを，というように続ける．残るページが十分に少なくなって最後に停止してコピーすることが許容できるようになるまで–あるいは，プロセスがコピーと同じ速さでページを汚すために停止が避けられなくなるまで–これを繰り返す．

Clark ら [24] は，実行中の Xen の仮想マシンを，ウェブサーバで 210 ms，ゲームサーバで 60 ms の停止時間で移送した．

この繰り返しはプリコピーマイグレーションと呼ばれる．ポストコピーはその逆で，まず移送先でプロセスを再開し，ページはネットワーク越しにフォールトで取り寄せる．

**例 8.30.** 50 000 ページのプロセスを，毎秒 25 000 ページをコピーする接続を通じてマイグレーションする．その間プロセスは毎秒 2000 個の異なるページを汚す．プロセスを停止してすべてをコピーすると 2 s かかる．各ラウンドが前のラウンドの間に汚れたページをコピーする繰り返しでは，どのラウンドも直前のラウンドの 8 % のページをコピーし，3 ラウンドの後には最後の停止は約 1 ms で済む．

| ラウンド | コピーしたページ数 | 所要時間 | その間に汚れたページ数 |
|---|---|---|---|
| 1（実行中） | 50 000 | 2 s | 4000 |
| 2（実行中） | 4000 | 160 ms | 320 |
| 3（実行中） | 320 | 12.8 ms | 26 |
| 最後（停止） | 26 | 1.04 ms | – |

**要点 — 章のまとめ**
仮想メモリはアドレス空間を物理メモリから切り離す．オペレーティングシステムはメモリを割り当て，すべてのアクセスを調停する．そのためにページングまたはセグメンテーションと，アドレスを変換しフォールトを起こす MMU とを用いる．ページテーブルは VPN を PFN に対応付け（式 (8.1)），そのエントリは有効，ダーティ，アクセス，保護の各ビットを持つ．ページフォールトは，MMU が完了できないアクセスをオペレーティングシステムに委ねる．平坦なページテーブルは大きすぎるので（式 (8.2)），多段にするか，逆ページテーブルとハッシュ法を用いる．そして TLB が変換を高速に保つ（式 (8.3)）．大きなページは，内部断片化を代価として，ページテーブルを小さくし TLB の到達範囲を広げる．アロケータは外部断片化と戦い，Linux はページフレームにバディアロケータを，カーネルオブジェクトにスラブアロケータを用いる．デマンドページングはページフォールトの際にページをスワップインし，ページアウトデーモンはセカンドチャンスアルゴリズムのような LRU の近似でページを選んでスワップアウトする．MMU はさらに，コピーオンライトと増分チェックポインティングを可能にし，後者はデバッグやプロセスマイグレーションを支える．

第 9 章

# プロセス間通信

| | |
|---|---|
| **講義動画** | P3L3 |
| **教科書** | Silberschatz ら [2] ch. 3（§§3.4–3.8）；Kerrisk [11] ch. 43–54，56–57 |
| **キーワード** | プロセス間通信，メッセージパッシング，ポート，パイプ，メッセージキュー，ソケット，共有メモリ，コピーとマップ，ローカル手続き呼び出し，共有メモリセグメント，System V IPC，POSIX 共有メモリ，メモリマップドファイル，プロセス共有ミューテックス |

第 2 章では，それぞれ独自のアドレス空間に隔離されたプロセスがデータをやり取りする方法としてプロセス間通信（IPC）を導入し，メッセージパッシングと共有メモリを区別した．この章では，オペレーティングシステムが両者のために提供する機構を調べる．共有メモリは第 8 章のアドレス変換の上に成り立つ．アドレス変換によってオペレーティングシステムは同じ物理メモリを複数のアドレス空間に対応付けられる．共有メモリはやり取りのたびにカーネルが介在することを避けるが，メッセージパッシングではカーネルが担う同期と通信プロトコルをプロセスに委ねる．

## 9.1 プロセス間通信

**出典** P3L3，プロセス間通信；Silberschatz ら ch. 3.

IPC とは，プロセスどうしの相互作用のためにオペレーティングシステムがサポートする機構の総称であり，あるアドレス空間から別のアドレス空間へデータを移す**通信**と，プロセスの動作の**協調**からなる．IPC が必要になるのは，オペレーティングシステムがプロセスのアドレス空間を互いに隔離しているからである（第 2 章）．どのプロセスも他のプロセスのメモリを読み書きできないので，データのやり取りはすべて，オペレーティングシステムが提供する機構を通る．これらの機構は次の 4 つに大別される．

**メッセージパッシング** プロセスは，カーネルが維持するチャネルを通じてメッセージをやり取りする．ソケット，パイプ，メッセージキューなどである．

**メモリベースの IPC** オペレーティングシステムが複数のプロセスからメモリにアクセスできるようにし，プロセスはそれを読み書きして通信する．共有メモリやメモリマップドファイルなどである．

**より高水準のセマンティクス** データ転送により多くの構造を与える機構．複数のプロセスが読み書きするファイルや，遠隔手続き呼び出し（第 13 章）である．

**同期プリミティブ** ミューテックスやセマフォなど，プロセスが協調するために用いる構成要素（第 10 章）である．

この章では，メッセージパッシングと共有メモリ，そして共有メモリが必要とする同期を中心に扱う．

## 9.2 メッセージベースの IPC

**出典** P3L3，メッセージベースの IPC；Silberschatz ら ch. 3.

メッセージベースの IPC では，プロセスがメッセージを作り，オペレーティングシステムが作成して維持するチャネル—典型的にはカーネルメモリ内の FIFO キューとして構成されたバッファ—を通じて送受信する．プロセスがこのバッファに直接アクセスすることはない．

▷ port

**定義 9.1** (ポート). **ポート**とは，プロセスがメッセージパッシングのチャネルを利用するためのインタフェースである．プロセスはポートへメッセージを送り（書き），ポートからメッセージを受け取り（読み），カーネルがポートの間でメッセージを移動させる．

x86-64 では何もしないシステムコールでも 100 ns の桁の時間がかかり（lmbench の `lat_syscall`），Meltdown 対策の KPTI があるとその倍になりうる．

カーネルは，通信の確立にも，個々の IPC 操作のすべてにも関与する．送信は，メッセージを送信側のアドレス空間からチャネルへコピーするシステムコールであり，受信は，それをチャネルから受信側のアドレス空間へコピーするシステムコールである．したがって，クライアントとサーバの間の要求・応答のやり取りには，ユーザ/ カーネル境界の横断が 4 回とデータコピーが 4 回必要になる（図 9.1）．

メッセージパッシングの利点は単純さである．チャネルの管理と同期はカーネルが行う—例えば空のチャネルに対する受信は，メッセージが届くまでブロックする．欠点はオーバーヘッドであり，境界の横断とコピーの代価をメッセージごとに支払う．

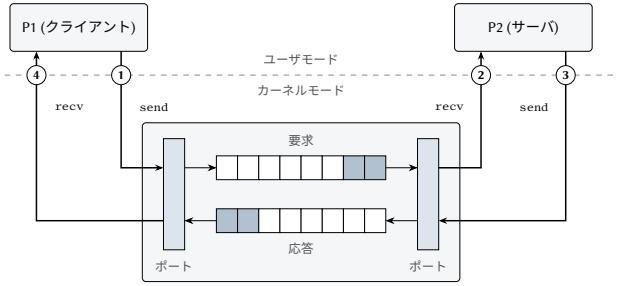



**図 9.1:** メッセージベースの IPC による要求・応答のやり取り．番号を付したユーザ/カーネル境界の横断はそれぞれ，1 回のシステムコールと，プロセスのアドレス空間とカーネルのチャネルとの間の 1 回のコピーである．

## 9.3 メッセージパッシングの形態

Unix 系システムは 3 つの代表的なメッセージパッシングの形態を提供する．それらは何を転送するかと，カーネルがどれだけの処理を行うかが異なる．

**出典** P3L3，メッセージパッシングの形態；OSTEP ch. 5；Silberschatz ら ch. 3；Kerrisk ch. 44，46，52，56–57.

**定義 9.2** (パイプ). **パイプ**とは，バイトストリームを運ぶ，2 つの端点の間のチャネルである．一方の端に書かれたバイトは，メッセージ境界を持たずに，同じ順序で他方の端から読まれる．

▷ pipe

POSIX の `pipe(fd)` 呼び出しはパイプを作り，各端のファイルディスクリプタを 2 要素の配列 `fd` に格納する．典型的な用途は，シェルが `ls | wc -l` に対して行うように，あるプロセスの出力を別のプロセスの入力につなぐことである．

このようなパイプには名前がない．使えるのは，そのディスクリプタを持つプロセス，すなわち作成したプロセスと，`fork` によってディスクリプタを継承した子だけである．`mkfifo(path, mode)` で作る**名前付きパイプ**（FIFO）はファイルシステム内に名前を持ち，無関係なプロセスもその名前を通じて，ファイルと同様に `open` で開ける．データはやはりカーネルのバッファを通り，ディスクには書かれない．

Linux のパイプは既定で 16 ページ，64 KiB を保持し，`fcntl` の `F_SETPIPE_SZ` で `pipe-max-size`（1 MiB）まで増やせる．`PIPE_BUF` = 4 kB 以下の書き込みはアトミックである（`pipe(7)`）.

▷ named pipe

**定義 9.3** (メッセージキュー). **メッセージキュー**とは，プロセスの間で離散的なメッセージを運ぶチャネルである．送信側は正しく整形されたメッセージを渡し，受信側は完全なメッセージを受け取る．

▷ message queue

カーネルはメッセージの始まりと終わりを知っているので，メッセージを管理できる．例えば，送信側が付けた優先度の順に配送したり（POSIX），受信側が型でメッセージを選べるようにしたり（System V）できる．Unix 系システムはメッセージキューに System V と POSIX の 2 つのインタフェースを提供する（表 9.1）．System V では，`msgget` がキーに結び付いたキューを作成または開き，その識別子を返す．どのメッセージも `long` 型の型フィールドで始まり，`msgrcv` は型 0 ではキューの先頭のメッセージを，正の型ではその型の最初のメッセージを取り出す．POSIX では，`mq_open` が名前でキューを開いてディスクリプタを返し，`mq_receive` は常に最も優先度の高いメッセージのうち最も古いものを返す．共有メモリと同様に，キューは `IPC_RMID` を指定した `msgctl` か `mq_unlink` で明示的に取り除かれるまで存続する．`mq_close` はディスクリプタを解放するだけである．

**ヒント** `ls -l /proc/self/fd | cat` を実行すると，2 つのコマンドをつなぐパイプが `ls` のファイルディスクリプタ 1 として `1 -> pipe:[…]` のように表示される．

Linux の既定値は，System V のメッセージ 1 つが 8 kB，1 つのキューが 16 kB，POSIX のキューが 10 メッセージである（`msgsnd(2)`, `mq_overview(7)`）.

**表 9.1:** 2 つのインタフェースのメッセージキュー呼び出し．`m` はメッセージのバッファ，`n` はその長さ，`flg` はフラグである．

| 操作 | System V | POSIX |
|---|---|---|
| 作成・オープン | `msgget(key, flg)` | `mq_open(name, flg, …)` |
| 送信 | `msgsnd(id, m, n, flg)` | `mq_send(q, m, n, prio)` |
| 受信 | `msgrcv(id, m, n, type, flg)` | `mq_receive(q, m, n, &prio)` |
| クローズ | — | `mq_close(q)` |
| 破棄 | `msgctl(id, IPC_RMID, NULL)` | `mq_unlink(name)` |

▷ socket

**定義 9.4 (ソケット).** **ソケット**とは，socket() システムコールで作られる通信の端点である．この呼び出しはカーネルレベルのソケットバッファを確保し，TCP/IP プロトコルスタックなど，その通信に必要なカーネルレベルの処理をそれに結び付ける．

▷ stream socket
▷ datagram socket

send() と recv() は，ユーザバッファからソケットバッファへ，またソケットバッファから外へデータを渡す．**ストリームソケット**（TCP）はバイトストリームを，**データグラムソケット**（UDP）は離散的なメッセージを配送する．2 つのストリームソケットの間の接続は非対称に確立される．サーバは socket() でソケットを作り，bind() でアドレスを割り当て，listen() で接続を受け付ける状態にし，到着する接続を 1 つずつ accept() で受け取る．accept() はそのクライアント用の新しいソケットを返す．クライアントはソケットを作り，connect() でサーバのアドレスに接続する．データグラムソケットには接続が要らない．sendto() と recvfrom() がメッセージごとに相手を指定する．

socket() で選ぶアドレスファミリがデータの経路を決める．AF_INET では，チャネルは TCP/IP プロトコルスタックを介してプロセスをネットワークデバイスに接続する．同じマシン上のプロセスの間でも，ループバックアドレス 127.0.0.1 を通るデータはスタック全体を通過し，省かれるのはネットワークデバイスだけである．ファイルシステム内のパスで名前付けされる **Unix ドメインソケット**（AF_UNIX）は，同じマシン上のプロセスを接続し，ネットワークプロトコルスタックを完全に迂回する．表 9.2 に 3 つの形態を比較する．

▷ Unix-domain socket

**表 9.2:** メッセージパッシングの 3 つの形態．

| | パイプ | メッセージキュー | ソケット |
|---|---|---|---|
| 転送の単位 | バイトストリーム | メッセージ | ストリーム/データグラム |
| カーネルの処理 | バッファリング | 境界，優先度 | TCP/IP または Unix ドメイン |
| マシン | 同一 | 同一 | 同一または別 |
| インタフェース | pipe() | System V，POSIX | socket() |

## 9.4 共有メモリ IPC

**出典** P3L3，共有メモリ IPC；Silberschatz ら ch. 3.

共有メモリ IPC では，複数のプロセスが，そのすべてからアクセスできるメモリ領域を読み書きする．オペレーティングシステムは，同じ物理ページをプロセスの仮想アドレス空間に対応付けることでこのチャネルを確立する．すなわち，参加するすべてのプロセスのページテーブルエントリが同じページフレームを指す（図 9.2）．ページングによる対応付け一般と同様に（第 8 章），プロセスが領域を見る仮想アドレスは同じである必要はなく，物理フレームも連続である必要はない．

**注意** $VA_1 \neq VA_2$ であるから，一方のプロセスが領域に格納したポインタは他方では有効でない．

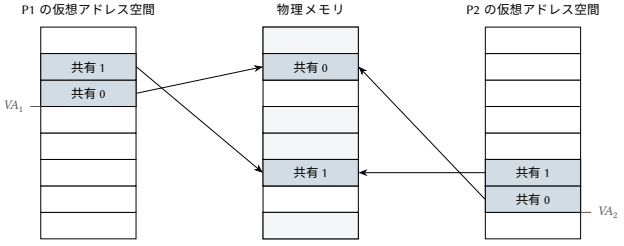



**図 9.2:** 共有メモリ．2 つのページフレームが両方のアドレス空間に対応付けられている．P1 はこの領域を仮想アドレス $VA_1$ に，P2 は別のアドレス $VA_2$ に見ており，フレームは物理メモリ上で隣接していない．

対応付けができてしまえば，オペレーティングシステムは介在しない．読み書きは通常のメモリアクセスであり，システムコールが必要なのは設定のときだけである．データコピーは削減されうるが，なくなるわけではない．プロセスが共有領域の中で生成したデータは直ちに他のプロセスから見えるが，そのアドレス空間の別の場所にすでに存在するデータは，やはり領域へコピーしなければならない．

対応付けが動かすのはデータではなくページテーブルエントリである（第 8 章）．2 MB のヒュージページなら，1 つのエントリで 512 個の基本ページを覆う（SHM_HUGETLB）．

代償として，オペレーティングシステムはメッセージパッシングに対して提供していたサービスをもはや行わない．プロセスは並行アクセスを明示的に同期しなければならず（9.8 節），通信プロトコル–データの形式，受信側がデータの到着をどう知るか，領

域内の空間をどう管理するか–はプログラマの責任である（9.10 節）．Unix と Linux は，System V 共有メモリ（9.6 節），POSIX 共有メモリ（9.7 節），およびメモリマップドファイルによって共有メモリをサポートする．Android は独自の機構 ashmem を加えている．[1]

[1] ashmem の領域はアンピンできる．メモリ逼迫時にカーネルはその捨ててよい内容を回収することがあり，再びピンすると破棄されたかが分かる．Android はこの用途を `memfd_create` へ移しつつある．

## 9.5 コピーとマップ

出典 P3L3，コピーとマップ；Silberschatz ら ch. 3.

メッセージパッシングと共有メモリは，あるアドレス空間から別のアドレス空間へデータを移すという同じ目的を追うが，CPU 時間を費やす操作が異なる．

**コピー（メッセージパッシング）** CPU 時間は，データをポートへ，そしてポートから外へコピーすることに費やされ，送信と受信のたびに–すなわちメッセージごとに–システムコールを伴う．

**マップ（共有メモリ）** CPU 時間は，物理メモリをアドレス空間に対応付けること–領域を設定するときの 1 回だけ–と，領域の中で生成されなかったデータを領域へコピーすることに費やされる．

対応付けは比較的高価である．1 回設定して多数の転送に使えば，第 2 章の例が示したように元が取れる．大きなデータであれば，1 回の転送でも元が取れうる．大量のデータをあるアドレス空間から別のアドレス空間へコピーする時間は，それを対応付ける時間よりはるかに長いからである: $t_{\text{copy}} \gg t_{\text{map}}$．単純なコストモデルによって，この 2 つの主張を精密にできる．システムコールのコストを $t_{\text{sys}}$，コピーのコストを 1 バイト当たり $c$ とし，$S$ バイトの転送があるプロセスから別のプロセスへ $n$ 回行われるとする．メッセージパッシングは転送ごとに 2 回のシステムコールと 2 回のコピーを必要とする．共有メモリは 1 回限りの設定コスト $t_{\text{setup}}$ を必要とし，転送ごとには，領域への 1 回のコピー–受信側はそこからデータをその場で読む–と，プロセスがアクセスの同期と互いへの通知に費やす時間 $t_{\text{sync}}$（9.8 節）を必要とする．

$$T_{\text{copy}}(n) = n(2t_{\text{sys}} + 2cS), \qquad T_{\text{map}}(n) = t_{\text{setup}} + n(cS + t_{\text{sync}}). \tag{9.1}$$

$t_{\text{sync}} < 2t_{\text{sys}} + cS$ であれば，共有メモリは $n > n^* = t_{\text{setup}}/(2t_{\text{sys}} + cS - t_{\text{sync}})$ のとき安く，そうでなければ決して追いつかない．競合のないプロセス共有ミューテックスが領域を保護し，受信側がブロックしない場合，同期コストはほぼゼロであるが，ブロックして起こされるプロセスはシステムコールを追加する．設定コストは，固定部分と，対応付けるページ数に比例する部分からなる．1 ページを対応付けるコストがその内容をコピーするコストより小さい限り，$n^*$ は $S$ が大きくなるにつれて下がり，大きなデータでは 1 を下回る．

ヒント 例 9.5 の定数で $n^* \leq 1$ を解くと $S \geq 640\,\text{kB}$ となる．これを超えれば 1 回の転送だけで対応付けの元が取れる．

**例 9.5.** $t_{\text{sys}} = 1\,\mu\text{s}$，$c$ を 1 キロバイト当たり $0.1\,\mu\text{s}$ とし，設定コストを $50\,\mu\text{s}$ に 4 kB ページ当たり $0.1\,\mu\text{s}$ を加えたものとする．$S = 40\,\text{MB}$（$10^4$ ページ）の 1 回の転送は，メッセージパッシングでは $2 + 2 \times 4000 = 8002\,\mu\text{s}$ かかるが，$t_{\text{sync}} \approx 0$ とすると共有メモリでは $1050 + 4000 = 5050\,\mu\text{s}$ であり，送信側がデータを領域の中で直接生成するなら設定の $1050\,\mu\text{s}$ だけで済む．$n^* = 1050/4002 \approx 0.26$ である．ブロックする待機と起床（$t_{\text{sync}} = 2\,\mu\text{s}$）があってもこれは変わらない（$n^* = 1050/4000 \approx 0.26$）が，$S = 4\,\text{kB}$ では $n^*$ が $50.1/2.4 \approx 21$ から $50.1/0.4 \approx 125$ へ上がる．

1 キロバイト当たり $0.1\,\mu\text{s}$ は $10\,\text{GB}\,\text{s}^{-1}$ であり，1 つのコアが主記憶に対して持つコピー帯域の桁である．

Windows は，同じマシン上のプロセスの間の通信のための機構である**ローカル手続き呼び出し**（LPC）でこのトレードオフを利用している．小さなメッセージはポートを介してコピーし，より大きなデータに対しては共有メモリ領域を両方のプロセスに対応付け，それを通じてデータをやり取りする．

▹ local procedure call

現在の Windows はこれを ALPC として実装している（Silberschatz ら ch. 3）．

**要点**
メッセージパッシングは単純である–カーネルがチャネルを管理し，プロセスを同期する–が，メッセージごとにシステムコールと 2 回のコピーの代価を払う．共有メモリは 1 回限りの対応付けのコストを払った後はカーネルを介さずに済み，その代わりに明示的な同期とプログラマが定めるプロトコルを要する．小さなメッセージはコピーするほうがよく，大きなデータや再利用されるデータは対応付けるほうがよい．

## 9.6 System V 共有メモリ

▹ System V IPC

出典 P3L3，SysV 共有メモリ，SysV 共有メモリ API；Kerrisk ch. 45，48.

最も古い Unix の IPC インタフェースの 1 つである **System V IPC** は，メッセージキュー，セマフォ集合，共有メモリからなり，各オブジェクトをキーで識別する．共有メモリはセグメントとして構成する．

▷ shared memory segment

**定義 9.6 (共有メモリセグメント).** **共有メモリセグメント**とは，必ずしも連続したページからなるとは限らない物理メモリの領域であって，オペレーティングシステムがシステム全体のオブジェクトとして管理し，任意個数のプロセスが自分のアドレス空間に対応付けられるものである．

/proc/sys/kernel にある Linux の既定値は，shmmni が 4096 セグメントであり，shmmax と shmall は Linux 3.16 以降事実上無制限である（それ以前のカーネルでは shmmax が 32 MiB）（shmget(2)，sysvipc(7)）．

セグメントはシステム全体のオブジェクトなので，オペレーティングシステムはその個数と合計サイズの両方を制限する．セグメントは**キー**で識別され，同じキーを使うプロセスはすべて同じセグメントにアクセスする．そのライフサイクルは 4 つの操作からなる（図 9.3）．

**作成** オペレーティングシステムがセグメントのメモリを確保し，それをキーに結び付ける．

▷ attach

**アタッチ** セグメントを**アタッチ**するとき，オペレーティングシステムはそれを呼び出し元プロセスのアドレス空間に対応付ける．すなわち，プロセスの仮想アドレスからセグメントの物理メモリへの有効な対応付けを確立する．複数のプロセスが同じセグメントを，それぞれ自分の仮想アドレスでアタッチできる．

▷ detach

**デタッチ** セグメントを**デタッチ**するとき，オペレーティングシステムはこれらの対応付けを無効にする．セグメントの仮想アドレス範囲に対するページテーブルエントリはもはや有効でない．セグメント自体は残る．

**破棄** セグメントを取り除くのは明示的な要求だけである．それまでは，どのプロセスもアタッチしていなくても存続し，何度でもアタッチとデタッチができる．

**注意** アタッチされたまま取り除かれたセグメントを，ipcs -m は状態 dest で示す．nattch が 0 になった時点で消える．

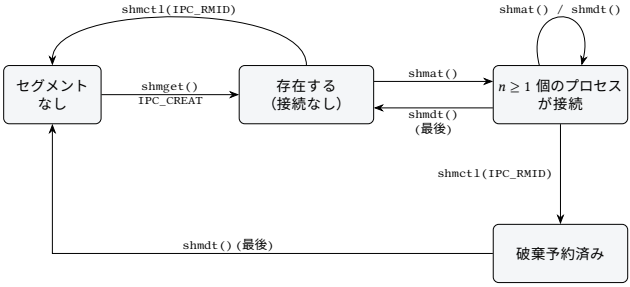



**図 9.3:** System V 共有メモリセグメントのライフサイクル．セグメントはそれを使うプロセスより長く残り，取り除かれるのは IPC_RMID によってのみである．アタッチされたまま取り除かれたセグメントは，最後のプロセスがデタッチした時点で破棄される．

したがってセグメントは，プロセス固有のメモリというよりファイルのように振る舞う．終了したプロセスはセグメントからデタッチされるが，セグメントは破棄されるかシステムが再起動するまで残る．

**注意** どのプロセスも破棄しなかったセグメントは，再起動までメモリを占有し続ける（9.9 節）．

## 9.6.1 System V の API

このインタフェースは 4 つの呼び出しと，キーを生成する補助関数からなる．

shmget(key, size, flags) は key に結び付いたセグメントを開く．存在せず，かつ flags に IPC_CREAT が含まれていれば，指定された大きさのセグメントを作成する（IPC_CREAT | IPC_EXCL では，セグメントがすでに存在すると呼び出しは失敗する）．セグメントの識別子を返す．flags の下位 9 ビットは，ファイルと同様にアクセスモードであり，0600 は所有者だけに読み書きを許す．既存のセグメントを開くプロセスは大きさに 0 を渡す．

size はページ単位に切り上げられるので，最小のセグメントは 1 ページ（x86-64 では 4 kB）である（shmget(2)）．

ftok(pathname, proj_id) は，既存のファイルの同一性（デバイスと i ノード番号）と，0 でない整数の下位 8 ビットからキーを生成する．ファイルが作り直されない限り，同じファイルと同じ番号は同じキーを与えるので，無関係なプロセスどうしも，ファイル名と番号を取り決めることでセグメントを見つけられる．

**注意** キーが保持するのは proj_id の 1 バイト，i ノード番号の下位 16 ビット，デバイス番号の下位 8 ビットだけなので，別のファイルどうしが同じキーになりうる（ftok(3)）．

shmat(id, addr, flags) はセグメントをアタッチし，それが対応付けられた仮想アドレスを返す．addr を NULL にするとオペレーティングシステムがアドレスを選ぶ．返される型のないポインタは，セグメントに置く任意のデータ構造にキャストできる．

shmdt(addr) は addr に対応付けられたセグメントをデタッチする．

shmctl(id, cmd, buf) はセグメントに対する制御コマンドをオペレーティングシステムに渡す．IPC_RMID はセグメントを破棄する．

**例 9.7.** プロセス A がセグメントを作成して文字列を書き込み，後から実行されるプロセス B が同じキーでそのセグメントを見つけ，文字列を表示してセグメントを破棄する（エラー処理は省略．ftok はファイルがないと −1 を返すので，/tmp/app が存在していなければならない）．

```
/* プロセス A */
key_t key = ftok("/tmp/app", 'A');
int id = shmget(key, 4096, IPC_CREAT | 0600);
char *p = shmat(id, NULL, 0);
strcpy(p, "hello");                 /* B から見える */
shmdt(p);                           /* セグメントは残る */

/* プロセス B */
key_t key = ftok("/tmp/app", 'A'); /* 同じキー */
int id = shmget(key, 0, 0);         /* 開く */
char *q = shmat(id, NULL, 0);
printf("%s\n", q);
shmdt(q);
shmctl(id, IPC_RMID, NULL);         /* 破棄する */
```

A の shmdt と B の shmat の間，どのプロセスもセグメントをアタッチしていないが，その内容は保たれている．

## 9.7 POSIX 共有メモリ

POSIX は第 2 の共有メモリインタフェースである **POSIX 共有メモリ**を標準化しているが，これは System V ほど広くサポートされてはいない．例えば Linux では，カーネル 2.4 以降でサポートされている．キーで識別されるセグメントの代わりに，名前で識別されファイルのようにアクセスされる**共有メモリオブジェクト**を用いる．Linux では，これらはメモリ上の一時ファイルシステム（tmpfs）内のファイルである．プロセスは，ファイルをメモリに対応付ける一般的な機構を通じてそれらにアクセスする．

▷ POSIX shared memory

**出典** P3L3，POSIX 共有メモリ API；Kerrisk ch. 49，54．

▷ shared memory object

**定義 9.8 (メモリマップドファイル).** **メモリマップドファイル**とは，その内容がプロセスの仮想アドレス空間に対応付けられ，プロセスがメモリの読み書きによってアクセスできるようになったファイルである．同じファイルを共有として対応付けたプロセスは，同じ物理ページを対応付け，互いの変更を見る．

▷ memory-mapped file

通常のファイルシステム上のメモリマップドファイルは，それ自体が IPC 機構であり，そのデータはファイルにも永続する．POSIX 共有メモリは，メモリ上にのみ存在するオブジェクトに同じ機構を適用したものである．その操作は System V の操作に対応する（表 9.3）．shm_open(name, flags, mode) は共有メモリオブジェクトを作成または開き，そのファイルディスクリプタを返す．ftruncate がその大きさを設定する．MAP_SHARED フラグ付きの mmap はオブジェクトをアタッチしてその仮想アドレスを返し，munmap はそれをデタッチする．close は，確立済みの対応付けに影響を与えずにファイルディスクリプタを解放し，shm_unlink(name) は名前を取り除く．どのプロセスもそれを開いておらず，対応付けてもいなくなった時点で，オブジェクトは破棄される．

オブジェクトは，Linux がこの tmpfs をマウントする /dev/shm の下に現れる．df /dev/shm でその大きさの上限が分かり，既定では RAM の半分である（shm_overview(7)，tmpfs(5)）．

**ヒント** shm_open の名前は 1 つのスラッシュで始まり，それ以外にスラッシュを含まない．glibc 2.34 より前では -lrt をリンクする（shm_open(3)）．

**注意** MAP_PRIVATE で対応付けると，代わりにコピーオンライトになる．最初の書き込みでそのプロセス専用のページができ，何も共有されない（mmap(2)）．

**表 9.3:** 2 つの共有メモリインタフェースの対応する操作．

| 操作 | System V | POSIX |
|---|---|---|
| 領域の名前付け | ftok() によるキー | オブジェクト名（/name） |
| 作成・オープン | shmget() | shm_open()，ftruncate() |
| アタッチ | shmat() | mmap() |
| デタッチ | shmdt() | munmap() |
| 破棄 | shmctl(IPC_RMID) | shm_unlink() |

## 9.8 共有メモリと同期

共有領域に並行してアクセスするプロセスは，1 つのアドレス空間の中で状態を共有する第 3 章のスレッドと同じ状況にある．同期がなければ，そのアクセスは競合状態をなす．利用できる機構は 2 種類ある．1 つは，プロセスをまたいで使えるように構成された Pthreads のミューテックスや条件変数のような，スレッドライブラリの機構である．もう 1 つは，メッセージキューやセマフォのような，オペレーティングシステムが提供す

**出典** P3L3，共有メモリと同期，IPC のための PThreads 同期，他の IPC のための同期；Kerrisk ch. 30，46–47，52–53．

る IPC 機構に基づく同期である．どちらの種類でも，プロセスは 2 つのことを協調させなければならない．1 つは共有領域への並行アクセスの数であり，これは相互排除が制限する．もう 1 つはデータが消費できるようになる時点であり，条件変数の場合のように，一方のプロセスがそれを待つことができ，他方がそれを知らせられなければならない．

### 9.8.1 Pthreads によるプロセス間の同期

▷ process-shared attribute

第 4 章の Pthreads のミューテックスと条件変数は，2 つの条件の下でプロセスを同期できる．第 1 に，それらの初期化に用いる属性オブジェクト pthread_mutexattr_t と pthread_condattr_t の両方で，**プロセス共有属性** を PTHREAD_PROCESS_SHARED に設定しなければならない．既定では，作成したプロセスのスレッドだけがそれらを使える．第 2 に，ミューテックスと条件変数自体が共有領域に置かれていなければならない．そうすることで，すべてのプロセスが同じデータ構造を操作する．

**例 9.9.** あるプロセスが領域のレイアウトを定める．領域は同期構成要素から始まる:

```
typedef struct {
  pthread_mutex_t mutex;
  pthread_cond_t  cond;
  char            data[1024];
} shm_data_t;
```

そしてセグメントを作成してアタッチする:

```
key_t key = ftok("/tmp/app", 120);
int id = shmget(key, sizeof(shm_data_t),
                IPC_CREAT | IPC_EXCL | 0600);
shm_data_t *shm = shmat(id, NULL, 0);
```

さらに領域の中でプロセス共有のミューテックスと条件変数を初期化する:

```
pthread_mutexattr_t ma;
pthread_condattr_t  ca;
pthread_mutexattr_init(&ma);
pthread_mutexattr_setpshared(&ma,
                             PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&shm->mutex, &ma);
pthread_condattr_init(&ca);
pthread_condattr_setpshared(&ca,
                            PTHREAD_PROCESS_SHARED);
pthread_cond_init(&shm->cond, &ca);
pthread_mutexattr_destroy(&ma);
pthread_condattr_destroy(&ca);
```

2 つの属性オブジェクトはどちらも PTHREAD_PROCESS_SHARED を持つ．構成要素を初期化した後は属性オブジェクトは不要なので，破棄する．[2] IPC_EXCL フラグは，セグメントがすでに存在する場合に shmget を失敗させるので，同期構成要素を初期化するのはただ 1 つのプロセス，すなわち作成者である．他のプロセスは IPC_EXCL を付けずにセグメントを開くが，作成者が初期化の完了を知らせるまで構成要素を使ってはならない．知らせるには，例えばメッセージ，セマフォ，あるいは作成者が上の呼び出しの後に設定するアトミックな「初期化済み」フラグを用いる．さもないと，pthread_mutex_init が実行される前にあるプロセスが shm->mutex をロックしうる．それ以降，セグメントをアタッチする各プロセスは，スレッドとまったく同じように shm->mutex をロックし shm->cond で待つ．

[2] プロセスはクリティカルセクションの中で死ぬことがあり，そのときミューテックスは永久にロックされたままになる．属性オブジェクトに PTHREAD_MUTEX_ROBUST を設定したミューテックスは，代わりに次にロックするプロセスへ EOWNERDEAD を返す．そのプロセスは共有状態を修復して pthread_mutex_consistent を呼ぶ．さもなければミューテックスは使えなくなる（ENOTRECOVERABLE）．

### 9.8.2 他の IPC 機構による同期

プロセスは，オペレーティングシステムの IPC 機構を通じて同期することもできる．メッセージキューは，送信と受信のプロトコルによって相互排除を提供できる（図 9.4）．P1 は共有領域にデータを書き，それから「ready」メッセージを送る．キューでの受信でブロックしていた P2 は，そのメッセージが届くとデータを読み，「ok」メッセージで応答する．P1 は領域に再びアクセスする前にそれを待つ．各プロセスは相手のメッセージを受け取った後にのみ領域にアクセスするので，アクセスが重なることはない．

双方向で 1 つのキューを使うため，各プロセスは相手のメッセージだけを受信しなければならない．例えばメッセージの型で選択する（System V）．

セマフォ（第 10 章）は，整数値を保持するオペレーティングシステムの同期構成要素である．値 0 と 1 で使えばミューテックスのように振る舞う．値 0 を見たプロセスはブロックし，値 1 を見たプロセスはそれを 0 に減らして先へ進み，クリティカルセクションを出るときに再び値を増やす．System V のセマフォは集合として扱われる．semget(key,

POSIX の名前付きセマフォ（sem_open）は作りからしてプロセス共有である．名前のないものは pshared を設定し，領域の中に置かなければならない（sem_overview(7)）．

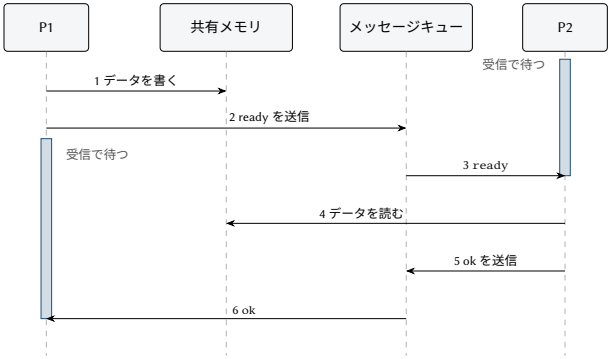



**図 9.4:** メッセージキューが共有メモリへのアクセスを協調させる．P2 は「ready」の後にのみ読み，P1 は「ok」の後にのみ先へ進む．

nsems, flg) は，セグメントと同様にキーで識別される nsems 個のセマフォの集合を作成または開く．semop(id, ops, nops) は，集合の要素に対する増分と減分の配列をアトミックに適用し，減分によって値が負になる間はブロックする．semctl は値を設定し（SETVAL），集合を破棄する（IPC_RMID）．POSIX のインタフェースはより単純である．sem_open は名前付きセマフォを作成または開き，sem_wait はその値を減らし（値が 0 の間はブロックする），sem_post は値を増やす（第 10 章）．

> **要点**
> 共有メモリには，マルチスレッドプロセスの共有状態と同じ同期が必要である．Pthreads のミューテックスと条件変数は，プロセス共有に設定され共有領域に置かれていればプロセスをまたいで働く．メッセージキューとセマフォは，同じ協調をオペレーティングシステムを通じて提供する．

## 9.9 IPC のコマンドラインツール

System V の IPC オブジェクトは，それを作成したプロセスより長く残るので，Unix 系システムはそれらを調べて取り除くコマンドを提供する．ipcs コマンドは，システム内のすべての IPC 機構—メッセージキュー，共有メモリセグメント，セマフォ集合—を，そのキー，識別子，所有者，アクセス権とともに一覧する．ipcs -m は一覧を共有メモリに限定する．ipcrm コマンドは機構を取り除く．ipcrm -m にセグメント識別子を続けるとそのセグメントを破棄でき，これはプログラムが破棄し損ねたセグメントを回収する通常の方法である．

**出典** P3L3，IPC のコマンドラインツール；Kerrisk ch. 45.

**ヒント** nattch の欄は，そのセグメントに現在アタッチしているプロセスの数である．どのプログラムも開き直さないのに nattch が 0 のセグメントはリークである．

ipcs -l はカーネルの上限を表示し，ipcrm -M にキーを続けるとキーでセグメントを取り除ける（ipcs(1)，ipcrm(1)）．

## 9.10 共有メモリ上の通信の設計

オペレーティングシステムは共有メモリを提供し，その後は介在しない．同期機構の選択，データの形式，データを渡すプロトコルはアプリケーションに委ねられる．この柔軟さのために，プログラマは特に 2 つの設計上の決定に責任を負う．

**出典** P3L3，共有メモリの設計上の考慮，セグメントはいくつか，設計上の考慮．

### 9.10.1 セグメントはいくつにするか

共有メモリで通信するプロセスは，それを 2 通りに構成できる（図 9.5）．

**1 つの大きなセグメント** すべての通信が単一のセグメントを使う．その場合アプリケーションは，セグメントの一部を個々のやり取りに割り当て，後で解放するメモリマネージャを必要とする．

**多数の小さなセグメント** 通信するプロセスの組ごとに専用のセグメントを使う．セグメントの作成は高価で個数にも限りがあるので，セグメントのプールをあらかじめ作り，空いているものの識別子をキューに保持する．プロセスはプールからセグメントを取り，その識別子をメッセージキューなど別の機構で相手に伝える．やり取りが終わるとセグメントはプールに戻る．

プールをあらかじめ確保してその要素を配るのは，第 3 章のボス・ワーカー型のスレッドプールと同じ論法をセグメントに適用したものである．

**注意** セグメントのプールは shmmni で上限が決まる．使い果たすと shmget が ENOSPC を返し，ディスクが満杯であるかのように誤解しやすい（shmget(2)）．

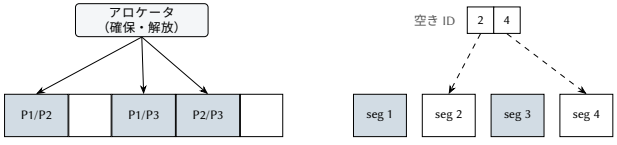



**図 9.5:** 複数のプロセスの組に対する共有メモリの構成: (a) アロケータが一部を配る 1 つの大きなセグメント，(b) 空き識別子をキューに保持する小さなセグメントのプール．

### 9.10.2 セグメントをどれだけ大きくするか

第 2 の決定はセグメントの大きさと，1 つに収まらないデータをどう扱うかである．

**データの大きさに等しいセグメント** どのデータ項目も 1 つのセグメントに収まる．データの大きさが既知で固定の場合にうまく働くが，データの大きさはセグメントの最大サイズに制限される．

**データより小さいセグメント** より大きなデータは，一度に一部ずつ，複数のラウンドに分けて転送する．その場合プロセスは，転送の進行を追うプロトコルを必要とする．典型的には，セグメントの先頭に置いたデータ構造が同期構成要素を保持し，セグメントがどれだけのデータを含むか，転送が完了したかを記録する．

**例 9.10.** セグメントは次のヘッダで始まり，残りの部分が 2 MB のペイロードを保持する．

```
typedef struct {
  pthread_mutex_t mutex;  /* プロセス共有 */
  pthread_cond_t  cond;   /* プロセス共有 */
  size_t nbytes;          /* buf 内のバイト数; 0 = 空 */
  int    done;            /* 最後の部分で設定 */
  char   buf[];           /* ペイロード */
} xfer_t;
```

送信側はミューテックスを保持したまま nbytes が 0 になるまで待ち，次の部分を buf へコピーし，nbytes を設定して（最後の部分では done も設定して）signal する．受信側は nbytes が 0 でなくなるまで待ち，その部分をコピーして取り出し，nbytes を 0 に戻して signal する．5 MB の転送は $\lceil 5/2 \rceil = 3$ ラウンドを要する:

これは第 4 章の生産者・消費者の有界バッファでスロットが 1 つの場合であり，スロットはセグメントのペイロード領域にあたる．

| ステップ | プロセス | 動作 | nbytes（MB） | done |
|---|---|---|---|---|
| 1 | 送信側 | バイト 0–2 MB を書く | 2 | 0 |
| 2 | 受信側 | それを読む | 0 | 0 |
| 3 | 送信側 | バイト 2–4 MB を書く | 2 | 0 |
| 4 | 受信側 | それを読む | 0 | 0 |
| 5 | 送信側 | バイト 4–5 MB を書く | 1 | 1 |
| 6 | 受信側 | それを読み，done を見る | 0 | 1 |

### 9.10.3 メッセージパッシングか共有メモリか

データの量は，この章の 2 つの機構のどちらを選ぶかも決める．小さなデータを低い頻度でやり取りするならメッセージパッシングが有利である．カーネルがチャネルと同期を提供し，わずかなコピーのコストは小さい．大きなデータや，多数のやり取りに再利用される領域には共有メモリが有利であり，1 回限りの設定のコストが償却される．損益分岐点がどこにあるかは，やり取り 1 回当たりの同期のコスト，すなわち式 (9.1) の $t_{\text{sync}}$ による．やり取りのたびにブロックして相手を起こすプロトコルは，小さなデータでは共有メモリの利点を打ち消しうる．

**要点 — 章のまとめ**

IPC 機構はアドレス空間をまたいでデータを移す．メッセージパッシング—パイプ，メッセージキュー，ソケット—は，ポートを介してすべてのメッセージをカーネルのチャネルに通し，その代価としてメッセージごとにシステムコールとコピーを払う．共有メモリは同じ物理ページを複数のアドレス空間に対応付けるので，1 回限りの設定の後，プロセスはカーネルを介さずに通信する．コピーとマップのどちらが安いかは，データの大きさと対応付けを何回再利用するかで決まり（式 (9.1)），Windows の LPC はこのトレードオフを利用している．System V のセグメント（`shmget`，`shmat`，`shmdt`，`shmctl`）と POSIX 共有メモリ（`shm_open`，`mmap`，`munmap`，`shm_unlink`）は明示的に破棄されるまで存続し，`ipcs` と `ipcrm` が System V のオブジェクトを調べて取り除く．メモリを共有するプロセスは，領域に置いたプロセス共有の Pthreads ミューテックスと条件変数か，メッセージキューとセマフォによって同期しなければならず，セグメントをいくつ使うか，セグメントより大きなデータをどう転送するかを決めなければならない．

第 10 章

# 同期機構

| | |
|---|---|
| 講義動画 | P3L4 |
| 教科書 | OSTEP [1] ch. 28，31；Silberschatz ら [2] ch. 6–7 |
| キーワード | スピンロック，ビジーウェイト，セマフォ，リーダ・ライタロック，モニタ，バリア，ウェイトフリー同期，RCU，アトミック命令，テストアンドセット，キャッシュ一貫性，書き込み無効化，テストアンドテストアンドセット，キューイングロック |

第 3 章ではミューテックスと条件変数を導入し，第 9 章では共有メモリを通じて通信するプロセスもスレッドと同じように同期を必要とすることを示した．この章では，ミューテックスと条件変数より単純か，あるいは表現力の高い同期機構—スピンロック，セマフォ，リーダ・ライタロック，モニタ，およびあまり一般的でないいくつかの機構—を概観する．続いて，そうした機構がすべて最終的に依拠するもの，すなわちハードウェアが提供するアトミック命令と，それが共有メモリ型マルチプロセッサのキャッシュとどう関わるかを扱う．

## 10.1 ミューテックスと条件変数の先へ

**出典** P3L4，同期についてさらに；OSTEP ch. 28．

ミューテックスと条件変数は第 3 章の同期方針を表現できるが，3 つの限界があり，それが他の機構を必要とする理由となる．

**誤りやすい利用** 正しさはすべてプログラマに委ねられる．共有状態へのアクセスはすべて正しいミューテックスをロックし，どの経路でもそれを解放しなければならず，状態の変化はすべて正しい条件変数にシグナルまたはブロードキャストしなければならない．第 3 章に挙げた落とし穴の大半，例えば誤ったミューテックスのアンロックや誤った条件変数へのシグナルは，これらの規則に違反するものであり，機構自体はそのどれも検出しない．

**表現力の限界** ミューテックスが強制する方針はちょうど 1 つ，一度に 1 つのスレッドという方針だけである．複数のリーダまたは 1 つのライタを受け入れる，待っているライタを優先するといったより豊かな方針は，第 3 章のリーダ・ライタ問題の解における代理変数のように，補助変数と明示的な検査で符号化しなければならない．

**低水準の支援への依存** ミューテックス自体が共有状態である．そのロック操作は，ミューテックスが空いていることを見つけ，それを所有済みと印を付けることを，1 つの不可分な手順で行わなければならない．実用的な実装はその手順を，ハードウェアが提供するアトミック命令から得る（10.7 節）．

セマフォ，リーダ・ライタロック，モニタ，および 10.3〜10.6 節の機構は，最初の 2 つの限界に対処する．10.2 節のスピンロックはそれらすべての土台となる低水準の構成要素であり，3 つ目の限界はこの章の後半の主題である．

## 10.2 スピンロック

**出典** P3L4，スピンロック；OSTEP ch. 28．

ミューテックスの次に単純な機構がスピンロックである．

▷ spinlock

**定義 10.1** (スピンロック). **スピンロック**とは，ミューテックスと同様にロック操作とアンロック操作によって相互排除を提供するロックであり，ロックが保持されていることを見つけたスレッドがブロックしない点だけが異なる．そのスレッドは実行を続け，ロックが空くか自分がプリエンプトされるまで，ロックを繰り返し検査する．

▷ busy waiting

条件が成立するまで CPU を手放すのではなく，ループの中で条件を何度も検査することを**ビジーウェイト**と呼び，スピンロック上でビジーウェイトするスレッドは**スピンする**という．スピンロックはミューテックスとまったく同じように使う．

```
spinlock_lock(s);
// クリティカルセクション
spinlock_unlock(s);
```

2 つの機構が異なるのは，ロックが他のスレッドに保持されている間にスレッドが何をするかだけであり，そのコストも異なる（図 10.1）．ミューテックスでブロックしたスレッドは待っている間 CPU 時間を消費しないが，自分から他のスレッドへのコンテキス

トスイッチと，ミューテックスが解放された後の起床および元へ戻るコンテキストスイッチの代価を払う．スピンするスレッドはどちらのコンテキストスイッチも避けられるが，ロックが保持されている間ずっと自分の CPU を占有する．

POSIX では pthread_spin_lock と pthread_spin_unlock．

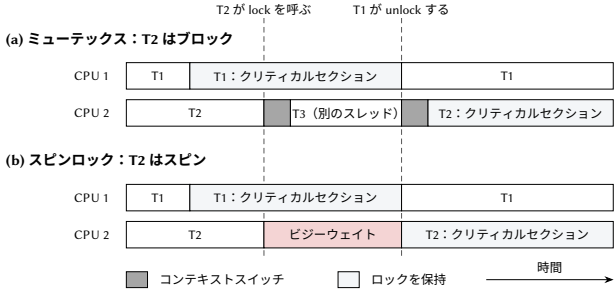



**図 10.1:** 別の CPU 上の T1 が保持するロックを T2 が要求する．(a) ミューテックスでは T2 はブロックし，CPU は別のスレッドを実行し，アンロックの後に T2 が戻される．(b) スピンロックでは T2 はビジーウェイトし，T1 がアンロックすると同時にクリティカルセクションに入る．

したがってスピンが引き合うのは，スレッドをブロックして再開するのに要する時間より短い時間しかロックが保持されず，かつ所有者が同時に別の CPU 上で動作しているときだけである．単一 CPU では，スピンするスレッドが CPU を占有している限り所有者はロックを解放できないので，スピンはタイムスライスの残りを浪費するだけである．第 5 章の適応型ミューテックスは両方の振る舞いを組み合わせ，所有者が別の CPU 上で動作しているときはスピンし，そうでなければブロックする．

**ヒント** ブロックのコストと同程度だけスピンしてからブロックすれば，最善の選択の 2 倍より悪くなることはない（Karlin ら，1991）．

**例 10.2.** スレッドをブロックして後に再開するのに合計 8 µs かかるとする．所有者がロックを保持する時間が最大 0.5 µs であれば，待っているスレッドのスピンは最大 0.5 µs であり，ブロックのコストの 16 分の 1 である．これに対し所有者がロックを保持したまま 5 ms のディスク読み出しを行えば，スピンするスレッドは 5 ms の CPU 時間を，すなわちブロックのコストの 5000/8 = 625 倍を浪費する．

2007 年のデュアル Xeon での実測では，コンテキストスイッチの直接コストは 3.8 µs，加えてその後のキャッシュ再充填がかかる（Li ら，ExpCS 2007；古い数値である）．

スピンロックが必要とするのは共有された 1 語とアトミック命令だけなので，より複雑な機構を組み立てる基本的な構成要素となる．例えばオペレーティングシステムのカーネルは，ミューテックスとそのブロックされたスレッドのキューの状態を更新する短いクリティカルセクションを，スピンロックで保護する．

Linux の spin_lock はローカル CPU のプリエンプションも無効にするので，他のスレッドがスピンしている間にロックの保持者がスケジュールから外されることはない（カーネルのロックに関する文書）．

## 10.3 セマフォ

Dijkstra [25] が提案したセマフォは，オペレーティングシステムのカーネルでよく使われる同期機構である．セマフォは，ミューテックスを一度に 1 つのスレッドから有限個のスレッドへと一般化する．

**出典** P3L4，セマフォ，POSIX セマフォ API；Dijkstra [25]；OSTEP ch. 31．

▹ semaphore

**定義 10.3** (セマフォ). **セマフォ**とは，非負の整数で初期化される整数のカウントと 2 つの操作で表される同期機構である．**ウェイト**: カウントが正ならそれを 1 減らして先へ進み，0 ならば正になるまでブロックする．**ポスト**: カウントを 1 増やし，ブロックしているスレッドがあればその 1 つを起こす．

カウントは，あと何個のスレッドが通過できるかを記録する．スレッドは入るときウェイトでそれを減らし，出るときポストで増やすので，初期値は同時に内部に入れるスレッドの最大数である．2 つの操作は P と V とも呼ばれ，オランダ語の *proberen*（試す）と *verhogen*（増やす）に由来すると説明されることが多い．

セマフォはカウントの取りうる範囲で区別される．**計数セマフォ**は任意の非負値を取り，初期値と同じ数のスレッドを同時に受け入れる．カウントが 0 と 1 の値しか取らないセマフォを**バイナリセマフォ**と呼ぶ．1 で初期化すれば一度に 1 つのスレッドだけを受け入れ，ミューテックスと同じように振る舞う．

▹ counting semaphore

▹ binary semaphore

**ヒント** カウントは通行許可の数と読む．1 ならミューテックス，*n* なら *n* 個の枠，0 なら他のスレッドのポストを待つイベントである．

**例 10.4.** 初期値 2 の計数セマフォが同一の 2 つのデバイスを守り，3 つのスレッドがそれを使う．表 10.1 はカウントとブロックされたスレッドを追跡したものである．

**表 10.1:** 3 つのスレッドが使う初期値 2 の計数セマフォ．T3 はカウントが 0 の間ブロックし，T1 のポストの後に先へ進む．

| 手順 | 事象 | カウント | ブロック中 |
|---|---|---|---|
| 1 | T1 がウェイト: カウント > 0，減らして進む | 1 | — |
| 2 | T2 がウェイト: カウント > 0，減らして進む | 0 | — |
| 3 | T3 がウェイト: カウント = 0，ブロック | 0 | T3 |
| 4 | T1 がポスト: 増やして T3 を起こす | 1 | — |
| 5 | T3 が再検査: カウント > 0，減らして進む | 0 | — |
| 6 | T2 がポスト | 1 | — |
| 7 | T3 がポスト | 2 | — |

### 10.3.1 POSIX セマフォ

POSIX は <semaphore.h> でセマフォを定義する．

```
#include <semaphore.h>

sem_t sem;
int sem_init(sem_t *sem, int pshared,
             unsigned int value);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
```

Linux 2.6（glibc の NPTL）以降，POSIX セマフォは futex の上に構築されている．競合のない sem_wait や sem_post はユーザ空間での 1 回のアトミック命令である．

**注意** セマフォに所有者はない．ウェイトしていないスレッドも含め，どのスレッドもポストできる．

sem_init は初期カウントを value に設定する．フラグ pshared は，セマフォを 1 つのプロセスのスレッドだけが使うのか（0），プロセス間で共有するのか（0 以外）を表す．後者の場合，sem_t 自体が，第 9 章の共有メモリセグメントのような，それらのプロセスが共有するメモリ上に置かれていなければならない．sem_wait と sem_post がウェイトとポストの操作である．

**例 10.5.** あるサーバは，バックエンドのデータベースに届く要求を一度に最大 4 つに制限する．

```
#include <semaphore.h>

#define MAX_BACKEND 4
sem_t backend_slots;

void init_slots(void) {
  sem_init(&backend_slots, 0, MAX_BACKEND);
}

void handle_request(struct request *r) {
  sem_wait(&backend_slots);   /* スロットを取る */
  forward_to_backend(r);      /* ここには最大4つ */
  sem_post(&backend_slots);   /* スロットを解放 */
}
```

4 つの要求が処理中の間に handle_request を呼んだ 5 つ目のスレッドは，そのうちの 1 つがポストするまで sem_wait でブロックする．

**注意** POSIX の sem_wait はシグナルが届くと EINTR で失敗することがある．呼び出し側は戻り値を検査して再試行しなければならない．

## 10.4 リーダ・ライタロック

第 3 章では，リーダ・ライタ方針–任意の数のリーダ，または 1 つのライター–を，ミューテックス，2 つの条件変数，代理変数から組み立てた．この方針は非常によく現れるので，オペレーティングシステムやスレッドライブラリはそれを独立した機構として提供している．

**出典** P3L4，リーダ/ライタロック，リーダ/ライタロックの利用；OSTEP ch. 31.

▷ reader-writer lock

**定義 10.6** (リーダ・ライタロック). **リーダ・ライタロック**とは，2 つのモードを持つロックである．**読み出し**（共有）モードは任意の数のスレッドが同時に保持でき，**書き込み**（排他）モードは，他のどのスレッドもどちらのモードでもロックを保持していないときに限り，1 つのスレッドが保持できる．

スレッドはどのモードでロックが必要かを表明し，機構が方針を強制する．Linux カーネルでの型は rwlock_t である．

Linux の rwlock_t はスピンする．ブロックする変種は struct rw_semaphore である．

```
#include <linux/spinlock.h>

rwlock_t m;

read_lock(&m);
/* クリティカルセクション: 共有状態を読む */
read_unlock(&m);

write_lock(&m);
/* クリティカルセクション: 読んで変更する */
write_unlock(&m);
```

POSIX スレッド，Java，Windows の .NET フレームワークも同じ機構を提供する．POSIX スレッドでは次のようになる．

```
pthread_rwlock_t m;
pthread_rwlock_rdlock(&m);    /* 読み出しモードで獲得 */
pthread_rwlock_wrlock(&m);    /* 書き込みモードで獲得 */
pthread_rwlock_unlock(&m);    /* どちらのモードも解放 */
```

実装は基本的な方針では一致するが，プログラムの振る舞いに影響する細部が異なる．

- **再帰的な読み出しロック．** スレッドが読み出しロックを 2 回獲得したとき，1 回の read_unlock が両方の獲得を解放するのか，一方だけを解放するのかは，実装によって異なる．
- **昇格と降格．** 実装によっては，リーダがロックを解放せずに書き込みロックへ変換したり，ライタが読み出しロックへ変換したりできる．これを許すかどうか，またそうした要求がすでに待っているスレッドに対してどの優先度を持つかは，実装によって異なる．
- **スケジューリングとの相互作用．** 実装は，ロックが現在読み出しモードでしか保持されていなくても，より高い優先度かもしれないライタが待っている間，到着したリーダをブロックすることがある．

ヒント リーダ・ライタロックを使うのは，リーダが多く，クリティカルセクションが余分な帳簿付けを償うほど長いときだけにする．短いセクションでは通常のミューテックスのほうが速いことが多い．

**例 10.7.** リーダが 3 ms ごとに到着し，各々が読み出しモードで 5 ms ロックを保持するとする．どのリーダも先行するリーダが去る前に到着するので，ロックが空くことはない．したがって，ロックが読み出しモードで保持されている間つねにリーダを受け入れる実装では，待っているライタは無限にスタベーションに陥る．ライタが待ち始めたら新しいリーダをブロックする実装なら，すでに内部にいるリーダが去り次第，待ち始めてから最大 5 ms でライタを受け入れる．

## 10.5 モニタ

ここまでの機構は，適切な場所でロックとアンロックを行う責任を，なおプログラマに残している．モニタはこの責任を機構の側へ移す．

出典 P3L4，モニタ；Hoare [26]；Lampson と Redell [10]；Silberschatz ら ch. 6.
▷ monitor
▷ entry procedure

**定義 10.8 (モニタ).** **モニタ**とは，共有資源をカプセル化する高水準の同期機構である．資源と，その資源にアクセスする唯一の手段である手続き–各々を**エントリ手続き**と呼ぶ–と，モニタの内部にいるスレッドが待つことのできる条件変数とを指定する．

相互排除は暗黙的である（図 10.2）．スレッドがエントリ手続きを呼ぶと，モニタのロックが自動的に獲得され，モニタが使用中であることを見つけたスレッドはその入口キューで待つ．手続きから戻ると，ロックは自動的に解放される．モニタの内部で条件変数上で待つスレッドはロックを解放し，起こされたときには先へ進む前にロックを再獲得する．これもまた明示的なロック操作を伴わない．概念的には，モニタは入口でロックと検査を行い，出口でアンロックと検査とシグナルを行う．とはいえ，古典的な設計ではシグナルそのものは明示的であり，スレッドが待つ条件も明示的である．Hoare のモニタはウェイトとシグナルを提供し，Mesa はシグナルをヒントにすぎないノーティファイで置き換え，ブロードキャストを加える（Lampson と Redell [10]）．

自動シグナルを備えたモニタの設計はさらに進んで，スレッドがモニタを去るたびに待機条件を再評価する．

**例 10.9.** 容量 N の有界バッファをモニタとして書くと，C 風の擬似コードで次のようになる．

```
monitor BoundedBuffer {
  item buf[N];
```

```
  int count = 0, in = 0, out = 0;
  condition not_full, not_empty;

  entry void put(item x) {
    while (count == N)
      wait(not_full);
    buf[in] = x;  in = (in + 1) % N;  count++;
    signal(not_empty);
  }

  entry item get(void) {
    while (count == 0)
      wait(not_empty);
    item x = buf[out];  out = (out + 1) % N;
    count--;
    signal(not_full);
    return x;
  }
}
```

コードにロック操作は現れないので，忘れることも，誤ったロックに適用することもできない．

**注意** 検査は `if` ではなく `while` である．Mesa のシグナルはスレッドを起こすだけで，先に別のスレッドが品目を取りうる．

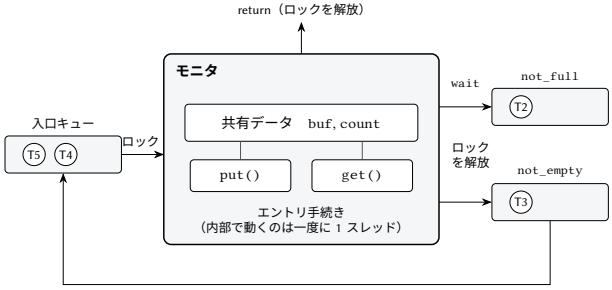



**図 10.2:** モニタ．スレッドはエントリ手続きを通じてのみ内部に入り，エントリ手続きはモニタのロックを暗黙的に獲得する．条件変数上で待つスレッドはロックを解放し，ここで示した Mesa のセマンティクスでは，起こされた後にロックを再獲得しなければならない．

Mesa と Java では，起こされたスレッドは `while` ループで条件を再検査する．

HotSpot では競合のない `synchronized` ブロックはオブジェクトヘッダへの 1 回のコンペアアンドスワップであり，競合するとOS のモニタへ昇格する．

第 3 章の条件変数と同じく，ループの中で待つ規律である．モニタはミューテックスを隠すだけである．

モニタは Xerox PARC で開発された Mesa 言語の一部であった [10]．今日では Java がモニタをサポートしている．すべてのオブジェクトが内部にロックを持ち，`synchronized` と宣言されたメソッドは，そのロックとアンロックを言語ランタイムが行うエントリ手続きである．ただし通知は `notify` と `notifyAll` の明示的な呼び出しのままである．

**モニタ**という語はプログラミングのスタイルも指す．第 3 章のように，ミューテックスと条件変数から組み立てた明示的な入口コードと出口コードですべてのクリティカルセクションを構成することは，言語の支援なしにモニタの規律を適用することである．

## 10.6 その他の同期機構

同期方針をより述べやすくすること，あるいは待つこと自体を避けることを目指す機構が，ほかにもいくつかある．

**出典** P3L4，さらなる同期機構．

▷ serializer

**シリアライザ シリアライザ** [27] はモニタと同様に資源を守るが，優先度を定義しやすくし，明示的なシグナルを隠す．スレッドは，自分が先へ進める条件とともにキューで待つ．条件が成立し，かつそのスレッドが自分のキューの先頭にあるとき，シリアライザが自動的にそれを再開する．したがってプログラムには条件変数もシグナル呼び出しも現れず，キューの順序が優先度を表す．

▷ path expression

**パス式 パス式** [28] は，資源に対する操作の名前を用いた正規表現として，操作が実行されうる順序を述べる．ランタイムシステムは，その式が許す順序でのみ操作を交錯させる．「多数の読み出しまたは 1 つの書き込み」という方針は `path {read}, write end` と書く．波括弧は任意個の読み出しの同時実行を許し，コンマは選択肢の 1 つを選び，パスは無限に繰り返される．

**バリア バリア**はセマフォの逆である．セマフォは最大 $n$ 個のスレッドを通し，その後に来たスレッドをブロックする．バリアは $n$ 個のスレッドが到着するまでスレッドをブロックし，その後すべてをまとめて進ませる（図 10.3）．POSIX スレッドは `pthread_barrier_t` と `pthread_barrier_wait` を提供する．

▹ barrier

**ランデブーポイント ランデブーポイント**とは，2 つ以上の特定のスレッドの実行における点であり，各スレッドは他のスレッドが対応する点に達するまでそこで待つ．その後，典型的にはデータを交換してから，スレッドは実行を続ける．

▹ rendezvous point

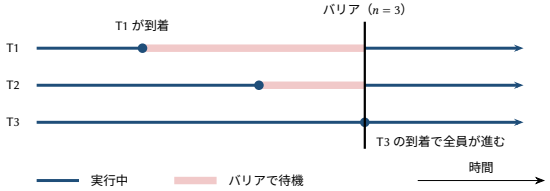



**図 10.3:** 3 つのスレッドのためのバリア．T1 と T2 はバリアに達したところでブロックし，T3 の到着が 3 つすべてを解放する．

### 10.6.1 ウェイトフリー同期と RCU

ここまでの機構はいずれもスレッドを待たせる．楽観的な代替手段は，並行する操作が衝突しないことに賭けてスレッドをロックなしに進ませ，変更されている間も並行するリーダが安全であるようにデータ構造を設計する．

**定義 10.10** (ウェイトフリー同期)**.** 他のスレッドがどれだけ速くても遅くても，どのスレッドも各操作を自分自身の有限回のステップで完了する同期を**ウェイトフリー同期** [29] と呼ぶ．どのスレッドも他のスレッドを待つことがない．

▹ wait-free synchronization

現在は Linux カーネルの一部である **RCU** 機構 [30] は，この考えを，書かれるよりはるかに多く読まれるデータに適用する．リーダはロックをまったく取らない．ライタは現在の版を複製し，複製を変更し，リーダがデータを見つけるために辿るポインタをアトミックに差し替えて公開する．そのためリーダは，古い版か新しい版のどちらかを見るのであって，中途半端に変更された版を見ることはない．古い版は，なおそれへの参照を保持しているかもしれないリーダが終えた時点で解放される．したがってリーダは決して待たず，ライタどうしは通常のロックで協調する．

▹ read-copy-update

表 10.2 はこの章の機構をミューテックスと比較したものである．スレッドを待たせるかどうかにかかわらず，そのいずれもがメモリ位置をアトミックに更新すること—ロック語の設定，カウントの増減，ポインタの差し替え—に依存しており，これにはハードウェアの支援が必要である．

**表 10.2:** 同期機構の比較．

| 機構 | 同時に受け入れる数 | 待つスレッド | ロック |
|---|---|---|---|
| ミューテックス | 1 つ | ブロック | 明示的 |
| スピンロック | 1 つ | スピン | 明示的 |
| 計数セマフォ | 初期カウントまで | ブロック | 明示的 |
| リーダ・ライタロック | 任意個のリーダまたは 1 つのライタ | ブロックまたはスピン | 明示的 |
| モニタ | 1 つ | ブロック | 暗黙的 |
| バリア | $n$ 個到着したら $n$ 個 | ブロック | 明示的 |
| RCU | 任意個のリーダと 1 つのライタ | リーダは待たない | ライタのみ |

### 10.6.2 Linux カーネルにおける同期

Linux カーネルは，この章の機構の大半を独自の名前で提供している．

- `spinlock_t` と `spin_lock`，`spin_unlock`: スピンロック．
- `rwlock_t`: 10.4 節のスピンするリーダ・ライタロック．
- `struct semaphore` と `down`，`up`，および `struct rw_semaphore`: ブロックする計数セマフォとリーダ・ライタロック．

**出典** Silberschatz ら ch. 7；Linux カーネルのロックに関する文書．

- struct mutex と mutex_lock，mutex_unlock: 所有者を持つ，ブロックするロック．
- RCU: 上で説明したもの．

その下では，型 atomic_t が atomic_inc や atomic_cmpxchg のようなアトミックな整数操作を提供する．これらは 10.7 節のアトミック命令の上に組み立てられている．スピンするロックは短い間しか保持してはならず，その保持者は眠ってはならない．ブロックするロックは入出力のように眠る操作をまたいで保持できるが，獲得できるのは眠ることが許される場所だけであり，割り込みハンドラの中では獲得できない．カーネルのクリティカルセクションの大半は数命令の長さしかなく，コンテキストスイッチよりはるかに短いので，カーネルは主にスピンする（10.2 節）．spin_lock はプリエンプションを無効にするので，保持者はロックを解放するまで実行を続け，スピンは短いままで済む．

> **要点**
> スピンロックはコンテキストスイッチと引き換えに CPU 時間を費やし，セマフォは有限個のスレッドを受け入れ，リーダ・ライタロックはリーダ・ライタ方針を 1 つの機構にまとめ，モニタはロックを暗黙的にする．シリアライザ，パス式，バリア，ランデブーポイントはさらに別の方針を表現し，RCU のようなウェイトフリーの方式は待つこと自体を避ける．

## 10.7 同期のためのハードウェア支援

**出典** P3L4，同期の構成要素: スピンロック，ハードウェア支援の必要性，アトミック命令；OSTEP ch. 28；Silberschatz ら ch. 6.

まず，通常の命令だけでスピンロックを実装してみる．

```
#define FREE 0
#define BUSY 1

void naive_lock(int *lock) {
  while (*lock == BUSY)
    ;                   /* スピン */
  *lock = BUSY;         /* 検査とは別の命令 */
}
```

**注意** コンパイラが単純な読み出しをループの外へ移すこともある．実際のコードは volatile かアトミックな読み出しを使う．

検査と更新は別々の命令であり，その間に別のスレッドが同じ空きロックを観測しうる．表 10.3 は，異なる CPU 上の 2 つのスレッドがともにクリティカルセクションに入る様子を示している．2 つの手順を別のロックで保護しても，問題をそのロックへ移すだけである．この形のロックが正しいのは，それを検査することと設定することが 1 つの不可分な手順として起こる場合だけであり，実用的な実装はその手順をハードウェアから得る．

Peterson のアルゴリズムのようなソフトウェアのアルゴリズムは，通常の読み書きだけで相互排除を達成する．

**表 10.3:** 異なる CPU 上で naive_lock を実行する 2 つのスレッド．どちらかが書き込む前に両方がロックを読むので，両方がクリティカルセクションに入る．

| 手順 | CPU 1 上の T1 | CPU 2 上の T2 | *lock |
|---|---|---|---|
| 1 | FREE を読み，ループを抜ける | | FREE |
| 2 | | FREE を読み，ループを抜ける | FREE |
| 3 | BUSY を書き込み，中に入る | | BUSY |
| 4 | | BUSY を書き込み，中に入る | BUSY |

▷ atomic instruction

**定義 10.11** (アトミック命令). **アトミック命令**とは，複数の操作，典型的にはメモリ位置の読み出しと書き込みを，1 つの不可分な手順として実行するプロセッサ命令である．ハードウェアは，アトミック性（複数の操作がまとめて効力を持つこと），相互排除（同じメモリ位置に対する他のアトミック命令がその間に割り込まないこと），および同じ位置に対する複数の並行実行のうち 1 つを除くすべてが待たされることを保証する．

したがってアトミック命令は，ハードウェアによって保護されたクリティカルセクションである．よくある例は次のものである．

▷ test-and-set

test_and_set **テストアンドセット**命令は，ロック語の現在の値を返し，その語を BUSY に設定する．

read_and_increment メモリ位置の値を返し，それを 1 増やす．

▷ compare-and-swap

**注意** コンペアアンドスワップが検査するのは値であって履歴ではない．A から B へ，そして A へ戻った位置も一致してしまう（ABA 問題）．

compare_and_swap **コンペアアンドスワップ**命令は，メモリ位置を期待値と比較し，等しい場合に限り新しい値を書き込む．書き込みが行われたかどうかは，この命令が報告する．

さらに別の変種，例えば read_and_decrement や read_and_clear，一般には演算 *op* に対する fetch_and_op も同じ型に従う．どれが存在するかはアーキテクチャごとに異なる．

プロセッサがアトミックに実行する C 関数として書けば，最初のものと最後のものは次のようになる．

```
/* 各関数は1つのアトミックな手順として実行される */
int test_and_set(int *lock) {
  int old = *lock;
  *lock = BUSY;
  return old;
}

int compare_and_swap(int *p, int expected,
                     int new_val) {
  if (*p != expected)
    return 0;             /* 値が変わっている */
  *p = new_val;
  return 1;
}
```

test_and_set を使えばスピンロックは正しくなる．

```
void spinlock_lock(int *lock) {
  while (test_and_set(lock) == BUSY)
    ;   /* スピン */
}

void spinlock_unlock(int *lock) {
  *lock = FREE;
}
```

ロックが空いていれば test_and_set は FREE を返し，すでにロックをビジーと印を付けているので，スレッドは所有者としてループを抜ける．ロックがビジーであれば，BUSY に BUSY を上書きして—これは何も変えない—BUSY を返すので，スレッドは再び試みる．空いているロックに対して同時に test_and_set を実行した複数のスレッドのうち，FREE を得るのはちょうど 1 つである．

x86 では，メモリオペランドを取る xchg は暗黙にロックされるので，lock プレフィックスは要らない（Intel SDM 第 2 巻）．

**例 10.12.** コンペアアンドスワップは，ロックなしに共有データを更新する．カウンタは，スワップが成功するまで読み直すことで 1 増やされる．

```
void increment(int *counter) {
  int old;
  do {
    old = *counter;
  } while (!compare_and_swap(counter, old, old + 1));
}
```

スワップが失敗するのは，読み出しとスワップの間に別のスレッドがカウンタを変更したときに限られる．そのときループは読み直して再試行するので，増分が失われることはない．

**注意** このループはブロックしないが，スレッドが無限に再試行しなければならないこともあるので，ウェイトフリーではない．

### 10.7.1 移植性

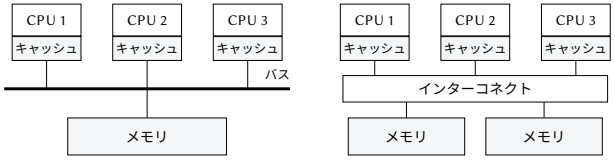
アトミック命令はプロセッサアーキテクチャに固有である．どの命令が存在し，それがどれだけ効率的かはプラットフォームによって異なる．例えば x86 では，メモリオペランドを取る xchg がテストアンドセットとして，lock cmpxchg がコンペアアンドスワップとして使える．したがって，カーネルやスレッドライブラリのように同期機構を実装するソフトウェアは，動作するすべてのアーキテクチャへ移植しなければならない．さらにプラットフォームごとに特化もされる．同じ機構の複数の実装のうち，そのプラットフォームが提供する最も効率的な命令の上に組み立てられたものを，条件コンパイルで選ぶ．

## 10.8 共有メモリ型マルチプロセッサとキャッシュ

同期機構が最も重要になるのは，スレッドが実際に同時に動作する複数 CPU のマシン上である．共有メモリ型マルチプロセッサ（第 7 章）は，複数の CPU と，そのすべてがアクセスできるメモリを持つ．こうしたシステムは対称型マルチプロセッサ（SMP）とも呼

**出典** P3L4，共有メモリ型マルチプロセッサ，キャッシュ一貫性，キャッシュ一貫性とアトミック命令；Silberschatz ら ch. 1.

ばれる．図 10.4 は 2 つの構成を示す．古いシステムに典型的なバス型の構成では，CPU と単一のメモリ部品が 1 本の共有バスに接続され，そのバスは一度に 1 つのメモリアクセスしか運べない．今日一般的なインターコネクト型の構成では，CPU と，場合によっては複数のメモリ部品が，複数のメモリアクセスを同時に進行させられるインターコネクトを通じて接続される．




(a) バス型　　(b) インターコネクト型

**図 10.4:** 共有メモリ型マルチプロセッサの構成．どちらでも，各 CPU が自分のキャッシュを持ち，すべての CPU がすべてのメモリにアクセスできる．

どの CPU も自分のキャッシュを持つ．メモリはキャッシュよりも到達に時間がかかる．物理的により遠いからでもあり，そこへ至るバスやインターコネクトを CPU どうしが奪い合うからでもある．キャッシュはこのレイテンシを隠す．

Skylake（2015）では L1 が 4 サイクル，L2 が 12，L3 が約 40，DRAM が約 80 ns．Intel 最適化マニュアル．

しかし複数の CPU が同じデータを扱うと，1 つのメモリ位置の複製が複数のキャッシュに同時に存在しうる．ある CPU がその位置に書き込むと，他の CPU が持つ複製は古くなり，そうした複製を読んだスレッドは古い値を見てしまう（図 10.5）．メモリ位置の複製が複数の CPU にキャッシュされていても，どの CPU もその位置の最新の値を観測するという性質を，**キャッシュ一貫性**と呼ぶ．

▷ cache coherence

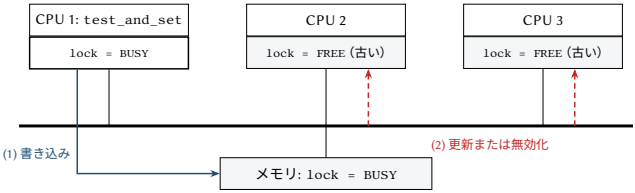



**図 10.5:** CPU 1 が lock を BUSY に設定する．CPU 2 と CPU 3 のキャッシュにある複製は古くなる．キャッシュ一貫性のあるプラットフォームでは，ハードウェアがそれらを更新または無効化する．

キャッシュ一貫性を誰が担うかはプラットフォームによって異なる．キャッシュ一貫性を持たない（NCC）プラットフォームでは，ハードウェアは書き込みを他のキャッシュの複製へ伝播せず，古い複製への対処はソフトウェアの仕事となる．キャッシュ一貫性のある（CC）プラットフォームでは，ハードウェアが 2 つの戦略のいずれかに従ってキャッシュを一貫に保つ．

▷ write-invalidate

**書き込み無効化**　**書き込み無効化**戦略では，書き込みが，他のキャッシュにあるその位置の複製を無効化する．バスやインターコネクトを渡るのは無効化だけなので，書き込み当たりの帯域は小さく，同じ CPU による以後の書き込みにはコストがかからない．ただし他の CPU での次の読み出しはキャッシュミスとなり，その位置を取得し直さなければならない．

▷ write-update

**書き込み更新**　**書き込み更新**戦略では，書き込みが新しい値を，複製を持つすべてのキャッシュへ運ぶ．他の CPU のスレッドは新しい値をただちに自分のキャッシュから読めるが，書き込み当たりのトラフィックは増える．

2 つのうちでは書き込み無効化のほうが一般的である．いずれにせよ，一貫性機構のコストは，共有された位置に書き込むソフトウェアが負担する．

## 10.8.1 マルチプロセッサ上のアトミック命令

実際のプロセッサは代わりにキャッシュラインの排他的な所有権を取得する．そのコストが高いのは，ラインを他のキャッシュから取ってこなければならないときだけである．

マルチプロセッサ上では，アトミック命令はすべての CPU にわたってアトミックでなければならない．ある CPU 上でのその実行は，他のあらゆる CPU 上での同じ位置に対する実行を排除しなければならない．したがってここで用いるモデルでは，アトミック命令は私的なキャッシュ複製に対しては働かない．キャッシュを迂回してメモリ位置そのものに適用され，すべてのキャッシュを一貫に保つ．その結果，マルチプロセッサ上のア

トミック命令は，単一 CPU のシステム上の同じ命令よりも，また同じデータに対する通常の命令よりも高価になる．メモリに到達し，バスやインターコネクトを他の CPU と奪い合い，一貫性機構を起動するからである．

スピンロックはこれをさらに悪化させる．test_and_set は，BUSY を BUSY で置き換えるだけのときでも必ず書き込みを行い，どの書き込みもキャッシュ一貫性機構を起動する．スピンするスレッドは test_and_set を絶え間なく実行するので，複数のスレッドが待っているスピンロックは，ロック語の複製を持つすべての CPU に対して一貫性更新の絶え間ない流れを生む．

> **要点**
> あらゆる同期機構は，テストアンドセットやコンペアアンドスワップといったアトミック命令の上に成り立つ．共有メモリ型マルチプロセッサでは，それらはすべての CPU にわたってアトミックでなければならず，ハードウェアは他の複製を無効化するか更新することでキャッシュを一貫に保つ．このため，アトミック命令は，とりわけ競合するロック語への繰り返しの書き込みは，高価になる．

## 10.9 スピンロックの性能

マルチプロセッサ上でのアトミック命令のコストは，スピンロックをどう書くべきかに影響する．これは Anderson [31] が研究した問題である．スピンロックの実装は 3 つの指標で特徴づけられる．

出典 P3L4，スピンロックの性能指標，テストアンドセットスピンロック，スピンロックの遅延の代替案，キューイングロック，スピンロックの性能比較；Anderson [31]．

**レイテンシ** 空いているロックをスレッドが獲得するのに要する時間．理想的には，スレッドはただちにアトミック命令を実行する．

**遅延** ロックの解放から，スピンしているスレッドがそれを獲得するまでの時間．理想的には，スピンしているスレッドは解放にただちに気づく．そのためにはロックを絶え間なく検査する必要がある．

**競合** **ロック競合**，すなわち複数のスレッドが同じロックを同時に獲得しようとする状況が引き起こすオーバーヘッド．とりわけ，それらのアトミック命令が生むキャッシュ一貫性トラフィックである．理想的には，ロックが保持されている間，スピンしているスレッドは何も書き込まない．

▷ lock contention

3 つの指標すべてで理想的な実装は存在しない．最初の 2 つは待っているスレッドにできるだけ頻繁にロックに触れることを求め，3 つ目はできるだけまれにしか触れないことを求めるからである．10.7 節の test_and_set ループはレイテンシと遅延では理想的だが，反復のたびに書き込むので競合では劣る．

ヒント レイテンシ＝空いたロック，遅延＝解放から獲得まで，競合＝待つ間のトラフィック．

### 10.9.1 読み出しでのスピン

**テストアンドテストアンドセット**という技法は，まず通常の命令でロックを読み，ロックが空いているように見えたときにだけ test_and_set を実行する．

▷ test-and-test-and-set

```
void spinlock_lock(int *lock) {
  while (*lock == BUSY || test_and_set(lock) == BUSY)
    ;   /* スピン */
}
```

短絡評価される || は，読み出しが FREE を見つけたときにだけ test_and_set を評価する．読み出しだけでは何も決まらない—読み出しと test_and_set の間に別のスレッドがロックを獲得しうる—が，その場合はアトミックな test_and_set が BUSY を返してループが続くので，ロックは正しいままである．代価は，すべての test_and_set の前に 1 回余分な読み出しが入ることであり，レイテンシと遅延がわずかに増える．

この技法が競合に対して何をもたらすかは，プラットフォームによって異なる．

**NCC** 読み出しもアトミック命令もキャッシュからは供給できないので，スピンするすべてのスレッドのすべての反復がメモリに到達し，競合は単純なループと変わらない．

**CC，書き込み更新** ロックが保持されている間，スピンするスレッドは自分のキャッシュ複製を読み，トラフィックをまったく生まない．解放はすべてのキャッシュへ伝播されるので，待っているスレッドはすべて同じ瞬間に FREE を見て，test_and_set 命令をいっせいに発行する．

**CC，書き込み無効化** 競合が改善するのは保持の間だけである．解放のたびに，待っているすべてのスレッドがミスし，読み出し，test_and_set を発行する．そして失敗する test_and_set はどれも再びすべての複製を無効化する．クリティカルセクションが短いと，これらのバーストのために単純なループとほとんど同じくらい悪くなる．

古い複製への対処はソフトウェアの仕事になる．第 15 章の分散共有メモリが同じ問題を扱う．

読み出しでのスピンが取り除くのは待機期間のトラフィックであって，解放のトラフィックではない．クリティカルセクションが短く，多数のスレッドが待っていると，解放が次々と続くのでこれらのバーストが支配的になる．そのため書き込み無効化のハードウェア上の激しい競合のもとでは，この技法は，Anderson がキューイングロックや遅延を用いたロックと比較した代替案のうちで最も性能が悪い．単純な test_and_set はさらに悪い．

1 回の解放にすべての待機スレッドが反応するのは，第 3 章の FUTEX_CMP_REQUEUE が避けるのと同じ「thundering herd」である．

**例 10.13.** 読み出しでのスピンが最もよく働く軽負荷の場合を考える．4 つの CPU があり，CPU 1 のスレッドが 1 ms ロックを保持する間，CPU 2–4 のスレッドがスピンする．単純な test_and_set ループで 1 回の反復に 50 ns かかるとすると，スピンする各スレッドは 1 ms/50 ns = 20 000 回反復するので，この保持は 3 × 20 000 = 60 000 回のロック語への書き込みを要し，その 1 回ごとに他のキャッシュの複製を無効化または更新する．テストアンドテストアンドセットでは，この保持に書き込みは 1 回もかからない．反復は読み出しであり，読み出しのほうが安価なので回数はむしろ増える．そして解放にかかるのは，待っているスレッド 1 つにつき 1 回，およそ 3 回の test_and_set 命令であり，そのうち 1 つが成功する．数百ナノ秒のクリティカルセクションを伴う高負荷のもとでは，そのバーストとそれに続くキャッシュミスが解放のたびに繰り返されることになる．

競合がなければ，Skylake 世代のコアで lock 付き命令は約 20 サイクルである．Agner Fog の命令表．

## 10.9.2 遅延を用いたスピンロック

バーストが生じるのは，待っているスレッドがすべて同じ瞬間に解放へ反応するからである．ロックに再び触れる前に少し待たせれば，反応がばらける．遅延は，ロックが空いたのを見つけた後に置くことができる．

```
void spinlock_lock(int *lock) {
  while (*lock == BUSY || test_and_set(lock) == BUSY) {
    while (*lock == BUSY)
      ;         /* 保持中は読み出しでスピン */
    delay();    /* 空いた：少し待つ */
  }
}
```

ロックの獲得に失敗したスレッドは，これまでどおり読み出しでスピンし，ロックが空いたらアトミック命令を試みる前に待つ．そのため待っているスレッドは互いに異なる時刻に test_and_set を試み，1 回の解放に反応するスレッドの数が減る．空いているロックに出会ったスレッドはループの本体に入らないので，レイテンシは読み出しでのスピンと同じである．競合は著しく改善するが，遅延は悪化する．ロックがすでに空いているのに，スレッドが自分の遅延の中に留まっていることがあるからである．

遅延は，反復のたびに置くこともできる．

```
void spinlock_lock(int *lock) {
  while (*lock == BUSY || test_and_set(lock) == BUSY)
    delay();
}
```

待っているスレッドは，絶え間なくではなく遅延の期間ごとに 1 回ロックを参照するようになる．これは一貫性トラフィックも減らし，読み出しでのスピンと違って，すべての参照がメモリに到達する NCC プラットフォームでも効果がある．代価はやはり遅延である．解放が，最長で 1 遅延期間のあいだ気づかれないままになりうる．

どちらの変種も遅延の値を必要とする．**静的な遅延**は，各スレッドを区別する何か，例えば CPU 識別子から導いた固定値を各スレッドに与え，試行が仕組みとしてばらけるようにする．単純だが，他に待っているスレッドがないときでもスレッドを遅らせる．**動的な遅延**は，スレッドが感じ取る競合—例えば失敗した test_and_set の試行回数から推定する—とともに広がる範囲から無作為に選ばれる．

**注意** 試行の失敗は，待っているスレッドが多いことではなく，クリティカルセクションが長いことを意味しているかもしれない．

## 10.9.3 キューイングロック

遅延は，解放への反応を時間的にばらけさせる．キューイングロックは，ちょうど 1 つのスレッドだけを反応させることで，バーストそのものを避ける．

▷ queueing lock

**定義 10.14** (キューイングロック). **キューイングロック**は，ロックを要求したすべてのスレッドに，キュー内の場所と専用のフラグを与える．待っているスレッドは自分のフラグ上でスピンし，ロックを解放するスレッドはキュー内の後継のフラグを設定する．これによって後継が所有者となる．

キューはスレッドごとに 1 つのフラグからなる配列であり，各フラグは HAS_LOCK か

MUST_WAIT のいずれかを取る．これに次の空き場所の添字が伴い，到着したスレッドは read_and_increment でそれを得る．

```
/* N は2の累乗．% N は桁あふれの後も周回を保つ */
int flags[N];   /* flags[0] = HAS_LOCK，残りは待機 */
unsigned queuelast = 0;

unsigned queue_lock(void) {
  unsigned me = read_and_increment(&queuelast) % N;
  while (flags[me] == MUST_WAIT)
    ;                   /* 専用のフラグ上でスピン */
  flags[me] = MUST_WAIT;  /* 次の周回に備えて戻す */
  return me;
}

void queue_unlock(unsigned me) {
  flags[(me + 1) % N] = HAS_LOCK;
}
```

解放は 1 つのフラグだけを書き込むので，起こされる待機スレッドはちょうど 1 つであり，無効化または更新されるキャッシュ複製も 1 つだけである．3 つの指標に照らすと，キューイングロックは次のようになる．

- **レイテンシ**は悪い．read_and_increment は test_and_set より複雑なアトミック命令であり，獲得のたびにそれを実行してから配列を添字で引くからである．
- **遅延**は良い．所有者がロックを後継へ直接手渡すので，後継が自分で解放を検出する必要がない．
- **競合**ははるかに良い．異なるスレッドのフラグが異なるキャッシュラインに置かれている限り，1 回の解放が生むトラフィックは 1 つのフラグ分だけである．

その代わりにロックは，1 語ではなく $N$ 項目の配列を占める．ここで $N$ は，そのロックを要求しうるスレッドの数である．また read_and_increment はすべてのプラットフォームで利用できるわけではない．

x86-64 のキャッシュラインは 64 B なので，各フラグをその大きさに詰める必要がある（偽共有，第 15 章）．

### 10.9.4 比較

表 10.4 は評価をまとめたものである．Anderson は，書き込み無効化キャッシュを持つ 20 CPU の Sequent Symmetry 上でこれらの実装を測定した [31]．高負荷–多数の CPU が同じロックを奪い合う–のもとでは，比較した代替案のうちキューイングロックが最も良く，テストアンドテストアンドセットが最も悪く，遅延を用いたロックはその中間であった．別に測定された単純な test_and_set はさらに悪かった．遅延を用いたロックの中では，高負荷のもとで静的な遅延が動的な遅延よりわずかに良く，参照のたびの遅延が解放後の遅延よりわずかに良かった．Anderson はこれを，Symmetry の一貫性が無効化に基づくためとしている．軽負荷では順序が逆転した．テストアンドテストアンドセットはレイテンシと遅延が最も小さいので最も良く，キューイングロックは獲得のたびに帳簿付けの代価を払うのでレイテンシが最も大きかった．Symmetry ではこの帳簿付けにロックが含まれていた．このマシンにはアトミックな read_and_increment がなかったので，キューイングロックは通し番号を，バックオフ付きの明示的なロックを通じて得ており，これが測定されたレイテンシの一部をなす．最後に，スピンするスレッドが少ないときには静的な遅延は不適切なので，軽負荷では動的な遅延のほうが良い．すべての条件で最良の実装は存在しない．選択は，予想される競合と，プラットフォームが提供する命令に依存する．[1]

Sequent Symmetry は 20 個の Intel 80386 を 1 本の共有バスに載せており，一貫性トラフィックはすべてそのバスを渡った．

**表 10.4:** 3 つの指標に照らしたスピンロックの実装．各項目は互いに相対的なもので，測定された順位は負荷に依存する．

| 実装 | レイテンシ | 遅延 | 競合 |
|---|---|---|---|
| test_and_set | 良 | 良 | 最悪 |
| テストアンドテストアンドセット | 中 | 中 | 書き込み無効化では悪 |
| 解放後に遅延 | 中 | 悪 | 改善 |
| 参照のたびに遅延 | 中 | 悪 | 改善，NCC でも有効 |
| キューイングロック | 悪 | 良 | 最良 |

[1] Anderson のキューイングロックはロックごとに $N$ 個のフラグの配列を必要とする．MCS ロック（Mellor-Crummey と Scott，1991）はこれをスレッドごとのノードの連結リストで置き換え，ロック自体は 1 語で済む．Linux は，8 コアの Opteron で不公平な引き渡しがスレッドをスタベーションに陥らせたことを受け（LWN，「Ticket spinlocks」，2008），2.6.25（2008）で FIFO のチケットロックを，4.2（2015）で MCS に基づく qspinlock を採用した．

**要点 — 章のまとめ**
ミューテックスと条件変数は誤りやすく，表現力に限界があり，ハードウェアの支援に依存する．スピンロックはブロックする代わりにビジーウェイトし，複数 CPU 上の短いクリティカルセクションでのみ引き合う．セマフォは与えられた数までのスレッドを受け入れ，バイナリセマフォはミューテックスとして働く．リーダ・ライタロックは共有モードと排他モードを提供し，モニタはエントリ手続きの中でロックを暗黙的に獲得・解放する．シリアライザ，パス式，バリア，ランデブーポイント，および RCU のようなウェイトフリーの方式がこれを補う．そのすべてが，テストアンドセットやコンペアアンドスワップといったアトミック命令に依存しており，共有メモリ型マルチプロセッサではさらに CPU のキャッシュを一貫に保たなければならない．テストアンドセットの前にロックを検査することは軽負荷では有効だが，書き込み無効化のプラットフォーム上の高負荷では，Anderson がキューイングロックや遅延を用いたロックと比較した代替案のうちで最も性能が悪く，単純なテストアンドセットはさらに悪い．待っているスレッドを遅延させること，とりわけ専用のフラグ上でキューに並ばせることが，競合するロックの一貫性トラフィックを抑える．

第 11 章

# I/O 管理

| | |
|---|---|
| **講義動画** | P3L5 |
| **教科書** | OSTEP [1] ch. 36–37，39–42；Silberschatz ら [2] ch. 11–15；Kerrisk [11] ch. 13–14 |
| **キーワード** | デバイスレジスタ，デバイスドライバ，ブロックデバイス，キャラクタデバイス，メモリマップド I/O，ポーリング，プログラム制御 I/O，DMA，OS バイパス，仮想ファイルシステム，i ノード，ext2，バッファキャッシュ，ジャーナリング |

第 7 章と第 8 章で述べた CPU とメモリの管理に加えて，オペレーティングシステムは，計算機が入力を受け取り，出力を生み，データを永続的に格納するためのデバイスも管理する．この章では，オペレーティングシステムがそうしたデバイスをどのように表現し制御するか，すなわちデバイスが見せるインタフェース，デバイス固有の知識を包み込むドライバ，そして CPU とデバイスがコマンド・データ・通知をやり取りする方法を調べる．続いて，ファイルアクセスがブロックデバイススタックを下る経路をたどり，ファイルシステム–とりわけ仮想ファイルシステムと ext2–がデータをディスク上にどう編成し，アクセスのコストをどう減らすかを見る．

## 11.1 I/O デバイス

**出典** P3L5，I/O デバイス，I/O デバイスの基本的な特徴；OSTEP ch. 36；Silberschatz ら ch. 12.

入出力（I/O）デバイスは，計算機をその環境に接続する．例えばキーボード，マウス，マイクロフォン，スピーカ，ディスプレイ，ネットワークインタフェースカード（NIC），ディスクがある．デバイスの機能は大きく異なり–計算機に入力を届けるだけのもの，計算機からの出力を受け取るだけのもの，その両方を行うもの–速度も，毎秒数文字を生むデバイスからギガビット毎秒で転送するネットワークインタフェースまで幅広い．オペレーティングシステムの I/O 管理は，次の 3 つの原則に従う．

- デバイスにアクセスするための標準的なインタフェースである**プロトコル**を定める．
- デバイス固有の処理を担う**専用のハンドラ**，すなわちデバイスドライバと割り込みハンドラを提供する．
- 個々のデバイスの詳細をシステムの他の部分から**切り離し**，アプリケーションがデバイスの操作ではなくファイルなどの抽象化を使えるようにする．

**定義 11.1** (デバイスレジスタ). **デバイスレジスタ**とは，CPU がデバイスとやり取りするために読み書きする，デバイス側のレジスタである．デバイスは通常，何をすべきかを CPU が書き込む**コマンド**レジスタ，デバイスとの間でデータを転送する**データ**レジスタ，そしてデバイスが何をしているかを CPU が読み取る**ステータス**レジスタを備える．

▷ device register

これらのレジスタがデバイスのインタフェースを成す（図 11.1）．その背後にあるのがデバイスの内部構成であり，デバイス自身の CPU であるマイクロコントローラ，デバイス上のメモリ，そしてマイクロフォンのアナログ・デジタル変換器のようなデバイス固有の回路からなる．オペレーティングシステムから見えるのはインタフェースだけであり，デバイスが内部構成をどう使ってコマンドを実行するかは隠されている．

**注意** デバイスレジスタは CPU のレジスタではない．デバイスに属し，命令中のレジスタ名ではなくアドレスで指定される．

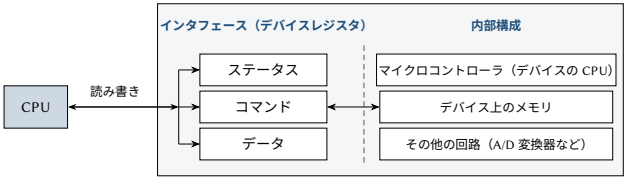



**図 11.1:** 基本的な I/O デバイス．CPU はコマンド・ステータス・データの各レジスタを通じてのみデバイスとやり取りする．コマンドを実行する内部構成はデバイス固有である．

## 11.2 CPU とデバイスのインターコネクト

**出典** P3L5，CPU とデバイスのインターコネクト；OSTEP ch. 36；Silberschatz ら ch. 12.

デバイスは，インターコネクト上のコントローラを介して CPU とメモリに接続される．

▷ Peripheral Component Interconnect

PCIe は慣習的にバスと呼ばれるが，各デバイスはポイントツーポイントのシリアルリンクとスイッチを通じて接続される．PCIe 4.0 の 1 レーンは片方向あたり約 2 GB s$^{-1}$，×16 のスロットは約 32 GB s$^{-1}$ である．PCIe 5.0 はこれを倍にし，1 レーンあたり約 4 GB s$^{-1}$ となる（PCI-SIG）．

**ヒント** `lspci -t` は PCI のトポロジをブリッジとデバイスの木として表示する．`lsusb -t` は USB について同じことを行う．

インターコネクトは，デバイスとシステムの他の部分との間でデータをどれだけ速く動かせるか，またいくつのデバイスを接続できるかを制限する．そうしたインターコネクトの標準的な一族が **PCI** である．当初の形態は共有の並列バスであり，PCI-X（PCI eXtended）がそれを高速化した．現在のプラットフォームは PCI Express（PCIe）を用いる．PCIe は PCI のソフトウェアインタフェースとの互換性を保ちながら，より大きな帯域幅，より速く低遅延なアクセス，より多くのデバイスの接続を提供する．

すべてのデバイスが PCIe に直接つながるわけではない（図 11.2）．PCIe に接続された SCSI コントローラは自身の SCSI バスにディスクをつなぎ，拡張バスインタフェースはキーボードのような低速な周辺機器を拡張（周辺）バスにつなぐ．ブリッジとなるコントローラがバスどうしの差異を吸収し，CPU とメモリを結ぶブリッジは両者を PCIe にも接続する．したがって CPU からデバイスへのアクセスは，複数のインターコネクトをまたぐことがある．

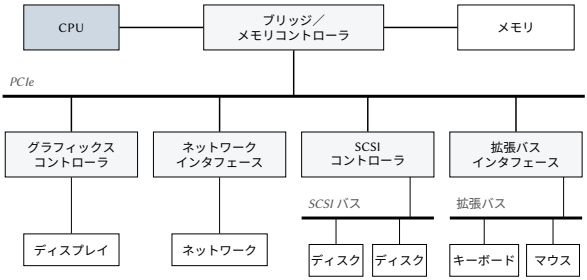



**図 11.2:** 典型的なプラットフォームのインターコネクト．デバイスコントローラは PCIe に接続され，ディスクとキーボードはそれぞれのコントローラの先の別のバスに接続される．PCIe は簡単のためバスとして描いてあるが，実際には各コントローラは自身のポイントツーポイントリンクで接続される．

## 11.3 デバイスドライバ

**出典** P3L5，デバイスドライバ；OSTEP ch. 36；Silberschatz ら ch. 12．

▷ device driver

オペレーティングシステム自身が，各デバイスを操作する方法の知識を持っているわけではない．

**定義 11.2** (デバイスドライバ). **デバイスドライバ**とは，ある一種類のデバイスに対するすべてのアクセス・管理・制御を担う，オペレーティングシステムのデバイス固有のソフトウェア構成要素である．

ドライバはデバイスの種類ごとに 1 つあり，通常はデバイスの製造者が提供する．モジュール構成のオペレーティングシステムでは，カーネルモジュール（第 1 章）として読み込まれる．一方でオペレーティングシステムは，ドライバが実装すべきインタフェース，すなわち**デバイスドライバフレームワーク**を標準化する．この標準化から 2 つの性質が得られる．

**デバイス独立性** オペレーティングシステムは，ある種類のすべてのデバイスを同じインタフェースを通じて使うので，個々のデバイスに特化する必要がない．

**デバイスの多様性** オペレーティングシステムは，新しいものを含め，任意に異なるデバイスを扱える．そのデバイス用のインタフェースを実装するドライバがあれば，デバイスはすぐに使える．

## 11.4 デバイスの種類

**出典** P3L5，デバイスの種類；Silberschatz ら ch. 12；Kerrisk ch. 14．

▷ block device

▷ character device

▷ network device

オペレーティングシステムはデバイスをいくつかのクラスに分類する．同じクラスのデバイスは同じやり方で使われ，クラスごとに固有のドライバインタフェースを持つ．区別されるクラスは 3 つである．

- ディスクのような**ブロックデバイス**は，データを固定長のブロックに格納し，個々のブロックを番号で直接読み書きできる．
- キーボードのような**キャラクタデバイス**は，文字の逐次的なストリームを渡し，あるいは受け取る．基本操作は 1 文字の取得と出力である．
- イーサネットインタフェースのような**ネットワークデバイス**は，両者の中間に位置

する．データのストリームを渡すが，固定長のブロックでも 1 文字ずつでもなく，大きさの異なる塊で渡す．

### 11.4.1 ファイルとしてのデバイス

オペレーティングシステムは，デバイスをファイルシステムの中に表現する．

**定義 11.3** (デバイスファイル). **デバイスファイル**は，スペシャルファイルとも呼ばれ，デバイスを表すファイルである．プロセスがそれに対して行う操作—open，read，write，close—は，そのデバイスのドライバによってデバイス上で実行される．

▷ device file

Unix 系システムでは，デバイスファイルは /dev ディレクトリに置かれ，このディレクトリは特別なファイルシステムが管理する．かつては devfs（Linux 2.6.18 で削除）であり，現在の Linux では devtmpfs である．devtmpfs は tmpfs（第 9 章の POSIX 共有メモリの置き場）の一種で，カーネルが中身を作り，udev が管理する．`ls -l` による詳細表示では，ブロックデバイスファイルは種別 b，キャラクタデバイスファイルは c と示される．デバイスがファイルに見えるので，デバイスについて何も知らないプログラムでもそれを使える．

Linux ではネットワークデバイスが例外で，デバイスファイルを持たず，ソケットを通じて使われる．

`ls -l /dev` に表示されるメジャー番号とマイナー番号は，ドライバとデバイスのインスタンスを識別する（Kerrisk ch. 14）．

**例 11.4.** 次のコマンドは，1 台目のディスクの先頭 512 バイトを通常のファイルへコピーする．

```
dd if=/dev/sda of=head.img bs=512 count=1
```

dd は open と read と write しか使わない．入力がブロックデバイスであることも，ブロックを取り出すディスクのプロトコルも，オペレーティングシステムとディスクのドライバが担う．

すべてのデバイスファイルがハードウェアに対応するわけではない．

**定義 11.5** (疑似デバイス). **疑似デバイス**とは，他と同じくデバイスファイルで表現されるが，物理的なハードウェアに対応しないデバイスである．そのドライバは，デバイスの機能をすべてソフトウェアで実装する．

▷ pseudo device

例えば /dev/zero を読むとゼロバイトの無限のストリームが得られ，指定した大きさのゼロで埋めたファイルを作るのに便利である．また/dev/null は書き込まれたものをすべて捨て，読み出しには常にファイルの終わりを返す．

> **要点**
> デバイスは，コマンド・データ・ステータスの各レジスタを通じて，デバイスドライバによって操作される．ドライバはデバイスを，オペレーティングシステムがクラス全体に対して標準化したインタフェースの背後に隠す．クラスはブロックデバイス，キャラクタデバイス，ネットワークデバイスであり，前の 2 つはデバイスファイルとして表現されるので，通常のファイル操作でデバイスを扱える．

## 11.5 CPU とデバイスの相互作用

デバイスを使うには，CPU がそのレジスタをアドレス指定でき，デバイスが注意を必要とするときに CPU へ伝えられなければならない．レジスタをアドレス指定する方法は 2 つある．

**出典** P3L5，CPU とデバイスの相互作用；OSTEP ch. 36；Silberschatz ら ch. 12．

**定義 11.6** (メモリマップド I/O). **メモリマップド I/O** では，デバイスレジスタが特定の物理アドレスのメモリ位置として CPU に見える．物理アドレス空間の一部がデバイスとのやり取りに充てられ，CPU はそのアドレスに対する通常のロード命令やストア命令でレジスタにアクセスする．プラットフォームはそのアクセスを，メモリではなくデバイスへ導く．

▷ memory-mapped I/O

**注意** メモリマップドレジスタは通常のメモリではない．通常キャッシュ不可（プリフェッチ可能なフレームバッファなどはライトコンバイニング）で対応付けられ，ドライバはポインタ参照ではなく `read` や `write` でアクセスする．

▷ I/O port

メモリマップド I/O は，ファイルの内容をプロセスの仮想アドレス空間に対応付けるメモリマップドファイル（第 9 章）と混同してはならない．PCI デバイスでは，コンフィギュレーション空間にあるベースアドレスレジスタ（BAR）が，そのデバイスの領域の大きさと位置を決める．BAR はブートの間に設定される．

2 つ目の方法は，x86 の `in` と `out` のような専用の CPU 命令を使う．**I/O ポート**とは，メモリアドレスとは別に，デバイスレジスタを指定するアドレスである．命令は対象のポートと，レジスタへ書き込む値を指定し，あるいは読み出した値を受け取る．

x86 のポート空間は 64 KiB（16 ビットのポート番号）しかなく，今日のデバイスとのやり取りの大半がメモリマップド I/O で行われる理由の 1 つである．

### 11.5.1 割り込みとポーリング

逆方向では，デバイスは要求を完了したこと，あるいはデータが到着したことを CPU に知らせなければならない．デバイスは割り込み（第 5 章）を発生させることができ，あるいはオペレーティングシステムがポーリングによって知ることもできる．

**定義 11.7** (ポーリング). デバイスが割り込みを発生させるのを待つのではなく，オペレーティングシステムが自ら選んだ時点でデバイスのステータスレジスタを読み，デバイスが注意を必要としているかどうかを調べる技法を**ポーリング**と呼ぶ．

▷ polling

表 11.1 に両者を比較する．割り込みはデバイスが生成すると同時に CPU に届くが，割り込みごとに処理のオーバーヘッドがかかる．割り込みハンドラの実行，割り込みマスクの設定と解除に加えて，割り込まれたスレッドのデータが CPU キャッシュから追い出されるといった間接的な影響もある．ポーリングはオペレーティングシステムにとって都合のよいときに行われるが，まれにしか行わなければイベントは次のポーリングまで待たされ，頻繁に行えばほとんどのポーリングは何も見つけずに CPU 時間を浪費する．どちらが望ましいかは，デバイスの種類と，イベントがどれだけ頻繁でどれだけ速く処理されねばならないかといった目的に依存する．[1]

[1] Linux のネットワークドライバは NAPI によって両者を切り替える．最初のパケットが届くとドライバはそのインタフェースの受信割り込みを禁止し，カーネルは滞留した分がなくなるまでデバイスをポーリングし，その後で割り込みを再び許可する．負荷の軽いインタフェースは割り込み駆動に，忙しいインタフェースはポーリングになる．

**表 11.1:** 割り込みとポーリング．

| | 割り込み | ポーリング |
|---|---|---|
| 開始する主体 | デバイス | オペレーティングシステム |
| イベントの認識 | 生成されると同時 | 次のポーリング時 |
| コスト | 割り込みごとの処理オーバーヘッド，キャッシュ汚染 | ポーリングごとに 1 回のステータス読み出し．何も見つからない回も含む |
| 欠点 | イベントが頻繁なときのオーバーヘッド | ポーリングがまれなら遅延，頻繁なら CPU のオーバーヘッド |

**例 11.8.** 割り込みを受けるコストが，イベント自体の処理を除いて CPU 時間で 4 µs であり，前回のポーリング以降に溜まったイベントをすべて回収する 1 回のポーリングのコストが 0.5 µs であるとする．毎秒 50 件のイベントを報告するデバイスは，割り込みでは毎秒 200 µs の CPU 時間を要する．1 ms ごとのポーリングなら毎秒 500 µs を要し，各イベントは最大 1 ms 遅れる．一方，毎秒 100 000 パケットを受信するネットワークインタフェースは，パケットごとに 1 回の割り込みでは毎秒 400 ms（CPU の 40 %）を要するのに対し，1 ms ごとのポーリングなら依然として毎秒 500 µs で済む．

## 11.6 デバイスアクセス：プログラム制御 I/O と DMA

CPU とデバイスがデータをやり取りする方法は 2 つあり，データが CPU を通るかどうかが異なる．どちらの方法も，コマンドの発行には 11.1 節のデバイスレジスタを使う．

**出典** P3L5，デバイスアクセス：PIO，デバイスアクセス：DMA；OSTEP ch. 36.

**定義 11.9** (プログラム制御 I/O). **プログラム制御 I/O**（PIO）では，CPU がやり取りのすべての段階をデバイスレジスタを通じて行う．操作を要求するためにコマンドレジスタへ書き込み，データレジスタへの書き込みまたは読み出しによって，レジスタの大きさずつデータを運ぶ．

▷ programmed I/O

PIO はレジスタ以外のハードウェアによる支援を必要としない．PIO でネットワークパケットを送るには，CPU は送信を要求するコマンドをコマンドレジスタに書き，パケットの先頭部分をデータレジスタにコピーし，パケット全体が転送されるまでコピーを繰り返す（図 11.3a）．CPU のアクセス回数はデータの大きさとともに増える．

**定義 11.10** (ダイレクトメモリアクセス). **DMA** では，CPU は依然としてコマンドレジスタを通じてコマンドを発行するが，データは CPU を通らない．CPU は主メモリ上のバッファにデータを置き，そのバッファのアドレスと大きさをデバイスに渡す．すると DMA コントローラが，メモリとデバイスの間でデータを自ら転送する．

▷ direct memory access

DMA でパケットを送るには，CPU はコマンドを書き，パケットをメモリのバッファに置き，バッファのアドレスと大きさを DMA コントローラに設定する（図 11.3b）．CPU のアクセス回数はもはやデータの大きさに依存しない．

**例 11.11.** 8 バイトのデータレジスタを持つネットワークインタフェースが 9000 バイトの

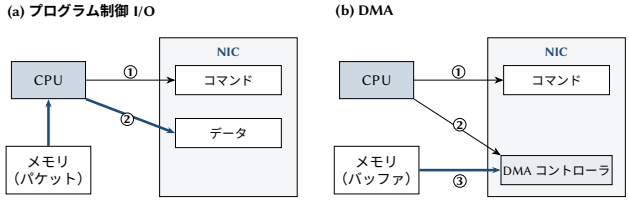



**図 11.3:** パケットの送信．(a) プログラム制御 I/O では，CPU がパケットを少しずつデータレジスタへコピーする．(b) DMA では，CPU はコマンドの書き込みと DMA の設定を行うだけで，デバイスがメモリからパケットを読む．

フレームを送る．PIO では，CPU はコマンドのために 1 回，データのために 9000/8 = 1125 回，合計 1126 回のアクセスを行う．DMA では 2 回，すなわちコマンドに 1 回と DMA の設定に 1 回である．

DMA は必要な段階が少ないが，DMA の設定は 1 回のレジスタ書き込みよりも複雑な操作である．そのため小さな転送では，PIO による数回のレジスタ書き込みの方が 1 回の DMA 設定より安く済むことがある．PIO が必要とするデータアクセスのコストが DMA の設定より小さい限り，PIO の方が安い．さらに，デバイスはバッファを物理アドレスでアクセスするため，別の要件が生じる．バッファは転送が完了するまで物理メモリに留まらなければならない．そのためバッファのページはピン留め（第 8 章）される，すなわちスワッピングの対象から外される．さもなければ，デバイスはバッファを置き換えたページを読んだり上書きしたりしかねない．

**ヒント** DMA が引き合うのは，転送がレジスタ幅の何倍にも及ぶときである．数バイトの転送では PIO の方が安い．

物理アドレスをそのまま渡すことは，メモリ全体をデバイスに委ねることでもある．そのため現在のプラットフォームは IOMMU（Intel VT-d，AMD-Vi）を介在させ，ドライバはアドレスを渡す代わりにバッファを DMA 用に対応付ける．

**要点**
CPU はメモリマップド I/O か I/O ポートを通じてデバイスレジスタにアクセスし，デバイスは割り込みによって CPU に知らせる．あるいはオペレーティングシステムがデバイスをポーリングする．データは CPU を経由してレジスタ単位で運ばれる（PIO）か，メモリバッファとデバイスの間で直接運ばれる（DMA）．後者は CPU のアクセス回数を減らすが，設定がより複雑になり，メモリのピン留めを要する．

## 11.7 典型的なデバイスアクセス

アプリケーションが自らデバイスとやり取りすることは通常なく，その要求はオペレーティングシステムのいくつもの層を通る．デバイスアクセスの典型的な経路は次のとおりである（図 11.4a）．

1. プロセスが，ソケットへのデータ送信やファイルからの読み出しのような操作を要求するシステムコールを行う．
2. カーネルが，そのデバイスに対応するカーネル内スタックを実行する．TCP/IP スタックがパケットを組み立てる，あるいはファイルシステムが要求されたデータを保持するブロックを決定する．
3. カーネルが適切なデバイスドライバ，ここではネットワークドライバかブロックデバイスドライバを呼び出す．
4. ドライバが，PIO によるレジスタ書き込みか DMA の設定によって，要求をデバイス上に設定する．
5. デバイスが要求を実行する．パケットを送信する，あるいはディスクからブロックを読む．

結果と完了通知は同じ経路を逆にたどる．デバイスが割り込みを発生させるかポーリングされ，ドライバがデータをスタックの上へ渡し，システムコールがプロセスへ戻る．

**出典** P3L5，典型的なデバイスアクセス，OS バイパス；OSTEP ch. 36.

### 11.7.1 OS バイパス

カーネルを経由することは必須ではない．

**定義 11.12** (OS バイパス). デバイスのレジスタとメモリをユーザプロセスから直接アク

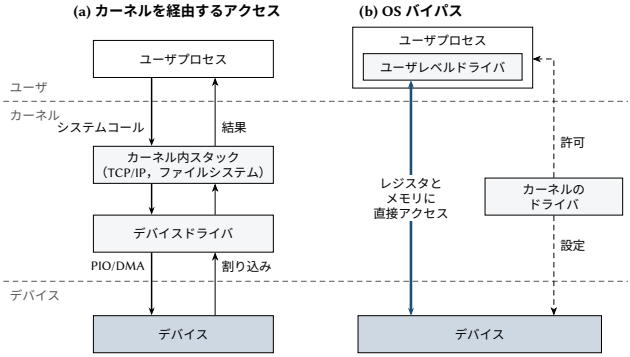



**図 11.4:** (a) 典型的なデバイスアクセスは，システムコール，カーネル内スタック，デバイスドライバを通る．(b) OS バイパスでは，プロセスはユーザレベルドライバを通じてデバイスにアクセスし，カーネルはアクセスの設定と制御を行うだけである．

▷ operating system bypass

セスできるようにするデバイスアクセスの方式を **OS バイパス**と呼ぶ．オペレーティングシステムはアクセスを設定した後は関与せず，プロセスはカーネルのコードを実行せずにデバイス操作を行う（図 11.4b）．

▷ user-level driver

本来カーネルのドライバで実行されるコードは，**ユーザレベルドライバ**，すなわちアプリケーションにリンクされるライブラリが提供する．オペレーティングシステムは粗い粒度の制御を保持する．例えばデバイスの有効化と無効化や，別のプロセスにアクセス権を与えることができる．[2] バイパスはデバイスの機能に依存する．

- デバイスは十分な数のレジスタを備えていなければならない．一部をユーザプロセスに対応付けつつ，制御に必要なレジスタはカーネルだけがアクセスできるように残すためである．
- 複数のプロセスがデバイスを使う場合，デバイスは**逆多重化**，すなわちデータがどのプロセスのものかを判別できなければならない．典型的なデバイスアクセスではカーネルがこれを行う．例えばネットワークスタックは，到着したパケットのヘッダを調べて受信プロセスを見つける．バイパスではデバイス自身がそれを行わねばならない．

[2] カーネルバイパスは高性能ネットワークでは日常的である．InfiniBand や RoCE のアダプタは verbs API を通じてキューペアをプロセスに見せ，DPDK はイーサネットアダプタを完全にユーザ空間から駆動する．カーネルは設定を行うだけである．第 15 章はこうしたアダプタをリモートダイレクトメモリアクセスに使う．

PCIe デバイスは SR-IOV でこれを行える．第 12 章を参照．

## 11.8 同期アクセスと非同期アクセス

**出典** P3L5，同期アクセスと非同期アクセス；Silberschatz ら ch. 12.

デバイスが要求を実行している間，それを発行したスレッドは待つこともできるし，実行を続けることもできる．

▷ synchronous I/O

**定義 11.13** (同期 I/O). **同期 I/O** では，I/O 要求を発行したスレッドは，その要求が完了するまでブロックされる．オペレーティングシステムはそのスレッドを，デバイスに対応する待ち行列，すなわち第 2 章の I/O キューに置き，デバイスが完了を報告したときに再び実行可能にする．

**非同期 I/O**（第 6 章）では，呼び出しは直ちに戻り，スレッドは実行を続ける（図 11.5）．スレッドは結果を後から 2 つの方法のいずれかで知る．スレッド自身が操作の完了を確認して結果を取り出すか，オペレーティングシステムが，操作が完了して結果が利用可能になったことを通知するかである．カーネルがデバイスへの非同期アクセスに対応していない場合には，第 6 章の Flash ウェブサーバのように，主スレッドに代わってブロッキング呼び出しを行うヘルパスレッドやヘルパプロセスを使って，ユーザ空間で模倣できる．

glibc は POSIX の `aio_read` をまさにこのヘルパスレッドによる模倣で実装している．`io_uring` は，この模倣を不要にする Linux カーネルのインタフェースである（`io_uring(7)`）．

> **要点**
> デバイスアクセスは通常，システムコールからカーネル内スタックとドライバを経てデバイスに至り，結果は同じ経路を戻る．OS バイパスは，デバイスが自身の資源を保護し逆多重化できるとき，この経路からカーネルを取り除く．経路にかかわらず，要求したスレッドはブロックする（同期 I/O）か，実行を続けて後から結果を回収する（非同期 I/O）．

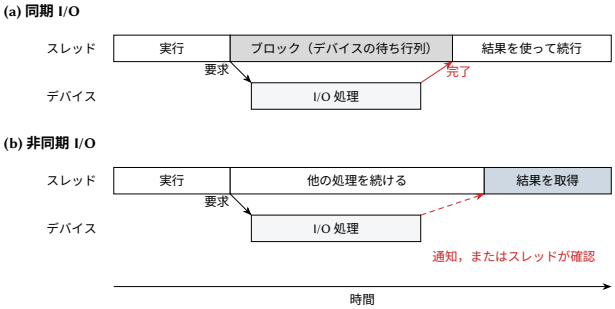



**図 11.5:** (a) 同期 I/O では，デバイスが操作を完了するまでスレッドはブロックする．(b) 非同期 I/O では，スレッドは実行を続け，通知を受けた後または自ら確認した後に結果を取得する．

## 11.9 ブロックデバイススタック

プロセスがブロックデバイスをブロック番号で指定することはまれである．プロセスは，論理的な記憶単位であるファイルを，`open`，`read`，`write`，`close` といった操作を持つ POSIX API を通じて使う．API の下でカーネルは層状に構成され，各層はその下の層のインタフェースを使う（図 11.6）．

**出典** P3L5，ブロックデバイススタック；OSTEP ch. 36，39．

**定義 11.14** (ファイルシステム). **ファイルシステム**とは，記憶デバイスのブロックをファイルとディレクトリへと編成するカーネルの構成要素である．各ファイルのデータがどこにあるか，またファイルをどう見つけてアクセスするかを決める．

▷ file system

オペレーティングシステムは，すべてのファイルシステムが実装するインタフェースを規定する．ファイルシステムの下には，ブロックデバイスどうしの差異を隠す層がある．

**定義 11.15** (汎用ブロック層). **汎用ブロック層**とは，任意のブロックデバイス上でブロックを読み書きするための，オペレーティングシステムが標準化した単一のインタフェースを提供し，各要求を個々のデバイスのドライバのインタフェースへ変換するカーネルの層である．

▷ generic block layer

したがって同じファイルシステムが，あらゆる種類のブロックデバイス上で動く．汎用ブロック層が各要求をデバイスのドライバに合わせ，ドライバが SCSI や ATA といったそのデバイスのプロトコルでハードウェアとやり取りする．

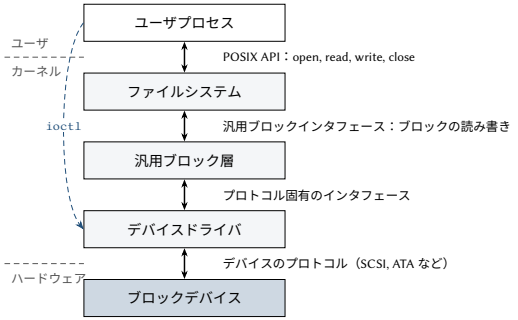



**図 11.6:** ブロックデバイススタック．各層は，その下に示したインタフェースを使う．`ioctl` はデバイス固有の要求をドライバへ運ぶ．

デバイスに対する操作の中には，`read` にも `write` にも当てはまらないものがある．Linux をはじめとする Unix 系システムは，デバイス固有の情報にプロセスがアクセスし操作するためのシステムコール `ioctl`（「I/O control」）を提供する．これは，開いたデバイスファイルのファイルディスクリプタ（11.11 節），デバイスのクラスまたはドライバに対して定義された要求コード，そして要求に応じて要求固有のデータへのポインタを取る．

▷ ioctl

**注意** `ioctl` の要求コードと引数の形式はドライバ固有なので，移植性のあるプログラムはこれを避ける（`ioctl(2)`）．

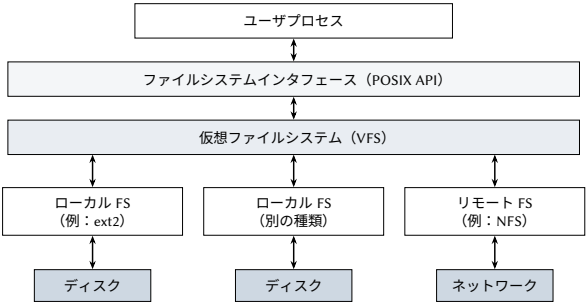



**図 11.7:** 仮想ファイルシステムは，ファイルシステムインタフェースの操作を，具体的なローカルおよびリモートのファイルシステムへ対応付ける．

**例 11.16.** あるプログラムが 1 台目の光学ドライブのトレイを開く．

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/cdrom.h>

int main(void)
{
  int fd = open("/dev/sr0", O_RDONLY | O_NONBLOCK);
  if (fd >= 0) {
    ioctl(fd, CDROMEJECT);  /* read/write では表せない */
    close(fd);
  }
  return 0;
}
```

O_NONBLOCK フラグにより，ディスクが入っていなくてもデバイスファイルを開ける．

## 11.10 仮想ファイルシステム

**出典** P3L5，仮想ファイルシステム；OSTEP ch. 39；Silberschatz ら ch. 15；Kerrisk ch. 14.

▷ virtual file system

ほとんどのシステムでは，単一のローカルデバイス上の単一のファイルシステムでは足りない．ファイルは複数のデバイスに存在しうるし，デバイスによって最も適したファイルシステムの実装も異なりうる．さらにファイルはローカルデバイス上にまったく存在せず，ネットワーク越しにアクセスするリモートサーバ上にあるかもしれない（第 14 章）．

**定義 11.17** (仮想ファイルシステム). **仮想ファイルシステム**（VFS）とは，アプリケーションに 1 つのファイルシステムインタフェースを見せ，各操作を，ファイルが存在する具体的なファイルシステムへ対応付けるカーネルの層である．

VFS は下層のファイルシステムの詳細をすべてアプリケーションから隠す（図 11.7）．アプリケーションはファイルがどこにあっても同じ API を使い，VFS は各操作を該当するファイルシステム—ディスク上のローカルな ext2 ファイルシステムや，NFS のようなリモートファイルシステム—へ転送する．

## 11.11 VFS の抽象化

**出典** P3L5，仮想ファイルシステムの抽象化，ディスク上の VFS；OSTEP ch. 39–40；Silberschatz ら ch. 13，15；Kerrisk ch. 14.

▷ file descriptor

記述子 0，1，2 はプロセスの標準入力・標準出力・標準エラーである．開いているファイル数の通常の既定のソフトリミットは 1024 である（`ulimit -n`，`getrlimit(2)` の `RLIMIT_NOFILE`）．

具体的なファイルシステムはいずれも，VFS が定める少数の抽象化を実装する．VFS が操作する要素は**ファイル**である．プロセスは，開いたファイルをファイルディスクリプタによって参照する（図 11.8）．

**定義 11.18** (ファイルディスクリプタ). **ファイルディスクリプタ**とは，プロセスにとっての開いたファイルの，オペレーティングシステムによる表現である．プロセスが open から得て，read，write，sendfile，ロック，close といったファイルへの操作に渡す小さな整数である．

ファイル自身の属性と，そのデータブロックの位置は，ファイルの i ノードに記録さ

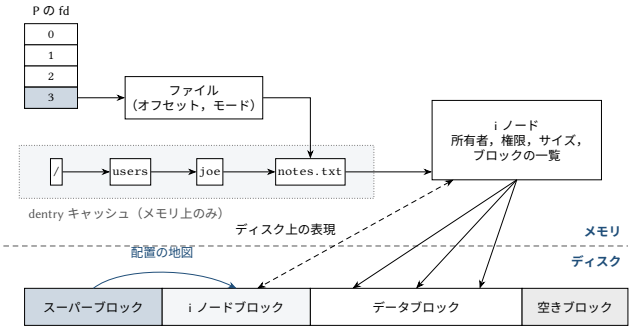



**図 11.8:** VFS の抽象化．プロセス P のファイルディスクリプタ 3 は開いたファイルを指し，そのファイルの dentry が i ノードへ導く．パスの dentry はメモリ上にのみ存在する．ディスク上では，スーパーブロックが，それが位置を示す構造とともに，i ノード・データ・空き領域を保持するブロックの地図となる．

れる．

**定義 11.19** (i ノード)．**i ノード**とは，ファイルの永続的な表現である．ファイルのすべてのデータブロックを列挙する索引であり，デバイス・権限・サイズといったメタデータを伴う．

▷ inode

**注意** 複数の名前が 1 つの i ノードを共有でき（ハードリンク），名前の削除はリンク数を 1 減らすだけである．リンク数が 0 になり開いた記述子もなくなるとファイルは解放される．

ディレクトリもまたファイルであり，そのデータは，ディレクトリが含むファイルとそのノードについての情報である．したがってファイル名はディレクトリに現れ，i ノードには現れない．パスの各構成要素は，さらに別のオブジェクトで表される．

**定義 11.20** (ディレクトリエントリ)．**ディレクトリエントリ**（dentry）とは，パスの 1 つの構成要素に対応する VFS のオブジェクトである．

▷ directory entry

パス `/users/joe` の解決では，`/`，`users`，`joe` のそれぞれに対して dentry が作られる．VFS は dentry を **dentry キャッシュ**に保持するので，同じ接頭辞を持つパスを後で解決するときに，対応するディレクトリを読み直さずに済む．dentry はソフトステートであり，メモリ上にのみ存在してディスク上の表現を持たない．

▷ dentry cache

**定義 11.21** (スーパーブロック)．**スーパーブロック**は，ファイルシステムの配置に関する，ファイルシステム固有の情報を保持する．

▷ superblock

**例 11.22.** あるプロセスが `/users/joe/notes.txt` を開く．`/` の dentry から始めて，VFS はパス上の各ディレクトリを読んで次の構成要素とその i ノードを見つけ，その過程で `users`, `joe`, `notes.txt` の dentry を作る．続いてそのプロセスが `/users/joe/todo.txt` を開くと，`/`，`users`，`joe` の dentry は dentry キャッシュで見つかり，最後の構成要素だけをディレクトリ `joe` の中で探せばよい．

### 11.11.1 ディスク上の VFS

ディスク上では，これらの抽象化は次のように配置される（図 11.8）．ファイルのデータはデータブロックに保持される．ファイルのブロックを管理する i ノードもまた，ディスク上のいずれかのブロックに置かれる．スーパーブロックは，それが位置を示す構造，例えば ext2 のビットマップ（11.12 節）とともに，どのブロックが i ノードを保持し，どれがデータを保持し，どれが空いているかという，ディスクブロック全体の地図を提供する．

## 11.12 第 2 拡張ファイルシステム

Linux は長らく，ディスク上の構造が VFS の抽象化と密接に対応するファイルシステムの一族を使ってきた．第 2 拡張ファイルシステム **ext2** は，長年にわたり Linux の既定のファイルシステムであり，その後継である ext3 と ext4 の基礎でもある．ext2 のパーティションは，ブート用に予約された 1024 B（ブロックが 1 KiB のときはブロック 0 全体）から始まり，残りはブロックのグループに分割される（図 11.9）．

▷ ext2

**出典** P3L5，ext2；OSTEP ch. 40–41；Silberschatz ら ch. 14．

▷ block group

**定義 11.23 (ブロックグループ).** **ブロックグループ**とは，ext2 がディスクパーティションを分割してできる，等しい大きさ（最後のものを除くことがある）の連続したブロックのグループの 1 つである．各ブロックグループは，自身の i ノードとデータブロックを，それらを記述するメタデータとともに保持する．

グループはディスクの幾何学的な領域ではない．1 ブロック分のビットマップで管理できるだけのブロックを保持する．ブロックが 4 KiB なら 8 × 4096 = 32 768 ブロック，すなわち 128 MiB である．

各ブロックグループは次の構造を含む．ただし最初の 2 つは，後で述べるとおり一部のグループにしかない．

**スーパーブロックのコピー** ファイルシステム全体の情報，すなわち i ノードの数とディスクブロックの数，空き i ノードと空きブロックの数，そしてブロックの大きさを記録する．

**グループ記述子表のコピー** グループごとに 1 つの記述子を持ち，そのグループのビットマップと i ノード表の位置，および空きブロック数・空き i ノード数・ディレクトリ数を記録する．

**ブロックビットマップと i ノードビットマップ** そのグループのどのブロックとどの i ノードが空いているかを管理する．

**i ノード表** ファイルシステムの i ノードのうち連続した範囲を保持する．i ノードは 1 から最大数まで番号付けされ，ファイルごとに 1 つある．

**データブロック** ファイルのデータを保持する．

スーパーブロックとグループ記述子表はファイルシステム全体を記述する．通常使われるのはブロックグループ 0 のコピーであり，他のコピーはバックアップとして働く．`mke2fs` が既定で有効にする `sparse_super` 機能のもとでは，こうしたコピーを持つのはグループ 0 と 1，および番号が 3，5，7 のべき乗であるグループだけであり（ext4(5)），他のグループはビットマップから直接始まる．あるファイルの i ノードとデータブロックを同じグループに置くことで，関連するブロックがディスク上で近くに配置され，ディスクヘッドがそれらの間を移動する距離が減る．

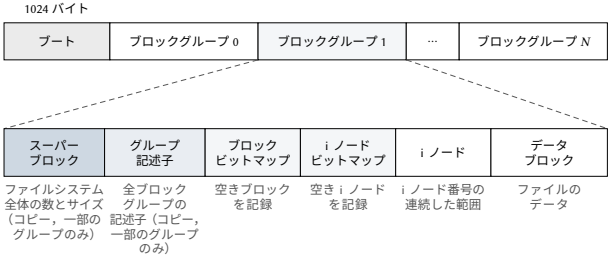



**図 11.9:** ext2 のパーティションの配置．ブート用に予約された領域の後，パーティションはブロックグループに分割され，各グループは自身のビットマップ，i ノード，データブロックを持つ．グループ 0 と 1，および番号が 3，5，7 のべき乗であるグループは，スーパーブロックとグループ記述子表のコピーも持つ（sparse_super）．

## 11.13 i ノード

**出典** P3L5，i ノード，間接ポインタを持つ i ノード；OSTEP ch. 40；Silberschatz ら ch. 14；Kerrisk ch. 14.

ファイルは i ノードで識別され，i ノードはファイルのすべてのディスクブロックの索引である．ブロックを順に列挙し，メタデータを伴う．最も単純な形では，i ノードはファイルの全ブロックのブロック番号を保持する．このとき逐次アクセスもランダムアクセスも同じくらい容易である．ブロックサイズを $B$ とすると，オフセット $o$ のバイトは，一覧のエントリ $\lfloor o/B \rfloor$ が示す番号のブロックにある．しかしファイルの大きさは，i ノードに収まるエントリの数で制限される．

**例 11.24.** 512 バイトの i ノードがメタデータに 64 バイトを使うとすると，4 バイトのブロック番号を 448/4 = 112 個収められる．4 KiB のブロックでは，どのファイルも 112 × 4 KiB = 448 KiB より大きくなれない．

### 11.13.1 間接ポインタ

小さな i ノードで大きなファイルを扱うために，i ノードは数種類のポインタを持つ（図 11.10）．

▷ direct pointer

**直接ポインタ** **直接ポインタ** は，データブロックを指す．

**間接ポインタ 間接ポインタ** は，データブロックへのポインタを含むブロックを指す．
**二重間接ポインタ 二重間接ポインタ** は，データブロックへのポインタのブロックを指すポインタの，そのブロックを指す．
三重間接ポインタはさらにもう 1 段を加える．ext2 と多くの Unix 系ファイルシステムの i ノードは，12 個の直接ポインタと，各種類の間接ポインタを 1 つずつ含む．

▷ indirect pointer
▷ double indirect pointer

ext2 の実際の値は，i ノードが 256 B（非常に小さなファイルシステムでは 128 B）でパーティション 16 KiB ごとに 1 つ，ブロック番号が 4 バイトである（`mke2fs.conf(5)`，`mke2fs(8)`）．

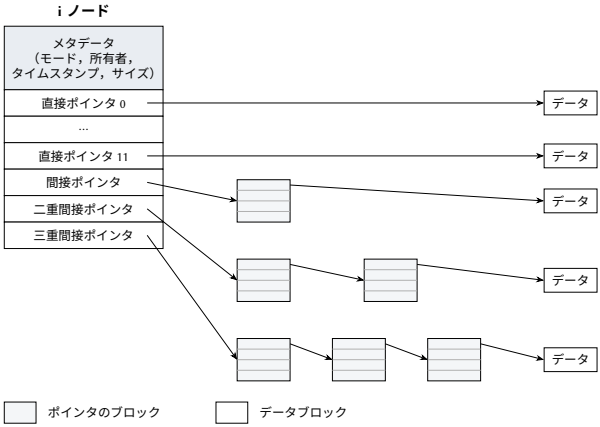



**図 11.10:** 12 個の直接ポインタと，単一・二重・三重の間接ポインタを持つ i ノード．間接の段が 1 つ増えるごとに，到達できるデータブロックの数は 1 ブロックあたりのポインタ数倍になる．

ブロックサイズを $B$，ポインタの大きさを $p$ とすると，1 ブロックには $P = B/p$ 個のポインタが入る．$n_d$ 個の直接ポインタと，3 種類の間接ポインタを 1 つずつ持つ i ノードが扱えるファイルの大きさは，最大で

$$S_{\max} = \left(n_d + P + P^2 + P^3\right) \times B. \tag{11.1}$$

**例 11.25.** 4 KiB のブロックと ext2 の 4 バイトのブロック番号では，1 ブロックに $P = 4096/4 = 1024$ 個のポインタが入る．12 個の直接ポインタは $12 \times 4\,\text{KiB} = 48\,\text{KiB}$ を，間接ポインタは $1024 \times 4\,\text{KiB} = 4\,\text{MiB}$ を，二重間接ポインタは $1024^2 \times 4\,\text{KiB} = 4\,\text{GiB}$ を，三重間接ポインタは $1024^3 \times 4\,\text{KiB} = 4\,\text{TiB}$ を扱える．式 (11.1) によれば上限はおよそ 4.004 TiB であり，1 KiB のブロックでは $P = 256$ で上限はおよそ 16.06 GiB となる．ただし 4 KiB のブロックでも，実際の ext2 のファイルは 2 TiB で頭打ちになる．i ノードはファイルのブロック数を，512 バイトのセクタ単位の 32 ビットのフィールドで数えるからである（$2^{32} \times 512\,\text{B}$）．

間接ポインタの利点は，小さな i ノードで非常に大きなファイルを扱えることである．代償はファイルアクセスが遅くなることであり，間接の段が 1 つ増えるごとに，データブロックに到達するまでのディスクアクセスが 1 回増える．直接ポインタを通じてブロックを読むには，i ノードに 1 回とデータブロックに 1 回の計 2 回のディスクアクセスが必要であり，二重間接ポインタを通じて読むには最大 4 回が必要である．

**例 11.26.** 前の例のパラメータで，あるプロセスがファイルのオフセット 600 MiB のバイトを読む．これはファイルのブロック $600 \times 1024/4 = 153\,600$ にある．ブロック 0–11 には直接に，ブロック 12–1035 には間接ポインタを通じて，ブロック 1036–1 049 611 には二重間接ポインタを通じて到達するので，この読み出しには i ノードと，2 つのポインタのブロックと，データブロックが必要であり，いずれもキャッシュされていなければ 4 回のディスクアクセスとなる．

**ヒント** ファイルのブロック $i$ にどのポインタで到達するかは，$i$ を累積和 $n_d$，$n_d + P$，$n_d + P + P^2$ と比べれば分かる．最初に下回った和がポインタの種類を示す．

**要点**
VFS は，ファイルディスクリプタ，dentry，i ノード，スーパーブロックを通じてファイルを操作する．ディスク上でのファイルは，その i ノードと，i ノードが索引付けするデータブロックである．直接ポインタと間接ポインタにより，固定長の i ノードで非常に大きなファイルを索引付けできるが，間接の段ごとに追加のディスクアクセスという代償を払う．

## 11.14 ディスクアクセスの最適化

**出典** P3L5，ディスクアクセスの最適化；OSTEP ch. 37，40，42；Silberschatz ら ch. 11，14；Kerrisk ch. 13–14.

ファイルへのアクセスにはディスクアクセスのコストがかかり，ハードディスクでは逐次的でないアクセスのたびにヘッドを動かす時間がかかる．ファイルシステムは，ファイルアクセスのオーバーヘッドを減らすために 4 つの技法を用いる（表 11.2）．

これらの技法が埋めようとする差は大きい．7200 rpm のディスクは逐次読み出しでは 150 MB s$^{-1}$ to 280 MB s$^{-1}$ に達する（ベンダのデータシート）が，4 KiB のランダムアクセスは毎秒 100 回程度で，毎秒数百キロバイトにしかならない（OSTEP ch. 37）．

**表 11.2:** ファイルアクセスのオーバーヘッドを減らす技法．

| 技法 | 効果 |
|---|---|
| キャッシングとバッファリング | ディスクアクセスの削減 |
| I/O スケジューリング | ヘッド移動の削減．逐次アクセスが増え，ランダムアクセスが減る |
| プリフェッチ | 局所性を利用してキャッシュヒットを増やす |
| ジャーナリング | 更新をメモリだけに留めることなく，書き込みのランダムアクセスを減らす |

### 11.14.1 キャッシングとバッファリング

▷ buffer cache

**定義 11.27** (バッファキャッシュ). **バッファキャッシュ**とは，主メモリ上のファイルデータのキャッシュである．読み出しは可能な限りキャッシュから提供され，書き込みはキャッシュを更新する．変更されたデータは定期的に，あるいはプロセスが `fsync` で要求したときに，ディスクへ書き出される．

このキャッシュの大きさは固定ではない．第 8 章の置換方針が残したメモリをそのまま使う．

▷ page cache

カーネルはファイルデータを**ページキャッシュ**に保持するので，繰り返しの読み出しと遅延書き込みはデバイスにアクセスせずに済む．Linux はバッファキャッシュとページキャッシュを統合している．

更新を確実に永続化しなければならないプロセスは，書き込みに続けて `fsync` を呼ぶ．

```
write(fd, rec, sizeof *rec);  /* キャッシュを更新 */
fsync(fd);                    /* ディスクへ書き出す */
```

**注意** `write` が成功しても，データがディスク上にあるとは限らない．クラッシュすると，まだキャッシュにあるデータは失われる．

`fsync` は，ファイルのデータがデバイスへ書き込まれた後にのみ戻る．

### 11.14.2 I/O スケジューリング

▷ I/O scheduling

▷ elevator algorithm

ディスクヘッドの移動を減らすように，保留中のディスク要求を並べ替えることを **I/O スケジューリング**と呼ぶ．要求は，逐次アクセスを最大にしランダムアクセスを最小にする順序で処理される．要求をブロックアドレス順に並べ替え，ヘッドの一度の掃引で処理する方式を**エレベータアルゴリズム**と呼び，Linux の I/O スケジューラはこれを改良している．

**例 11.28.** ヘッドがブロック 10 に位置しており，ブロック 90，12，91，13，50 への書き込みがこの順に到着する．ブロック番号がディスク上の位置を反映すると仮定すると，到着順に処理した場合ヘッドは 80 + 78 + 79 + 78 + 37 = 352 ブロック分を移動する．12，13，50，90，91 の順に処理すれば移動は 2 + 1 + 37 + 40 + 1 = 81 ブロック分で済み，12 と 13，90 と 91 の組は逐次書き込みになる．

エレベータアルゴリズムは SCAN とも呼ばれる．Linux は `mq-deadline`，BFQ，Kyber を備え，NVMe では通常 `none` を使う．

### 11.14.3 プリフェッチ

▷ prefetching

要求されたよりも多くのブロックを読むことを**プリフェッチ**あるいは先読みと呼ぶ．あるブロックを読むときに，次に必要になりそうなブロックを一緒に読む．プリフェッチは局所性—逐次的に読まれているファイルは間もなく後続のブロックを必要とする—を利用し，バッファキャッシュのヒット率を高める．プロセスがファイルのブロック 40 を読み，ファイルシステムが同じディスク操作でブロック 41–43 も読めば，続く 3 回の読み出しはヘッドを動かさずにメモリから提供される．

Linux の既定の先読み量は 128 KiB であり（`VM_READAHEAD_PAGES`），ブロックデバイスごとに `/sys/block/*/queue/read_ahead_kb` で設定される（sysfs-block の文書）．

### 11.14.4 ジャーナリング

▷ journaling

▷ journal

すべての更新を直ちに本来の位置へ書くとランダムアクセスが生じ，更新をバッファキャッシュだけに留めると失う危険がある．**ジャーナリング**，すなわちロギングの技法は，この両方の問題を避ける．ファイルシステムは各更新を**ジャーナル**と呼ばれるログに記録し，ディスクの専用領域へ逐次的に書き，記録した更新を定期的に本来のディスク上の位置へ反映する（図 11.11）．ジャーナルはディスク上にあるので，そこに記録された更新はク

ラッシュを生き延び，後から反映できる．ext3 と ext4 のファイルシステムは，ext2 の設計にこのようなジャーナルを加えたものである．そのジャーナリング層（ext3 では JBD，ext4 では JBD2）は，変更の記述ではなく，変更された各ブロックの新しい内容全体をログに記録する．既定のモード `data=ordered` では，メタデータブロックだけをログに記録し，それをコミットするのはファイルデータが最終的な位置へ書き込まれた後である．モード `data=journal` では，データブロックもログに記録する．

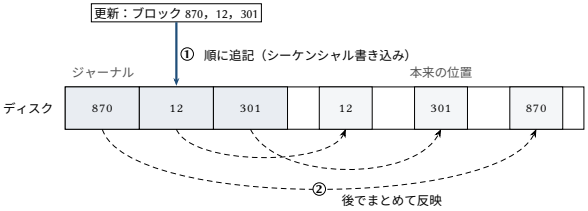



**図 11.11:** ジャーナリング．散らばったブロックへの更新は，まず順にジャーナルへ追記され，後から本来の位置へ反映される．

**注意** ジャーナリングは書き込みの総量を減らすわけではない．記録したブロックは 2 回書かれる．散在した書き込みを，1 回の逐次書き込みと後でまとめて行う更新に置き換えるのである．

### 要点 — 章のまとめ

I/O デバイスはコマンド・データ・ステータスの各レジスタを見せ，PCIe などのインターコネクトを通じて到達され，オペレーティングシステムが標準化したインタフェースの背後でデバイスドライバによって操作される．デバイスはブロックデバイス，キャラクタデバイス，ネットワークデバイスのいずれかであり，デバイスファイルとして表現される．CPU はメモリマップド I/O か I/O ポートでレジスタをアドレス指定し，割り込みかポーリングでデバイスのイベントを知り，プログラム制御 I/O か DMA でデータを転送する．デバイスアクセスは通常カーネルを通るが，OS バイパスはデータ経路からカーネルを取り除ける．要求したスレッドはブロックするか，非同期に実行を続ける．ファイルへのアクセスは，ファイルシステム，汎用ブロック層，ドライバを通じてブロックデバイスに達する．仮想ファイルシステムは，ファイルディスクリプタ，dentry，i ノード，スーパーブロックによってファイルシステムを統一する．ext2 はパーティションをブロックグループに編成し，その i ノードは直接ポインタと間接ポインタを使う．キャッシング，I/O スケジューリング，プリフェッチ，ジャーナリングがディスクアクセスのコストを減らす．

第 12 章

# 仮想化

| | |
|---|---|
| **講義動画** | P3L6 |
| **教科書** | OSTEP [1] app. B；Silberschatz ら [2] ch. 18 |
| **キーワード** | 仮想化，仮想マシン，ハイパーバイザ，ベアメタル型ハイパーバイザ，ホスト型ハイパーバイザ，保護リング，トラップアンドエミュレート，バイナリ変換，準仮想化，ハイパーコール，シャドウページテーブル，分割デバイスドライバモデル，ハードウェア支援仮想化，拡張ページテーブル |

これまでの章では，オペレーティングシステムが，その上で動作するプロセスに代わってマシンの CPU，メモリ，デバイスを管理する様子を見てきた．仮想化はこの役割を 1 段階下へ移す．薄いソフトウェアの層が，複数の完全なオペレーティングシステムに代わってハードウェアを管理し，その各々がマシンを所有しているかのように振る舞う．この章では，仮想化を定義し，それが広く使われるようになった理由を説明し，仮想化ソフトウェアの 2 つの構成方式を比較する．続いて，CPU，メモリ，デバイスがどのように仮想化されるかを，まずソフトウェアだけの技法から，次に現在のプロセッサが提供する支援へと順に調べる．第 1 章の保護境界，第 8 章のページテーブル，第 11 章のデバイスドライバが，オペレーティングシステムの下にもう 1 層のソフトウェアを伴って，繰り返し現れる．

## 12.1 仮想化とは何か

**出典** P3L6，仮想化とは何か，仮想化の定義；Popek と Goldberg [32]；OSTEP app. B；Silberschatz ら ch. 18.

仮想化は古い考えである．1960 年代の IBM に始まる．当時，計算は少数の大型で高価なメインフレーム上で行われており，そうしたマシンの所有者は，異なるオペレーティングシステム向けに開発されたワークロードにそれを使いたいと考えた．

▷ virtualization

**定義 12.1** (仮想化). 複数のオペレーティングシステムが，それぞれのアプリケーションとともに，同じ物理マシン上で並行に実行できるようにすることを**仮想化**と呼ぶ．

これらのオペレーティングシステムは，もはやどれもハードウェアを制御していない．各々は自分専用の環境で動作し，ハードウェア資源–CPU，メモリ，デバイス–を所有しているという幻想を与えられる．ただし，そこで見える資源は物理資源の一部を代理する仮想資源である（図 12.1）．

▷ virtual machine

**定義 12.2** (仮想マシン). **仮想マシン**（VM）とは，オペレーティングシステムと，そのアプリケーション，およびそれらが動作する仮想資源の総体である．

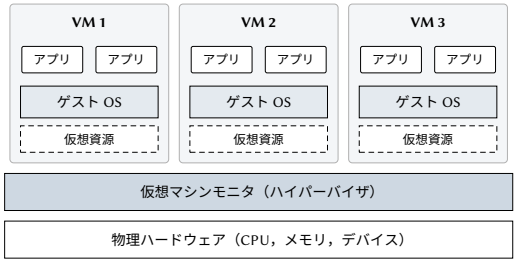



**図 12.1:** 仮想化．各 VM は，ゲスト OS，そのアプリケーション，およびそれに提示される仮想資源からなる．仮想マシンモニタは，すべての VM の下で物理ハードウェアを管理する．

▷ virtual machine monitor

**定義 12.3** (仮想マシンモニタ). **仮想マシンモニタ**（VMM）は，**ハイパーバイザ**とも呼ばれ，VM の下にある仮想化の層である．物理ハードウェアを管理し，VM に仮想資源を提供し，物理資源へのアクセスを調停し，VM どうしを隔離する．

▷ guest operating system

VM の中で動作するオペレーティングシステムを**ゲスト OS** と呼び，VM 全体はゲストドメインとも呼ばれる．

Popek と Goldberg [32] は古典的な定義を与えた．仮想マシンとは「実マシンの効率的で隔離された複製」である．それを支える VMM は 3 つの性質を備えなければならない．

Popek と Goldberg はこの 3 つの性質を等価性，効率性，資源制御と呼ぶ．

**忠実性** プログラムは，オペレーティングシステムも含めて，元のマシンと本質的に同一の環境で動作し，タイミングと利用可能な資源量を除けば，元のマシン上でと同じように振る舞う．この**忠実性**があるからこそ，改変されていないオペレーティングシステムが VM の中で動作できる．準仮想化（12.8 節）はこれを意図的に緩める．

▹ fidelity

**性能** VM 内のプログラムの速度低下は，最悪でもわずかである．したがってほとんどの命令は，VMM が解釈するのではなく，ハードウェア上で直接実行されなければならない．

**安全性と隔離** VMM がシステム資源を完全に制御する．VM は，VMM が割り当てていない資源にアクセスできず，他の VM や VMM 自体に干渉することもできない．

**注 12.4.** この定義は，同じく仮想マシンと呼ばれる技術を除外する．高水準言語仮想マシンは，実マシン向けではなく抽象的な命令セット向けにコンパイルされたプログラムを実行するので，その下のハードウェアの複製ではない．異なるハードウェアプラットフォームをソフトウェアで実装するエミュレータは，実マシンではなくそのプラットフォームを複製しており，しかもすべての命令を解釈または変換によってソフトウェアで実行し，どれ 1 つとしてプロセッサ上で直接実行しないので，性能の要件を満たさない．

## 12.2 仮想化の利点

1 台の物理マシン上で複数のオペレーティングシステムを動かすことには，いくつもの利点がある．

**出典** P3L6，仮想化の利点；Rosenblum と Garfinkel [33]；Silberschatz ら ch. 18．

**サーバ統合** それぞれ独自のオペレーティングシステムとアプリケーションを持つ複数の物理マシンのワークロードを，より少ない台数のマシン上で VM として動作させられる．この**サーバ統合**はコストを下げ–購入，設置，給電，冷却すべきマシンが減る–，管理すべきマシンが減るので管理性も高まる．

▹ consolidation

**マイグレーション** VM は物理マシンから切り離されているので，オペレーティングシステムとアプリケーションを含むその状態全体を，例えば保守のためにマシンを停止する前に，あるマシンから別のマシンへ移せる．マイグレーションは可用性と信頼性を高める．

**セキュリティ** バグや悪意ある振る舞いは，それが起こった VM の中に封じ込められ，他の VM に影響しない．

**デバッグ** 新しいオペレーティングシステムやその新しい機能を，専用のマシンなしに VM 上で素早く立ち上げて試験できる．クラッシュしても落ちるのはその VM だけである．

**レガシー OS のサポート** 現在のハードウェア上ではもはや動作しないオペレーティングシステムを，それに依存するアプリケーションとともに，現行のシステムの隣で VM の中で動かし続けられる．

IBM のメインフレームは 1960 年代から仮想化をサポートしていたが，1990 年代後半までそれはメインフレームに限られていた [33].[1] メインフレームは高価で数も少なかった．コモディティハードウェアが安価になり，そのオペレーティングシステムがマルチタスクをサポートするようになると，1 台のマシンを複数のオペレーティングシステムで分け合うよりも，別のアプリケーションのために別のマシンを買う方が簡単であり，VMM への関心は薄れた．この慣行が，やがて VMM を復活させる問題を生んだ．データセンタは，それぞれ 1 つのアプリケーションだけを動かす低稼働率のマシンで埋まり，それらが消費する場所，電力，管理の手間が，果たす仕事に見合わないほど大きくなった．アプリケーションをより少ないマシンに統合すればこれらのコストは一挙に減るが，そのためにはコモディティな x86 ハードウェアを仮想化しなければならず，12.6 節で見るように，x86 はそのために設計されてはいなかった．

[1] 最初のそのようなシステムは CP-40 と CMS であり，IBM ケンブリッジ科学センターが 1967 年 1 月から運用した．続く CP-67 は，ハードウェアがアドレスを変換する System/360 モデル 67 の上で動いた．この系譜は 1972 年に発表された VM/370 へ続き，現在の z/VM に至る．

**例 12.5.** ある企業が，それぞれ 1 つのアプリケーション専用の 24 台のサーバを運用しており，CPU 使用率は平均 8 % である．合計の負荷は $24 \times 0.08 = 1.92$ 台分の全力稼働に相当する．負荷のピークと仮想化のオーバーヘッドに余裕を残して各物理マシンの負荷を高々 70 % に抑え，メモリと I/O の容量が足りるなら，24 のワークロードは $\lceil 1.92/0.7 \rceil = 3$ 台のマシンに VM として収まり，各マシンの使用率は約 64 % となる．給電，冷却，管理すべきマシンの台数は 8 分の 1 に減る．

低い稼働率は例外ではない．Barroso と Hölzle は，データセンタのサーバが時間の大半を CPU 使用率 10 % から 50 % の間で過ごすと報告している [34]．

## 12.3 仮想化のモデル

仮想化ソフトウェアの構成には 2 つの方式があり，何がハードウェア上で直接動作するか，とりわけ誰がデバイスドライバを所有するかが異なる．

出典 P3L6，仮想化のモデル: ベアメタル型，ホスト型；Barham ら [35]；Kivity ら [36]；Silberschatz ら ch. 18.

### 12.3.1 ベアメタル型ハイパーバイザ

▷ bare-metal hypervisor

**定義 12.6** (ベアメタル型ハイパーバイザ). **ベアメタル型ハイパーバイザ**は，ハイパーバイザ型 VMM あるいは**タイプ** *1* VMM とも呼ばれ，ハードウェア上で直接動作する．すべてのハードウェア資源を管理し，VM の実行を支える．

▷ service VM

このようなハイパーバイザはデバイスも制御しなければならない．デバイスは多種多様であり，そのすべてをサポートすれば大量のデバイスドライバがハイパーバイザに入り込む．そこでベアメタル型の設計では，ゲスト VM の隣に特権的な**サービス VM** を動かすことが多い（図 12.2a）．サービス VM は，デバイスドライバを備えた標準的なオペレーティングシステムを動かし，デバイスに直接アクセスでき，構成と管理の作業を担う．こうしてハイパーバイザ自体は小さく保たれる．

**注意** タイプ 1 とタイプ 2 という呼び分けは，きれいに割り切れるものではない．KVM は Linux のモジュールなので，そのハイパーバイザはハードウェア上で動作している．それでもタイプ 2 に数えられるのは，その周りに完全なホスト OS が残っているからである．

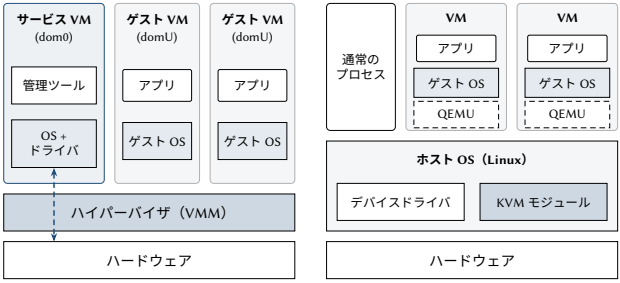



**図 12.2:** 仮想化のモデル．(a) デバイスドライバを所有する特権的なサービス VM を伴うベアメタル型ハイパーバイザ．Xen がこの形である．(b) ホスト型ハイパーバイザ: Linux における KVM のようにホスト OS のモジュールであり，各 VM はホストのプロセスで，その中で QEMU がデバイスをエミュレートする．

▷ Xen

▷ dom0

ハイパーバイザ **Xen** [35] は，オープンソースとしても商用の XenServer（2019 年から 2023 年までの名称は Citrix Hypervisor）としても入手でき，この設計に従う．その VM は**ドメイン**と呼ばれる．特権的なサービス VM は **dom0** であり，デバイスドライバと管理ツールを備えた Linux を動かす．ゲスト VM は非特権のドメイン *domU* であり，Xen ハイパーバイザ自体は，ディスクやネットワークカードといった I/O デバイスのドライバを一切含まない．VMware ESX（現在の ESXi）はもう一方の道を採り，ドライバをハイパーバイザに置く．ESX が対象とするサーバのベンダは限られており，各ベンダが ESX 向けのドライバを供給するので，これが実用的である．ESX は遠隔管理 API を通じて管理され，この API が以前の Linux ベースの制御用中核を置き換えた．

Barham らが記述した最初の Xen [35] では，これらのドライバはまだハイパーバイザの中にあった．

### 12.3.2 ホスト型ハイパーバイザ

▷ hosted hypervisor

▷ host operating system

**定義 12.7** (ホスト型ハイパーバイザ). **ホスト型ハイパーバイザ**，すなわち**タイプ** *2* VMM は，従来型のオペレーティングシステムの上で動作する．このオペレーティングシステムを**ホスト OS** と呼び，ホスト OS がハードウェアのすべてを所有する．ホスト OS 内の特別な VMM モジュールが，VM にハードウェアインタフェースを提供し，VM 間の切り替えを扱う．

ホスト型ハイパーバイザは，ホスト OS がすでに提供しているものを再利用する．デバイスドライバ，スケジューリング，メモリ管理などのサービスを実装し直す必要はなく，通常のアプリケーションも VM の隣でホスト上でそのまま動き続ける（図 12.2b）．

**KVM** [36] は，Linux のホスト型ハイパーバイザである．ロード可能なカーネルモジュールが Linux をハイパーバイザに変える．各 VM は通常の Linux プロセスであり，その仮想 CPU はそのプロセスのスレッドである．したがって Linux のスケジューラとメモリマネージャは，他のプロセスと同じように VM を扱う．VM のデバイスは，ユーザレベルで動作するハードウェアエミュレータ **QEMU** によってそのプロセスの中でエミュレートされ，一方ゲストの命令は 12.11 節で述べるハードウェア支援によって CPU 上で直接実行される．KVM は Linux の一部であるため，Linux のオープンソースコミュニティの恩恵を受ける．新しい機能，バグ修正，新しいデバイスのドライバが，別途の労力なしに利用できるようになる．

▷ Kernel-based Virtual Machine

▷ QEMU

**ヒント** Linux のホストでは，ps は各 KVM ゲストを 1 つの qemu-system-* プロセスとして表示し，その仮想 CPU はそのプロセスのスレッドである．

> **要点**
> ベアメタル型ハイパーバイザはハードウェア上で直接動作し，デバイスドライバは Xen の dom0 のような特権的なサービス VM に委ねるのが普通である．ホスト型ハイパーバイザは Linux における KVM のようにホスト OS のモジュールであり，その OS のドライバとサービスを再利用する．

## 12.4 ハードウェアの保護レベル

ゲスト OS は，カーネルモードで動作し特権命令を実行するように書かれているが，仮想化の下ではハイパーバイザだけがハードウェアを制御してよい．第 1 章の 2 つのモードでは，アプリケーション，ゲスト OS，ハイパーバイザという 3 種類のソフトウェアを分離できないが，コモディティプロセッサは 2 つより多くの特権レベルを備えている．

**出典** P3L6，ハードウェアの保護レベル；Silberschatz ら ch. 18.

**定義 12.8** (保護リング). **保護リング**とは，プロセッサが強制する特権レベルの階層の 1 つであり，あるリングのソフトウェアは，そのレベルで許された命令だけを実行し，許された資源だけにアクセスできる．x86 プロセッサは 4 つのリングを備え，最も特権的なリング 0 から最も特権の低いリング 3 までである．

▷ protection ring

仮想化を用いない場合，OS カーネルはリング 0，アプリケーションはリング 3 で動作し，リング 1 と 2 は使われない（図 12.3a）．仮想化を用いる場合，ハイパーバイザがリング 0 を占め，ゲスト OS はリング 1 へ移り，アプリケーションはリング 3 のままである（図 12.3b）．ゲスト OS がリング 1 から特権的な操作を試みると，プロセッサはリング 0 のハイパーバイザへトラップする．

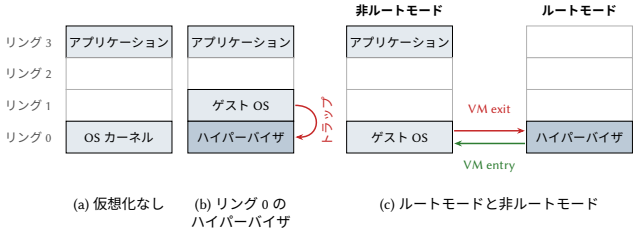



**図 12.3:** x86 の保護リングの使い方．(a) 仮想化なしの実行．(b) ハードウェア支援のない仮想化: ゲスト OS はリング 1 へ降格され，その特権的な操作はハイパーバイザへトラップする．(c) 仮想化支援を備えたプロセッサ: ゲスト OS は非ルートモードのリング 0 で動作し，VM exit が制御をルートモードのハイパーバイザへ移す．

仮想化支援を備えたプロセッサは，現在の x86-64 プロセッサを含め，さらに進んで，それぞれ 4 つのリングを持つ 2 つの動作モードを提供する（図 12.3c）．

**定義 12.9** (ルートモードと非ルートモード). **ルートモード**では，プロセッサはハイパーバイザをリング 0 で，ハードウェアを完全に制御した状態で実行する．**非ルートモード**では VM を実行し，ゲスト OS はリング 0，アプリケーションはリング 3 で動作するが，特権的な操作は制限される．ホスト型ハイパーバイザでは，ホスト OS とそのプロセスもルートモードで動作する．

▷ root mode

▷ non-root mode

こうしてゲスト OS は，それが書かれたときに想定された特権レベルで動作する．ハイパーバイザが横取りすると決めた操作は，VM ごとに制御構造の中で選ばれ（12.11 節），**VM exit** と呼ばれるトラップを起こす．VM exit によってプロセッサはルートモードへ切

▷ VM exit

▷VM entry

り替わり，制御がハイパーバイザに渡る．ハイパーバイザがその操作を処理し終えると，**VM entry** がプロセッサを非ルートモードへ戻し，VM を再開する．

## 12.5 プロセッサの仮想化

**出典** P3L6，プロセッサの仮想化；Popek と Goldberg [32]；OSTEP app. B．

12.1 節の性能の要件を満たすため，ハイパーバイザは VM の命令を解釈するのではなく，ハードウェア上で直接実行させる．問題は，そのような命令が特権命令であるときに何が起こらなければならないかである．

▷trap-and-emulate

**定義 12.10** (トラップアンドエミュレート)．**トラップアンドエミュレート**による仮想化では，ゲストの命令は，ゲスト OS を降格させたうえで CPU 上で直接実行される．非特権命令はハードウェアの速度で動作する．特権命令はハイパーバイザへトラップし，ハイパーバイザが何をすべきかを判断する．その操作が VM にとって不正であれば VM を終了させ，正当であればゲスト OS がハードウェアに期待する振る舞いをエミュレートしてゲストを再開する．

図 12.4 はこれらの判断をまとめたものである．エミュレーションは，物理ハードウェアではなく VM の仮想状態に対して働く．例えばゲスト OS が割り込みを禁止すると，ハイパーバイザはこの VM について割り込みが禁止されていることを記録し，その VM 宛ての割り込みを保留する．一方，物理的な割り込みは有効なままであり，ハイパーバイザと他の VM は自分宛ての割り込みを受け取り続ける．トラップのたびにハイパーバイザへの遷移の代価がかかるので，トラップアンドエミュレートのオーバーヘッドは，ゲストが行う特権的な操作の数とともに増える．

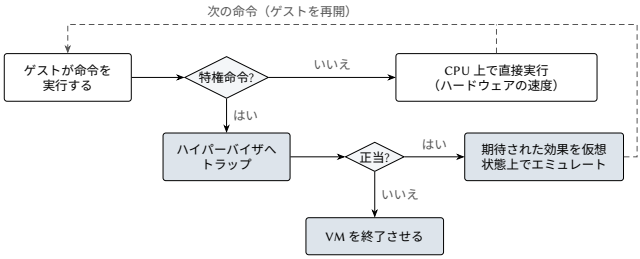



**図 12.4:** トラップアンドエミュレート．ハイパーバイザが関与するのは特権命令だけであり，正当なものは VM の仮想状態上でエミュレートされる．

**注 12.11.** トラップアンドエミュレートが正しく働くのは，ハイパーバイザが効果を制御しなければならない命令がすべて実際にトラップする場合に限られる．Popek と Goldberg [32] はそのような命令を**センシティブ**命令と呼ぶ．マシンの構成を変更あるいは露出させる命令，または特権レベルによって振る舞いが変わる命令である．すべてのセンシティブ命令が特権命令である，すなわち最も特権的なレベルの外で実行されるとトラップするなら，そのアーキテクチャはこの方法で仮想化できる．

## 12.6 ハードウェア支援以前の x86 の仮想化

**出典** P3L6，過去の x86 の仮想化；Robin と Irvine [37]．

2005 年以前の x86 プロセッサは 4 つのリングを備えていたが，ルートモードと非ルートモードは持たなかったので，ハイパーバイザはリング 0 で動作し，ゲスト OS をリング 1 へ降格させた．このアーキテクチャはトラップアンドエミュレートの条件を満たしていなかった．Robin と Irvine [37] は，特権の低いリングで実行してもトラップしないセンシティブ命令を 17 個特定した．リング 1 で実行すると，それらは黙って失敗するか，ハイパーバイザが隠そうとしているマシンの状態を露出させるので，トラップアンドエミュレートだけではプロセッサを正しく仮想化できない．

最もよく知られた例は PUSHF と POPF であり，フラグレジスタをスタックに保存し，また復元する．フラグレジスタには割り込み許可フラグ IF が含まれており，カーネルは，割り込みを禁止して実行した区間の終わりで，保存したフラグを POPF で復元することによって割り込みを再び許可するのが普通である．リング 0 では POPF は IF を復元する．リング 1 では，プロセッサは復元する値の IF ビットを，トラップすることなく無視する．ハイパーバイザは呼び出されないので，ゲストが割り込みの状態を変えようとしたこと

ハイパーバイザは I/O 特権レベルを 0 に設定するので，リング 1 の POPF は IF を変更しないままにし，一方 CLI はそこでトラップする．

を知ることができず，その変更をエミュレートできない．ゲスト OS はエラーを受け取らないので，変更は成功したものと思い込む．表 12.1 はその帰結を追ったものである．

**表 12.1:** リング 1 のゲストカーネルが，フラグを保存し，割り込みを禁止し，後でフラグを復元する．CLI はトラップしてエミュレートされるが，POPF はトラップしない．仮想 *IF* は，ハイパーバイザがこの VM について記録している割り込みフラグであり，物理的な割り込みは終始有効なままである．

| ステップ | ゲストカーネルの実行 | トラップ | 仮想 IF | ゲストの想定 |
|---|---|---|---|---|
| 1 | PUSHF（IF = 1 を保存） | しない | 1 | 許可 |
| 2 | CLI | する | 0 | 禁止 |
| 3 | クリティカルセクション | しない | 0 | 禁止 |
| 4 | POPF（IF = 1 を復元） | しない | 0 | 許可 |
| 5 | デバイス割り込みを待つ | しない | 0 | 許可 |

ステップ 4 以降，ハイパーバイザはこの VM について割り込みが禁止されていると考えてその割り込みを保留し，一方ゲスト OS は，許可されていると信じている割り込みを待ち続ける．ゲストはハングする．これに対する対策がソフトウェアで 2 つ開発された．1 つはゲスト OS を改変しないもの，もう 1 つは改変するものである．

**注意** 32 ビットのソフトウェア VMM は，自分のメモリをセグメント限界で守る．AMD64 のロングモードはこの検査をやめ，ページングはリング 1 とリング 0 を区別しないので，64 ビットの準仮想化 Xen ゲストはカーネルをリング 3 で動かし，VMware のバイナリ変換には，後の AMD プロセッサが復活させたセグメント限界か VT-x のどちらかが必要だった．

## 12.7 バイナリ変換

第 1 の対策は，ゲストのバイナリコードを書き換えて，問題のある命令を決して実行しないようにする．この手法はスタンフォード大学の Mendel Rosenblum のグループが先鞭をつけ，VMware が製品化した [38]．その目標は，ゲスト OS をまったく改変せずに動かすことである．

**出典** P3L6，バイナリ変換；Bugnion ら [38]；Silberschatz ら ch. 18．

**定義 12.12** (完全仮想化). ゲストオペレーティングシステムが，物理マシン上で動作するのと同じように，改変されずに動作する仮想化を**完全仮想化**と呼ぶ．

▷ full virtualization

**定義 12.13** (バイナリ変換). 動的**バイナリ変換**では，ハイパーバイザが実行時に，実行の直前にゲストのコードを書き換える．問題のある命令は，意図した効果を持つ別の命令列に変換され，それ以外の命令はそのまま残されてハードウェアの速度で動作する．

▷ binary translation

変換はブロック単位で進む（図 12.5）．

1. これから実行されるコードのブロック，例えば次の分岐までの命令列を検査する．
2. ブロックに問題のある命令が含まれていれば，望ましい振る舞いをエミュレートする命令列，例えばハイパーバイザのエミュレーションルーチンの呼び出しに変換する．元の命令がトラップしたはずの箇所でも，変換によってトラップを避けられることさえある．
3. そうでなければ，制御の移動だけを調整してブロックをそのまま引き取る．こうしてブロックはハードウェアの速度で動作しつつ，実行は変換器の制御下に留まる．

VMware の設計では，変換されるのはゲストのカーネルコードだけであり，リング 3 のアプリケーションは直接動作する [38]．

得られたブロックは**変換キャッシュ**に保持されるので，ブロックを変換する代価は一度だけ支払われ，その後の実行にわたって償却される．[2] どのコードが実行されるか，間接分岐がどこへ向かうかは実行時にしか分からないので，変換は動的でなければならない．

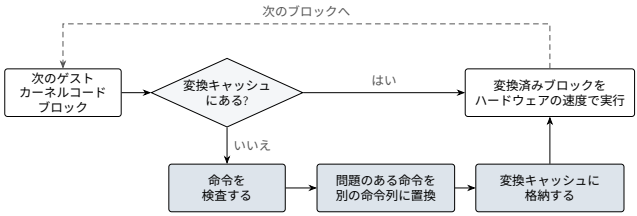



**図 12.5:** 変換キャッシュを伴う動的バイナリ変換．ブロックが検査され変換されるのは，最初に実行されようとするときだけである．

[2] バイナリ変換は，ハードウェア支援が現れても時代遅れにはならなかった．最初の VT-x プロセッサでは，VM exit が高価で，どちらにせよページテーブルをシャドウ化しなければならず，ほとんどのワークロードではソフトウェアの VMM の方が速かった [39]．

**例 12.14.** ゲストのカーネルコードのあるブロックは，ネイティブには 5 µs で実行される．その変換には 2 ms かかり，1 命令がハイパーバイザへの呼び出しに置き換えられるため，変換後のブロックの実行には 6 µs かかる．このブロックが 20 000 回実行されるなら，一度だけ変換して結果をキャッシュする代価は $2\,\text{ms} + 20\,000 \times 6\,\mu\text{s} = 122\,\text{ms}$ であり，ネイティブの 100 ms に対してオーバーヘッドは 22 % である．実行のたびに変換し直せば $20\,000 \times (2\,\text{ms} + 6\,\mu\text{s}) \approx 40.1\,\text{s}$ となり，ネイティブの約 400 倍の時間になる．

## 12.8 準仮想化

出典 P3L6，準仮想化；Barham ら [35].

第 2 の対策は，性能と引き換えに，改変しないゲストを諦める．ゲスト OS は，オペレーティングシステムを新しいハードウェアプラットフォームへ移植するのとほぼ同じように，ハイパーバイザへ移植される．

▷ paravirtualization

**定義 12.15** (準仮想化). **準仮想化**では，ゲストオペレーティングシステムが，自分は VM の中で動作していると知るように改変される．特権的な操作を実行してトラップに頼るのではなく，それをハイパーバイザに明示的に要求する．

▷ hypercall

ゲスト OS からハイパーバイザへの明示的な要求が**ハイパーコール**である．これは，アプリケーションにとってのシステムコール（第 1 章）と同じ役割を，ゲスト OS に対して果たす．ゲストは文脈情報–引数–を定められた場所に置き，どのハイパーコールを要求するかを指定してハイパーバイザへトラップする．ハイパーバイザはその操作を実行し，制御をゲストへ返す．したがってハイパーバイザは，トラップした個々の命令から意図された操作を推測する必要がなく，12.9 節で見るように，1 回のハイパーコールで複数の操作を運べる．改変されるのはゲスト OS だけであり，ゲスト OS がアプリケーションに提供するインタフェースは変わらないので，アプリケーションはそのまま動作する [35].

Xen はケンブリッジ大学で開発され，後に XenSource と Citrix によって製品化された，最もよく知られた準仮想化ハイパーバイザである．12.11 節の表 12.4 は，準仮想化とプロセッサ仮想化の他の手法を比較したものである．

> **要点**
> トラップアンドエミュレートは，すべてのセンシティブ命令がトラップすることを必要とするが，2005 年以前の x86 プロセッサはそれを保証しなかった．バイナリ変換は問題のある命令を実行時に書き換え，ゲスト OS を改変しないまま保つ．準仮想化はゲスト OS を改変してハイパーコールを発行させ，性能を得る．

## 12.9 メモリの仮想化

### 12.9.1 完全仮想化

出典 P3L6，メモリの仮想化: 完全仮想化，準仮想化；OSTEP app. B；Barham ら [35].

ゲスト OS は，オペレーティングシステムが物理マシン上で見出すもの，すなわちアドレス 0 から始まる連続した物理メモリを期待し，第 8 章で述べたページテーブルでそれを管理する．ハイパーバイザは，マシンのメモリを VM の間で分割しながら，すべての VM にこの幻想を与えなければならず，しかもアドレス変換はハードウェアの MMU と TLB に行わせたい．そこで 3 種類のアドレスを区別する．

**仮想アドレス** 第 2 章と同じく，VM 内のソフトウェアが使うアドレスである．
**物理アドレス** ゲスト OS が物理メモリのアドレスだと信じているアドレスである．
**マシンアドレス** マシンのメモリの実際のアドレスである．完全仮想化では，**マシンアドレス**はハイパーバイザだけが知っている．

▷ machine address

ページ番号についても同じ区別があり，仮想ページ番号，物理フレーム番号，マシンフレーム番号となる．ゲスト OS は仮想アドレスを物理アドレスへ対応付けるページテーブルを維持し，ハイパーバイザは物理アドレスからマシンアドレスへの対応を維持する（図 12.6）．

ヒント 仮想 → 物理はゲストのもの，物理 → マシンはハイパーバイザのものであり，シャドウページテーブルはその合成である．

第 1 の選択肢は，すべてのアドレスをこの 2 段階で変換することである．しかし MMU が辿るページテーブルは 1 つだけであり，TLB はアドレスごとに 1 つの変換しかキャッシュしない．メモリ参照のたびに第 2 の変換をソフトウェアで行うのは，あまりにも高価である．

▷ shadow page table

**定義 12.16** (シャドウページテーブル). **シャドウページテーブル**とは，ハイパーバイザが維持するページテーブルで，ゲストプロセスの仮想アドレスをマシンアドレスへ直接対応付ける．ハイパーバイザはこれをゲストのページテーブルの代わりに MMU に設定するので，ハードウェアは各アドレスを 1 段階で変換し，TLB がその結果をキャッシュする．

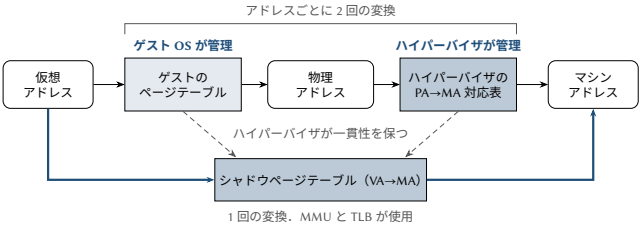



**図 12.6:** メモリの仮想化．ゲストのページテーブルを通り，次にハイパーバイザの対応表を通る変換は 2 段階を要する．シャドウページテーブルは両者を，MMU が直接使う 1 つの対応にまとめる．拡張ページテーブルを備えたハードウェアは，この 2 段階の変換を自ら行う（12.11 節）．

ゲスト OS は自分のページテーブルを作り，変更し続けるが，ハードウェアはもはやそれを使わない．そのためハイパーバイザは，シャドウページテーブルをゲストのページテーブルと整合させ続けなければならない．

- ゲストのページテーブルを保持するメモリを書き込み禁止にする．ゲスト OS が対応付けを追加・変更すると書き込みがトラップし，ハイパーバイザがその書き込みを行ったうえで，対応するマシンアドレスでシャドウページテーブルを更新する．
- ゲスト OS がプロセスを切り替えるときは，別のページテーブルをページテーブルベースレジスタに読み込む．これは特権的な操作なのでトラップする．そこでハイパーバイザは，新しいプロセスのシャドウページテーブルを設定するか，現在のシャドウエントリを無効化して，使われるにつれて作り直す．

これらのトラップによって，ゲスト内でのページテーブルの更新とコンテキストスイッチは，物理マシン上よりはるかに高価になる．

書き込みを知るためにページを書き込み禁止にする仕掛けは，第 8 章のコピーオンライトと同じである．ここではページを複製するためではなく，ゲストを観察するために使う．

第 8 章のページテーブルベースレジスタは x86 では CR3 であり，シャドウページングの下では，ゲストによる CR3 への書き込みはすべてトラップする．

**例 12.17.** あるゲストプロセスが仮想ページ 2 と 3 を使っている．ゲストのページテーブルはそれらを物理フレーム 5 と 9 に対応付けており，ハイパーバイザはそれらをマシンフレーム 41 と 17 で裏付けているので，シャドウページテーブルはページ 2 をマシンフレーム 41 に，ページ 3 をマシンフレーム 17 に対応付ける．ここでゲスト OS が仮想ページ 4 を物理フレーム 12 に対応付ける．保護されたページテーブルへの書き込みはトラップし，ハイパーバイザはフレーム 12 がマシンフレーム 26 で裏付けられていることを見つけ，書き込みを行い，シャドウページテーブルにエントリ 4 → 26 を追加してゲストを再開する．表 12.2 に，その結果の対応を示す．

**表 12.2:** 例のゲストプロセスの対応．最後のシャドウエントリは，ゲストのページテーブル更新がトラップしたときにハイパーバイザが追加したものである．

| 仮想ページ | ゲストのページテーブル: 物理フレーム | ハイパーバイザ: マシンフレーム | シャドウページテーブル: マシンフレーム |
|---|---|---|---|
| 2 | 5 | 41 | 41 |
| 3 | 9 | 17 | 17 |
| 4 | 12 | 26 | 26 |

## 12.9.2 準仮想化

準仮想化ではゲスト OS は自分が VM の中で動作していることを知っており，メモリの仮想化はゲストとハイパーバイザの協調になる．物理メモリが連続していてアドレス 0 から始まるという厳しい要求は消える．Xen では，ゲスト OS は自分に与えられたマシンフレームを知っており，そのページテーブルは仮想アドレスをマシンアドレスへ直接対応付ける [35]．ゲストは自分のページテーブルをハイパーバイザに明示的に登録し，それ以降そのページテーブルに対しては読み出しの権限しか持たない．ページテーブルを変更するにはハイパーコールを発行し，ハイパーバイザはその変更を—例えば新しい対応がこの VM に属するマシンフレームを指していることを—検証してから適用する．ハイパーコールのたびにハイパーバイザへの遷移の代価がかかるので，ゲストはページテーブルの更新を**まとめ**，多数の更新が 1 回のハイパーコールを共有するようにする．他の最適化も，遷移を減らすという同じ考えに従う．例えば Xen は，すべてのゲストのアドレス

**注意** 呼び方はシステムごとに異なる．この章と Xen が物理アドレス，マシンアドレスと呼ぶものを，Intel はゲスト物理アドレス，ホスト物理アドレスと呼び，VMware も同様に呼ぶ．

空間の上位 64 MB を占めるので，ハイパーバイザに入るのにページテーブルの切り替えも TLB のフラッシュも必要ない [35].

**例 12.18.** Xen 上の準仮想化された Linux ゲストが，1 回のハイパーコールで 3 つのページテーブルエントリを変更する．各要求は，ページテーブルエントリのマシンアドレスと，そのエントリの新しい内容を与える:

```
struct mmu_update req[3];
int done;

for (int i = 0; i < 3; i++) {
  /* 変更するエントリのマシンアドレス */
  req[i].ptr = pte_maddr[i] | MMU_NORMAL_PT_UPDATE;
  /* 新しいエントリ: マシンフレーム番号とフラグ */
  req[i].val = (mfn[i] << PAGE_SHIFT) | flags;
}
/* 3 つの更新すべてに対して Xen への遷移は 1 回 */
HYPERVISOR_mmu_update(req, 3, &done, DOMID_SELF);
```

Xen は各要求を適用する前に検査し，いくつの更新が成功したかを done に報告する．1 つずつ発行すれば，同じ更新のためにハイパーバイザへ 3 回入ることになる．

より新しいプラットフォームでは，12.11 節のハードウェア支援が，メモリの仮想化のオーバーヘッドを取り除くか大幅に減らす．

## 12.10 デバイスの仮想化

**出典** P3L6，デバイスの仮想化，パススルー，ハイパーバイザ直接，分割デバイスドライバの各モデル；Silberschatz ら ch. 18.

CPU とメモリの多様性は小さい．命令セットアーキテクチャがそのインタフェースを標準化しているので，ある系列のすべてのプロセッサにわずかな技法で足りる．デバイスは非常に多様であり，そのインタフェースと振る舞いの標準仕様は存在しない．第 11 章で見たように，デバイスの種類ごとに専用のドライバが必要である．デバイスを仮想化するモデルは 3 つ現れた．物理デバイスを制御するドライバがどこに置かれるか，VM のデバイスアクセスがどのようにそこへ届くかが異なる（図 12.7）.

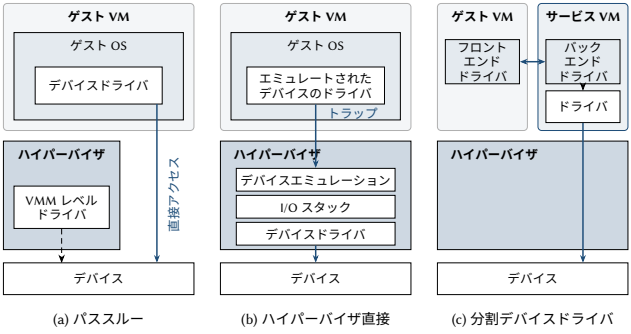



**図 12.7:** デバイスの仮想化のモデル．(a) VM 自身のドライバがデバイスに直接アクセスする．(b) ハイパーバイザがデバイスアクセスを横取りし，自分の I/O スタックとドライバでそれをエミュレートする．(c) ゲスト内のフロントエンドドライバが，サービス VM 内のバックエンドドライバに要求を渡し，バックエンドドライバがデバイスを駆動する．

### 12.10.1 パススルーモデル

▷ passthrough model

**定義 12.19** (パススルーモデル). **パススルーモデル**では，VMM レベルのドライバがデバイスのアクセス権を設定し，VM が自分のドライバで，ハイパーバイザを迂回してデバイスに直接アクセスする．

SR-IOV（PCI-SIG，2007 年 [40]）はこれを和らげる．1 枚のカードが 1 つの物理ファンクションと複数の仮想ファンクション—仕様上は 65 535 個まで，実際には数百個まで（Intel E810 では 256 個）—を提示し，その各々を VM に割り当てられる．

VM はデバイスへの排他的なアクセスを得て，そのデバイス操作は物理マシン上と同じ経路をたどる．この *VMM バイパス*は，第 11 章の OS バイパスに対応するもので，最良の性能を与える．このモデルには 3 つの欠点がある．

- デバイスの共有が難しい．ある VM から別の VM へ割り当て直さなければならない．

- VM 内のドライバがハードウェアを直接操作するので，マシンはそのドライバが期待する種類のデバイスをちょうど備えていなければならない．
- VM の状態の一部がデバイスの中にあり，移送先のマシンがその状態を移せる同一のデバイスを備えていなければならないので，マイグレーションが厄介である．

第 11 章の OS バイパスと同じく，直接割り当てられたネットワークカードはラインレートで動作するが，それを使う VM はライブマイグレーションが難しい．

### 12.10.2 ハイパーバイザ直接モデル

▷ hypervisor-direct model

**定義 12.20** (ハイパーバイザ直接モデル). **ハイパーバイザ直接モデル**では，ハイパーバイザが VM のデバイスアクセスをすべて横取りし，デバイス操作をエミュレートする．すなわち，その操作を汎用の I/O 操作に翻訳し，ハイパーバイザ内にある I/O スタックを通し，物理デバイス用のハイパーバイザ自身のドライバを呼び出す．

主な利点は，VM が物理デバイスから切り離されることである．VM はエミュレートされたデバイスを見る．それは物理デバイスとは無関係に，どのマシンでも同じでありうる．デバイスは VM の間で共有でき，マイグレーションは簡単になり，各デバイスの固有事情はハイパーバイザが扱う．VMware ESX はこのモデルを用いる．代価は，横取りとエミュレーションがデバイス操作のたびに加えるレイテンシと，ハイパーバイザ内部のデバイスドライバのエコシステムの複雑さである．ハイパーバイザは，サポートするすべてのデバイスのドライバを含み，しかも堅牢であり続けなければならない．

エミュレートされるデバイスはわざと古く，ありふれたものが選ばれる．QEMU の PC マシンは Intel e1000 のネットワークカードと IDE のディスクコントローラを提示する．どのゲスト OS もそのドライバをすでに持っているからである．

### 12.10.3 分割デバイスドライバモデル

▷ split-device driver model

▷ front-end driver

▷ back-end driver

**定義 12.21** (分割デバイスドライバモデル). **分割デバイスドライバモデル**では，デバイスへのアクセスを，協調する 2 つのドライバが制御する．1 つはゲスト VM 内の**フロントエンドドライバ**で，ゲスト OS にデバイスインタフェースを提示する．もう 1 つはサービス VM またはホスト OS 内の**バックエンドドライバ**で，通常のドライバを通じて物理デバイスにアクセスする．

フロントエンドドライバは，トラップしてエミュレートしなければならないようなデバイスレジスタを操作しない．送信すべきパケットのような完結した要求をバックエンドドライバに渡す．Xen では，2 つの VM の間で共有されるメモリ内の記述子リングを通じて渡す．これによってハイパーバイザ直接モデルのエミュレーションのオーバーヘッドがなくなり，あるデバイスへの要求はすべてバックエンドドライバを通るので，サービス VM はそのデバイスが VM の間でどう共有されるかを管理できる．代価はゲストの改変である．そのドライバを，ハイパーバイザ向けに書かれたフロントエンドドライバに置き換えなければならないので，このモデルは準仮想化されたゲストと，そうしたドライバが導入されたゲストに適用される．表 12.3 に 3 つのモデルをまとめる．

Linux の virtio（Russell，2008 年 [41]．2016 年以降は OASIS の仕様，virtio 1.0）が，共通のフロントエンド・バックエンドのインタフェースである．その virtqueue は第 9 章の共有メモリである．

**表 12.3:** デバイスの仮想化のモデルの比較．

| モデル | 物理デバイスを駆動するもの | ゲスト内のドライバ | 共有とマイグレーション | デバイス操作ごとの代価 |
|---|---|---|---|---|
| パススルー | ゲスト VM | ネイティブドライバ | 難しい | ネイティブ以上の代価なし |
| ハイパーバイザ直接 | ハイパーバイザ | エミュレートされたデバイスのドライバ | 可能 | 横取りとエミュレーション |
| 分割デバイスドライバ | サービス VM またはホスト OS | 改変されたフロントエンド | 可能，バックエンドが管理 | バックエンドへの要求の受け渡し |

**要点**
デバイスには標準のインタフェースがないので，その仮想化は性能と柔軟性を引き換えにする．パススルーは最速だが VM をデバイスに縛りつけ，ハイパーバイザ直接はエミュレーションの代価で VM を切り離し，分割デバイスドライバモデルはゲストのドライバを改変することでエミュレーションを避ける．

## 12.11 仮想化のためのハードウェア支援

2005 年頃，AMD と Intel はプロセッサに仮想化のための支援を加えた．AMD は Pacifica というコード名の技術，現在の AMD-V であり，Intel は Vanderpool，現在の Intel VT である．

**出典** P3L6，ハードウェアによる仮想化，x86 の VT 革命；Uhlig ら [42]；Rosenblum と Garfinkel [33]．

**定義 12.22 (ハードウェア支援仮想化).** プロセッサ自体が，それを仮想化するために必要な機構を提供し，改変されていないゲストオペレーティングシステムが，バイナリ変換も準仮想化もなしにトラップアンドエミュレートで動作するような仮想化を，**ハードウェア支援仮想化**と呼ぶ．

▷ hardware-assisted virtualization

これらの拡張は，これまでの節で述べた問題に対処する．

**x86 ISA の穴をふさぐ** 非ルートモードではゲスト OS はリング 0 で動作し，そこでは POPF のような命令もゲストの期待どおりに振る舞う．また，センシティブ命令，制御レジスタへのアクセス，I/O，割り込みなど，ハイパーバイザが制御しなければならない操作はすべて，VM exit を起こすようにできる．これによってこのアーキテクチャは，トラップアンドエミュレートで仮想化可能になる．

**注意**「ルート／非ルート」は Intel の呼び名であり，ホスト／ゲストモードは AMD の呼び名である．ARM（例外レベル EL2）と RISC-V（H 拡張）も同様の分離を備える．

**ルートモードと非ルートモード** 12.4 節のモードは，AMD ではホストモードとゲストモードと呼ばれ，ハイパーバイザが制御を保ったままゲスト OS をリング 0 で動作させる．

**VM 制御構造** VM ごと，より正確には仮想 CPU ごとに 1 つのデータ構造で，ハードウェアがそれを読み書きする．この **VM 制御構造**（Intel の VMCS，AMD の VMCB）は，仮想 CPU の状態を保持し，VM exit で保存され VM entry で復元される．また，どの命令とイベントが VM exit を起こすかを指定するので，ハイパーバイザは VM ごとに何をトラップするかを決められる．

▷ VM control structure

**拡張ページテーブル** MMU が図 12.6 の 2 段階の変換を自ら行う．ゲストのページテーブルは従来どおり仮想アドレスを物理アドレスへ対応付け，ハイパーバイザが維持する**拡張ページテーブル**（EPT）が物理アドレスをマシンアドレスへ対応付ける．TLB ミスの際，MMU は両方のテーブルを辿り，TLB は結果として得られる仮想アドレスからマシンアドレスへの変換をキャッシュする．ゲスト OS は VM exit なしに自分のページテーブルを変更し切り替えられるようになり，シャドウページテーブルは不要になる．AMD における同等のものは，ネステッドページテーブルと呼ばれる．

▷ extended page table

**タグ付き TLB** **タグ付き TLB** は，各エントリに，アドレス空間（AMD の ASID）または仮想プロセッサ（Intel の VPID）の識別子を付ける．異なる VM とハイパーバイザのエントリが TLB の中に共存できるので，VM exit や VM entry でも VM の切り替えでも，それをフラッシュする必要がない．

▷ tagged TLB

**マルチキューデバイスと割り込みのルーティング** ネットワークカードのようなデバイスは，複数の論理インタフェース，すなわちキューを提供し，その各々を別々の VM に割り当てられる．割り込みは，そのキューを所有する VM が動作しているコアへ配送される．

**セキュリティと管理の支援** さらなる拡張が，ハイパーバイザを保護し，VM の管理を支える．

新しい命令が，非ルートモードへの遷移，VM 制御構造の読み書き，ハイパーコールの発行を行う．KVM のようなホスト型ハイパーバイザは，これらに依存する．

**ヒント** ゲストの $g$ 段の下に拡張の $n$ 段があるとき，1 回の探索には $(g+1)(n+1)-1$ 回の参照が要る．$g=n=4$ なら 24 回であり，ネイティブの 4 回に対する代価である [43]．

Intel では，VM entry に VMLAUNCH と VMRESUME，制御構造に VMREAD と VMWRITE，ハイパーコールに VMCALL を用いる．

VT-x は 2005 年 11 月の Pentium 4 662 と 672 で，AMD-V は 2006 年 5 月に，すでに ASID タグ付きの TLB を備えて出荷された．ネステッドページテーブルは Barcelona（2007 年），拡張ページテーブルと 16 ビットの VPID タグは Nehalem（2008 年）で導入された．

Intel は支援をプロセッサから外側へ向かって段階的に広げた．*VT-x* はプロセッサを対象とし，ルートモードと非ルートモード，VM exit と VM entry，VM 制御構造を，そして後のプロセッサでは拡張ページテーブルとタグ付き TLB エントリを含む．

*VT-d*（directed I/O）はチップセットに I/O メモリ管理ユニットを加え，DMA（第 11 章）で使われるアドレスを変換し割り込みを再対応付けするので，パススルーモデルで VM に割り当てられたデバイスが，他の VM のメモリにアクセスできないようにする．*VT-c*（connectivity）はネットワークデバイスに仮想化支援をもたらす．デバイスごとに複数のキューを備え，自らを複数の仮想デバイスとして提示して，その各々を VM に割り当てられるようにする．

**表 12.4:** x86 におけるプロセッサの仮想化の手法．

| 手法 | ゲスト OS | 問題のある命令 | 主なオーバーヘッド | 例 |
|---|---|---|---|---|
| トラップアンドエミュレートのみ，2005 年以前 | 改変なし | 一部が黙って失敗する: 不正確 | トラップ | – |
| バイナリ変換 | 改変なし | 実行時に書き換え | 変換，エミュレーション | VMware |
| 準仮想化 | 改変あり | ハイパーコールに置換 | ハイパーコール | Xen |
| ハードウェア支援 | 改変なし | VM exit を起こす | VM exit | KVM |

**要点 — 章のまとめ**
仮想化は 1 台のマシン上で複数のオペレーティングシステムを並行に動かす．ハイパーバイザは各 VM に，効率的で隔離されたマシンの複製を与える．ベアメタル型ハイパーバイザはハードウェア上で動作し，通常はドライバを所有するサービス VM を伴う．ホスト型ハイパーバイザはホスト OS を拡張する．CPU はトラップアンドエミュレートで仮想化されるが，2005 年以前の x86 プロセッサはこれを完全にはサポートしていなかったため，改変しないゲストにはバイナリ変換が，改変するゲストにはハイパーコールを用いる準仮想化が使われた．メモリの仮想化は物理アドレスの下にマシンアドレスを加え，シャドウページテーブルか，検証されまとめられたページテーブル更新によって扱われる．デバイスはパススルー，ハイパーバイザ直接，分割デバイスドライバの各モデルで仮想化される．ルートモードと非ルートモード，VM exit，拡張ページテーブル，タグ付き TLB を備えたハードウェア支援仮想化は，これらのオーバーヘッドの大半を取り除く．

第 13 章

# 遠隔手続き呼び出し

| | |
|---|---|
| 講義動画 | P4L1 |
| 教科書 | OSTEP [1] ch. 48；Silberschatz ら [2] ch. 3；Birrell と Nelson [44]；RFC 5531 [45]，RFC 4506 [46]，RFC 1833 [47] |
| キーワード | 遠隔手続き呼び出し，スタブ，RPC ランタイム，インタフェース定義言語，マーシャリング，アンマーシャリング，バインディング，レジストリ，部分障害，Sun RPC，XDR，rpcgen，rpcbind，Java RMI |

これまでの章が扱ってきたのは，1 つのオペレーティングシステムが管理する単一のマシン，あるいは第 12 章のように，複数のオペレーティングシステムの下にあるハイパーバイザが管理する単一のマシンであった．この章から分散システムの部が始まる．そこでは協調するプロセスが，ネットワークで結ばれた別々のマシン上で動作する．マシンをまたぐと，プロセスは–通常は第 9 章のソケットを通じて–メッセージを交換することしかできず，それを使うアプリケーションはどれも同じ低水準の通信コードを繰り返し書くことになる．遠隔手続き呼び出しは，このコードを通常の手続き呼び出しのインタフェースの背後に隠す．この章では，RPC システムの構造と，どの RPC システムも下さなければならない設計上の決定–インタフェースの定義，マーシャリング，バインディング，ポインタ引数，部分障害–を示し，続いて Sun RPC と Java RMI がそれらをどう決めているかを調べる．

光ファイバ中の光の速さはおよそ 200 km ms$^{-1}$ であり，5000 km 離れたマシンへの呼び出しは，両端のマシンがどれほど速くても 50 ms 未満では戻らない．

## 13.1 なぜ遠隔手続き呼び出しか

**出典** P4L1，なぜ RPC か，RPC の利点；Birrell と Nelson [44]；OSTEP ch. 48．

2 つのクライアント・サーバアプリケーションが問題を示す．第 1 のものでは，クライアントがサーバにファイルを要求し，サーバはそのファイルの内容を返す．第 2 のものでは，クライアントが画像とともに画像処理アルゴリズムの名前とそのパラメータを送り，サーバは処理した画像を返す．サービスは異なるが，2 つのアプリケーションの通信コードはほとんど同じである．どちらも次のことを行わなければならない．

- クライアントとサーバが通信するためのソケットを作り，初期化する；
- メッセージのためのバッファを確保し，そこに内容を詰める；
- メッセージをどう解釈すべきかを相手側に伝えるプロトコル情報–要求された操作（ファイルの取得，アルゴリズムの適用）や後続のデータの大きさなど–を含める；
- データをバッファへコピーする．第 1 のアプリケーションではファイル名とファイルの内容，第 2 のアプリケーションではアルゴリズムのパラメータと画像である．

そして受信側は逆の手順を行う．アプリケーションに固有なのは，操作と，やり取りされるデータの型だけである．

▷ remote procedure call

**定義 13.1** (遠隔手続き呼び出し). **遠隔手続き呼び出し**（RPC）とは，プロセスが，別のアドレス空間–通常は別のマシン上–で実行される手続きを，ローカルな手続きを呼び出すのと同じやり方で呼び出せるようにするプロセス間通信機構である．手続きへ引数を渡し結果を呼び出し元へ返す通信は，RPC システムが行う．

RPC は，アドレス空間やマシンをまたぐやり取りの開発を単純にすることを目的とする．その利点は次のとおりである．

**より高水準のインタフェース** データの移動と通信は，ソケットに書きソケットから読むバイトバッファとしてではなく，型の付いた引数と結果を持つ手続きの呼び出しとして表現される．

**エラー処理** RPC システムは通信のエラーを検出し，統一された形でアプリケーションへ報告する．

**複雑さの隠蔽** 接続の管理から，マシンごとのデータ表現の違いに至るまで，マシン間のやり取りの詳細は，RPC システムとそれが生成するコードが扱う．

Birrell と Nelson [44] は，1984 年に Xerox PARC の Cedar プログラミング環境のための RPC 機構の設計と実装を記述した．彼らが選んだ構造，すなわち生成されたスタブと両側のランタイムライブラリからなる構造（13.3 節）は，この章で扱う 2 つを含む後の RPC システムの基礎になっている．[1]

[1] 彼らの測定は，2.94 Mbit s$^{-1}$ の Ethernet で結ばれた 2 台の Dorado ワークステーションの間で行われ，オーバーヘッドの規模を示している．引数も結果もない呼び出しの所要時間は中央値で 1097 µs，そのうち転送が 131 µs であり，同じ呼び出しをローカルに行うと 9 µs であった [44]．

## 13.2 RPC への要件

遠隔の操作を手続き呼び出しのように見せるという目標から，RPC システムには次の要件が生じる．

**クライアント・サーバのやり取り** RPC はクライアント・サーバモデルを対象とする．サーバが一組の手続きを提供し，クライアントがそれらを呼び出す．

**手続き呼び出しインタフェース** RPC は同期的な呼び出しのセマンティクスを持つ．呼び出したプロセス，マルチスレッドのプロセスであれば呼び出したスレッドは，呼び出しを行うとブロックし，手続きが完了してその結果が返されて初めて実行を続ける．ローカルな呼び出しとまったく同じである．

**型検査** 引数と結果には型が宣言されているので，RPC システムはそれらを検査し，呼び出しが手続きに合致しないときはエラーを報告できる．型はまた，メッセージのバイト列を受信側がどう解釈すべきかを伝える．ソケットはそれを解釈されないバイト列として配送するだけである．

**マシン間の変換** クライアントとサーバは，データの表現が異なるマシン上で動作しうるので，RPC システムは表現の間の変換を行わなければならない．

**より高水準のプロトコル** RPC はトランスポート層の上のプロトコルである．クライアントを認証しその呼び出しを認可するアクセス制御や，耐障害性の機構を組み込むことができ，TCP や UDP など異なるトランスポートプロトコルを使えることが望ましい．

表現の違いのうち最も一般的なのは，複数バイトからなる値のバイトを格納する順序である．整数の最上位バイトを最も低いアドレスに格納するマシンを**ビッグエンディアン**，最下位バイトをそこに格納するマシンを**リトルエンディアン**と呼ぶ．x86 プロセッサはリトルエンディアンであり，多くのネットワークプロトコルと 13.13 節 の符号化はビッグエンディアンの順序を使う．

出典 P4L1，RPC への要件；OSTEP ch. 48．

Sun RPC は UDP と TCP の上で動作する [45]．NFS は UDP で始まり，NFSv4 は TCP のような輻輳制御を持つトランスポートを要求する [48]．

▷ big-endian
▷ little-endian

**例 13.2.** 32 ビット整数 300 = 0x0000012C は，アドレスの昇順に，ビッグエンディアンのマシンでは 00 00 01 2C のバイト列を，リトルエンディアンのマシンでは 2C 01 00 00 を占める．リトルエンディアンのクライアントがこの整数のバイトをそのままメッセージへコピーすると，ビッグエンディアンのサーバはそれを 0x2C010000 = 738 263 040 と読む．

## 13.3 RPC システムの構造

ローカルな手続き呼び出しでは，呼び出し元が引数をスタックまたはレジスタに置き，手続きへジャンプする．手続きは結果を計算して戻る．RPC では，呼び出し元と手続きは異なるアドレス空間にあり，引数も手続きのコードも相手側からはアクセスできない．そこで呼び出しは，遠隔の手続きと同じインタフェースを持ちながら代わりに通信を行う，ローカルな手続きへジャンプする（図 13.1）．

出典 P4L1，RPC の構造，RPC の手順；Birrell と Nelson [44]；OSTEP ch. 48．

**定義 13.3 (スタブ).** **スタブ**とは，RPC システムが生成する手続きであり，手続き呼び出しとメッセージの間の変換を行う．**クライアントスタブ**はクライアントにリンクされ，遠隔の手続きのインタフェースを持つ．引数をメッセージに詰め，応答から結果を取り出す．**サーバスタブ**はサーバにリンクされ，引数を取り出し，手続きの実際の実装を呼び出し，その結果を応答に詰める．

▷ stub

第 1 章のシステムコールのラッパと同じ考え方である．本体が境界を越えるローカルな手続きである．

**定義 13.4 (RPC ランタイム).** **RPC ランタイム**とは，両側に存在し，クライアントとサーバの間でメッセージを転送するライブラリである．接続を管理し，メッセージを送受信し，失われたメッセージを再送し，受信した各メッセージを適切なスタブへ渡す．

▷ RPC runtime

呼び出し元にも手続きにも通信のコードは含まれず，どちらもローカルな呼び出しだけを見る．

Birrell と Nelson のシステムも同じ部品からなり，そのスタブ生成器は Lupine と呼ばれた [44]．

### 13.3.1 RPC の手順

RPC は次の手順で進む．番号は 図 13.1 のものである．

**–1 登録** サーバは，提供する手続き，その引数の型，および自身の所在を登録し，クライアントがそのサーバを見つけられるようにする（13.6 節）．

**0 バインド** クライアントは，目的の手続きを提供するサーバを見つけ，例えば接続を開いてそれにバインドする．

**1 呼び出し** クライアントが手続きを呼び出す．制御はクライアントスタブへ移り，呼び出し元はブロックする．

**ヒント** トレース上では，手順 2–5 と帰り道のそれに対応する手順は純粋なオーバーヘッドである．手順 6 のローカルな呼び出しにスタブとランタイムが付け加えるコストである．

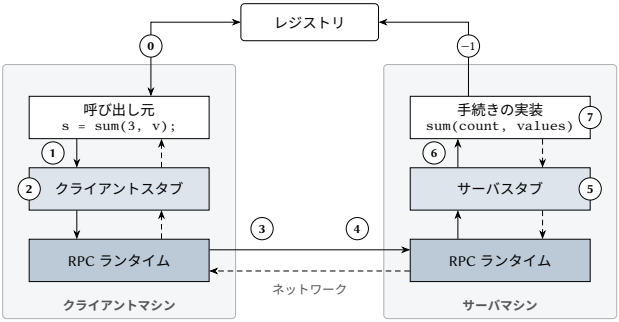



**図 13.1:** RPC システムの構造と呼び出しの手順．実線の矢印は呼び出しを，破線の矢印は結果の返送を表す．手順 −1 と 0 は最初の呼び出しの前に行われる．

**2 マーシャル** クライアントスタブが引数をマーシャルする．すなわちバッファへ直列化する（13.5 節）．
**3 送信** クライアントの RPC ランタイムが，バインドの際に合意したトランスポートプロトコルでメッセージをサーバへ送る．
**4 受信** サーバの RPC ランタイムがメッセージを受信し，サーバスタブへ渡す．アクセス制御の検査はこの時点で行われる．
**5 アンマーシャル** サーバスタブが引数をアンマーシャルする．すなわちバッファから取り出し，対応するデータ構造を作る．
**6 実際の呼び出し** サーバスタブが手続きのローカルな実装を呼び出す．
**7 結果** 手続きが結果を計算し，結果は逆の経路でクライアントへ戻る．サーバスタブがそれをマーシャルし，両側のランタイムが応答を転送し，クライアントスタブがそれをアンマーシャルして呼び出し元へ返し，呼び出し元は実行を再開する．

手順 −1 と 0 は最初の呼び出しの前に一度だけ行われ，手順 1–7 は呼び出しのたびに繰り返される．

**注意** Sun RPC の既定の資格情報 AUTH_SYS は，クライアントが自称するユーザ ID とグループ ID にすぎない．本当の認証には RPCSEC_GSS が要る [49]．

> **要点**
> RPC は，呼び出し元にとっても手続きにとってもローカルな手続き呼び出しに見える．スタブが呼び出しとメッセージの間を変換し，RPC ランタイムがマシン間でメッセージを運ぶ．

## 13.4 インタフェース定義言語

クライアントとサーバは，サーバが何をできるか，そして各手続きの引数と結果について合意していなければならない．クライアントはあるサーバにバインドするかどうかを決めるためにこの情報を必要とし，RPC システムはスタブを自動的に生成するために，正確で機械可読な形でそれを必要とする．

**出典** P4L1，インタフェース定義言語，IDL の記述；OSTEP ch. 48．

▷ interface definition language

**定義 13.5 (インタフェース定義言語).** **インタフェース定義言語**（IDL）とは，サーバが公開するインタフェース–手続きの名前，その引数と結果の型，およびバージョン番号–を記述するための言語である．

RPC システムは 2 種類の IDL のいずれかを使える．
**言語非依存の IDL** クライアントやサーバを書くプログラミング言語とは独立した専用の言語であり，そこから対応する各言語向けのスタブが生成される．Sun RPC は XDR を使う（13.9 節）．
**言語固有の IDL** インタフェースをアプリケーション自身のプログラミング言語で記述するので，プログラマは別の言語を学ぶ必要がない．Java RMI は Java を使う（13.14 節）．
いずれの場合も，IDL が記述するのはサーバのインタフェースだけであり，その実装ではない．

バージョン番号はサービスの進化を支える．サービスが更新されるとき，すべてのサーバが同時に更新されるとは限らず，1 つのサーバが古いバージョンと新しいバージョンのインタフェースを並べて公開することもある．期待するバージョンを示すことで，クライアントはそのバージョンを実装するサーバにだけバインドする．

## 13.5 マーシャリングとアンマーシャリング

呼び出しの引数は呼び出し元のメモリにある．スタックやレジスタ上の値と，ポインタが指す，アドレス空間の別の場所にあるデータである．一方メッセージは連続したバイト列である．

出典 P4L1，マーシャリング，アンマーシャリング；OSTEP ch. 48.

**定義 13.6** (マーシャリング). 呼び出しの引数や結果を，メッセージとして送れ受信側が正しく解釈できるように，クライアントとサーバの間で合意された符号化で連続したバッファへ変換することを**マーシャリング**，あるいは**直列化**と呼ぶ．

▷ marshalling

**定義 13.7** (アンマーシャリング). その逆の変換を**アンマーシャリング**と呼ぶ．受け取ったバッファをインタフェースが与える型に従って解析し，値を取り出し，必要に応じてメモリを確保・初期化しながら，受信側のアドレス空間に対応するデータ構造を作る．

▷ unmarshalling

大きさが固定のデータについては，インタフェースの型が，ある値がどこで終わり次の値がどこから始まるかを受信側に伝える．配列や文字列のように大きさが可変のデータには追加の情報が要る．要素の前に長さを符号化するか，あるいは C の文字列を NUL 文字が終端するように，特別な値で終わりを示すかである．

注意 ここではマーシャリングと直列化は同義である．区別する場合，直列化はバイト表現だけを，マーシャリングは値の再構成に要るもの（Java ならクラスのコードベース）も含めて指す．

**例 13.8.** あるサーバのインタフェースが，手続き `int sum(int count, int *values)` を宣言し，`values` は `count` 個の整数を指すと定めているとする．クライアントは `sum(3, v)` を呼び出す．ここで `v` はアドレス 0x5a10 にある配列で，7，−2，300 を保持している．整数を 4B とすると，クライアントスタブは図 13.2 の 16B のバッファ，すなわち個数に続いて 3 つの要素を並べたものを作る．アドレス 0x5a10 は送られない．サーバスタブは個数を読み，整数 3 つ分のメモリを—ここでは 0x9c40 に—確保し，要素をそこへコピーし，`count` = 3，`values` = 0x9c40 で実装を呼び出す．

注意 C の文字列は NUL で終端される．XDR の文字列は長さに続いてバイト列として送られ，終端記号は付かない（例 13.16）．

データは行きと帰りで少なくとも 2 度—送信バッファへ，受信バッファから—複製される．第 9 章の共有メモリは，まさにこの複製を避けるためにある．

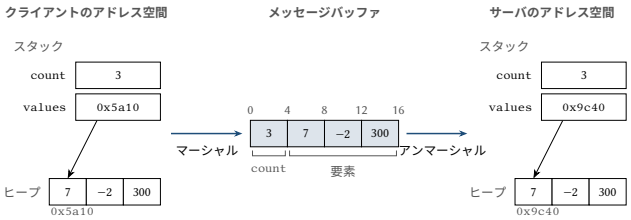



**図 13.2:** `sum(3, v)` の引数のマーシャリングとアンマーシャリング．バッファには，ポインタが指す配列を含めて引数の値が入るが，ポインタそのものは入らない．その上に付した数はバイトオフセットである．

ランタイムが送るメッセージは，呼び出される手続きも識別する．Sun RPC が符号化した引数の前に置くヘッダは 13.13 節 で示す．

マーシャリングとアンマーシャリングのコードは手で書くものではない．IDL のコンパイラが，インタフェースからスタブとともにそれを生成する．Sun RPC については 13.10 節で示す．

## 13.6 バインディングとレジストリ

クライアントがサーバの手続きを呼び出せるようになる前に，どのサーバを使い，どうやってそこへ到達するかを決めなければならない．

出典 P4L1，バインディングとレジストリ；Birrell と Nelson [44].

**定義 13.9** (バインディング). **バインディング**では，クライアントが，サービスの名前とバージョンで識別される通信相手のサーバを決め，ネットワークアドレス，トランスポートプロトコル，ポートで与えられるそのサーバへの接続方法を決め，通信を確立する．

▷ binding

**定義 13.10** (レジストリ). **レジストリ**とは，利用可能なサービスのデータベースである．サーバは自分のサービスをそこに登録し，クライアントはサービス名で検索して，そのサービスを提供するサーバと，そのサーバに接続するために必要な情報を見つける．

▷ registry

レジストリの構成には 2 通りがある．

**分散レジストリ** 複製されることもあるレジストリサービスであり，どこで動作していて

も任意の RPC サービスがそこに登録できる．クライアントはサービスの所在を知らなくてもそれを見つけられる．

Birrell と Nelson はバインディングに分散データベース Grapevine を用いた [44].

**マシンごとのレジストリ** 各マシンが，そのマシン上のサービスのためのレジストリを動かす．クライアントはマシンのアドレスをあらかじめ知っていなければならず，レジストリはそのサービスに到達できるポート番号を与える．

レジストリには命名の規約も要る．最も単純なものは，クライアントのサービス名が登録された名前と厳密に一致することを求める．より凝ったものは同義語も考慮し，sum という名前のサービスを探すクライアントが add や summation として登録されたものも見つけられるようにできる．

## 13.7 引数としてのポインタ

出典 P4L1，RPC におけるポインタ；OSTEP ch. 48.

例 13.8 の sum のようなローカルな手続きは，ポインタを受け取ってそれを間接参照できる．呼び出し元と手続きがアドレス空間を共有しているからである．RPC では，ポインタはクライアントのアドレス空間を指す．サーバ上で同じアドレスは無意味であり，割り当てられていないかもしれず，無関係なデータを保持しているかもしれない．RPC システムはこの問題を 2 通りのいずれかで解決する．

**ポインタを許さない** 遠隔インタフェースにポインタ引数を含めてはならないとする．

**指されたデータを直列化する** クライアントスタブは，ポインタが指すデータ構造を送信バッファへコピーし，サーバスタブはそれをサーバのアドレス空間に再構成して，その複製へのポインタを渡す（図 13.2）．連結されたデータ構造については，スタブがポインタを再帰的にたどる．

ポインタを直列化する場合，サーバが操作するのは複製である．サーバが指されたデータに加えた変更は，インタフェースがそのデータを結果の一部として返さない限りクライアントからは見えない．

## 13.8 部分障害

出典 P4L1，部分障害の扱い，RPC の設計上の選択のまとめ；OSTEP ch. 48.

ローカルな手続き呼び出しは，戻るか，さもなければそれを行ったプロセスもろとも失敗する．呼び出し元と手続きは，1 台のマシン上の 1 つのプロセスという運命を共にしている．分散システムは部分的に故障しうる．

▷ partial failure

**定義 13.11** (部分障害). **部分障害**とは，分散システムの一部の構成要素–マシン，プロセス，ネットワークのリンク，あるいは個々のメッセージなど–が故障し，他の構成要素は動作し続けることをいう．

RPC が戻らないとき，その原因は，サーバのマシンが停止している，サーバのプロセスがクラッシュした，または過負荷である，ネットワークが停止している，あるいは要求か応答が失われた，のいずれでもありうる．クライアントはどの場合にも同じこと–応答がないこと–しか観測できず，どの構成要素が故障したのかを見分けられない（図 13.3）．

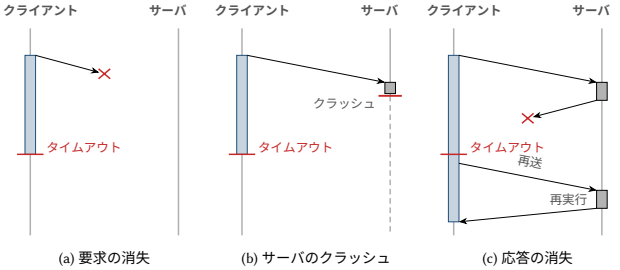



**図 13.3:** クライアントが区別できない 3 つの部分障害．網掛けの帯はクライアントがブロックしている時間を，灰色の箱は手続きの実行を，× は失われたメッセージを表す．(a) 要求が失われる．(b) 呼び出しの実行中にサーバがクラッシュする．(c) 応答が失われ，再送が手続きを 2 度目に実行する．

RPC システムは**タイムアウト**で待ち時間に上限を設け，期限が切れたら要求を再送できるが，タイムアウトと再送は何も保証しない．[2] サーバが停止している間は再送は成功しえないし，応答が失われていた場合には，図 13.3c のように再送は手続きをもう一度実

[2] 応答が届くまで再送すれば少なくとも *1* 回の意味論に，最初の試みで諦めれば高々 *1* 回の意味論になる．どちらもちょうど 1 回ではない．サーバが最近の要求識別子と応答の対を保持し，再送に対しては手続きを実行し直さずその記録から答えれば，ちょうど 1 回に近づく．NFS のサーバはこれを重複要求キャッシュと呼ぶ．

行させる．繰り返しの実行が無害なのは，1 回の実行と同じ効果しかもたらさない**冪等**な手続きに限られる．

**例 13.12.** ファイルにレコードを追加する手続きは冪等ではない．200 B のレコードを追加する呼び出しの応答が失われると，再送はレコードを 2 度目に追加し，ファイルは 400 B 大きくなる．例えばオフセット 4096 のように，与えられたオフセットに 200 B を書く手続きは冪等である．2 度実行しても内容は同じである．

そこで RPC システムは，呼び出しが部分的に失敗しうる様々な場合をまとめて受け止める特別なエラー通知–エラーコード，シグナル，または例外–を提供する．呼び出し元はすべての呼び出しでそれを受け取る用意をしておかなければならない．ただしそれは，どの構成要素が故障したのかも，手続きが実行されたのかどうかも示さない．

Birrell と Nelson の意味論では，呼び出しが戻れば手続きはちょうど 1 回実行され，さもなければ例外が上がる [44]．

> **要点 — RPC の設計上の選択**
> どの RPC システムも次を決めなければならない．
> - サーバをどう見つけるか．レジストリを介したバインディング；
> - サーバとどうやり取りし，データをどう詰めるか．IDL とマーシャリング；
> - ポインタ引数をどう扱うか．許さないか，指されたデータを直列化するか；
> - 部分障害をどう扱うか．特別なエラー通知による．

## 13.9 Sun RPC

Sun Microsystems は 1980 年代に Unix システムのために **Sun RPC** を開発した．最もよく知られた応用はネットワークファイルシステム（第 14 章）である．現在ではほとんどのプラットフォームで利用でき，ONC RPC（Open Network Computing RPC）として標準化されている [45]．Sun RPC は，13.8 節 の末尾にまとめた 4 つの設計上の選択を次のように行う．

▷ Sun RPC

**出典** P4L1，SunRPC とは何か，SunRPC の概要，SunRPC の XDR の例；RFC 5531 [45]；RFC 4506 [46]．

**バインディング** RPC サービスを提供するマシンはどれもレジストリデーモンを動かす（13.11 節）．
**IDL** XDR がインタフェースの記述にも，通信路上のデータの符号化にも使われる．
**ポインタ** ポインタは許され，指されたデータが直列化される．
**障害** ランタイムはタイムアウトを用いて呼び出しを再送し，障害の原因についてできる限りの情報を返す．

**定義 13.13 (外部データ表現).** Sun RPC の言語非依存の IDL と，それが記述するデータの標準的なバイナリ符号化とを合わせて，**外部データ表現**（XDR）と呼ぶ [46]．

▷ external data representation

クライアントとサーバは手続き呼び出しを通じてやり取りする．サービスのインタフェースは .x ファイルに XDR で記述され，そこから rpcgen が特定の言語のスタブを生成する．以下では C である（13.10 節）．サーバは自分のマシンのレジストリデーモンに登録し，デーモンはサービスの番号，バージョン，トランスポートプロトコル，ポートを記録する．バインディングは**クライアントハンドル**を作る．クライアントはそれをすべての呼び出しに渡し，ランタイムはその中にクライアントの RPC の状態を保持する．クライアントとサーバは同じマシン上で動いても，別々のマシン上で動いてもよい．現在の実装は TI-RPC（トランスポート非依存 RPC）の系譜にあり，そのライブラリは特定のトランスポートプロトコルに依存しない．

### 13.9.1 インタフェースの記述

次の XDR の記述 sum.x は，最大 64 個の整数の和を返すサービスを記述する．

```
const MAX_VALUES = 64;

struct sum_in {
  int values<MAX_VALUES>;     /* 可変長配列 */
};
struct sum_out {
  int total;
};

program SUM_PROG {                      /* サービス */
  version SUM_VERS {
    sum_out SUM_PROC(sum_in) = 1;       /* 手続き 1 */
  } = 1;                                /* バージョン 1 */
```

```
} = 0x20000123;                                    /* プログラム番号 */
```

データ型 sum_in と sum_out は引数と結果を記述する．rpcgen は既定では引数を 1 つ，結果を 1 つしか許さない（-N を付ければ引数は複数でもよい）ので，複数の値は構造体にまとめる．サービス自体は**プログラム**であり，1 つ以上のバージョンを含み，各バージョンが手続きを含む．.x 言語のこの部分は RPC 言語であり，プロトコルとともに定義された XDR の拡張である [45]．プログラム，バージョン，手続きは番号で識別され，要求はその 3 つすべてを指定する．32 ビットのプログラム番号はサービスを一意に識別する．values に 3 つの整数 7，−2，300 を設定して SUM_PROC を呼び出したクライアントは，total = 305 の sum_out を受け取る．

プログラム番号
0x00000001–0x1fffffff は IANA が割り当て—NFS は 100003 である—，0x20000000–0x3fffffff は各サイトの管理者に委ねられる．上の 0x20000123 はこの範囲にある [45]．

## 13.10 rpcgen によるインタフェースのコンパイル

▹ rpcgen
**出典** P4L1，XDR のコンパイル，XDR のコンパイルのまとめ；OSTEP ch. 48.

Sun RPC のスタブは，XDR の記述を C のコードに翻訳するコンパイラ **rpcgen** が生成する．コマンド rpcgen -C sum.x（-C は ANSI C を選ぶ．現在の Linux の rpcgen では既定）は，4 つのファイルを生成する（図 13.4）．

sum.h データ型とスタブ手続きの C の宣言，および SUM_PROG，SUM_VERS，SUM_PROC などの定数．他のすべてのファイルがこれをインクルードする．

sum_clnt.c クライアントスタブ sum_proc_1．手続きとバージョンにちなんで名付けられる．引数をマーシャルし，ランタイムを通じて要求を送り，応答を待ち，結果をアンマーシャルする．

sum_svc.c サーバスタブと，サーバプログラムの骨組み．その main 関数はトランスポートを作り，プログラムとバージョンをレジストリデーモンに登録し，要求を待つループに入る．要求ごとに，ディスパッチ手続き sum_prog_1 が手続き番号を判別し，引数をアンマーシャルし，sum_proc_1_svc を呼び，結果をマーシャルして送る．

sum_xdr.c sum_in と sum_out をマーシャル・アンマーシャルする XDR ルーチン．両側で使われる（13.12 節）．

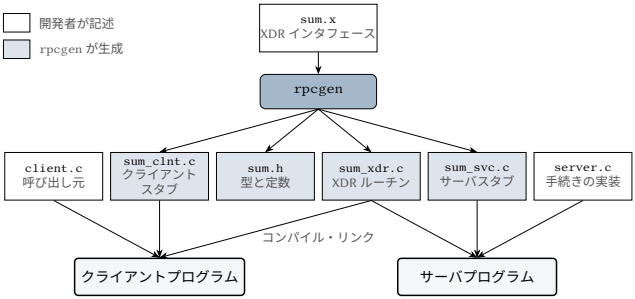



**図 13.4:** Sun RPC のクライアントとサーバの構築．開発者はインタフェース，クライアントプログラム，手続きの実装を書き，残りは rpcgen が生成する．sum.h はすべての C ファイルがインクルードする．

開発者が書くコードは 2 つである．サーバ側では手続きの実装 sum_proc_1_svc（以下に示す），クライアント側では sum.h をインクルードして sum_proc_1 を呼ぶプログラムである（13.11 節）．それぞれを，生成されたファイルとともにコンパイルしリンクする．残りは RPC ランタイムライブラリが行う．ソケットの利用などオペレーティングシステムとのやり取りと，通信の管理である．

**ヒント** 生成される名前には規則がある．手続き名を小文字にしてバージョン番号を続け—sum_proc_1—，開発者が書く手続きにはさらに _svc が付く．

```
#include "sum.h"

sum_out *
sum_proc_1_svc(sum_in *in, struct svc_req *req)
{
  static sum_out out;        /* 呼び出しより長く生きる */
  u_int i;

  out.total = 0;
  for (i = 0; i < in->values.values_len; i++)
    out.total += in->values.values_val[i];
```

```
    return &out;                /* スタブがマーシャルする */
}
```

結果はポインタで返され，手続きから戻った後も，サーバスタブがそれをマーシャルし終えるまで有効でなければならないので，静的変数に保持する．生成されるクライアントスタブも同様に静的変数へのポインタを返し，次の呼び出しがそれを上書きする．このため rpcgen -C が生成するコードは，マルチスレッドのクライアントやサーバでは安全でない．-M オプションを付けると，rpcgen はスレッドセーフなコードを生成し，スタブの呼び出し元が結果のためのメモリを与える．このオプションはサーバをマルチスレッドにするわけではない．生成される main は依然として一度に 1 つの要求しか扱わない．Solaris では，さらに -A オプションを付けると，サーバのランタイムが要求のためのスレッドを自動的に生成する．

## 13.11 Sun RPC のレジストリとバインディング

Sun RPC のレジストリはマシンごとである．RPC サービスを提供する各マシンが自分のレジストリデーモンを動かし，クライアントはよく知られたポートでそれに接続する．

出典 P4L1，SunRPC のレジストリとバインディング；RFC 1833 [47]．

**定義 13.14** (rpcbind)．Sun RPC のレジストリデーモンが **rpcbind** である．TCP と UDP の両方でポート 111 を待ち受け，プログラム番号，バージョン番号，トランスポートプロトコルを，対応するサーバが待ち受けるアドレスへ対応付ける．

▷ rpcbind

rpcbind デーモンは，より古い *portmapper* デーモンを引き継いだもので，その名前は下の出力にもなお現れる．ポート 111 は特権ポートなので，デーモンはスーパーユーザの権限で起動しなければならない．サーバが起動すると，生成された main が，プログラムの各バージョンを各トランスポートについて登録する（図 13.5，手順 1）．あるマシンの登録は rpcinfo -p で一覧できる．以下はその抜粋である．

rpcinfo -p host は遠隔の rpcbind に問い合わせる．Linux のパッケージ名は rpcbind であり，以前は portmap であった．

```
$ rpcinfo -p
   program vers proto   port  service
    100000    4   tcp    111  portmapper
    100000    2   udp    111  portmapper
    100003    3   tcp   2049  nfs
 536871203    1   udp  39871
 536871203    1   tcp  40123
```

各行はプログラム番号，バージョン，トランスポートプロトコル，ポートを示す．デーモン自身はポート 111 のプログラム 100000 として現れ，NFS サーバはプログラム 100003 として現れる．最後の 2 行が sum サーバであり，生成された main が UDP と TCP について登録したものである．そのプログラム番号 0x20000123 が 10 進で表示されている．

NFS は登録されたポート 2049 でも待ち受ける．NFSv4.1 のクライアントは RPC のバインディングプロトコルを使うべきでない [50]．

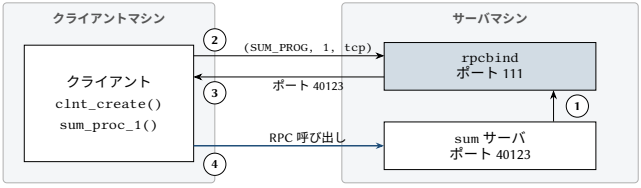



**図 13.5:** Sun RPC における登録とバインディング．(1) サーバが自分のプログラム，バージョン，トランスポート，ポートを rpcbind に登録する．(2, 3) clnt_create がサーバマシンの rpcbind にポートを尋ねる．(4) 呼び出しは直接サーバへ送られる．

クライアントは clnt_create でバインドする．その引数は，サーバマシンの名前，プログラム番号，バージョン番号，トランスポートプロトコルである．clnt_create はそのマシンの rpcbind にサーバのポートを尋ね（手順 2 と 3），CLIENT 型のクライアントハンドルへのポインタを返す．クライアントはそれをすべてのスタブに渡す．ハンドルはサーバのアドレスのほか，認証情報と直近の呼び出しの状態を保持し，エラーメッセージはそこから得られる．クライアントの全体を以下に示す．

サーバ自身のポートは動的に割り当てられ，再起動のたびに変わるので，ファイアウォールであらかじめ開けておくことができない．また rpcbind は誰の問い合わせにも答えるので，露出したデーモンは UDP のリフレクション攻撃に使われてきた [51]．

```
#include <stdio.h>
#include <stdlib.h>
#include "sum.h"

int main(int argc, char *argv[])
```

```
{
  int v[3] = { 7, -2, 300 };
  sum_in in = { { 3, v } };   /* values_len, values_val */
  sum_out *out;
  CLIENT *clnt;

  if (argc != 2) {
    fprintf(stderr, "usage: %s host\n", argv[0]);
    exit(1);
  }
  /* バインド：argv[1] の rpcbind にポートを尋ねる */
  clnt = clnt_create(argv[1], SUM_PROG, SUM_VERS, "tcp");
  if (clnt == NULL) {
    clnt_pcreateerror(argv[1]);
    exit(1);
  }
  out = sum_proc_1(&in, clnt);          /* RPC 呼び出し */
  if (out == NULL) {                    /* RPC が失敗 */
    clnt_perror(clnt, argv[1]);
    exit(1);
  }
  printf("total = %d\n", out->total);
  clnt_destroy(clnt);
  return 0;
}
```

NULL を返すスタブは呼び出しの失敗を報告する．ランタイムはその理由を—例えばサーバマシンに到達できなかった，呼び出しがタイムアウトしたなどを— ハンドルに記録しており，clnt_perror がそれを表示する．応答を受け取れない呼び出しはタイムアウトする．UDP の場合，ランタイムは待つ間に要求を再送する．[3]

[3] UDP 上の RPC メッセージは既定で 8800 B（UDPMSGSIZE）に制限される．libtirpc は 15 s ごとに再送するので，rpcgen のスタブの 25 s のタイムアウトでは要求は 1 度だけ再送される．glibc はバージョン 2.32 で Sun RPC を削除しており，Linux は libtirpc を使う．

## 13.12 XDR のデータ型とルーチン

XDR は，整数，浮動小数点数，列挙，ブール，構造体など，C でおなじみの基本型を提供する．さらに次のものを加える．
const 記号定数．#define に翻訳される；
hyper 64 ビット整数；
quadruple 128 ビット浮動小数点数；
opaque 解釈されないバイナリデータ．C では char のバイト列として表現される．
配列と文字列は 表 13.1 のように宣言する．可変長配列は最大長を山括弧で与え，長さのフィールドと要素へのポインタを持つ C の構造体になる．この構造体の大きさは最大長に依存しない．要素は別に確保されたメモリに置かれる．文字列は，メモリ上では NUL で終端された C の文字列へのポインタになるが，通信路上では長さに続いて文字が並ぶ形で転送される（13.13 節）．

**出典** P4L1，XDR のデータ型，XDR ルーチン；RFC 4506 [46]．

最大長は強制される．生成されたルーチンはそれより長い配列を拒むので，受信側をあふれさせる代わりに呼び出しが失敗する．

**表 13.1:** XDR の配列と文字列，および rpcgen が生成するその C の表現．

| 種類 | XDR の宣言 | C の表現 |
|---|---|---|
| 固定長配列 | `int data[8];` | `int data[8];` |
| 可変長配列 | `int data<8>;` | `struct { u_int data_len; int *data_val; } data;` |
| 文字列 | `string name<32>;` | `char *name;` |
| 固定長 opaque | `opaque id[16];` | `char id[16];` |
| 可変長 opaque | `opaque id<16>;` | `struct { u_int id_len; char *id_val; } id;` |

ポインタは C と同じくアスタリスクで宣言する．そうした**省略可能なデータ**は，ポインタが非 null かどうかを示すブール値と，非 null ならそれに続く指されたデータとして符号化される．こうして連結リストは要素ごとに直列化される [46]．

**注意** 循環のある構造はこの符号化を破る．スタブがポインタを永久にたどってしまうからである．Java のオブジェクト直列化は，各オブジェクトにハンドルを与え，2 度目には後方参照を書く．

### 13.12.1 XDR ルーチン

ファイル sum_xdr.c は，インタフェースのデータ型ごとに 1 つの XDR ルーチンを含む．rpcgen が生成する sum_in 用のルーチンは，ほぼそのままの形で次のとおりである．

```
bool_t
xdr_sum_in(XDR *xdrs, sum_in *objp)
{
  if (!xdr_array(xdrs,
      (char **)&objp->values.values_val,
      (u_int *)&objp->values.values_len, MAX_VALUES,
      sizeof(int), (xdrproc_t)xdr_int))
    return FALSE;
  return TRUE;
}
```

同じルーチンがマーシャリング，アンマーシャリング，解放のいずれにも使われ，XDR ストリーム xdrs のフィールド x_op が方向を選ぶ．符号化するときは長さと要素をバッファへ書き，復号するときはそれらを読み，values_val が null なら配列を確保し，解放するときはこのメモリを返却する．アンマーシャリングの際に確保したメモリは，いつかは解放しなければならない．クライアントでは，不要になった結果に xdr_free が解放の方向でルーチンを適用する．-M を付けて生成したサーバでは，結果をクライアントへ返した後に，サーバスタブが開発者の書いた手続き sum_prog_1_freeresult を呼ぶ．この手続きは通常 xdr_free を呼ぶ．

## 13.13 XDR の符号化

出典 P4L1，符号化，XDR の符号化；RFC 5531 [45]；RFC 4506 [46].

通信路上の RPC メッセージは 3 つの部分からなる．

**トランスポートヘッダ** TCP や UDP などのトランスポートプロトコルと，その下位のプロトコルのヘッダ．

**RPC ヘッダ** 呼び出しの識別情報．クライアントが応答を要求に対応付けるための要求識別子（*xid*），プログラム番号，バージョン番号，手続き番号，および認証データ．

**データ** 引数または結果を，そのデータ型に従ってバイト列へ符号化したもの．

このように XDR は，IDL と，通信路上のデータのバイナリ表現である符号化の両方を包含する．その符号化の規則は少ない．

- すべての項目は 4 B の倍数で符号化される．短いデータはゼロバイトで詰められる．
- 複数バイトの値はビッグエンディアンの順序で転送される．
- 整数は 2 の補数で表現される．
- 浮動小数点数は IEEE 754 形式で表現される．

各マシンは，マーシャルとアンマーシャルの際に自分のネイティブな表現と XDR との間で変換するので，どちらの側も相手の表現を知る必要がない．

バイトストリームの上では，1 つのメッセージが 1 つ以上のフラグメントからなる 1 つのレコードであり，各フラグメントの前に 4 B のヘッダが置かれる．最上位ビットが最終フラグメントの印，残る 31 ビットが長さである [45].

**ヒント** *n* バイトの項目は通信路上で 4⌈*n*/4⌉ バイトを占め，可変長の配列や文字列はさらに 4 B の長さフィールドを加える．

**例 13.15.** 13.11 節のクライアントは，値 7，−2，300 で SUM_PROC を呼び出す．XDR はこの可変長配列を図 13.2 の 16 B に，長さに続いて 3 つの要素という形で符号化する．

```
00 00 00 03   00 00 00 07   ff ff ff fe   00 00 01 2c
```

要素 −2 はクライアントのバイト順序によらず 2 の補数で ff ff ff fe と，300 は 00 00 01 2c と表される．これらのバイトの前には，RPC ヘッダが呼び出しの要求識別子と，プログラム番号，バージョン番号，手続き番号を運ぶ．

文字列は，メモリ上の表現と通信路上の表現の違いを示す．

**例 13.16.** string name<32> と宣言された文字列 "kernel" は，クライアントとサーバのメモリ上では 7 B を占める．6 文字と終端の NUL である．XDR はこれを 12 B に符号化する．4 B の長さ 6，6 文字，そして 4 の倍数にするための 2 バイトのパディングであり，終端記号は付かない．

```
00 00 00 06   6b 65 72 6e   65 6c 00 00
```

> **要点**
> XDR は IDL であり符号化でもある．すべての項目は 4 B の倍数で，ビッグエンディアンの順序で，整数は 2 の補数，浮動小数点数は IEEE 形式で符号化され，可変長のデータは自身の長さを伴う．メッセージは，トランスポートヘッダに，要求，プログラム，バージョン，手続きを識別する RPC ヘッダを加えたものである．

## 13.14 Java RMI

出典 P4L1，Java RMI；Wollrath ら [52].

Java は，そのオブジェクト指向のセマンティクスに合う RPC の対応物を提供する．

▷ remote method invocation

**定義 13.17 (リモートメソッド呼び出し).** 別のアドレス空間にあるオブジェクトのメソッドを，ローカルなメソッド呼び出しの構文とセマンティクスで呼び出すことを，**リモートメソッド呼び出し**（RMI）と呼ぶ．Java RMI では，そのアドレス空間は，同じマシン上あるいは別のマシン上の Java 仮想マシン（JVM）のアドレス空間である．

Java RMI の IDL は Java 自身であり，言語固有の IDL である．遠隔インタフェースは `java.rmi.Remote` を継承した Java のインタフェースであり，その各メソッドは例外 `RemoteException` を宣言する．これが，部分障害を報告するエラー通知である．次の遠隔インタフェースは sum サービスを宣言し，その下のクライアントのコードがそのサービスを呼び出す．

RMI レジストリ自体も遠隔オブジェクトであり，`LocateRegistry.getRegistry` は，別のポートを指定しない限り TCP のポート 1099 にあるレジストリのスタブを返す．

```
public interface Summation extends java.rmi.Remote {
  int sum(int[] values) throws java.rmi.RemoteException;
}

/* クライアント */
Registry reg = LocateRegistry.getRegistry("server");
Summation s = (Summation) reg.lookup("Summation");
int total = s.sum(new int[] { 7, -2, 300 });
```

サーバはインタフェースを実装するオブジェクトを作り，*RMI レジストリ*に名前を付けて登録する．クライアントはその名前を検索して遠隔オブジェクトのスタブを得て，そのメソッドを呼び出す（図 13.6）．上の整数の配列のような通常のオブジェクトである引数は，値渡しされる．すなわち，それが参照するオブジェクトとともに直列化される．遠隔オブジェクトはスタブとして参照渡しされる．

**注意** 複製は呼び出しのたびに作られ，サーバが複製に加えた変更はクライアントからは見えない．13.7 節 の直列化されたポインタと同じ落とし穴である．

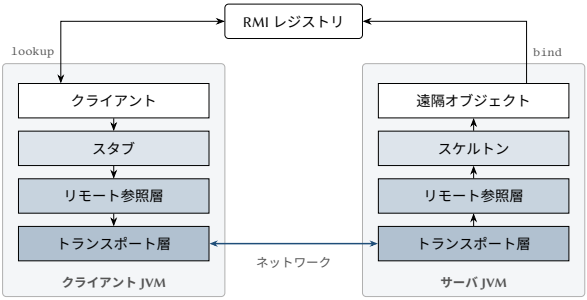



**図 13.6:** Java RMI．呼び出しはクライアントからスタブ，リモート参照層，トランスポート層を経てサーバへ渡り，サーバではスケルトンがそれを遠隔オブジェクトへ配送する．

**表 13.2:** Sun RPC と Java RMI の設計上の選択．

| 選択 | Sun RPC | Java RMI |
|---|---|---|
| バインディング | マシンごとのレジストリデーモン `rpcbind` | RMI レジストリが名前を遠隔オブジェクトへ対応付ける |
| IDL | XDR，言語非依存，`rpcgen` でコンパイル | Java の遠隔インタフェース，言語固有 |
| 符号化 | XDR の符号化 | Java のオブジェクト直列化 |
| ポインタと参照 | 許される，指されたデータを直列化 | オブジェクトは複製，遠隔オブジェクトはスタブとして渡す |
| 部分障害 | タイムアウトと再送，エラー状態はクライアントハンドルに | `RemoteException` |

JDK 1.2 以降，スケルトンは不要である．サーバのランタイムがリフレクションでディスパッチする．J2SE 5.0 以降はスタブが動的に生成され，`rmic` は JDK 15 で削除された．

指されたデータを複製する Sun RPC と対照的に，RMI は直列化可能な引数を複製し，遠隔オブジェクトは参照渡しする．gRPC は同じ 4 つの選択を別の形で行う．IDL と符号化には Protocol Buffers を使い，レジストリの代わりに DNS とロードバランサを置き，すべての呼び出しがステータスコードを返す．

Java RMI は各側に 3 つの層を持つ．最上位は，クライアント側ではスタブ，サーバ側では**スケルトン**であり，スケルトンは呼び出しをアンマーシャルして遠隔オブジェクトへディスパッチする．その下の 2 つの層は RMI ランタイムが提供する．

**リモート参照層** 呼び出しのセマンティクスを決める．単一のオブジェクトへの**ユニキャスト**か，複製など一群のオブジェクトへの**ブロードキャスト**かであり，後者は最初の応答を返すか，すべての応答が一致したときだけ返す．

**トランスポート層** 接続を設定・管理し，データを転送する．この構成は，TCP，UDP，共有メモリなど異なるトランスポートを許す．

表 13.2 は Sun RPC と Java RMI の設計上の選択を比較したものである．

**要点 — 章のまとめ**
RPC は，プロセスが別のアドレス空間—通常は別のマシン上—の手続きを，ローカルな呼び出しと同じ同期的なセマンティクスで呼び出せるようにする．IDL から生成されたスタブが引数と結果をマーシャル・アンマーシャルし，RPC ランタイムがメッセージを転送する．どの RPC システムも，クライアントがレジストリを介してどうサーバにバインドするか，インタフェースをどう記述しデータをどう符号化するか，ポインタ引数をどう扱うか，そしてクライアントには診断できない部分障害をどう報告するかを決めなければならない．Sun RPC は XDR を IDL と符号化に用い，`rpcgen` でスタブを生成し，マシンごとの `rpcbind` レジストリを使う．Java RMI は Java を IDL とし，呼び出しのセマンティクスを定めるリモート参照層を加える．

第 14 章

# 分散ファイルシステム

| | |
|---|---|
| **講義動画** | P4L2 |
| **教科書** | OSTEP [1] ch. 49–50；Silberschatz ら [2] ch. 15，19；Nelson ら [53] |
| **キーワード** | 分散ファイルシステム，レプリケーション，パーティショニング，アップロード・ダウンロードモデル，ステートレスサーバ，ステートフルサーバ，UNIX セマンティクス，セッションセマンティクス，リース，NFS，ファイルハンドル，Sprite，遅延書き込み，並行書き込み共有 |

第 11 章の仮想ファイルシステムは，アプリケーションが使うファイルがどこにあるかをアプリケーションから隠す．第 13 章の遠隔手続き呼び出しは，プロセスが別のマシンで操作を実行させることを可能にする．分散ファイルシステムはこの 2 つを組み合わせたものであり，ネットワークで結ばれたマシン上にファイルを格納し，通常のファイルシステムインタフェースを使うクライアントにそれらを提供する．この章では，そうしたシステムの設計空間–クライアントとサーバの間で作業をどう分けるか，サーバがどのような状態を保持するか，キャッシュされた複製をどう一貫させるか，ファイルを共有したときクライアントが何を観測するか–を調べ，続いて 2 つの具体的な設計，すなわち NFS と，ファイルが実際にどう使われるかの測定から設計を導いた Sprite を扱う．

## 14.1 分散ファイルシステム

**出典** P4L2，分散ファイルシステム，DFS のモデル；Silberschatz ら ch. 19．

オペレーティングシステムは，記憶装置の詳細をファイルシステムインタフェースの背後に隠す．その下では，仮想ファイルシステムが各操作を，ファイルが存在するファイルシステムへ渡す．そのファイルシステムは，ファイルをローカルなデバイスに置くこともあれば，ネットワーク上の他のマシンに置くこともある．

▷ distributed file system

**定義 14.1** (分散ファイルシステム). **分散ファイルシステム**（DFS）とは，ネットワークで結ばれたマシン上に格納され，そこから提供されるファイルからなるファイルシステムであって，プロセスは，ファイルが自分のマシンにあるかどうかにかかわらず，通常のファイルシステムインタフェースを通じてアクセスする．

DFS を特徴づける性質は 3 つある．

- 明確に定義されたインタフェースを通じてアクセスされる．アプリケーションはローカルなファイルに対するのと同じシステムコールを使い，VFS はリモートファイルに対する各操作を DFS へ渡す．
- 一貫した状態を保たなければならない．複数のクライアントが同じファイルを使うとき，DFS はどのクライアントがどのファイルにアクセスしているかを追跡し，更新を伝播し，キャッシュされた複製の整合を保たなければならない．
- 複数の分散モデルを組み合わせうる．ファイルは複数のサーバに複製することも分割することもでき，クライアントとサーバの役割が別々のマシンに存在する必要もない．

分散のモデルを 図 14.1 に示す．

**クライアントとサーバが別のマシンにある場合** 単一のファイルサーバがすべてのファイルを格納し，他のマシン上のクライアントがネットワーク越しにそれらにアクセスする．

**分散ファイルサーバ** ファイルサービス自体が複数のマシンで動作する．**複製された**ファイルサーバでは，すべてのマシンがすべてのファイルを保持し，**分割された**ファイルサーバでは，各マシンがファイルの一部を保持する．両者は 14.7 節 で比較する．

**ピア** すべてのマシンがファイルを格納し，かつ提供する．

最後のモデルでは，クライアントとサーバの区別は曖昧になる．このような構成を**ピアツーピア**と呼ぶ．どのマシンも，ファイルサービスを利用すると同時に提供するピアである．各ピアはファイルの一部を格納し，要求–典型的には自分が格納しているファイルに対する要求–を処理することで負荷の一部を担う．

▷ peer-to-peer

## 14.2 遠隔ファイルサービス

**出典** P4L2，遠隔ファイルサービス：両極端，遠隔ファイルサービス：折衷案；OSTEP ch. 49–50；Silberschatz ら ch. 19．

サーバが 1 台であっても，リモートファイルにアクセスする作業をクライアントとサーバの間で分ける方法は大きく異なりうる．設計空間を画する両極端が 2 つある（図 14.2）．

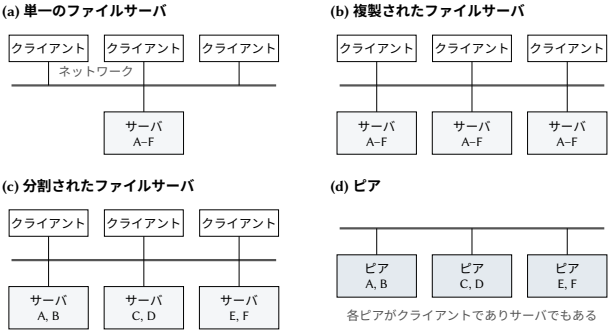



**図 14.1:** DFS のモデル．(a) 単一のサーバがすべてのファイルを格納する．(b) 複製されたファイルサーバでは，どのサーバもすべてのファイルを保持する．(c) 分割されたファイルサーバでは，各サーバがファイルの一部を保持する．(d) すべてのマシンがピアであり，ファイルを格納して提供する．

**定義 14.2** (アップロード・ダウンロードモデル). **アップロード・ダウンロードモデル**では，ファイルにアクセスするクライアントはサーバからファイル全体をダウンロードし，すべての操作をローカルな複製に対して行い，終わったらファイル全体をサーバへアップロードする．

▹ upload/download model

FTP によるファイル転送や，Subversion のようなバージョン管理システムはこの方式で動作する．読み出しも書き込みもクライアント側でローカルに行われるので高速である．しかし，クライアントがファイルのごく一部しか必要としない場合でもファイル全体が転送され，さらにサーバは制御を手放す．いったんファイルがダウンロードされると，サーバは，クライアントがそれをどう扱うかも，変更されたファイルがいつアップロードされるかも知りえない．

**定義 14.3** (遠隔アクセスモデル). **遠隔アクセスモデル**，すなわち真の遠隔ファイルアクセスでは，ファイルはサーバ上に留まり，ファイルに対するあらゆる操作–すべての読み出しと書き込み–がサーバへ送られる．クライアント側でローカルに行われることは何もない．

▹ remote access model

すべてのアクセスがサーバを経由するので，ファイルアクセスは集中化され，一貫性は，複数のプロセスで共有されるローカルなファイルシステムと同じくらい容易に理解できる．しかし，すべてのファイル操作がネットワークの往復とサーバでの処理の代価を払い，またサーバがすべてのクライアントのすべての操作を実行するので，DFS が支えられるクライアント数はサーバの処理能力によって制限される．

1 つのデータセンタ内の往復はおよそ 0.5 ms，ディスクのランダムシークはおよそ 10 ms である（Dean，“Numbers everyone should know”，2009）．例 14.4 で仮定した 1 ms は LAN の値である．

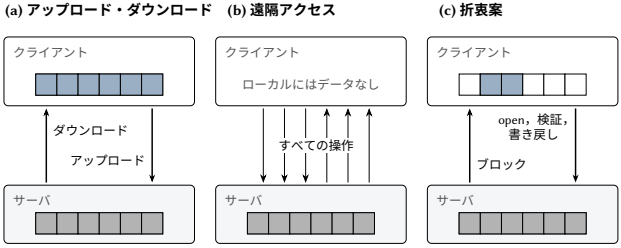



**図 14.2:** 遠隔ファイルサービス．(a) クライアントはファイル全体をダウンロードし，終わったらアップロードする．(b) ファイルはサーバに留まり，サーバがすべての操作を実行する．(c) クライアントは一部のブロックをキャッシュするが，サーバと頻繁にやり取りする．

**例 14.4.** あるクライアントが 1 GiB のファイルを，100 MiB/s で転送するネットワーク越しに扱う．要求 1 回ごとに 1 ms の往復がかかる．クライアントが 4 KiB のブロックを 50 個読むなら，遠隔アクセスモデルでは 50 回の要求，およそ 50 ms で済むのに対し，ファ

**注意** ネットワークの速度はビット毎秒で表される．1 Gbit $s^{-1}$ が運べるのは高々 125 MB $s^{-1}$ であり，ここで仮定した速度はギガビット回線をほぼ使い切った値である．

イルのダウンロードには 1024/100 ≈ 10.2 s かかる．一方，クライアントが 200 000 ブロックを読むなら，遠隔アクセスモデルでは往復だけで 200 s を要するが，ダウンロードは依然として 10.2 s であり，その後の読み出しはすべてローカルである．

**ヒント** クライアントが読むブロック数が $T_{\text{dl}}/t_{\text{RTT}}$ を超えると，ダウンロードが有利になる．ここで $T_{\text{dl}} = S/r$ は，大きさ $S$ のファイルを速度 $r$ でダウンロードする時間である（ここではおよそ 10 000）．

### 14.2.1 折衷案

実用的な DFS は両極端の間に位置し，次の 2 つの原則を組み合わせる．

1. クライアントはファイルの一部，すなわち個々のブロックをローカルに保持してよい．ローカルなブロックに対する操作はレイテンシが低く，その分だけ要求が減ってサーバの負荷が下がるので，1 台のサーバがより多くのクライアントを支えられる．
2. クライアントはサーバとやり取りしなければならず，しかも頻繁にやり取りする．ファイルを open するとき，ローカルなブロックがまだ最新かを確かめるとき，そして自分の変更を送り返すときである．これによりサーバは，クライアントが何をしているかを把握し，どのアクセスを許すかを制御できるようになり，一貫性の維持が容易になる．

Linux の NFS は `rsize`/`wsize` の単位でデータを転送し，その値は最大 1 MiB まで交渉で決まる（`nfs(5)`）．キャッシュされる「ブロック」はディスクのセクタよりはるかに大きい．

その代価は，サーバがより複雑になること，そしてファイル共有セマンティクスがローカルなファイルシステムのそれとは異なるものになることである．ブロックのローカルな複製の上で作業するクライアントは，他のクライアントが加えた変更をすべて，それが起きた瞬間に観測することはできない（14.5 節）．

## 14.3 ステートレスなファイルサーバとステートフルなファイルサーバ

**出典** P4L2，ステートレスなファイルサーバとステートフルなファイルサーバ；OSTEP ch. 49；Silberschatz ら ch. 19．

クライアントとサーバの間の作業の分け方は，サーバがクライアントについて何を覚えていなければならないかを決める．逆に，サーバが保持する状態は，14.2 節 のモデルのうちどれを支えられるかを決める．

▷ stateless server

**定義 14.5** (ステートレスサーバ). **ステートレスサーバ**は，クライアントについての情報を要求と要求の間に保持しない．どのファイルが open されているかも，クライアントがどのデータをキャッシュしているかも，クライアントがファイルのどこを読んでいるかも保持しない．すべての要求は自己完結的であり，それを処理するのに必要な情報をすべて含む．

▷ stateful server

**定義 14.6** (ステートフルサーバ). **ステートフルサーバ**は，クライアントについての情報—どのクライアントがどのファイルにどのモードでアクセスしているか，どのデータをキャッシュあるいはロックしているか—を維持する．

**表 14.1:** ステートレスなファイルサーバとステートフルなファイルサーバ．

| | ステートレスサーバ | ステートフルサーバ |
|---|---|---|
| 適するモデル | アップロード・ダウンロードモデルと遠隔アクセスモデル | 折衷モデル |
| 要求 | 自己完結的；要求当たりのビット数が多い | サーバが保持する状態を参照しうる |
| 支えられるもの | 一貫したキャッシュもロックも支えられない | ロック，キャッシュ，増分的な操作 |
| サーバの資源 | クライアント状態のためには不要 | 状態と一貫性のためにメモリと CPU 時間 |
| 障害の後 | 再起動して続行する | 状態をチェックポイントし復元しなければならない |

ステートレスサーバは両極端の 2 つのモデルには十分だが，折衷案には足りない．クライアントがキャッシュしたブロックの一貫性を保つには，どのクライアントがどのファイルをキャッシュしているかを知る必要があるからである．その利点は，クライアントの状態にメモリも CPU 時間も費やさないこと，そして障害からの回復が単に再起動するだけで済むことである．失われた状態はなく，要求に応答を得られなかったクライアントはそれを送り直せばよい．送り直しが安全なのは，要求が冪等である限りにおいてであり（第 13 章），明示的なオフセットを伴う読み出しや書き込みはまさに冪等である．欠点は，一貫性管理を伴うキャッシュもロックも支えられないこと，そしてすべての要求が操作を完全に—どのファイルか，どのオフセットか，何バイトか—記述しなければならず，要求当たりの転送ビット数が増えることである．ステートレスサーバのクライアン

**注意** NFSv3 のすべての操作が冪等なわけではない．繰り返された REMOVE や排他的な CREATE はエラーを返すので，サーバは再送に備えて重複要求キャッシュを保持する（第 13 章）．

**注意** ステートレスとはクライアントについての状態を指す．サーバはファイルとそのメタデータを依然として保持しており，要求と要求の間に忘れるのは，誰が何を読んでいるかである．

トでも，データをキャッシュし，ファイルを open するときや周期的に自分の複製が最新かをサーバに尋ねることはできる（クライアント駆動の検証．14.4 節．NFS がその例である）．しかしサーバは，変更をクライアントに通知することも，複製が整合していることを保証することもできない．したがって NFSv3 が提供するのは，クライアント駆動の検証とより弱い一貫性だけである（14.9 節）．

折衷モデルにはステートフルサーバが必要である．サーバはクライアントが何をしているかを知っているので，ロック，一貫性の保証を伴うクライアントキャッシュ，そしてファイルの次のブロックを読むといった増分的な操作を支えられる．その代価は 2 つある．サーバが故障すると，サーバが状態をチェックポイントし（第 8 章），復旧機構がそれを復元しない限り，状態は失われる．また，状態を維持し一貫性を強制することはサーバの資源を消費し，その量はキャッシュ機構と一貫性プロトコルに依存する．表 14.1 にこの比較をまとめる．

**例 14.7.** あるクライアントが 12 KiB のファイルを 4 KiB ずつ 3 回の要求で読み，2 回目の読み出しの後にサーバが再起動する（表 14.2）．ステートレスサーバは読み出しのたびにファイルとオフセットを受け取る．ファイルハンドルが 32 バイト（NFSv2 での固定長），オフセットが 4 バイト（これも NFSv2 のもの），個数が 4 バイトなら，各要求は 40 バイトの引数を運ぶ．ステートフルサーバは open されたファイルとそのオフセットを記録しているので，読み出しには 4 バイトのディスクリプタと個数，合わせて 8 バイトで足りる．しかし再起動の後，ステートフルサーバはディスクリプタ 5 をもはや知らず，状態が復元されない限り 3 回目の読み出しは失敗する．一方ステートレスサーバは，何事もなかったかのようにそれを処理する．

**表 14.2:** 2 回目の読み出しの後に再起動するステートレスサーバとステートフルサーバからのファイルの読み出し．

| 段階 | ステートレスサーバ | ステートフルサーバ |
|---|---|---|
| open | `lookup(dir,"data")` がハンドル *h* を返す | `open("data")` が 5 を返す；オフセット 0 を記録 |
| read 1 | `read(`*h*`,0,4096)` | `read(5,4096)`；オフセット 4096 |
| read 2 | `read(`*h*`,4096,4096)` | `read(5,4096)`；オフセット 8192 |
| 再起動 | 失われるものはない | open されたファイルとオフセットが失われる |
| read 3 | `read(`*h*`,8192,4096)` が処理される | `read(5,4096)` が失敗する |

**要点**
実用的な DFS は，クライアントにブロックをキャッシュさせつつ，サーバとのやり取りを頻繁に行わせる．これはアップロード・ダウンロードモデルと遠隔アクセスモデルの折衷案である．この折衷にはステートフルサーバが必要であり，サーバの資源と障害後の復旧という代価と引き換えに，キャッシュ，ロック，増分的な操作を支える．ステートレスサーバはより単純で，再起動によって障害を乗り切るが，クライアントのキャッシュが一貫していることを保証できない．

## 14.4 DFS におけるキャッシュ状態

折衷モデルでは，クライアントはファイルの状態の一部—典型的にはそのブロックのいくつか—をローカルに保持し，open，read，write といった操作をこのキャッシュされた状態に対して行う．クライアントはこれらのブロックを自身のメモリに，第 11 章のバッファキャッシュのように保持する．最も高速な場所だが，容量に限りがあり，クライアントがクラッシュすれば失われる．ローカルなディスクや SSD になら，クライアントはより多くのデータをキャッシュでき，クラッシュを越えて保持できる．その代価はメモリより遅いことだが，それでもネットワークよりは速い．サーバもまた自身のバッファキャッシュにブロックをキャッシュし，それによってディスクアクセスを節約する．どれだけ節約できるかは負荷に依存する．多数のクライアントによる異なるファイルへの要求がそこで交錯し，互いのデータを追い出すからである．同じファイルが複数の場所にキャッシュされると，それらの複製は食い違いうる．クライアント 1 と 2 がともにファイル *F* をキャッシュしており，クライアント 2 が自分の複製を *F′* に変更したとしよう（図 14.3）．この変更はサーバに届かなければならず，クライアント 1 は自分の *F* の複製が古くなったことを知らなければならない．

**出典** P4L2，DFS におけるキャッシュ状態；OSTEP ch. 49–50；Silberschatz ら ch. 19．

NFS（14.8 節）と Sprite（14.10 節）のクライアントは，ブロックをメモリにキャッシュする．

**注意**「ネットワークより速い」は遅いネットワークを前提とする．7200 rpm のディスクのランダムな 1 ブロックはおよそ 13 ms を要し（OSTEP ch. 37），データセンタ内部の往復より遅い．

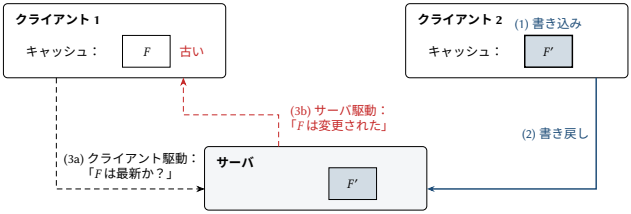



**図 14.3:** クライアント 2 がキャッシュした $F$ の複製を変更し，書き戻す．クライアント 1 は，サーバに尋ねる（クライアント駆動）か通知を受ける（サーバ駆動）ことで，自分の複製が古いことを知る．

これは共有メモリ型マルチプロセッサにおけるキャッシュ一貫性の問題（第 10 章）と同じである．そこでは，ある CPU による書き込みが他の CPU のキャッシュ内のメモリ位置の複製を古くし，キャッシュ一貫性を持つプラットフォームではハードウェアがそれらを更新あるいは無効化する．DFS では一貫性の機構はソフトウェアで実装され，次のいずれかである．

サーバ駆動の通知は，キャッシュされた複製を無効化することも更新することもできる．これは第 15 章の分散共有メモリが直面する，書き込み無効化と書き込み更新の間の選択である．

**クライアント駆動** クライアントが，ファイルを open するときあるいは周期的に，キャッシュしたデータがまだ最新かをサーバに尋ねる．

**サーバ駆動** どのクライアントがファイルをキャッシュしているかを知っているサーバが，ファイルが変更されたときにそれらに通知する．[1]

クライアント駆動の確認はサーバを単純に保つが，1 回ごとに要求の代価がかかり，次の確認まではクライアントが古いデータを持ち続ける．サーバ駆動の通知はステートフルサーバを必要とする．いずれの機構がいつ，どれだけ頻繁に働くかは，DFS が提供するファイル共有セマンティクスによって決まる（14.5 節）．

[1] Sprite と同じころカーネギーメロン大学で作られた Andrew ファイルシステムは，サーバ駆動の設計をさらに推し進めている．クライアントはファイル全体を自分のローカルディスクにキャッシュし，サーバは，ファイルが変更されたときにそれを知らせるというコールバックの約束を保持する．約束が有効な間，open は何の代価も要さない．OSTEP ch. 50 に記述がある．

## 14.5 ファイル共有セマンティクス

単一のマシン上では，プロセス B も open しているファイルにプロセス A が書き込むと，B の次の読み出しは新しいデータを返す．両プロセスとも同じバッファキャッシュを通してファイルにアクセスするので，すべての書き込みが直ちに見えるからである．クライアントがデータをキャッシュする DFS では，クライアント A による書き込みはまず A のキャッシュだけを変更する．それは A がサーバへ送ったときにサーバに届き，クライアント B は，B がそれをサーバから取得した後にはじめてそれを見る．あるクライアントが別のクライアントの書き込みをいつ観測するかについての保証が，DFS の**ファイル共有セマンティクス**である．

**出典** P4L2，DFS のファイル共有セマンティクス；Silberschatz ら ch. 15；OSTEP ch. 49–50．

▷ UNIX semantics

**定義 14.8** (UNIX セマンティクス). **UNIX セマンティクス**の下では，ファイルへのすべての書き込みは，どのマシン上であれ，その後のすべての読み出しから直ちに見える．

UNIX セマンティクスは，アプリケーションがローカルなファイルシステムに期待するものである．しかし DFS でこれを提供するには，他のどのクライアントがファイルを読むよりも前にすべての書き込みを伝播しなければならず，その代価は大きい．

▷ session semantics

**定義 14.9** (セッションセマンティクス). **セッションセマンティクス**の下では，クライアントはファイルを open するときにサーバから最新の版を取得し，ファイルを close するときに自分の変更をサーバへ書き戻す．open からそれに対応する close までの期間が**セッション**である．セッションの間に他のクライアントが加えた変更は，そのセッションからは見えない．

セッションセマンティクスは理解しやすいが，不十分なこともある．ともにファイルを open したままそれを共有するクライアントは互いの変更を見ず，セッションが長く続くほど，異なるクライアントの複製が不整合なままである時間も長くなる．

**周期的な更新**では，クライアントは自分の変更を周期的にサーバへ書き戻し，キャッシュされたデータも周期的に無効化あるいは再検証される．この周期が，クライアントが不整合なデータを観測しうる時間の上限を与える．また DFS は，アプリケーションが自分の変更を直ちにサーバへ反映させるための `flush` や sync といった明示的な操作を提供してもよい．こうしてクライアントがキャッシュしたデータを使う権利は時間的に制限される．すなわちクライアントはデータに対するリースを保持している．

**ヒント** POSIX では，`fsync` がファイルのダーティなデータを安定な記憶装置へ強制的に書き出す．NFS では `close` もサーバへ書き出す（close-to-open 一貫性，14.9 節）．

▷ lease

**定義 14.10** (リース). **リース**とは，サーバがクライアントに限られた期間だけ与える権利

であり，キャッシュしたデータを使う権利やロックを保持する権利がこれにあたる．期間が満了すると，クライアントが更新していない限りその権利は失われる．

キャッシュされたデータに対するリースは排他的である必要はない．複数のクライアントが同じファイルに対するリースを同時に保持できる．さらに 2 つの方法が，不整合な見え方そのものを回避する．

**定義 14.11** (不変ファイル). **不変ファイル**とは，作成された後は決して変更されないファイルである．内容を変えるには新しいファイルを作成し，古いものは削除してもよい．

▷ immutable file

不変ファイルの複製はすべて同一なので，それを共有しても不整合は生じえない．たとえば写真を保管するサービスは，アップロードされた写真をその場で変更することは通常なく，編集された写真は新しいファイルとして保存される．最後に，DFS は**トランザクション**を提供することもできる．アプリケーションは，1 つあるいは複数のファイルに対する複数の更新を 1 つのトランザクションにまとめ，DFS はそれらを，データベースシステムがそうするように，原子的に—すべてが反映されるか，どれも反映されないかのいずれかに—適用する．

第 8 章のコピーオンライトと同じ仕掛けである．新しい版は別の場所に書かれ，古いブロックは後で解放される．

**例 14.12.** ファイル $F$ の初期内容は x である．クライアント A は $t = 0$ s に $F$ を open し，4 s に y を書き，10 s に $F$ を close する．クライアント B は 2 s に $F$ を open し，6 s と 12 s に読み出し，14 s に close する．さらに 15 s に $F$ をもう一度 open し，16 s に読み出す（図 14.4）．周期的な更新については，A のクライアントがダーティなデータを 5 s ごと，すなわち 5, 10, 15, ... 秒に書き戻し，B のクライアントが 3, 8, 13, ... 秒に自分のキャッシュを再検証するものとする．B が何を読むかを表 14.3 に示す．セッションセマンティクスの下では，B は 2 回目のセッションではじめて y を見る．周期的な更新の下では，A の書き戻しに続く最初の再検証の後，すなわち書き込みの 4 s 後に y を見る．一般に，どの書き込みも 2 つの周期の和である 10 s より長く B から見えないままになることはない．

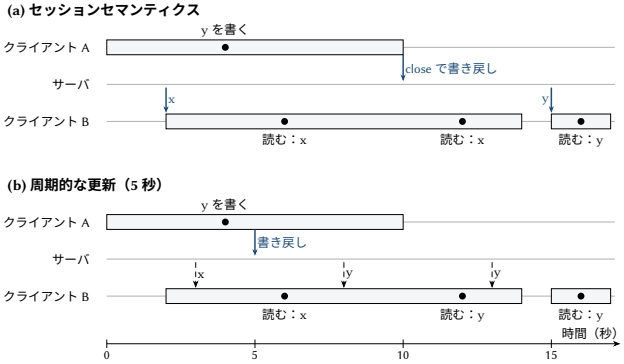



**図 14.4:** 例 14.12 の状況．網掛けの帯がセッションである．(a) セッションセマンティクスの下では，A は close で書き戻し，B は open でファイルを取得する．(b) 周期的な更新の下では，A は 5 s に書き戻し，B は 5 s ごとに再検証する（破線）．

**表 14.3:** 例 14.12 でクライアント B が読む $F$ の内容．

| B が読む時刻 | UNIX セマンティクス | セッションセマンティクス | 周期的な更新 |
|---|---|---|---|
| 6 s | y | x | x |
| 12 s | y | x | y |
| 16 s | y | y | y |

## 14.6 ファイルとディレクトリ

出典 P4L2，ファイルサービスとディレクトリサービス．

どの DFS にも最適といえる唯一のセマンティクスは存在しない．適切な選択はワークロードに依存する．ファイルがどれだけ頻繁に共有されるか，どれだけ頻繁に書き込まれるか，そして一貫した見え方がアプリケーションにとってどれだけ重要かである．こうしたアクセスパターンを知っていれば，設計者は頻繁に起こる場合の最適化（第 1 章）を行える．

DFS は，ワークロードの異なる 2 種類のファイルを扱う．通常のファイルはたいてい一度に 1 つのクライアントだけが使い，変更が他のクライアントから見えるまでに遅れがあっても気づかれることはまれである．ディレクトリははるかに頻繁に共有される—共有ディレクトリでファイルを作成，削除，改名するクライアントはすべてそれを変更する—うえ，ディレクトリの不整合な見え方は有害である．たとえば，2 つのクライアントが同じ名前のファイルをともに作成してしまいかねない．そのため DFS は，両者に異なる方針を適用できる．たとえば，ファイルにはセッションセマンティクス，ディレクトリには UNIX セマンティクスを用いる，あるいは周期的な更新を用いつつ，書き戻しの頻度をファイルについてはディレクトリより低くする，といった具合である．

> **要点**
> ファイル共有セマンティクスは，あるクライアントが別のクライアントの書き込みをいつ観測するかを定める．UNIX セマンティクスはすべての書き込みを直ちに見えるようにするが DFS では高価であり，セッションセマンティクスは close と open で変更を伝播し，周期的な更新は不整合を更新周期で抑える．不変ファイルとトランザクションは不整合な見え方そのものを回避する．選択はワークロードに従い，ファイルとディレクトリを別に扱ってもよい．

## 14.7 レプリケーションとパーティショニング

出典 P4L2，レプリケーションとパーティショニング；Silberschatz ら ch. 19．
▹ replication

複数のマシン上で動作するファイルサービスは，ファイルをそれらのマシンに 2 通りのいずれかの方法で分散させる（図 14.1b と c）．

**定義 14.13** (レプリケーション). **レプリケーション**では，どのサーバマシンも，すべてのファイルの完全な複製，すなわち**レプリカ**を保持する．

レプリケーションは負荷を均す．読み出しはどのレプリカでも処理できるからである．また可用性と耐障害性をもたらす．1 台のマシンが故障しても，残りがすべてのファイルを提供し続けられる．しかし書き込みはより複雑になる．すべての書き込みをすべてのレプリカに対して同期的に行うか—この場合書き込みは遅くなる—，あるいは 1 つのレプリカに対して行って後から他へ伝播するかであり，後者ではしばらくの間レプリカが食い違う．食い違ったレプリカは整合させなければならない．たとえば投票によって，過半数のレプリカが保持する内容を採用する．

▹ partitioning

**定義 14.14** (パーティショニング). **パーティショニング**では，各サーバマシンがファイルの部分集合を保持し，それらの部分集合が合わさってファイルシステムを構成する．

パーティショニングは単一のサーバに比べて可用性を高める．1 台のマシンの故障はそのマシンが保持するファイルにしか影響しないからである．またファイルシステムの規模に対してスケールする．ファイルが増えてもマシンを増やせばよい．1 つのファイルへの書き込みにはそれを保持するマシンだけが関わるので，書き込みは単純なままである．他方で，故障したマシンが保持していたデータは失われ，負荷の均衡をとるのはより難しい．人気のあるファイルが少数のマシンに集中すると，それらがホットスポットになる．表 14.4 に両者を比較する．2 つを組み合わせることもできる．ファイルを分割し，各パーティションを複数のマシンに複製するのである．

大規模なシステムは両者を組み合わせる．GFS と HDFS はファイルをチャンクに分割し（パーティショニング），それぞれのチャンクについて複数のレプリカを保持する（第 16 章）．

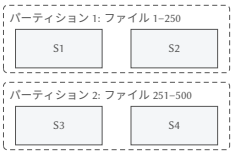



**図 14.5:** 2 つのパーティションを，それぞれ 2 台のマシンに複製した構成．

**例 14.15.** 4 台のサーバマシンがそれぞれ 250 個のファイルを格納できるとする．複製すれば DFS は 250 個のファイルを保持し，1 台の—実際には 3 台までの— マシンの故障でも 1 つも失われない．分割すれば 1000 個のファイルを保持するが，1 台の故障でそのうち 250/1000 = 25 % が失われる．図 14.5 のように組み合わせ，250 個ずつの 2 つのパーティションをそれぞれ 2 台のマシンに複製すると，DFS は 500 個のファイルを保持する．1 台の故障では 1 つも失われず，同じパーティションの 2 台がともに故障したときにのみ 50 % が失われる．

表 14.4: レプリケーションとパーティショニング．

| | レプリケーション | パーティショニング |
|---|---|---|
| 容量 | 1 台のマシンの容量 | 全マシンの合計 |
| 読み出し | どのレプリカでも処理できる；負荷が均される | そのファイルを保持するマシンが処理する；ホットスポットが生じうる |
| 書き込み | すべてのレプリカに対して，同期的にあるいは後から伝播；レプリカを整合させる必要がある | 1 台のマシン上で；単純 |
| マシンの故障 | ファイルは失われない | そのマシンのファイルが失われる |

## 14.8 Network File System

▹ Network File System

1980 年代に Sun Microsystems で開発された **NFS**（Network File System）[54] は，クライアントがリモートのサーバ上にあるファイルにアクセスする，広く使われている DFS である．その構造を図 14.6 に示す．クライアント上のアプリケーションは通常のファイルシステムコールを発行する．VFS は，そのファイルが NFS ファイルシステムに属することを見出し，操作を *NFS* **クライアント**へ渡す．これは，操作をサーバへの遠隔手続き呼び出しに変換するファイルシステム実装である．サーバ側では *NFS* **サーバ**の構成要素が要求を受け取り，サーバ自身の VFS とローカルファイルシステムを通じてサーバ上のファイルに対して操作を行い，結果を返す．結果はアプリケーションまで戻っていく．NFS は Sun RPC とそのデータ表現 XDR（第 13 章）の上に構築されている．

**出典** P4L2，NFS の設計；Sandberg ら [54]；OSTEP ch. 49；Silberschatz ら ch. 15．

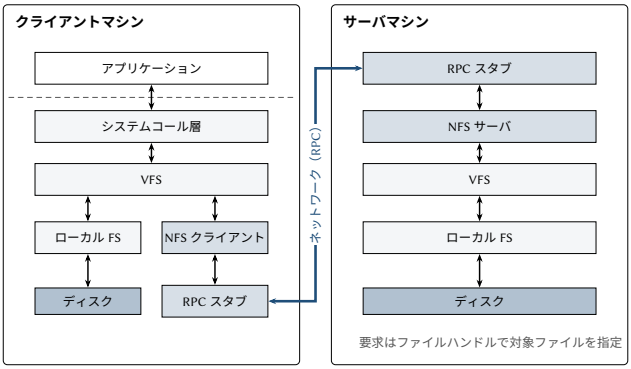



図 14.6: NFS．クライアントの VFS は NFS ファイルに対する操作を NFS クライアントへ渡し，NFS クライアントはそれを RPC 要求として NFS サーバへ送る．サーバは自身の VFS とローカルファイルシステムを通じてそれを実行する．

クライアントがどのリモートファイルに到達できるかは，NFS ファイルシステムがクライアントのディレクトリツリーに取り付けられるときに決まる．サーバは自分のディレクトリのいくつかを**エクスポート**し，クライアントは**マウント**操作でエクスポートされたディレクトリを利用可能にする．マウントはそれをローカルなディレクトリ，すなわち**マウントポイント**に取り付ける．マウントポイント以下のパスはサーバ上のファイルを指すことになる．プロセスがそのようなファイルを open すると，NFS クライアントはそのパスをサーバで解決し，サーバはファイルハンドルを返す．

▹ mount

**定義 14.16** (ファイルハンドル)．**ファイルハンドル**とは，NFS クライアントと NFS サーバが要求の中でファイルを識別するために用いるバイト列である．これを作るのはサーバであり，ファイルのサーバローカルな同一性—ファイルシステムと，その中でのファイル—を符号化する．クライアントはそれを不透明なものとして扱う．

▹ file handle

ハンドルは NFSv2 では 32 B の固定長である．以降の版では可変長になり，NFSv3 では高々 64 B [55]，NFSv4 では 128 B [48] である．

NFS クライアントは，open されたファイルを表す自身のデータ構造にハンドルを保持し，読み出し，書き込み，ファイルの属性の要求といったその後のすべての要求でそれを渡す．サーバは通常，ハンドルにファイルシステムの識別子，ファイルの i ノード番号，そして i ノードが再利用されるたびに変わる**世代番号**を符号化する．ファイルがサーバ上で，ローカルなプロセスによってあるいは別のクライアントによって削除されると，その

▹ generation number

▷ stale file handle

ハンドルを用いた要求は**失効したファイルハンドル**というエラー（ESTALE．パスを open し直せば新しいハンドルが得られる）で失敗する．ハンドルはもはや存在しないファイルを指しており，世代番号は，そのiノードを再利用した新しいファイルをハンドルが黙って指してしまうことを防ぐ．

**例 14.17.** あるクライアントのディレクトリ /home がサーバからマウントされており，マウント操作によってクライアントはエクスポートされたディレクトリのファイルハンドルを得ている．次のプログラムはそれを知らない．コメントは，何もキャッシュされていないときに各呼び出しが引き起こす主要な NFSv3 の要求を示す．[2]

```
char buf[4096];
int fd = open("/home/ana/log.txt", O_RDONLY);
        /* /home のハンドルで "ana" を LOOKUP，
           ana のハンドルで "log.txt" を LOOKUP：
           サーバがファイルハンドルを返す          */
ssize_t n = read(fd, buf, sizeof buf);
        /* READ(ファイルハンドル，オフセット 0，4096) */
close(fd);  /* 読み出し専用：要求は不要 */
```

[2] NFSv3 は実のところ RPC プログラムの一族である．NFS 本体 [55]，パスをエクスポートされたディレクトリのハンドルに変える MOUNT プロトコル，ロックマネージャ，そして再起動したことを互いに伝える状態モニタである．いずれも rpcbind（第 13 章）に登録される．NFSv4 はこれらを 1 つのプロトコルにまとめ，ポート 2049 だけを使う．

## 14.9 NFS のバージョン

**出典** P4L2，NFS のバージョン；RFC 1813 [55]；RFC 7530 [48]；OSTEP ch. 49．

NFS は 1980 年代から使われており，いくつかのバージョンを経てきた．現在使われているのは NFSv3 [55] と NFSv4 [48] である．両者は状態の扱いにおいて根本的に異なる．NFSv3 はステートレスなプロトコルである．ただし実際には，ロックマネージャのような状態を保持する補助プロトコルによって補われている．NFSv4 はステートフルであり，ファイルの open，ロック，そして後述するデリゲーションがプロトコルの一部になっている．

NFSv4.0 は RFC 3530（2003）で定められ，RFC 7530（2015）として改訂された．NFSv4.1（RFC 8881 [50]）と 4.2（RFC 7862）はこれを拡張したものであり，今日実際に使われているのはこれらである．

### 14.9.1 キャッシュ

▷ close-to-open consistency

並行にアクセスされないファイルについて，NFS はセッションに基づくセマンティクスを提供する．これを **close-to-open 一貫性**と呼ぶ．クライアントはファイルを close するときに変更をサーバへ書き戻し，open するときに新しい版がないかを確認する．さらに，ファイルの使用中も，クライアントは周期的に，ファイルが変更されていないかをサーバに尋ねる．既定では，ファイルについては 3 s ごと，ディレクトリについては 30 s ごとである．これらの周期は，クライアントの**属性キャッシュ**のタイムアウトである．属性キャッシュは，大きさや変更時刻といったファイルとディレクトリの属性を保持し，変更時刻が変わっていれば，クライアントはキャッシュしたデータが古いことを知る．ディレクトリの周期が長いのは意図的である．ディレクトリの内容はファイルのデータよりはるかに変更が少ない一方でその中の探索は頻繁なので，確認をまれにすれば多くの要求を節約でき，代価は，別のクライアントが作成あるいは削除した名前を知るのが遅れることだけである．この方針は 14.6 節 の例とは逆だが，それが許容されるのは，ディレクトリを変更する操作はいずれにせよサーバへ行き，そこで直列化されるからである．

セッションに似た側，すなわち close-to-open 一貫性とは，open で再検証し，close で書き戻すことである．nocto マウントオプションは open での確認を止める．nfs(5)．

**例 14.18.** クライアント B が最後にファイル $F$ をサーバと照合したのが $t = 20.0$ s なら，$F$ についての B の情報は 23.0 s まで最新とみなされる．クライアント A が 21.0 s に $F$ を close し，変更を書き戻す．B による 22.5 s の読み出しは B のキャッシュから処理され，古いデータを返す．23.0 s より後の B の最初の読み出しで B はサーバと照合し，変更を見つけて新しいデータを取得する．B が 20.0 s に照合したディレクトリに A が 21.0 s に作成したファイルは，50.0 s まで B から見えないままでありうる．

Linux では，この周期はマウントオプション acregmin と acdirmin である．既定でいずれも 60 s の acregmax と acdirmax が上限を与える．

NFSv4 はこうした確認を避けられる．サーバはクライアントに，ファイルの**デリゲーション**を一定期間与えてよい．デリゲーションが続く間，クライアントはファイルを open し，更新の有無をサーバに確認することなくキャッシュしたデータを使う．そしてサーバは，別のクライアントがそのファイルに対してデリゲーションと衝突するアクセスを必要としたときにデリゲーションを回収する．

▷ delegation

**注 14.19.** このように NFS は，セッションセマンティクスと周期的な再検証を組み合わせている．周期的な確認によって，まだ open しているセッションの中に他のクライアントの変更が持ち込まれることがある．純粋なセッションセマンティクスではこれは起こらない．また 2 回の確認の間に，クライアントはサーバ上ですでに変更されたデータを読むことがある．したがって UNIX セマンティクスは提供されない．

### 14.9.2 ロック

NFSv4 のロックはリースに基づく．サーバはロックを限られた期間だけ与え，クライアントはこのリースが満了する前に，ロックを解放するかリースを更新しなければならない．クライアントが故障したり到達不能になったりすれば，そのリースは期限切れとなり，サーバは故障したクライアントをいつまでも待つことなく，他のクライアントにロックを与えられる（図 14.7）．NFSv4 はさらに**共有予約**を支える．クライアントはファイルを open するとき，自分が必要とするアクセス–読み出し，書き込み，あるいはその両方–と，他のクライアントに拒否するアクセスを宣言する．読み出しは共有しつつ他への書き込みを拒否する予約は，リーダ・ライタロック（第 10 章）のように働く．クライアントは，ファイルを close することなく，予約を読み出しから書き込みへ昇格させたり，逆に降格させたりできる．

NFSv3 のロックマネージャは独立したプロトコルであり，Network Lock Manager（NLM）と呼ばれる．

**注意** リースに時刻の同期は要らない．必要なのは，1 回のリースの間に大きくずれない時計だけである．リースは期間であり，クライアントはそれを要求を送った時点から計るので，サーバより先に権利を手放す．

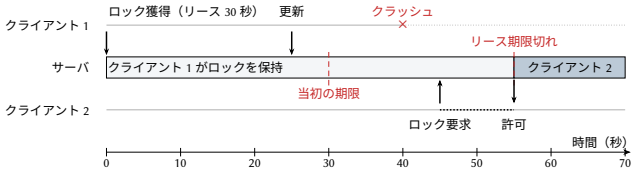



**図 14.7:** リースに基づくロック．クライアント 1 は 25 s にリースを更新し，40 s にクラッシュする．リースが 55 s に期限切れになると，サーバはクライアント 2 にロックを与える．

**例 14.20.** あるサーバが 30 s のリースを与える．クライアント 1 は $t = 0$ にロックを獲得し，25 s にリースを更新して 55 s まで延長するが，40 s にクラッシュする．クライアント 2 は 45 s にロックを要求して待つ．55 s にクライアント 1 のリースは更新されないまま期限切れとなり，サーバはクライアント 2 にロックを与える（図 14.7）．仮にクライアント 1 がクラッシュしたのではなく，単にサーバとの連絡を失っただけだったとしても，自分の時計で 55 s の少し前にはロックの使用をやめなければならない．ロックが別のクライアントに与えられたかどうかを知りえないからである．

期間はプロトコルが定めるものではない．1 つのサーバが与えるリースはすべて同じ長さであり，クライアントはそれをサーバの `lease_time` 属性から読む [48]．Linux の NFS サーバの既定のリースは 90 s である（`/proc/fs/nfsd/nfsv4leasetime`）．ここでの 30 s は例のための値である．

> **要点**
> NFS のクライアントは VFS，NFS クライアント，RPC を通じてリモートファイルに到達し，サーバが生成するファイルハンドルでファイルを識別する．NFSv3 はステートレス，NFSv4 はステートフルである．NFS はセッションに基づくキャッシュと周期的な確認–既定でファイルは 3 s ごと，ディレクトリは 30 s ごと–を組み合わせる．NFSv4 はこれにデリゲーション，リースに基づくロック，共有予約を加える．

## 14.10 Sprite 分散ファイルシステム

分散オペレーティングシステム **Sprite** は 1980 年代にカリフォルニア大学バークレー校で開発された．そのファイルシステムは，ネットワーク上のワークステーションがサーバに格納されたファイルを共有できるようにするものである．Nelson，Welch，Ousterhout [53] は，そのファイルシステムがどのようにデータをキャッシュするかを述べている．この論文の要点はその設計過程にある．設計上の各決定が，ファイルが実際にどうアクセスされるかの分析から導かれ，それによって正当化されている．表 14.5 に観測と，そこから導かれた結論を示す．

▷ Sprite

**出典** P4L2，Sprite 分散ファイルシステム，Sprite DFS のアクセスパターンの分析，Sprite DFS の分析から設計へ；Nelson ら [53]．

これらの観測から Sprite の設計が導かれる．

- クライアントはファイルのブロックをキャッシュし，変更されたブロックを 30 秒の遅延をもってサーバへ書き戻す．詳細は後述する．
- 別のクライアントが書き込んでいたファイルをあるクライアントが open すると，サーバはまずそのクライアントからダーティなブロックを回収する．
- すべての open はサーバへ行き，ディレクトリはクライアントにキャッシュされない．
- ファイルが並行に書き込まれているときは，そのファイルのキャッシュを無効にし，そのファイルに対する操作はすべてサーバへ行く．

Linux もページキャッシュに対して同じことをする．フラッシャスレッドが 5 s ごとに起き，30 s を超えてダーティなままのデータを書き戻す（`dirty_expire_centisecs`）．

最初の項目が書き込み方針を定める．変更されたブロックをクライアントのキャッシュに保持し，書き込みのたびでも close の時点でもなく，遅延の後にはじめて書き戻すことを**遅延書き込み**，あるいは遅延書き戻しと呼ぶ．Sprite では，クライアントは 30 秒ごとに，直前の 30 秒間に変更されなかったダーティなブロックを書き戻す．したがって，まだ変更され続けているブロックが繰り返し転送されることはなく，書き込みから 30 秒以

▷ delayed write

古典的な UNIX も同じ周期で動いていた．`update` デーモンが 30 s ごとに `sync` を実行していた．

**表 14.5:** アクセスパターンと，それが Sprite の設計にもたらした帰結．測定は時分割の 4.2BSD システムで行われた（Nelson らが引用する BSD のトレース研究 [56]）．

| 観測 | 帰結 |
|---|---|
| 全ファイルアクセスの 33 % が書き込みである | アクセスの大半を占める読み出しについてはキャッシュが引き合う．しかし UNIX セマンティクスが要求するように，すべての書き込みを直ちに伝播するのは高価すぎる． |
| ファイルの 75 % は 0.5 s 未満，90 % は 10 s 未満しか open されない | セッションセマンティクスでもなおオーバーヘッドが大きすぎる．これほど短いセッションでは，open と close のたびの更新がきわめて頻繁にサーバへ届くことになる． |
| 新しいデータの 20–30 % は 30 s 以内に，50 % は 5 min 以内に削除される | close の時点でデータを書き戻す必要はない．データの多くは，他のどのクライアントもそれを必要とする前に削除される． |
| ファイルへの並行アクセスはまれである | 並行アクセスは最適化する必要はないが，支えなければならない． |

内に削除されたデータはまったく転送されない．ブロックがサーバに届くのは，その最後の変更から 30 秒後から 60 秒後の間である．

**注意** 遅延書き込みは通信量と引き換えに耐久性を手放す．クライアントがクラッシュすると，まだ書き戻されていないブロックはすべて失われ，最大で 1 分ぶんの作業が失われる．Sprite がこれを受け入れるのは，遅延によって書き込みを速く保ちつつ通信量も減らせるからである．サーバはディスクへ書く前にさらに 30 s to 60 s を加える．

**例 14.21.** あるクライアントが $t = 0, 30, 60, 90, \ldots$ 秒に書き戻すとする．$t$ は最初のフラッシュからの秒数であり，ブロックは，最後の変更から少なくとも 30 s 経った最初のフラッシュで書き戻される．ブロック $b_1$ は 12 s に変更される．30 s の時点では最後の変更から 18 s しか経っていないので，書き戻されるのは 60 s の時点である．ブロック $b_2$ は 12 s と，さらに 50 s に変更される．60 s の時点では最後の変更から 10 s しか経っていないので，書き戻されるのは 90 s の時点である．ブロック $b_3$ は，20 s に書かれ 45 s に削除される一時ファイルに属しており，サーバへ送られることはない．

Sprite が提供するセマンティクスは，ファイルがどのように共有されるかに依存する．

**定義 14.22 (逐次書き込み共有と並行書き込み共有).** あるクライアントがファイルを変更して close し，別のクライアントが後からそれを open するとき，そのファイルは**逐次書き込み共有**の下にある．すなわち，複数のクライアントが次々にファイルを使う．ファイルが複数のクライアントで同時に open されており，そのうち少なくとも 1 つが書き込みのために open しているとき，そのファイルは**並行書き込み共有**の下にある．

▷ sequential write sharing

▷ concurrent write sharing

逐次書き込み共有の下では Sprite はファイルをキャッシュし，それを open するどのクライアントも，直前の書き手が書いたデータを見る．並行書き込み共有の下では Sprite はファイルをキャッシュせず，サーバがすべての読み出しと書き込みを順に実行する．これは UNIX セマンティクスを与える．

**注意** ここでの逐次はクライアントを指すのであって，ファイル内のアクセスの仕方を指すのではない．ブロックを順に読むことではなく，クライアントが次々にファイルを使うことである．

## 14.11 Sprite におけるファイルアクセス操作

**出典** P4L2，Sprite におけるファイルアクセス操作；Nelson ら [53]．

以下では，ファイルを複数のクライアントが読み，1 つ以上のクライアントが書く．すべての open はサーバを経由し，ファイルがキャッシュ可能である限りすべてのクライアントがブロックをキャッシュし，書き手は，遅延書き込みが要求するとおり，変更した各ブロックについてタイムスタンプを保持する．クライアントとサーバは，ファイルごとに次の状態を保持する（図 14.8）．

**クライアント** そのファイルについてキャッシュが有効かどうか，キャッシュしているブロック，ダーティな各ブロックの最終変更時刻，そしてキャッシュしているデータの版番号．

**サーバ** そのファイルを読み出しのために open しているクライアントと書き込みのために open しているクライアント，最後にそのファイルに書き込んだクライアント，現在の版番号，そしてファイルがキャッシュ可能かどうか．

**逐次書き込み共有．** クライアントがファイルを書き込みのために open するたびに，サーバは版番号を増やす．ファイルを open したクライアントは，サーバが返した版番号を自分のキャッシュしたブロックの版番号と比較し，古ければそのブロックを捨てる．遅延書き込みのため，最後にファイルに書き込んだクライアントは，close した後もなおそのファイルのダーティなブロックを保持していることがある．そこでサーバは，別のクライアントがファイルを open する前にそのクライアントにそれらを書き戻すよう求め，新し

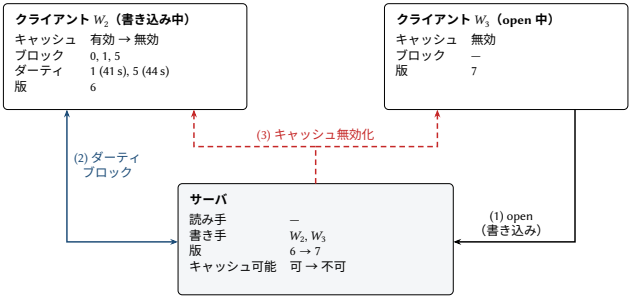



**図 14.8:** Sprite のファイルごとの状態．$W_2$ が書き込みのために open しているファイルを $W_3$ が open すると (1)，サーバは $W_2$ のダーティなブロックを回収し (2)，両クライアントのキャッシュを無効にする (3)．

いクライアントが最新のデータを見られるようにする．キャッシュは有効なままである．

**並行書き込み共有．** 別のクライアントが open しているファイルをあるクライアントが書き込みのために open したとき，あるいは別のクライアントが書き込みのために open しているファイルを読み出しのために open したとき，サーバは再び書き手のダーティなブロックを回収し，ファイルをキャッシュ不可と印し，それを open しているすべてのクライアントに通知する．それ以降，どのクライアントもそのファイルをキャッシュせず，すべての読み出しと書き込みがサーバへ行く．並行書き込み共有が終わると，ファイルはその後の open に対して再びキャッシュ可能になる．すでにそれを open しているクライアントは，キャッシュしないまま使い続ける．

**例 14.23.** 表 14.6 は，はじめ版 4 にあるファイル $F$ について，クライアント $R_1$，$W_1$，$W_2$，$W_3$ が open と close を行う間のサーバの状態を追ったものである．段階 5 では $W_1$ は $F$ を close しているがなおダーティなブロックを保持しており，サーバは $W_2$ を進める前にそれを回収する．共有は逐次的であり，$W_2$ は $F$ をキャッシュしてよい．段階 6 では $W_2$ がまだ書き込みのために open している $F$ を $W_3$ が open し（図 14.8），キャッシュが無効にされる．段階 7 では $R_1$ が，キャッシュしている版 4 のブロックが古いことを見出し，それを捨ててサーバから読む．

**ヒント** 逐次共有でキャッシュが生き残るのは，サーバが先にブロックを回収できる「最後の書き手」が必ずいるからである．並行共有にはそれがいないので，サーバがすべてのアクセスを自ら処理するほかない．

**表 14.6:** Sprite におけるファイル $F$ のサーバ状態．版とキャッシュ可否は各段階の後の値を示す．

| | 事象 | サーバの動作 | open 中 | 版 | キャッシュ |
|---|---|---|---|---|---|
| 1 | $R_1$ が読み出しで open | — | $R_1$ | 4 | 可 |
| 2 | $R_1$ が close | — | — | 4 | 可 |
| 3 | $W_1$ が書き込みで open | 版を増やす | $W_1$ | 5 | 可 |
| 4 | $W_1$ が書き込み，close | —（ダーティなブロックは $W_1$ に残る） | — | 5 | 可 |
| 5 | $W_2$ が書き込みで open | $W_1$ のダーティなブロックを回収；版を増やす | $W_2$ | 6 | 可 |
| 6 | $W_3$ が書き込みで open | $W_2$ のダーティなブロックを回収；版を増やす；キャッシュを無効化 | $W_2$, $W_3$ | 7 | 不可 |
| 7 | $R_1$ が読み出しで open | $R_1$ にキャッシュしないよう伝える | $R_1$, $W_2$, $W_3$ | 7 | 不可 |
| 8 | $W_2$, $W_3$, $R_1$ が close | キャッシュを有効化 | — | 7 | 可 |

**要点 — 章のまとめ**

分散ファイルシステムは，ネットワークで結ばれたマシン上のファイルを，通常のファイルシステムインタフェースを通じて提供する．その構成は，単一のサーバ，複製あるいは分割されたサーバ，そしてピアのいずれかである．実用的な設計はアップロード・ダウンロードモデルと遠隔アクセスモデルの間に位置する．クライアントはブロックをキャッシュし，ステートフルサーバと頻繁にやり取りし，サーバはキャッシュされた複製の整合を保たなければならない．ファイル共有セマンティクス—UNIX セマンティクス，セッションセマンティクス，リースを伴う周期的な更新，不変ファイル，トランザクション—がクライアントの観測するものを定め，ファイルとディレクトリで異なってよい．レプリケーションは書き込みが複雑になる代価と引き換えに可用性と負荷の均衡を高め，パーティショニングはファイルシステムの規模に対してスケールする．NFS は RPC の上でファイルハンドルを用い，ステートレスな NFSv3 かステートフルな NFSv4 で，セッションに基づくキャッシュと周期的な確認，リースに基づくロックを提供する．Sprite は測定されたアクセスパターンから設計を導いた．すなわち，30 秒の遅延を伴う遅延書き込み，逐次書き込み共有の下でのキャッシュ，並行書き込み共有の下でのキャッシュの停止である．

第 15 章

# 分散共有メモリ

| | |
|---|---|
| 講義動画 | P4L3 |
| 教科書 | OSTEP [1] ch. 48；Protić ら [57]；Nitzberg と Lo [58]；Tanenbaum と Bos [6] ch. 8；Tanenbaum と van Steen [59] ch. 7 |
| キーワード | 分散共有メモリ，RDMA，共有の粒度，偽共有，アクセスアルゴリズム，マイグレーション，レプリケーション，一貫性管理，ページフォールト，一貫性モデル，厳密一貫性，逐次一貫性，因果一貫性，弱一貫性 |

第 14 章では分散ファイルシステムを扱った．そこではサーバがファイルを所有し，その一部をキャッシュするクライアントにそれらを提供する．しかし多くの分散アプリケーションでは，どのノードも状態の一部を所有し，かつそれを他のノードへ提供する．この章では，そのように分散した状態がどう管理されるかの例として，分散共有メモリを調べる．分散共有メモリは，異なるマシン上のプロセスが，あたかも単一の共有メモリ型マルチプロセッサ上で動作しているかのようにメモリを共有できるようにするものである．DSM システムの設計上の選択–粒度，アクセスパターン，マイグレーションとレプリケーションの選択，そして一貫性管理–と，DSM がページをどう探し出し MMU を使ってアクセスをどう捕捉するか，そして，プロセスがメモリを共有したときに何を観測するかを定める一貫性モデルを扱う．

## 15.1 ピア分散アプリケーション

**出典** P4L3，レッスンの概要，分散ファイルシステムの復習，ピア分散アプリケーション；OSTEP [1] ch. 48；Tanenbaum と van Steen [59] ch. 2.

第 14 章の分散ファイルシステムでは，マシンの役割が明確に分かれている．サーバは状態，すなわちファイルを所有して管理し，サービス，すなわちファイルアクセスを提供する．クライアントはサーバへ要求を送る．クライアントは状態の一部をキャッシュし，それによって自身のアクセスのレイテンシが下がり，またサーバはより多くのクライアントを支えられるようになるが，状態そのものはサーバに属する．

多くの分散アプリケーションには，そのような分担が存在しない．ビッグデータ解析，ウェブ検索，コンテンツ共有では，アプリケーションの状態はその全ノードに分散している．各ノードは状態の一部を所有し，その部分に対するサービスを他のノードへ提供すると同時に，他のノードが所有する状態を利用する．したがってどのノードもクライアントでありサーバでもあり，すべてのノードがピアである．このようなアプリケーションを**ピア分散アプリケーション**と呼ぶ．ここで「ピア」という語は緩い意味で使われている．ピアツーピアシステム（第 14 章）では，システム全体の制御と管理までもがすべてのピアによって行われるのに対し，ピア分散アプリケーションは，どのノードが状態のどの部分を保持しているかを追跡するといった管理作業を，特定のノードに割り当ててよい．

第 16 章では，こうしたアプリケーションをデータセンタで扱う．そこではフレームワークが，各計算を，その入力をすでに保持しているノードへ送る．

ピアにまたがって広がる状態の管理は 3 つの問題を生じる．これらはこの章を通じて繰り返し現れる．

**配置** 状態の各部分がどのノードに存在するか．最も頻繁にアクセスするプロセスのノードに状態が存在するとき，アクセスは最も速い．

**マイグレーション** 他のノードに存在する状態を，移動によって，あるいは複製によって，それにアクセスするプロセスのノードへどのように，いつ持ってくるか．

**共有の規則** 同じ状態に対する並行な操作がどの順序で効果を持つか，そして各ノードが他のノードの行った更新をいつ観測するかを定める規則．

最初の 2 つは 1 台のマシンの中にも現れる．NUMA 対応スケジューリング（第 7 章）は計算をそのメモリのもとへ動かす．

状態がメモリであるピア分散アプリケーションである分散共有メモリが，この章でこれらの問題を調べるための例である．

**注意** この章のマイグレーションはノード間でメモリを移動する．プロセス全体を別のマシンへ移す第 8 章のプロセスマイグレーションではない．

## 15.2 分散共有メモリ

**出典** P4L3，分散共有メモリ；Protić ら [57]；Nitzberg と Lo [58]；OSTEP ch. 48；Tanenbaum と Bos ch. 8.

共有メモリ型マルチプロセッサ（第 7 章）は，異なる CPU 上のスレッドが，そのすべてがアクセスするメモリを通じて協調することを可能にするが，そのメモリは 1 台のマシンが保持できる量に限られる．分散共有メモリは，このモデルを多数のマシンへ拡張する．

**定義 15.1** (分散共有メモリ). ネットワークのノードにまたがってメモリを管理し，その上で動作するアプリケーションに単一の共有メモリマシン上で動作しているかのような錯覚を与えるサービスを**分散共有メモリ**（DSM）と呼ぶ．どのノードのプロセスも 1 つの共有アドレス空間を読み書きし，その内容は異なるノードの物理メモリに存在する．

▷ distributed shared memory

他のあらゆるピア分散アプリケーションと同様に，DSM システムの各ノードは次のことを行う（図 15.1）．

- 状態，すなわちシステムの物理メモリの一部–自身のメモリのうち DSM に提供する部分–を所有する．
- その状態に対するサービス，すなわち自身のメモリに対する読み出しと書き込みを提供する．これらはどのノードのプロセスが発行してもよい．
- 他のノードとの一貫性プロトコルに参加する．これは，複数のノードが同じメモリにアクセスしたときにも共有メモリへのアクセスが意味を持ち続けること–たとえば読み出しが最も新しい書き込みの値を返すこと–を保証する．

**注意** これは第 9 章の共有メモリ IPC ではない．そちらでは 1 台のマシンのメモリをプロセスどうしが共有し，その見え方はハードウェアが一貫させる．

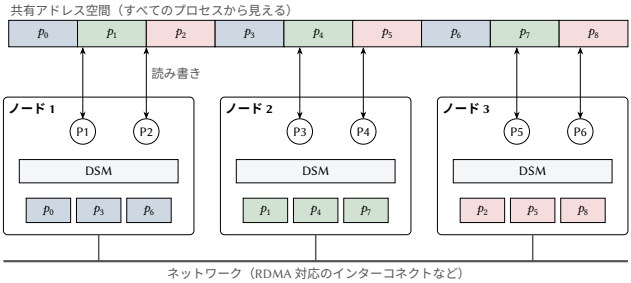



**図 15.1:** 分散共有メモリ．共有アドレス空間の各ページ $p_i$ は，いずれかのノードが提供したメモリに存在する．各ノードの DSM 層は，ローカルなページへのアクセスを処理し，リモートのページをネットワーク越しに取得する．

ローカルメモリへのアクセスは 100 ns 程度，データセンタのイーサネット越しの往復は数十マイクロ秒であるのに対し，400 Gbit $s^{-1}$ の NDR InfiniBand リンク上の片側 RDMA 読み出し（往復）は約 2 µs である（perftest `ib_read_lat`）．ベンダは 1 マイクロ秒未満の片道レイテンシを公表している．

DSM は，アプリケーションが 1 台のマシンのメモリの限界を超えて規模を拡大することを可能にする．1 台のマシンが保持できる以上のメモリを必要とするアプリケーションは，それを他のマシンから得る．また DSM は共有メモリをより低い費用で提供する．非常に大きなメモリを持つ 1 台のマシンは，中程度のメモリを持つ複数のコモディティマシンよりも不釣り合いに高価だからである．その代価はレイテンシである．他のノードに存在するメモリへのアクセスはネットワークを越えるので，ローカルメモリへのアクセスより桁違いに遅い．コモディティのインターコネクトは，リモートダイレクトメモリアクセスを提供することでこの代価を下げる．

▷ remote direct memory access

**定義 15.2 (リモートダイレクトメモリアクセス).** **RDMA** では，ネットワークインタフェースが，自身のマシンのメモリとリモートマシンのメモリとの間でデータを直接転送する．その際，リモートマシンの CPU もオペレーティングシステムも転送に関与しない．

RDMA は DMA（第 11 章）をネットワーク越しに拡張したものである．通常は OS バイパス（第 11 章）と組み合わされ，どちらのマシンのカーネルもデータ経路に乗らないようにする．

InfiniBand は RDMA をネイティブに提供する．RoCE と iWARP はイーサネット上で運ばれる RDMA プロトコルである．

## 15.3 ハードウェア DSM とソフトウェア DSM

DSM はハードウェアで提供することも，ソフトウェアで提供することも，両者の組み合わせで提供することもできる．

**出典** P4L3，ハードウェア DSM 対ソフトウェア DSM；Protić ら；Nitzberg と Lo [58]．

**ハードウェア支援 DSM** 専用のインターコネクトに依存する．各ノードのオペレーティングシステムには，自ノードのメモリよりはるかに大きい，複数のノードにまたがる物理メモリを管理しているかのように見える．他のノードに存在するアドレスへのアクセスは，ネットワークインタフェースカード（NIC）によってそのノードへのメッセージに変換され，NIC は，リモートノードの CPU を関与させることなく，リモートメモリに対するアトミック操作を含めて共有メモリの管理のあらゆる側面に参加する．このようなハードウェアは高価であり，高性能計算のように性能が最優先されるハイエンドのシステムで用いられる．

Sequent の NUMA-Q の IQ-Link が実装した Scalable Coherent Interface（IEEE 1596-1992）はそうしたインターコネクトであり，ノードのメモリをハードウェアで一貫させる．

**ソフトウェア支援 DSM** すべてをソフトウェアで行う．メモリを保持するノードの特定，その取得，アクセスの検出，そして複製の一貫性の維持である．そのソフトウェアはオペレーティングシステムの一部か，プログラミング言語のランタイムの一部である．前者はページを，後者はオブジェクトを共有する（15.4 節）．

**ハイブリッド DSM** 作業を両者で分担する．ハードウェア，とくに MMU は，あらゆるアクセスで起こらなければならないこと，すなわちアドレスの変換と，現在のマッピングが許可しないアクセスに対する例外の発生を担う．ソフトウェアは，システ

ム全体の知識を要する判断–たとえばどのページをプリフェッチするか，すなわちアクセスされる前に他のノードから取得するか–を担う．

## 15.4 共有の粒度

DSM の最初の設計上の決定は共有の単位である．すなわち，DSM がノード間で転送し，複製し，全体として一貫させるメモリの単位である．共有メモリ型マルチプロセッサでは，ハードウェアが CPU キャッシュを，数十バイトの単位であるキャッシュラインの粒度で一貫させる（第 10 章）．これが可能なのは，ハードウェアがナノ秒のうちにキャッシュ間を調整するからである．DSM では調整のたびにネットワーク越しのメッセージが必要になるので，このような細かい粒度は費用がかかりすぎる．候補は次のものである．

出典 P4L3，DSM の設計：共有の粒度；Protić ら；Nitzberg と Lo；Tanenbaum と Bos ch. 8.

**変数** 個々の変数を共有するのも細かすぎる．変数は数バイトしか占めないので，アクセスごとの調整に必要なメッセージが，データそのものの価値をはるかに超える代価となる．

**ページ** ページ–普通は 4 KiB–は，オペレーティングシステムがすでに管理している単位である．ページに基づく DSM は，オペレーティングシステムが MMU と協調して実装する（15.9 節）．アプリケーションからは透過であり，アプリケーションが書かれた言語にも依存しない．

2 MiB のラージページ（第 8 章）なら一度に 512 個の基本ページを覆う．転送は少なく大きくなるが，偽共有もそれだけ増える．

**オブジェクト** 言語ランタイムは，プログラムが生成するオブジェクトを知っている．ランタイムで実装されるオブジェクトに基づく DSM は，プログラムのデータ構造に対応する単位を共有し，オペレーティングシステムの変更を必要としないが，そのランタイムを使うプログラムだけがその恩恵を受ける．

粒度が粗いほど，転送や調整 1 回あたりの代価をより多くのデータで償却できるが，同時に無関係なデータが同じ単位に置かれることにもなる．

**定義 15.3 (偽共有).** 共有されていないデータ–それぞれ別のノードのプロセスがアクセスするデータ–が同じ共有の単位に存在し，その結果，DSM がそれらのデータが共有されているかのようにその単位へのアクセスを調整してしまう状況を**偽共有**と呼ぶ．

▷ false sharing

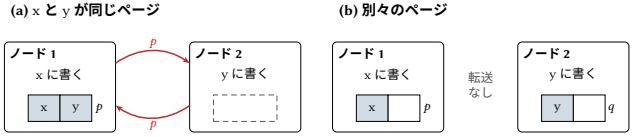



**図 15.2:** 偽共有．(a) ノード 1 が x に書き，ノード 2 が y に書く．両者は同じページ $p$ にあるので，ページが往復する．(b) 別々のページ $p$ と $q$ にあれば，どちらのアクセスもローカルに留まる．

**例 15.4.** 変数 x と y が同じページ $p$ にあるとする．ノード 1 のプロセスは x だけに書き，ノード 2 のプロセスは y だけに書く．各プロセスは 10 ms ごとに自分の変数に書き，2 つのプロセスの書き込みは交互に起こる．両プロセスはデータを何も共有していないが，ページに基づく DSM からは 2 つのノードがページ $p$ に書いているように見える（図 15.2a）．どの書き込みの時点でも $p$ の最新の内容は他方のノードにあるので，$p$ は 5 ms ごと，毎秒 200 回転送され，200 × 4 KiB = 800 KiB のネットワークトラフィックが毎秒生じる．転送に 0.5 ms かかるとすれば，各プロセスは 10 ms ごとに 0.5 ms，自分の時間の 5 % を待つことになる．x と y を別々のページに置けば（図 15.2b），最初のアクセスの後はページは転送されない．

**注意** 偽共有は単一のマシンでも起こる．2 つの変数が同じキャッシュラインを共有し，一貫性プロトコル（第 10 章）がそのラインを往復させる場合である．4 KiB のページは 64 B のラインを 64 本含み，その移動 1 回はキャッシュミスではなくネットワーク転送である．

偽共有は，異なるノードがアクセスするデータを異なる単位に配置することで避けられる．アクセスパターンが分かっていれば，プログラマ，コンパイラ，あるいはメモリアロケータがそれを行える．

## 15.5 アクセスアルゴリズム

DSM が何を提供しなければならないかは，その上で動作するアプリケーションのアクセスアルゴリズムに依存する．アクセスアルゴリズムは，同じメモリの単位を同時にいくつのノードが読み書きするかを表す．次の 3 つが区別される．

出典 P4L3，DSM の設計：アクセスアルゴリズム；Protić ら；Nitzberg と Lo．

**SRSW 単一リーダ単一ライタ**（SRSW）アクセスでは，ある時点で 1 つのノードだけが単位を読み書きする．DSM の主な仕事は，他のノードに存在するメモリをアクセス

▷ single reader/single writer

可能にすることである．2 つのノードが同時に 1 つの単位を使うことはないので，調整すべき並行アクセスは存在しない．

▷ multiple readers/single writer

**MRSW 複数リーダ単一ライタ**（MRSW）アクセスでは，複数のノードが同時に単位を読んでよいが，書くノードは 1 つだけである．DSM は，あらゆる読み出しが最も新しい書き込みの値を返すことを保証しなければならない．

▷ multiple readers/multiple writers

**MRMW 複数リーダ複数ライタ**（MRMW）アクセスでは，複数のノードが単位を読みも書きもしてよい．DSM はさらに，すべてのノードが同じ順序で書き込みを観測し，読み出しが最も新しい書き込みを返すように，書き込みを正しく順序づけなければならない．

**注意** アクセスアルゴリズムはアプリケーションが何を行うかを述べるものであって，DSM が課す規律ではない．MRMW の下で DSM がすべきことは，並行なライタに対処することであって，それを禁じることではない．

## 15.6 マイグレーションとレプリケーション

DSM の主要な性能指標はアクセスレイテンシ，すなわち共有メモリの読み出しや書き込みにかかる時間である．リモートアクセスはローカルアクセスより桁違いに遅いので，リモートアクセスの割合がわずかであっても平均は急激に上がる．1 万回に 1 回のリモートアクセスですでに平均は 5 割増しになり，100 回に 1 回では平均は約 50 倍になる．

**出典** P4L3，DSM の設計：マイグレーション対レプリケーション；Protić ら；Nitzberg と Lo．

**例 15.5.** ローカルメモリへのアクセスに 100 ns かかり，まず他のノードから取得しなければならないページへのアクセスに 0.5 ms かかるとする．アクセスのうち割合 $f$ がリモートであるとき，平均アクセスレイテンシは次式で与えられる．

$$\bar{t} = (1 - f) \times 100\,\text{ns} + f \times 0.5\,\text{ms}. \tag{15.1}$$

例 15.5 の 0.5 ms は控えめな値である．4 KiB のページを回線へ送り出す時間は 100 Gbit s$^{-1}$ では 0.33 µs にすぎない．フォールト，要求，プロトコルのオーバーヘッドを加えても，そうしたインターコネクト越しの取得は数マイクロ秒である．

$f = 10^{-4}$ では $\bar{t} \approx 100\,\text{ns} + 50\,\text{ns} = 150\,\text{ns}$ となり，ローカルメモリのレイテンシの 1.5 倍である．$f = 10^{-2}$ では $\bar{t} = 99\,\text{ns} + 5000\,\text{ns} \approx 5.1\,\mu\text{s}$ となり，約 51 倍である．

したがって低いレイテンシは局所性を要求する．プロセスがアクセスするメモリは，そのプロセス自身のノードに存在すべきである．メモリを，それにアクセスするプロセスのもとへ持ってくる技法は 2 つある（図 15.3）．

**マイグレーション** DSM は各単位のコピーを 1 つだけ保ち，それをアクセスするノードへ移動する．コピーが 1 つしかないので，一貫させるべきものは何もない．マイグレーションは SRSW アクセスに適する．そこでは，1 つのノードがある期間その単位を使い続けた後に，別のノードがそれを必要とする．マイグレーションは，異なるノードがアクセスするたびに単位を転送するので，ノードが短い間隔で交代する場合には代価が高く，また複数のノードに同時にサービスすることもできない．

**レプリケーション** DSM は単位を複数のノードにコピーし，各ノードは自分のローカルな複製にアクセスする．キャッシングはレプリケーションの一形態である．レプリケーションはより一般的な技法であり，MRSW アクセスや MRMW アクセスが要求するように，複数のノードが同時に単位を読み，あるいは書くことさえ可能にする．その代価は一貫性管理（15.7 節）である．1 つの複製が書かれたとき，他の複製は無効化されるか更新されなければならない．このオーバーヘッドは複製の数とともに増え，DSM は各単位の複製の数を制限することでそれを抑えられる．

**ヒント** メッセージを数えよ．複製が $n$ 個あれば，書き込み 1 回に $n-1$ 回の無効化がかかり，無効化された各ノードでの次のアクセスに取得がかかる．レプリケーションは，それによってローカルで済む読み出しの数がこれらを上回る間は引き合う．

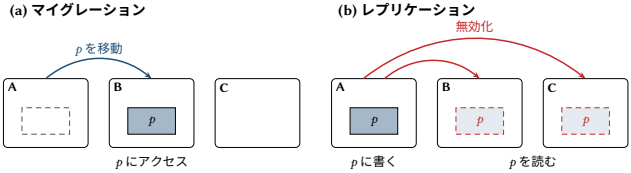



**図 15.3:** ページ $p$ を，それにアクセスするノードのもとへ持ってくる．(a) マイグレーションは唯一のコピーを移動する．(b) レプリケーションは複数のノードに複製を保ち，それらは $p$ をローカルに読む．所有者 A による書き込みは B と C の複製を無効化する（破線）．

**例 15.6.** ページ $p$ は初めノード A に存在し，ノード A，B，C が表 15.1 の順序でそれにアクセスする．ページの転送には 0.5 ms，無効化メッセージには 0.05 ms かかる．マイグレーションは，$p$ を保持していないノードがアクセスするたびに $p$ を移動する．7 回の転送，3.5 ms である．レプリケーションでは，B と C は複製を一度取得し，ステップ 6 の A の書き込みがそれらを無効化するまではローカルに読む．4 回の転送と 2 回の無効化，2.1 ms である．8 回のアクセスをすべて A が行っていたなら，どちらの技法も何も転送しなかっただろう．逆に A と B が交互に書いていたなら，書き込みのたびに他方のノード

無効化はページの識別子—数十バイト—しか運ばないので，その代価はデータの代価ではなくメッセージのやり取りの代価である．

の複製が無効化され，次のアクセスのたびにそれを取得し直すことになり，レプリケーションはマイグレーションと同じ回数の転送に加えて無効化の代価を払うことになる．

**表 15.1:** マイグレーションの下と，書き込み時に無効化するレプリケーションの下での，ページ $p$ へのアクセス．

| | | マイグレーション | | レプリケーション | |
|---|---|---|---|---|---|
| ステップ | アクセス | $p$ の位置 | 転送 | 複製の位置 | メッセージ |
| 0 | （初期状態） | A | — | A | — |
| 1 | A が書く | A | — | A | — |
| 2 | B が読む | B | A→B | A, B | A から取得 |
| 3 | C が読む | C | B→C | A, B, C | A から取得 |
| 4 | B が読む | B | C→B | A, B, C | — |
| 5 | C が読む | C | B→C | A, B, C | — |
| 6 | A が書く | A | C→A | A | B, C を無効化 |
| 7 | B が読む | B | A→B | A, B | A から取得 |
| 8 | C が読む | C | B→C | A, B, C | A から取得 |
| 合計 | | | 7 回の転送 | | 4 回の取得，2 回の無効化 |

どちらの技法もあらゆるワークロードに最良というわけではない．マイグレーションは，1 つの単位が一度に 1 つのノードによって長い期間使われるときに適する．レプリケーションは，複数のノードが同時に単位にアクセスするときに必要であり，読み出しが支配的なときに引き合う．複製された単位が頻繁に書かれるときは，書き込みのたびに起こる一貫性管理が，複製の局所性によって節約される分より高くつきうる．

書き込み時に無効化するレプリケーションは，第 10 章の書き込み無効化戦略をソフトウェアで行うものである．キャッシュラインの代わりにページを，バストランザクションの代わりにメッセージを用いる．

**要点**
DSM の性能はアクセスレイテンシによって決まり，低いレイテンシは局所性を要求する．マイグレーションは唯一のコピーを移動し，SRSW アクセスに適する．レプリケーションは複数のコピーを保ち，同時アクセスを支えるが，その代価として，複製の数と書き込みの頻度とともに増える一貫性管理を払う．

## 15.7 一貫性管理

いったんメモリが複製されると，DSM は複製を一貫させなければならない．共有メモリ型マルチプロセッサでは，ソフトウェアがロックで自身のアクセスを順序づけ，ハードウェアが CPU キャッシュ内のメモリの複製を一貫させる（第 10 章）．DSM にはそのようなハードウェアがない．その機構は，分散ファイルシステムがクライアントのキャッシュを一貫させる機構（第 14 章）に似ている．

**出典** P4L3，DSM の設計：一貫性管理；Protić ら；Nitzberg と Lo．

**プッシュ** 複製が書かれたとき，DSM は同じ単位の他の複製を保持するノードへ，直ちに無効化を押し出す．この方式は**先回り的**（proactive）かつ**即時的**（eager）である．また**悲観的**（pessimistic）でもある．古い複製は読まれて害をなすと仮定し，古い複製が生じ次第それを取り除くからである．DSM は無効化の代わりに新しい内容を複製へ押し出してもよい．第 10 章のキャッシュと同じく，書き込み無効化か書き込み更新かの選択である．

**プル** ノードは，単位の所有者から変更に関する情報を，周期的に，あるいは必要になったときに引き出す．この方式は**反応的**（reactive，オンデマンド）かつ**遅延的**（lazy）である．また**楽観的**（optimistic）でもある．古い複製をしばらく使っても害はないと期待し，即時的な伝播に要するメッセージを節約するからである．

**注意** DSM における一貫性管理はキャッシュ一貫性ではない．ソフトウェアによって，ページやオブジェクトの粒度で，しかも一貫性モデルが要求する時点でのみ行われる．

DSM が実際にこれらの機構を起動するのがいつか—書き込みのたびか，アクセスのたびか，周期的にか，あるいはアプリケーションが指定する時点でのみか—は，DSM が共有状態に対して提供する一貫性モデルによって決まる（15.10 節）．

プッシュとプルは，第 14 章のサーバ駆動およびクライアント駆動のキャッシュ一貫性に対応する DSM 版である．

**例 15.7.** ノード A がページ $p$ を所有し，ノード B が複製を保持している．A は $t = 0.5$ s に $p$ に書き，B は 1 s と 3 s に $p$ を読む（図 15.4）．プッシュでは，A は 0.5 s に B へ無効化を送る．B の 1 s の読み出しは有効な複製を見出せず，新しい内容を取得する．プルでは，B は 2 s ごとに $p$ が変更されたかを A に尋ねる．B の 1 s の読み出しは古い内容を返し，2 s のポーリングが変更を明らかにし，3 s の読み出しが新しい内容を返す．このように複製は最大で 1 ポーリング周期のあいだ古いままでありうる．その代わり，プルは一定の頻度でメッセージを要する．2 s ごとにポーリングする 10 個の複製は毎秒 5 回のポーリング（10 個のメッセージ）を行う．$p$ が毎秒 100 回書かれようと，まったく書かれまいと

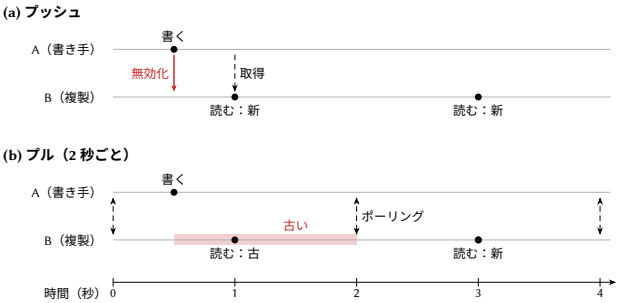



**図 15.4:** ノード A は 0.5 s にページ $p$ に書き，ノード B は複製を保持している．(a) A が無効化を押し出し，B は次の読み出しで新しい内容を取得する．(b) B は 2 s ごとにポーリングし，その複製は次のポーリングまで古いままである．

同じである．プッシュは，$p$ が書かれない間はメッセージを送らず，書き込み 1 回につき複製ごとに高々 1 回の無効化を送る．

**ヒント** プッシュは書き込みに比例して，プルは時間に比例してメッセージを要する．どちらが安いかは，書き込みの頻度とポーリングの頻度を比べればよい．

## 15.8 DSM のアーキテクチャ

**出典** P4L3，DSM のアーキテクチャ，分散した状態の索引付け；Nitzberg と Lo．

ここでは，オペレーティングシステムが実装するページに基づく DSM を考える．その構成は，これまでの設計上の選択から導かれる．

- どのノードも主記憶の一部，いくつかのページフレームを DSM に提供する．提供されたページはシステム全体で 1 つの共有メモリのプールをなし，すべてのノードからアクセスできる．
- どのノードも，自分のプロセスがアクセスするページのローカルなコピー，すなわち複製のキャッシュを保持し，アクセスレイテンシを下げる．
- 管理もまた分散される．どのノードも共有メモリの一部について責任を持つ．

各ページについて，DSM は次の役割を区別する．

**ホームノード** ページを管理するノードであり，その**管理ノード**とも呼ばれる．ホームノードはページの状態，すなわちどのノードがそれにアクセスしているか，変更されているか，そのキャッシングが有効か，ロックされているか，そして複製がどこにあるかを保持する．ページのアドレスはそれを初めに管理するノードを名指す．管理はアドレスを変えずに他のノードへ引き渡せる（15.8.1 節）．

ホームノードは，分散ファイルシステム（第 14 章）でサーバが果たす役割を果たす．ただし，どのノードもメモリの一部についてのホームノードである点が異なる．

**所有者** 現在，ページの正本のコピーを保持し，その更新を制御するノード．たとえば，直近にそれを書く権利を得たノードである．所有権はページがアクセスされるにつれて変わるので，所有者はホームノードである必要はない．ホームノードが現在の所有者を追跡する．

**明示的な複製** アクセスの副作用としてではなく，DSM が意図的に作る複製．負荷分散のため，性能のため，あるいは信頼性のためであり，たとえばデータセンタではデータを 3 つのコピーで保持するのが普通である．これらの複製をどこに置き，どう管理するかはホームノードが制御する．

3 つというのは HDFS の既定のレプリケーション数であり，HDFS は 2 つの複製を同じラックに，3 つ目を別のラックに置く（Hadoop のドキュメント）．

### 15.8.1 分散した状態の索引付け

アクセスを処理するために，ノードはページのメタデータを見つけなければならない．メタデータはそのページの管理ノードが保持している．DSM はこのメタデータを次のように組織する（図 15.5）．

- どのページも，ノード ID と物理フレーム番号（PFN，第 8 章）からなるアドレスで識別される．ノード ID は，そのフレームを提供し，初めはそのページを管理するノードを指す．PFN はそのノード上のフレームを指す．
- ページ ID から管理ノードの ID へのグローバルマップがすべてのノードに複製されるので，どのノードも任意のページの管理ノードを，メッセージを交換せずにローカルに見つけられる．
- ページ自体のメタデータは分割される．各管理ノードは自分が管理するページのメタデータを保持する．

マップを複製できるのは，それが小さいからである．後述の例の 8 ビットのノード ID フィールドで索引付けされるマッピングテーブルのエントリは 256 個しかない．はるかに数の多いページごとのメタデータは分割されたままである．

ページ ID のビットから管理ノードを直接導く代わりに，グローバルマップを**マッピングテーブル**として実装することもできる．ページ ID の一部がテーブルへの索引となり，そ

のエントリが管理ノードを示す．こうすれば，負荷分散のためや故障の後に，すべてのノードでテーブルのエントリを更新するだけで，あるページ集合の管理ノードを変更できる．ページ ID も，それを使うあらゆる参照も変わらない．いったんテーブルを介在させれば，ノード ID のフィールドはテーブルへの索引にすぎない．ページの管理ノードはエントリが示すノードであり，もはやそのフィールドが指すノードである必要はない．

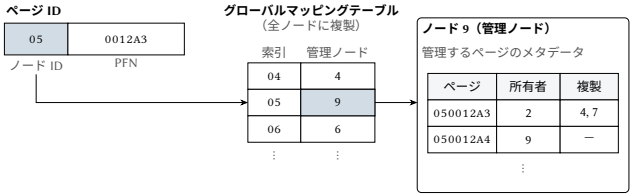



**図 15.5:** 分散した状態の索引付け．ページ ID のノード ID フィールドがグローバルマッピングテーブルの索引となり，テーブルはページの管理ノードを示す．管理ノードは自分が管理するページのメタデータを保持する．エントリ 05 はノード 9 に変更されているので，ノード ID フィールドが 05 であるページは，そのフレームがノード 5 にあるにもかかわらずノード 9 が管理する．

**例 15.8.** ある DSM は各ページを 32 ビットのアドレス，すなわち 8 ビットのノード ID とそれに続く 24 ビットの PFN で識別する．これにより，最大 256 のノードがそれぞれ最大 $2^{24}$ 個の 4 KiB フレーム，64 GiB のメモリを提供できる．ページ 0x050012A3 はノード 5 のフレーム 0x0012A3 であり，初めはノード 5 がそれを管理する．マッピングテーブルはノード ID フィールドで索引され，初めはすべての索引 $i$ をノード $i$ へ写す．ノード 5 が過負荷になると，すべてのノードでエントリ 5 を 9 に設定し，ページのメタデータをノード 5 からノード 9 へ移すことによって，ノード 5 のページの管理がノード 9 へ引き渡される（図 15.5）．フレーム自体はノード 5 に留まり，ページ ID は 1 つも変わらない．管理ノードをノード ID フィールドから直接取っていたなら，これらのページはすべて新しい ID を必要とし，それへのあらゆる参照を更新しなければならなかっただろう．

## 15.9 DSM の実装

DSM は 2 つの理由から共有メモリへのアクセスを捕捉しなければならない．プロセスが自ノードに存在しないメモリにアクセスするとき，DSM はそれを保持するノードへメッセージを送り，アクセスを要求しなければならない．プロセスが，他のノードが複製を保持するメモリを更新するとき，DSM はその更新を検出し，必要な一貫性操作を行わなければならない．同時に，ローカルに存在し調整を必要としないメモリへのアクセスは，DSM もオペレーティングシステムも関与させてはならない．あらゆるメモリアクセスにソフトウェアが介入すれば，局所性がもたらすはずの性能が失われてしまう．

ページに基づく DSM は，MMU のハードウェア支援によってこの相反する要求を解決する（図 15.6）．MMU は，仮想アドレスに有効なマッピングがないとき，あるいはマッピングが試みられた種類のアクセスを許可しないときはいつでも，ページフォールト（第 8 章）によってオペレーティングシステムへトラップする．それ以外のアクセスはすべてハードウェアで完了する．DSM は，捕捉しなければならないアクセスがちょうどフォールトするようにマッピングを設定する．

- 自ノードに存在しないページには有効なマッピングがない．それへのアクセスはトラップし，オペレーティングシステムはフォールトを DSM に渡す．DSM は所有者にページを要求し，受け取ったコピーのマッピングを設定してからプロセスを再開する．読み出しであれば読み出し専用でマップする．書き込みであれば，DSM は以下の一貫性操作も行ったうえでページを読み書き可でマップし，存在しないページへの書き込みが 2 回ではなく 1 回のフォールトで済むようにする．
- ノードが複製を保持するページは，保護ビット（第 8 章）によって読み出し専用でマップされる．読み出しはトラップせずに進む．書き込みはトラップし，DSM は一貫性操作–他の複製を無効化し，所有権を引き受ける–を行ってから書き込みを許可し，プロセスを再開する．[1]

MMU が記録する他の情報も有用である．ページのダーティビットは，そのページが変更されたか，したがって変更を伝播しなければならないかを示す．アクセスビットは，ローカルな複製が最近使われたかを示し，メモリが足りなくなったときにどの複製を追い出すかを，ページ置換（第 8 章）と同様に決めるのに役立つ．変更された部分だけを伝播

**ヒント** マッピングテーブルとページの管理ノードの関係は，ページテーブルとフレームの関係と同じである．すなわち，名前を変えないまま名前の指す先を変えられるようにする間接の層である．

**出典** P4L3，DSM の実装；Nitzberg と Lo；Tanenbaum と Bos ch. 8.

**注意** このページフォールトはスワップ領域から処理されるのではない．DSM はページを他のノードのメモリから取得する．

[1] 最初のページに基づく DSM である IVY [60] はこのように動作し，MRSW アクセスの下で逐次一貫性を提供する．後の TreadMarks [61] は，より弱いモデルの下で，変更を加えていない UNIX システム上に，完全にユーザ空間で DSM を実装した．これらの MMU の操作はただではない．Appel と Li [62] は，1991 年当時のマシンで，ページあたり約 1200 µs のフォールトと再保護のオーバヘッドを測定した．

するために，DSM は書き込み保護したページのコピー（ツイン，twin）を保持しておき，一貫性操作の時点でその差分（diff）を送ることがある（TreadMarks [61] がそうである）．オブジェクトに基づく DSM は，代わりに言語ランタイムでアクセスを捕捉する．ランタイムがプログラムの共有オブジェクトに対する操作を仲介するからである．

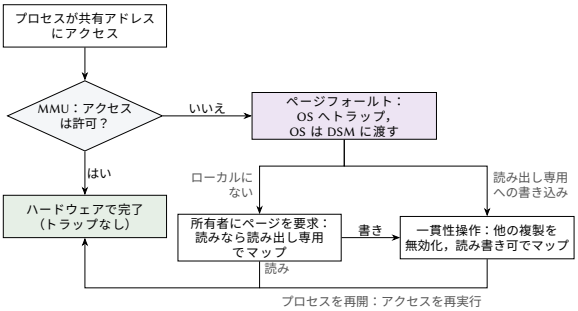



**図 15.6:** ページに基づく DSM におけるアクセスの捕捉．現在のマッピングが許可するアクセスはハードウェアから出ない．それ以外はフォールトし，DSM はページを取得するか一貫性操作を行い，その後アクセスがやり直される．

**例 15.9.** ページ $p$ は初め，所有者であるノード 1 にのみ存在し，ノード 1 はそれを読み書き可でマップしている．表 15.2 は，書き込み時に無効化する MRSW アクセスの下での 3 つのノードによるアクセスを追ったものである．ステップ 1 でノード 2 が複製を得ると，ノード 1 のマッピングは読み出し専用に格下げされ，後にノード 1 が書き込めばフォールトして複製を無効化するようになる．7 回のアクセスのうち 4 回がフォールトする．ステップ 2 と 3 の読み出し，およびステップ 5 の書き込みは，マッピングがすでにそれらを許可しているのでハードウェアで完了する．

**ヒント** ノードごとに見れば，ページのマッピングは「なし，RO，RW」の 3 状態マシンであり，フォールトするのは遷移だけである．読み出しはなしから RO へ，書き込みはなしまたは RO から RW へ移し，他のノードのアクセスは RO またはなしへ格下げする．

**表 15.2:** ページ $p$ へのアクセスと，それが引き起こすフォールト．最後の列は各ステップの後のノード 1/2/3 における $p$ のマッピングを示す（RW は読み書き可，RO は読み出し専用，– はマッピングなし）．

| ステップ | アクセス | フォールト | DSM の動作 | マッピング |
|---|---|---|---|---|
| 0 | （初期状態） | – | – | RW/–/– |
| 1 | ノード 2 が読む | ページフォールト | 所有者のノード 1 から $p$ を取得；ノード 1 を RO に格下げ | RO/RO/– |
| 2 | ノード 2 が読む | なし | – | RO/RO/– |
| 3 | ノード 1 が読む | なし | – | RO/RO/– |
| 4 | ノード 2 が書く | 保護違反 | ノード 1 のコピーを無効化；ノード 2 が所有者になる | –/RW/– |
| 5 | ノード 2 が書く | なし | – | –/RW/– |
| 6 | ノード 3 が読む | ページフォールト | 所有者のノード 2 から $p$ を取得；ノード 2 を RO に格下げ | –/RO/RO |
| 7 | ノード 2 が書く | 保護違反 | ノード 3 のコピーを無効化 | –/RW/– |

**要点**
ページに基づく DSM は，共有されるページをすべてのノードが提供したメモリに置き，複製されたグローバルマップを通じて各ページの管理ノードを見つけ，MMU を使って，必要なアクセスだけ–存在しないページへのアクセスと，複製されたページへの書き込み–をちょうど捕捉する．それ以外のアクセスはすべてハードウェアの速度で進む．

## 15.10 一貫性モデル

15.7 節 の機構は異なる時点で起動でき，その結果得られる保証も異なる．分散ファイルシステムでは，これらの保証は第 14 章のファイル共有セマンティクスである．メモリについて，そしてより一般に共有状態について，それを述べるのが一貫性モデルである．

出典 P4L3，一貫性モデル，厳密一貫性，逐次一貫性，因果一貫性，弱一貫性；Protić ら；Tanenbaum と van Steen [59] ch. 7.

**定義 15.10** (一貫性モデル). **一貫性モデル**とは，メモリ– あるいは任意の共有状態–と，その上のソフトウェア層との間の取り決めである．ソフトウェアが特定の規則に従う限り，メモリは，アクセスの順序づけと，更新の伝播および可視性に関して正しく振る舞うことを保証する．

▷ consistency model

したがってファイル共有セマンティクスは，ファイルに対する一貫性モデルである．メモリに対する一貫性モデルを記述するために，メモリ位置 $m$ から値 $v$ を返す読み出しを $R_m(v)$，$m$ への $v$ の書き込みを $W_m(v)$ と書く．操作はプロセスごとに 1 本のタイムラインに描き，時間は右へ進む．すべての位置は初め 0 を保持している．

タイムラインにおいて $m_a$ と $m_b$ は共有アドレス空間の 2 つの位置であり，$t_1 < t_2 < \ldots$ は外部の観測者が見る実時刻の時点である．

### 15.10.1 厳密一貫性

**定義 15.11** (厳密一貫性). **厳密一貫性**の下では，あらゆる更新がただちにどこからも見える．すなわち，どのノードのどの読み出しも，その位置に対する実時間で最も新しい書き込みの値を返し，すべてのノードが更新を，それがなされた順序で観測する．

▷ strict consistency

厳密一貫性は，プログラマがメモリに対して直観的に期待するものだが，理論上のモデルである．単一の共有メモリ型マルチプロセッサ上でさえ，異なる CPU 上の並行な更新は，ソフトウェアがロックと同期を加えない限り，保証された順序で効果を持つわけではない．分散システムでは，更新はゼロでないレイテンシを持つネットワークを渡り，メッセージは順序が入れ替わったり失われたりし，異なるノード上の 2 つの書き込みのどちらがより新しいかを定めるグローバルな時計も存在しない．厳密一貫性を保証することはできない．

### 15.10.2 逐次一貫性

**定義 15.12** (逐次一貫性). **逐次一貫性**の下では，あらゆる実行の結果が，すべてのプロセスの操作をある逐次順序で実行したときと同じになる．その順序では，各プロセスの操作はそのプロセスが発行した順に現れ，どの読み出しもその順序においてその位置に対する最も新しい書き込みの値を返し，すべてのプロセスがこの同一の順序を観測する [63].

▷ sequential consistency

したがって，異なるプロセスからの更新は任意にインタリーブされてよく，インタリーブが実時間の順序と一致する必要もないが，すべてのプロセスが同じインタリーブを見なければならない．

注意 逐次一貫な実行は実時間と一致する必要はない．すべてのプロセスが同じ順序を観測する限り，読み出しが，実時間でそれに最も近く先行する書き込みより古い値を返してもよい．

### 15.10.3 因果一貫性

ある書き込みが以前の書き込みに**因果的に依存する**とは，同じプロセスが以前の書き込みを発行していたこと，あるいはそれを発行したプロセスが，直接に–たとえばその書き込んだ値を読むことによって–あるいはそうした依存の連鎖を通じて，以前の書き込みを観測していたことをいう．どちらも他方に依存しない書き込みは**並行**であるという．

**定義 15.13** (因果一貫性). **因果一貫性**の下では，すべてのプロセスが，因果的に関係する書き込みをその因果順序で観測し，各プロセスの操作をそのプロセスが発行した順序で観測する．並行な書き込みは，プロセスごとに異なる順序で観測されてよい．

▷ causal consistency

因果一貫性を提供するには，DSM は因果関係がありうる箇所を検出しなければならない．すなわち，各書き込みの前にそのプロセスがどの更新を観測していたかを記録しなければならない．その代わり，順序を調整する必要があるのは因果的に関係する更新だけである．

ベクタタイムスタンプはこうした依存を記録する一般的な方法である（Tanenbaum と van Steen ch. 6）．

### 15.10.4 弱一貫性

弱一貫性は**同期点**を導入する．同期点とは，DSM がその上のソフトウェアに提供する sync のような操作である．

**定義 15.14** (弱一貫性). **弱一貫性**の下では，あるノードが自身の同期点より前に行った更新を別のノードが観測することが保証されるのは，観測する側のノード自身がその後に

▷ weak consistency

同期操作を行ってからである．同期点の間では，ノードがどの更新を観測するかについて何の保証も与えられない．

DSM は，共有状態全体に対して 1 つの同期操作を提供することも，個々のページやオブジェクトといった状態の部分集合に対して別々の操作を提供し，アプリケーションが必要な状態だけを同期できるようにすることもできる．また操作を 2 つに分けることもできる．プロセスが共有状態を使う前に行い，他のプロセスの更新をそのプロセスから見えるようにする *acquire* と，更新を終えた後に行い，自身の更新を伝播する *release* である．弱一貫性は，データの移動と調整を同期点に限り，そこでは前の同期点以降になされた更新をまとめて伝播できる．その代わり，DSM はこれらの更新を記録し集約するための追加の状態を保持しなければならず，同期点の間ではプロセスが古いデータを観測しうる．

セッションセマンティクス（第 14 章）は弱一貫性のファイル版である．open と close が同期点にあたる．

## 15.10.5 モデルの比較

厳密一貫性，逐次一貫性，因果一貫性は階層をなす．厳密一貫な実行はすべて逐次一貫であり，逐次一貫な実行はすべて因果一貫である．より弱いモデルはより多くの実行を許すので調整が少なくて済むが，アプリケーションに与える保証は少ない．

**注意** 実際のハードウェアも逐次一貫ではない．x86 では，ロードが別の位置への先行するストアを追い越しうる（total store order）．ARM や POWER はさらに順序を入れ替える．フェンスが同期点の役割を果たす（Intel SDM vol. 3A，メモリ順序付け）．

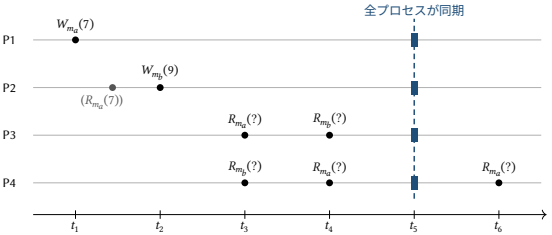



**図 15.7:** 例 15.15 の操作．P2 による $m_a$ の読み出し（灰色）は，P2 の書き込みが P1 の書き込みに依存する変種でのみ起こる．破線はすべてのプロセスによる同期操作を示す．

**例 15.15.** 図 15.7 では，P1 が $t_1$ に $m_a$ へ 7 を書き，P2 が $t_2$ に $m_b$ へ 9 を書く．P3 と P4 は $t_3$ と $t_4$ に両方の位置を，互いに逆の順序で読む．表 15.3 は，これらの読み出しの 3 通りの結果と，それが満たすモデルを挙げたものである．

- 結果 (a) は厳密一貫である．どの読み出しも実時間で最も新しい書き込みを返している．
- 結果 (b) はそうではない．P4 は，$t_2$ に 9 が書かれた後の $t_3$ に $m_b$ から 0 を読んでいる．しかし逐次一貫ではある．P3 の $R_{ma}(0)$，P4 の $R_{mb}(0)$，$W_{ma}(7)$，$W_{mb}(9)$，P3 の $R_{mb}(9)$，P4 の $R_{ma}(7)$ という順序が，どの読み出しも説明し，各プロセスの操作の順序も保っているからである．
- 結果 (c) は逐次一貫ではない．P3 は $W_{ma}(7)$ を $W_{mb}(9)$ より先に観測し，P4 は逆の順序で観測しており，両者を説明する単一の順序は存在しない．2 つの書き込みが並行であれば，(c) は因果一貫である．しかし P2 が $m_b$ に書く前に $m_a$ から 7 を読んでいたなら，$W_{mb}(9)$ は $W_{ma}(7)$ に因果的に依存し，$m_b$ から 9 を読みその後 $m_a$ から 0 を読む P4 は，原因より先に結果を観測していることになる．

4 つのプロセスすべてが $t_5$ に sync を実行すれば，3 つの結果はいずれも弱一貫である．$t_5$ より前には何の保証も適用されないからである．ただし $t_6$ における P4 の $m_a$ の読み出しは 7 を返さなければならない．

**ヒント** 逐次一貫性を確かめるには，各プロセスの順序を保ったまま全操作を 1 本の列にまとめ，すべての読み出しを説明できるものを探せばよい．

**注意** モデルは結果を制約するのであって，結果を決定するのではない．逐次一貫性を提供する DSM は，ある実行で (a) を，次の実行で (b) を生じうるし，弱一貫な DSM は 3 つのいずれをも生じうる．

**表 15.3:** 例 15.15 における $t_3$ と $t_4$ の読み出しの結果．因果の列は，書き込みが並行である場合／P2 が $m_b$ に書く前に $m_a$ を読んだ場合の結果を示す．

| | P3 | P4 | 厳密 | 逐次 | 因果 | 弱 |
|---|---|---|---|---|---|---|
| (a) | $R_{m_a}(7), R_{m_b}(9)$ | $R_{m_b}(9), R_{m_a}(7)$ | はい | はい | はい／はい | はい |
| (b) | $R_{m_a}(0), R_{m_b}(9)$ | $R_{m_b}(0), R_{m_a}(7)$ | いいえ | はい | はい／はい | はい |
| (c) | $R_{m_a}(7), R_{m_b}(0)$ | $R_{m_b}(9), R_{m_a}(0)$ | いいえ | いいえ | はい／いいえ | はい |

**要点 — 章のまとめ**
分散共有メモリは，どのノードもメモリを提供し，それへのアクセスを処理し，一貫性プロトコルに参加するピア分散アプリケーションであり，それによって多数のマシン上のプロセスが 1 つのアドレス空間を共有する．その設計は，共有の粒度（ページかオブジェクトか，偽共有の危険を伴う），アクセスアルゴリズム（SRSW，MRSW，MRMW），局所性を得るためのマイグレーションとレプリケーションの選択，そして無効化をプッシュするか変更をプルするかという一貫性管理によって決まる．ページに基づく DSM は，複製されたマッピングテーブルを通じて各ページの管理ノードを見つけ，調整を必要とするアクセスだけを捕捉するために MMU のフォールトに依存する．一貫性モデルはプロセスが何を観測するかを述べる．厳密一貫性は実際には達成できない．逐次一貫性はすべての操作に共通の 1 つの順序を要求する．因果一貫性は因果的に関係する書き込みだけを順序づける．弱一貫性は同期点においてのみ可視性を保証する．

第 16 章

# データセンタ技術

| | |
|---|---|
| **講義動画** | P4L4 |
| **教科書** | Silberschatz ら [2] ch. 1；OSTEP [1] ch. 48；Armbrust ら [64]；Mell と Grance [65] |
| **キーワード** | インターネットサービス，多層アーキテクチャ，スケールアウト，ロードバランサ，ホモジニアス構成，ヘテロジニアス構成，クラウドコンピューティング，伸縮性，大数の法則，ユーティリティコンピューティング，IaaS，PaaS，SaaS，マルチテナンシ |

これまでの章は，異なるマシン上のプロセスが協調するための機構–遠隔手続き呼び出し，分散ファイルシステム，そして第 15 章の分散共有メモリ–を扱ってきた．これらの機構は，より前の章のスレッド，サーバ，スケジューリング，仮想化とともに，インターネット越しに使われるサービスを動かすデータセンタにおいて，最大の規模で用いられている．この最後の章では，データセンタ技術を大まかに概観する．インターネットサービスがどう構成され，多数のマシンへスケールアウトされるか，クラウドコンピューティングとは何か，それがなぜ成り立ち，どのようなモデルを提供するか，基盤となるシステムに何を要求するか，そしてどのような技術がそれを可能にするかである．

## 16.1 インターネットサービス

**出典** P4L4，レッスンの概要，インターネットサービス．

▷ datacenter

日常的に使われるアプリケーションの多くは，その利用者のマシン上では動作しない．それらは**データセンタ**のマシン上で動作する．データセンタとは，ネットワークで結ばれた多数の計算機をそのストレージ，電源，冷却設備とともに収容し，計算サービスを提供するために運用する施設である．利用者はインターネット越しにこれらのアプリケーションに到達する．

▷ Internet service

**定義 16.1** (インターネットサービス). **インターネットサービス** とは，Web インタフェースを通じてインターネット越しに利用者へ提供されるあらゆるサービスである．

よく使われるオープンソースによる実現は LAMP スタックである．Linux，Apache（プレゼンテーション），PHP（ロジック），MySQL（データベース）からなる．

Web メール，オンラインショップ，検索エンジン，ソーシャルネットワークはインターネットサービスである．ほとんどのインターネットサービスは 3 つの構成要素からなる（図 16.1）．

**プレゼンテーション** 利用者とやり取りする．その要求を受け取り，Web ページを返し，ページのレイアウトや画像のように要求によらず同じである**静的コンテンツ**を提供する．

**ビジネスロジック** サービスの機能を実装する．各要求を処理し，検索の結果やショッピングカートの中身のような**動的コンテンツ**を生成する．

**データベース** 利用者，商品，注文といったサービスのデータを格納し，ビジネスロジックのためにそれらを取り出す．

▷ multi-tier architecture

**定義 16.2** (多層アーキテクチャ). 一連の構成要素として構成され，各構成要素が次の構成要素のサービスを利用するサービスは，**多層アーキテクチャ**を持つ．各構成要素が 1 つの層であり，上記の 3 つの構成要素は三層アーキテクチャを成す．

層の境界はコストの境界でもある．同じ呼び出しでも，1 つのプロセス内ならナノ秒，マシン内の IPC 経由なら数マイクロ秒（第 9 章），データセンタ内の別マシンへの RPC なら約 0.5 ms である（Dean の「誰もが知るべき数値」[66]）．

層は論理的な構成要素である．別々のプロセスである必要はなく，別々のプロセスが別々のマシンで動作する必要もない．Web サーバ，アプリケーションサーバ，データベースサーバといった多数のオープンソースおよび商用のミドルウェア製品が個々の層と層どうしのやり取りを実装しており，さまざまな構成で組み合わせられる．層が別々のプロセスとして動作する場合，プロセスは何らかの IPC で通信する．同じマシン上ではメッセージパッシングや共有メモリ（第 9 章），マシンをまたぐときは RPC や RMI（第 13 章）である．

## 16.2 スケールアウト構成

**出典** P4L4，インターネットサービスの構成，ホモジニアス構成，ヘテロジニアス構成．

1 台のマシンが毎秒処理できる要求の数は，サーバソフトウェアをどれほど効率よく設計しても限られる（第 6 章）．より多くの要求を処理しなければならないサービスは，したがって多数のマシン上の多数のプロセスとして動作する．これらのマシンがサービスの**ノード**である．

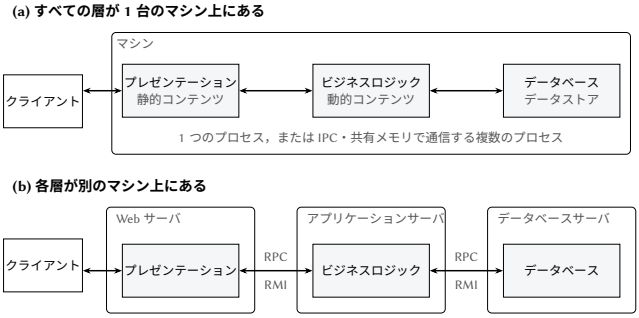



**図 16.1:** 三層のインターネットサービス．(a) すべての層が 1 台のマシン上で，1 つのプロセスとして，あるいは複数のプロセスとして動作する．(b) 各層が自分のマシンで動作し，層どうしは RPC または RMI で通信する．

**定義 16.3** (スケールアウト)**.** ノードをより強力なマシンで置き換える（**スケールアップ**）のではなく，ノードを追加することでサービスの処理能力を高める構成を**スケールアウト**構成と呼ぶ．

▷ scale-out

スケールアウト構成は，第 3 章のボス・ワーカーパターンをスレッドではなくマシンに適用することが多い．フロントエンドのノードがすべての要求を受け取り，その各々を，要求を処理するノードの 1 つへ渡す．

**定義 16.4** (ロードバランサ)**.** **ロードバランサ**とは，サービスに到着する要求を，それを処理するノードの間へ分配するフロントエンドである．

▷ load balancer

代表的な分配方針はラウンドロビンと最小接続数であり，NGINX と HAProxy はいずれも両方を提供する（各ドキュメント）．

ボス・ワーカーパターンのワーカーがすべて等しいか特化しているかのいずれかであるのと同様に，スケールアウト構成のノードは 2 通りに編成できる（図 16.2）．

**すべて等しい** どのノードも，あらゆる要求の処理のあらゆる段階を実行できる．ノードは機能的にホモジニアスである．

**特化する** 各ノードは処理の一部の段階だけ，あるいは一部の種類の要求だけを実行する．ノードは機能的にヘテロジニアスである．

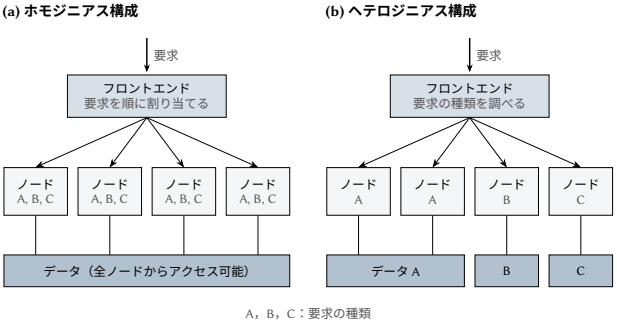



**図 16.2:** スケールアウト構成．(a) どのノードもあらゆる種類の要求を処理し，どのノードもすべてのデータにアクセスできる．(b) ノードは要求の種類に特化しており，フロントエンドはノードを選ぶために各要求を調べなければならない．

### 16.2.1 ホモジニアス構成

**定義 16.5** (ホモジニアス構成)**.** **ホモジニアス構成** では，すべてのノードが機能的に同一である．どのノードも，あらゆる要求に対するあらゆる処理段階を実行できる．

▷ homogeneous architecture

ホモジニアス構成のフロントエンドは単純である．要求を調べる必要はなく，ラウンドロビンで順に，あるいは最も負荷の小さいノードへ割り当てればよい．機能的にホモジ

ニアスであっても，すべてのノードがすべてのデータを格納している必要はない．必要なのは，どのノードもすべてのデータに**アクセスできる**ことだけであり，たとえば分散ファイルシステムや共有のデータベース層を通じてであってよい．その層自体は複製されていても分割されていてもよい（第 14 章）．処理能力を高めるには，単にノードを追加する．

どのノードがどのデータを持つかは，利用者，セッション，ファイル名といった鍵のハッシュで決めるのが普通である．$N$ 番目のノードを追加しても鍵のおよそ $1/N$ しか移動しないコンシステントハッシュ法 [67] がよく使われる．

その代価は，サービスがキャッシュからほとんど利益を得られないことである．フロントエンドは，どのノードが最近似た要求を処理し，必要なデータをホットキャッシュに保持しているかを知らない．あるノードがキャッシュから処理できたはずの要求も，したがって，まずデータを取得しなければならないノードへ送られる可能性が高い．これは，等しいワーカーについて第 3 章が指摘した局所性の欠如である．

### 16.2.2 ヘテロジニアス構成

▷ heterogeneous architecture

**定義 16.6** (ヘテロジニアス構成)**. ヘテロジニアス構成**では，ノードが機能的に特化している．異なるノードが異なる処理段階を実行し，あるいは異なる種類の要求を処理する．

このときデータは，すべてのノードから一様にアクセスできる必要はない．各ノードには，自分の処理段階や要求の種類に必要なデータだけがあればよい．利点は局所性である．ある種類の要求はすべて同じ少数のノードに到達し，それらのノードは，第 3 章の特化したスレッドと同様に，その要求が必要とするコードとデータをホットキャッシュに保つ．しかしフロントエンドはより複雑になる．どのノードがその要求を処理できるかを判定するために，たとえば URL を解析して，あらゆる要求を調べなければならない．サービスの管理もまた複雑になる．

接続を転送するだけのフロントエンドは，トランスポート層のヘッダより先を見なくてよい．URL や cookie でノードを選ぶものは，接続を終端して要求そのものを解析しなければならない．

### 16.2.3 スケーリングと管理

スケールアウトとスケールアップは，水平スケーリングおよび垂直スケーリングとも呼ばれる．いずれもサービスのスループットを高めるが，どちらも単一の要求の処理を短くはしない．

2 つの構成の違いは，負荷が増えたときに最も大きく現れる．ホモジニアス構成をスケールさせるには，ノードと，それが必要とするプロセス，マシン，ストレージへのアクセスを追加すればよい．新しいノードはどれもあらゆる種類の要求を分担するので，どの要求が負荷を生んでいるかを知る必要はない．ヘテロジニアス構成をスケールさせるには，運用者はどの種類の要求に需要があるかを知り，その種類のノードを追加しなければならず，そのためにはワークロードのプロファイリングが必要である．要求の種類ごとの需要は時とともに変化するので，ノードの構成比もそれに追随しなければならない．さもなければ，人気のある種類のノードは未処理の要求の長い待ち行列を抱えた**ホットスポット**となり，他の種類のノードは遊休する．表 16.1 にこの比較をまとめる．

同じ不均衡は 1 台のマシンの中でも第 7 章に現れる．そこでは負荷分散が CPU ごとのランキューの間でスレッドを移動させる．特化したノードは互いの仕事を引き受けられない．必要なデータを持たないからである．

**表 16.1:** ホモジニアスなスケールアウト構成とヘテロジニアスなスケールアウト構成．

| | ホモジニアス | ヘテロジニアス |
|---|---|---|
| ノード | 機能的に同一 | 処理段階または要求の種類に特化 |
| フロントエンド | 単純；要求を調べる必要がない | あらゆる要求を調べなければならない |
| データ | どのノードからもアクセスできる | 一部のノードにだけ必要 |
| キャッシュと局所性 | ほとんど利益がない | ある種類の要求が同じノードに到達する |
| スケーリング | ノードを追加する | 需要をプロファイルし，必要な種類のノードを追加する |
| 要求の構成比の変化 | 介入なしに吸収される | ノードを割り当て直さない限りホットスポットが生じる |

**例 16.7.** あるオンラインショップが 2 種類の要求を受け取る．ホモジニアス構成のノード上では，検索 1 件の処理に 5 ms，チェックアウト 1 件に 20 ms かかる．毎秒 1200 件の検索と 200 件のチェックアウトという負荷は，1 秒当たり $1200\times0.005+200\times0.020=6+4=10$ 秒の処理を要し，これは 10 ノード分の処理能力である．構成比が毎秒 600 件の検索と 350 件のチェックアウトに変化しても，必要量は $3+7=10$ で変わらない．同じ 10 ノードで足り，何の介入も必要ない．

**ヒント** これは第 6 章のリトルの法則である．同時に動いているノードの数は到着率と処理時間の積であり，要求の構成比によらず負荷は $\sum_i \lambda_i t_i$ ノードとなる．

ヘテロジニアス構成では，ホットキャッシュによって，特化したノードが要求を 20 % 短い時間で，すなわち検索 1 件を 4 ms，チェックアウト 1 件を 16 ms で処理できるとする．元の負荷には $\lceil 1200\times0.004\rceil=5$ 個の検索ノードと $\lceil 200\times0.016\rceil=4$ 個のチェックアウトノード，合計 9 個が必要である．変化の後には $\lceil 2.4\rceil=3$ 個の検索ノードと $\lceil 5.6\rceil=6$ 個のチェックアウトノードが必要となる．合計は同じ 9 個だが，構成比は異なる．運用者が 2 個のノードを検索からチェックアウトへ移すまで，4 個のチェックアウトノード

は毎秒 4/0.016 = 250 件しか完了できず，未処理の待ち行列は毎秒 100 件ずつ伸び，検索ノードの稼働率は 48 % にすぎず，2.6 ノード分の処理能力が遊休する．

**要点**
インターネットサービスは層に分けて構成され，ロードバランサの背後で多数のノードへスケールアウトされる．ホモジニアスなノードはフロントエンドとスケーリングを単純に保つが，キャッシュからほとんど利益を得られない．ヘテロジニアスなノードは局所性を得るが，その代価として，要求を調べるフロントエンドと，要求の構成比に追随する管理を必要とする．

## 16.3 クラウドコンピューティングの動機

講義によれば，オンライン小売業者である Amazon は，ホリデーシーズンの商戦のピーク負荷に合わせてハードウェアを用意しており，1 年の残りの期間，そのハードウェアの多くは遊休していた．Amazon は Web ベースの API を通じてそれへのアクセスを開放し，第三者が料金を払って Amazon のハードウェア上で自分のワークロードを実行できるようにした．これが Amazon Web Services（AWS）である．2006 年に開始されたその Elastic Compute Cloud（EC2）は**コンピュートインスタンス**，すなわち顧客が API を通じて起動・停止し，当時は時間単位で料金を支払った（2017 年以降は秒単位，最低 60 s）仮想マシン（第 12 章）を貸し出す．

利用者の写真から動画を生成するサービスである Animoto は，コンピュートインスタンスを EC2 から借りていた．2008 年 4 月に Animoto が Facebook の利用者にも使えるようになると需要は急増し，3 日のうちに利用するサーバ数は 50 台から 3500 台へ増え，その後トラフィックはピークをはるかに下回る水準まで下がった [64]．数日で 2 桁近く成長することは，マシンを購入し，設置し，構成しなければならない場合には不可能である．借りた資源では，必要なのは API の呼び出しだけであった．

**出典** P4L4，クラウドコンピューティングの代表例：Animoto，クラウドコンピューティングの要件；Armbrust ら [64]．

**注意** この話には異論がある．Amazon の CTO である Werner Vogels は，余剰容量の販売という話を作り話だと公に述べており，AWS は Amazon が自社のチームのために構築した共通の基盤から生まれたとしている．

EC2 は 2006 年 8 月に限定的なパブリックベータとして公開され，2008 年 10 月に一般提供となった．

### 16.3.1 変動する需要に対する資源の用意

従来は，サービスは自分の資源を購入し構成する．その容量は期待される需要，通常はその期待されるピークから事前に決められ，後から容量を増やせるのは大きな刻みに限られ，そのたびに購入による遅れを伴う．需要は時とともに変動するので，固定された容量はほとんどの時点で適切でない（図 16.3a）．

- 需要が容量を下回っている間，遊休する資源は無駄になる．支払い済みでありながら使われていない．
- 需要が容量を超えると，要求は捨てられ，それらの利用者に提供する機会と，それがもたらしたはずの収益が失われる．

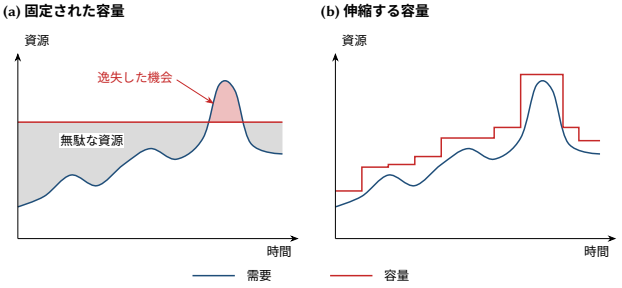



**図 16.3:** 時間に対する容量と需要．(a) 固定された容量は，需要が低い間は資源を無駄にし，需要が急増する間は要求を失う．(b) 伸縮する容量は需要に追随する．

理想的には，サービスの容量は需要に追随するのが望ましい（図 16.3b）．

- 容量は需要に応じて，増える方向にも減る方向にも伸縮する．
- スケーリングは即座に行われる．
- 費用は需要に，したがって収益の機会に比例する．
- 以上のすべてが，特別な専門知識なしに自動的に行われる．
- 資源にはいつでも，どこからでもアクセスできる．

**注意**「即座に」は理想であって機構ではない．仮想マシンの起動には数十秒，コンテナでも 1 秒ほどかかり，1 分ごとの測定値を見る自動スケーラはそれを何回か待つ．伸縮する容量は数分の遅れで需要に追随する．

▷ elasticity

**定義 16.8** (伸縮性). 需要の増減に応じてシステムが資源を素早く自動的に獲得し解放し，その容量が需要に追随する能力を**伸縮性**と呼ぶ．

**注意** 伸縮性はスケーラビリティではない．スケーラブルなシステムは大きくなれるが，伸縮的なシステムはさらに自動的に再び小さくもなる．

この理想の代価は，サービスが，自分の動作する基盤である資源をもはや所有せず，そのデータが他者の運用するマシン上に置かれることである．

**例 16.9.** あるサービスの需要が，毎日 16 時間はサーバ 20 台，残りの 8 時間はサーバ 100 台であるとする．1 日当たり $20\times16+100\times8=1120$ サーバ時である．所有するサーバは，使っていても使っていなくても 1 時間当たり $c$ の費用がかかる．ピークに合わせて用意すると，サービスは $100\times24=2400$ サーバ時，すなわち 1 日当たり $2400c$ を支払い，そのうち $1-1120/2400\approx53\,\%$ が無駄になる．平均の需要である $1120/24\approx47$ 台に合わせて用意しても，需要が低い間に $27\times16=432$ サーバ時を無駄にし，ピークの間は要求の $53/100=53\,\%$ を捨てる．50 % 高い価格，すなわち 1 サーバ時当たり $1.5c$ で伸縮する容量を借りると，サービスの支払いは 1 日当たり $1120\times1.5c=1680c$ となり，ピークに合わせて用意する場合より 30 % 少なく，要求を 1 つも捨てない．

**注意** 容量を借りても，資源をどれだけ用意するかという判断がなくなるわけではない．その判断の時間軸が短くなるだけである．いくつのインスタンスを動かすかは依然として決めなければならず，それが数年ごとではなく数分ごとになる．

**出典** P4L4，クラウドコンピューティングの概観，クラウドコンピューティングが成り立つ理由，クラウドコンピューティングの構想，クラウドコンピューティングの定義；Armbrust ら [64]；Mell と Grance [65]；Silberschatz ら ch. 1.

## 16.4 クラウドコンピューティング

クラウドコンピューティングは，上で述べた理想に近づくものである．その本質的な要素は次のとおりである．

**共有される資源** プロバイダは，計算，ストレージ，ネットワークといったインフラストラクチャに加えて，ソフトウェアやサービスを提供し，それらを多数の顧客の間で共有する．

**アクセスと設定のための API** 顧客は，Web ベースの API，プログラミング言語のライブラリ，あるいはコマンドラインツールを通じて資源を要求し，設定し，解放する．

**課金と会計** 利用量は計測され課金される．多様な価格モデルが併存する．たとえば，オンデマンド価格，長期間予約した容量に対するより低い価格，そして需給に応じて変動する余剰容量の**スポット価格**であり，プロバイダは事実上の市場を運営することになる．資源は離散的な大きさで提供される．たとえば CPU 数とメモリ量の異なる small，medium，extra-large といったインスタンスである．

**プロバイダによる管理** プロバイダが，データセンタ，ハードウェア，およびサービスを実装するソフトウェアを運用する．

実際の価格の目安．EC2 の最初のインスタンス種別 m1.small（仮想コア 1 つ，メモリ 1.7 GB）はオンデマンドで 1 時間 0.10 ドルであり [64]，AWS はスポット容量を最大 90 % 引きとうたっている．

**定義 16.10** (クラウドコンピューティング). 多数の顧客の間で共有される資源から，API を通じて需要に応じて獲得・解放され，利用に応じて課金され，プロバイダによって管理される計算資源を，ネットワーク越しにサービスとして提供することを，**クラウドコンピューティング**と呼ぶ．

▷ cloud computing

**注 16.11.** 公表されている定義もこれらの要素と一致する．NIST の定義 [65] は 5 つの本質的な特徴，すなわちオンデマンドのセルフサービス，幅広いネットワークアクセス，資源のプール化，迅速な伸縮性，計測されたサービスを挙げる．Armbrust ら [64] はこの語を，インターネット越しにサービスとして提供されるアプリケーションと，それを提供するデータセンタ内のハードウェアおよびシステムソフトウェアの両方に用いる．後者を彼らは**クラウド**と呼ぶ．彼らはプライベートクラウドをクラウドコンピューティングに含めず，また IaaS や PaaS のような，アプリケーションより下の層に対する「X as a service」という呼び名を避けている（16.5 節）．

### 16.4.1 クラウドコンピューティングが成り立つ理由

プロバイダは，すべての顧客の需要を賄うだけの資源を所有し，なお顧客が自前の資源に費やすであろう額より安く課金しなければならない．これを可能にする原理が 2 つある．

**定義 16.12** (大数の法則). **大数の法則**によれば，同じ分布に従う $N$ 個の独立な確率量の平均は，$N$ が大きくなるにつれてその期待値に近づく．

▷ law of large numbers

単一の顧客の資源需要は時とともに大きく変動するが，異なる顧客はそれぞれ別の時点でピークを迎えるので，多数の顧客にわたって平均した需要はほぼ一定である．$N$ 人の顧客の需要が独立で，各々の平均が $\mu$，標準偏差が $\sigma$ であれば，その和の平均は $N\mu$，標準偏差は $\sigma\sqrt{N}$ となり，平均に対する変動の割合 $\sigma/(\mu\sqrt{N})$ は $N$ とともに小さくなる．したがってプロバイダは，個々のピークの和よりはるかに小さい，合計の需要に合わせて資源を用意できる．

**ヒント** 相対的な変動は $1/\sqrt{N}$ でしか減らない．独立な顧客が 10 倍になっても約 3 分の 1 に，100 倍になってようやく 10 分の 1 になるにすぎない．

**例 16.13.** あるプロバイダが 100 の顧客にサービスを提供している．各顧客は普段 10 台

のサーバを必要とし，ピーク時には 100 台を必要とする．ピークは時間全体の 10 % を占め，他の顧客のピークとは独立に生じる．すべての顧客のピークに合わせて用意すると $100 \times 100 = 10\,000$ 台のサーバが要る．しかし，ある時点でピークにある顧客の数 $K$ は，平均が $100 \times 0.1 = 10$，標準偏差が $\sqrt{100 \times 0.1 \times 0.9} = 3$ であり，合計の需要である $1000 + 90K$ 台は平均 1900 台となる．平均より標準偏差 3 つ分上の $K \leq 19$ を賄える 2710 台のサーバがあれば，需要が容量を超える確率はわずか 0.2 % 程度である．プロバイダに必要なサーバは，顧客が自前で所有する場合の台数の 30 % に満たない．

**注意** ここで効いている仮定は独立性である．現実の需要は 1 日の周期，休日，ライブ配信などを共有するので，ピークは実際に同時に訪れる．予備の容量はそのためにあり，スポットインスタンスが 2 分前の通知で回収されるのもそのためである．

第 2 の原理は**規模の経済**である．資源を提供する単位当たりの費用は，提供する量が増えるにつれて下がる．プロバイダは，ハードウェア，電力，ネットワーク容量をまとめて購入し，データセンタの固定費を多数のマシンに分散し，管理者 1 人に多数のマシンを管理させる．

▷ economies of scale

### 16.4.2 ユーティリティとしてのコンピューティング

クラウドコンピューティングの構想は，その技術より古い．1961 年，John McCarthy は，計算がいつか電話網のように公共のユーティリティとして組織されうること，そしてそのような計算ユーティリティが重要な新産業の基盤となりうることを示唆した．

**定義 16.14** (ユーティリティコンピューティング)**. ユーティリティコンピューティング**では，計算資源が電力や電話サービスと同じように提供される．すなわち，計測され，代替可能な–どの単位も他の単位と変わらない–商品として提供され，顧客は必要に応じてそれを消費し，利用に応じて支払う．

▷ utility computing

Armbrust ら [64] の用語では，ユーティリティコンピューティングとは，クラウドが従量課金で一般に公開されるときに販売するものである．現在のクラウドがこの構想を実現しているのは一部にとどまる．計算が完全に代替可能なユーティリティにはなっていないからである．その限界は次のとおりである．

**API のロックイン** プロバイダはそれぞれ独自の API を持ち，あるプロバイダ向けに書かれたアプリケーションは，変更なしに別のプロバイダへ移せない．

**ハードウェアへの依存** アプリケーションは，特定のプロセッサやアクセラレータといった特定のハードウェアに依存することがあり，それをすべてのプロバイダやインスタンス種別が提供するわけではない．

**レイテンシ** 資源は遠隔のデータセンタに存在し，それとのやり取りはすべてネットワークのレイテンシを伴う．

**プライバシとセキュリティ** データと計算は，顧客が所有も制御もせず，他の顧客と共有されるマシン上に存在する．

> **要点**
> クラウドコンピューティングは，共有され，プロバイダによって管理される資源を，API を通じて伸縮的に，利用に応じた課金で提供する．これが成り立つのは，多数の顧客の合計の需要が個々の顧客の需要よりはるかに変動しないから（大数の法則）であり，また単位当たりの費用が規模とともに下がるから（規模の経済）である．ユーティリティとしての計算という構想を実現しているのは一部にとどまる．

## 16.5 デプロイメントモデルとサービスモデル

クラウドは，誰が使えるか，そして何を提供するかによって異なる．**デプロイメントモデル**は，誰がクラウドを使うかを表す．

**出典** P4L4，クラウドのデプロイメントモデル，クラウドのサービスモデル；Mell と Grance [65]；Silberschatz ら ch. 1.

**パブリッククラウド** **パブリッククラウド**は，プロバイダが第三者の顧客，すなわちそのテナントのために運用し，テナントが資源を共有するクラウドである．

▷ public cloud

**プライベートクラウド** **プライベートクラウド**では，組織が，仮想化，API，自動化された資源割り当てといったクラウドの技術を，自分のインフラストラクチャに，自分のアプリケーションのために適用する．NIST の定義では，そのインフラストラクチャを第三者が運用し，組織の構外に置いてもよい．

▷ private cloud

**ハイブリッドクラウド** **ハイブリッドクラウド**は，2 つ以上の別個のクラウド（プライベート，コミュニティ，パブリック），典型的にはプライベートクラウドとパブリッククラウドを組み合わせる．組織は自分のプライベートクラウドでアプリケーションを動かし，フェイルオーバのため，自分の容量を超える需要の急増を吸収するため，あるいはテストのためにパブリッククラウドを使う．

▷ hybrid cloud

**コミュニティクラウド** **コミュニティクラウド**は，使命，セキュリティ要件，方針，コンプライアンスといった関心を共有する組織からなる特定のコミュニティが専用に使

▷ community cloud

うために用意されるクラウドである．政府機関や研究機関がその例である．コミュニティの組織の 1 つ以上，第三者，あるいはその両方が所有し運用し，構内にも構外にも置かれうる．

**サービスモデル**は，ハードウェアとソフトウェアのスタックのうちどこまでをプロバイダが管理するかを表す（図 16.4）．オンプレミスの設置では，組織がスタック全体を自分で管理する．ネットワーク，ストレージ，サーバから，仮想化，オペレーティングシステム，ミドルウェア，ランタイムを経て，自分のデータとアプリケーションまでである．

**注意** 講義はコミュニティクラウドを，ある種類の利用者に利用が限られたパブリッククラウドとして説明するが，NIST [65] はそれをパブリッククラウドの一種とはしていない．

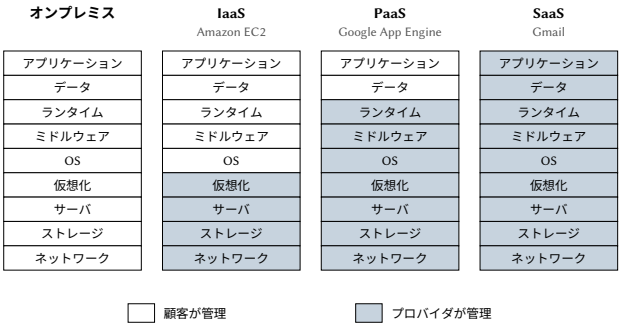



**図 16.4:** クラウドのサービスモデルと，それぞれの例．網かけの層はプロバイダが，それ以外の層は顧客が管理する．

▷ infrastructure as a service

**IaaS** では，プロバイダが物理的なインフラストラクチャ，すなわちネットワーク，ストレージ，サーバと，その仮想化を管理する．顧客は仮想マシンを借り，オペレーティングシステムとそれより上のすべてを自分で設置し管理する．Amazon EC2 がその例である．

▷ platform as a service

**PaaS** では，プロバイダがオペレーティングシステム，ミドルウェア，ランタイム環境も管理する．顧客は，プラットフォームのインタフェースに対して書かれたアプリケーションとそのデータだけを与え，プラットフォームがアプリケーションを配備して実行する．Google App Engine がその例である．

▷ software as a service

**SaaS** では，プロバイダがアプリケーションを含むスタック全体を管理し，顧客は Gmail の利用者のようにアプリケーションを使うだけである．

NIST SP 800-145 [65] が挙げるサービスモデルはちょうど 3 つ（IaaS，PaaS，SaaS），デプロイメントモデルは 4 つ（パブリック，プライベート，コミュニティ，ハイブリッド）である．

**例 16.15.** 16.1 節の三層のオンラインショップの運用者は，仮想マシンを借り，そこにオペレーティングシステム，Web サーバ，アプリケーションサーバ，データベースサーバを設置し，これらのソフトウェアとその更新およびスケーリングをすべて管理することで，IaaS 上でショップを運用できる．PaaS 上では，運用者はビジネスロジックのコードと静的コンテンツだけをアップロードする．プラットフォームがそれらを実行し，管理されたデータベースを提供し，負荷が増えればインスタンスを追加する．同時にショップの従業員は，SaaS として提供される電子メールを読むかもしれない．そこで彼らが管理するのは，自分のアカウントとメッセージだけである．

## 16.6 クラウドへの要件

プロバイダの視点から見ると，クラウドコンピューティングは基盤となる技術にいくつもの要件を課す．

**出典** P4L4，クラウドへの要件，クラウドにおける障害の確率；OSTEP ch. 48.

1. **代替可能な資源．**資源は互いに交換可能であり，異なるオペレーティングシステムやソフトウェアスタックのように異なる要件を持つ顧客のために，容易に転用できなければならない．代替可能性がなければ，すべての顧客が専用の資源を必要とし，共有に立脚するクラウドの経済的な機会（16.4 節）は存在しないことになる．
2. **伸縮的で動的な資源割り当て．**機構と方針は，需要に応じて資源を割り当て，解放されたときにはそれを回収しなければならない．
3. **規模．**管理と資源割り当ては，数千台規模のマシンからなるデータセンタと，多数の顧客に対して機能しなければならない．
4. **障害への対処．**この規模では障害は通常の事象であり，システムはそれにもかかわらず動作を続けなければならない．[1]
5. **マルチテナンシ．**テナントは互いに隔離されなければならず，各テナントの性能が保証されなければならない．

[1] Google が自社のディスク 100 000 台以上を調べた結果では，年間故障率は 1 年目の個体で 1.7 %，3 年目の個体では 8 % を超えた [68]．年 5 % とすれば，100 000 台のディスクからは 1 日におよそ 14 台が失われる．マシン，スイッチ，電源はまだ数えていない．

6. **セキュリティ．**各テナントのデータと計算は，他のテナントからも，クラウドの外部の攻撃者からも保護されなければならない．

**定義 16.16** (マルチテナンシ)**.** 複数の顧客が同じ物理資源を共有し，各顧客が自分の隔離された資源の集合を使うことを，**マルチテナンシ**と呼ぶ．

▷ multi-tenancy

マルチテナンシは，VMM が VM に対して提供するのと同様のテナントの状態の隔離（第 12 章）を必要とし，さらに性能の隔離も必要とする．

**定義 16.17** (性能隔離)**.** あるテナントが得る性能が，資源を共有する他のテナントのワークロードに依存しないとき，そのシステムは**性能隔離**を提供するという．

▷ performance isolation

性能隔離がなければ，ディスクやメモリを飽和させる VM と同じマシンを共有するテナントの VM は予測できない応答時間を観測し，プロバイダは約束した SLA（第 6 章）を守れない．

**ヒント** 同じマシンに同居するテナントによる性能への干渉は「うるさい隣人」（noisy neighbour）問題と呼ばれる．それを防ぐのが性能隔離である．

### 16.6.1 規模がもたらす障害

障害は，構成要素の数が多いというだけの理由で頻繁に起こる．$N$ 個の構成要素の各々が，ある期間内に独立に確率 $p$ で故障するとすれば，そのうち少なくとも 1 つが故障する確率は

$$P_{\text{fail}} \;=\; 1-(1-p)^N, \tag{16.1}$$

であり，$N$ が大きくなるにつれて 1 に近づく．一方，故障する構成要素の期待個数 $Np$ は $N$ に比例して増える．したがって，単体ではめったに故障しない構成要素も，データセンタのどこかでは常に故障していることになる．クラウドのシステムは，たとえばタイムアウトによって障害を検出し，それにもかかわらず，これまでの章で扱った機構によって動作を続けなければならない．故障した複製を引き継ぐ複製（第 14 章），計算を再開できるチェックポイント（第 8 章），故障しつつあるマシンから退避させる VM のマイグレーション（第 12 章）である．

**注意** 式 (16.1) は構成要素が独立に故障することを仮定している．相関のある故障—電源や冷却の喪失，ネットワーク分断，欠陥のあるソフトウェア更新—は多数のマシンを一度に停止させ，実際に大規模障害を引き起こすのはそれらである．

**ヒント** $p$ が小さければ $(1-p)^N \approx e^{-Np}$ であり，$P_{\text{fail}} \approx 1-e^{-Np}$ となる．$Np \ll 1$ の間はおよそ $Np$ に等しく，$Np \geq 3$ となれば 95 % を超える．

**例 16.18.** あるマシンが，他のマシンとは独立に，ある 1 日のうちに確率 $p = 0.001$ で故障するとする．単体では平均しておよそ 3 年に 1 度故障する計算である．式 (16.1) により，ある 1 日に少なくとも 1 台のマシンが故障する確率は，10 台では 1.0 %，100 台では 9.5 %，1000 台では 63 % である．10 000 台では事実上 100 % であり，平均して毎日 10 台が故障する．

> **要点**
> クラウドは，代替可能な資源，伸縮的な割り当て，規模に耐える管理，障害への耐性，そしてテナントの状態と性能の隔離を必要とする．構成要素が多ければ，ほとんど常にどこかの構成要素が故障している（式 (16.1)）．

## 16.7 クラウドを可能にする技術

いくつもの技術が合わさってクラウドコンピューティングを可能にしている．

**出典** P4L4，クラウドを可能にする技術；Hindman ら [69]；Dean と Ghemawat [70]；Zaharia ら [71]；McKeown ら [72]．

**仮想化．** 仮想化（第 12 章）は資源を代替可能にする．1 台の物理マシンは，どの顧客の VM でも，どのようなオペレーティングシステムやソフトウェアスタックのものでも載せることができ，VM はテナントを互いに隔離する．VM は需要の変化に応じて生成，破棄，マイグレートできる．

**資源の割り当て．** Mesos [69] や Hadoop YARN のようなクラスタの資源マネージャは，多数のマシンの資源を，その上で動作するフレームワークに割り当て，Mesos の場合は提供（オファー）し，フレームワークがそれらの資源の上に自分のタスクを配置する（2 段階スケジューリング）．

**ビッグデータ処理．** Hadoop MapReduce や Spark のようなフレームワークは，大規模なデータ集合を多数のマシン上で並列に処理する．

Hadoop は，Google が記述したスタックのオープンソース版に当たる．HDFS は Google File System に，Hadoop MapReduce は MapReduce に，HBase は Bigtable に対応する．

**定義 16.19** (MapReduce)**.** **MapReduce** [70] とは，クラスタ上で大規模なデータ集合を処理するためのプログラミングモデルであり，またそれを実装するランタイムシステムである．プログラマは，各入力レコードを中間的なキーと値の組へ変換する *map* 関数と，

▷ MapReduce

同じキーを持つすべての中間の値を結合する *reduce* 関数を書く．ランタイムは入力を分割し，map タスクと reduce タスクを多数のマシン上で並列に実行し，中間の組をキーごとにまとめ，故障したマシンのタスクを再実行する．

Dean と Ghemawat [70] によれば，2004 年 8 月に Google では 29 423 件の MapReduce ジョブが実行され，読み込んだ入力は合計 3288 TB であった．

たとえば，文書の集まりの中で各単語が何回現れるかを数えるには，map 関数が単語 $w$ の出現ごとに組 $(w, 1)$ を出力し，reduce 関数が単語ごとに値を合計する．MapReduce は中間結果をディスクに格納するが，Spark [71] はそれをメモリに保持するので，同じデータを再利用する反復的なアルゴリズムや対話的な問い合わせが高速になる．

こうしたシステムは大きな単位で扱う．GFS はチャンクを 64 MB，HDFS はブロックを 128 MB とし，既定ではそれぞれを 3 台のマシンに置く．走査は逐次的になり，1 台を失ってもデータは失われない．

**ストレージ．** クラウドはデータを，上書きよりも追記が中心の非常に大きなファイルのために設計された分散ファイルシステム（第 14 章）に，関係データモデルと従来のデータベースの一貫性保証の一部を手放すことでスケールアウトする NoSQL データベースに，そして多数のマシンのメモリにデータを保持する分散インメモリストアやキャッシュに格納する．

**ソフトウェア定義のネットワーク，ストレージ，データセンタ．** ネットワーク，ストレージ，そしてやがてはデータセンタ全体が，手作業ではなく API を通じてソフトウェアによって構成される．

▷ software-defined networking

**定義 16.20** (ソフトウェア定義ネットワーキング)**.** **SDN** では，トラフィックをどう転送するかを決めるネットワークの**制御プレーン**が，パケットを転送するスイッチの**データプレーン**から分離される．ソフトウェアで実装され論理的に集中化されたコントローラが，OpenFlow [72] のような開かれたインタフェースを通じてスイッチの転送表をプログラムする．

SDN によって，クラウドはネットワークをソフトウェアで再構成できる．たとえば，各テナントに，他のテナントのものから隔離された専用の仮想ネットワークを与えられる．ソフトウェア定義ストレージとソフトウェア定義データセンタは，同じ原理をストレージと，データセンタのすべての資源に適用する．

OpenFlow は 2008 年に仕様が定められ，この種のインタフェースとして最初に広く実装されたものであり，コントローラがスイッチの転送表にマッチ・アンド・アクションの規則を設定できるようにする．

**監視．** 数千台のマシンと，その上で動作するサービスは，ログと測定値を絶えず生み出す．Flume，CloudWatch，Log Insight のようなリアルタイムのログ処理システムは，障害，異常，過負荷を検出し，伸縮的なスケーリングを起動するために，それらを生成されるそばから収集し分析する．

## 16.8 ビッグデータエンジンとしてのクラウド

大規模なインターネットサービスの利用者活動のログ，索引付けすべき Web ページ，科学的な測定値など，単一のマシンには収まらないデータ集合を分析するアプリケーションは多い．

**出典** P4L4，ビッグデータエンジンとしてのクラウド，ビッグデータスタックの例．

**定義 16.21** (ビッグデータ)**.** 単一のマシンが格納し処理できる大きさを超え，多数のマシン上で格納し分析しなければならないデータ集合を，**ビッグデータ**と呼ぶ．

▷ big data

クラウドは，そうしたデータのストレージと，それを並列に処理する多数のマシンの両方を提供する．それを行うソフトウェアは，次の層からなる**ビッグデータエンジン**を成す（図 16.5）．

**データストレージ** データを多数のマシンに分散し複製して格納する．
**データ処理** データに対する計算を多数のマシン上で並列に実行する．
**キャッシュ** 頻繁に使うデータをメモリに保持し，ストレージ層のレイテンシを避ける．
**言語フロントエンド** 利用者が処理層を直接プログラムする代わりに，SQL 風の問い合わせ言語のような高水準の言語でデータを問い合わせ分析できるようにする．
**分析ライブラリ** 機械学習，グラフ分析，統計などのために，処理層の上に構築されたアルゴリズムを提供する．
**ストリームデータ** ログ，クリック，センサの測定値といったデータは絶えずエンジンへ入ってくるので，格納された後だけでなく，到着するそばから処理されなければならない．

各層は桁違いに離れている．メモリ参照は約 100 ns，ディスクのシークは約 10 ms であり [66]，フラッシュ SSD からの 4 kB のランダム読み出しは約 100 µs to 150 µs である（同じ一覧の後の拡張版）．

広く使われている 2 つのオープンソースのスタックが，これらの層を実装している（表 16.2）．*Hadoop* スタックは，Hadoop Distributed File System（HDFS）と MapReduce を中心に構築されている．カリフォルニア大学バークレー校で開発された *Berkeley Data Analytics Stack*（BDAS）は，Spark とクラスタの資源マネージャ Mesos を中心に構築されており，その下で HDFS や他のストレージシステムを利用できる．

いくつかの構成要素はその後に改称あるいは置き換えられている．Tachyon は 2016 年に Alluxio となり，Shark は Spark SQL に取って代わられた．表 16.2 は元の名称のままである．

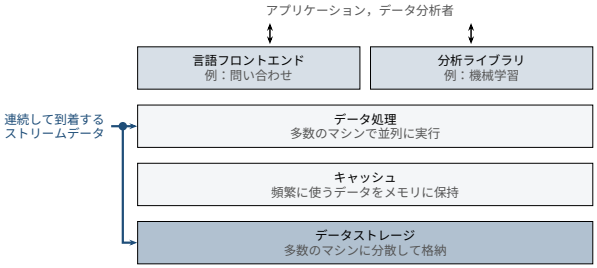



**図 16.5:** ビッグデータエンジンの層．ストリームデータは絶えずエンジンへ入り，到着するそばから処理され，かつ格納される．

**表 16.2:** 2 つのビッグデータスタックの構成要素（層ごと）．

| 層 | Hadoop | BDAS |
|---|---|---|
| データストレージ | HDFS（ファイルシステム），HBase（列指向ストア） | HDFS，Amazon S3 その他のストレージシステム |
| キャッシュ | — | Tachyon（インメモリストレージ） |
| 資源管理 | YARN | Mesos，YARN |
| データ処理 | MapReduce | Spark |
| ストリームデータ | Flume（ログ収集） | Spark Streaming |
| 言語フロントエンド | Hive（SQL），Pig（データフロー） | Shark（SQL），BlinkDB（近似問い合わせ） |
| 分析ライブラリ | Mahout（機械学習），R コネクタ（統計） | MLlib，MLbase（機械学習），GraphX（グラフ） |
| 協調 | ZooKeeper（協調），Oozie（ワークフロー），Sqoop（関係データベースとのデータ交換） | — |

**要点 — 章のまとめ**

インターネットサービスは，プレゼンテーション，ビジネスロジック，データベースの層からなり，ロードバランサの背後で多数のノードへスケールアウトされる．ホモジニアスなノードは管理が単純であり，ヘテロジニアスなノードは局所性を活かせるが，要求を調べるフロントエンドと需要に従う管理を要する．クラウドコンピューティングは，共有された資源を API を通じて伸縮的に，利用に応じた課金で提供し，プロバイダが管理する．これが成り立つのは大数の法則と規模の経済のゆえであり，ユーティリティコンピューティングの構想に近づく．クラウドはパブリック，プライベート，ハイブリッド，コミュニティのいずれかであり，インフラストラクチャ，プラットフォーム，ソフトウェアをサービスとして提供する．クラウドが要するのは，代替可能な資源，規模に耐える伸縮的な割り当て，頻繁な障害への耐性，テナントの隔離であり，それらを与えるのが仮想化，クラスタの資源管理，MapReduce のようなビッグデータフレームワーク，分散ストレージ，ソフトウェア定義ネットワーキング，監視である．これらの技術の上で，クラウドはビッグデータエンジンとして働く．

# 参考文献


[1] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. 1.10. Arpaci-Dusseau Books, 2023. URL: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[2] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. 10th ed. Wiley, 2018.

[3] Livio Soares and Michael Stumm. "FlexSC: Flexible System Call Scheduling with Exception-Less System Calls". In: *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2010, pp. 33–46.

[4] Gerwin Klein et al. "seL4: Formal Verification of an OS Kernel". In: *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*. 2009, pp. 207–220.

[5] Jochen Liedtke. "Improving IPC by Kernel Design". In: *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP)*. 1993, pp. 175–188.

[6] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. 4th ed. Pearson, 2015.

[7] Chuanpeng Li, Chen Ding, and Kai Shen. "Quantifying the Cost of Context Switch". In: *Proceedings of the 2007 Workshop on Experimental Computer Science (ExpCS)*. San Diego, CA: ACM, June 2007.

[8] Andrew D. Birrell. *An Introduction to Programming with Threads*. SRC Research Report 35. Palo Alto, CA: Digital Equipment Corporation, Systems Research Center, Jan. 1989.

[9] Ulrich Drepper and Ingo Molnar. *The Native POSIX Thread Library for Linux*. White paper. Red Hat, Inc., Feb. 2005.

[10] Butler W. Lampson and David D. Redell. "Experience with Processes and Monitors in Mesa". In: *Communications of the ACM* 23.2 (1980), pp. 105–117.

[11] Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco: No Starch Press, 2010.

[12] *IEEE Standard for Information Technology—Portable Operating System Interface (POSIX) Base Specifications, Issue 7*. IEEE Std 1003.1-2017. IEEE and The Open Group. 2018.

[13] Ulrich Drepper. *Futexes Are Tricky*. Revision 1.6. 2011. URL: https://www.akkadia.org/drepper/futex.pdf.

[14] J. R. Eykholt et al. "Beyond Multiprocessing: Multithreading the SunOS Kernel". In: *Proceedings of the Summer 1992 USENIX Technical Conference*. San Antonio, TX, June 1992.

[15] Dan Stein and Devang Shah. "Implementing Lightweight Threads". In: *Proceedings of the Summer 1992 USENIX Technical Conference*. San Antonio, TX, June 1992.

[16] Thomas E. Anderson et al. "Scheduler Activations: Effective Kernel Support for the User-Level Management of Parallelism". In: *ACM Transactions on Computer Systems* 10.1 (Feb. 1992), pp. 53–79.

[17] Vivek S. Pai, Peter Druschel, and Willy Zwaenepoel. "Flash: An Efficient and Portable Web Server". In: *Proceedings of the 1999 USENIX Annual Technical Conference*. Monterey, CA: USENIX Association, June 1999, pp. 199–212.

[18] Jeffrey Dean and Luiz André Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (Feb. 2013), pp. 74–80.

[19] Alexandra Fedorova et al. "Chip Multithreading Systems Need a New Operating System Scheduler". In: *Proceedings of the 11th ACM SIGOPS European Workshop*. 2004.

[20] Fernando J. Corbató, Marjorie Merwin-Daggett, and Robert C. Daley. "An Experimental Time-Sharing System". In: *Proceedings of the AFIPS Spring Joint Computer Conference*. 1962.

[21] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. 3rd ed. O'Reilly Media, 2005.

[22] Alexandra Fedorova et al. "Performance of Multithreaded Chip Multiprocessors and Implications for Operating System Design". In: *Proceedings of the USENIX Annual Technical Conference*. 2005.

[23] Jeff Bonwick. "The Slab Allocator: An Object-Caching Kernel Memory Allocator". In: *Proceedings of the USENIX Summer 1994 Technical Conference*. 1994, pp. 87–98.

[24] Christopher Clark et al. "Live Migration of Virtual Machines". In: *Proceedings of the 2nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '05)*. 2005, pp. 273–286.

[25] Edsger W. Dijkstra. *Cooperating Sequential Processes*. Technical Report EWD-123. Eindhoven, The Netherlands: Technological University Eindhoven, 1965.

[26] C. A. R. Hoare. "Monitors: An Operating System Structuring Concept". In: *Communications of the ACM* 17.10 (1974), pp. 549–557.

[27] Carl E. Hewitt and Russell R. Atkinson. "Specification and Proof Techniques for Serializers". In: *IEEE Transactions on Software Engineering* SE-5.1 (1979), pp. 10–23.

[28] Roy H. Campbell and A. Nico Habermann. "The Specification of Process Synchronization by Path Expressions". In: *Operating Systems, International Symposium*. Vol. 16. Lecture Notes in Computer Science. Springer, 1974, pp. 89–102.

[29] Maurice Herlihy. “Wait-Free Synchronization”. In: *ACM Transactions on Programming Languages and Systems* 13.1 (1991), pp. 124–149.

[30] Paul E. McKenney and John D. Slingwine. “Read-Copy Update: Using Execution History to Solve Concurrency Problems”. In: *Proceedings of the International Conference on Parallel and Distributed Computing and Systems (PDCS)*. Las Vegas, NV, 1998.

[31] Thomas E. Anderson. “The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors”. In: *IEEE Transactions on Parallel and Distributed Systems* 1.1 (1990), pp. 6–16.

[32] Gerald J. Popek and Robert P. Goldberg. “Formal Requirements for Virtualizable Third Generation Architectures”. In: *Communications of the ACM* 17.7 (1974), pp. 412–421.

[33] Mendel Rosenblum and Tal Garfinkel. “Virtual Machine Monitors: Current Technology and Future Trends”. In: *Computer* 38.5 (2005), pp. 39–47.

[34] Luiz André Barroso and Urs Hölzle. *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2009.

[35] Paul Barham et al. “Xen and the Art of Virtualization”. In: *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*. Bolton Landing, NY, 2003, pp. 164–177.

[36] Avi Kivity et al. “kvm: the Linux Virtual Machine Monitor”. In: *Proceedings of the Linux Symposium*. Vol. 1. Ottawa, Canada, 2007, pp. 225–230.

[37] John Scott Robin and Cynthia E. Irvine. “Analysis of the Intel Pentium’s Ability to Support a Secure Virtual Machine Monitor”. In: *Proceedings of the 9th USENIX Security Symposium*. Denver, CO, Aug. 2000.

[38] Edouard Bugnion et al. “Bringing Virtualization to the x86 Architecture with the Original VMware Workstation”. In: *ACM Transactions on Computer Systems* 30.4 (2012), 12:1–12:51.

[39] Keith Adams and Ole Agesen. “A Comparison of Software and Hardware Techniques for x86 Virtualization”. In: *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. San Jose, CA, 2006, pp. 2–13.

[40] *Single Root I/O Virtualization and Sharing Specification, Revision 1.0*. PCI-SIG. 2007.

[41] Rusty Russell. “virtio: Towards a De-Facto Standard for Virtual I/O Devices”. In: *ACM SIGOPS Operating Systems Review* 42.5 (2008), pp. 95–103.

[42] Rich Uhlig et al. “Intel Virtualization Technology”. In: *Computer* 38.5 (2005), pp. 48–56.

[43] Advanced Micro Devices. *AMD-V Nested Paging*. White paper. Advanced Micro Devices, Inc., July 2008.

[44] Andrew D. Birrell and Bruce Jay Nelson. “Implementing Remote Procedure Calls”. In: *ACM Transactions on Computer Systems* 2.1 (Feb. 1984), pp. 39–59.

[45] Robert Thurlow. *RPC: Remote Procedure Call Protocol Specification Version 2*. RFC 5531. Internet Engineering Task Force, May 2009.

[46] Mike Eisler. *XDR: External Data Representation Standard*. RFC 4506. Internet Engineering Task Force, May 2006.

[47] R. Srinivasan. *Binding Protocols for ONC RPC Version 2*. RFC 1833. Internet Engineering Task Force, Aug. 1995.

[48] Thomas Haynes and David Noveck. *Network File System (NFS) Version 4 Protocol*. RFC 7530. Internet Engineering Task Force, Mar. 2015.

[49] Mike Eisler, Alex Chiu, and Lin Ling. *RPCSEC_GSS Protocol Specification*. RFC 2203. Internet Engineering Task Force, Sept. 1997.

[50] David Noveck and Charles Lever. *Network File System (NFS) Version 4 Minor Version 1 Protocol*. RFC 8881. Internet Engineering Task Force, Aug. 2020.

[51] US-CERT. *UDP-Based Amplification Attacks*. Alert TA14-017A. Jan. 2014. URL: https://www.cisa.gov/news-events/alerts/2014/01/17/udp-based-amplification-attacks.

[52] Ann Wollrath, Roger Riggs, and Jim Waldo. “A Distributed Object Model for the Java System”. In: *Proceedings of the 2nd USENIX Conference on Object-Oriented Technologies and Systems (COOTS)*. Toronto, Canada, June 1996.

[53] Michael N. Nelson, Brent B. Welch, and John K. Ousterhout. “Caching in the Sprite Network File System”. In: *ACM Transactions on Computer Systems* 6.1 (Feb. 1988), pp. 134–154.

[54] Russel Sandberg et al. “Design and Implementation of the Sun Network Filesystem”. In: *Proceedings of the Summer 1985 USENIX Conference*. Portland, OR, June 1985, pp. 119–130.

[55] Brent Callaghan, Brian Pawlowski, and Peter Staubach. *NFS Version 3 Protocol Specification*. RFC 1813. Internet Engineering Task Force, June 1995.

[56] John K. Ousterhout et al. “A Trace-Driven Analysis of the UNIX 4.2 BSD File System”. In: *Proceedings of the Tenth ACM Symposium on Operating Systems Principles (SOSP)*. Orcas Island, WA, Dec. 1985, pp. 15–24.

[57] Jelica Protić, Milo Tomašević, and Veljko Milutinović. “Distributed Shared Memory: Concepts and Systems”. In: *IEEE Parallel & Distributed Technology: Systems & Applications* 4.2 (1996), pp. 63–79.

[58] Bill Nitzberg and Virginia Lo. “Distributed Shared Memory: A Survey of Issues and Algorithms”. In: *Computer* 24.8 (Aug. 1991), pp. 52–60.

[59] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms*. 2nd ed. Pearson Prentice Hall, 2007.

[60] Kai Li and Paul Hudak. “Memory Coherence in Shared Virtual Memory Systems”. In: *ACM Transactions on Computer Systems* 7.4 (Nov. 1989), pp. 321–359.

[61] Pete Keleher et al. “TreadMarks: Distributed Shared Memory on Standard Workstations and Operating Systems”. In: *Proceedings of the Winter 1994 USENIX Conference*. 1994, pp. 115–131.

[62] Andrew W. Appel and Kai Li. “Virtual Memory Primitives for User Programs”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*. 1991, pp. 96–107.

[63] Leslie Lamport. “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs”. In: *IEEE Transactions on Computers* C-28.9 (Sept. 1979), pp. 690–691.

[64] Michael Armbrust et al. *Above the Clouds: A Berkeley View of Cloud Computing*. Technical Report UCB/EECS-2009-28. EECS Department, University of California, Berkeley, Feb. 2009.

[65] Peter Mell and Timothy Grance. *The NIST Definition of Cloud Computing*. Special Publication 800-145. National Institute of Standards and Technology, Sept. 2011.

[66] Jeffrey Dean. *Designs, Lessons and Advice from Building Large Distributed Systems*. Keynote, 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems and Middleware (LADIS), Big Sky, MT. Oct. 2009.

[67] David Karger et al. “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web”. In: *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC)*. El Paso, TX, May 1997, pp. 654–663.

[68] Eduardo Pinheiro, Wolf-Dietrich Weber, and Luiz André Barroso. “Failure Trends in a Large Disk Drive Population”. In: *Proceedings of the 5th USENIX Conference on File and Storage Technologies (FAST)*. San Jose, CA, Feb. 2007, pp. 17–28.

[69] Benjamin Hindman et al. “Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center”. In: *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. Boston, MA, 2011.

[70] Jeffrey Dean and Sanjay Ghemawat. “MapReduce: Simplified Data Processing on Large Clusters”. In: *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. San Francisco, CA, Dec. 2004, pp. 137–150.

[71] Matei Zaharia et al. “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing”. In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. San Jose, CA, 2012.

[72] Nick McKeown et al. “OpenFlow: Enabling Innovation in Campus Networks”. In: *ACM SIGCOMM Computer Communication Review* 38.2 (Apr. 2008), pp. 69–74.

# 索引


**A**
AMPED, → 非対称マルチプロセスイベント駆動モデル
AMTED, → 非対称マルチスレッドイベント駆動モデル
Apache, 66

**B**
broadcast 操作, 25

**C**
CFS, 79
clone, 57
CMT, → チップマルチスレッディング
Completely Fair Scheduler, → CFS
COW, → コピーオンライト
CPI, 83
CPU 使用率, 59
CPU スケジューラ, 16
CPU バウンドタスク, 75
cycles per instruction, → CPI

**D**
dentry, → ディレクトリエントリ
dentry キャッシュ, 131
DFS, → 分散ファイルシステム
direct memory access, → DMA
DMA, 126, 174
dom0, 138
DSM, → 分散共有メモリ

**E**
EPT, → 拡張ページテーブル
`exec`, 15
ext2, 131

**F**
FCFS, → 先着順スケジューリング
Flash, 64
`fork`, 15
`fork`, 97

**I**
i ノード, 131
IaaS, 190
IDL, → インタフェース定義言語
infrastructure as a service, → IaaS
`init`, 15
I/O スケジューリング, 134
I/O バウンドタスク, 75
I/O ポート, 125
`ioctl`, 129
IPC, → プロセス間通信

**J**
Java RMI, → リモートメソッド呼び出し

**K**
Kernel-based Virtual Machine, → KVM
KVM, 139

**L**
least recently used, → LRU
LPC, → ローカル手続き呼び出し
LRU, 96
LWP, → 軽量プロセス

**M**
MapReduce, 191
memory management unit, → MMU
MLFQ, → 多段フィードバックキュー
MMU, 86
MRMW, → 複数リーダ複数ライタ
MRSW, → 複数リーダ単一ライタ
MSI, → メッセージシグナル割り込み

**N**
Native POSIX Thread Library, → NPTL
Network File System, → NFS
NFS, 167
nice 値, 78
non-uniform memory access, → NUMA
NPTL, 57
NUMA, 81
NUMA 対応スケジューリング, 81

**O**
*O*(1) スケジューラ, 78
ONC RPC, → Sun RPC
OS, → オペレーティングシステム
OS バイパス, 128, 144, 174

**P**
PaaS, 190
PC, → プログラムカウンタ
PCB, → プロセス制御ブロック
PCI, 124
Peripheral Component Interconnect, → PCI
PFN, → 物理フレーム番号
PIO, → プログラム制御 I/O
platform as a service, → PaaS
Portable Operating System Interface, → POSIX
portmapper, → rpcbind
POSIX, 34
POSIX 共有メモリ, 105
POSIX スレッド, 34
PTE, → ページテーブルエントリ
Pthreads, → POSIX スレッド

**Q**
QEMU, 139

**R**
RCU, 115
RDMA, 174
read-copy-update, → RCU
remote direct memory access, → RDMA

RMI, → リモートメソッド呼び出し
RPC, → 遠隔手続き呼び出し
RPC ランタイム, 149
rpcbind, 155
rpcgen, 154

**S**
SaaS, 190
SDN, 192
service level agreement, → SLA
signal 操作, 25
SJF, → 最短ジョブ優先
SLA, 60, 191
SMT, → 同時マルチスレッディング
software as a service, → SaaS
software-defined networking, → SDN
SPED, → 単一プロセスイベント駆動モデル
Sprite, 169
SRSW, → 単一リーダ単一ライタ
Sun RPC, 153, 167
System V IPC, 103

**T**
TLB, 90
translation lookaside buffer, → TLB

**U**
UNIX セマンティクス, 164
Unix ドメインソケット, 102

**V**
VFS, → 仮想ファイルシステム
VM, → 仮想マシン
VM entry, 140
VM exit, 139
VM 制御構造, 146
VMCS, → VM 制御構造
VMM, → 仮想マシンモニタ
VPN, → 仮想ページ番号

**W**
wait 操作, 24

**X**
XDR, → 外部データ表現
Xen, 138

**Z**
Zygote, 15

**ア**
赤黒木, 79
アクセスビット, 88, 179
アクティブ配列, 78
アタッチ, 104
アップロード・ダウンロードモデル, 161
アトミック命令, 116
アドレス空間, 10
アドレス変換, 85
アンマーシャリング, 151
一貫性モデル, 181
一対一モデル, 30
イベント駆動モデル, 61
イベントディスパッチャ, 61
イベントハンドラ, 61
因果一貫性, 181
インターネットサービス, 184
インタフェース定義言語, 150
ウェイトフリー同期, 115
エージング, 72
エレベータアルゴリズム, 134
遠隔アクセスモデル, 161
遠隔手続き呼び出し, 148
エントリ手続き, 113
応答時間, 59
オフセット, 87
オペレーティングシステム, 1
親プロセス, 14

**カ**
カーネル, 3
カーネルモード, 3
カーネルモジュール, 7
カーネルレベルスレッド, 30
階層化パターン, 32
外部断片化, 93
外部データ表現, 153
書き込み更新, 118
書き込み無効化, 118
拡張ページテーブル, 146
隔離, 2, 191
仮想アドレス, 11
仮想化, 136
仮想実行時間, 79
仮想ファイルシステム, 130
仮想ページ番号, 87
仮想マシン, 136
仮想マシンモニタ, 136
仮想メモリ, 85
間接ポインタ, 133
完全仮想化, 141
偽共有, 175
期限切れ配列, 78
機構, 2
機構と方針の分離, 3
疑似デバイス, 125
規模の経済, 189
逆ページテーブル, 91
キャッシュアフィニティ, 80
キャッシュ一貫性, 118, 164, 175
キャラクタデバイス, 124
キューイングロック, 120
競合状態, 22, 38
共有メモリ, 18, 102
共有メモリオブジェクト, 105
共有メモリ型マルチプロセッサ, 80, 117, 173
共有メモリセグメント, 104
クラウドコンピューティング, 188
クリティカルセクション, 23
close-to-open 一貫性, 168
計数セマフォ, 111
軽量プロセス, 47
ゲスト OS, 136

厳密一貫性, 181
コールドキャッシュ, 13
合成ワークロード, 67
コピーオンライト, 97
子プロセス, 14
コミュニティクラウド, 189
コンテキストスイッチ, 13
コンペアアンドスワップ, 116

**サ**
サーバ統合, 137
サービス VM, 138
最短ジョブ優先, 71
シグナル, 4
シグナルハンドラ, 52
シグナルマスク, 53
システムコール, 4
システムスコープ, 31, 35
実行可能状態, 14
実行完了型スケジューリング, 71
実行時間, 59
失効したファイルハンドル, 168
実行状態, 14
ジャーナリング, 134
ジャーナル, 134
弱一貫性, 181
シャドウページテーブル, 142
終了状態, 14
準仮想化, 142
ジョイン, 23
ジョイン可能スレッド, 36
条件変数, 24, 41
初回アクセス時の割り当て, 87
シリアライザ, 114
新規状態, 14
伸縮性, 188
スーパーブロック, 131
スケールアウト, 185
スタック, 10
スタックポインタ, 12
スタブ, 149
スタベーション, 72
ステートフルサーバ, 162
ステートレスサーバ, 162
ストリームソケット, 102
スピンロック, 110
スプリアスウェイクアップ, 28, 41
スラブアロケータ, 94
スループット, 59
スレッド, 20
スレッド生成, 22
スレッド属性, 35
スレッドプール, 31
スワッピング, 95
スワップ領域, 95
生産者・消費者問題, 24, 41
性能隔離, 191
性能指標, 59
セカンドチャンスアルゴリズム, 96
セグメンテーション, 85
セグメント, 85
世代番号, 167
セッションセマンティクス, 164
セマフォ, 111
先着順スケジューリング, 71
相互排除, 23
ソケット, 102

**タ**
ダーティビット, 88, 179
大数の法則, 188
タイムシェアリング, 74
タイムスライス, 16, 74
代理変数, 26
タグ付き TLB, 146
タスク, 56
多層アーキテクチャ, 184
多対一モデル, 30
多対多モデル, 30
多段フィードバックキュー, 77
多段ページテーブル, 89
ダチョウアルゴリズム, 29
単一プロセスイベント駆動モデル, 61
単一リーダ単一ライタ, 175
チェックポインティング, 97, 163, 191
遅延書き込み, 169
逐次一貫性, 181
逐次書き込み共有, 170
チップマルチスレッディング, 82
忠実性, 137
抽象化, 1
調停, 2
直接ポインタ, 132
直列化, → マーシャリング
データグラムソケット, 102
データセグメント, 10
データセンタ, 184
ディスパッチ, 16
ディレクトリエントリ, 131
適応型ミューテックス, 50, 111
テキストセグメント, 10
テストアンドセット, 116
テストアンドテストアンドセット, 119
テストベッド, 59
デタッチ, 104
デタッチスレッド, 36
デッドロック, 28
デッドロックの検出と回復, 29
デッドロック防止, 29
デバイスドライバ, 124
デバイスファイル, 125
デバイスレジスタ, 123
デマンドページング, 95
デリゲーション, 168
同期, 22
同期 I/O, 128
同期シグナル, 52
同時マルチスレッディング, 81
特化, 21, 186
特権命令, 3
トップハーフ, 54
トラップ, 4
トラップアンドエミュレート, 140
トレース, 67

**ナ**
内部断片化, 92
名前付きパイプ, 101
二重間接ポインタ, 133
ネットワークデバイス, 124

**ハ**
パーティショニング, 166
ハードウェア支援仮想化, 146
ハードウェア性能カウンタ, 83
ハードプロセス状態, 46
バイナリセマフォ, 111
バイナリ変換, 141
ハイパーコール, 142
ハイパースレッディング, → 同時マルチスレッディング
ハイパーバイザ, → 仮想マシンモニタ
ハイパーバイザ直接モデル, 145
パイプ, 101
パイプラインパターン, 31
ハイブリッドクラウド, 189
バインディング, 151
バウンドスレッド, 30
パス式, 114
パススルーモデル, 144
バックエンドドライバ, 145
ハッシュページテーブル, 91
バッファキャッシュ, 134
バディアロケータ, 94
パブリッククラウド, 189
バリア, 115
汎用ブロック層, 129
ヒープ, 10
ピアツーピア, 160, 173
ビジーウェイト, 110
非対称マルチスレッドイベント駆動モデル, 64
非対称マルチプロセスイベント駆動モデル, 64
ビッグエンディアン, 149
ビッグデータ, 192
非同期 I/O, 63, 128
非同期シグナル, 52
ヒュージページ, 92
非ルートモード, 139
ピン留めページ, 95, 127
頻繁に起こる場合の最適化, 3, 166
ファイルシステム, 129
ファイルディスクリプタ, 130
ファイルハンドル, 167
負荷分散, 81
複数リーダ単一ライタ, 176
複数リーダ複数ライタ, 176
物理アドレス, 11
物理フレーム番号, 87, 178
部分障害, 152
不変ファイル, 165
プライベートクラウド, 189
プリエンプション, 16
プリエンプティブスケジューリング, 71
プリフェッチ, 134, 175
プログラムカウンタ, 12
プログラム制御 I/O, 126
プロセス, 10
プロセス間通信, 18, 100
プロセス共有属性, 106
プロセス状態, 14
プロセススコープ, 30, 35
プロセス制御ブロック, 12
プロセスマイグレーション, 98
ブロックグループ, 132
ブロックデバイス, 124
フロントエンドドライバ, 145
分割デバイスドライバモデル, 145
分散共有メモリ, 173
分散ファイルシステム, 160
ページ, 85
ページアウトデーモン, 96
ページキャッシュ, 134
ページ置換, 96
ページテーブル, 11, 87
ページテーブルエントリ, 87
ページテーブルベースレジスタ, 88
ページディレクトリ, 89
ページフォールト, 88, 179
ページフレーム, 85
ページング, 85
ベースライン, 69
ベアメタル型ハイパーバイザ, 138
並行書き込み共有, 170
並行性, 22
並列化, 21
ヘテロジニアス構成, 186
ヘルパ, 64
ベンチマーク, 68
ポート, 100
ポーリング, 126
方針, 2
保護, 2
保護ビット, 88, 179
保護リング, 139
ホスト OS, 138
ホスト型ハイパーバイザ, 138
ボス・ワーカーパターン, 31, 185
ホットキャッシュ, 13, 186
ボトムハーフ, 54
ホモジニアス構成, 185

**マ**
マーシャリング, 151
マイクロカーネル, 7
マウント, 167
マシンアドレス, 142
待ちグラフ, 28
待ち時間, 59
待ち状態, 14
マルチコアプロセッサ, 80
マルチスレッド, 20
マルチスレッドモデル, 61
マルチテナンシ, 191
マルチプロセスモデル, 61
ミューテックス, 23, 39
メッセージキュー, 101
メッセージシグナル割り込み, 51
メッセージパッシング, 18, 100
メモリアロケータ, 93

メモリバウンドタスク, 82
メモリマップド I/O, 125
メモリマップドファイル, 105
モードビット, 3
モジュール型 OS, 7
モニタ, 113
モノリシック OS, 6

**ヤ**
ユーザモード, 3
ユーザレベルスレッド, 30
ユーザレベルドライバ, 128
ユーティリティコンピューティング, 189
有界バッファ, 41
有効ビット, 88
優先度, 72
優先度逆転, 73
優先度継承, 73
優先度スケジューリング, 72

**ラ**
ラージページ, 92
ライトプロセス状態, 46
ラウンドロビンスケジューリング, 74
ランキュー, 70
ランデブーポイント, 115
リース, 164
リーダ・ライタ問題, 25
リーダ・ライタロック, 112, 169
リーパスレッド, 50
リアルタイムシグナル, 54
リトルエンディアン, 149
リモートメソッド呼び出し, 158
ルートモード, 139
レジストリ, 151
レッドゾーン, 47
レディキュー, 16
レプリケーション, 166, 176
ローカル手続き呼び出し, 103
ロードバランサ, 185
ロック競合, 119
ロック順序, 29, 40

**ワ**
ワークロード, 66, 186
割り込み, 51
割り込みハンドラ, 51
割り込みハンドラテーブル, 51
割り込みマスク, 53
ワンショットシグナル, 54

# English index


**A**
abstraction, 1
access bit, 88
active array, 78
adaptive mutex, 50
address space, 10
address translation, 85
allocation on first touch, 87
Apache, 66
arbitration, 2
asymmetric multi-process event-driven model, 64
asymmetric multithreaded event-driven model, 64
asynchronous I/O, 63
asynchronous signal, 52
atomic instruction, 116
attach, 104

**B**
back-end driver, 145
bare-metal hypervisor, 138
barrier, 115
baseline, 69
benchmark, 68
big data, 192
big-endian, 149
binary semaphore, 111
binary translation, 141
binding, 151
block device, 124
block group, 132
boss-workers pattern, 31
bottom half, 54
bound thread, 30
bounded buffer, 41
broadcast operation, 25
buddy allocator, 94
buffer cache, 134
busy waiting, 110

**C**
cache affinity, 80
cache coherence, 118
causal consistency, 181
character device, 124
checkpointing, 97
child process, 14
chip multithreading, 82
clone, 57
close-to-open consistency, 168
cloud computing, 188
cold cache, 13
community cloud, 189
compare-and-swap, 116
Completely Fair Scheduler, 79
concurrency, 22
concurrent write sharing, 170
condition variable, 24
consistency model, 181
consolidation, 137
context switch, 13
copy-on-write, 97
counting semaphore, 111
CPU scheduler, 16
CPU utilization, 59
CPU-bound task, 75
critical section, 23
cycles per instruction, 83

**D**
data segment, 10
datacenter, 184
datagram socket, 102
deadlock, 28
deadlock detection and recovery, 29
deadlock prevention, 29
delayed write, 169
delegation, 168
demand paging, 95
dentry cache, 131
detach, 104
detached thread, 36
device driver, 124
device file, 125
device register, 123
direct memory access, 126
direct pointer, 132
directory entry, 131
dirty bit, 88
dispatch, 16
distributed file system, 160
distributed shared memory, 173
dom0, 138
double indirect pointer, 133

**E**
economies of scale, 189
elasticity, 188
elevator algorithm, 134
entry procedure, 113
event dispatcher, 61
event handler, 61
event-driven model, 61
exec, 15
execution time, 59
expired array, 78
ext2, 131
extended page table, 146
external data representation, 153
external fragmentation, 93

**F**
false sharing, 175
fidelity, 137
file descriptor, 130

file handle, 167
file system, 129
first-come first-served scheduling, 71
Flash, 64
fork, 15
front-end driver, 145
full virtualization, 141

**G**
generation number, 167
generic block layer, 129
guest operating system, 136

**H**
hard process state, 46
hardware performance counter, 83
hardware-assisted virtualization, 146
hashed page table, 91
heap, 10
helper, 64
heterogeneous architecture, 186
homogeneous architecture, 185
host operating system, 138
hosted hypervisor, 138
hot cache, 13
huge page, 92
hybrid cloud, 189
hypercall, 142
hypervisor-direct model, 145

**I**
I/O port, 125
I/O scheduling, 134
I/O-bound task, 75
immutable file, 165
indirect pointer, 133
infrastructure as a service, 190
init, 15
inode, 131
inter-process communication, 18
interface definition language, 150
internal fragmentation, 92
Internet service, 184
interrupt, 51
interrupt handler, 51
interrupt handler table, 51
interrupt mask, 53
inverted page table, 91
ioctl, 129
isolation, 2

**J**
join, 23
joinable thread, 36
journal, 134
journaling, 134

**K**
kernel, 3
kernel mode, 3
kernel module, 7
Kernel-based Virtual Machine, 139
kernel-level thread, 30

**L**
large page, 92
law of large numbers, 188
layered pattern, 32
lease, 164
least recently used, 96
light process state, 46
lightweight process, 47
little-endian, 149
load balancer, 185
load balancing, 81
local procedure call, 103
lock contention, 119
lock ordering, 29

**M**
machine address, 142
many-to-many model, 30
many-to-one model, 30
MapReduce, 191
marshalling, 151
mechanism, 2
memory allocator, 93
memory management unit, 86
memory-bound task, 82
memory-mapped file, 105
memory-mapped I/O, 125
message passing, 18
message queue, 101
message signaled interrupt, 51
microkernel, 7
mode bit, 3
modular operating system, 7
monitor, 113
monolithic operating system, 6
mount, 167
multi-level feedback queue, 77
multi-level page table, 89
multi-process model, 61
multi-tenancy, 191
multi-tier architecture, 184
multicore processor, 80
multiple readers/multiple writers, 176
multiple readers/single writer, 176
multithreaded model, 61
multithreading, 20
mutex, 23
mutual exclusion, 23

**N**
named pipe, 101
Native POSIX Thread Library, 57
network device, 124
Network File System, 167
new state, 14
nice value, 78
non-root mode, 139
non-uniform memory access, 81
NUMA-aware scheduling, 81

**O**
O(1) scheduler, 78
offset, 87
one-shot signal, 54
one-to-one model, 30
operating system, 1
operating system bypass, 128
optimize for the common case, 3
ostrich algorithm, 29

**P**
page, 85
page cache, 134
page directory, 89
page fault, 88
page frame, 85
page replacement, 96
page table, 11
page table base register, 88
page table entry, 87
page-out daemon, 96
paging, 85
parallelization, 21
paravirtualization, 142
parent process, 14
partial failure, 152
partitioning, 166
passthrough model, 144
path expression, 114
peer-to-peer, 160
performance isolation, 191
performance metric, 59
Peripheral Component Interconnect, 124
physical address, 11
physical frame number, 87
pinned page, 95
pipe, 101
pipeline pattern, 31
platform as a service, 190
policy, 2
polling, 126
port, 100
Portable Operating System Interface, 34
POSIX shared memory, 105
POSIX threads, 34
preemption, 16
preemptive scheduling, 71
prefetching, 134
priority, 72
priority aging, 72
priority inheritance, 73
priority inversion, 73
priority scheduling, 72
private cloud, 189
privileged instruction, 3
process, 10
process control block, 12
process migration, 98
process scope, 30
process state, 14
process-shared attribute, 106
producer-consumer problem, 24
program counter, 12
programmed I/O, 126
protection, 2
protection bits, 88
protection ring, 139
proxy variable, 26
pseudo device, 125
public cloud, 189

**Q**
QEMU, 139
queueing lock, 120

**R**
race condition, 22
read-copy-update, 115
reader-writer lock, 112
readers-writers problem, 25
ready queue, 16
ready state, 14
real-time signal, 54
reaper thread, 50
red zone, 47
red-black tree, 79
registry, 151
remote access model, 161
remote direct memory access, 174
remote method invocation, 158
remote procedure call, 148
rendezvous point, 115
replication, 166
response time, 59
root mode, 139
round robin scheduling, 74
RPC runtime, 149
rpcbind, 155
rpcgen, 154
run-to-completion scheduling, 71
running state, 14
runqueue, 70

**S**
scale-out, 185
second chance algorithm, 96
segment, 85
segmentation, 85
semaphore, 111
separation of mechanism and policy, 3
sequential consistency, 181
sequential write sharing, 170
serializer, 114
service level agreement, 60
service VM, 138
session semantics, 164
shadow page table, 142
shared memory, 18
shared memory multiprocessor, 80
shared memory object, 105
shared memory segment, 104
shortest job first, 71
signal, 4

signal handler, 52
signal mask, 53
signal operation, 25
simultaneous multithreading, 81
single reader/single writer, 175
single-process event-driven model, 61
slab allocator, 94
socket, 102
software as a service, 190
software-defined networking, 192
specialization, 21
spinlock, 110
split-device driver model, 145
Sprite, 169
spurious wake-up, 28
stack, 10
stack pointer, 12
stale file handle, 168
starvation, 72
stateful server, 162
stateless server, 162
stream socket, 102
strict consistency, 181
stub, 149
Sun RPC, 153
superblock, 131
swap partition, 95
swapping, 95
synchronization, 22
synchronous I/O, 128
synchronous signal, 52
synthetic workload, 67
system call, 4
system scope, 31
System V IPC, 103

**T**
tagged TLB, 146
task, 56
terminated state, 14
test-and-set, 116
test-and-test-and-set, 119
testbed, 59
text segment, 10
thread, 20
thread attribute, 35
thread creation, 22
thread pool, 31
throughput, 59
timesharing, 74
timeslice, 16
top half, 54
trace, 67
translation lookaside buffer, 90
trap, 4
trap-and-emulate, 140

**U**
UNIX semantics, 164
Unix-domain socket, 102
unmarshalling, 151
upload/download model, 161
user mode, 3
user-level driver, 128
user-level thread, 30
utility computing, 189

**V**
valid bit, 88
virtual address, 11
virtual file system, 130
virtual machine, 136
virtual machine monitor, 136
virtual memory, 85
virtual page number, 87
virtual runtime, 79
virtualization, 136
VM control structure, 146
VM entry, 140
VM exit, 139

**W**
wait graph, 28
wait operation, 24
wait time, 59
wait-free synchronization, 115
waiting state, 14
weak consistency, 181
workload, 66
write-invalidate, 118
write-update, 118

**X**
Xen, 138

**Z**
Zygote, 15